Embedded Linux

BeagleBone Black ile Adım Adım

İçindekiler

- Önsöz

Kısım I — Başlarken

- Bölüm 1 — BeagleBone Black Donanımı
- Bölüm 2 — Seri Konsol ve Minicom

Kısım II — Boot Süreci

- Bölüm 3 — Linux Boot Süreci: RBL → SPL → U-Boot
- Bölüm 4 — SD Karttan Boot
- Bölüm 5 — eMMC'den Boot

Kısım III — U-Boot ile Çalışmak

- Bölüm 6 — U-Boot Komut Satırı ve uEnv.txt
- Bölüm 7 — Seri Porttan (UART) Boot
- Bölüm 8 — TFTP ile Ağdan Boot
- Bölüm 9 — Cross Toolchain, SPL ve U-Boot Derleme

Kısım IV — Linux Kernel

- Bölüm 10 — Linux Kernel: Kaynak Ağacı ve Derleme
- Bölüm 11 — Linux Device Tree

Kısım V — Root Filesystem

- Bölüm 12 — BusyBox ile Minimal Root Filesystem
- Bölüm 13 — Buildroot

Kısım VI — Uygulama Geliştirme ve Donanım

- Bölüm 14 — Eclipse ile Uygulama Geliştirme
- Bölüm 15 — AM335x GPIO'ları, Pin Ayarları ve sysfs
- Bölüm 16 — Harici LED Bağlamak
- Bölüm 17 — 7-Segment Gösterge
- Bölüm 18 — 4 Haneli 7-Segment Gösterge
- Bölüm 19 — MPU6050 ile Eğim Ölçmek (I2C)
- Bölüm 20 — 16x2 Karakter LCD (HD44780)

Kısım VII — Kernel Sürücüleri

- Bölüm 21 — Kernel Modülü Yazmak
- Bölüm 22 — Karakter Aygıt Sürücüsü
- Bölüm 23 — Device Tree ile GPIO Platform Sürücüsü
- Bölüm 24 — MPU6050 için I2C Sürücüsü (IIO)

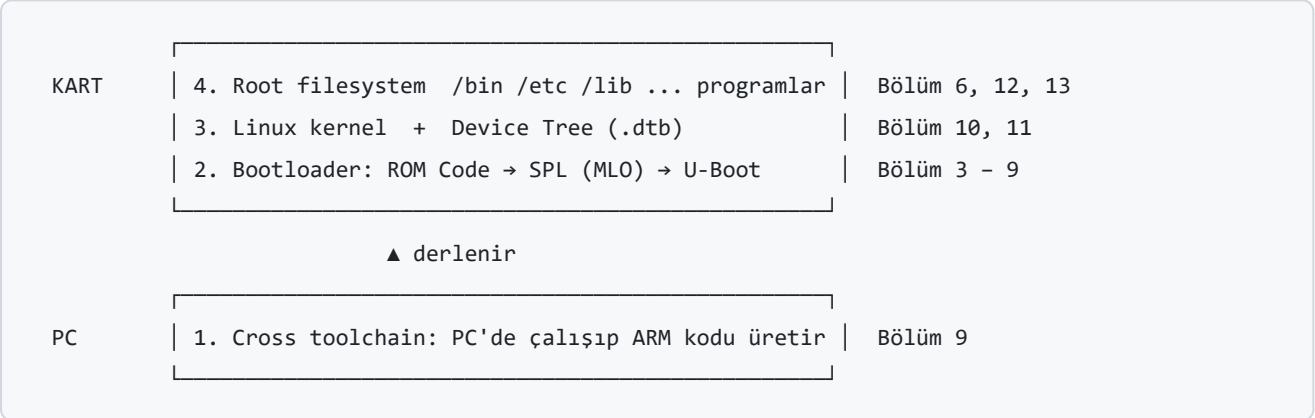
Önsöz

Bu kitap, gömülü Linux'a (Embedded Linux) yeni başlayan biri için yazıldı. Bir **BeagleBone Black** kartını sıfırdan alıp önce hazır dosyalarla, sonra kendi derlediğin bootloader ile açacaksın. Her adımda **neyi neden yaptığını** görmen hedefleniyor.

Kitap, bir Embedded Linux kursunu (https://github.com/niekiran/EmbeddedLinuxBBB) takip ederken alınan notlardan oluştu. Kursla birlikte de tek başına da okunabilir.

Bir Embedded Linux Sistemi Nelerden Oluşur?

Masaüstü bilgisayarda bu parçaları hiç görmeyiz; hepsi hazır gelir. Gömülü sistemde ise her birini tek tek hazırlamak, karta koymak ve birbirine bağlamak bizim işimizdir:



Parça	Görevi	Kitaptaki yeri
Cross toolchain	PC'de (x86) çalışıp karta (ARM) uygun program üreten derleyici ve araçlar	Bölüm 9
Bootloader	Kart açılınca belleği hazırlar, kernel'i ve device tree'yi RAM'e yükleyip başlatır	Bölüm 3 – Bölüm 9
Linux kernel + Device Tree	İşletim sisteminin çekirdeği ve kartın donanımını kernel'e tarif eden dosya	Bölüm 10, Bölüm 11
Root filesystem	Kernel açıldıktan sonra çalışan her şey: kabuk, komutlar, ayar dosyaları	Bölüm 6, Bölüm 12, Bölüm 13

Kitabın Haritası

Kısım	Bölüm	Ne yapacaksın?
I — Başlarken	1. BeagleBone Black Donanımı	Kartı, butonları, LED'leri ve debug header'ını tanı

Kısım	Bölüm	Ne yapacaksın?
	2. Seri Konsol ve Minicom	Kartın konsolunu bilgisayarda gör
II — Boot Süreci	3. Linux Boot Süreci	Kart açılırken hangi programın hangi sırayla çalıştığını öğren
	4. SD Karttan Boot	SD kartı hazırla, kartı adım adım SD karttan aç
	5. eMMC'den Boot	Kartın dahili belleğine güncel Debian yükle
III — U-Boot ile Çalışmak	6. U-Boot Komut Satırı ve uEnv.txt	U-Boot'u durdur, ayarlarını değiştir, rootfs'i tanı
	7. Seri Porttan (UART) Boot	Kartı SD kart ve eMMC olmadan, sadece seri kabloyla aç
	8. TFTP ile Ağdan Boot	Kernel'i her seferinde ağdan çekerek geliştirmeyi hızlandır
	9. Cross Toolchain, SPL ve U-Boot Derleme	Bootloader'ı kaynak koddan kendin derle
IV — Linux Kernel	10. Linux Kernel: Kaynak Ağacı ve Derleme	Kernel kaynağında gezin, kernel'i ve modülleri derle
	11. Linux Device Tree	Kernel'in donanımı nasıl tanıdığını öğren
V — Root Filesystem	12. BusyBox ile Minimal Root Filesystem	Kendi rootfs'ini sıfırdan kur, NFS ile bilgisayardan aç
	13. Buildroot	Bütün sistemi tek komutla derle, hazır SD kart imajı üret
VI — Uygulama Geliştirme ve Donanım	14. Eclipse ile Uygulama Geliştirme	Eclipse'te ya da `make` + VS Code ile ARM için program yaz, kartta çalıştır, `gdbserver` ile hata ayıkla
	15. AM335x GPIO'ları, Pin Ayarları ve sysfs	Pinmux'u çöz, LED'leri ve GPIO'ları komut satırından ve C'den kontrol et
	16. Harici LED Bağlamak	LED'i dirençle bir GPIO'ya bağla, C ile ve device tree ile sür
	17. 7-Segment Gösterge	Tek haneli göstergeyi sür, sayaç ve butonla sayma programı yaz
	18. 4 Haneli 7-Segment Gösterge	Çoğullama ile dört haneyi sür, sayaç ve dijital saat yaz
	19. MPU6050 ile Eğim Ölçmek (I2C)	I2C ile ivmeölçer/jiroskop oku, eğim açısını hesapla
	20. 16x2 Karakter LCD (HD44780)	LCD'yi 4 bit modunda sür, kayan yazı ve Türkçe karakter göster
VII — Kernel Sürücüleri	21. Kernel Modülü Yazmak	İlk modülünü yaz, kartta yükle, parametre ve sembol paylaşımını dene
	22. Karakter Aygıt Sürücüsü	`/dev` altında kendi aygıtını aç; read, write, ioctl'ü yaz

Kısım	Bölüm	Ne yapacaksın?
	23. Device Tree ile GPIO Platform Sürücüsü	LED'i ve butonu DT'den tanıt, kesmeyle süren bir sürücü yaz
	24. MPU6050 için I2C Sürücüsü (IIO)	Sensörü kernel'de oku, IIO dosyalarıyla dışarı aç

Okuma sırası: Bölümler birbirinin üzerine kurulu. İlk kez okuyorsan sırayla git. Her bölümün başındaki **"Ön koşul"** satırı, o bölümden önce hangilerini bilmen gerektiğini söyler.

Her bölüm aynı düzende yazıldı:

1. **Bu bölümde:** Bölümün sonunda neleri yapabileceğin.
2. **Konu anlatımı ve adım adım uygulama:** Komutlar hem terminal görüntüsü olarak hem de kopyalanabilir kod bloğu olarak verilir.
3. **Özet:** Bölümün tek bakışta tekrarı.
4. **Karşılaşılabilecek problemler:** Sık görülen hatalar, sebepleri ve çözümleri.

Gerekenler

Donanım	Neden?
BeagleBone Black (Rev C)	Kitabın kartı
microSD kart (en az 4 GB) + kart okuyucu	SD karttan boot için
USB-TTL seri dönüştürücü (**3.3V**)	Kartın konsolunu görmek için. En önemli debug aracın.
5V / 2A adaptör (barrel jack)	Seri boot ve TFTP bölümlerinde karta güvenilir güç için
Ethernet kablosu	TFTP ile ağdan boot için
Mini USB kablo	Karta güç vermek ve USB üzerinden SSH için
Breadboard, erkek-erkek / erkek-dişi kablolar	Donanım bölümlerindeki devreler (Bölüm 16 – Bölüm 20)
LED, dirençler (470 Ω, 1 kΩ, 10 kΩ), buton	Harici LED ve butonla sayma (Bölüm 16, Bölüm 17)
Tek haneli ve 4 haneli 7-segment gösterge (5161, 5641 tipi), 4 adet PNP transistör (BC557)	Bölüm 17, Bölüm 18
MPU6050 modülü (GY-521)	I2C ile eğim ölçme (Bölüm 19)
16x2 karakter LCD (HD44780) + 10 kΩ potansiyometre	Bölüm 20

Yazılım	Not
Ubuntu (host PC)	Gerçek kurulum ya da sanal makine. Sanal makinede ağ ayarı **Bridged** olmalı (Bölüm 8).
`minicom`, `lrzsz`	Seri konsol ve Xmodem/Ymodem ile dosya gönderme (Bölüm 2)
`gparted`	SD kartı bölümlemek için (Bölüm 4)
`tftpd-hpa`	TFTP sunucusu (Bölüm 8)
`nfs-kernel-server`	Rootfs'i ağdan bağlamak için NFS sunucusu (Bölüm 12)
Eclipse (Eclipse IDE for C/C++ Developers)	Uygulama geliştirme (Bölüm 14)
`i2c-tools` (kartta)	I2C aygıtlarını komut satırından denemek (Bölüm 19)
Cross toolchain, U-Boot, kernel, BusyBox kaynakları	Derleme bölümlerinde kurulur (Bölüm 9, Bölüm 10, Bölüm 12)

Bu Kitaptaki Sürümler

Komutlarda ve ekran görüntülerinde geçen sürümler şunlardır. Kendi kurulumunda farklı bir sürüm kullanıyorsan komutlardaki sürüm numaralarını ona göre değiştir.

Ne?	Değer	Bölüm
Kursun ilk hazır imajları (Ångström): U-Boot / kernel	2013.04 / 3.8.13	4, 6, 7, 8
Kursun seri boot imajları: SPL / U-Boot	2017.03 / 2016.11-rc1	7
Kursun güncel SD kart imajları: U-Boot / kernel	2022.04 / 5.10.168	4, 8
eMMC'ye yüklenen Debian imajı (kursta: `bone-debian-10.3-iot-armhf-2020-04-06-4gb.img.xz`)	Debian 13.7 IoT, v5.10-ti: `am335x-debian-13.7-base-v5.10-ti-armhf-2026-10-09-4gb.img.xz`	5, 14–20
Kendi derlediğimiz U-Boot	2022.04 (`am335x_evm_defconfig`)	9
Cross toolchain	Linaro GCC 7.5.0 (7.5-2019.12)	9, 10, 12
Kendi derlediğimiz kernel	5.10.162 (BeagleBoard, dal `5.10`, `bb.org_defconfig`)	10
BusyBox / Dropbear	1.36.1 / 2022.83	12
Buildroot (LTS)	2026.02.3 (kernel 6.18.1, U-Boot 2026.01)	13
Eclipse	2026-09 (Eclipse IDE for C/C++ Developers)	14
Host PC IP / kart IP	192.168.27.1 / 192.168.27.2	8

Yazım Kuralları

Komut satırları: Bir komutun nerede çalıştırılacağını başındaki işaretten anlarsın.

Görünüm	Nerede?
`mehmet@ubuntu:~$ komut` veya kod bloğunda `bash`	Host PC'deki Ubuntu terminali
`U-Boot# komut` veya `=> komut`	Kartın U-Boot komut satırı (seri konsolda). Eski U-Boot `U-Boot#`, yenisi `=>` gösterir.
`root@beaglebone:~# komut` veya `/ # komut`	Kartta açılmış Linux (seri konsol veya SSH)

Ekran görüntüleri: Koyu renkli terminal görüntüleri adımları göstermek için hazırlanmış **temsili** çıktılardır. Sürüm, tarih, boyut ve hız değerleri senin kartında farklı görünebilir. Kursun slaytlarından, videolarından ve başka kaynaklardan (AM335x TRM, beagleboard.org, Bootlin) alınan görseller olduğu gibi kullanıldı; her birinin altında kaynağı yazılıdır.

Kutular:

> **Not**
>
> Ek bilgi, arka plan.

> **İpucu**
>
> İşini kolaylaştıracak bir yol.

> **Önemli**
>
> Atlarsan adımın çalışmayacağı bilgi.

> **Uyarı**
>
> Dikkat etmezsen zaman kaybettirecek veya bir şeyi bozacak durum.

> **Dikkat**
>
> Veri kaybına veya donanım hasarına yol açabilecek durum.

Kaynaklar

- Kursun deposu: https://github.com/niekiran/EmbeddedLinuxBBB
- BeagleBone Black resmi dokümanı
- AM335x Technical Reference Manual (TRM): repodaki `am335x_referance_manuel.pdf`
- U-Boot dokümantasyonu
- Bootlin eğitim materyalleri (CC BY-SA 3.0)

Kısım I
Başlarken

Bölüm 1 — BeagleBone Black Donanımı

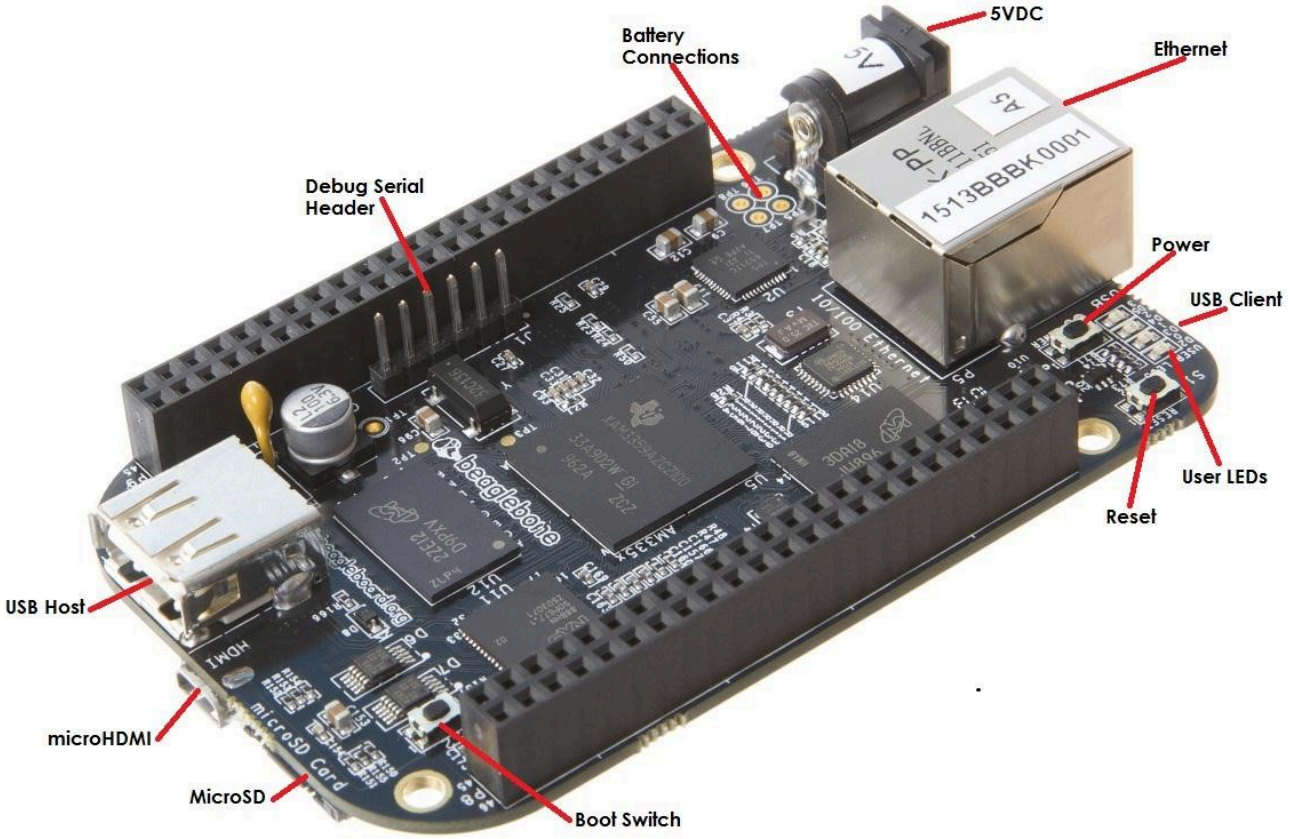
> **Bu bölümde:**
>
> - BeagleBone Black'in işlemcisini, belleklerini ve bağlantı noktalarını tanıyacaksın.
> - Kartın üç butonunun (Power, Reset, Boot) ne işe yaradığını öğreneceksin.
> - Seri konsol için kullanacağın J1 debug header'ının pinlerini göreceksin.
>
> **Ön koşul:** Yok. Kitabın ilk bölümü.

> Kaynak: BeagleBone Black resmi dokümanı (docs.beagleboard.org). Bu bölümdeki kart görselleri bu dokümandan alınmıştır (CC BY-SA).

1.1 Genel Özellikler

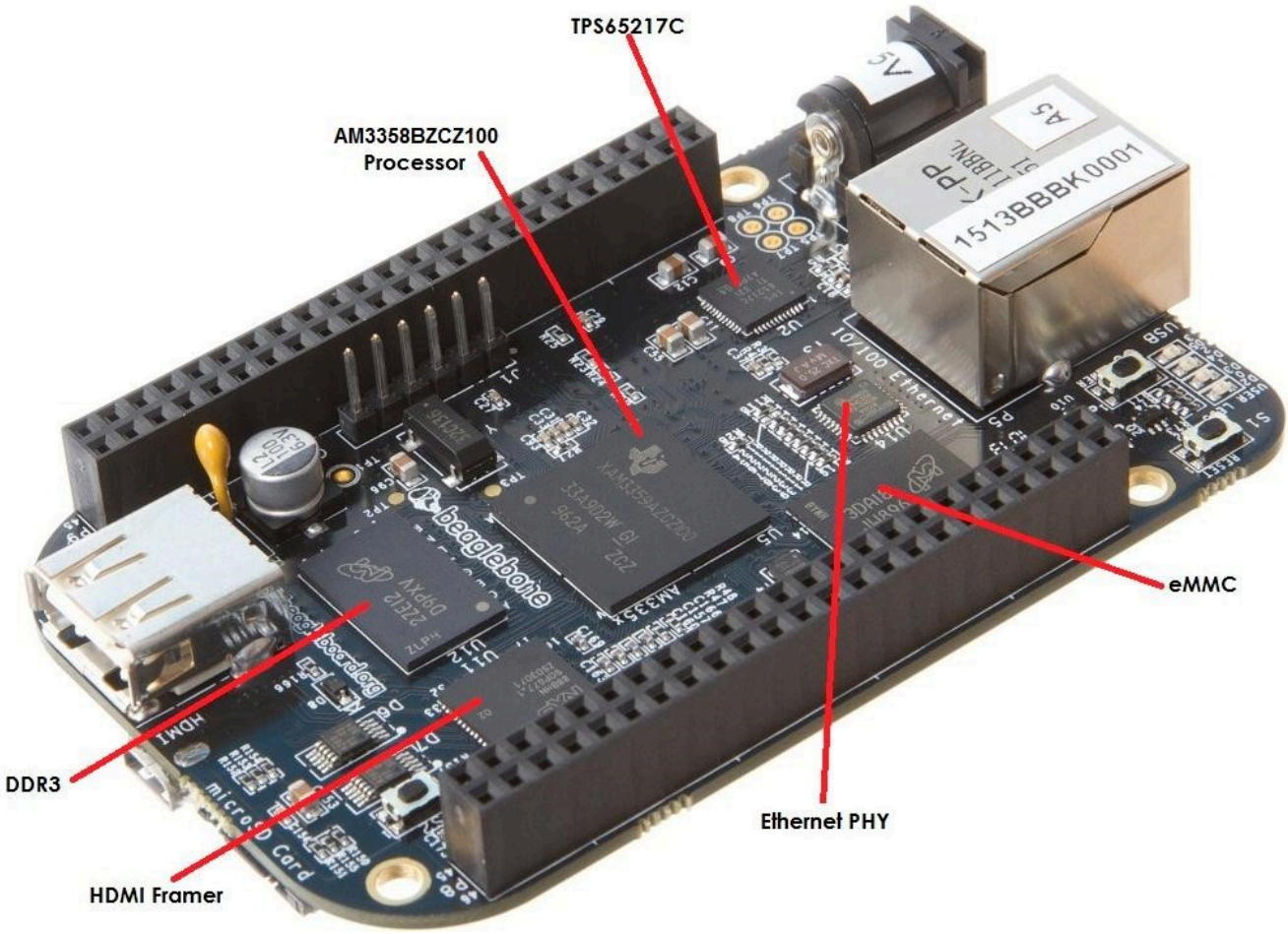
Özellik	Değer
İşlemci (SoC)	TI Sitara **AM3358BZCZ100**, ARM **Cortex-A8**, **1 GHz**'e kadar
Cache	64 KB L1, 256 KB L2
RAM	**512 MB DDR3L** (400 MHz clock, 800 MHz efektif)
Dahili depolama	**4 GB eMMC** (Rev C, eski revizyonlarda 2 GB), MMC1 üzerinden 8-bit
Harici depolama	**microSD** yuvası (MMC0)
EEPROM	4 KB, I2C0 üzerinde. Kart adı, seri numarası ve revizyon bilgisini tutar.
PMIC	**TPS65217C** (güç yönetimi) + 3.3V için harici LDO (TLV70233)
Ethernet	10/100 Mbit (RJ45)
USB	USB0: mini-USB **client** (PC bağlantısı), USB1: Type-A **host** (500 mA)
Görüntü	micro-HDMI (NXP TDA19988 HDMI framer)
Genişleme	**P8** ve **P9** (2 x 46 pin) header'ları, en fazla 4 cape takılabilir
PRU-ICSS	2 adet 200 MHz gerçek zamanlı yardımcı çekirdek (PRU)
Debug	**J1**: 6 pinli seri debug header (UART0), opsiyonel 20 pinli JTAG (lehimli gelmez)

1.2 Kart Üzerindeki Bileşenler



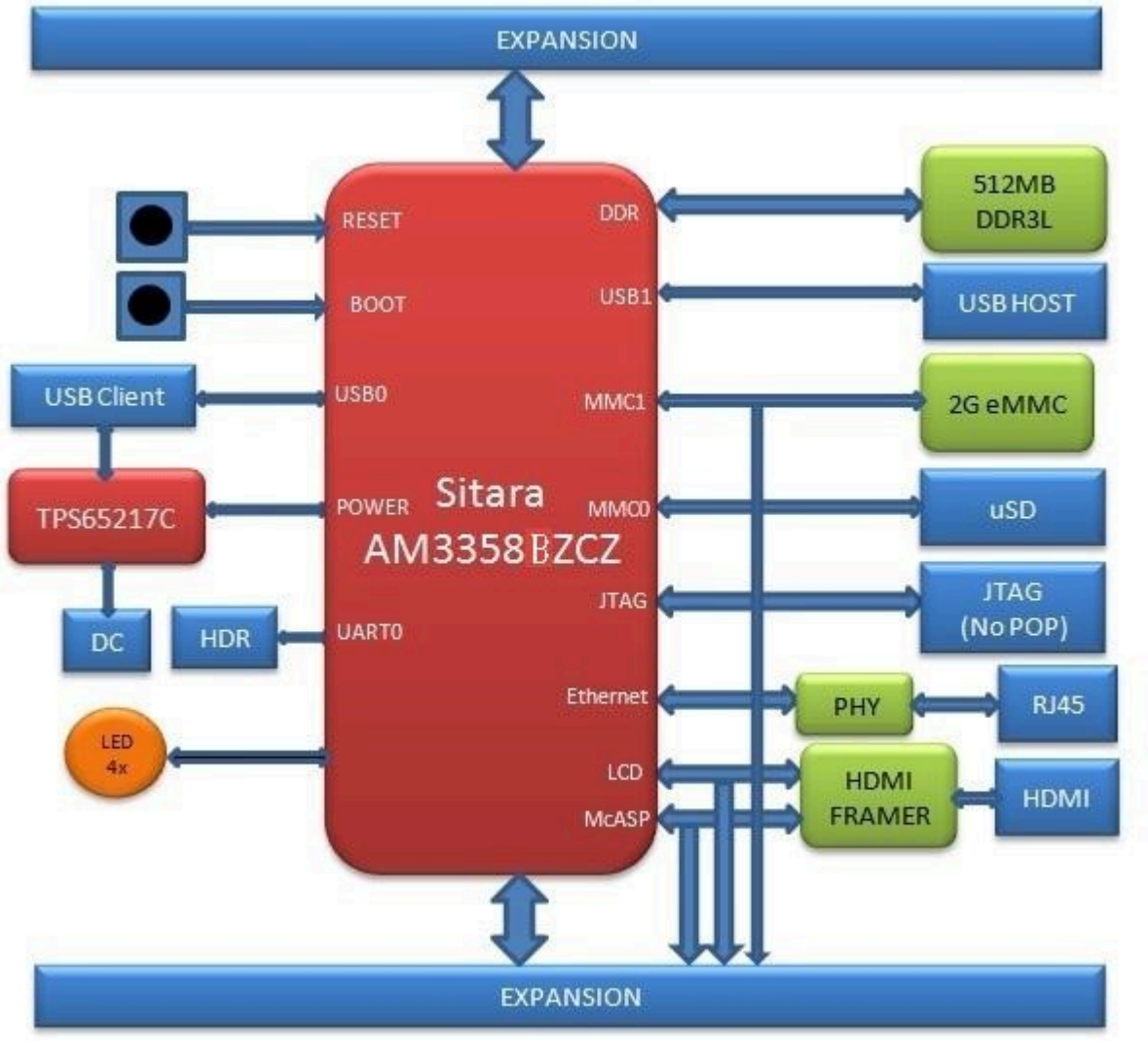
- **5VDC**: Barrel jack güç girişi (5V, en az 1A).
- **USB Client (mini-USB)**: Kartı bilgisayara bağlarsın. Hem güç verir hem de USB üzerinden ağ ve depolama sağlar. Kartın USB ağ adresi Windows'ta `192.168.7.2`, Linux ve macOS'ta `192.168.6.2`'dir (5.4).
- **Debug Serial Header (J1)**: Seri konsol bağlantısı. Kitap boyunca en çok kullanacağın yer burası.
- **Power / Reset / Boot Switch**: Kartın üç butonu (1.5).
- **User LEDs**: 4 adet mavi LED (USR0-USR3).

Kart üzerindeki çipler



Çip	Görevi
AM3358BZCZ100	Ana işlemci (SoC)
DDR3	512 MB RAM
eMMC	Kart üstündeki flash bellek, varsayılan boot kaynağı
TPS65217C	PMIC. Tüm voltaj raylarını üretir ve power butonunu yönetir.
Ethernet PHY	Ethernet fiziksel katman çipi
HDMI Framer	LCD sinyallerini HDMI'a çevirir

1.3 Blok Diyagram



> **Not**
>
> Diyagramda **2G eMMC** yazıyor çünkü eski revizyona ait. Rev C kartlarda eMMC **4 GB**'dır.

Diyagramdan çıkan önemli noktalar:

- **MMC1** → eMMC, **MMC0** → microSD. Kernel ve U-Boot tarafında bu isimleri sık göreceksin.
- **UART0** → Debug header (J1). Linux'ta `/dev/ttyS0` (eski kernel'lerde `/dev/ttyO0`) olarak görünür.
- **USB0** → Client portu, **USB1** → Host portu.

1.4 Güç

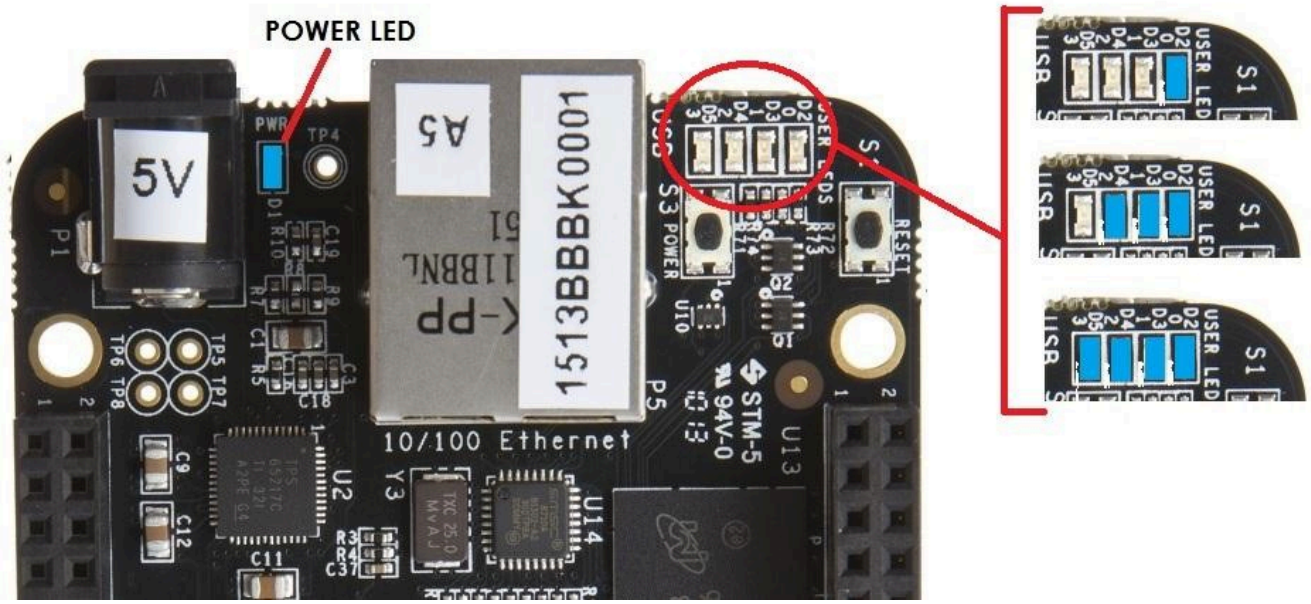
- **USB'den güç**: En fazla ~500 mA çeker. Bu modda P9'daki **VDD_5V** pinlerine güç gelmez.
- **DC jack'ten güç**: 5V ±0.25V, 1A önerilir. Cape takıyorsan daha yüksek akımlı bir adaptör gerekebilir.
- **Tipik tüketim (5V'ta)**: U-Boot'ta boşta ~210 mA, kernel boot sırasında ~460 mA (tepe), kernel boşta ~350 mA.

> **Uyarı**
>
> Kartı kapatmak için kabloyu çekme. `sudo poweroff` komutunu kullan veya **Power** butonuna kısa bas. Ani güç kesilmesi eMMC/SD karttaki dosya sistemini bozabilir.

1.5 Butonlar

Buton	Kart üstündeki adı	Görevi
Power	S3	Kısa basış işletim sistemine düzgün kapanma sinyali gönderir. **8 saniyeden uzun** basılı tutarsan kart zorla kapanır.
Reset	S1	Kartı yeniden başlatır (warm reset). Boot modunu **değiştirmez**.
Boot	S2	Güç verilirken basılı tutulursa boot sırasını değiştirir (1.7).

1.6 LED'ler



- **Power LED** (mavi): Güç varken yanar. Açılışta yanıp sönüyorsa PMIC aşırı akım nedeniyle kartı kapatmış demektir.
- **User LED'ler (USR0-USR3)**: Yazılımdan GPIO ile kontrol edilir. Hazır Debian imajındaki varsayılan görevleri:

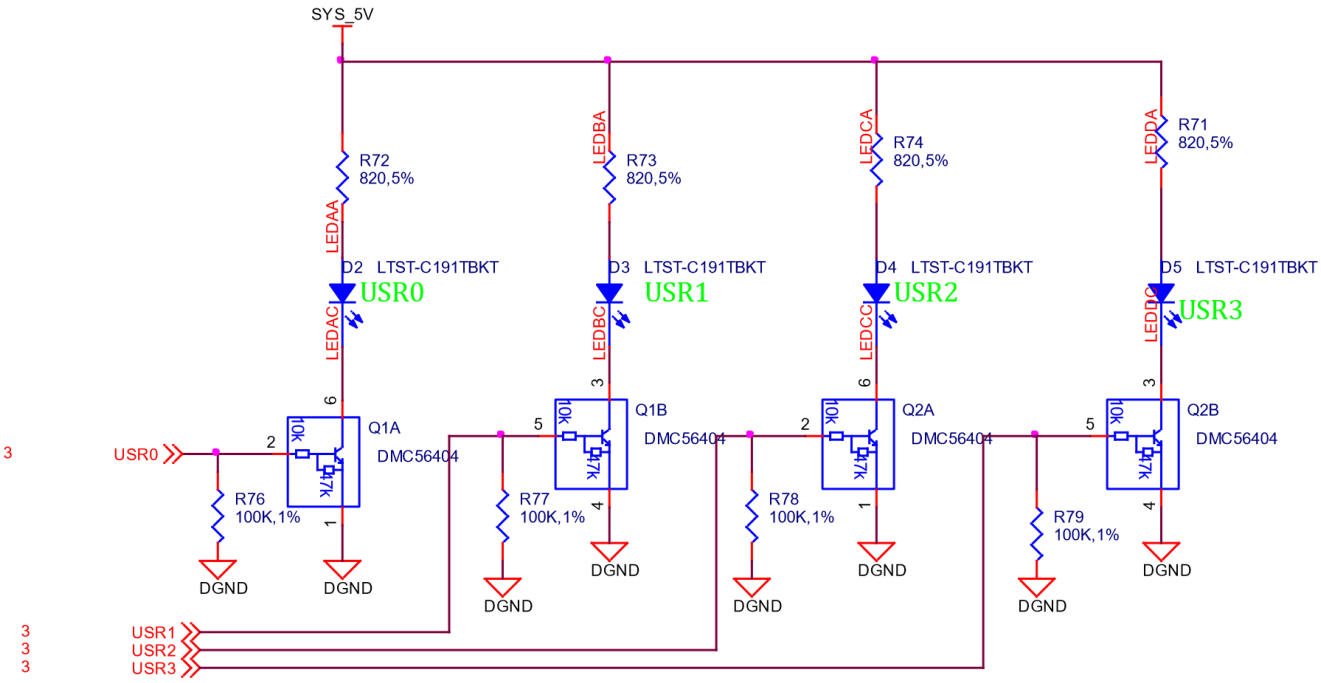
LED	GPIO	Varsayılan görev (Debian imajı)
USR0	GPIO1_21	**heartbeat**: Kalp atışı gibi yanıp söner, "kernel çalışıyor" demektir
USR1	GPIO1_22	microSD erişimi
USR2	GPIO1_23	CPU aktivitesi

LED	GPIO	Varsayılan görev (Debian imajı)
USR3	GPIO1_24	eMMC erişimi

Linux'ta LED'leri şu dizinden kontrol edebilirsin:

```
ls /sys/class/leds/
echo none > /sys/class/leds/beaglebone:green:usr0/trigger
echo 1 > /sys/class/leds/beaglebone:green:usr0/brightness
```

User LED'lerin devre şeması (her LED bir transistör üzerinden GPIO ile sürülür):



> **İpucu**
>
> USR0'ın kalp atışı gibi yanıp sönmesi kernel kodunda değil, **Device Tree**'de tanımlıdır (11.5).

1.7 Boot Sırası ve S2 Butonu

Kitap açısından **en önemli başlık budur.**

AM3358 açılırken **SYSBOOT** pinlerinin seviyesine bakar ve hangi kaynaktan boot edeceğine bu pinlere göre karar verir. **Boot (S2)** butonu bu bitlerden birini değiştirir.

SYSBOOT[15:14]	SYSBOOT[13:12]	SYSBOOT[11:10]	SYSBOOT[9]	SYSBOOT[8]	SYSBOOT[7:6]	SYSBOOT[5]	SYSBOOT[4:0]	Boot Sequence			
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11100b	MMC1	MMC0	UART0	USB0[5]
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11000b	SPI0	MMC0	USB0[5]	UART0

Durum	Boot sırası
S2 basılı değil (varsayılan)	**MMC1 (eMMC)** → MMC0 (microSD) → UART0 → USB0
S2 basılı (güç verilirken)	SPI0 → **MMC0 (microSD)** → USB0 → UART0

> **Önemli**
>
> Kendi derlediğin U-Boot/kernel'i **SD karttan** açmak için: Kartın gücünü kes, **S2**'yi basılı tut, gücü ver, birkaç saniye sonra bırak. Boot modu **sadece güç verilirken** okunur. Reset butonu boot modunu değiştirmez.

Boot zincirinin (ROM Code → SPL → U-Boot → Kernel) ayrıntıları ve SYSBOOT tablosunun nasıl okunduğu Bölüm 3'te anlatılıyor.

1.8 Seri Debug Header (J1)

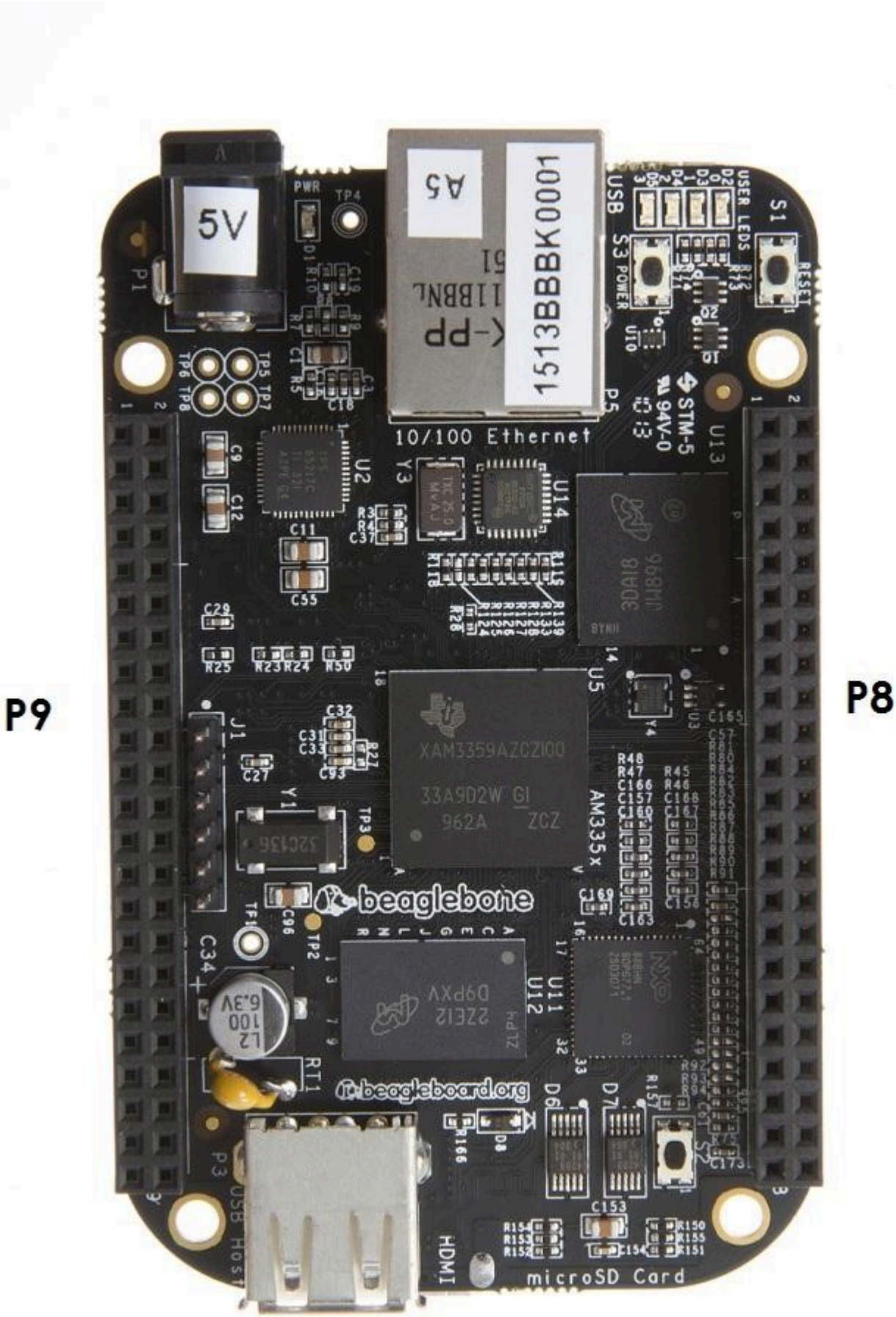
J1 Pini	Sinyal	USB-TTL dönüştürücüde bağlanacağı yer
1	GND	GND

J1 Pini	Sinyal	USB-TTL dönüştürücüde bağlanacağı yer
2	—	—
3	—	—
4	RX (kartın alıcısı)	Dönüştürücünün **TX**'i
5	TX (kartın vericisi)	Dönüştürücünün **RX**'i
6	—	—

- 1 numaralı pin kart üzerinde **beyaz nokta** ile işaretlidir. FTDI kablosu kullanıyorsan siyah kablo (GND) bu pine gelir.
- Ayarlar: **115200 8N1**, flow control **kapalı** (Bölüm 2).

> **Uyarı**
>
> Sinyaller **3.3V**'tur. USB-TTL dönüştürücünün **3.3V** modda olduğundan emin ol, 5V'luk dönüştürücü kartı bozabilir. Dönüştürücünün **VCC** pinini karta **bağlama**, sadece GND, TX ve RX yeterli.

1.9 Genişleme Header'ları (P8 ve P9)



- Her biri 2x23 = **46 pin**. Toplam 92 pin.
- Pinler **multiplexed**: Aynı pin GPIO, UART, I2C, SPI, PWM gibi farklı görevlerde kullanılabilir. Hangi görevin seçileceği **pinmux** (device tree / overlay) ile belirlenir.

Pin(ler)	Sinyal
P8_1, P8_2, P9_1, P9_2, P9_43-46	**GND**
P9_3, P9_4	**3.3V**
P9_5, P9_6	**VDD_5V** (sadece DC jack'ten güç verilirken var)
P9_7, P9_8	**SYS_5V**
P9_9	PWR_BUT
P9_10	SYS_RESETn
P9_32	VADC (ADC referansı)
P9_33-40	**AIN0-AIN6** (analog girişler)
P9_34	AGND

> **Dikkat**
>
> - GPIO pinleri **3.3V**'tur ve **5V toleranslı değildir**. 5V sinyal bağlarsan işlemci bozulur.
> - Analog girişler (AIN) en fazla **1.8V** kabul eder.
> - Kart tamamen açılmadan (SYS_RESETn HIGH olmadan) pinlere dışarıdan sinyal verme.

1.10 Özet

1. Seri konsol (J1, 115200 8N1) en önemli debug aracın.
2. Varsayılan boot **eMMC**'den. Kendi imajını **SD karttan** açmak için **S2**'yi basılı tutarak güç ver.
3. Boot zinciri: **ROM → MLO (SPL) → U-Boot → Kernel + DTB → RootFS**.
4. eMMC = **MMC1**, microSD = **MMC0**.
5. Tüm I/O **3.3V**, ADC **1.8V**.

Bölüm 2 — Seri Konsol ve Minicom

> **Bu bölümde:**
>
> - USB-TTL dönüştürücüyü bilgisayara tanıtıp port adını (`/dev/ttyUSB0`) bulacaksın.
> - Minicom'u kurup BeagleBone Black için doğru ayarlarla (115200 8N1, flow control kapalı) kaydedeceksin.
> - Minicom'un sık kullanılan kısayollarını ve Windows'taki karşılığı Tera Term'i öğreneceksin.
>
> **Ön koşul:** Bölüm 1 — J1 debug header'ının pinleri.

2.1 Seri Konsol Neden Önemli?

Kart açılırken ekrana bir şey basan ilk programlar (ROM Code, SPL, U-Boot) HDMI veya ağ kullanmaz. Bütün mesajlarını **UART0** üzerinden, yani J1 header'ından gönderirler. Bu mesajları görmenin ve U-Boot'a komut yazmanın tek yolu seri konsoldur:

```
[Klavye] ⇄ [minicom / Tera Term] ⇄ USB-TTL kablo ⇄ J1 (UART0) ⇄ [BeagleBone: ROM / SPL / U-Boot / Linux]
```

Bilgisayardaki program sadece bir penceredir. Komutları bilgisayar değil, **kartın üzerindeki** program (U-Boot veya Linux) çalıştırır.

2.2 Minicom'u Kur

Minicom, Linux'ta kullanılan bir seri port terminal programıdır. `lrzsz` paketi de Minicom'un Xmodem/Ymodem ile dosya göndermesi için gerekir (Bölüm 6 ve Bölüm 7'de kullanacağız):

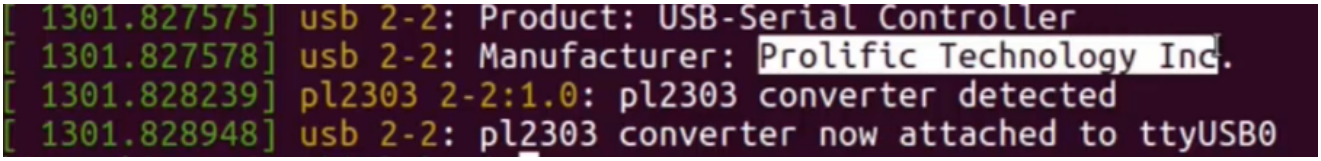
```
sudo apt update
sudo apt install minicom lrzsz
```

2.3 USB-TTL Dönüştürücünün Port Adını Bul

USB-TTL dönüştürücüyü bilgisayara tak ve terminalde şunu çalıştır:

```
sudo dmesg | grep tty
```

Çıktı aşağıdakine benzer olmalı:



Görsel: kursun videosundan ekran görüntüsü.

- Kernel, dönüştürücünün çipini tanıyıp sürücüsünü yükler. Bu örnekte çip **pl2303**. Senin dönüştürücünde **cp210x**, **ftdi_sio** veya **ch341** de yazabilir.
- Satırın sonundaki `ttyUSB0`, Minicom'da kullanacağımız portun adıdır: `/dev/ttyUSB0`.

Portu her seferinde `sudo` ile açmamak için kullanıcını `dialout` grubuna ekle (bir kez yapılır, sonra oturumu kapatıp açman gerekir):

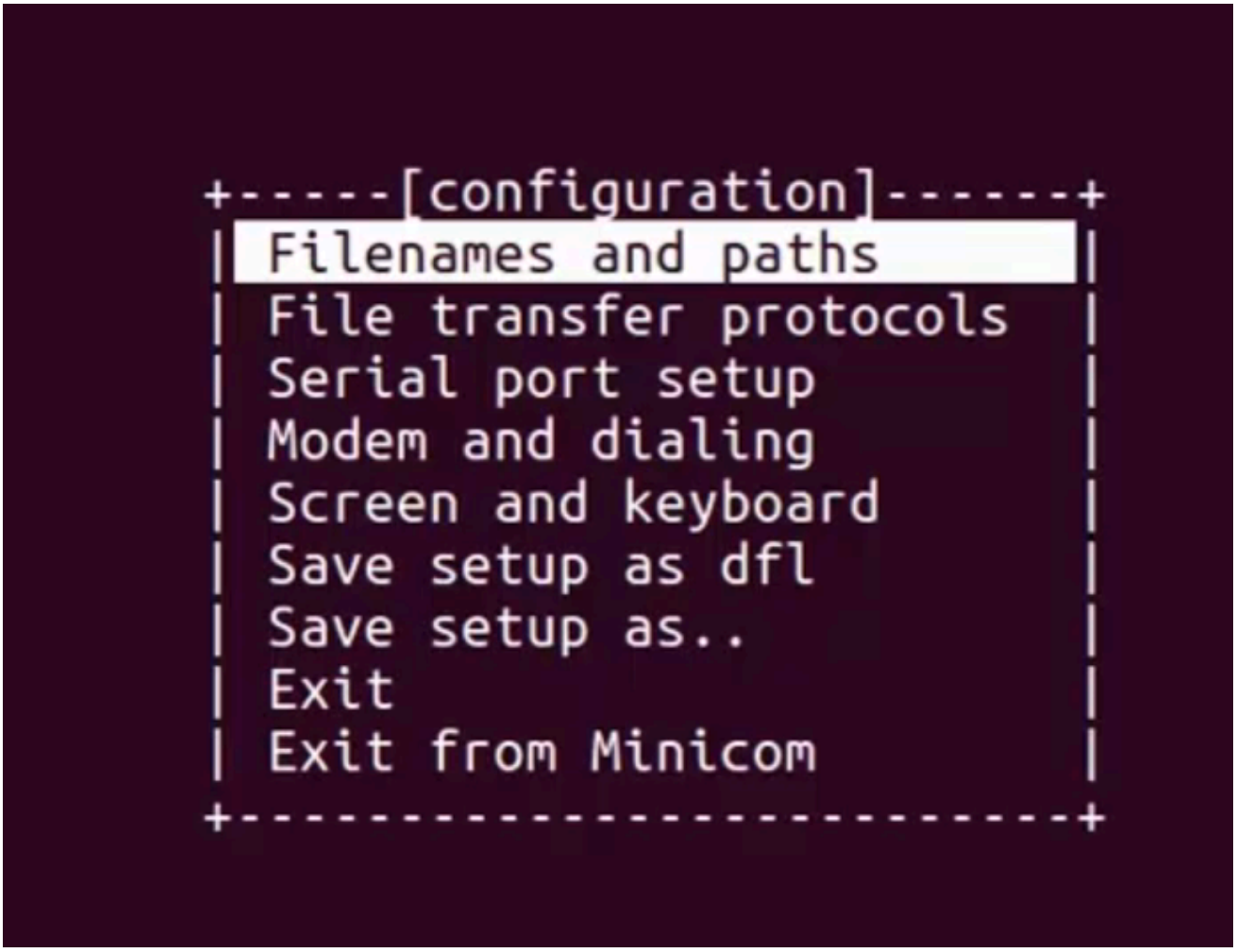
```
sudo usermod -aG dialout $USER
```

2.4 Minicom Ayarlarını Yap

Ayar ekranını aç:

```
sudo minicom -s
```

Yön tuşlarıyla menüde gezinebilirsin. **Serial port setup**'ı seç:



Görsel: kursun videosundan ekran görüntüsü.

Seri port ayarları ekranı açılır:

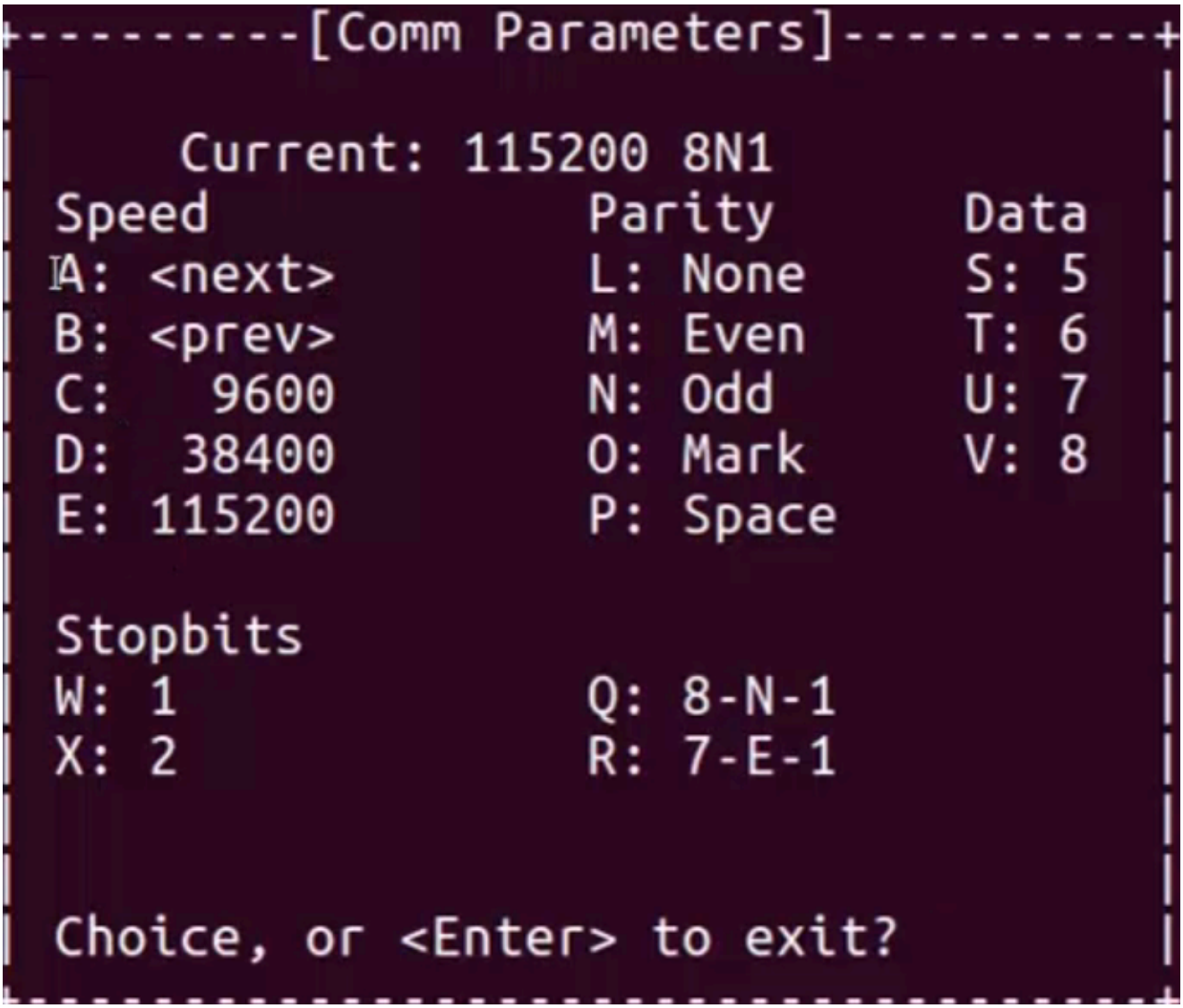
Görsel: kursun videosundan ekran görüntüsü.

Bu ekranda her ayarın başında bir harf vardır. Değiştirmek istediğin ayarın harfine basarsın. Örneğin **Serial Device**'ı değiştirmek için **A**'ya bas. İmleç o satıra geçer:

Görsel: kursun videosundan ekran görüntüsü.

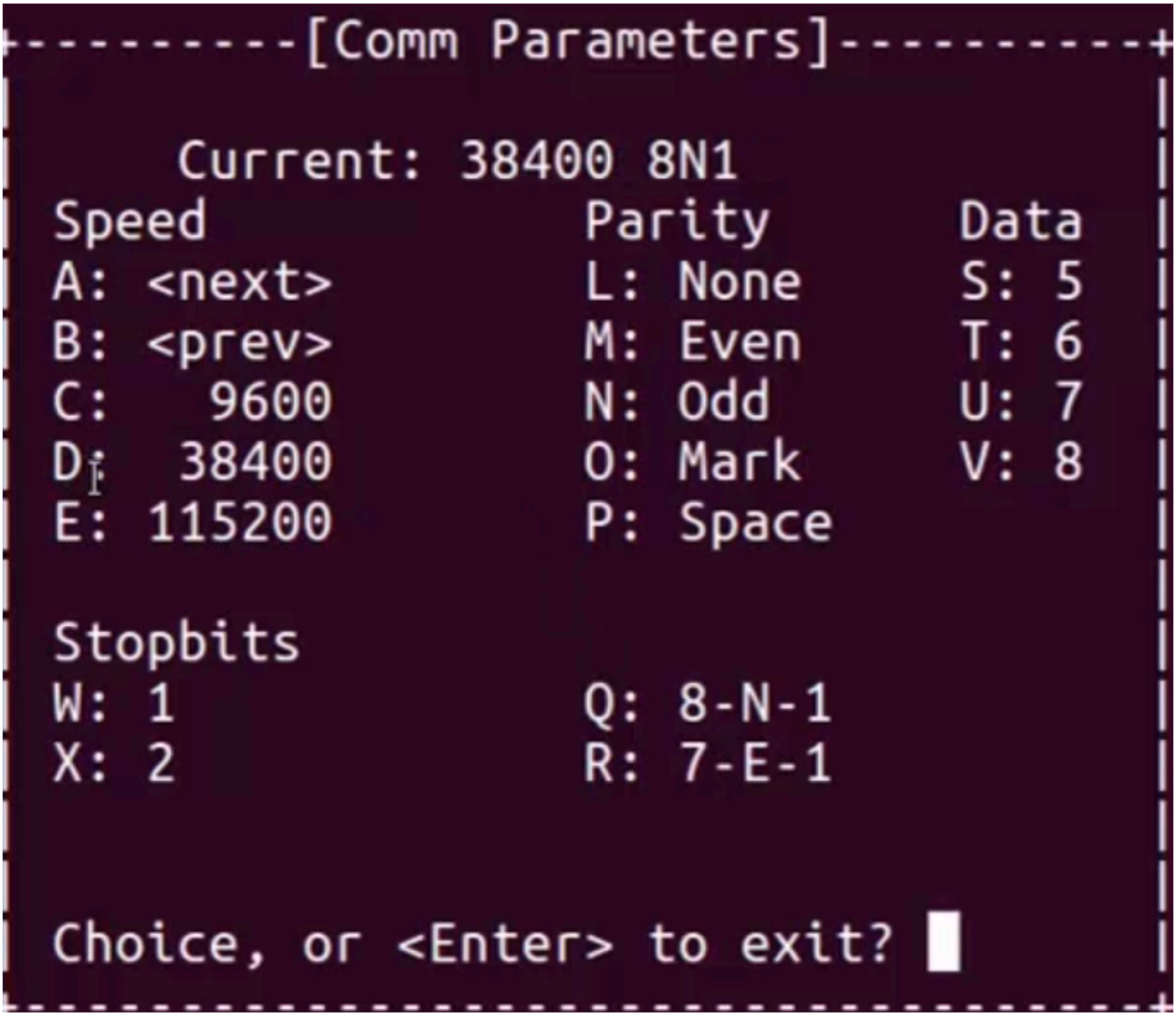
Eski değeri silip `/dev/ttyUSB0` yaz ve **Enter**'a bas.

Baud rate'i değiştirmek için **E**'ye bas. Hız ve veri formatı ekranı açılır:



Görsel: kursun videosundan ekran görüntüsü.

Bu ekranda da her seçeneğin bir harfi var. Örneğin **D**'ye basınca hız 38400 olur:



Görsel: kursun videosundan ekran görüntüsü.

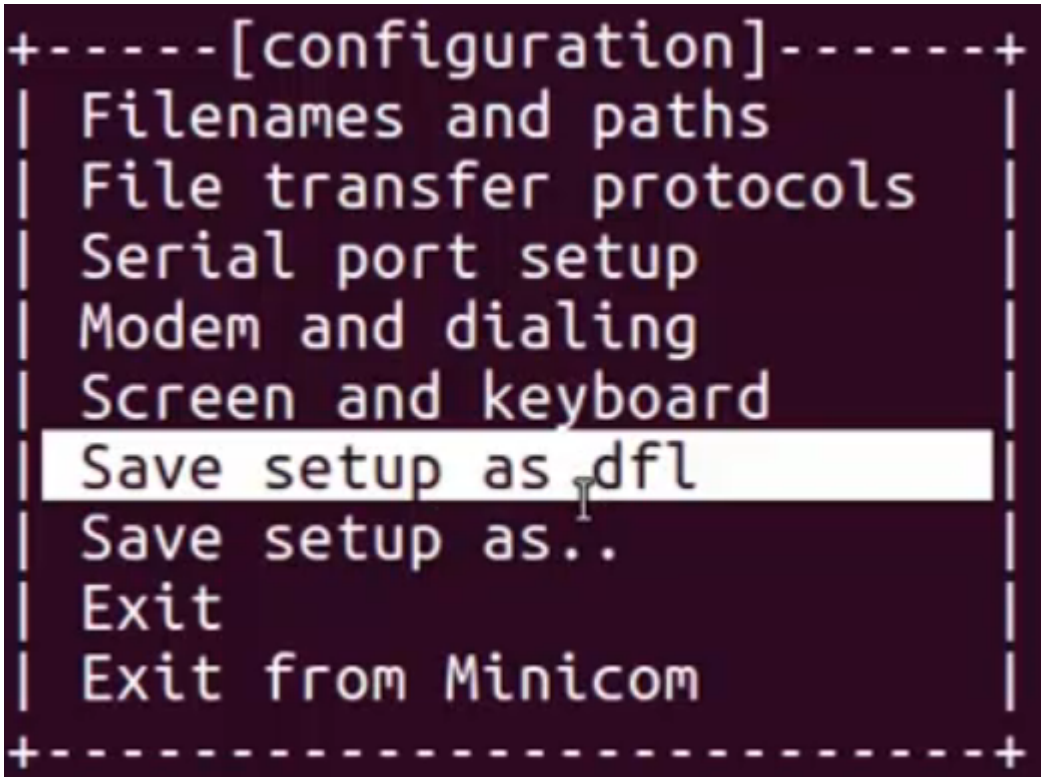
Biz BeagleBone Black için **115200** seçeceğiz. Bu ekrandaki harflerle hızı 115200, formatı **8N1** (8 bit, parity yok, 1 stop bit) yap ve Enter ile çık.

Ayarların son hali:

Ayar	**Değer**
Serial Device	`/dev/ttyUSB0`
Bps/Par/Bits	`115200 8N1`
Hardware Flow Control	**No**
Software Flow Control	**No**

> **Önemli**
>
> BeagleBone Black'in seri hattında **CTS** ve **RTS** pinleri yoktur. Bu yüzden **Hardware Flow Control** kullanılamaz ve **No** olmalıdır. Yes kalırsa ekranda kartın mesajlarını görürsün ama klavyeden yazdıkların karta gitmez. **Software Flow Control** da **No** kalsın.

Ayarları bitirince ana menüye dön ve **Save setup as dfl**'yi seç. Böylece ayarlar varsayılan olarak kaydedilir ve her seferinde yeniden yapman gerekmez. Sonra **Exit** ile ayar ekranından çık:



Görsel: kursun videosundan ekran görüntüsü.

2.5 Minicom'u Kullanmak

Ayarları kaydettikten sonra Minicom'u doğrudan açabilirsin. Ayarları komut satırında açıkça vermek de mümkün:

```
minicom                              # kayıtlı ayarlarla aç
minicom -D /dev/ttyUSB0 -b 115200    # portu ve hızı açıkça ver
```

Minicom'da bütün kısayollar önce `Ctrl-A`, ardından bir harf ile kullanılır:

Kısayol	Görevi
`Ctrl-A` → `Z`	Yardım: bütün kısayolların listesi
`Ctrl-A` → `S`	Dosya gönder (xmodem / ymodem). Bölüm 6 ve Bölüm 7'de kullanılır.

Kısayol	Görevi
`Ctrl-A` → `P`	Baud rate'i değiştir
`Ctrl-A` → `O`	Ayar menüsü (`minicom -s` ile aynı)
`Ctrl-A` → `C`	Ekranı temizle
`Ctrl-A` → `X`	Minicom'dan çık

> **İpucu**
>
> Minicom'u kartı açmadan **önce** başlat. Böylece açılıştaki ilk mesajları (SPL, U-Boot) kaçırmazsın.

2.6 Windows'ta: Tera Term

Kursun bazı derslerinde Windows'ta **Tera Term** kullanılıyor. Bu bir zorunluluk değil, Tera Term de Minicom gibi bir seri terminal programıdır. Hangisini kullanırsan kullan, komutlar ve çıktılar aynıdır.

	Linux	Windows
Program	`minicom`	Tera Term
Port adı	`/dev/ttyUSB0`	`COM3` gibi (Aygıt Yöneticisi → Bağlantı Noktaları'na bak)
Ayarlar	115200, 8N1, flow control yok	Setup → Serial port: 115200, 8 bit, none, 1 stop, Flow control: none

2.7 Özet

Ne?	Komut / Değer
Kurulum	`sudo apt install minicom lrzsz`
Port adını bulmak	`sudo dmesg | grep tty` → `/dev/ttyUSB0`
Ayarlar	`sudo minicom -s` → 115200 8N1, Hardware/Software Flow Control: **No** → Save setup as dfl
Açmak	`minicom -D /dev/ttyUSB0 -b 115200`
Çıkmak	`Ctrl-A` → `X`

2.8 Karşılaşılabilecek Problemler

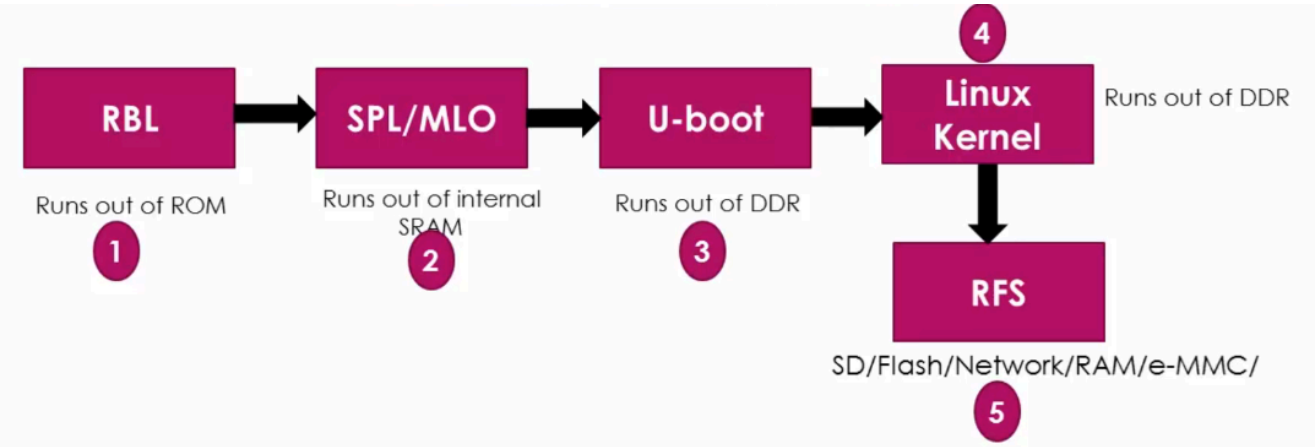
Belirti	Sebep	Çözüm
`cannot open /dev/ttyUSB0: Permission denied`	Kullanıcın `dialout` grubunda değil	`sudo usermod -aG dialout $USER` , sonra oturumu kapatıp aç

Belirti	Sebep	Çözüm
`dmesg`'de `ttyUSB` hiç görünmüyor	Kablo/sürücü sorunu ya da sanal makine USB'yi görmüyor	Başka USB portu dene. Sanal makinedeysen USB aygıtını VM'e bağla (VirtualBox: Devices → USB).
Ekranda hiçbir şey yok	TX/RX düz bağlanmış ya da GND bağlı değil	Dönüştürücünün TX'i → kartın RX'i (J1 pin 4), RX'i → kartın TX'i (J1 pin 5), GND → pin 1 (1.8)
Anlamsız / bozuk karakterler	Baud rate yanlış	115200 8N1 olmalı
Mesajlar geliyor ama yazdıklarım gitmiyor	Hardware Flow Control açık	`minicom -s` → Serial port setup → Hardware Flow Control: **No**
`sx: not found` (dosya gönderirken)	`lrzsz` kurulu değil	`sudo apt install lrzsz`

Kısım II
Boot Süreci

Bölüm 3 — Linux Boot Süreci: RBL → SPL → U-Boot

> **Bu bölümde:**
>
> - Kart açılırken çalışan programları (RBL, SPL/MLO, U-Boot, kernel) ve her birinin görevini öğreneceksin.
> - Neden tek bir bootloader yerine bir **zincir** olduğunu anlayacaksın.
> - SYSBOOT pinlerinin boot sırasını nasıl belirlediğini ve S2 butonunun bunu nasıl değiştirdiğini göreceksin.
>
> **Ön koşul:** Bölüm 1 — kartın bellekleri (SRAM, DDR, eMMC, SD) ve S2 butonu.



Görsel: kursun slaytı.

BeagleBone Black'e güç verildiğinde Linux tek adımda açılmaz. Açılış, birbirini yükleyen bir **zincir** şeklinde ilerler. Her aşama bir sonrakini belleğe yükler ve kontrolü ona devreder:

```
Güç → RBL (ROM) → SPL / MLO (SRAM) → U-Boot (DDR) → Linux kernel (DDR) → Root filesystem → init
```

Şemadaki **"Runs out of ..."** ifadeleri her aşamanın **hangi bellekte çalıştığını** gösterir. Zincirin neden bu kadar uzun olduğu da buradan anlaşılır.

> **Not**
>
> **Neden tek bir bootloader yok?** Açılışın ilk anında 512 MB'lık **DDR RAM henüz kullanılamaz**. DDR'ın çalışması için önce saat (clock), zamanlama ve voltaj ayarlarının yapılması gerekir ve bu ayarlar karttan karta değişir. İşlemcinin hemen kullanabildiği tek RAM, SoC'nin içindeki **çok küçük dahili SRAM**'dir. Bu yüzden önce SRAM'e sığacak kadar küçük bir program (SPL) çalışır, DDR'ı hazırlar, ardından büyük programlar (U-Boot, Kernel) DDR'a yüklenebilir.

3.1 RBL (ROM Boot Loader)

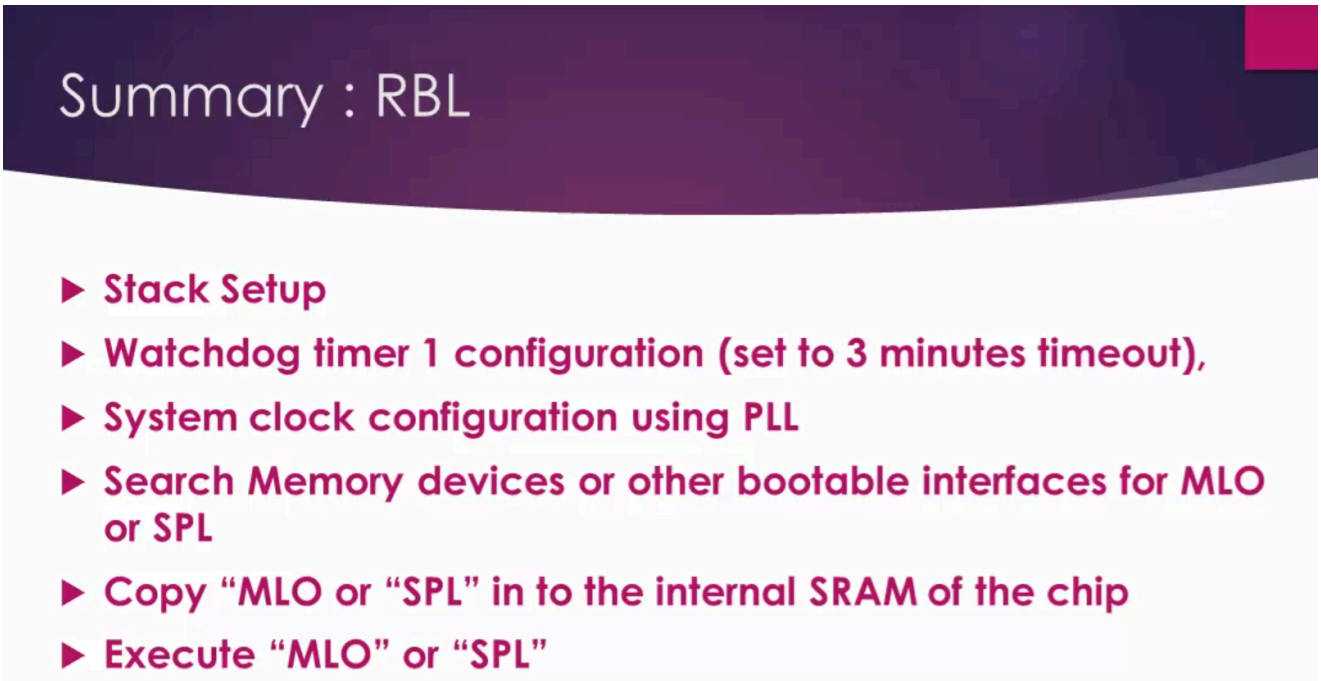
- Karta enerji verildiğinde SoC (System on Chip) üzerinde çalışan ***ilk kod parçasıdır***.
- Üretici firma (Texas Instruments) tarafından yazılır ve çip üretilirken SoC'nin içindeki **ROM** belleğe kaydedilir. ROM'da olduğu için **değiştirilemez**.
- **Nerede çalışır:** Doğrudan **ROM** üzerinden (Runs out of ROM).

Görevleri:

1. İşlemcinin temel donanımını (saat, watchdog vb.) minimum seviyede başlatır.
2. **SYSBOOT** pinlerinin seviyesini okuyarak **boot sırasını** belirler. Beaglebone'da bu sırayı **S2 (Boot)** butonu değiştirir: - S2 basılı değilse: **eMMC** → microSD → UART → USB - S2 basılıysa: SPI → **microSD** → USB → UART
3. Bu sırayla cihazları tek tek dener ve ilk bulduğu geçerli **ikinci aşama bootloader**'ı (**MLO**) dahili SRAM'e kopyalar.
4. Kontrolü MLO'ya devreder.

> **Not**
>
> SD karttan boot ederken RBL, kartın **FAT formatlı ilk bölümünde** (boot partition) `MLO` adlı dosyayı arar. Bu yüzden kendi boot SD kartımızı hazırlarken MLO dosyasını bu bölüme koyacağız (Bölüm 4).

RBL'in yaptığı işler

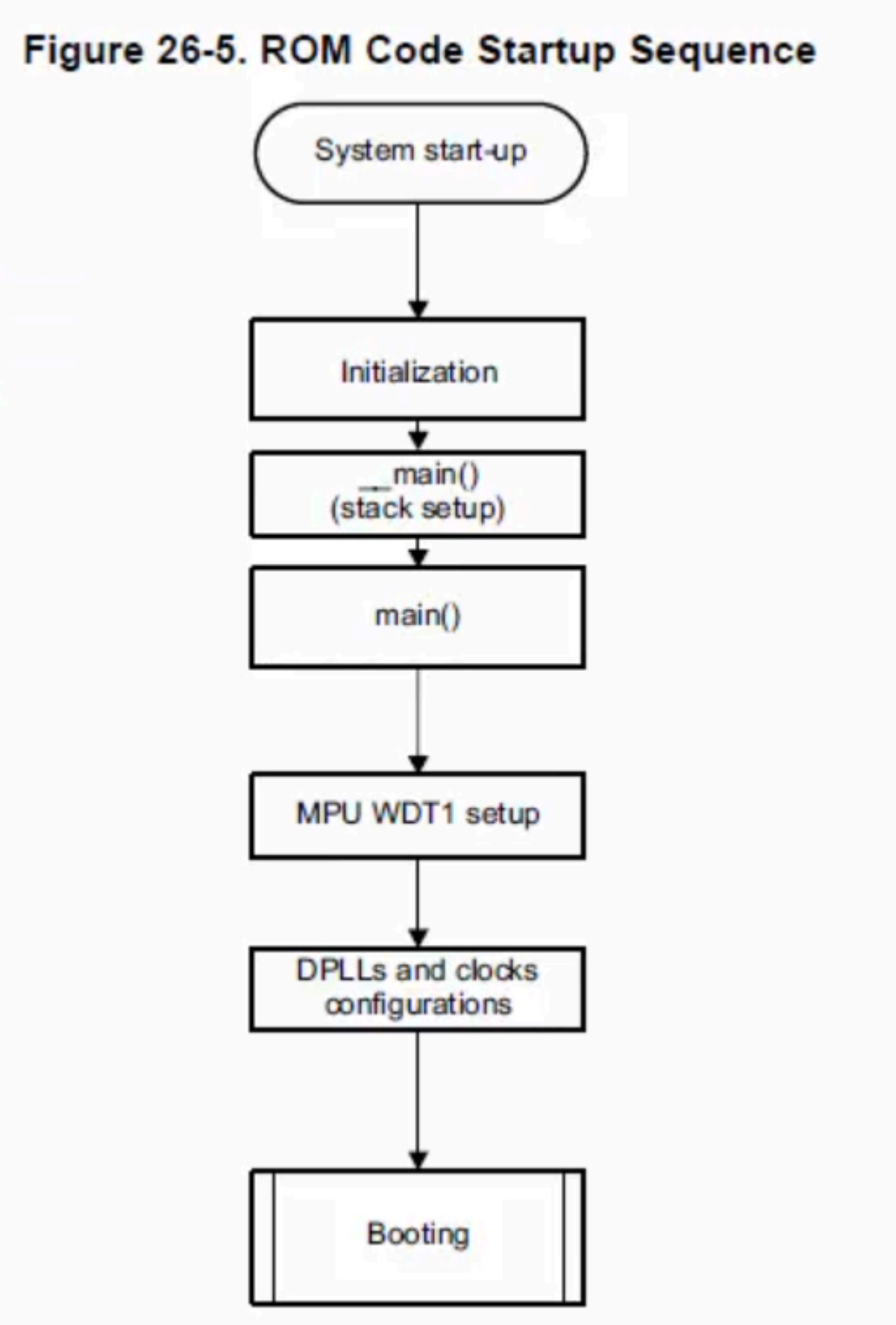


Görsel: kursun slaytı.

- **Stack Setup:** RBL'in C ile yazılmış fonksiyonları çalıştırabilmesi için önce bir **stack** (yığın) alanı gerekir. Fonksiyon çağrıları ve yerel değişkenler burada tutulur. RBL stack'i dahili SRAM'de kurar.

- **Watchdog Timer 1 (3 dakika):** Watchdog, belirli bir süre içinde "beslenmezse" (sıfırlanmazsa) sistemi **resetleyen** bir sayaçtır. Amacı, yazılım bir yerde takılırsa kartı kendiliğinden yeniden başlatmaktır. RBL bu sayacı **3 dakikaya** kurar ve açık bırakır. Bu yüzden sonraki aşamalar (SPL, U-Boot, Linux) bu watchdog'u ya kapatmalı ya da düzenli olarak beslemelidir. Yoksa kart 3 dakika sonra resetlenir.
- **System clock configuration using PLL:** İşlemcinin ve çevre birimlerinin saatlerini PLL ile kurar (aşağıda açıklanıyor).
- **MLO/SPL'i arar, SRAM'e kopyalar ve çalıştırır:** Yukarıdaki görev listesinin 2-4. maddeleri.

ROM Code Startup Sequence

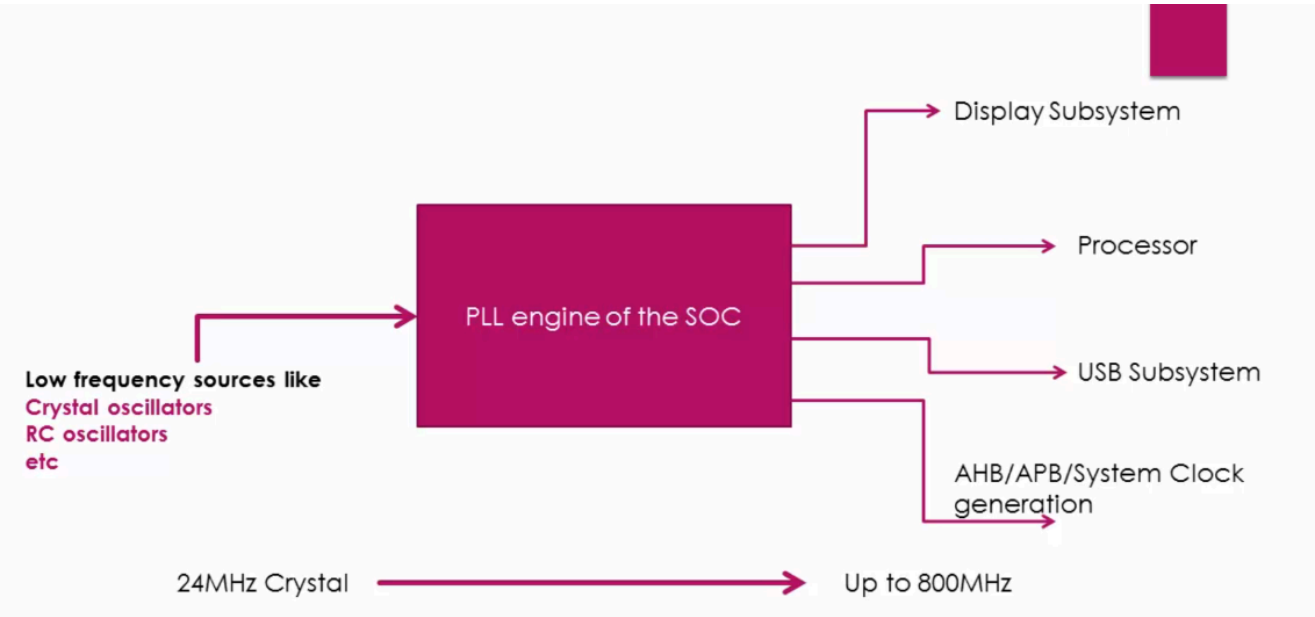


Bu akış şeması AM335x TRM'den (Figure 26-5) alınmıştır ve RBL'in açılışta yaptığı işleri sırasıyla gösterir:

1. **System start-up:** Karta güç verilir veya reset gelir.
2. **Initialization:** İşlemcinin en temel ayarları yapılır.
3. **__main() (stack setup):** Stack kurulur. Bundan sonra C kodu çalışabilir.
4. **main():** RBL'in ana fonksiyonu başlar.
5. **MPU WDT1 setup:** Watchdog Timer 1 kurulur (3 dakika).
6. **DPLLs and clocks configuration:** PLL'ler ve saatler ayarlanır.

7. **Booting:** Boot cihazları aranır ve MLO yüklenir. Bu adımın detayı aşağıdaki **ROM Code Booting Procedure** şemasındadır.

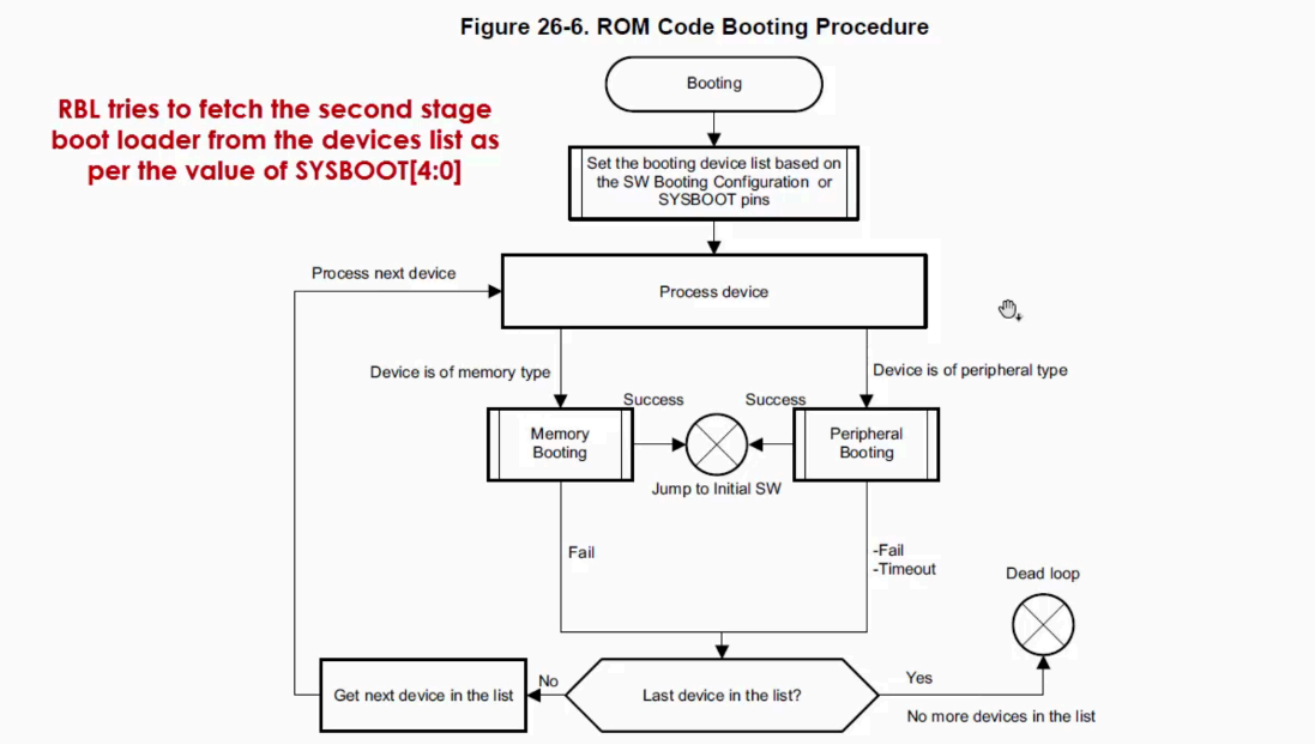
PLL (Phase Locked Loop) nedir?



Görsel: kursun slaytı.

- Kart üzerindeki kristal osilatör **düşük frekanslıdır** (BBB'de **24 MHz**). Ama işlemci yüzlerce MHz ile çalışmak ister.
- **PLL**, SoC içindeki bir devredir. Kristalden gelen düşük frekansı **çarparak** (ve gerektiğinde bölerek) yüksek frekanslar üretir. Şemadaki örnekte 24 MHz'lik kristalden **800 MHz**'e kadar frekans elde edilir.
- Her birim farklı bir frekansa ihtiyaç duyar. Bu yüzden SoC'de birden fazla PLL vardır ve her biri farklı bir birimi besler: **işlemci**, **USB**, **ekran (display)**, **bus saatleri** (AHB/APB) gibi.
- Doğrudan yüksek frekanslı bir kristal kullanılmamasının sebebi: Düşük frekanslı kristaller ucuz ve kararlıdır. Tek bir kristalden PLL'ler ile ihtiyaç duyulan tüm frekansları üretmek çok daha pratiktir.

ROM Code Booting Procedure



Bu şema (TRM Figure 26-6), RBL'in ikinci aşama bootloader'ı (MLO) nasıl aradığını gösterir:

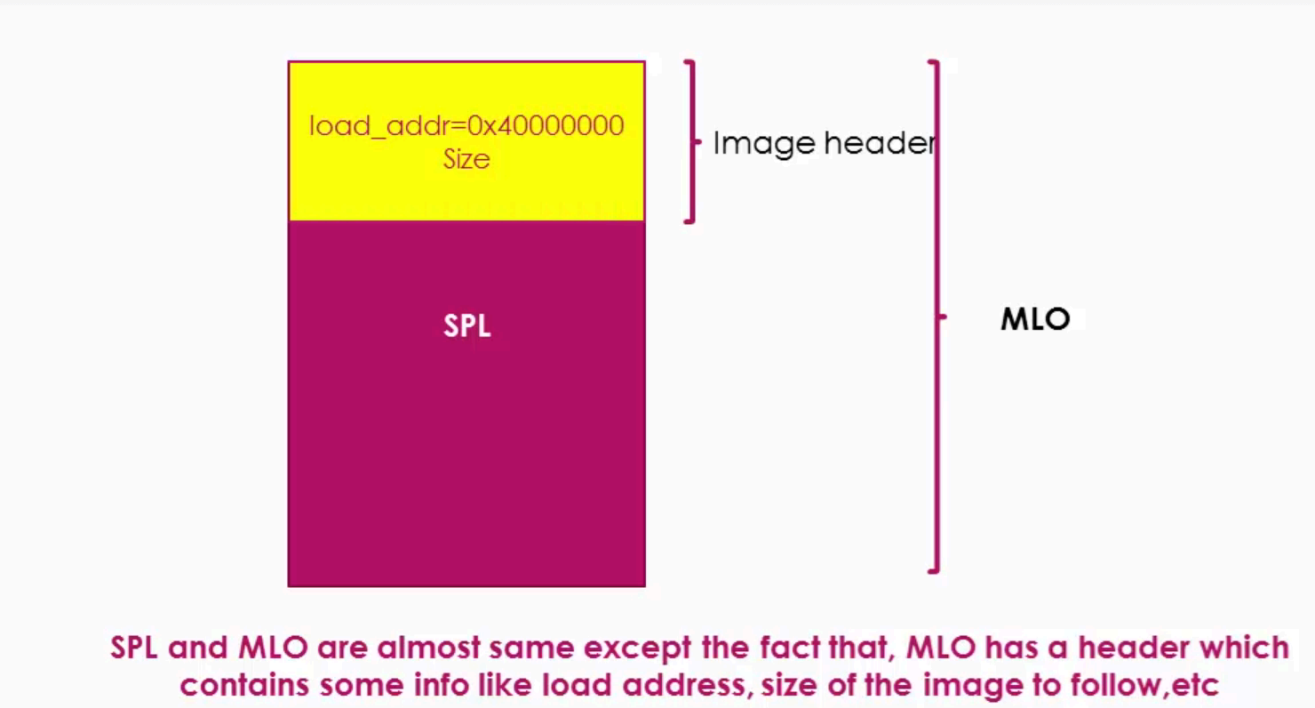
1. **Boot cihaz listesi oluşturulur:** RBL, **SYSBOOT[4:0]** değerine bakarak hangi cihazları hangi sırayla deneyeceğini belirler (3.5).
2. **Process device:** Listedeki sıradaki cihaz denenir. Cihazlar iki türe ayrılır: - **Memory type (bellek türü):** eMMC, SD kart, NAND, SPI flash gibi. MLO bu belleğin içinde **hazır durur**, RBL sadece okur. - **Peripheral type (çevre birimi türü):** UART, USB, Ethernet gibi. MLO kartta yoktur, **dışarıdan** (bir bilgisayardan) gönderilmesi beklenir. Bu yüzden burada **timeout** (zaman aşımı) vardır: Belirli süre içinde imaj gelmezse başarısız sayılır.
3. **Success:** MLO bulunduysa **"Jump to Initial SW"**, yani MLO'ya atlanır ve kontrol ona geçer.
4. **Fail / Timeout:** Listede başka cihaz varsa **bir sonrakine** geçilir.
5. **Dead loop:** Listedeki hiçbir cihazda MLO bulunamazsa RBL sonsuz bir döngüye girer ve kart hiçbir şey yapmaz.

> **İpucu**
>
> BBB'nin varsayılan boot listesinde **UART0** da vardır. eMMC'de ve SD kartta geçerli bir MLO yoksa RBL sırayı UART0'a getirir ve seri konsoldan X-Modem ile imaj bekler. Bu sırada seri konsolda sürekli `CCCCCCCC...` yazdığını görürsün. Bu çıktı, **"RBL MLO bulamadı"** anlamına gelir. Bu davranışı Bölüm 7'de kartı seri porttan açmak için kullanacağız.

3.2 SPL / MLO (Secondary Program Loader)

- **SPL** = Secondary Program Loader (İkinci Aşama Önyükleyici). TI işlemcilerinde bu dosyanın adı **MLO**'dur (MLO = MMC Loader). SPL ile MLO **neredeyse** aynı şeydir (farkı aşağıda).
- SPL ayrı bir program değildir. **U-Boot'un kaynak kodundan derlenir**, ama sadece gerekli kısımları içeren çok küçük bir versiyonudur. U-Boot'u derlediğimizde hem `MLO` hem de `u-boot.img` dosyaları birlikte oluşur (Bölüm 9).
- **Nerede çalışır:** SoC içindeki **dahili SRAM** üzerinde (Runs out of internal SRAM). SRAM çok küçük olduğu için SPL de küçük olmak zorundadır.

SPL ile MLO arasındaki fark



Görsel: kursun slaytı.

- **MLO = Image header + SPL**. Yani MLO, başına küçük bir **başlık (header)** eklenmiş SPL'dir.
- Header'da RBL'in ihtiyaç duyduğu bilgiler bulunur:
- **load_addr:** SPL'in belleğin **hangi adresine** kopyalanacağı
- **Size:** Header'dan sonra gelen SPL kodunun **boyutu**
- RBL önce header'ı okur. Böylece SPL'i nereye kopyalayacağını ve ne kadar veri okuyacağını öğrenir, kopyalamayı yapar ve o adrese atlar.
- U-Boot derlendiğinde önce header'sız SPL (`u-boot-spl.bin`) oluşur. Başına header eklenince `MLO` dosyası elde edilir.

> **Not**
>
> Slayttaki `0x40000000` adresi bir örnektir. AM335x'te MLO gerçekte dahili SRAM'deki `0x402F0400` adresine yüklenir.

Görevleri:

1. **DDR RAM'i başlatır.** En kritik görevi budur. Saat (PLL) ve DDR zamanlama ayarlarını yapar, böylece 512 MB'lık RAM kullanılabilir hale gelir.
2. Seri konsol (UART0) gibi gerekli birkaç çevre birimini hazırlar. Seri konsolda gördüğün ilk `U-Boot SPL ...` satırı buradan gelir.
3. Üçüncü aşama bootloader'ı, yani **U-Boot**'u (`u-boot.img`) eMMC/SD karttan okuyup **DDR**'a yükler.
4. Kontrolü U-Boot'a devreder.

Soru: AM335x'in RBL'i neden U-Boot'u doğrudan DDR'a yükleyemez?

Cevap: Çünkü RBL, karta **hangi DDR belleğin takılı olduğunu bilmez**. Bilmediği bir belleği de başlatamaz.

- DDR RAM'den veri okuyup yazabilmek için önce SoC içindeki **DDR kontrolcüsünün** ve RAM'in **doğru parametrelerle** ayarlanması gerekir: bellek tipi (DDR2, DDR3, DDR3L...), boyut, hız, zamanlama (timing) değerleri vb.
- Bu parametreler **tamamen karta (ürüne) özeldir**. Aynı AM335x işlemcisini kullanan farklı firmalar farklı bellekler seçebilir:
- X firması: Micron marka **DDR3**
- Y firması: Başka bir markanın **DDR2**'si
- Z firması: Hiç DDR kullanmayabilir
- RBL çip üretilirken ROM'a yazılır. O an çipin hangi kartta, hangi bellekle kullanılacağı belli değildir. Bu yüzden RBL **DDR'ı hiç başlatmaz**. Sadece her işlemcide garanti olarak bulunan **dahili SRAM**'i kullanır.
- U-Boot ise dahili SRAM'e **sığmayacak kadar büyüktür**. Yani RBL'in U-Boot'u yükleyebileceği bir yer yoktur.

Çözüm SPL'dir: SPL, dahili SRAM'e sığacak kadar küçüktür ve kartın DDR ayarlarını **bilir**, çünkü o kart için derlenmiştir. RBL, SPL'i SRAM'e yükler. SPL de DDR'ı başlatıp U-Boot'u DDR'a yükler.

> **Not**
>
> Kendi kartını tasarlayıp BBB'den farklı bir DDR kullanırsan, SPL kaynak kodundaki **DDR parametrelerini** kendi belleğine göre değiştirip SPL'i yeniden derlemen gerekir.

> **Not**
>
> **Saat (clock) ayarları:** RBL'in kodu ROM'da olduğu için saat ayarları **değiştirilemez**. SYSBOOT[15:14] ile sadece karttaki kristalin frekansı bildirilir. İşlemci (MPU), DDR ve çevre birimi frekanslarını asıl kuran **SPL**'dir. SPL'i kendimiz derlediğimiz için bu ayarları kaynak kodda değiştirip yeniden derleyebiliriz. Açılıştan sonra Linux da **cpufreq** ile işlemci frekansını çalışma sırasında değiştirebilir.

3.3 U-Boot (Universal Bootloader)

- Açık kaynak kodlu ve gömülü sistemlerde en yaygın kullanılan bootloader'dır. Üçüncü (ve son) aşama önyükleyicidir.
- **Nerede çalışır: DDR RAM** üzerinde (Runs out of DDR). Artık bellek kısıtı olmadığı için çok daha fazla özellik içerebilir.

Görevleri:

1. Linux Kernel'i çalıştırmak için gereken donanımları (eMMC, SD kart, Ethernet, USB vb.) başlatır.
2. **Linux Kernel** imajını (`zImage` / `uImage`) bir kaynaktan (SD kart, eMMC, seri port, ağ üzerinden TFTP vb.) okuyup **DDR**'a yükler.
3. Kartın donanımını tarif eden **Device Tree** dosyasını (BeagleBone Black için `am335x-boneblack.dtb`) DDR'a yükler.
4. Kernel'e **bootargs** (komut satırı parametreleri) iletir. Örneğin root filesystem'in nerede olduğu (`root=`) veya hangi konsolun kullanılacağı (`console=`) bu parametrelerle bildirilir.
5. Kontrolü Linux Kernel'e devreder. Bu noktadan sonra U-Boot'un işi biter.

Neden önemli:

- U-Boot'un kendi **komut satırı** vardır. Açılışta seri konsolda geri sayım (`Hit any key to stop autoboot`) sırasında bir tuşa basarsan U-Boot komut satırına (`U-Boot#` veya `=>`) düşersin. Buradan bellek okuma/yazma, dosya yükleme, boot ayarlarını değiştirme gibi işlemler yapılabilir (Bölüm 6).
- Boot ayarları **environment değişkenlerinde** (`bootcmd` , `bootargs` vb.) tutulur ve `printenv` / `setenv` komutlarıyla görüntülenip değiştirilebilir.

3.4 Linux Kernel ve Root Filesystem

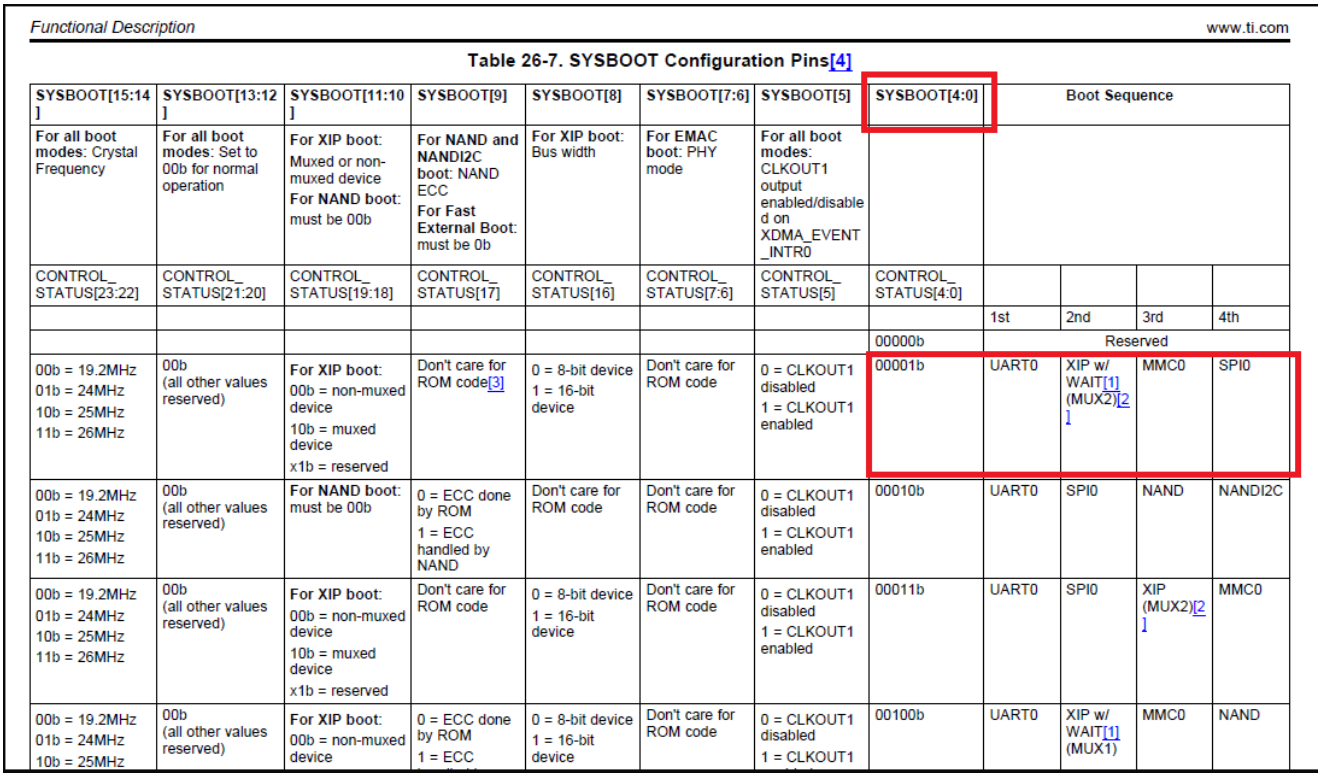
U-Boot `bootm` (veya `bootz`) komutuyla kernel'e atladıktan sonra zincirin son iki halkası çalışır:

1. **Kernel** kendini RAM'de açar (`zImage` sıkıştırılmıştır), U-Boot'un verdiği **Device Tree**'yi okuyup kartta hangi donanımın olduğunu öğrenir ve sürücüleri başlatır (Bölüm 11).
2. Kernel, `bootargs` 'taki **`root=`** parametresine bakarak **root filesystem**'i (`/`) bağlar. Bu bir SD kart bölümü (`/dev/mmcblk0p2`), eMMC bölümü veya RAM'deki bir initramfs olabilir.
3. Rootfs'teki ilk program olan **`init`** 'i (systemd veya BusyBox init) çalıştırır. Bundan sonra servisler başlar ve `login:` ekranı gelir.

> **Not**
>
> `console=` parametresindeki konsol adı kernel sürümüne göre değişir. Kursun hazır kernel'inde (3.8.13) UART0'ın adı `ttyO0` , yeni kernel'lerde `ttyS0` 'dır. Yanlış ad yazılırsa kernel açılır ama mesajlarını göremezsin.

3.5 Boot Kaynakları ve Boot Sırası (SYSBOOT Pinleri)

AM335x işlemcisi tek bir yerden boot etmek zorunda değildir. **Birçok farklı çevre biriminden** (eMMC, SD kart, UART, USB, SPI flash, NAND, Ethernet...) boot edebilir. Peki RBL hangisinden boot edeceğine nasıl karar verir? Bunu işlemcinin **SYSBOOT** pinleri belirler.



Functional Description www.ti.com

Table 26-7. SYSBOOT Configuration Pins[4]

SYSBOOT[15:14]	SYSBOOT[13:12]	SYSBOOT[11:10]	SYSBOOT[9]	SYSBOOT[8]	SYSBOOT[7:6]	SYSBOOT[5]	SYSBOOT[4:0]	Boot Sequence			
For all boot modes: Crystal Frequency	**For all boot modes:** Set to 00b for normal operation	**For XIP boot:** Muxed or non-muxed device **For NAND boot:** must be 00b	**For NAND and NANDI2C boot:** NAND ECC **For Fast External Boot:** must be 0b	**For XIP boot:** Bus width	**For EMAC boot:** PHY mode	**For all boot modes:** CLKOUT1 output enabled/disabled on XDMA_EVENT_INTR0					
CONTROL_STATUS[23:22]	CONTROL_STATUS[21:20]	CONTROL_STATUS[19:18]	CONTROL_STATUS[17]	CONTROL_STATUS[16]	CONTROL_STATUS[7:6]	CONTROL_STATUS[5]	CONTROL_STATUS[4:0]				
								1st	2nd	3rd	4th
							00000b	Reserved			
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	**For XIP boot:** 00b = non-muxed device 10b = muxed device x1b = reserved	Don't care for ROM code[3]	0 = 8-bit device 1 = 16-bit device	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	00001b	UART0	XIP w/ WAIT[1] (MUX2)[2]	MMC0	SPI0
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	**For NAND boot:** must be 00b	0 = ECC done by ROM 1 = ECC handled by NAND	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	00010b	UART0	SPI0	NAND	NANDI2C
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	**For XIP boot:** 00b = non-muxed device 10b = muxed device x1b = reserved	Don't care for ROM code	0 = 8-bit device 1 = 16-bit device	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	00011b	UART0	SPI0	XIP (MUX2)[2]	MMC0
00b = 19.2MHz 01b = 24MHz 10b = 25MHz	00b (all other values reserved)	**For XIP boot:** 00b = non-muxed device	0 = ECC done by ROM 1 = ECC	0 = 8-bit device 1 = 16-bit device	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1	00100b	UART0	XIP w/ WAIT[1] (MUX1)	MMC0	NAND

Bu tablo, AM335x'in **Technical Reference Manual (TRM)** dokümanındaki **Table 26-7: SYSBOOT Configuration Pins** tablosudur.

SYSBOOT pinleri nedir?

- **SYSBOOT[15:0]**, işlemcinin 16 adet pinidir. Karta **güç verildiği anda** RBL bu pinlerin seviyesini (1 veya 0) okur.
- Okunan değer, **CONTROL_STATUS** adlı bir register'a kaydedilir. Tablonun 3. satırındaki `CONTROL_STATUS[...]` ifadeleri, her SYSBOOT bitinin bu register'da hangi bite karşılık geldiğini gösterir.
- Bu pinlerin seviyesi kart üzerindeki **pull-up / pull-down dirençleriyle** sabitlenir. Yani boot ayarı, kartı tasarlayan kişi tarafından donanımsal olarak belirlenir.

Tablonun sütunları

Sütun	Ne işe yarar?
SYSBOOT[15:14]	Karttaki kristalin (osilatörün) frekansını bildirir: 19.2 / 24 / 25 / 26 MHz. Beaglebone Black **24 MHz** kristal kullanır (**01b**).
SYSBOOT[13:12]	Normal çalışma için **00b** olmalıdır.

Sütun	Ne işe yarar?
SYSBOOT[11:10], [9], [8]	Sadece belirli boot türlerinde (XIP, NAND) kullanılan ek ayarlar. Diğer boot türlerinde önemsizdir ("Don't care").
SYSBOOT[7:6]	Ethernet (EMAC) üzerinden boot için PHY modu.
SYSBOOT[5]	CLKOUT1 saat çıkışının açık/kapalı olması.
SYSBOOT[4:0]	**En önemli kısım.** Boot modunu, yani **hangi cihazların hangi sırayla** deneneceğini belirler.
Boot Sequence	SYSBOOT[4:0] değerine göre RBL'in deneyeceği cihazlar: **1st → 2nd → 3rd → 4th**.

Boot Sequence (Boot sırası) nasıl çalışır?

Resimde kırmızıyla işaretlenen satırı örnek alalım. **SYSBOOT[4:0] = 00001b** ise boot sırası şöyledir:

```
1st: UART0  →  2nd: XIP (NOR flash)  →  3rd: MMC0 (SD kart)  →  4th: SPI0
```

1. RBL önce **1. cihazı** (UART0) dener.
2. O cihazda geçerli bir boot imajı (MLO) bulamazsa veya cihaz yanıt vermezse **bir sonraki cihaza** geçer.
3. Geçerli bir imaj bulduğu **ilk cihazdan** boot eder ve diğerlerine bakmaz.
4. Dört cihazın hiçbirinde imaj bulamazsa boot başarısız olur.

SYSBOOT[4:0] 5 bit olduğu için 32 farklı kombinasyon vardır. Her biri farklı bir boot sırası tanımlar. Resimde bunların sadece ilk birkaçı görünüyor, tablonun tamamı TRM'de.

Tablodaki boot kaynakları

İsim	Anlamı
MMC0	Beaglebone'da **microSD kart** yuvası
MMC1	Beaglebone'da kart üstündeki **eMMC**
UART0	Seri port üzerinden boot. İmaj bilgisayardan seri hat ile (X-Modem protokolü) gönderilir (Bölüm 7).
USB0	USB üzerinden boot. Kart, bilgisayara bir ağ cihazı gibi görünür ve imajı oradan alır.
SPI0	SPI ile bağlı harici flash bellekten boot
XIP	e**X**ecute **I**n **P**lace: Kod, NOR flash'a kopyalanmadan doğrudan orada çalıştırılır.
NAND / NANDI2C	Harici NAND flash bellekten boot
EMAC	Ethernet üzerinden ağdan boot

> **Not**
>
> UART0 ve USB0 boot modları, kartta hiç yazılım olmadığında (örneğin eMMC boşsa veya bozulduysa) kartı **kurtarmak** için kullanılır.

BeagleBone Black'te durum

BeagleBone Black'in boot ayarları, resmi dokümandaki şu tabloda görünür:

SYSBOOT[15:14]	SYSBOOT[13:12]	SYSBOOT[11:10]	SYSBOOT[9]	SYSBOOT[8]	SYSBOOT[7:6]	SYSBOOT[5]	SYSBOOT[4:0]	Boot Sequence			
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11100b	MMC1	MMC0	UART0	USB0[5]
00b = 19.2MHz 01b = 24MHz 10b = 25MHz 11b = 26MHz	00b (all other values reserved)	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	Don't care for ROM code	0 = CLKOUT1 disabled 1 = CLKOUT1 enabled	11000b	SPI0	MMC0	USB0[5]	UART0

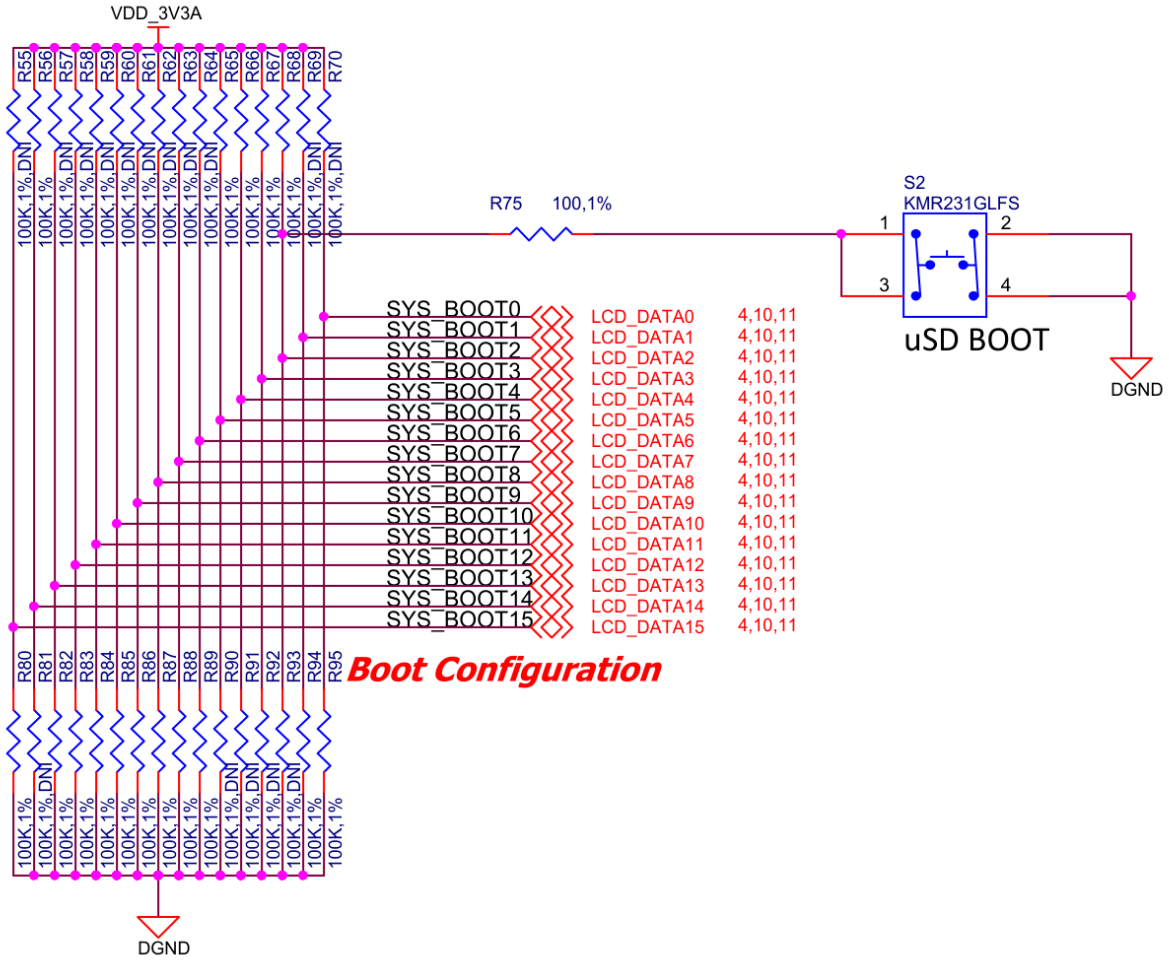
Durum	SYSBOOT[4:0]	Boot Sequence
S2 basılı değil (varsayılan)	**11100b**	**MMC1 (eMMC)** → MMC0 (SD) → UART0 → USB0
S2 basılı (güç verilirken)	**11000b**	SPI0 → **MMC0 (SD)** → USB0 → UART0

- İki değer arasında sadece **SYSBOOT[2]** biti farklıdır: 11**1**00 → 11**0**00. Boot (**S2**) butonu tam olarak bu biti **0**'a çeker.
- Varsayılan durumda eMMC ilk sırada olduğu için kart **eMMC'deki** hazır Debian'dan açılır.
- S2'ye basılıyken eMMC listede **hiç yoktur**. SPI0'da flash olmadığı için o adım atlanır ve kart **SD karttan** açılır. Kendi derlediğimiz MLO/U-Boot/Kernel'i bu yüzden SD karta koyup S2 ile boot edeceğiz.
- Beaglebone'da SYSBOOT pinleri **LCD veri pinleriyle** (LCD_DATA0–15) ortaktır ve P8 header'ına çıkar. Bu pinlere boot sırasında dışarıdan sinyal verilirse boot sırası bozulabilir. Bu yüzden kart açılmadan P8'e bir şey bağlarken dikkatli olmak gerekir.
- Listede **EMAC (Ethernet) yoktur**. ROM Code BBB'de ağdan boot edemez. Ağdan boot için önce SD karttan U-Boot'u açmak gerekir (Bölüm 8).

> **Önemli**
>
> SYSBOOT pinleri **sadece güç verildiği anda** okunur. Kart açıldıktan sonra S2'ye basmak veya Reset butonuna basmak boot sırasını değiştirmez. S2 ile boot etmek için gücü tamamen kesip, S2'yi basılı tutarak tekrar güç vermelisin.

3.6 Schematic'te Boot Devresi

Yukarıda anlatılan boot ayarının kart üzerindeki karşılığı, BBB schematic'inin **6. sayfasındaki** devredir.



> Kaynak: BeagleBone Black Rev C schematic, sayfa 6 (niekiran/EmbeddedLinuxBBB reposundaki `BBB_SCH.pdf`)

Schematic'te dikkat edilecekler:

- Her **SYS_BOOTx** hattına üstte bir pull-up (R55–R70), altta bir pull-down (R80–R95) direnci için yer var. Bunlardan sadece biri karta takılıdır. **DNI** (Do Not Install) yazan direnç kartta **yoktur**.
- Takılı dirençlere bakınca SYS_BOOT[4:0] = **11100b** çıkar. Bu değer, yukarıdaki tablodaki varsayılan değerle aynıdır.
- **S2** butonu (üzerinde **"uSD BOOT"** yazar), **R75** üzerinden **SYS_BOOT2** hattını **GND**'ye bağlar.

3.7 Özet

Aşama	Kim yazar?	Nerede saklanır?	Nerede çalışır?	Ana görevi
RBL	TI (çip üreticisi)	SoC içindeki ROM	ROM	Boot cihazını seçip MLO'yu SRAM'e yüklemek

Aşama	Kim yazar?	Nerede saklanır?	Nerede çalışır?	Ana görevi
SPL/MLO	U-Boot projesi (biz derleriz)	eMMC / SD kart	Dahili SRAM	DDR'ı başlatıp U-Boot'u DDR'a yüklemek
U-Boot	U-Boot projesi (biz derleriz)	eMMC / SD kart	DDR	Kernel + DTB'yi yükleyip Kernel'i başlatmak
Linux kernel	Linux projesi (biz derleriz)	eMMC / SD kart / ağ	DDR	Donanımı başlatmak, rootfs'i bağlamak, `init`'i çalıştırmak

- Boot sırasını **SYSBOOT[4:0]** pinleri belirler. BBB'de varsayılan **eMMC → SD → UART0 → USB0**, S2 basılıyken **SPI0 → SD → USB0 → UART0**.
- S2 sadece **güç verilirken** etkilidir.

Bölüm 4 — SD Karttan Boot

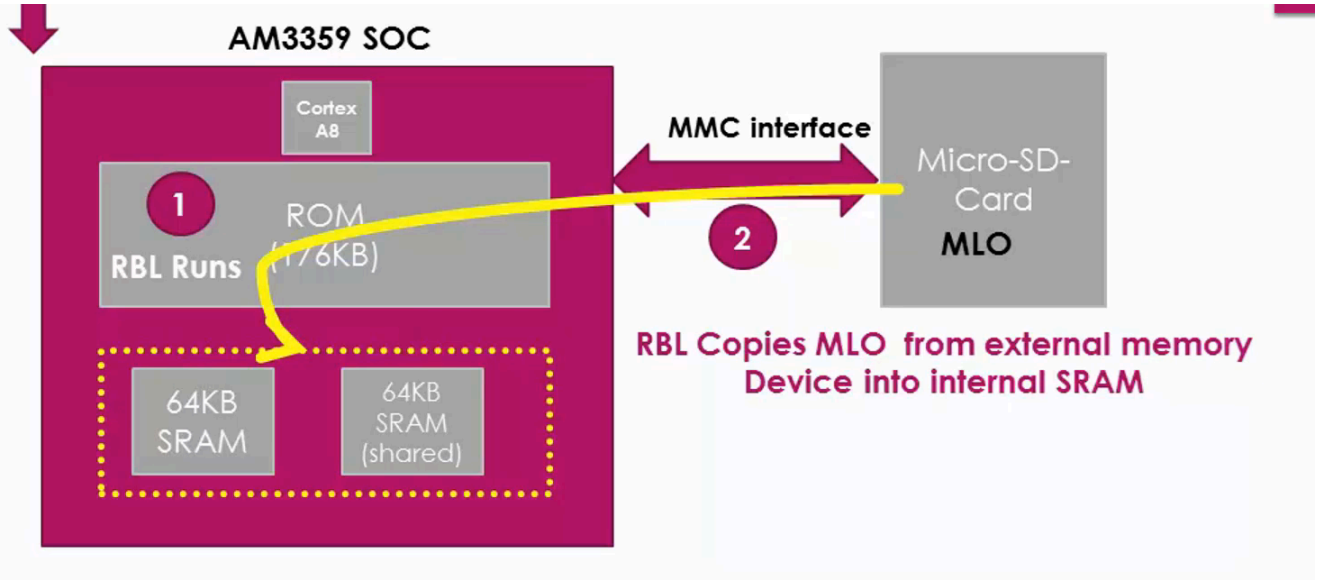
> **Bu bölümde:**
>
> - Bir microSD kartı GParted ile BOOT (FAT) ve ROOTFS (ext3) bölümlerine ayıracaksın.
> - Kursun hazır dosyalarını karta **tek tek** koyup her seferinde kartı açarak boot zincirinin her halkasının çalıştığını göreceksin.
> - `uEnv.txt`'nin bu açılışta ne yaptığını ve `uImage` dosyasının yapısını öğreneceksin.
> - U-Boot'u durdurup kernel'in başlığını (header) bellekten byte byte okuyacaksın.
>
> **Ön koşul:** Bölüm 2 — seri konsol, Bölüm 3 — boot zinciri.

4.1 Büyük Resim

Kart açılırken programlar sırayla birbirini çalıştırır. Buna **boot zinciri** denir (Bölüm 3):

```
Güç → RBL → MLO → U-Boot → Linux Kernel → Root Filesystem → login ekranı
```

Aşama	Ne yapar?	Nerede durur?
RBL (ROM Boot Loader)	Kart açılınca ilk çalışan kod. MLO'yu bulup çalıştırır.	İşlemcinin içinde, değiştirilemez
MLO	RAM'i (DDR) hazırlar, U-Boot'u yükler.	SD kart → BOOT bölümü
U-Boot	Kernel'i ve device tree'yi RAM'e yükleyip Linux'u başlatır.	SD kart → BOOT bölümü
Linux Kernel	İşletim sisteminin çekirdeği.	SD kart → ROOTFS bölümü (`/boot`)
Root Filesystem	Linux'un tüm dosyaları (`/bin`, `/etc`, ...).	SD kart → ROOTFS bölümü



Görsel: kursun slaytı.

SD kartın son hali şöyle olacak:

```
SD Kart
├── BOOT   (FAT16, ~1 GB)  →  MLO, u-boot.img, uEnv.txt
└── ROOTFS (ext3, kalan)   →  Linux dosyaları (kernel ve .dtb dahil)
```

Yol haritası

Dosyalar karta **tek tek** eklenir ve her seferinde kart açılıp test edilir. Böylece zincirin her halkasının çalıştığı görülür:

Adım	Ne yapılıyor?	Sonuç
4.2 – 4.4	Dosyaları indir, SD kartı bölümle	SD kart hazır
4.6	MLO'yu kopyala, test et	RBL ✓ → MLO ✓ → U-Boot ✗
4.7	U-Boot'u kopyala, test et	… → U-Boot ✓
4.8	Rootfs ve uEnv.txt'yi kopyala, test et	Linux açılır ✓
4.10	U-Boot'u durdur, uImage header'ını elle oku	Header çözüldü ✓

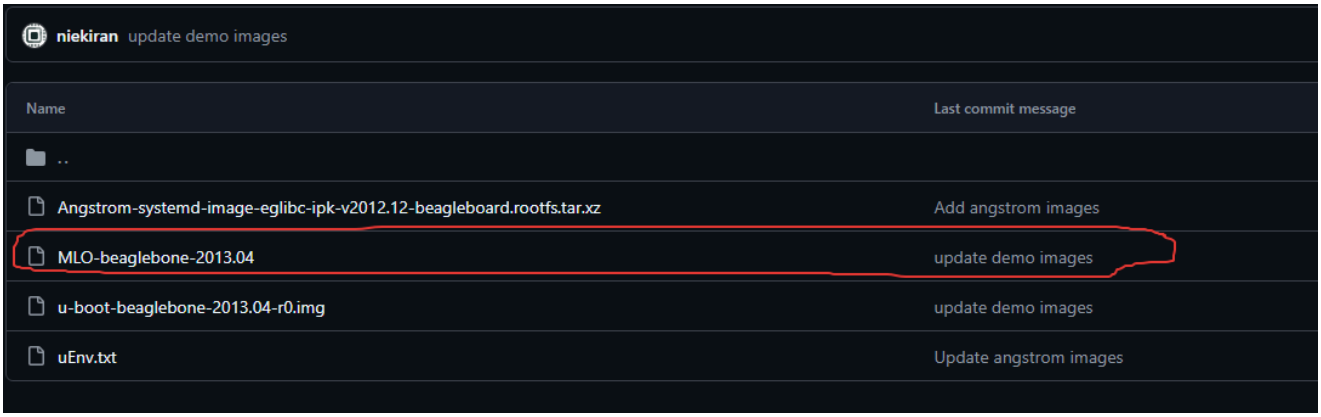
Her testte kart aynı şekilde açılır. Bu işlem 4.5'te bir kez anlatılıyor.

4.2 Hazırlık

Gerekenler:

- microSD kart (en az 4 GB) ve kart okuyucu
- USB-TTL seri dönüştürücü (3.3V). Kartın açılış mesajlarını görmek için gerekli.
- Mini USB kablo (karta güç vermek için)

Dosyaları indir. Kursun GitHub reposundaki **4 dosyanın hepsini** indir:
https://github.com/niekiran/EmbeddedLinuxBBB/tree/master/pre-built-images/Angstrom_Demo



Görsel: kursun GitHub reposu (niekiran/EmbeddedLinuxBBB).

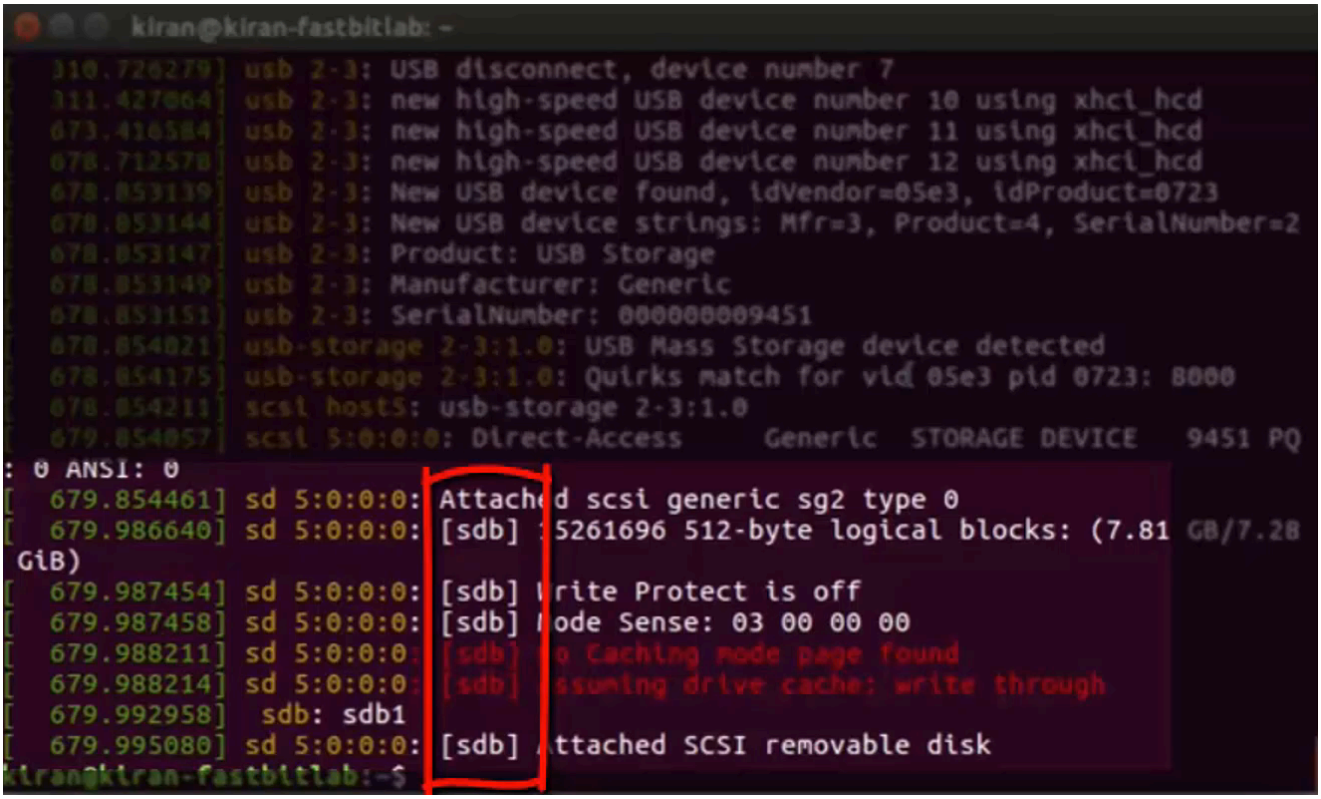
İndirilen dosya	Ne?	Nereye kopyalanacak?
`MLO-beaglebone-2013.04`	MLO	BOOT → adı `MLO` olacak
`u-boot-beaglebone-2013.04-r0.img`	U-Boot	BOOT → adı `u-boot.img` olacak
`uEnv.txt`	U-Boot ayar dosyası	BOOT → adı aynı kalacak
`Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz`	Root filesystem (içinde kernel `/boot/uImage` ve device tree `/boot/am335x-boneblack.dtb` da var)	ROOTFS → arşivden çıkarılacak

---

4.3 SD Kartı Bilgisayarda Bulma

Kartı takıp terminalde çalıştır:

```
sudo dmesg
```



Görsel: kursun videosundan ekran görüntüsü.

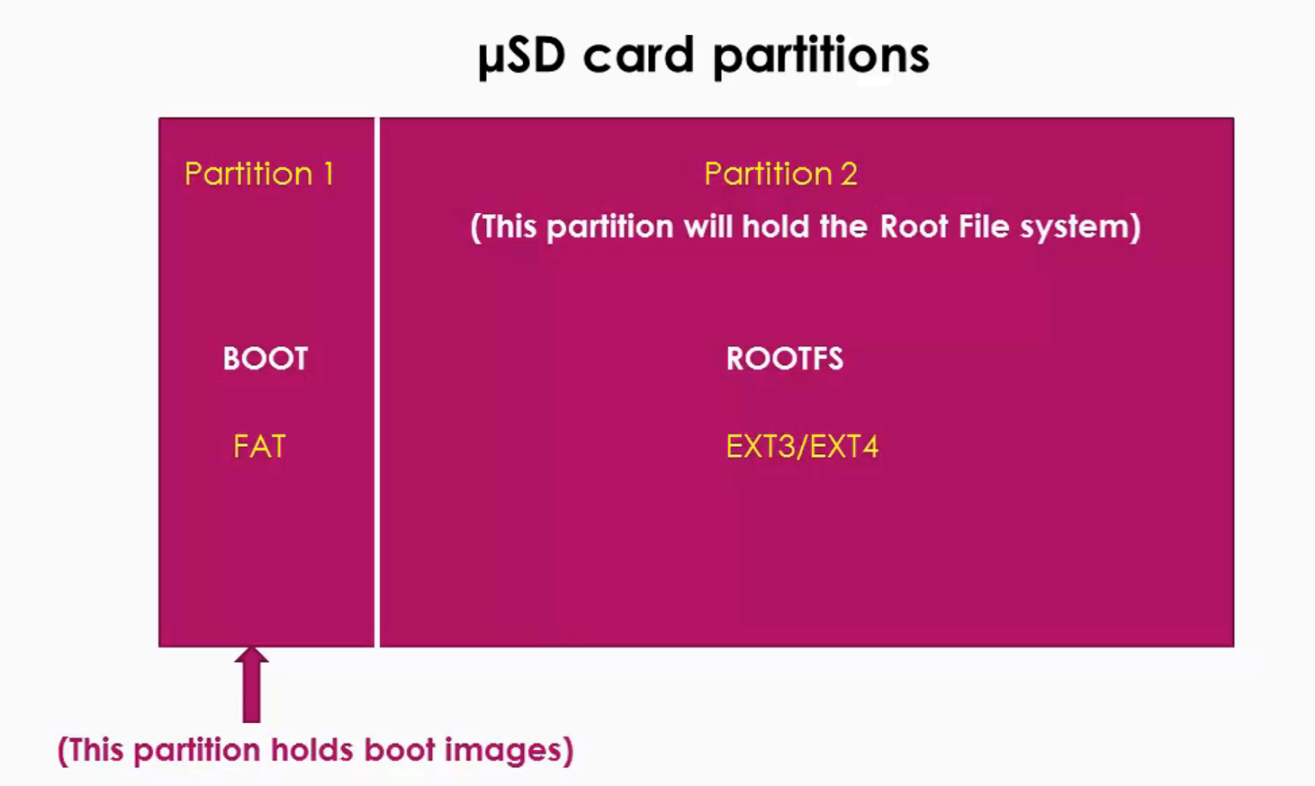
- `sdb` → SD kartın adı (`/dev/sdb`). Yanında boyutu yazar (7.81 GB). Doğru cihaz olduğunu boyuttan anlarsın.
- `sdb1` → kartta zaten var olan bölüm (partition).

> **İpucu**
>
> `lsblk` komutu tüm diskleri boyutlarıyla birlikte daha okunaklı gösterir.

> **Not**
>
> GParted'da aynı kart **7.28 GiB** görünür. Kart eksik değil, birimler farklı: 1 GB = 1.000.000.000 byte, 1 GiB = 1.073.741.824 byte.

4.4 SD Kartı Bölümlere Ayırma (GParted)

Kart 2 bölüme ayrılacak. **Karttaki tüm veriler silinir.**



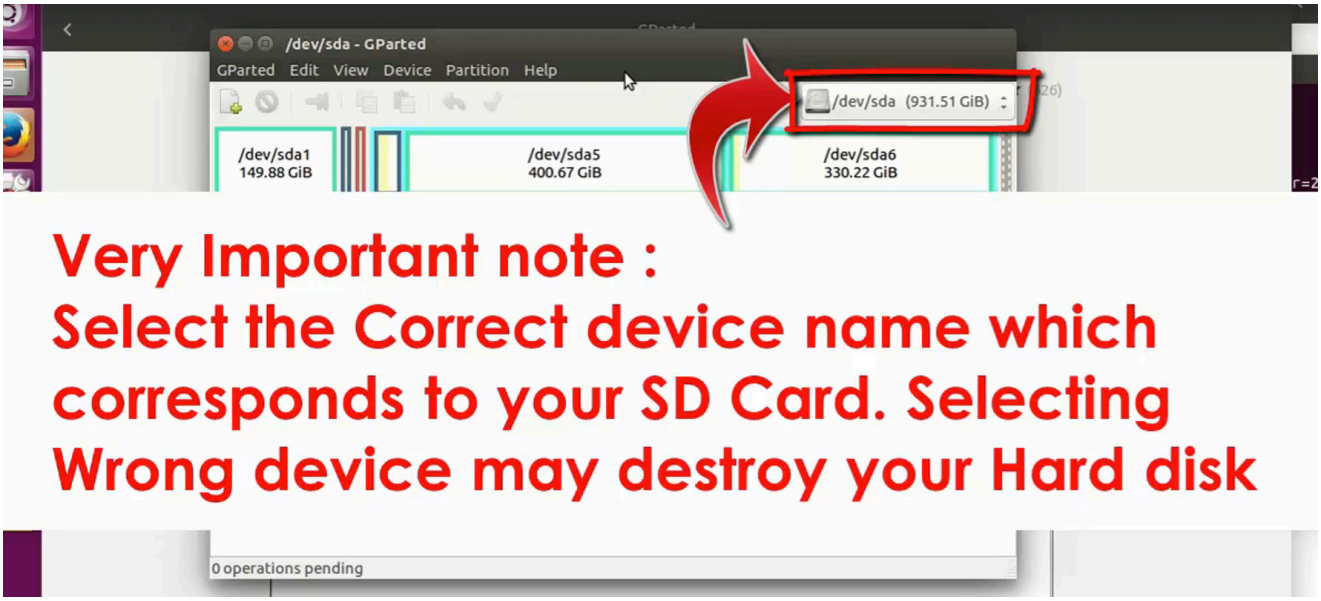
Görsel: kursun slaytı.

Bölüm	Label	Dosya sistemi	Boyut	Neden?
1	`BOOT`	`fat16`	~1 GB	RBL, MLO'yu sadece FAT bölümde arar
2	`ROOTFS`	`ext3`	Kalan alan	Linux dosyaları Linux dosya sistemi ister

GParted'ı kur ve aç:

```
sudo apt install gparted
sudo gparted
```

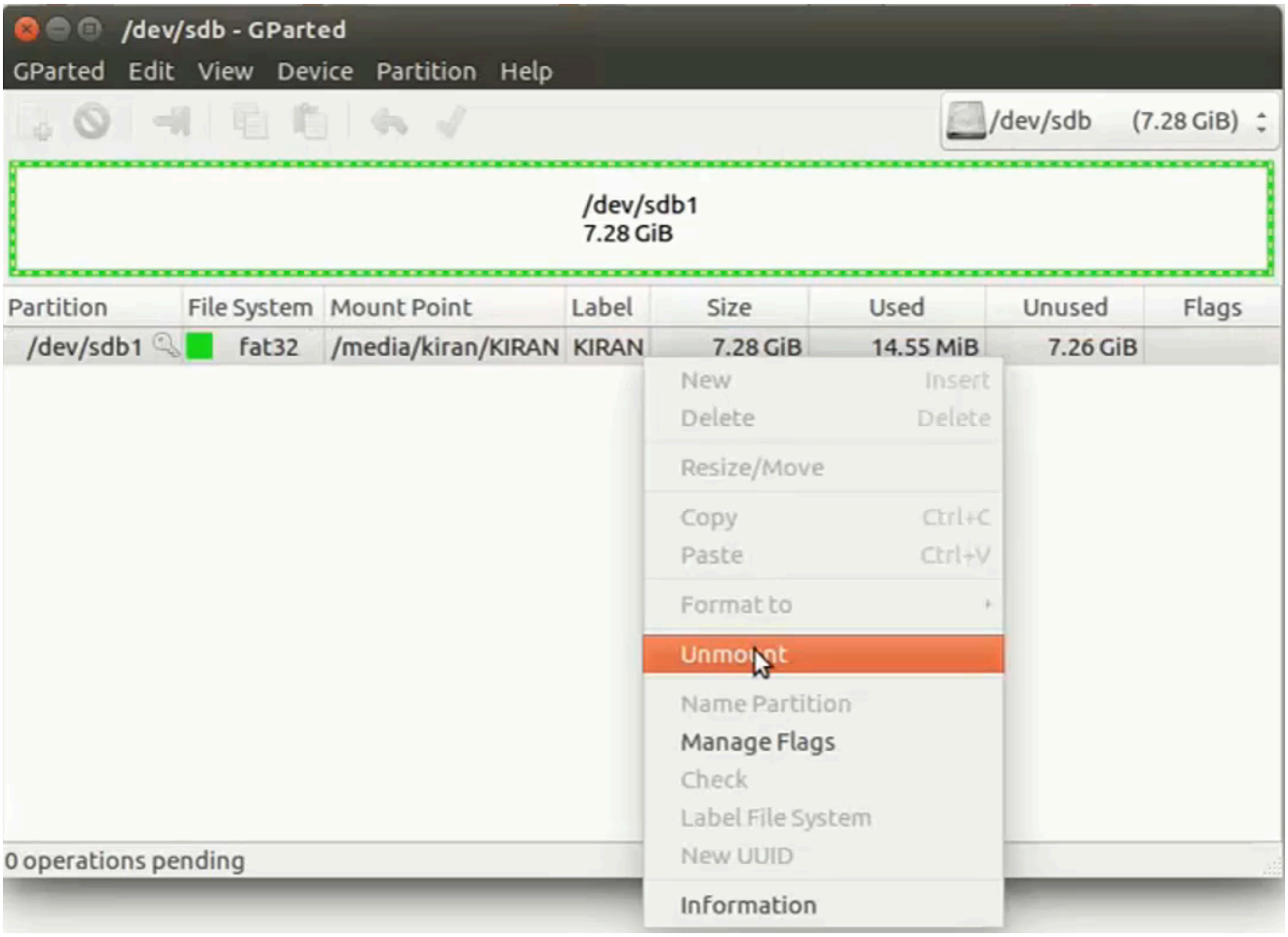
> **Dikkat**
>
> Sağ üstteki listeden **SD kartı seç** (`/dev/sdb` , 7.28 GiB). Yanlışlıkla bilgisayarının diskini (`/dev/sda`) seçersen **bilgisayarındaki veriler silinir**.



Görsel: kursun videosundan ekran görüntüsü.

4.4.1 Unmount et

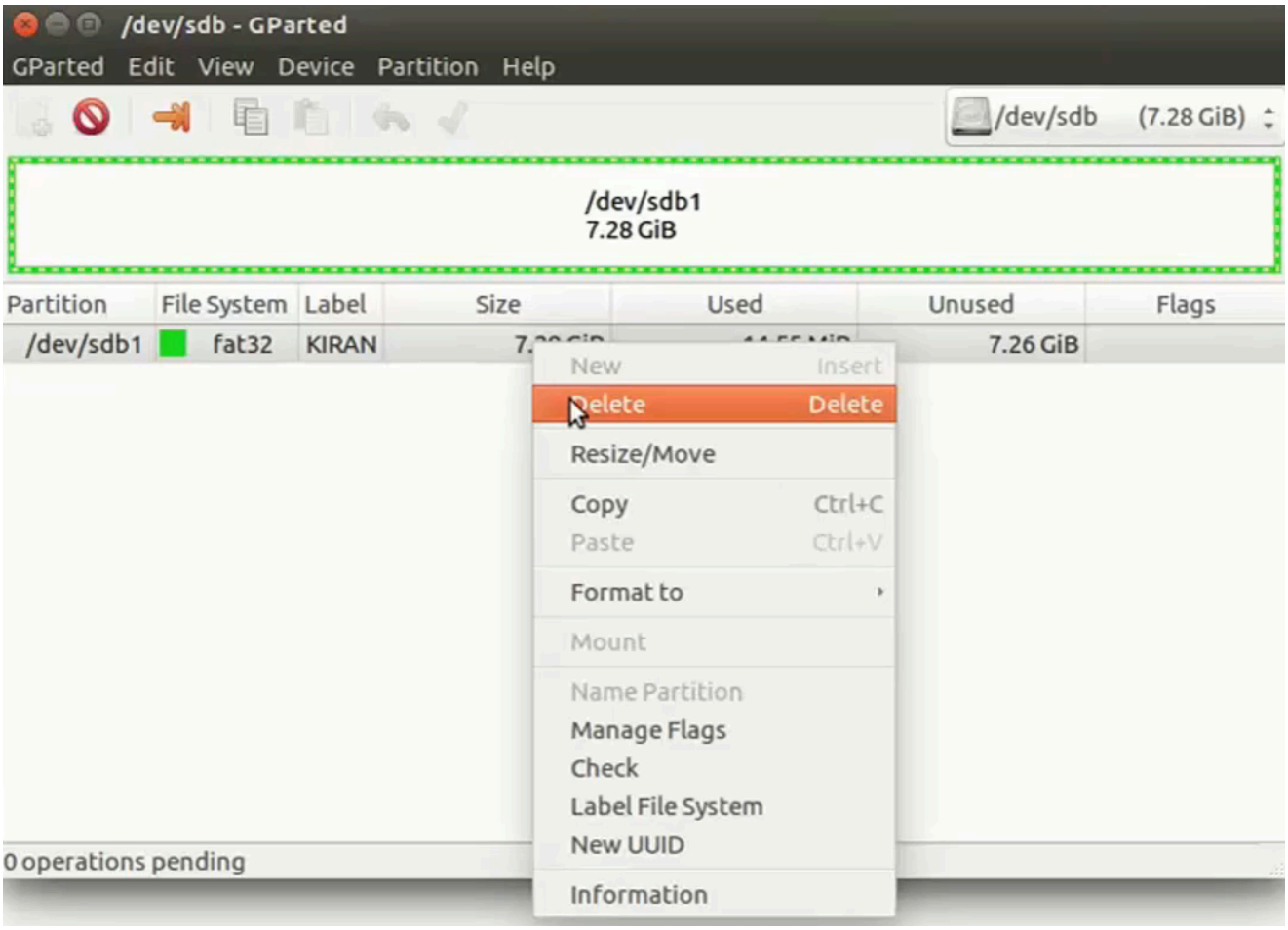
Bölüm şu an sistem tarafından kullanılıyor (yanındaki 🔑 simgesi). Kullanımdayken silinemez. **Sağ tık → Unmount.**

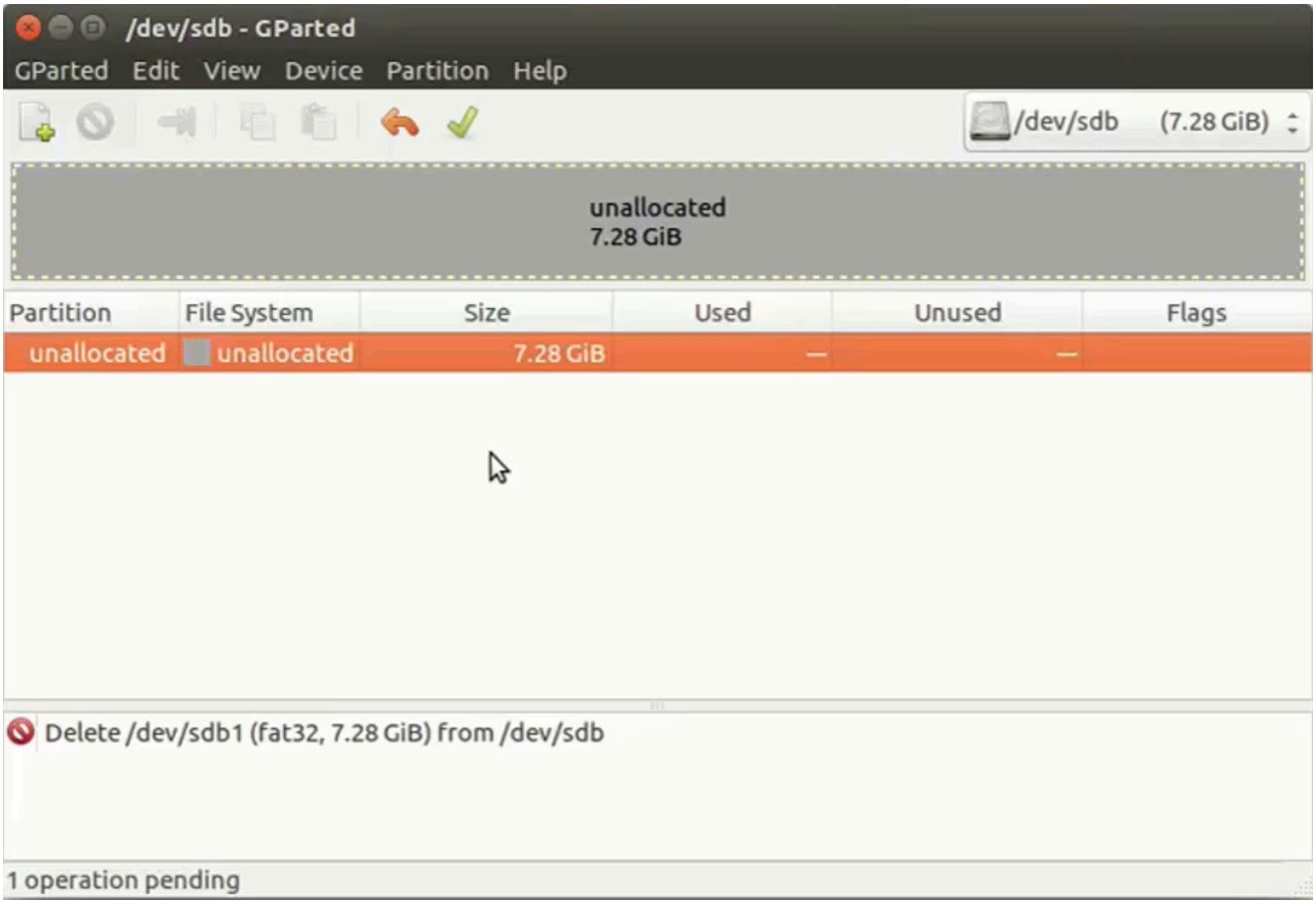


Görsel: kursun videosundan ekran görüntüsü.

4.4.2 Eski bölümü sil

Sağ tık → Delete. Kartın tamamı `unallocated` (boş alan) olur.



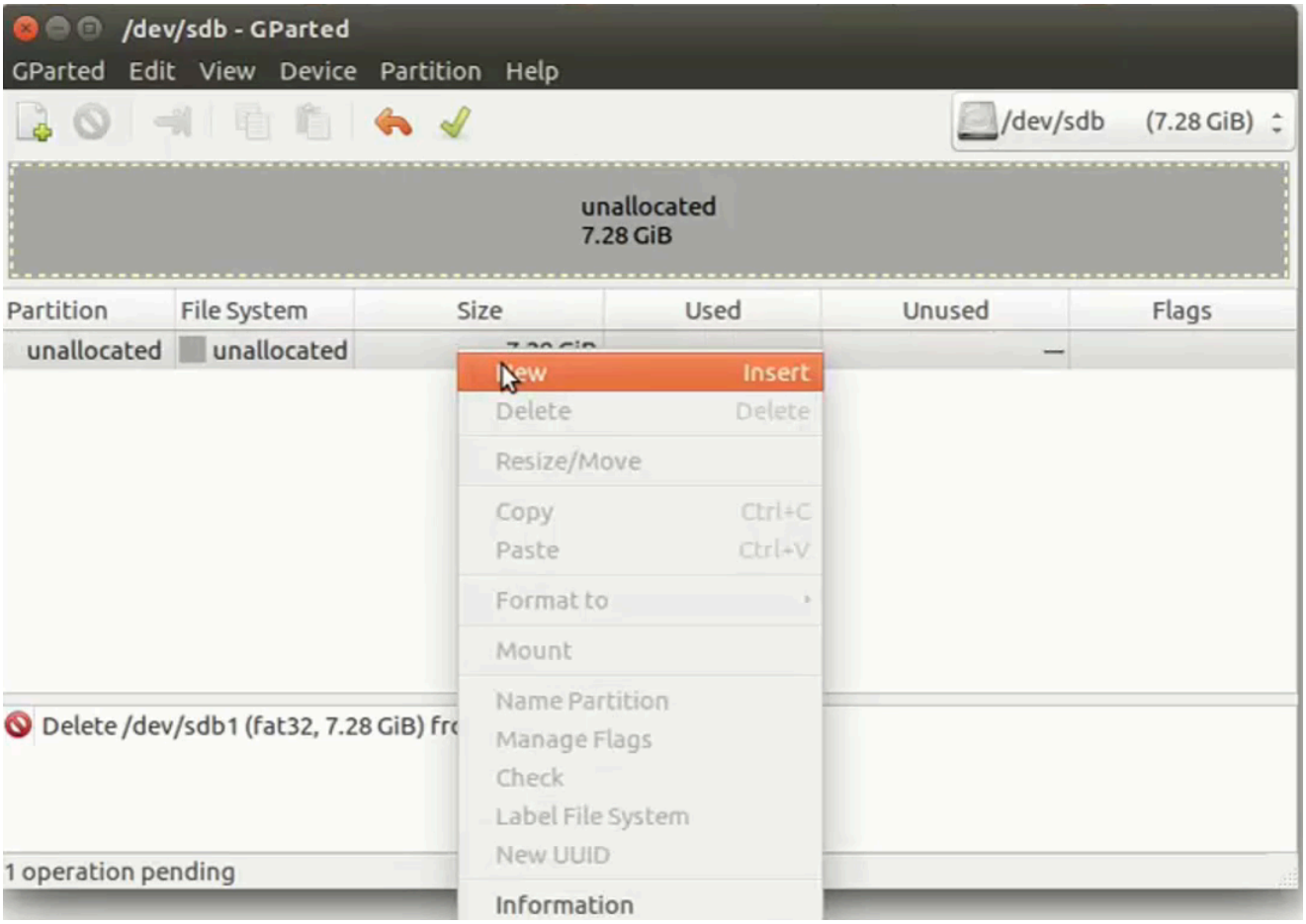


Görseller: kursun videosundan ekran görüntüleri.

> **Not**
>
> GParted işlemleri hemen uygulamaz, sıraya koyar ("1 operation pending"). Hepsi en sonda 4.4.5'te uygulanır. O ana kadar her şey geri alınabilir.

4.4.3 BOOT bölümünü oluştur

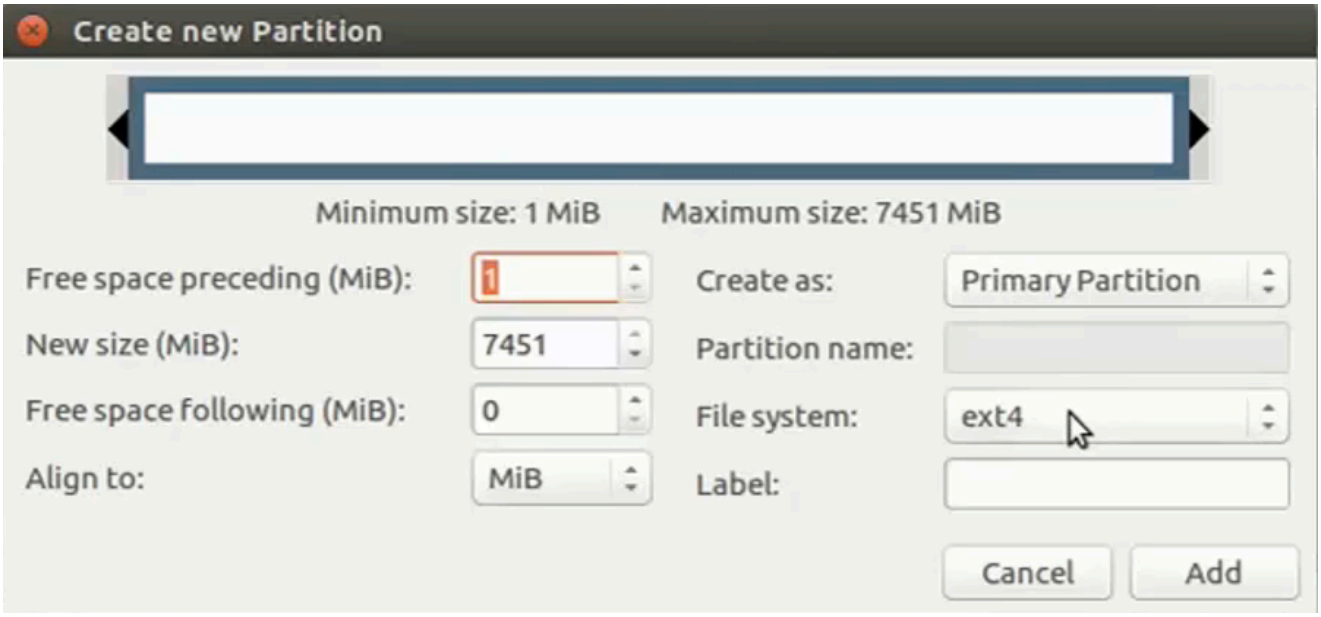
`unallocated` alana **sağ tık → New.**

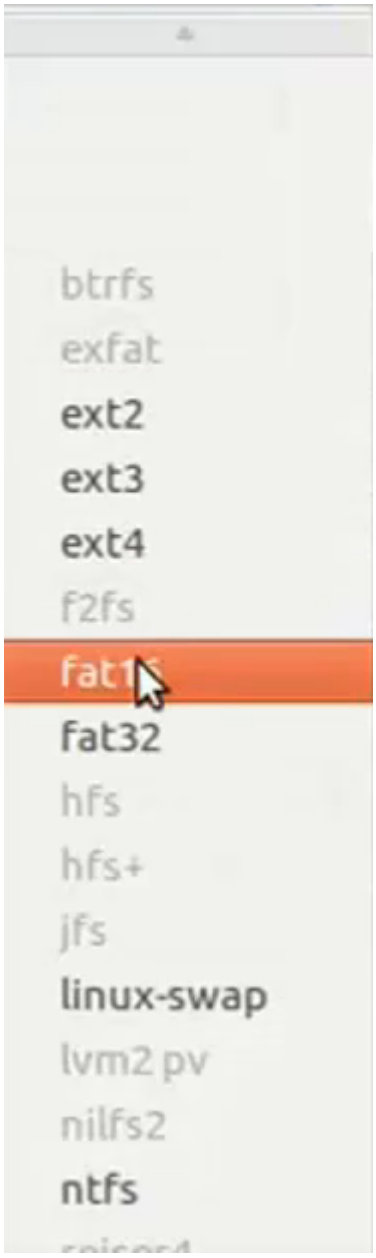


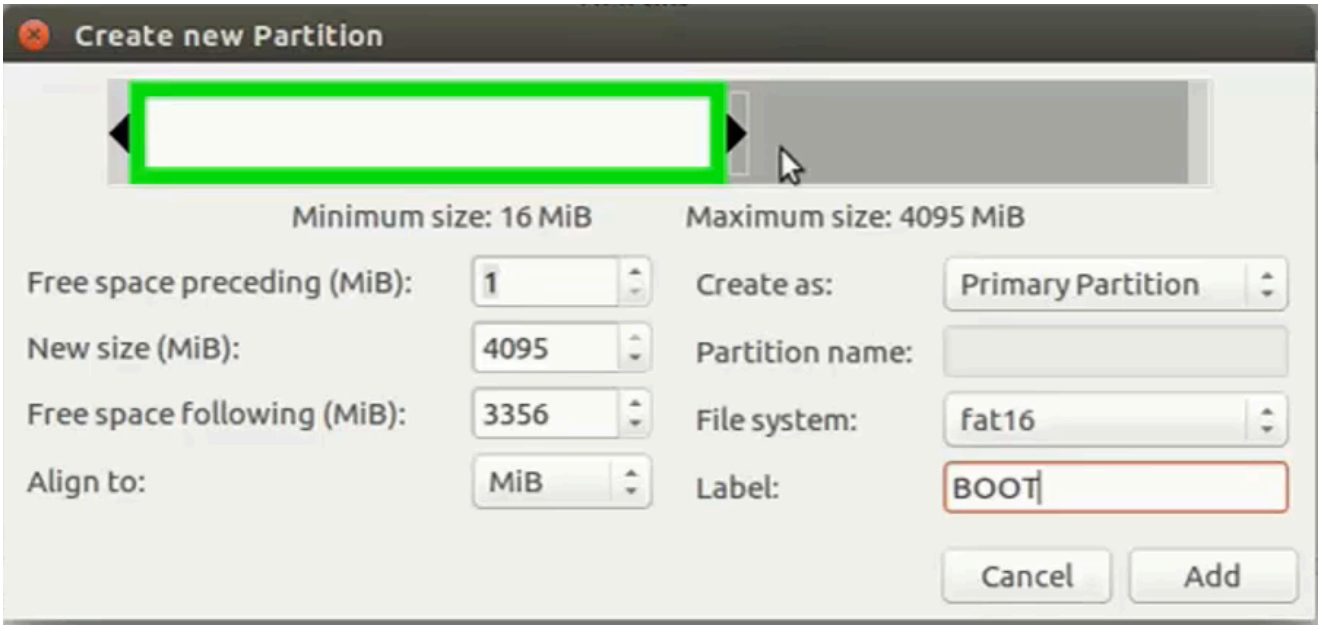
Görsel: kursun videosundan ekran görüntüsü.

Açılan pencerede 3 ayar yap, sonra **Add**'e bas:

Alan	**Değer**
File system	`fat16`
Label	`BOOT`
New size (MiB)	`1013` (yaklaşık 1 GB)

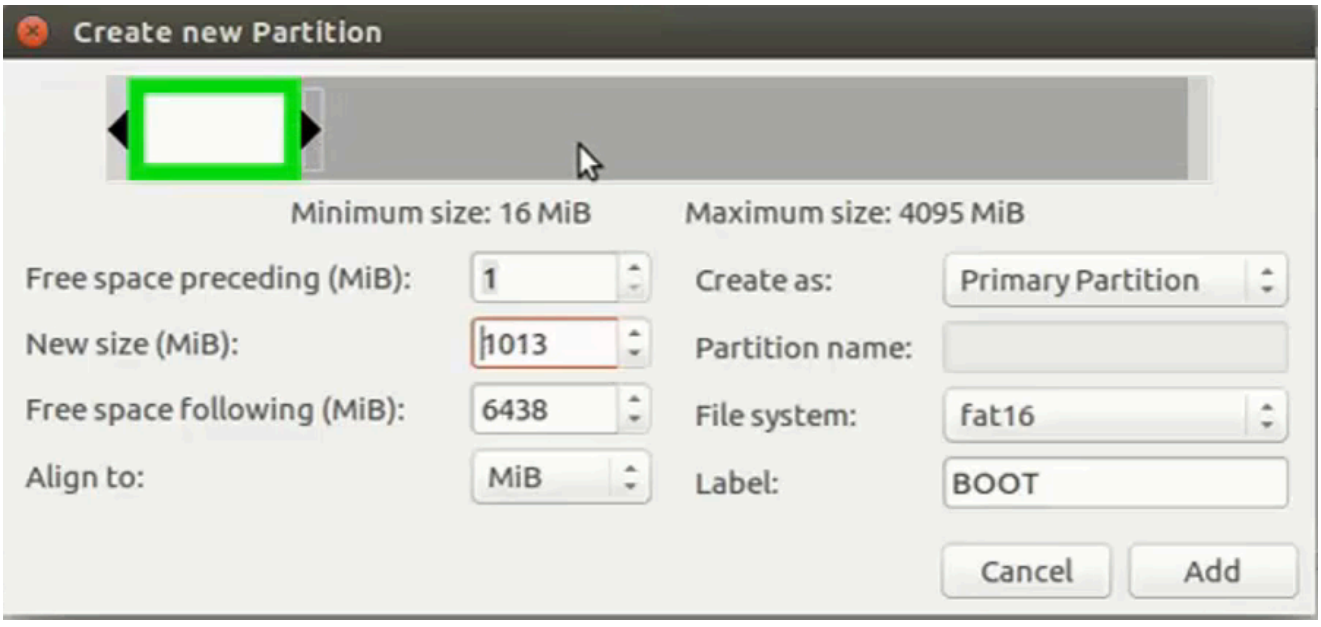

Create new Partition
Minimum size: 1 MiB
Maximum size: 7451 MiB
Free space preceding (MiB):
1
New size (MiB):
7451
Free space following (MiB):
0
Align to:
MiB
Create as:
Primary Partition
Partition name:
File system:
ext4
Label:
Cancel
Add

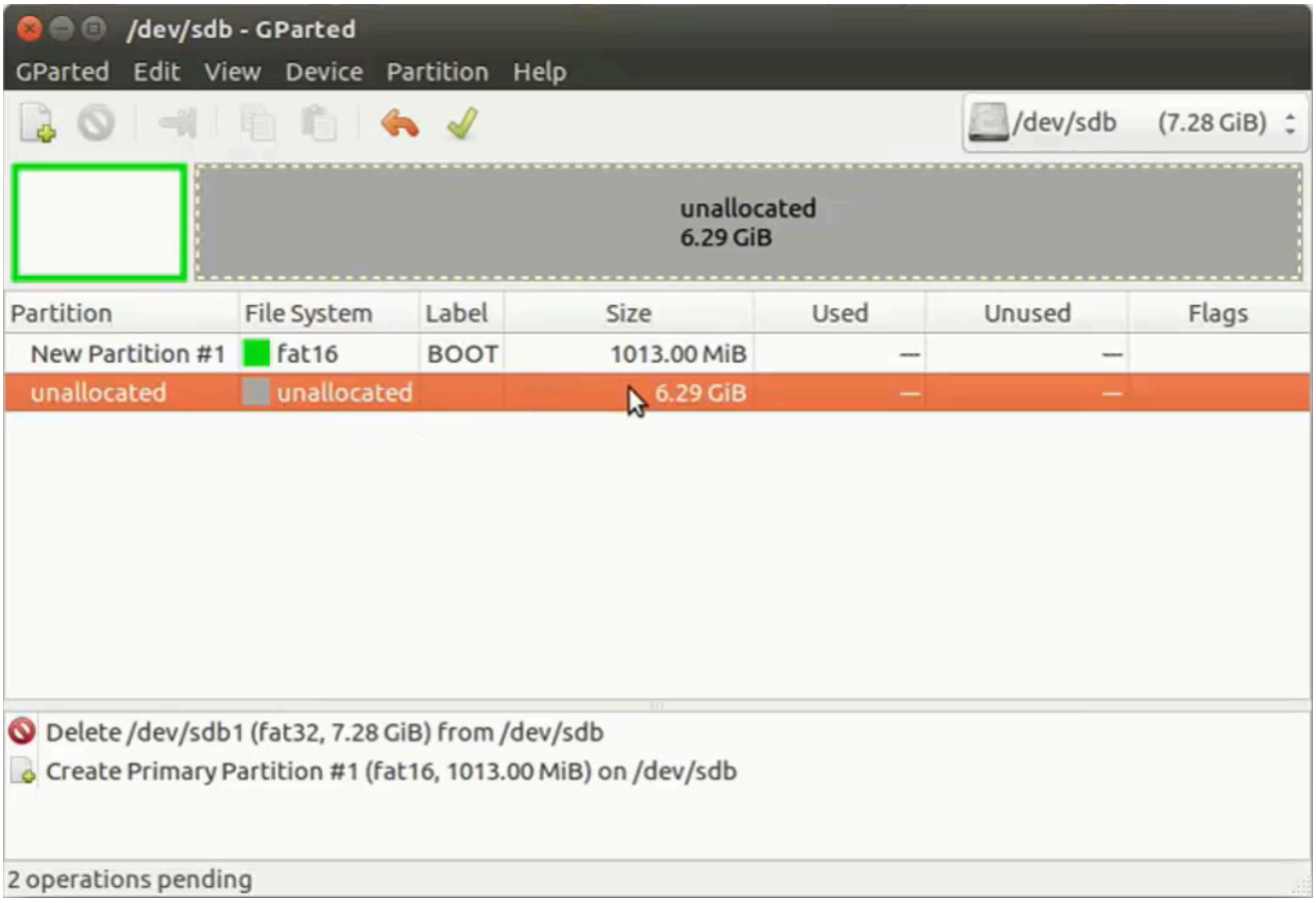

btrfs
exfat
ext2
ext3
ext4
f2fs
fat16
fat32
hfs
hfs+
jfs
linux-swap
lvm2 pv
nilfs2
ntfs



Görseller: kursun videosundan ekran görüntüleri.

> **Not**
>
> fat16 seçilince boyut kendiliğinden **4095 MiB** olur, çünkü FAT16 en fazla ~4 GiB olabilir. Bunu 1013'e düşürüyoruz. Boot dosyaları zaten birkaç MB.



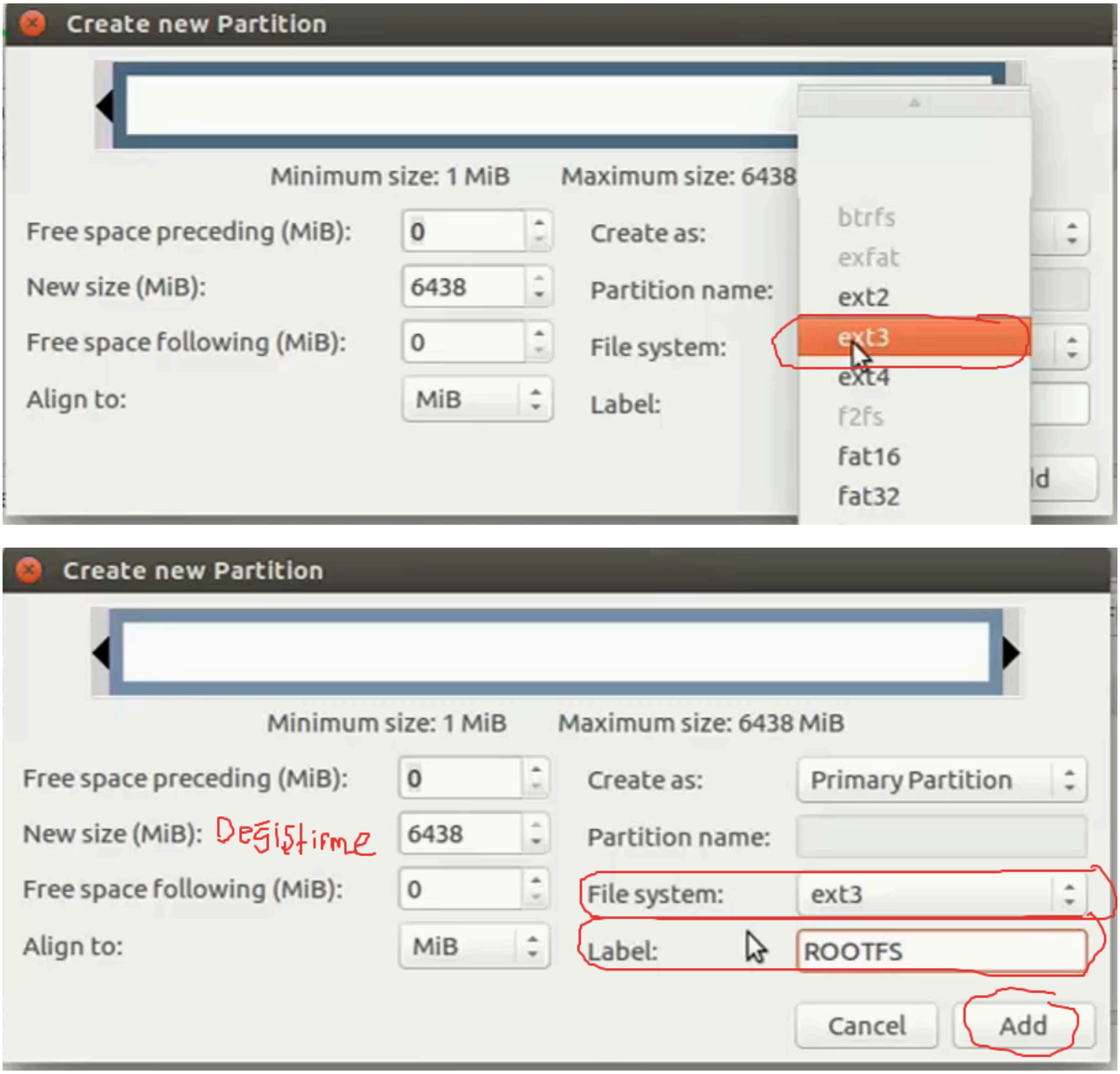


Görseller: kursun videosundan ekran görüntüleri.

4.4.4 ROOTFS bölümünü oluştur

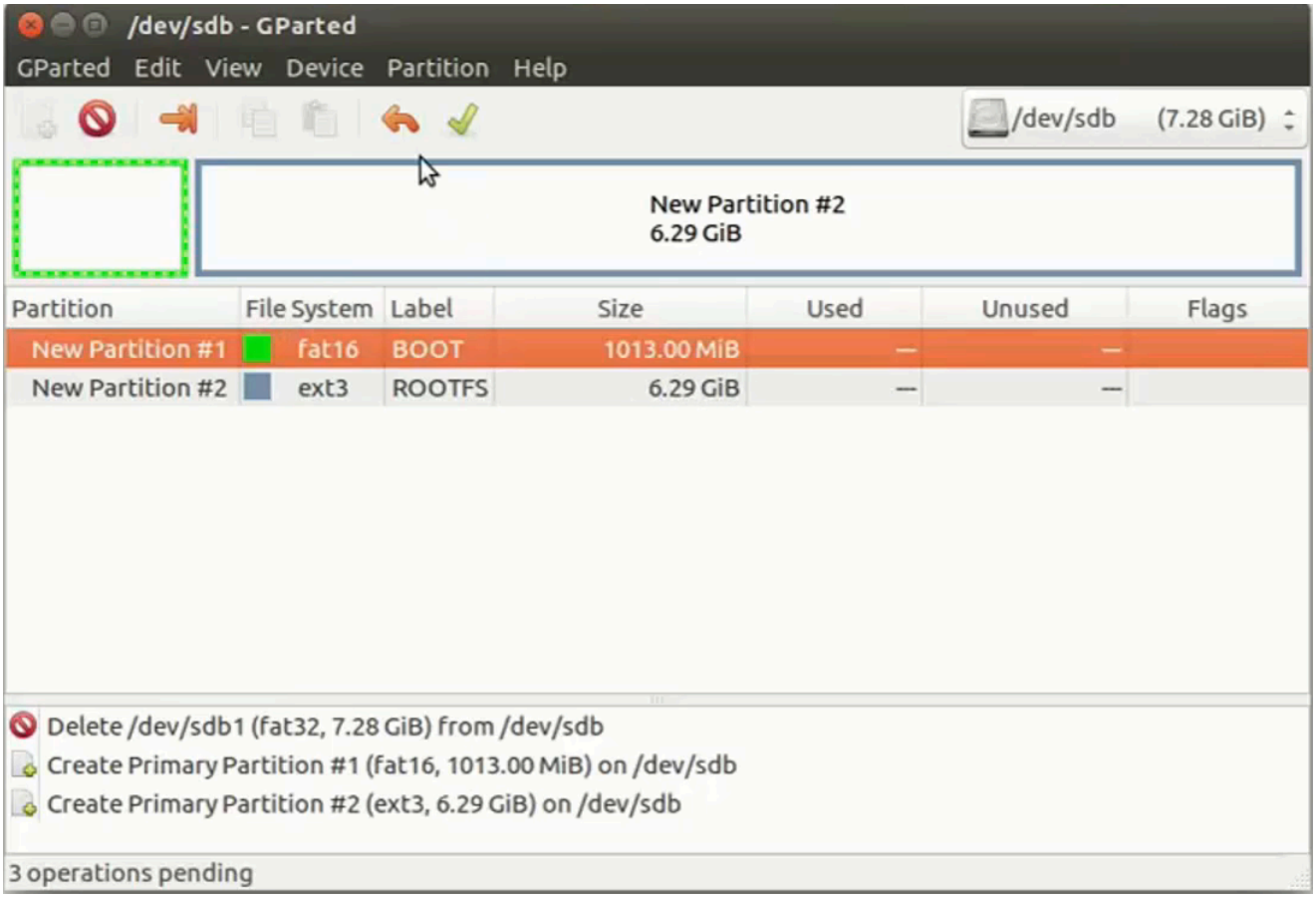
Kalan `unallocated` alana yine **sağ tık** → **New**, sonra **Add**:

Alan	Değer
File system	`ext3`
Label	`ROOTFS`
New size (MiB)	Değiştirme (kalan alanın tamamı)



Görseller: kursun videosundan ekran görüntüleri.

Planlanan durum: 1 GB BOOT + 6.29 GiB ROOTFS.

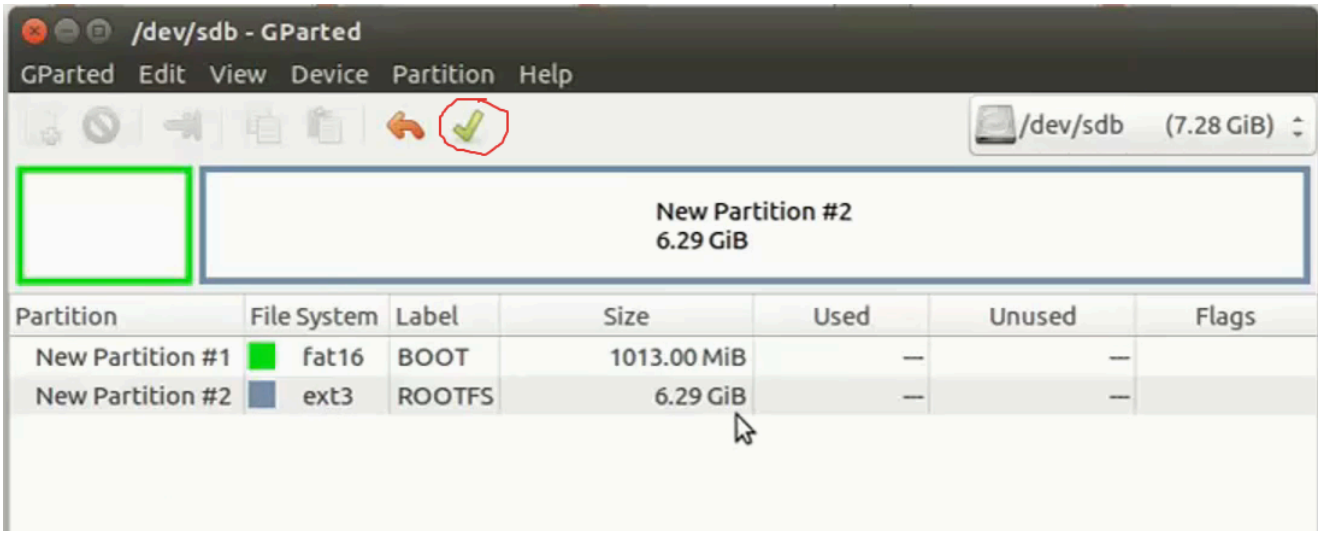


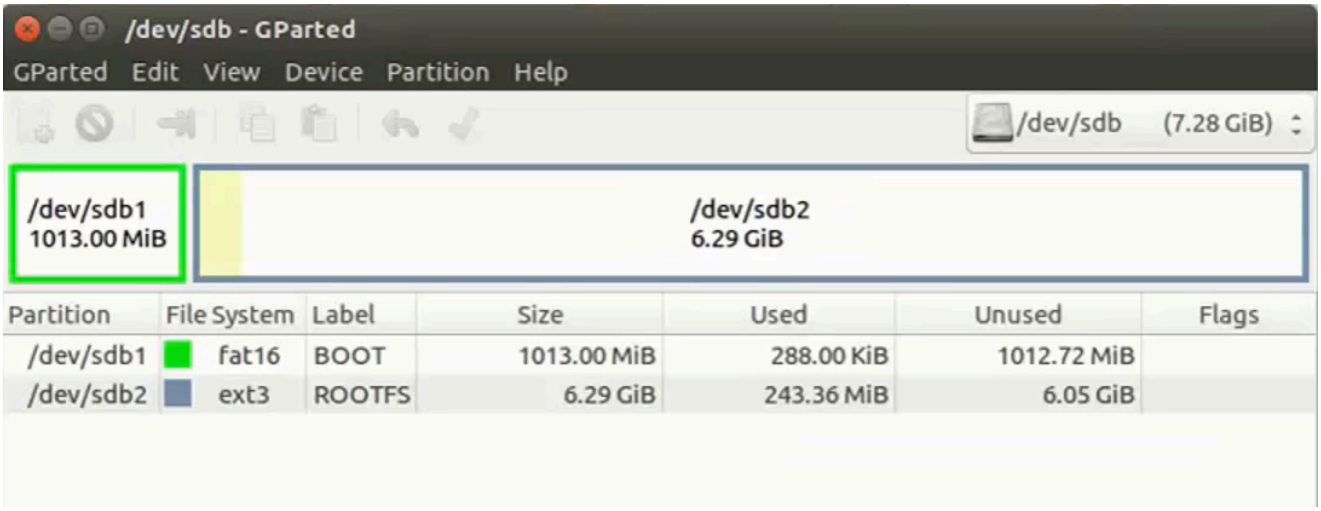
Görsel: kursun videosundan ekran görüntüsü.

4.4.5 Değişiklikleri uygula

Yeşil tik (Apply All Operations) → onay penceresinde **Apply**.

> **Uyarı**
>
> Bu adımdan sonra geri dönüş yok, karttaki eski veriler silinir.



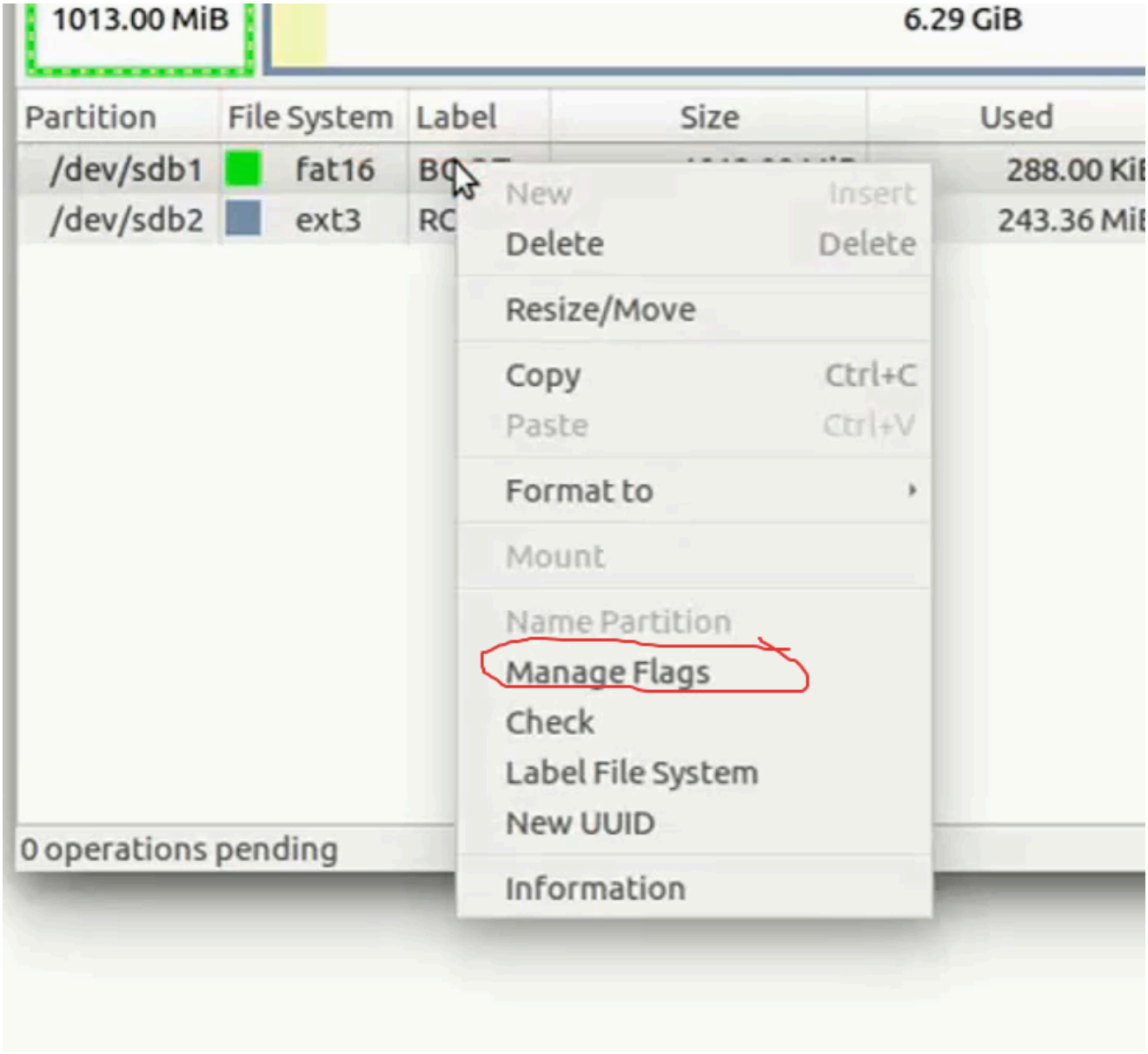


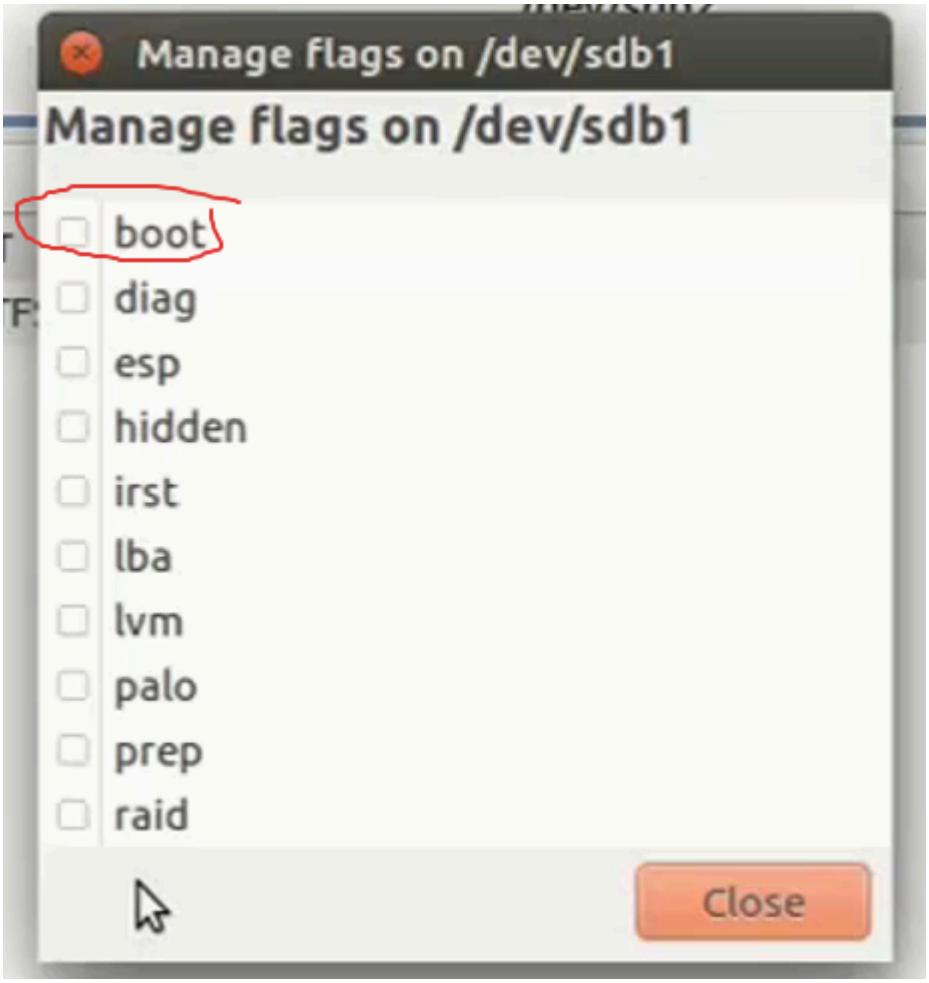
Görseller: kursun videosundan ekran görüntüleri.

4.4.6 BOOT bölümüne `boot` bayrağı ekle

Bu bayrak, bölümü "açılış bölümü" olarak işaretler. RBL, MLO'yu bu bölümde arar.

BOOT'a sağ tık → Manage Flags → `boot` kutucuğunu işaretle → Close.


1013.00 MiB
6.29 GiB
Partition
File System
Label
Size
Used
/dev/sdb1
fat16
288.00 KiB
/dev/sdb2
ext3
243.36 MiB
New
Insert
Delete
Delete
Resize/Move
Copy
Ctrl+C
Paste
Ctrl+V
Format to
Mount
Name Partition
Manage Flags
Check
Label File System
New UUID
Information
0 operations pending



Görseller: kursun videosundan ekran görüntüleri.

Flags sütununda `boot` yazıyorsa tamam. GParted'ı kapatabilirsin.

Partition	File System	Label	Size	Used	Unused	Flags
/dev/sdb1	fat16	BOOT	1013.00 MiB	288.00 KiB	1012.72 MiB	boot
/dev/sdb2	ext3	ROOTFS	6.29 GiB	243.36 MiB	6.05 GiB	

Görsel: kursun videosundan ekran görüntüsü.

4.5 Boot Prosedürü (her testte tekrarlanır)

Aşağıdaki aşamaların her birinde dosya kopyaladıktan sonra bu adımları uygula.

4.5.1 Kartı bilgisayardan güvenli çıkar

```
sync
sudo umount /media/$USER/BOOT
sudo umount /media/$USER/ROOTFS
```

- `sync` → Bekleyen yazma işlemlerini karta yazar. Atlanırsa kopyalanan dosya eksik kalabilir.

- `umount` → Bölümlerin bağlantısını keser. Artık kart çıkarılabilir.

4.5.2 Seri konsolu hazırla (sadece ilk seferde)

1. USB-TTL'yi kartın **J1** header'ına bağla (1.8).
2. Minicom'u aç: **115200 baud, 8N1, Hardware Flow Control kapalı** (Bölüm 2).

4.5.3 Kartı SD karttan aç (S2 butonu)

BeagleBone Black normalde **kendi eMMC belleğinden** açılır. SD karttan açılması için **kart açılırken S2 (BOOT) butonu basılı olmalı**. RBL bu butona sadece açılış anında bakar (3.5).

SD kartı BeagleBone'a tak, sonra iki yöntemden birini uygula:

Yöntem	Ne zaman?	Adımlar
A: Güç vererek	Kartta güç yokken	S2'yi basılı tut → USB kablosunu tak → 2–3 sn sonra S2'yi bırak
B: Power butonuyla	Kart zaten güç alıyorsa	Power'a LED'ler sönene kadar uzun bas (kapanır) → S2'yi basılı tut → Power'a kısa bas (açılır) → S2'yi bırak

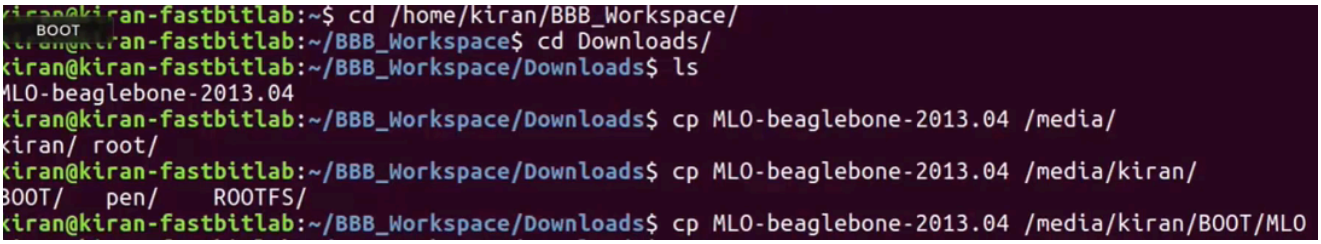
---

4.6 Aşama 1: MLO

Kopyala

İndirme klasörüne git ve MLO'yu BOOT bölümüne `MLO` **adıyla** kopyala:

```
cd ~/BBB_Workspace/Downloads/
cp MLO-beaglebone-2013.04 /media/$USER/BOOT/MLO
```



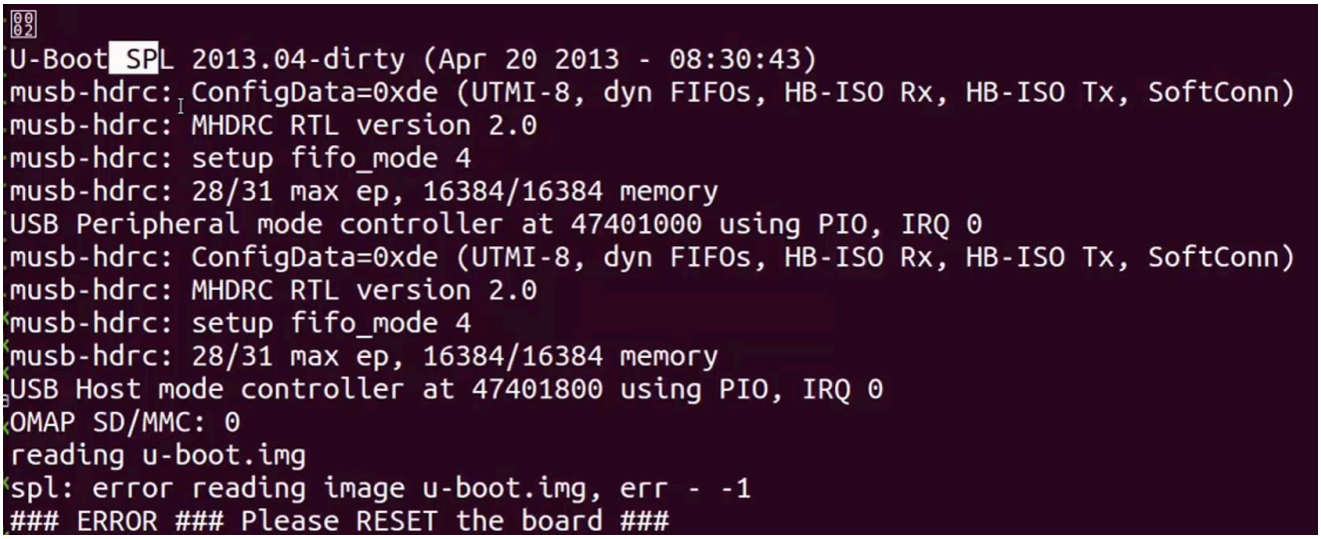
Görsel: kursun videosundan ekran görüntüsü.

> **Önemli**
>
> **Neden sonda `MLO` yazıyoruz?** `cp` komutunda hedefin sonuna bir ad yazılırsa dosya o adla kopyalanır:
>
> ```
> cp  MLO-beaglebone-2013.04   /media/$USER/BOOT/MLO
>     └──────── kaynak ────────┘     └─ hedef klasör + YENİ AD ─┘
> ```
>
> RBL, BOOT bölümünde **tam olarak `MLO` adlı** dosyayı arar. Dosya uzun adıyla kalırsa bulunamaz.

> **İpucu**
>
> `$USER` senin kullanıcı adındır (kursta `kiran`). Dosya adlarını elle yazmak yerine ilk birkaç harfi yazıp **Tab**'a bas, terminal tamamlar.

Test et

4.5'teki adımlarla kartı aç. Minicom'da şunu görmelisin:



Görsel: kursun videosundan ekran görüntüsü.

Satır	Anlamı
`U-Boot SPL 2013.04-dirty ...`	**MLO çalıştı.** RBL dosyayı buldu.
`musb-hdrc ...`, `USB ... controller`	MLO USB donanımını başlatıyor, önemli değil.
`reading u-boot.img`	MLO bir sonraki aşamayı (U-Boot) arıyor.
`spl: error reading image u-boot.img`	U-Boot bulunamadı, çünkü henüz kopyalamadık. **Beklenen hata.**

Sonuç: **RBL ✓ → MLO ✓ → U-Boot ✗**

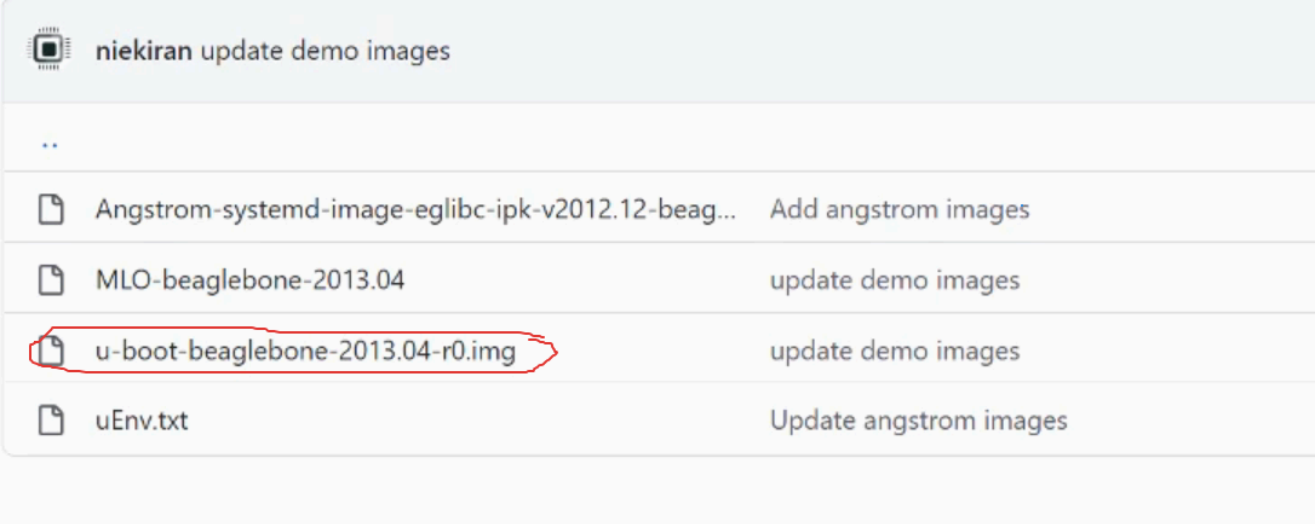
> **Not**
>
> "dirty" bir hata değildir. MLO derlenirken kaynak kodda kaydedilmemiş (commit edilmemiş) değişiklik olduğunu belirtir.

4.7 Aşama 2: U-Boot

Kopyala

Kartı tekrar bilgisayara tak ve U-Boot'u `u-boot.img` **adıyla** kopyala. MLO, U-Boot'u bu adla arar:

```
cp u-boot-beaglebone-2013.04-r0.img /media/$USER/BOOT/u-boot.img
```



Görsel: kursun GitHub reposu (niekiran/EmbeddedLinuxBBB).

Test et

4.5'teki adımlarla kartı aç.

- `U-Boot SPL` satırlarından sonra **U-Boot'un kendi mesajları** görünüyorsa bu aşama başarılı.
- Kernel ve `uEnv.txt` henüz olmadığı için Linux SD karttan açılmaz.

Sonuç: **RBL** ✓ → **MLO** ✓ → **U-Boot** ✓ → **Kernel** ✗

4.8 Aşama 3: Root Filesystem ve uEnv.txt

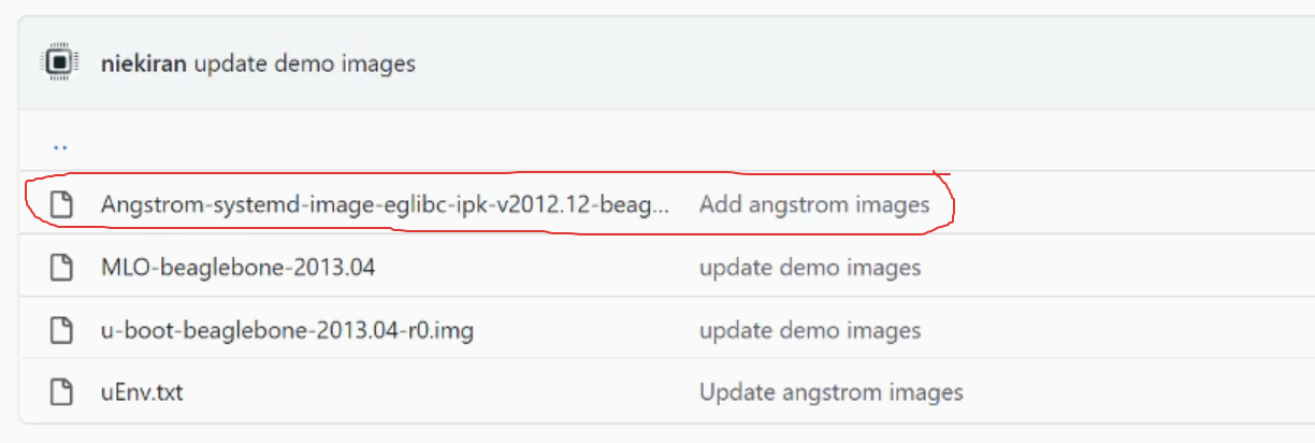
Kopyala

Kartı tekrar bilgisayara tak.

1) Root filesystem'i ROOTFS bölümüne çıkar:

```
sudo tar -xvf Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz -C
/media/$USER/ROOTFS
```

Parça	Anlamı
`sudo`	Linux dosyalarının sahibi `root`. Dosya izinlerinin korunması için gerekli.
`-x` `-v` `-f`	Arşivi aç (e**x**tract), açılan dosyaları göster (**v**erbose), arşiv dosyası (**f**ile)
`-C /media/$USER/ROOTFS`	Dosyaları bu klasöre çıkar



Görsel: kursun GitHub reposu (niekiran/EmbeddedLinuxBBB).

2) uEnv.txt'yi BOOT bölümüne kopyala (adı değişmiyor, U-Boot bu adla arar):

```
cp uEnv.txt /media/$USER/BOOT/
```

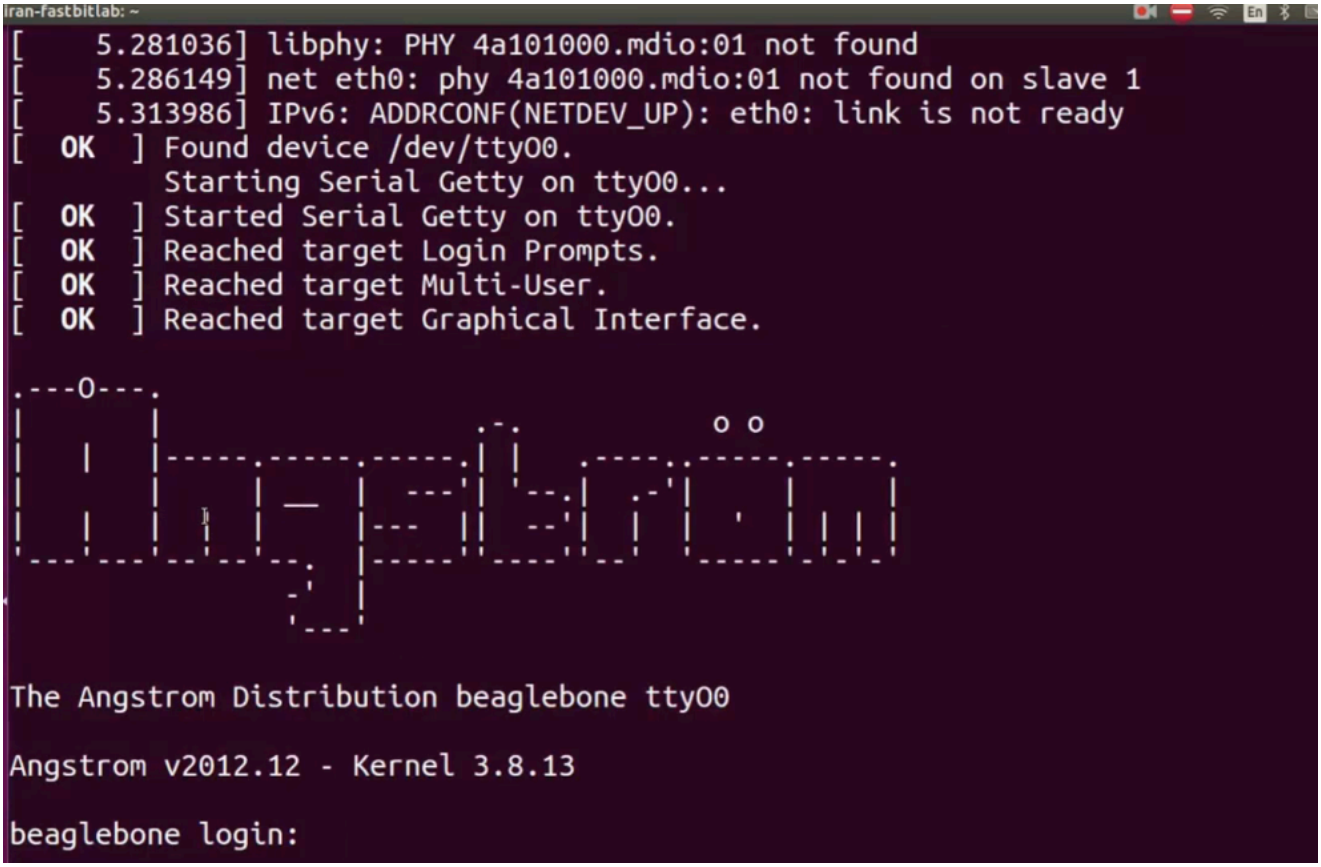
Kontrol et:

```
ls /media/$USER/BOOT/           # MLO  u-boot.img  uEnv.txt
ls /media/$USER/ROOTFS/boot/    # uImage ve am335x-boneblack.dtb burada olmalı
```

Test et

4.5'teki adımlarla kartı aç. Rootfs büyük olduğu için `sync` biraz uzun sürebilir, bitmesini bekle.

Kernel mesajları akar ve sonunda **Ångström** logosu ile `beaglebone login:` satırı çıkar:



Görsel: kursun videosundan ekran görüntüsü.

Sonuç: **RBL ✓ → MLO ✓ → U-Boot ✓ → Kernel ✓ → Rootfs ✓** 🎉

> **İpucu**
>
> Giriş için kullanıcı adı `root` yaz. Bu demo imajında genelde şifre yoktur.

Bu açılışta uEnv.txt ne yaptı?

`uEnv.txt`, U-Boot'un davranışını değiştiren ayar dosyasıdır. BBB'nin U-Boot'u açılırken BOOT bölümündeki bu dosyayı okur ve içindeki `uenvcmd` komutunu çalıştırır.

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
loadaddr=0x82000000 ──→ DDR Temel Adresi
fdtaddr=0x88000000 ──→ DTB File Adresi
loadfromsd=load mmc 0:2 ${loadaddr} /boot/uImage;load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
linuxbootargs=setenv bootargs console=${console} root=/dev/mmcblk0p2 rw
uenvcmd=setenv autoload no; run loadfromsd; run linuxbootargs; bootm ${loadaddr} - ${fdtaddr}
```

Görsel: kursun videosundan ekran görüntüsü.

Satır	Anlamı
`console=ttyO0,115200n8`	Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın.
`ipaddr`, `serverip`	Ağdan boot için IP adresleri. SD karttan boot'ta kullanılmıyor.

Satır	Anlamı
`loadaddr=0x82000000`	Kernel'in (uImage) **RAM'de (DDR) yükleneceği adres**.
`fdtaddr=0x88000000`	Device tree (.dtb) dosyasının RAM'de yükleneceği adres.
`loadfromsd=load mmc 0:2 ...`	`mmc 0:2` = SD kart, 2. bölüm (ROOTFS). Oradaki `/boot/uImage` → `loadaddr`'e, `/boot/am335x-boneblack.dtb` → `fdtaddr`'e yüklenir.
`linuxbootargs=... root=/dev/mmcblk0p2 rw`	Kernel'e "root filesystem SD kartın 2. bölümünde, okuma-yazma modunda" bilgisini verir.
`uenvcmd=...`	U-Boot'un çalıştırdığı komut: önce `loadfromsd`, sonra `linuxbootargs`, en son `bootm` ile kernel'i başlat. `bootm`'deki `-` "initramfs yok" demek.

> **Not**
>
> AM335x'te DDR'in başlangıç adresi `0x80000000`'dır. `0x82000000` ise DDR'in içinde, kernel için seçilmiş bir adres.

Aynı dosya farklı yazılarak U-Boot'a kernel'i **ağdan (TFTP)**, rootfs'u da **bilgisayardaki bir klasörden (NFS)** aldırmak mümkün (Bölüm 8):

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
absolutepath=/var/lib/tftp/
rootpath=/srv/nfs/bbb,nolock,wsize=1024,rsize=1024 rootwait rootdelay=5
loadtftp=echo Booting from network ...;tftpboot ${loadaddr} ${absolutepath}uImage; tftpb
netargs=setenv bootargs console=${console} root=/dev/nfs rw nfsroot=${serverip}:${rootpa
uenvcmd=setenv autoload no; run loadtftp; run netargs; bootm ${loadaddr} - ${fdtaddr}
```

Görsel: kursun videosundan ekran görüntüsü.

`uEnv.txt`'nin her satırını ve U-Boot komut satırını Bölüm 6'da ayrıntılı göreceğiz.

4.9 uImage Nedir?

`uImage`, U-Boot'un `bootm` komutuyla başlatabildiği kernel formatıdır. Yeni bir derleme yapılmaz. `zImage` dosyasının başına **64 byte'lık bir U-Boot header'ı** eklenir. Bu işi `mkimage` aracı yapar.

```
+------------------+--------------------------------------+
| U-Boot Header    |              zImage                  |
|   (64 byte)      | (sıkıştırılmış, kendini açan kernel) |
+------------------+--------------------------------------+
|<---------------------- uImage ---------------------->|
```

Kernel imajları zinciri:

Dosya	Açıklama
`vmlinux`	Derlenmiş kernel. ELF formatında, sıkıştırılmamış.
`Image`	`vmlinux` 'tan `objcopy` ile çıkarılmış ham binary
`zImage`	Sıkıştırılmış `Image` ve başında onu RAM'de açan küçük bir program
`uImage`	64 byte U-Boot header + `zImage`

Header'ın içeriği (`struct image_header` , tüm alanlar big-endian):

Offset	Boyut	Alan	Anlamı
0x00	4	`ih_magic`	Sihirli sayı: `0x27051956`
0x04	4	`ih_hcrc`	Header'ın CRC32 değeri
0x08	4	`ih_time`	Oluşturulma zamanı
0x0C	4	`ih_size`	zImage'in boyutu
0x10	4	`ih_load`	RAM'de kopyalanacağı adres
0x14	4	`ih_ep`	Çalıştırmanın başlayacağı adres (entry point)
0x18	4	`ih_dcrc`	zImage'in CRC32 değeri
0x1C	1	`ih_os`	İşletim sistemi (Linux)
0x1D	1	`ih_arch`	Mimari (ARM)
0x1E	1	`ih_type`	İmaj tipi (kernel, ramdisk, script...)
0x1F	1	`ih_comp`	Sıkıştırma tipi
0x20	32	`ih_name`	İmaj adı

U-Boot bu header sayesinde dosyanın geçerli bir imaj olup olmadığını (magic number), bozulup bozulmadığını (CRC) ve RAM'de nereye yüklenip nereden çalıştırılacağını anlar.

4.10 Uygulama: uImage Header'ını U-Boot'ta Elle Okuma

Amaç: uImage'ın başındaki 64 byte'lık U-Boot header'ını (4.9) kendi gözümüzle görmek.

Neredeyiz? 4.8'de U-Boot, `uEnv.txt`'deki komutları **otomatik** çalıştırıp Linux'u açtı. Bu sefer U-Boot'u durdurup aynı işin ilk yarısını **elle** yapacağız:

```
4.8:   RBL → MLO → U-Boot → Kernel → Rootfs → login     (otomatik)
4.10:  RBL → MLO → U-Boot ■ DUR → komutları elle yaz
```

Adım	Komut	Ne yapar?
1	Açılışta bir tuşa bas	U-Boot'u durdurur, `U-Boot#` komut satırı gelir
2	`load mmc 0:2 0x82000000 /boot/uImage`	uImage'ı SD karttan DDR'e kopyalar
3	`md.b 0x82000000 40`	DDR'deki ilk 64 byte'ı (header) ekrana döker
4	—	Byte'ları header tablosuyla eşleştirip çözeriz

> **Not**
>
> **Neden önce DDR'e yüklüyoruz?** `md` (memory display) komutu sadece **belleği** okuyabilir. SD kart bir bellek adresi değil, depolama cihazıdır. Bu yüzden dosyayı önce RAM'e kopyalıyoruz.

Kursun bu dersinde Windows'ta Tera Term kullanılıyor. Minicom ile de komutlar ve çıktılar aynıdır (2.6).

4.10.1 U-Boot'u durdur

1. Seri terminali aç ve kartı 4.5.3'teki gibi SD karttan başlat.
2. Ekranda `Hit any key to stop autoboot` yazısı görünür görünmez bir tuşa (ör. **Space**) bas.
3. `U-Boot#` komut satırı geldiyse hazırsın.

> **İpucu**
>
> Geri sayım çok kısa (1–3 saniye). Kaçırırsan Linux açılır. Kartı yeniden başlatıp daha erken bas. Kart açılırken Space'e art arda basmak işe yarar.

4.10.2 uImage'ı DDR'e yükle

Önce dosyanın orada olduğunu gör:

```
U-Boot# ls mmc 0:2 /boot
```

Sonra DDR'e yükle:

```
U-Boot# load mmc 0:2 0x82000000 /boot/uImage
```

Parça	Anlamı
`load`	Bir dosya sisteminden dosyayı belleğe kopyala
`mmc 0:2`	MMC cihazı 0 (SD kart), 2. bölüm (ROOTFS)
`0x82000000`	DDR'de dosyanın yazılacağı adres (`uEnv.txt` 'deki `loadaddr` ile aynı)
`/boot/uImage`	Bölüm içindeki dosyanın yolu

`mmc X:Y` ne demek?

```
load  mmc 0:2      0x82000000   /boot/uImage
      └─ NEREDEN ─┘ └─ NEREYE ─┘ └─ HANGİ DOSYA ┘
      SD kart,      DDR'deki      partition içindeki
      2. partition  bu adrese     dosya yolu
```

- `X` = **hangi MMC cihazı.** BeagleBone Black'te:
- `mmc 0` → **microSD kart**
- `mmc 1` → **eMMC** (karta lehimli dahili bellek)
- `Y` = **o cihazın kaçıncı partition'ı.** Numaralandırma **1'den** başlar.

> **Önemli**
>
> `load` her zaman **DDR'e** yükler. `mmc 0:2` dosyanın **nereye** yazılacağını değil, **nereden okunacağını** söyler. Yani "2. partition'a yükle" değil, "**2. partition'dan oku**, DDR'e yükle".

Bizim SD kartımız için:

Yazım	Anlamı	İçinde ne var?
`mmc 0:1`	SD kart, 1. partition	BOOT (FAT16): `MLO` , `u-boot.img` , `uEnv.txt`
`mmc 0:2`	SD kart, 2. partition	ROOTFS (ext3): `/boot/uImage` , `/boot/am335x-boneblack.dtb` , Linux dosyaları
`mmc 1:...`	eMMC'nin partition'ları	Kartla fabrikadan gelen sistem (ör. Ångström/Debian)

Örnekler:

```
U-Boot# ls mmc 0:1                                  # BOOT bölümünü listele
U-Boot# ls mmc 0:2 /boot                            # ROOTFS'taki /boot klasörünü listele
U-Boot# load mmc 0:1 0x82000000 uEnv.txt            # BOOT'taki uEnv.txt'yi DDR'e yükle
U-Boot# mmc list                                    # U-Boot'un gördüğü MMC cihazlarını listele
U-Boot# mmc dev 0                                   # SD kartı seç
U-Boot# mmc part                                    # Seçili cihazın partition tablosunu göster
```

> **Not**
>
> Linux'ta da benzer bir numaralandırma vardır: `/dev/mmcblk0p2` = MMC cihazı 0, partition 2. `uEnv.txt` 'deki `root=/dev/mmcblk0p2` ifadesi, U-Boot'taki `mmc 0:2` ile aynı yeri gösterir. Ancak Linux'un numaraları kernel sürümüne göre U-Boot'tan farklı olabilir, bu yüzden ikisinin her zaman aynı olacağını varsayma.

Çıktıda kaç byte okunduğu yazar (`xxxxxx bytes read`). Bu sayı uImage'ın boyutudur, yani 64 byte header + zImage.

> **Not**
>
> Bu komut `uEnv.txt` 'deki `loadfromsd` satırının yaptığı işin aynısı. Fark sadece bu sefer elle yazmamız.

> **İpucu**
>
> `load` komutu tanınmazsa eski adı olan `ext2load mmc 0:2 0x82000000 /boot/uImage` komutunu kullan (ext3 de bu komutla okunur). `ls` yerine de `ext2ls mmc 0:2 /boot` kullanılır.

4.10.3 Header'ı ekrana dök

```
U-Boot# md.b 0x82000000 40
```

Parça	Anlamı
`md`	memory display: belleği ekrana dök
`.b`	byte byte göster
`0x82000000`	Başlangıç adresi (uImage'ı yüklediğimiz yer)
`40`	Kaç byte gösterilecek. **U-Boot sayıları hex kabul eder:** `0x40` = **64 byte** = header'ın tamamı

Çıktı 4 satırdır. Her satırda adres, 16 byte ve sağda bu byte'ların ASCII karşılığı bulunur.

> **Uyarı**
>
> `md` komutu nokta olmadan (`md 0x82000000 10`) yazılırsa veriyi **4 byte'lık word'ler** halinde gösterir. ARM işlemci **little-endian**, header ise **big-endian** olduğu için byte'lar ters görünür. Örneğin magic number `27051956` yerine **`56190527`** olarak görünür. Header'ı okurken `md.b` kullanmak daha kolaydır.

4.10.4 Byte'ları çöz

Çıktının her satırı header'ın şu alanlarına denk gelir:

```
82000000: [27 05 19 56] [.. .. .. ..] [.. .. .. ..] [.. .. .. ..]
           ih_magic      ih_hcrc       ih_time       ih_size
82000010: [.. .. .. ..] [.. .. .. ..] [.. .. .. ..] [..][..][..][..]
           ih_load       ih_ep         ih_dcrc      os arch type comp
82000020: [ ih_name: 32 byte imaj adı, sağdaki ASCII sütununda okunur ...
82000030:   ...                                                      ]
```

Alan	Nasıl kontrol edilir?
`ih_magic`	`27 05 19 56` olmalı. Değilse bu adreste geçerli bir uImage yok.
`ih_time`	Unix zaman damgası. Linux'ta tarihe çevir: `date -d @$((0xXXXXXXXX))`
`ih_size`	zImage boyutu. Ondalığa çevir: `echo $((0xXXXXXXXX))`. Bu değer + 64 = `load` komutunun okuduğu byte sayısı olmalı.
`ih_load`, `ih_ep`	Kernel'in DDR'de çalışacağı adres ve başlangıç noktası
`os`	`05` = Linux
`arch`	`02` = ARM
`type`	`02` = Kernel
`comp`	`00` = sıkıştırma yok. zImage zaten kendini açtığı için U-Boot ayrıca açmaz.
`ih_name`	Sağdaki ASCII sütununda imajın adı okunur

> **İpucu**
>
> Hex'i ondalığa çevirmek için Windows Hesap Makinesi'nin **Programcı** modunu da kullanabilirsin.

4.10.5 Sonucu U-Boot'a doğrulat

Elle çözdüğün değerleri U-Boot'un kendi çözümüyle karşılaştır:

```
U-Boot# iminfo 0x82000000
```

`iminfo` komutu header'ı okuyup Image Name, Created, Image Type, Data Size, Load Address ve Entry Point bilgilerini yazar. Ayrıca CRC'yi kontrol eder (`Verifying Checksum ... OK`). Değerler senin bulduklarınla aynı olmalı.

4.10.6 Linux'u başlat

İşin bitince Linux'u normal şekilde açmak için:

```
U-Boot# boot
```

Ya da kartı yeniden başlat (`reset` veya Power butonu).

Komut özeti

Komut	Ne yapar?
`ls mmc 0:2 /boot`	SD kartın 2. bölümündeki `/boot` klasörünü listeler
`load mmc 0:2 0x82000000 /boot/uImage`	uImage'ı DDR'de `0x82000000` adresine yükler
`md.b 0x82000000 40`	O adresten başlayarak 64 byte'ı byte byte gösterir
`iminfo 0x82000000`	O adresteki uImage header'ını çözüp yazar
`boot`	Normal açılışa devam eder (Linux'u başlatır)
`reset`	Kartı yeniden başlatır
`help`	Tüm U-Boot komutlarını listeler (`help md` gibi tek komut için de olur)

4.11 Kursun Güncel SD Kart İmajları (U-Boot 2022.04 + Kernel 5.10.168)

Bu bölümde kullandığımız Ångström dosyaları kursun ilk sürümünden kalma (2013). Kursun reposundaki `pre-built-images/SD-boot` klasöründe güncel bir set de var:

Dosya	İçerik	SD kartta nereye?
`MLO` , `u-boot.img`	U-Boot 2022.04	BOOT
`uImage`	Linux 5.10.168	BOOT
`am335x-boneblack.dtb`	Aynı kernel'in device tree'si	BOOT
`uEnv.txt`	Aşağıda	BOOT

Farklar:

- Kernel ve DTB artık ROOTFS'teki `/boot` klasöründe değil, **BOOT (FAT) bölümünde** duruyor. `load mmc 0:1 ...` ile okunuyor.
- Bu set bir rootfs içermiyor. ROOTFS bölümüne Bölüm 12'de BusyBox ile hazırlayacağımız dosya sistemini koyacağız. Kernel 5.10 ile 2012 tarihli Ångström rootfs'i birlikte çalışmaz.

- Bu U-Boot'ta `bootdelay=0`: geri sayım yok. U-Boot'u durdurmak için kart açılırken **Space'e basılı tut**. Bekleme süresini uzatmak 9.4'ün konusu.

Kursun `uEnv.txt`'si:

```
console=ttyS0,115200n8
netargs=setenv bootargs console=ttyO0,115200n8 root=/dev/mmcblk0p2 rw rootfstype=ext4 rootwait
debug earlyprintk mem=512M
netboot=echo Booting from microSD ...; setenv autoload no ; load mmc 0:1 ${loadaddr} uImage ;
load mmc 0:1 ${fdtaddr} am335x-boneblack.dtb ; run netargs ; bootm ${loadaddr} - ${fdtaddr}
uenvcmd=run netboot
```

Parça	Anlamı
`rootfstype=ext4`	Rootfs'in dosya sistemi türü. ext4 sürücüsü ext3 bölümleri de bağlayabilir.
`rootwait`	Kernel, rootfs'in bulunduğu aygıt (SD kart) hazır olana kadar bekler. MMC aygıtları geç algılandığı için önemli.
`debug earlyprintk`	Açılışta daha fazla mesaj basar. Sorun ayıklarken işe yarar.
`mem=512M`	Kernel'e kullanacağı RAM miktarını söyler.
`${loadaddr}`, `${fdtaddr}`	U-Boot'un kendi varsayılan değişkenleri (`0x82000000`, `0x88000000`). Dosyada tanımlanmadıkları halde çalışır.

> **Uyarı**
>
> Dosyanın ilk satırında `console=ttyS0` tanımlanmış ama `netargs` bu değişkeni kullanmıyor, `console=ttyO0` yazıyor. Kernel 5.10'da UART0'ın adı `ttyS0`'dır (3.4). Açılışta `Starting kernel ...`'dan sonra mesaj göremezsen `netargs` içinde `console=${console}` kullan.

4.12 Özet

- SD kart iki bölümden oluşur: **BOOT** (FAT16, `boot` bayraklı: `MLO`, `u-boot.img`, `uEnv.txt`) ve **ROOTFS** (ext3: kernel, DTB ve Linux dosyaları).
- Dosya adları önemli: RBL tam olarak `MLO`'yu, MLO tam olarak `u-boot.img`'yi arar.
- Kartı SD karttan açmak için güç verirken **S2**'yi basılı tut.
- U-Boot `uEnv.txt`'deki `uenvcmd`'yi çalıştırır: kernel ve DTB'yi `mmc 0:2`'den RAM'e yükler, `bootargs`'ı ayarlar, `bootm` ile başlatır.
- `uImage` = 64 byte U-Boot header + `zImage`. `md.b` ile byte byte, `iminfo` ile çözülmüş halde okunabilir.

4.13 Karşılaşılabilecek Problemler

Belirti	Sebep	Çözüm
Minicom'da hiçbir şey yok	TX/RX düz bağlı, baud yanlış ya da Hardware Flow Control açık	Bağlantıyı ve 115200 8N1 ayarını kontrol et (2.8)
Sürekli `CCCCCCCC...`	RBL, MLO'yu bulamadı	Dosya adı tam `MLO` mu, BOOT bölümü FAT mı, `boot` bayrağı açık mı? (4.4)
`error reading image u-boot.img`	BOOT'ta `u-boot.img` yok veya adı yanlış	Dosyayı `u-boot.img` adıyla kopyala (4.7)
Eski Linux açılıyor (Debian vb.)	Kart eMMC'den açıldı; S2 açılış anında basılı değildi	Gücü kes, S2 basılıyken tekrar ver (4.5.3)
U-Boot `/boot/uImage` bulamıyor	Rootfs ROOTFS bölümüne çıkarılmamış veya yanlış klasöre çıkarılmış	`ls /media/$USER/ROOTFS/boot/` ile kontrol et, arşivi `-C` ile doğru yere çıkar
`Kernel panic ... Unable to mount root fs`	ROOTFS bölümü boş veya `sync` yapılmadan kart çıkarılmış	Rootfs'i yeniden çıkar, `sync` + `umount` ile güvenli çıkar

Bölüm 5 — eMMC'den Boot

> **Bu bölümde:**
>
> - eMMC'nin ne olduğunu ve DDR RAM'den farkını öğreneceksin.
> - Güncel bir Debian imajını önce SD karta yazıp sonra **flasher script** ile eMMC'ye aktaracaksın.
> - Karta USB üzerinden SSH ile bağlanmayı göreceksin.
>
> **Ön koşul:** Bölüm 3 — boot sırası ve S2, Bölüm 4 — SD karttan boot.

5.1 Hatırlatma: eMMC Nedir?

eMMC (embedded MultiMediaCard), BeagleBone Black'in üzerine lehimli olarak gelen dahili depolama birimidir. Aslında içinde bir SD kart vardır ve doğrudan karta lehimlenmiştir. Kapasitesi Rev C'de **4 GB**, eski Rev B'de **2 GB**'tır.

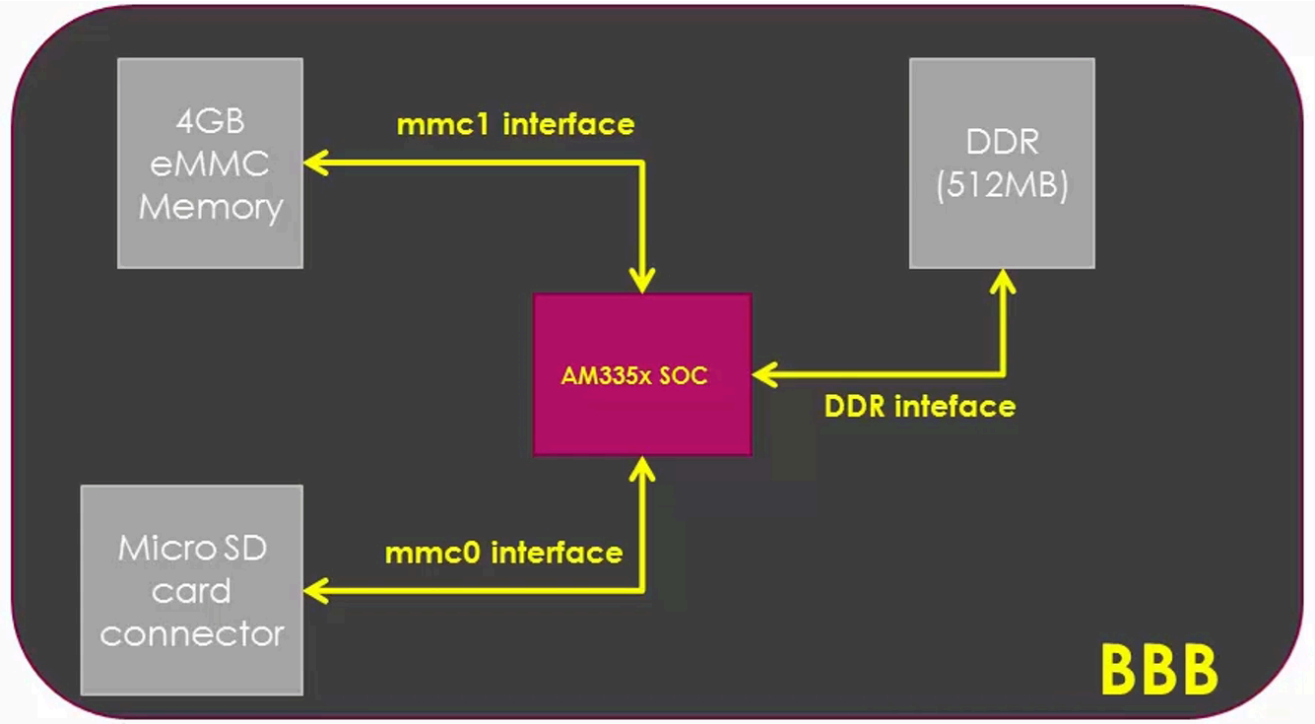
- Kartın **kalıcı depolama alanıdır**. Bilgisayardaki HDD/SSD gibi düşünebilirsin.
- Kart fabrikadan eMMC'de yüklü bir Debian ile gelir. Bu yüzden SD kart takmadan güç verildiğinde Linux açılır.
- SD karta göre **daha hızlı ve güvenilirdir**: 8-bit veri yolu ve dahili denetleyicisi vardır. Kartın üzerinde olduğu için çıkmaz, kaybolmaz, temas sorunu yaşamaz. Bu yüzden ürünlerde (sahadaki cihazlarda) tercih edilir.

DDR RAM ile farkı

	DDR RAM	**eMMC**
PC karşılığı	RAM	HDD / SSD
Kalıcılık	Geçici, güç gidince silinir	Kalıcı
Görevi	Çalışma alanı	Depolama
BBB'de boyut	512 MB DDR3L	4 GB (Rev C)

> **Not**
>
> İşlemci kodu doğrudan eMMC'den çalıştırmaz. Açılışta MLO, U-Boot, kernel ve device tree eMMC'den okunur, adım adım RAM'e yüklenir ve orada çalıştırılır.

5.2 Donanım Bağlantısı



Görsel: kursun slaytı.

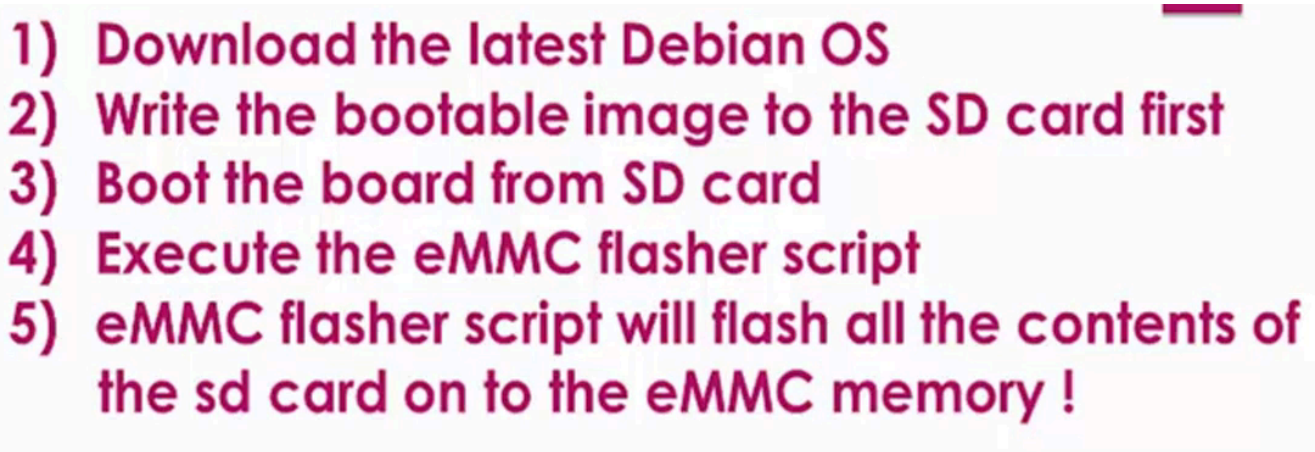
Bileşen	AM335x'e bağlantısı	Linux'ta (genelde)
4 GB eMMC	**mmc1** arayüzü (8-bit)	`mmcblk1`
microSD kart	**mmc0** arayüzü	`mmcblk0`
DDR (512 MB)	DDR arayüzü	-

> **Dikkat**
>
> Numaralandırma kernel sürümüne ve device tree'ye göre değişebilir. Bir diske bir şey yazmadan önce `lsblk` **ile mutlaka kontrol et**. Yanlış diske yazmak oradaki sistemi siler.

Varsayılan boot sırasında eMMC ilk sıradadır. eMMC'de geçerli bir MLO varsa kart oradan açılır, SD kart takılı olsa bile ona bakmaz. SD karttan açmak için **güç verirken S2'ye basılı tutulmalıdır** (3.5).

5.3 eMMC'ye Debian Yükleme (Teori)

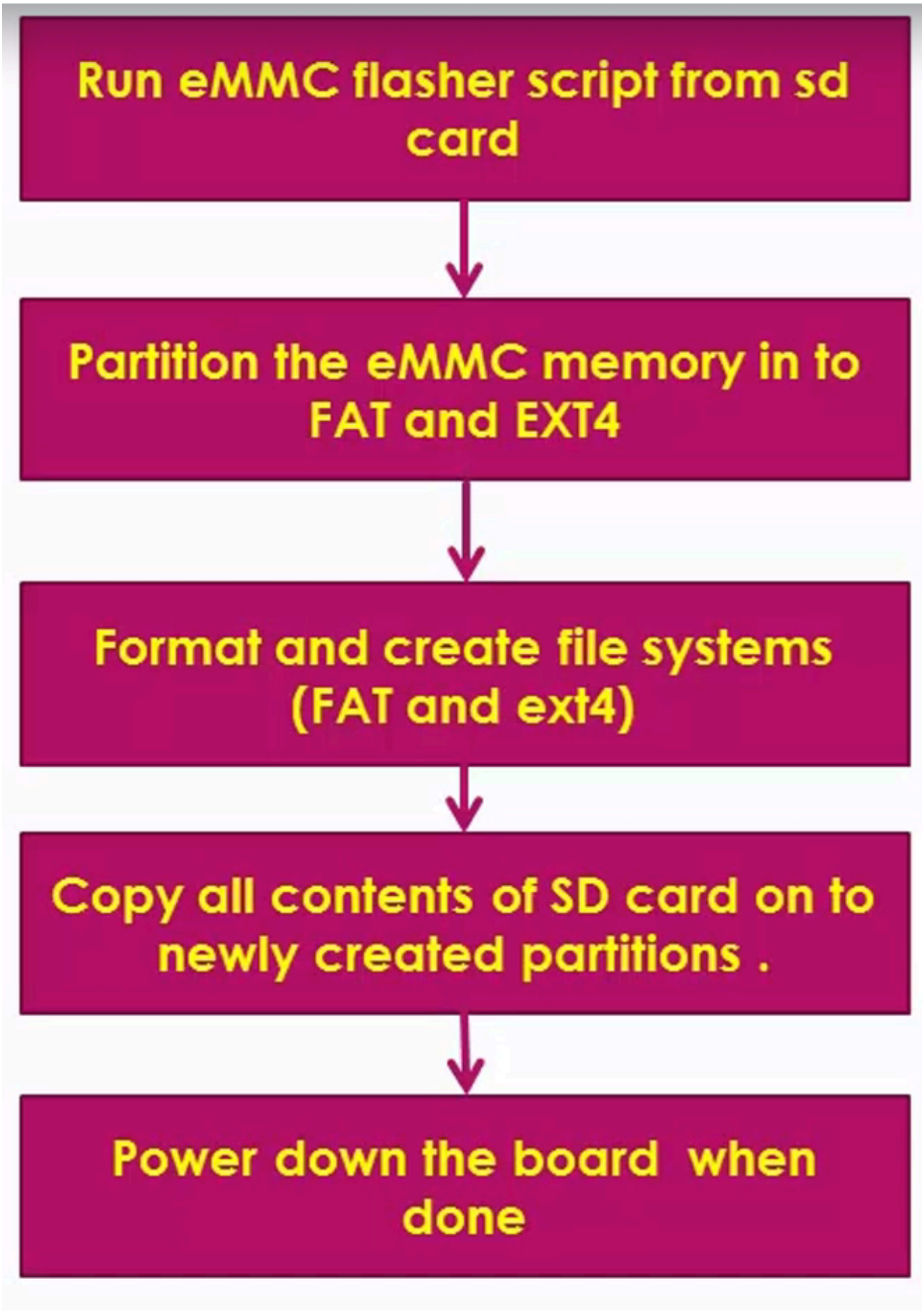
eMMC'ye doğrudan bilgisayardan yazılamaz, çünkü kartın üzerine lehimlidir. Bu yüzden önce **SD karttan** boot edilir, sonra SD karttaki her şey eMMC'ye kopyalanır.



Görsel: kursun slaytı.

1. En güncel **Debian OS** image'ını indir.
2. Bootable image'ı önce **SD karta** yaz.
3. Kartı **SD karttan boot et** (güç verirken S2'ye basılı tut).
4. **eMMC flasher script**'ini çalıştır.
5. Flasher script, **SD karttaki her şeyi eMMC'ye kopyalar**.

eMMC flasher script ne yapar?



Görsel: kursun slaytı.

1. Script **SD karttan** çalıştırılır.
2. eMMC iki bölüme (partition) ayrılır: **FAT** ve **EXT4**.
3. Bu bölümler formatlanır ve dosya sistemleri oluşturulur: - **FAT**: Boot dosyaları (MLO, u-boot.img vb.) - **EXT4**: Root dosya sistemi (rootfs)
4. SD karttaki tüm içerik yeni oluşturulan bölümlere kopyalanır.
5. İş bitince kart **kapatılır**.

> **Not**
>
> Bu, Bölüm 4'te SD kart için GParted ile elle yaptığımız bölümlendirme ve kopyalama işleminin eMMC için otomatik yapılan halidir.

5.4 Uygulama: eMMC'yi Güncelleme

5.4.1 Kartın IP adresini öğren

Kartı bilgisayara bağladığın yönteme göre IP adresi değişir. Güncel liste: https://beagleboard.org/getting-started

IP Address	Connection Type	Operating System(s)	Status
192.168.7.2	USB	Windows	
192.168.6.2	USB	Mac OS X, Linux	
192.168.8.1	WiFi	all	
beaglebone.local	all	mDNS enabled	
beaglebone-2.local	all	mDNS enabled	

Görsel: beagleboard.org.

Bağlantı	Bilgisayarın işletim sistemi	IP adresi
USB	Windows	`192.168.7.2`
USB	Mac OS X, **Linux**	`192.168.6.2`
WiFi	Hepsi	`192.168.8.1`

Bağlantı	Bilgisayarın işletim sistemi	IP adresi
Hepsi (mDNS)	Hepsi	`beaglebone.local`

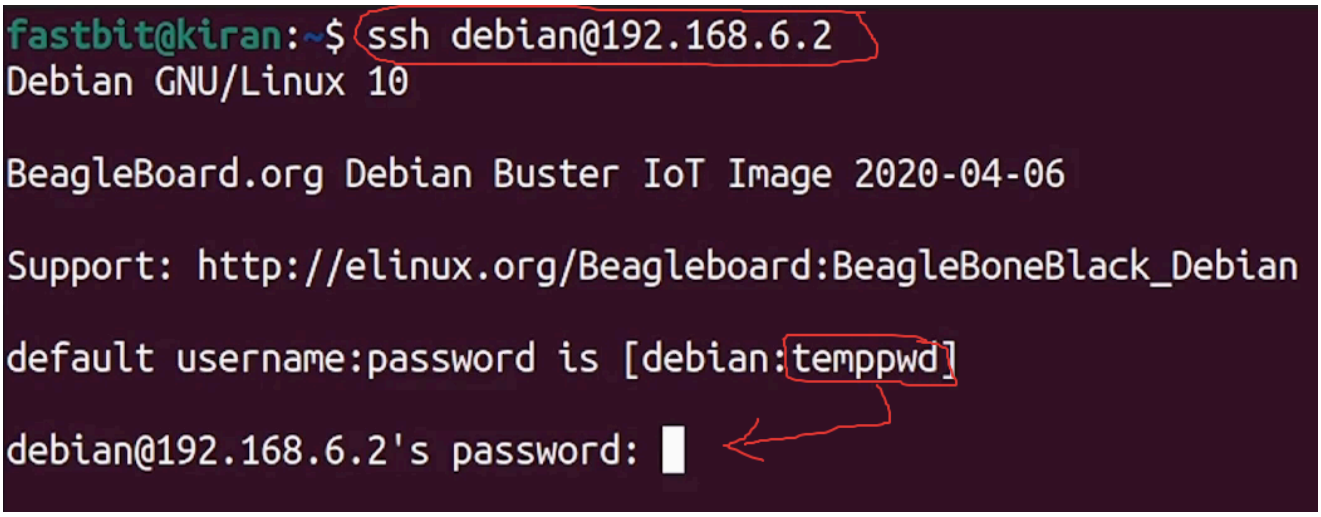
Bağlantıyı test etmek için:

```
ping 192.168.6.2
```

5.4.2 SSH ile karta bağlan

> **İpucu**
>
> SSH'ın bilgisayarında yüklü olup olmadığını `ssh -V` komutuyla kontrol edebilirsin. Yüklü değilse Ubuntu'da `sudo apt install openssh-client`, Fedora'da `sudo dnf install openssh-clients` ile kurabilirsin.

```
ssh debian@192.168.6.2
```



Görsel: kursun videosundan ekran görüntüsü.

Bağlanınca kart karşılama mesajında varsayılan kullanıcı adını ve şifreyi gösterir. `password:` satırına şifreyi yaz:

Kullanıcı adı	Şifre
`debian`	`temppwd`

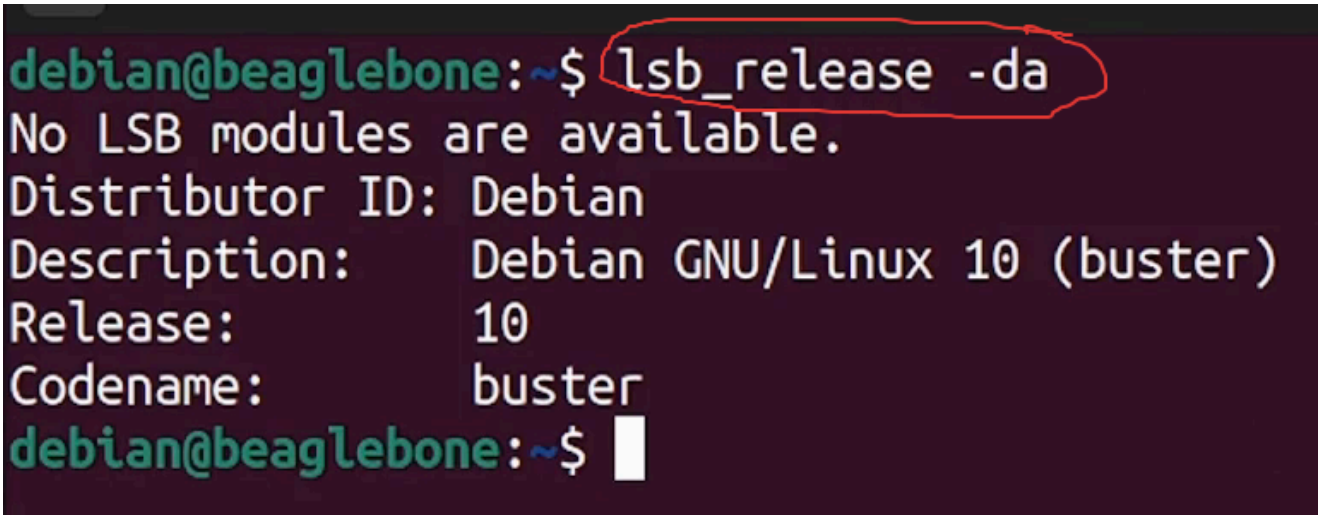
> **Not**
>
> Şifre yazarken ekranda hiçbir karakter görünmez. Bu normaldir, yazmaya devam edip Enter'a bas.

> **Önemli**
>
> `temppwd` şifresi kartla gelen **eski image'larda** geçerlidir. En güncel image'larda varsayılan şifre yoktur. Kullanıcı adı ve şifre, SD karta yazdıktan sonra FAT bölümündeki `sysconf.txt` dosyasından ilk açılıştan önce ayarlanmalıdır (5.4.4).

İlk bağlantıda `Are you sure you want to continue connecting (yes/no)?` diye sorarsa `yes` yaz.

5.4.3 Karttaki Debian sürümünü kontrol et

```
lsb_release -da
```



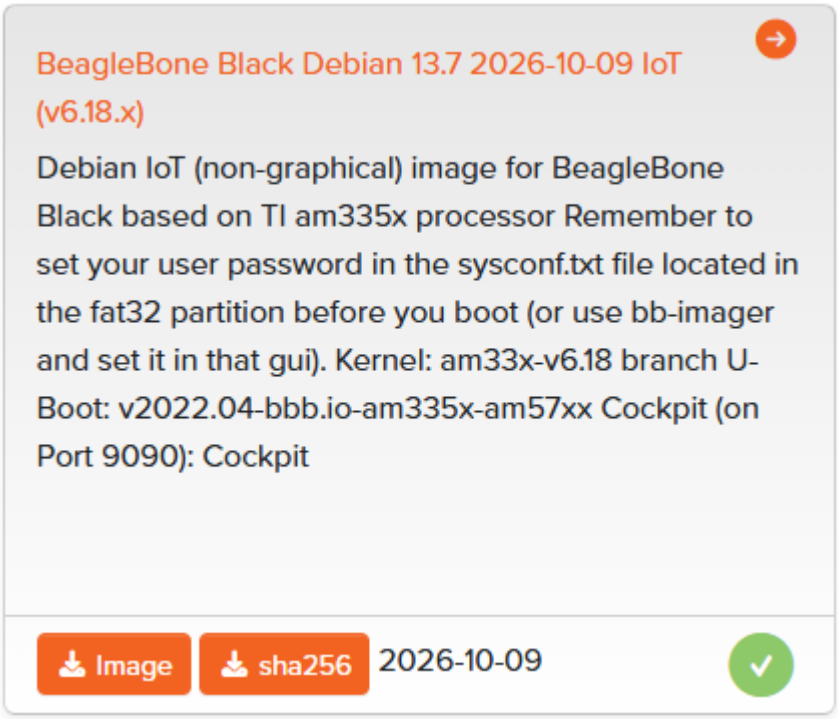
Görsel: kursun videosundan ekran görüntüsü.

Bu örnekte kartta **Debian 10 (buster)** yüklü.

> **İpucu**
>
> Kartında zaten en güncel Debian image'ı yüklüyse bu bölümün geri kalanını atlayabilirsin.

5.4.4 En güncel image'ı indir

Güncel image'lar: https://beagleboard.org/latest-images



*Görsel: beagleboard.org. Görselde aynı tarihli imajın v6.18.x çeşidi görünüyor; listede başlığında **(v5.10-ti)** yazanı seç.*

Bu kitap yazılırken en güncel sürüm **Debian 13.7 (2026-10-09, IoT)** idi. Karttaki Debian 10'dan çok daha yeni olduğu için eMMC'yi güncellemek gerekiyor.

Aynı tarihli imaj **üç farklı kernel** ile yayınlanıyor: v6.18.x, v6.12.x ve **v5.10-ti**. Bu kitapta **v5.10-ti** çeşidini indir:

Çeşit	Kernel	Bu kitapta
IoT (v5.10-ti)	BeagleBoard'un 5.10 LTS kernel'i	**Kullanılan.** Bölüm 15 – Bölüm 20'deki GPIO numaraları, `config-pin` ve cape-universal bu kernel'e göre. Bölüm 10'da derlediğimiz kernel de 5.10.
IoT (v6.12.x / v6.18.x)	Daha yeni kernel'ler	GPIO numaraları 512'den başlar (15.6); donanım bölümlerindeki `echo 60 > export` gibi komutlar başka pine gider.

İndireceğin dosya: `am335x-debian-13.7-base-v5.10-ti-armhf-2026-10-09-4gb.img.xz` (imajın sayfası).

- **Image** butonu ile `.img.xz` dosyasını indir.
- İstersen **sha256** ile indirdiğin dosyanın bozulmadığını kontrol edebilirsin:

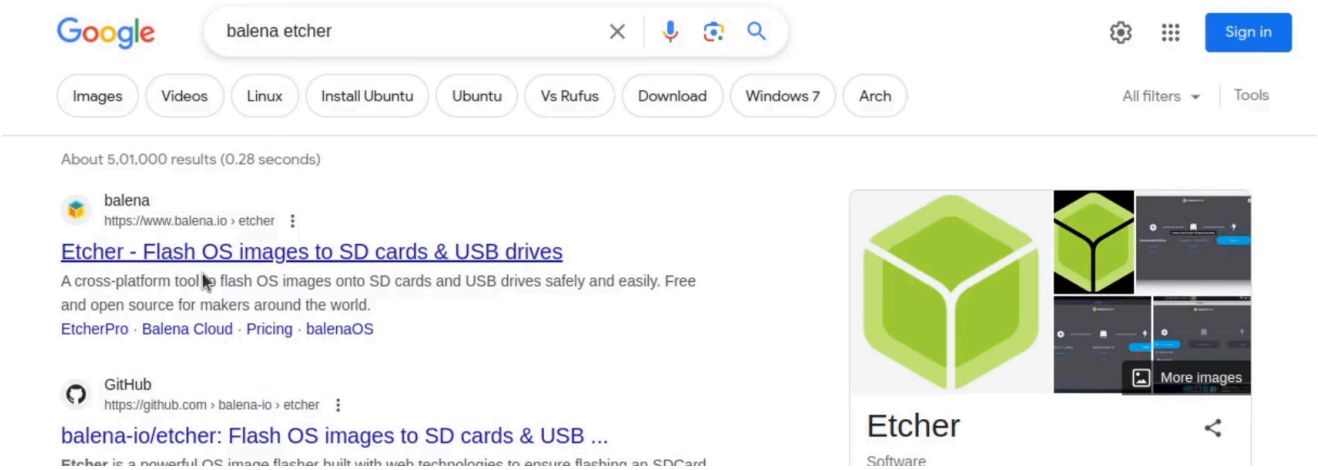
  ```
  sha256sum <indirilen-dosya>.img.xz
  ```

  Çıkan değer sitedeki sha256 değeriyle aynı olmalı.

> **Önemli**
>
> Sitedeki açıklamada da yazdığı gibi, bu image'da kullanıcı şifresi **ilk açılıştan önce** FAT32 bölümündeki `sysconf.txt` dosyasından ayarlanmalıdır.

5.4.5 Image'ı SD karta yaz (balenaEtcher)

Bu image doğrudan eMMC'ye yazılamaz. Önce **SD karta** yazılır, ardından kart SD karttan açılıp eMMC'ye aktarılır (5.3). SD karta yazmak için bir programa ihtiyacımız var: **balenaEtcher** (https://etcher.balena.io).



Görsel: kursun videosundan ekran görüntüsü.

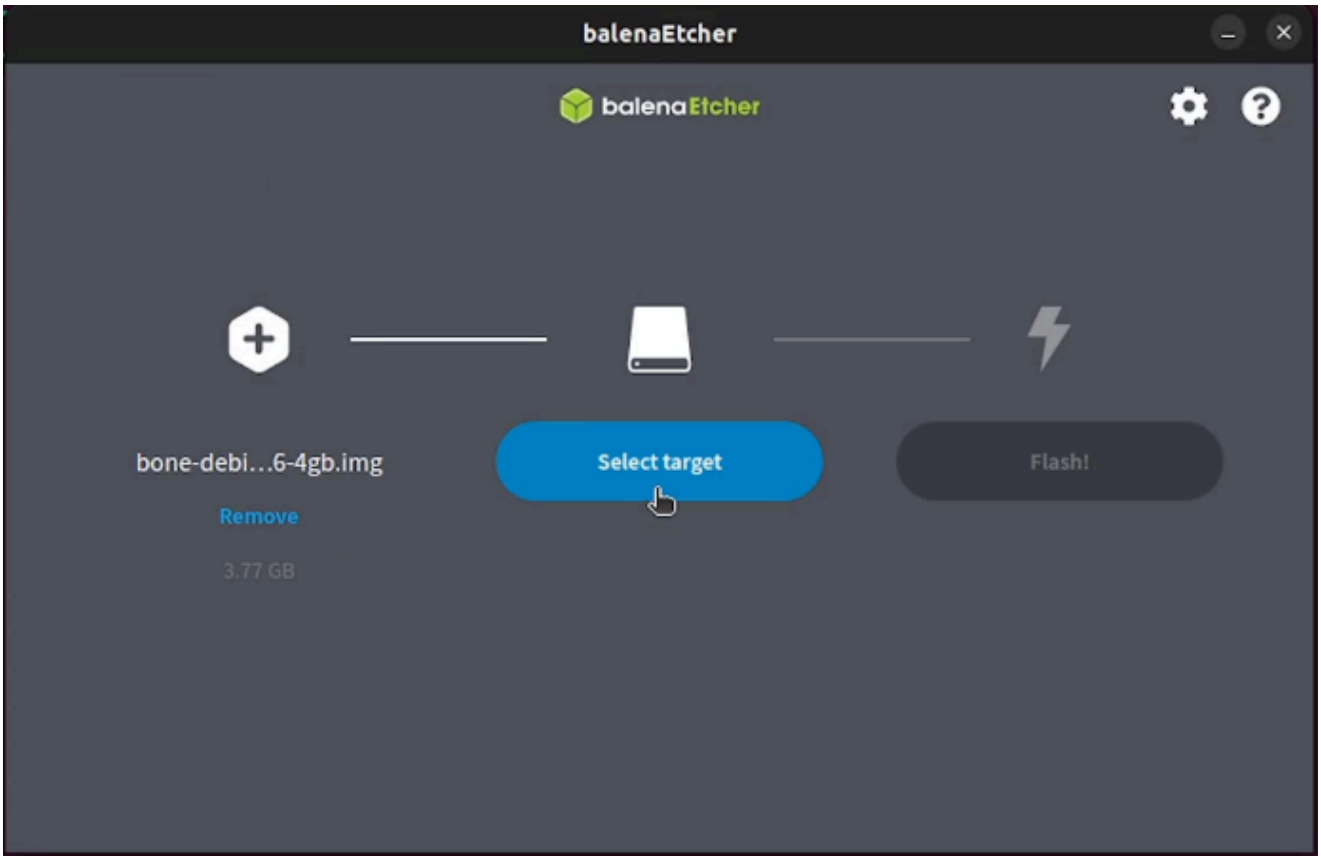
Neden dosyayı SD karta kopyalamak yetmiyor?

İndirilen `.img.xz` dosyası normal bir dosya değil, **bütün bir diskin birebir kopyasıdır**:

```
Debian image'ı (.img)
├── Bölüm tablosu (partition table)   → diskin hangi kısmı hangi bölüm
├── BOOT bölümü (FAT)                  → MLO, u-boot.img, sysconf.txt ...
└── rootfs bölümü (EXT4)               → Debian'ın tamamı
```

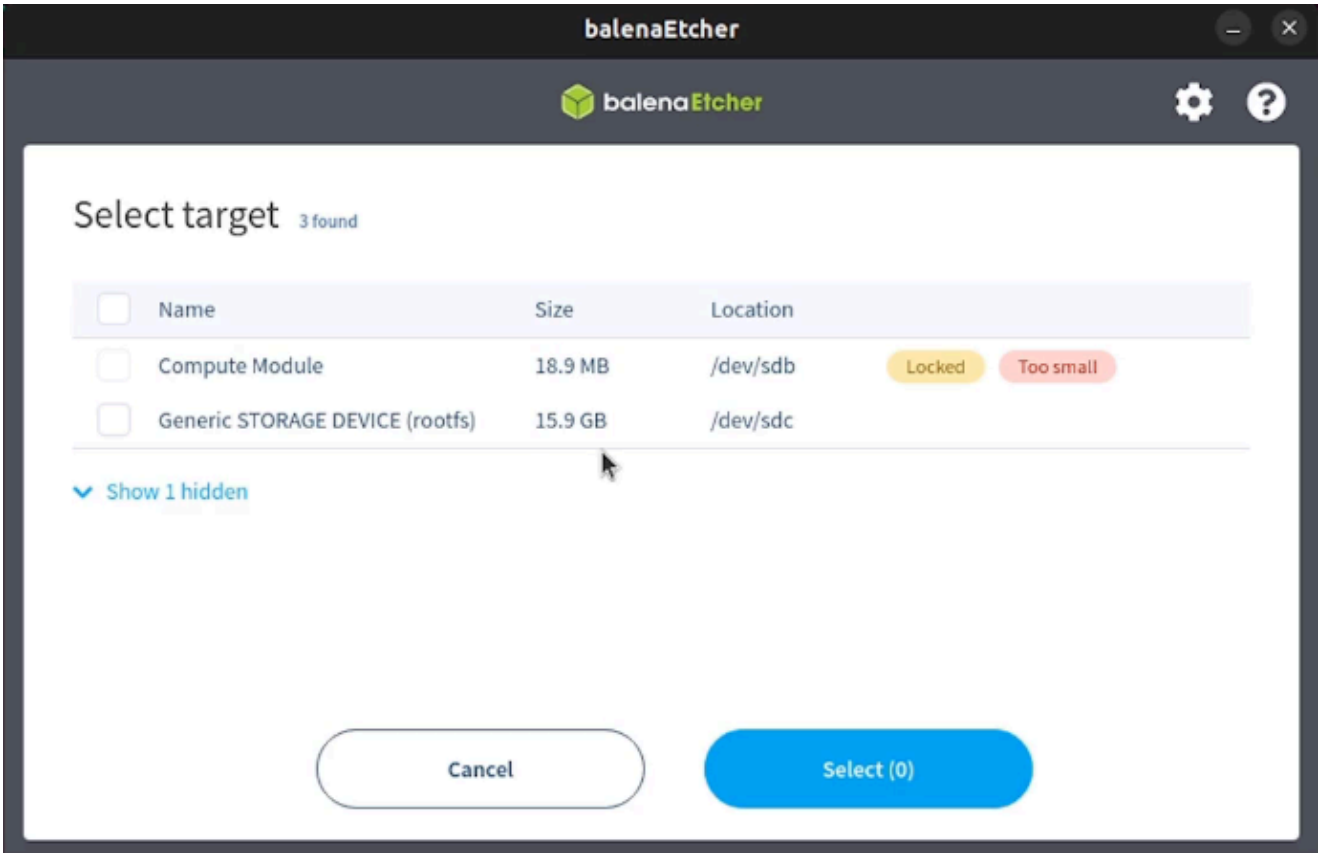
Dosyayı sürükle-bırak ile kopyalarsan SD kartta sadece tek bir büyük `.img` dosyası olur ve kart açılmaz. Etcher ise image'ı **diskin en başından itibaren ham olarak (byte byte) yazar**. Böylece SD kart image'daki bölümlere birebir sahip olur ve boot edilebilir hale gelir.

Etcher ile yazma adımları



Görsel: kursun videosundan ekran görüntüsü.

1. **Flash from file** ile indirdiğin image dosyasını seç (`am335x-debian-13.7-base-v5.10-ti-armhf-2026-10-09-4gb.img.xz` ; kursta `bone-debian-10.3-iot-armhf-2020-04-06-4gb.img.xz`).
2. **Select target** ile SD kartı seç. Kart okuyucu bilgisayarda bir USB depolama aygıtı olarak görünür.
3. **Flash!** butonuna bas. Yazma ve doğrulama (verify) bitene kadar bekle.



Görsel: kursun videosundan ekran görüntüsü.

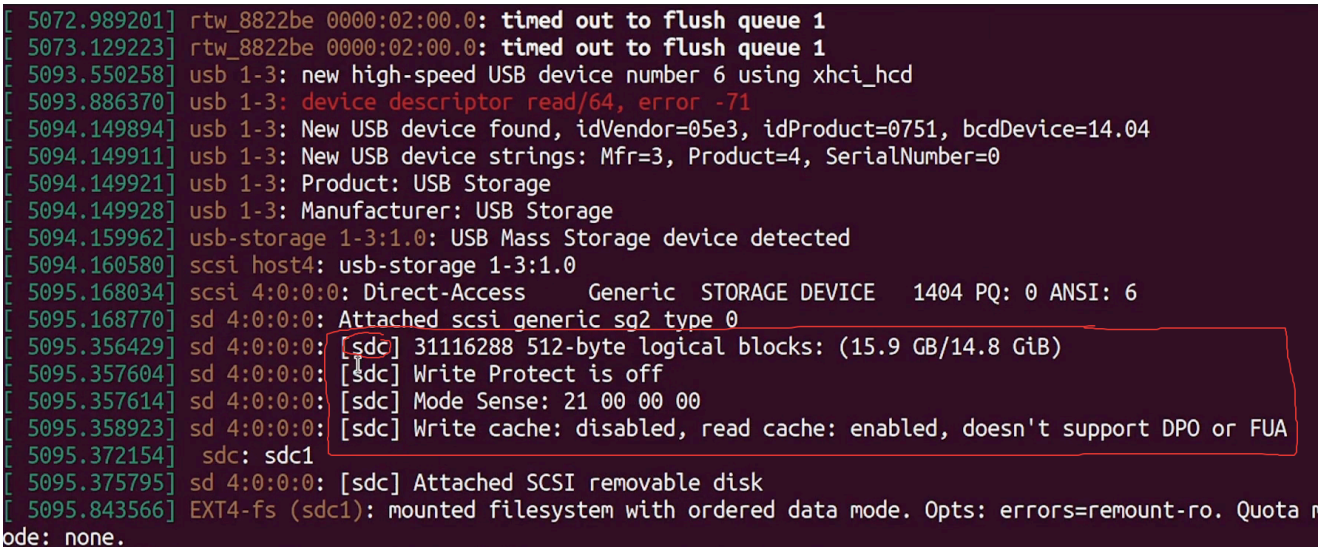
Bu örnekte iki aygıt görünüyor:

Aygıt	Boyut	Konum	Açıklama
Compute Module	18.9 MB	`/dev/sdb`	**Locked / Too small**, seçilemez
Generic STORAGE DEVICE	**15.9 GB**	**`/dev/sdc`**	**SD kart** → bunu seç

Hangisi SD kart? (`dmesg`)

Listede birden fazla aygıt çıkarsa ve hangisinin SD kart olduğundan emin değilsen, SD kartı taktıktan hemen sonra terminalde şunu çalıştır:

```
sudo dmesg | tail -20
```



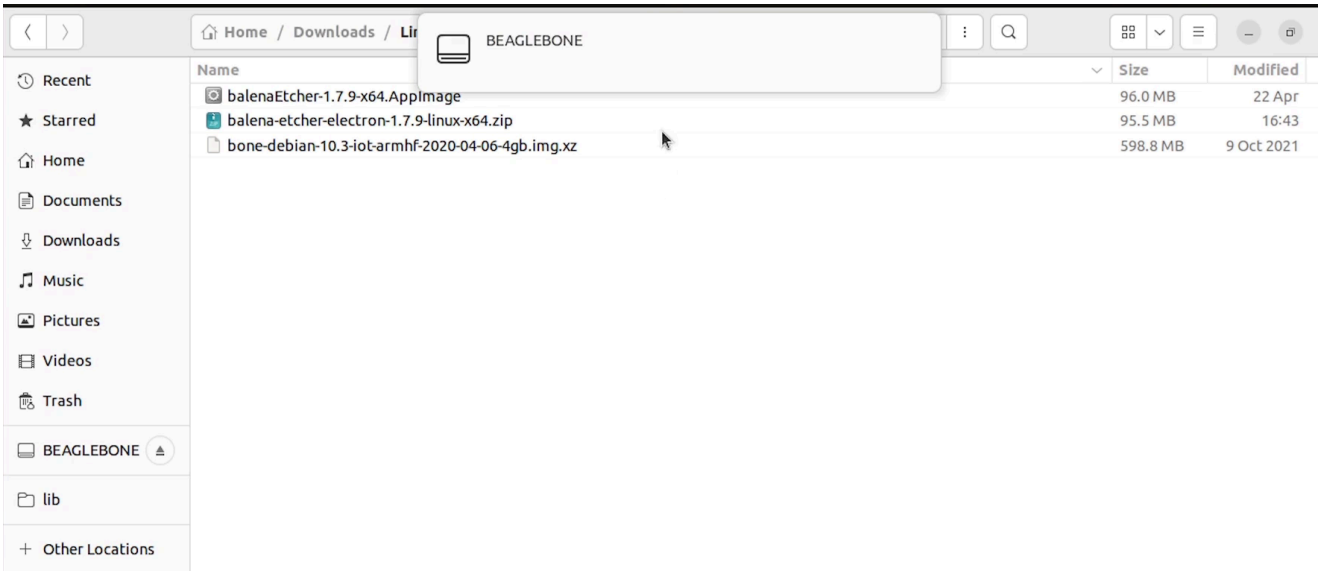
Görsel: kursun videosundan ekran görüntüsü.

Son satırlarda yeni takılan aygıtın adı ve boyutu görünür. Bu örnekte `[sdc] ... (15.9 GB/14.8 GiB)` satırı, SD kartın `/dev/sdc` olduğunu gösteriyor.

> **İpucu**
>
> `lsblk` komutu da aygıtları boyutlarıyla listeler. Harf (`sdb` , `sdc` ...) bilgisayardan bilgisayara değişir. Her seferinde boyuta bakarak kontrol et.

5.4.6 Kartı SD karttan boot et

1. Kartın **gücünü kes**.
2. SD kartı BeagleBone'a **tak**.
3. Güç kablosunu tak.
4. **Power** butonuna LED'ler sönene kadar basılı tut, LED'ler sönünce bırak. Kart artık kapalı.
5. **S2** butonunu basılı tut ve bu sırada **Power** butonuna bir kez basıp bırak.
6. S2'yi **2-3 saniye** daha basılı tut, sonra bırak.
7. Kart SD karttan açılmaya başlar. Bu sırada LED'ler hızlı hızlı yanıp söner.



Görsel: kursun videosundan ekran görüntüsü.

Bilgisayarında **BEAGLEBONE** adında bir sürücü göründüyse kart açılmıştır. Şimdi 5.4.2'deki SSH komutuyla tekrar bağlan:

```
ssh debian@192.168.6.2
```

> **Not**
>
> Bağlanırken `REMOTE HOST IDENTIFICATION HAS CHANGED!` uyarısı alırsan bu normaldir. Kart farklı bir sistemden (SD karttan) açıldığı için SSH anahtarı değişmiştir. Eski kaydı silip tekrar bağlan:
>
> ```
> ssh-keygen -R 192.168.6.2
> ```

5.4.7 eMMC flasher script'ini çalıştır

Flasher'ın adı ve yeri imajın yaşına göre değişir:

İmaj	Flasher
Güncel imajlar (Debian 11 ve sonrası, ör. 13.7)	`beagle-flasher` paketi
Kursun imajı (Debian 10.3)	`/opt/scripts/tools/eMMC/init-eMMC-flasher-v3.sh`

Güncel imajlarda: `beagle-flasher`

```
sudo enable-beagle-flasher      # bir sonraki açılışta SD karttaki sistemi eMMC'ye yazar
sudo reboot
```

`enable-beagle-flasher`, SD kartı bir sonraki açılışta flasher olarak çalışacak şekilde ayarlar. Kart yeniden açılınca kopyalama kendiliğinden başlar, bitince kart kapanır. Aynı paketteki `sudo beagle-flasher` komutu ise kopyalamayı hemen, çalışan sistemin içinden yapar. Daha hızlıdır ama kaynak sistem kopyalanırken çalıştığı için önerilen yol `enable-beagle-flasher`'dır.

> **Not**
>
> Komut bulunamazsa paketini ara ve kur: `sudo apt update && apt search beagle-flasher`, ardından listede çıkan paketi `sudo apt install <paket>` ile kur.

Kursun imajında: `init-eMMC-flasher-v3.sh`

Flasher script'leri `/opt/scripts/tools/eMMC` klasöründe bulunur:

```
cd /opt/scripts/tools/eMMC
ls
```

```
debian@beaglebone:~$ cd /opt/scripts/tools/
debian@beaglebone:/opt/scripts/tools$ ls
beaglebone-black-eMMC-flasher.sh  dtc       grow_partition.sh     readme.txt        version.sh
config_checking                   eMMC      init-eMMC-flasher.sh  software          wm
developers                        graphics  non-mmc-rootfs        update_kernel.sh
debian@beaglebone:/opt/scripts/tools$
```

```
debian@beaglebone:/opt/scripts/tools/eMMC$ ls
bbb-eMMC-flasher-eewiki-12mb.sh                  init-eMMC-flasher-v3-bbbw.sh
bbb-eMMC-flasher-eewiki-ext4.sh                  init-eMMC-flasher-v3-bbg.sh
beaglebone-black-make-microSD-flasher-from-eMMC.sh  init-eMMC-flasher-v3-bbgw.sh
functions.sh                                     init-eMMC-flasher-v3-m10a.sh
init-eMMC-flasher-a335.sh                        init-eMMC-flasher-v3-me06.sh
init-eMMC-flasher-bp00.sh                        init-eMMC-flasher-v3-no-eeprom.sh
init-eMMC-flasher-from-usb-media.sh              init-eMMC-flasher-v3.sh
init-eMMC-flasher-from-usb-media-v1-bbgw.sh      init-eMMC-flasher-v3-x15_b1.sh
init-eMMC-flasher-from-usb-media-v2-bbgw.sh      mmc_resize_usbflasher_example.md
init-eMMC-flasher-v2.sh                          mmc_usbflasher_example.md
init-eMMC-flasher-v3-bbbl.sh                     readme.md
debian@beaglebone:/opt/scripts/tools/eMMC$ sudo ./init-eMMC-flasher-v3
```

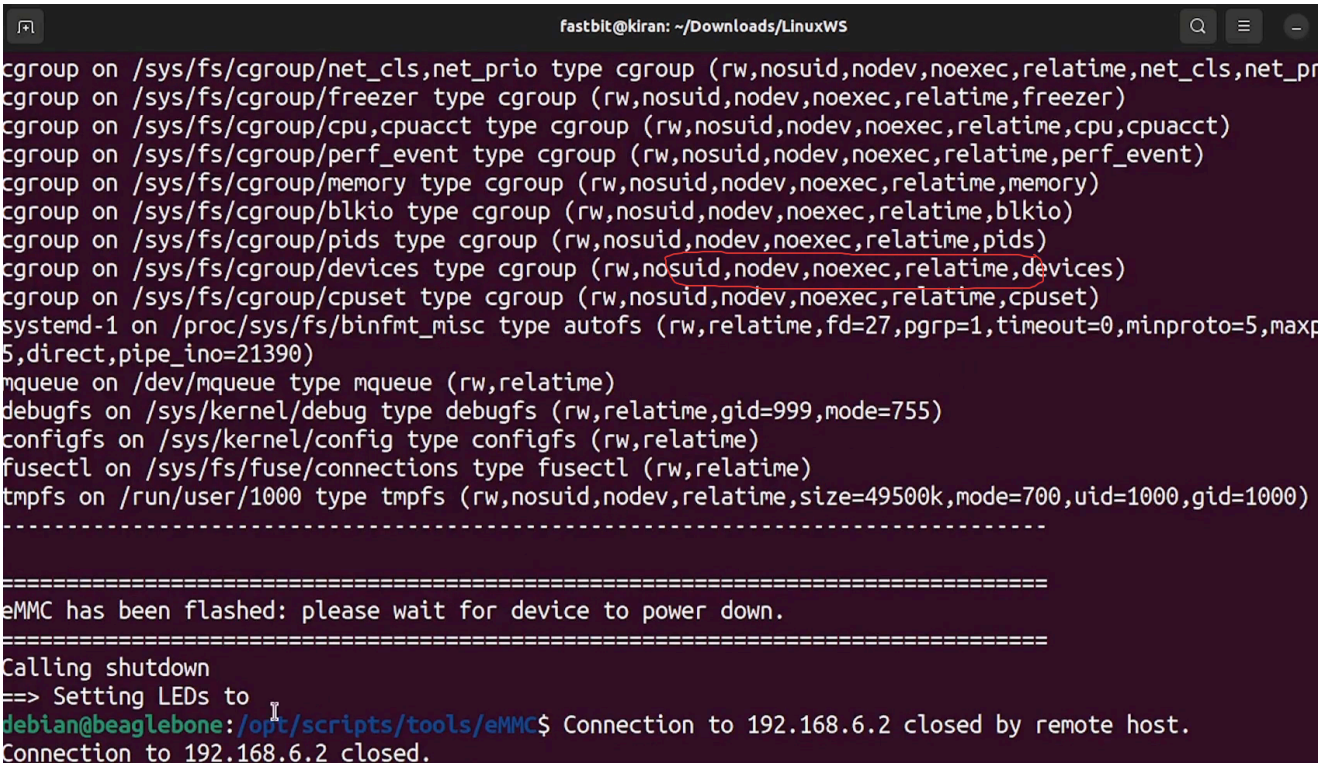
Görseller: kursun videosundan ekran görüntüleri.

Listede birçok script var. BeagleBone Black için kullanılan `init-eMMC-flasher-v3.sh`:

```
sudo ./init-eMMC-flasher-v3.sh
```

> **Dikkat**
>
> Bu script eMMC'deki **her şeyi siler** ve yerine SD karttaki sistemi yazar. eMMC'de saklamak istediğin dosya varsa önce yedekle.

Script çalışırken 5.3'te anlatılan adımları sırayla yapar: bölümlendirme → formatlama → kopyalama. Bu birkaç dakika sürer.



Görsel: kursun videosundan ekran görüntüsü.

Flash işlemi başarılıysa şu mesajı görürsün:

```
eMMC has been flashed: please wait for device to power down.
```

Ardından kart kendini kapatır ve SSH bağlantısı kopar (`Connection to 192.168.6.2 closed by remote host.`). Bu normaldir.

5.4.8 eMMC'den boot et

1. SD kartı BeagleBone'dan **çıkar**. Artık ona ihtiyacın yok.
2. **Power** butonuna bir kez bas (S2'ye **basmadan**). Kart artık eMMC'den açılır.
3. SSH ile tekrar bağlan:

   ```
   ssh debian@192.168.6.2
   ```

4. Yeni sürümün yüklendiğini kontrol et:

   ```
   lsb_release -da
   ```

5.5 Özet

- eMMC kartın kalıcı, dahili deposudur (MMC1). DDR ise geçici çalışma belleğidir.
- eMMC'ye doğrudan yazılamaz: imaj önce **balenaEtcher** ile SD karta yazılır, kart **S2** ile SD karttan açılır, **flasher** (`enable-beagle-flasher` ; eski imajlarda `init-eMMC-flasher-v3.sh`) SD karttaki sistemi eMMC'ye kopyalar.
- Bu kitapta Debian 13.7'in **v5.10-ti** çeşidi kullanılıyor; donanım bölümlerindeki GPIO numaraları bu kernel'e göre.
- USB üzerinden kartın adresi Linux'ta `192.168.6.2` , Windows'ta `192.168.7.2` . Bağlantı: `ssh debian@192.168.6.2` .

5.6 Karşılaşılabilecek Problemler

Belirti	Sebep	Çözüm
`ping 192.168.6.2` cevap vermiyor	Kart henüz açılmadı ya da Windows'tasın	1–2 dakika bekle. Windows'ta adres `192.168.7.2` . `beaglebone.local` da denenebilir.
`REMOTE HOST IDENTIFICATION HAS CHANGED!`	Kart farklı bir sistemden açıldı	`ssh-keygen -R 192.168.6.2`
Şifre kabul edilmiyor	Yeni image'da varsayılan şifre yok	SD karttaki `sysconf.txt` ile kullanıcı/şifre belirle (5.4.4)
`/opt/scripts/tools/eMMC` yok	Güncel imaj; flasher artık `beagle-flasher` paketinde	`sudo enable-beagle-flasher` (5.4.7)
Donanım bölümlerinde `echo 60 > /sys/class/gpio/export` hata veriyor ya da başka pin değişiyor	İmajın v6.x çeşidi yüklendi; GPIO numaraları 512'den başlıyor	v5.10-ti çeşidini yükle ya da numarayı `cat /sys/kernel/debug/gpio` ile bul (15.6)
Kart flash'tan sonra yine eski sistemle açılıyor	SD kart hâlâ takılı ve S2'ye basıldı / flash yarıda kaldı	SD kartı çıkar, S2'ye basmadan aç. Gerekirse flash'ı tekrarla.

Kısım III
U-Boot ile Çalışmak

Bölüm 6 — U-Boot Komut Satırı ve uEnv.txt

> **Bu bölümde:**
>
> - `uEnv.txt` dosyasının **satır satır ne yaptığını** anlayacaksın.
> - Bu dosyayı SD karta kopyalamadan, **seri port (UART) üzerinden** doğrudan U-Boot'a göndereceksin (`loady` , `env import`).
> - Kursun challenge'ı: boot imajlarını eMMC yerine **SD karttan (MMC0)** yükleyeceksin. Ek alıştırmada da tersini yapıp rootfs'i **eMMC'den** bağlayacaksın.
> - **ROOTFS** bölümünde hangi klasör ve dosyaların olduğunu ve bunlara **hangi komutlarla** ulaşacağını göreceksin.
>
> **Ön koşul:** Bölüm 4 — SD karttan boot, Bölüm 5 — eMMC'deki Debian.

> **Not**
>
> Bu bölümdeki terminal görüntüleri temsilidir. Senin kartında U-Boot sürümü, dosya boyutları ve bölüm boyutları farklı görünebilir.

6.1 Temel Kavramlar

Kavram	Ne işe yarar?
`uEnv.txt`	U-Boot ortam değişkenlerini **metin olarak** tanımlayan dosya. Her satır `isim=değer` biçimindedir. U-Boot'un bu dosyayı açılışta otomatik okuyup okumaması, U-Boot'un yapılandırmasına bağlıdır (BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt` 'yi arar).
Ortam değişkeni (environment)	U-Boot'un ayarları ve makroları. `printenv` ile görülür, `setenv` ile değiştirilir, `saveenv` ile kalıcı kaydedilir.
Minicom	Bilgisayardan seri port (UART) üzerinden karta bağlanmayı sağlayan terminal programı (Bölüm 2).
`loady`	U-Boot'ta **YMODEM** protokolüyle seri porttan dosya alıp RAM'e yazan komut.
`env import -t <adres> <boyut>`	RAM'deki `isim=değer` metnini okuyup U-Boot'un **aktif** ortamına ekler. `-t` "düz metin" demektir, `<boyut>` metnin bayt cinsinden uzunluğudur.

Bütün akış şöyle:

```
Bilgisayar (uEnv.txt) ──YMODEM / UART──► RAM 0x82000000 ──env import──► U-Boot ortamı ──boot──► Linux
```

6.2 `uEnv.txt` Satır Satır

Kullanacağımız dosya bu:

```
mypcip=setenv serverip 192.168.1.2
ipaddr=192.168.27.1
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000 uImage;load mmc
0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
```

`mypcip` ve `ipaddr` — ağ ayarları

Satır	Anlamı
`mypcip=setenv serverip 192.168.1.2`	Bir **makro**. Kendi başına hiçbir şey yapmaz; `run mypcip` yazınca içindeki komut çalışır ve `serverip` (TFTP/NFS sunucusunun, yani bilgisayarın IP'si) `192.168.1.2` olur.
`ipaddr=192.168.27.1`	Kartın kendi statik IP adresi.

> **Uyarı**
>
> Bu örnekte kart `192.168.27.x`, sunucu `192.168.1.x` ağında. Ağ geçidi (`gatewayip`) tanımlanmadığı sürece TFTP/NFS çalışmaz. Ağdan boot yapacaksan iki adresi **aynı alt ağda** seç (Bölüm 8). Ayrıca kitabın ağ ayarlarında `192.168.27.1` **bilgisayarın** adresidir; kursun dosyasındaki `ipaddr=192.168.27.1` onunla çakışır, kart için `192.168.27.2` kullan. SD karttan boot için bu iki satır kullanılmaz.

`bootargs` — kernel'e verilen parametreler

`bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw`

Parça	Anlamı
`console=ttyO0,115200`	Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın. `ttyO0` eski OMAP kernel'lerindeki addır; yeni kernel'lerde `ttyS0` olur.
`root=/dev/mmcblk0p2`	Root filesystem: MMC aygıtı **0**, bölüm **2** (`p2`).
`rw`	Rootfs okuma-yazma modunda bağlansın.

`bootcmd` — otomatik açılış komutu

`bootcmd`, `;` ile ayrılmış komutlardan oluşur. U-Boot bunları sırayla çalıştırır:

Sıra	Komut	Ne yapar?
1	`echo "****Booting from memory****"`	Ekrana mesaj basar (hangi yoldan açıldığını görmek için).
2	`load mmc 0:1 0x82000000 uImage`	`mmc 0:1` = SD kart, 1. bölüm (BOOT). Oradaki `uImage` dosyasını RAM'de `0x82000000` adresine yükler.
3	`load mmc 0:1 0x88000000 am335x-boneblack.dtb`	Aynı bölümdeki device tree dosyasını (DTB) `0x88000000` adresine yükler.
4	`bootm 0x82000000 - 0x88000000`	Kernel'i başlatır. Ortadaki `-` "initramfs yok" demektir, son adres DTB'nin adresidir.

```
RAM (DDR, 0x80000000'dan başlar)
├── 0x82000000  →  uImage   (kernel)
└── 0x88000000  →  am335x-boneblack.dtb
```

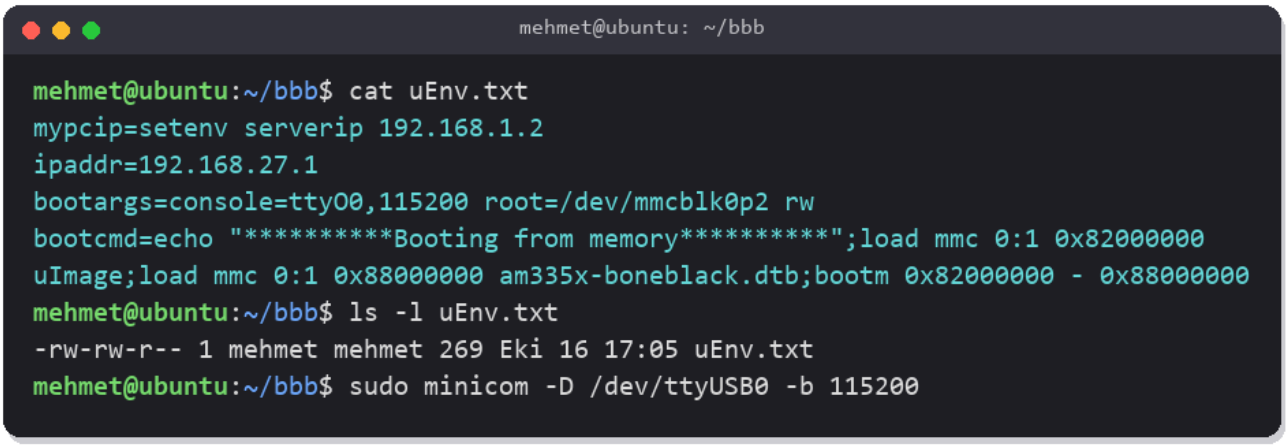
> **Not**
>
> `mmc 0` ↔ `/dev/mmcblk0` eşleşmesi çoğu kurulumda doğrudur ama **garanti değildir**. U-Boot tarafında `mmc list`, Linux tarafında `lsblk` ile kontrol et (6.5).

6.3 Uygulama: `uEnv.txt`'yi Seri Porttan Göndermek

Normalde `uEnv.txt` SD kartın BOOT bölümüne kopyalanır. Her değişiklikte SD kartı çıkar-tak yapmak yerine dosyayı **seri porttan** gönderip anında deneyebiliriz.

Adım 1 — Bilgisayarda dosyayı hazırla ve Minicom'u aç

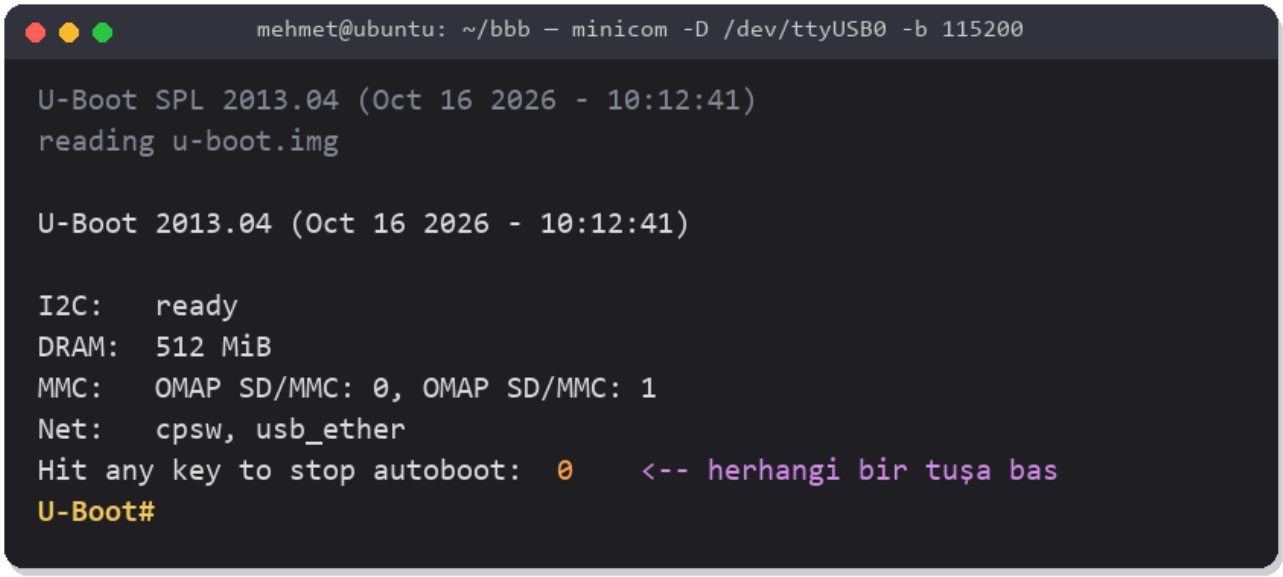
Dosyanın içeriğini ve **boyutunu** kontrol et, sonra Minicom ile karta bağlan:



> **İpucu**
>
> Windows'ta port adı `/dev/ttyUSB0` yerine `COM3` gibi olur (2.6).

Adım 2 — U-Boot'u durdur

Kartı aç. `Hit any key to stop autoboot` yazısı çıkınca **bir tuşa bas**. `U-Boot#` komut satırı gelmeli:



Adım 3 — `loady` ile dosyayı RAM'e al

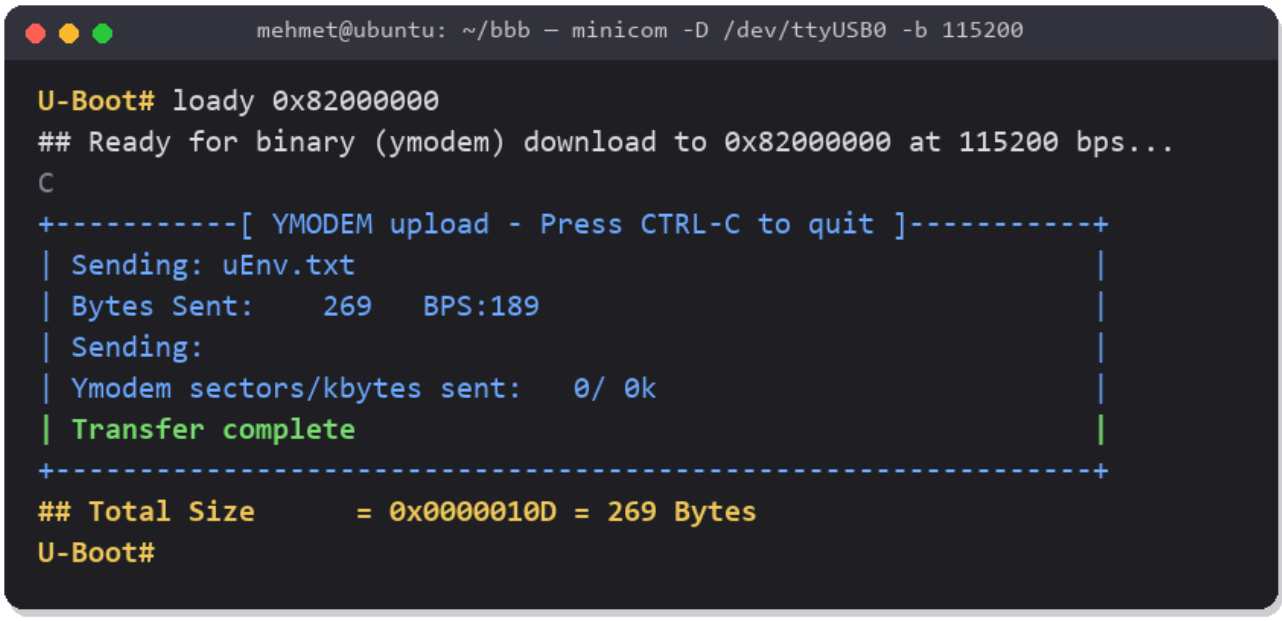
U-Boot'ta YMODEM alımını başlat:

```
U-Boot# loady 0x82000000
```

U-Boot `C` karakterleri basarak beklemeye başlar. Şimdi Minicom'da:

1. `Ctrl-A` → `S` (Send files)
2. Protokol olarak **ymodem** seç
3. `uEnv.txt` dosyasını seçip gönder

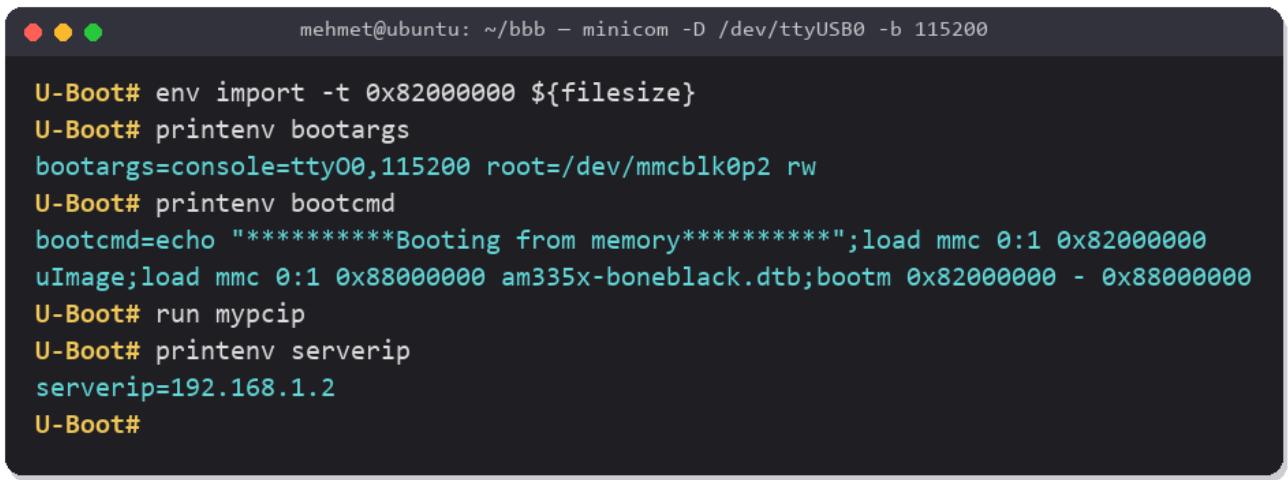
Transfer bitince U-Boot alınan dosyanın boyutunu yazar:



> **Not**
>
> `## Total Size = 0x0000010D = 269 Bytes` satırı önemli: Bu değer otomatik olarak **`filesize`** değişkenine yazılır. Bir sonraki adımda boyutu elle yazmak yerine `${filesize}` kullanabiliriz.

Adım 4 — `env import` ile ortama aktar ve kontrol et

```
U-Boot# env import -t 0x82000000 ${filesize}
U-Boot# printenv bootargs
U-Boot# printenv bootcmd
```



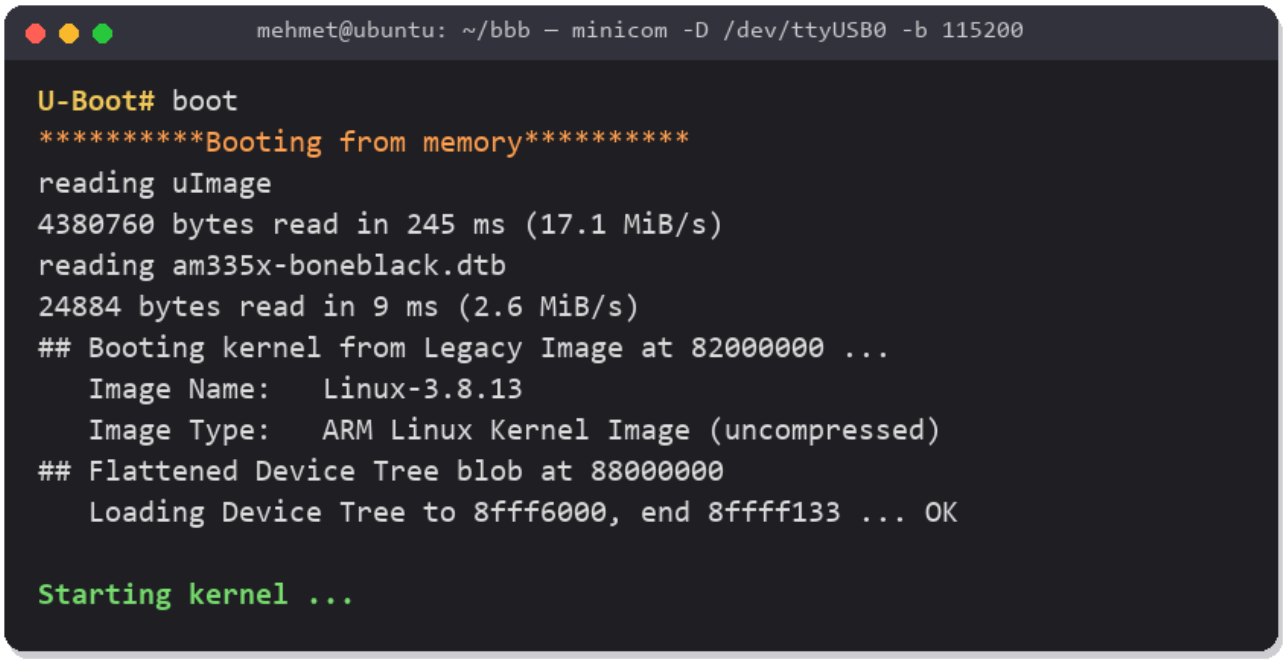
`printenv` çıktısı `uEnv.txt`'deki değerlerle aynıysa aktarım başarılıdır. `run mypcip` ile makronun da çalıştığını görebilirsin.

> **Uyarı**
>
> `${filesize}` yerine boyutu elle yazarsan **hex** olarak yaz (`0x10d` veya `10d`). U-Boot sayıları varsayılan olarak hex kabul eder; `269` yazarsan `0x269` = 617 bayt okur ve dosyanın sonundaki çöp veriyi de içe aktarmaya çalışır.

Adım 5 — Boot et

```
U-Boot# boot
```

`boot` komutu `bootcmd` 'yi çalıştırır. Ekrana bizim `echo` mesajımız gelmeli:



Dikkat edilecekler

Konu	Açıklama
Adres çakışması	`uEnv.txt` `0x82000000` 'a yüklendi; `bootcmd` de kernel'i aynı adrese yüklüyor. Sorun değil, çünkü `env import` dosyayı önce okuyup bitiriyor, kernel **sonra** üzerine yazılıyor. Sıra ters olursa kernel bozulur.
Geçicilik	`env import` sadece RAM'deki aktif ortamı değiştirir. Kart yeniden açılınca değişiklikler kaybolur.
Kalıcı yapmak	Her şeyin doğru olduğundan emin olduktan sonra `saveenv` ile kaydedilebilir. Bu komut kalıcı U-Boot ortamının üzerine yazar; geri dönmek için `env default -a` + `saveenv` gerekir.

Konu	Açıklama
Otomatik okumada `bootcmd`	BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt`'yi kendi `bootcmd`'si **çalışırken** okur ve `uenvcmd` değişkenini arar. Bu yüzden SD karttaki `uEnv.txt`'de `bootcmd` tanımlamak o açılışı etkilemez; orada `uenvcmd` kullanılır (4.8'deki örnek gibi). Seri porttan yüklemede ise `boot` komutunu biz verdiğimiz için `bootcmd` çalışır.

6.4 Challenge: Boot İmajlarını MMC0'dan (SD Kart) Yüklemek

Görev

Kursun bu bölümdeki challenge'ı:

> *Derste boot imajları (uImage ve DTB) **MMC1**'den (eMMC) yüklendi, dosya sistemi olarak **MMC0**'daki (microSD) rootfs kullanıldı. `uEnv.txt`'yi, boot imajları da **MMC0**'dan (microSD) yüklenecek şekilde değiştir.*

	Önce	Sonra
Kernel + DTB	eMMC (`mmc 1`)	**SD kart (`mmc 0`)**
Rootfs	SD kart, 2. bölüm (`/dev/mmcblk0p2`)	Değişmez

Çözüm

Değişen tek şey `load` komutlarındaki **aygıt numarası** (ve dosyanın o aygıttaki yeri). `bootargs` değişmez, çünkü rootfs zaten SD kartta.

1) Dosyaları SD karta koy. `uImage` ve `am335x-boneblack.dtb`'yi SD kartın BOOT bölümüne kopyala:

```
cp uImage am335x-boneblack.dtb /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT /media/$USER/ROOTFS
```

2) `load` satırlarını `mmc 0:1`'e çevir.

```
eski:  load mmc 1:2 0x82000000 /boot/uImage                  (eMMC, 2. bölüm)
yeni:  load mmc 0:1 0x82000000 uImage                        (SD kart, 1. bölüm: BOOT)

eski:  load mmc 1:2 0x88000000 /boot/am335x-boneblack.dtb
yeni:  load mmc 0:1 0x88000000 am335x-boneblack.dtb
```

6.2'deki dosyanın `bootcmd` satırı tam olarak bunu yapıyor:

```
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000 uImage;load mmc
0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
```

3) Doğrula. U-Boot'u durdurup dosyaların orada olduğunu gör, sonra boot et:

```
U-Boot# ls mmc 0:1
U-Boot# boot
```

Ekranda `**********Booting from memory**********` yazısından sonra `reading uImage` görüyorsan imajlar SD karttan geliyor.

> **İpucu**
>
> SD kart dosyayı U-Boot'a FAT bölümünden de ext bölümünden de verebilir. `uImage` 'ı BOOT (FAT) bölümüne koymak, dosyayı Windows'tan bile kopyalayabilmeni sağlar. ROOTFS bölümündeki `/boot` klasöründe tutmak istersen `load mmc 0:2 ... /boot/uImage` yaz.

6.5 Ek Alıştırma: Kernel ve DTB SD Karttan, Rootfs eMMC'den

Görev

Challenge'ın tersi: kernel ve DTB SD karttan gelsin, ama rootfs eMMC'deki Debian olsun.

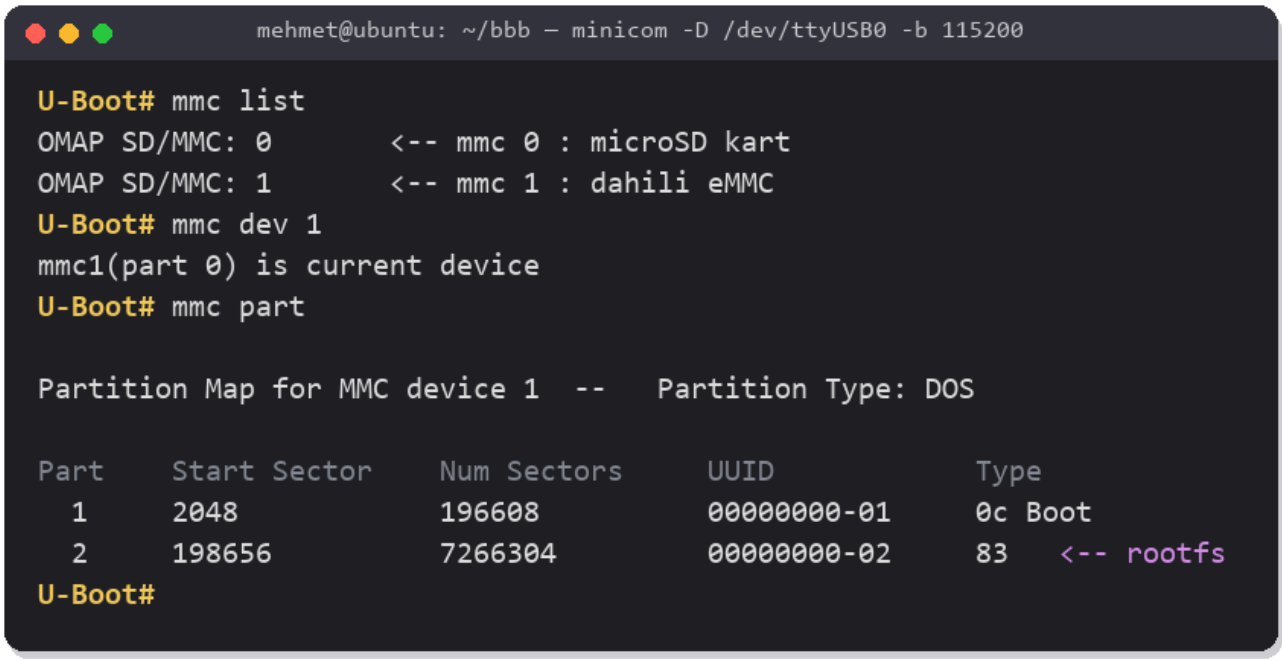
Ne?	Nereden?
Kernel (`uImage`)	**MMC0** — microSD kart, 1. bölüm
Device tree (`.dtb`)	**MMC0** — microSD kart, 1. bölüm
Root filesystem	**MMC1** — dahili eMMC, 2. bölüm

```
microSD (mmc 0)                      eMMC (mmc 1)
├── p1 BOOT  → uImage, .dtb  ──┐     ├── p1 BOOT
└── p2 ROOTFS                  │     └── p2 ROOTFS  ◄── root=/dev/mmcblk1p2
                               ▼
                    U-Boot RAM'e yükler → kernel başlar → rootfs'i eMMC'den bağlar
```

Adım 1 — Aygıt numaralarını doğrula

Önce varsayımımızı kontrol edelim: `mmc 0` gerçekten SD kart, `mmc 1` gerçekten eMMC mi ve eMMC'de 2. bölüm var mı?

```
U-Boot# mmc list
U-Boot# mmc dev 1
U-Boot# mmc part
```



eMMC'de `83` (Linux) tipinde 2. bölüm görünüyorsa rootfs için doğru yerdeyiz.

Adım 2 — Sadece `bootargs`'ı değiştir

Kernel ve DTB hâlâ SD karttan yükleneceği için `bootcmd` **aynı kalır**. Değişen tek şey, kernel'e "rootfs nerede?" bilgisini veren `bootargs`:

	Değer
Eski — rootfs SD kartta	`bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw`
Yeni — rootfs eMMC'de	`bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw`

Bunu iki yolla yapabilirsin:

- **a)** `uEnv.txt`'deki satırı değiştirip 6.3'teki gibi tekrar gönder.
- **b)** Doğrudan U-Boot'ta `setenv` ile değiştir:

```
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
**********Booting from memory**********
reading uImage
...
Starting kernel ...
```

Adım 3 — Linux'ta sonucu doğrula

Kart açılınca rootfs'in gerçekten eMMC'den bağlandığını kontrol et:

```
cat /proc/cmdline   # kernel'e giden bootargs
lsblk               # hangi bölüm / olarak bağlı?
findmnt /           # rootfs'in kaynağı
```

```
root@beaglebone: ~ (seri konsol)

root@beaglebone:~# cat /proc/cmdline
console=ttyO0,115200 root=/dev/mmcblk1p2 rw
root@beaglebone:~# lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
mmcblk0      179:0    0  7.4G  0 disk
|-mmcblk0p1  179:1    0   96M  0 part
`-mmcblk0p2  179:2    0  7.3G  0 part
mmcblk1      179:8    0  3.6G  0 disk
|-mmcblk1p1  179:9    0   96M  0 part
`-mmcblk1p2  179:10   0  3.5G  0 part /          <-- rootfs eMMC'de
root@beaglebone:~# findmnt /
TARGET SOURCE         FSTYPE OPTIONS
/      /dev/mmcblk1p2 ext4   rw,relatime
```

`/` bağlama noktası `mmcblk1p2`'nin yanında görünüyorsa challenge tamam.

Sonuç

Bileşen	Durum
Kernel ve DTB	microSD'nin 1. bölümünden RAM'e yüklenir.

Bileşen	Durum
Rootfs	Kernel açıldıktan sonra eMMC'nin 2. bölümünden bağlanır.
`bootcmd`	Değişmez; yükleme kaynağı hâlâ SD kart.
`bootargs`	`root=/dev/mmcblk1p2` olarak güncellenir.

> **Önemli**
>
> Bunun çalışması için:
>
> - eMMC'nin 2. bölümünde gerçekten bir Linux rootfs olmalı.
> - Kernel'de MMC sürücüsü ve rootfs'in dosya sistemi (ör. ext4) desteği **built-in** olmalı (veya bir initramfs'ten yüklenmeli).
> - Linux'taki numaralandırma U-Boot'unkiyle aynı olmalı. Değilse kernel `VFS: Unable to mount root fs` hatasıyla durur; bu durumda `root=/dev/mmcblk0p2` dene veya `root=PARTUUID=...` kullan.

6.6 ROOTFS Bölümünde Neler Var?

Root filesystem (rootfs), Linux'un `/` (kök) dizinidir. Kernel açıldıktan sonra `root=` ile gösterilen bölümü `/` olarak bağlar ve bundan sonra çalışan **her program, her ayar dosyası ve her kütüphane** buradan okunur. Kernel tek başına bir işe yaramaz; kullanıcıya kabuk (shell), komutlar ve servisler rootfs ile gelir.

Rootfs'teki klasör düzeni rastgele değildir; **FHS (Filesystem Hierarchy Standard)** denen ortak bir standarda uyar. Bu yüzden Ubuntu'da, BeagleBone'daki Ångström'de veya eMMC'deki Debian'da aynı klasörleri görürüz.

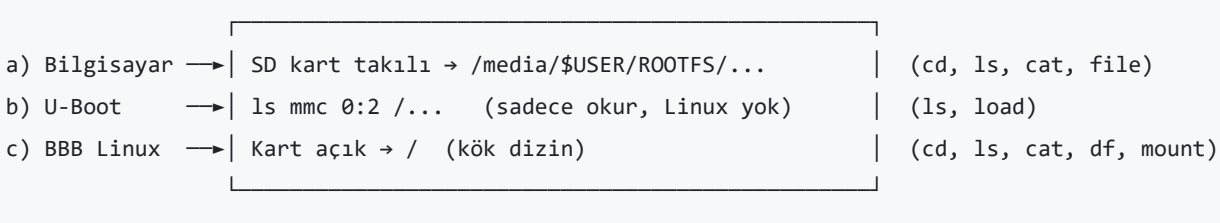
6.6.1 Klasörler ve görevleri

Klasör	İçinde ne var?	BBB'de örnek	Bakmak için
`/bin`	Temel kullanıcı komutları (`ls` , `cp` , `cat` , `sh` ...)	`bash` , `busybox`	`ls /bin`
`/sbin`	Sistem yönetimi komutları (genelde root için)	`init` , `mount` , `ifconfig` , `reboot`	`ls /sbin`
`/boot`	Kernel ve device tree dosyaları	`uImage` , `am335x-boneblack.dtb`	`ls -l /boot`
`/etc`	Sistem **ayar** dosyaları (metin)	`hostname` , `fstab` , `passwd` , `os-release`	`cat /etc/hostname`
`/lib`	Paylaşılan kütüphaneler (`.so`) ve kernel modülleri	`libc.so.6` , `modules/3.8.13/`	`ls /lib/modules`

Klasör	İçinde ne var?	BBB'de örnek	Bakmak için
`/usr`	Ek programlar, kütüphaneler, belgeler (rootfs'in en büyük kısmı)	`/usr/bin`, `/usr/lib`	`du -sh /usr`
`/home`	Kullanıcıların kişisel klasörleri	`/home/root` (Ångström'de root'un ev dizini)	`ls /home`
`/root`	root kullanıcısının ev dizini (Debian/Ubuntu'da)	eMMC'deki Debian'da kullanılır	`ls /root`
`/var`	Zamanla değişen veriler: loglar, kuyruklar, önbellek	`/var/log`	`ls /var/log`
`/tmp`	Geçici dosyalar (açılışta temizlenir, genelde RAM'de)		`ls /tmp`
`/media`, `/mnt`	Harici diskleri (USB, SD) bağlama noktaları	`mount /dev/mmcblk1p1 /mnt`	`ls /mnt`
`/dev` *(sanal)*	Donanım aygıt dosyaları	`mmcblk0p2`, `ttyO0`	`ls /dev`
`/proc` *(sanal)*	Kernel ve süreç bilgileri	`cpuinfo`, `meminfo`, `cmdline`	`cat /proc/cpuinfo`
`/sys` *(sanal)*	Aygıt ve sürücü ayarları (sysfs)	`/sys/class/leds/...`	`ls /sys/class`
`/run` *(sanal)*	Çalışma anı bilgileri (PID dosyaları, soketler)		`ls /run`

> **Önemli**
>
> *(sanal)* işaretli klasörler **sanal (virtual)** dosya sistemleridir. Diskte gerçek dosya içermezler; kernel çalışırken içlerini **RAM'de** doldurur (`devtmpfs`, `proc`, `sysfs`, `tmpfs`). Bu yüzden SD kartı bilgisayara takıp baktığında bu klasörler **boş** görünür. Bu bir hata değildir.

6.6.2 ROOTFS'e üç yerden ulaşabiliriz



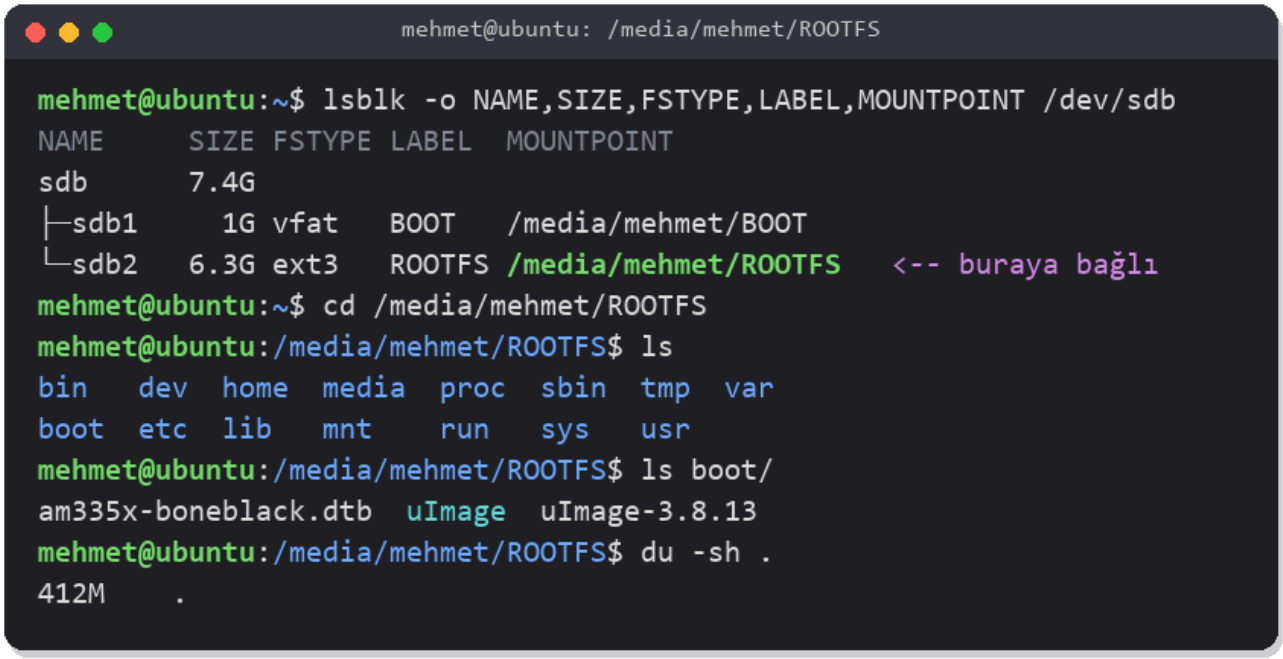
Nereden?	Ne zaman?	Rootfs'in yolu
a) Bilgisayar	SD kart okuyucuya takılıyken, kartı açmadan dosya kopyalamak / düzeltmek için	`/media/$USER/ROOTFS`

Nereden?	Ne zaman?	Rootfs'in yolu
b) U-Boot	Linux açılmadan önce, dosya gerçekten orada mı diye bakmak için	`mmc 0:2` (SD) veya `mmc 1:2` (eMMC)
c) BBB Linux	Kart açıldıktan sonra, seri konsol (Minicom) veya SSH üzerinden	`/`

6.6.3 a) Bilgisayardan ulaşmak

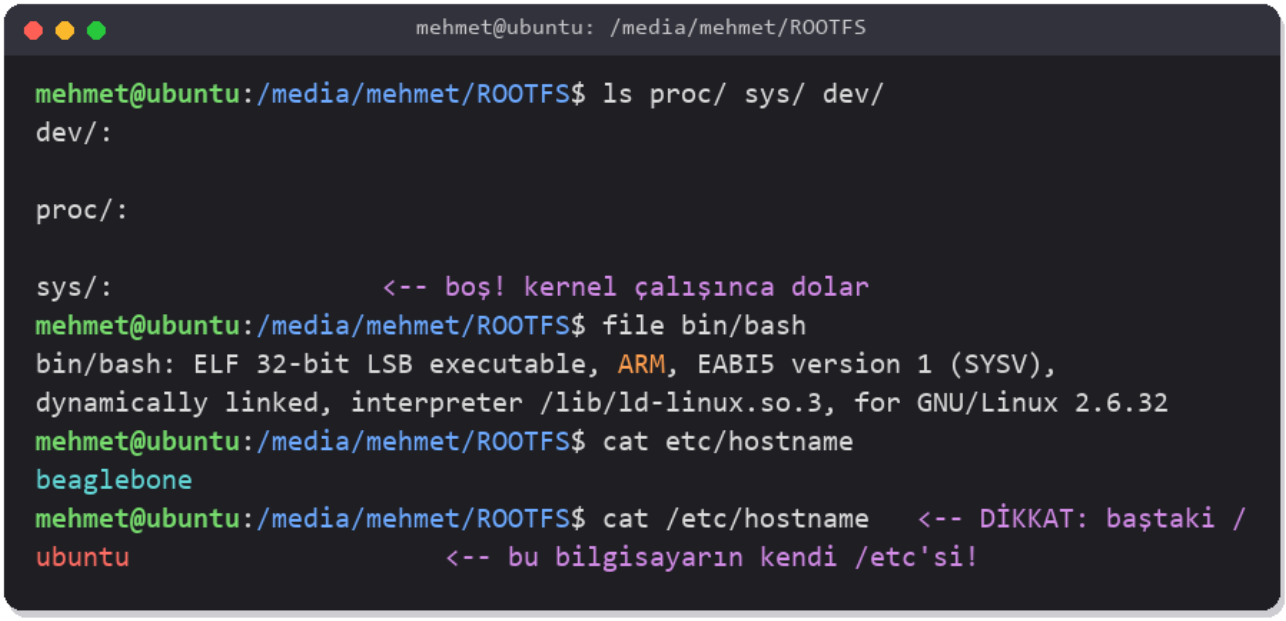
SD kartı bilgisayara tak. Ubuntu bölümleri **etiket (label)** adlarıyla otomatik bağlar. Önce kartın nereye bağlandığını bul, sonra klasöre gir:

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT   # SD kart hangi aygıt, nereye bağlı?
cd /media/$USER/ROOTFS                       # rootfs'in içine gir
ls                                           # kök klasörleri listele
ls boot/                                     # kernel ve dtb burada
du -sh .                                     # rootfs'in toplam boyutu
```



Burada iki şeyi gözlemle: sanal klasörler boş, programlar ise bilgisayarın değil **ARM** işlemcinin komut setiyle derlenmiş:

```
ls proc/ sys/ dev/      # boş görünür
file bin/bash           # "ARM" yazısını gör
cat etc/hostname        # BBB'nin hostname'i (başta / YOK)
```



> **Uyarı**
>
> Bilgisayardayken yolun başına `/` **koyma**. `cat /etc/hostname` bilgisayarın **kendi** dosyasını okur; BBB'ninki `etc/hostname` (ROOTFS klasöründeyken) veya `/media/$USER/ROOTFS/etc/hostname` yolundadır. Bu karışıklıkla bilgisayarın `/etc/fstab` gibi dosyalarını yanlışlıkla bozmak mümkündür. Rootfs'e yazmak için `sudo` gerekir ve işin bitince kartı çıkarmadan önce `sync` + `umount` yap.
>
> `file bin/bash` çıktısında `ARM` yazdığı için bu programları bilgisayarda **çalıştıramazsın** (`./bin/bash` → `Exec format error`). Sadece okuyabilir/kopyalayabilirsin. Bunun sebebini Bölüm 9'da (cross compile) göreceğiz.

6.6.4 b) U-Boot'tan ulaşmak

U-Boot dosya sistemini **okuyabilir** (ext2/3/4 ve FAT). Linux açılmadan önce kernel'in gerçekten orada olup olmadığını görmek için kullanışlıdır:

```
U-Boot# ls mmc 0:2 /          # SD kartın 2. bölümü (ROOTFS) kökü
U-Boot# ls mmc 0:2 /boot      # kernel ve dtb
U-Boot# ls mmc 1:2 /          # eMMC'nin 2. bölümü (Debian rootfs)
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# ls mmc 0:2 /
<DIR>       4096 .
<DIR>       4096 ..
<DIR>       4096 bin
<DIR>       4096 boot
<DIR>       4096 dev
<DIR>       4096 etc
<DIR>       4096 home
<DIR>       4096 lib
   ...
<DIR>       4096 usr
<DIR>       4096 var
U-Boot# ls mmc 0:2 /boot
            24884 am335x-boneblack.dtb
          4380760 uImage
U-Boot# ls mmc 1:2 /          # eMMC'deki rootfs (Debian)
<DIR>       4096 bin
   ...
```

Komut	Anlamı
`ls mmc <aygıt>:<bölüm> <yol>`	Bölümdeki klasörü listeler
`load mmc 0:2 0x82000000 /boot/uImage`	Dosyayı RAM'e yükler (4.10.2)
`ext4ls` / `ext2ls` / `fatls`	Eski U-Boot sürümlerinde dosya sistemine özel `ls` komutları

> **Not**
>
> U-Boot'ta `cd` yoktur ve dosya içeriğini `cat` ile göremezsin; her komutta tam yolu yazarsın. U-Boot rootfs'e normalde **yazmaz**, sadece okur.

6.6.5 c) BBB Linux'tan ulaşmak

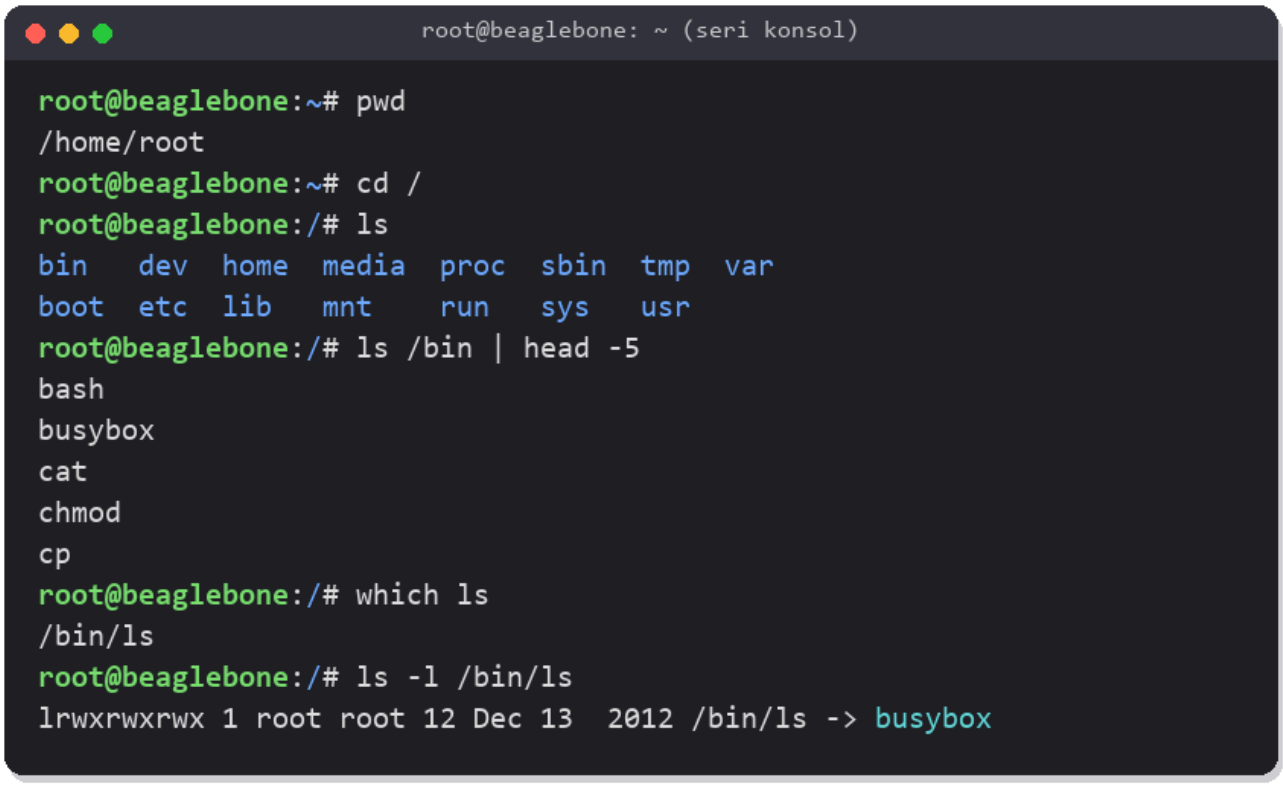
Kart açılıp `root` olarak giriş yaptıktan sonra rootfs artık `/` dizinidir. Dolaşmak için temel komutlar:

Komut	Görevi
`pwd`	Şu an hangi klasördeyim?
`cd /` · `cd /etc` · `cd ..` · `cd ~`	Kök dizine / bir klasöre / bir üste / ev dizinine git
`ls` · `ls -l` · `ls -a`	Listele / ayrıntılı (izin, boyut, tarih) / gizli dosyalarla
`cat <dosya>` · `head -n 5` · `less`	Metin dosyasını göster / ilk satırları / sayfa sayfa

Komut	Görevi
`which <komut>`	Bir komutun hangi dosya olduğunu bul
`find / -name "*.dtb" 2>/dev/null`	Rootfs'te dosya ara
`df -h /` · `du -sh <klasör>`	Rootfs'in doluluk oranı / klasör boyutu
`mount`	Hangi bölüm nereye bağlı?

Kök dizin ve `/bin`

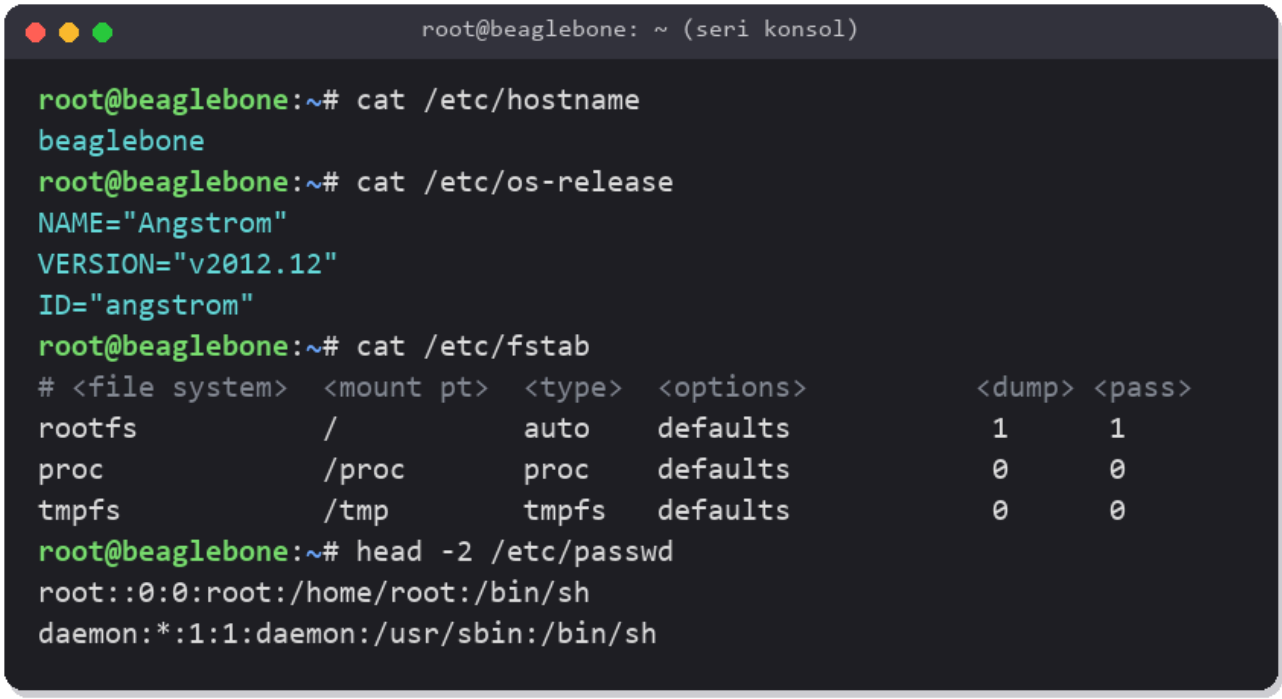
```
pwd                # /home/root  (Ångström'de root'un ev dizini)
cd /
ls
ls /bin | head -5
which ls
ls -l /bin/ls      # Ångström'de birçok komut busybox'a link
```



> **İpucu**
>
> **BusyBox**, `ls`, `cp`, `mount` gibi yüzlerce komutu **tek bir küçük programda** toplayan araçtır. Gömülü sistemlerde yer kazanmak için yaygın kullanılır; `/bin/ls` gibi dosyalar aslında `busybox`'a giden **sembolik linklerdir**.

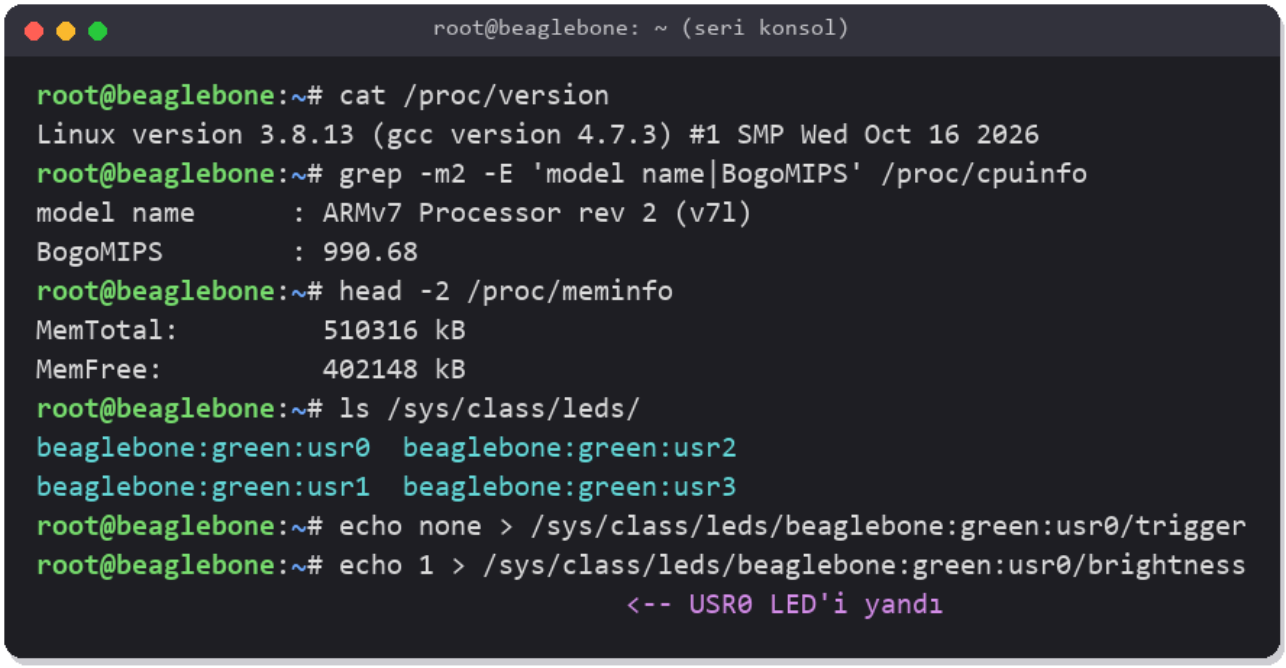
`/etc` — ayar dosyaları

```
cat /etc/hostname      # kartın adı
cat /etc/os-release    # hangi dağıtım, hangi sürüm
cat /etc/fstab         # açılışta hangi bölüm nereye bağlanacak
head -2 /etc/passwd    # kullanıcı listesi
```



`/proc` ve `/sys` — kernel ile konuşmak

```
cat /proc/version                                  # kernel sürümü
cat /proc/cmdline                                  # U-Boot'un verdiği bootargs
grep -m2 -E 'model name|BogoMIPS' /proc/cpuinfo    # işlemci bilgisi
head -2 /proc/meminfo                              # RAM bilgisi
ls /sys/class/leds/                                # kartın USR LED'leri
echo none > /sys/class/leds/beaglebone:green:usr0/trigger
echo 1    > /sys/class/leds/beaglebone:green:usr0/brightness   # LED'i yak (0 = söndür)
```



`/sys` altındaki dosyalara yazmak, sürücüye doğrudan komut vermek demektir. `trigger` dosyasını `none` yapmadan `brightness`'a yazarsan LED'in varsayılan görevi (ör. `heartbeat`) senin değerini hemen ezer.

`/dev`, `/boot` ve disk kullanımı

```
ls /dev/mmcblk* /dev/ttyO*   # MMC bölümleri ve seri portlar
ls -l /boot                  # kernel ve dtb
df -h /                      # rootfs ne kadar dolu, hangi bölümden geliyor
du -sh /usr /lib /etc        # en çok yer kaplayan klasörler
mount | grep -E ' / | /proc | /sys | /dev '   # gerçek ve sanal bağlamalar
```

```
root@beaglebone: ~ (seri konsol)
root@beaglebone:~# ls /dev/mmcblk* /dev/ttyO*
/dev/mmcblk0    /dev/mmcblk0p2  /dev/mmcblk1p1  /dev/ttyO0
/dev/mmcblk0p1  /dev/mmcblk1    /dev/mmcblk1p2
root@beaglebone:~# ls -l /boot
-rw-r--r-- 1 root root   24884 Dec 13  2012 am335x-boneblack.dtb
lrwxrwxrwx 1 root root      14 Dec 13  2012 uImage -> uImage-3.8.13
-rw-r--r-- 1 root root 4380760 Dec 13  2012 uImage-3.8.13
root@beaglebone:~# df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/mmcblk0p2  6.2G  412M  5.5G   7% /
root@beaglebone:~# du -sh /usr /lib /etc 2>/dev/null
287M    /usr
61M     /lib
3.1M    /etc
root@beaglebone:~# mount | grep -E ' / | /proc | /sys | /dev '
/dev/mmcblk0p2 on / type ext3 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,relatime)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
```

`mount` çıktısında `/` gerçek bir diskten (`/dev/mmcblk0p2` , `ext3`) gelirken `/dev` , `/proc` ve `/sys` 'in tipi `devtmpfs` , `proc` , `sysfs` olarak görünür. Bu, 6.6.1'deki sanal klasörlerin kanıtıdır. Rootfs'i eMMC'den açtıysan (6.5) `df -h /` satırında `/dev/mmcblk1p2` görürsün.

6.7 Özet

Komut / Değişken	Görevi
`uEnv.txt`	U-Boot ortam değişkenlerini metin olarak tanımlar
`loady <adres>`	YMODEM ile seri porttan RAM'e dosya alır, boyutu `filesize` 'a yazar
`env import -t <adres> ${filesize}`	RAM'deki metin değişkenlerini aktif ortama aktarır
`printenv <isim>`	Değişkenin değerini gösterir
`setenv <isim> <değer>`	Değişkeni değiştirir (geçici)
`run <makro>`	Değişkendeki komutları çalıştırır (ör. `run mypcip`)
`boot`	`bootcmd` 'yi çalıştırır
`bootcmd`	Otomatik açılışta çalışan komut dizisi
`bootargs`	Linux kernel'ine verilen açılış parametreleri
`root=/dev/mmcblk1p2`	Rootfs'in eMMC'nin 2. bölümünde olduğunu söyler

Komut / Değişken	Görevi
`mmc list` / `mmc part`	U-Boot'un gördüğü MMC aygıtlarını ve bölümleri listeler
`saveenv`	Aktif ortamı kalıcı olarak kaydeder (dikkatli kullan)
`/media/$USER/ROOTFS`	SD kart bilgisayara takılıyken rootfs'in yolu
`ls mmc 0:2 /boot`	U-Boot'tan rootfs'teki klasörü listeler
`/bin`, `/sbin`, `/usr`	Programlar
`/etc`	Ayar dosyaları (`hostname`, `fstab`, `passwd`)
`/boot`	Kernel (`uImage`) ve device tree (`.dtb`)
`/lib`	Kütüphaneler ve kernel modülleri
`/dev`, `/proc`, `/sys`	Sanal klasörler; kernel çalışırken dolar
`cat /proc/cmdline`	Kernel'e giden bootargs
`df -h /` · `mount`	Rootfs hangi bölümden geliyor?

6.8 Karşılaşılabilecek Problemler

Belirti	Sebep	Çözüm
`loady` sonrası transfer başlamıyor	Minicom'da xmodem seçildi ya da `lrzsz` yok	Protokol **ymodem** olmalı; `sudo apt install lrzsz`
`env import` sonrası değişkenler bozuk / fazladan satırlar	Boyut ondalık yazıldı	`${filesize}` kullan ya da boyutu hex yaz
`## Error: "mypcip" not defined`	`env import` yapılmadı ya da kart yeniden açıldı	Adım 3–4'ü tekrarla; değişiklikler kalıcı değildir
`VFS: Unable to mount root fs` (challenge)	Linux'ta eMMC'nin numarası farklı	`root=/dev/mmcblk0p2` dene; Linux'ta `lsblk` ile kontrol et
Açılışta `uEnv.txt`'deki `bootcmd` çalışmıyor	Otomatik okumada `bootcmd` değil `uenvcmd` çalıştırılır	SD karttaki dosyada komutları `uenvcmd`'ye yaz

Bölüm 7 — Seri Porttan (UART) Boot

> **Bu bölümde:**
>
> - BeagleBone Black'i **SD kart ve eMMC kullanmadan**, bütün boot dosyalarını bilgisayardan **seri port (UART0)** üzerinden göndererek açacaksın.
> - Xmodem ve Ymodem protokollerinin farkını ve ROM Code'un UART boot davranışını öğreneceksin.
> - Kernel, DTB ve initramfs'i RAM'de nereye yükleyeceğini ve `bootm` ile nasıl başlatacağını göreceksin.
>
> **Ön koşul:** Bölüm 2 — Minicom, Bölüm 3 — boot sırası, Bölüm 6 — `loady` ile dosya gönderme.

7.1 Dosya Transfer Protokolleri

Görsel: kursun slaytı.

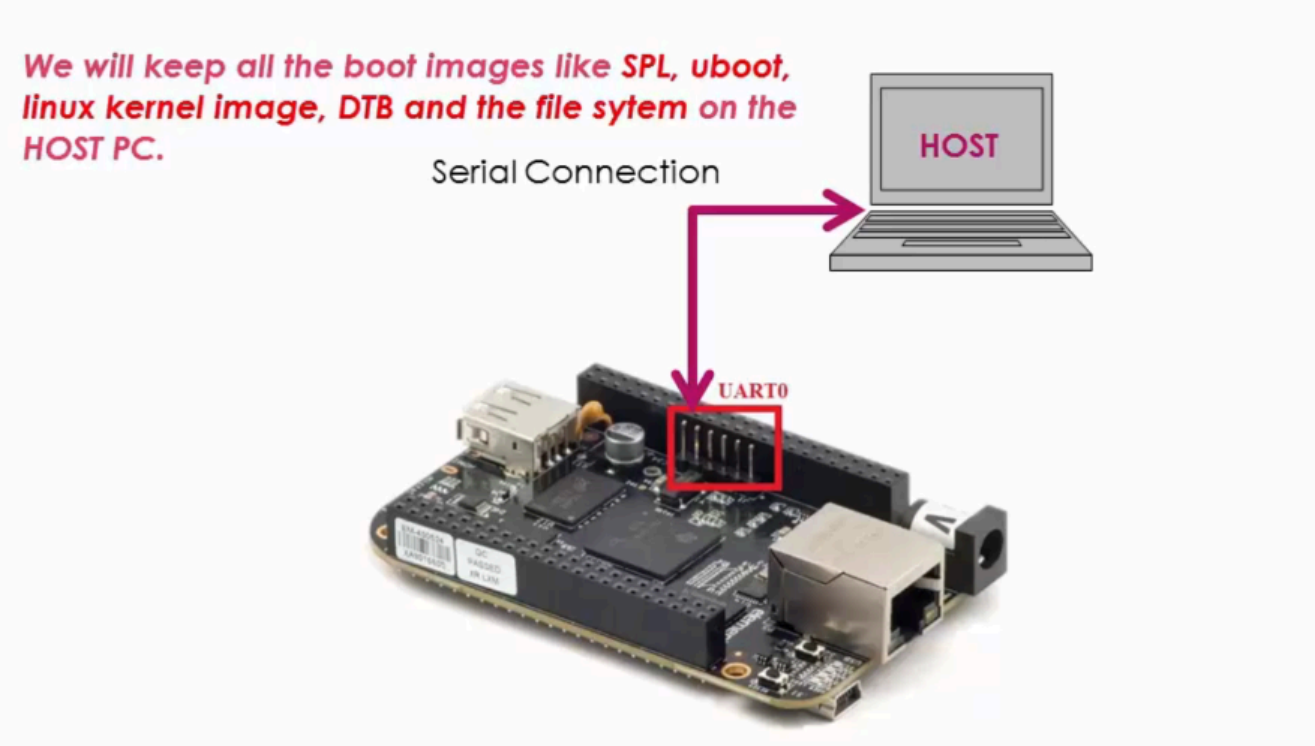
Bilgisayardan karta dosya göndermenin iki yolu var:

Bağlantı	Protokoller	Ne zaman kullanılır?
IP ağı (Ethernet)	TFTP, HTTP, FTP, SMTP...	Kartta çalışan bir ağ yığını (U-Boot veya Linux) varsa. Hızlıdır (Bölüm 8).
UART (seri port)	Xmodem, Ymodem, Zmodem, Kermit...	Ağ yokken veya kartta henüz hiçbir yazılım yokken. Yavaştır ama çok basittir.

UART sadece ham baytları taşır; "dosya nerede başlıyor, nerede bitiyor, veri bozuldu mu?" gibi bilgileri taşımaz. **Xmodem / Ymodem** gibi protokoller veriyi paketlere bölüp her pakete kontrol bilgisi (checksum / CRC) ekler, bozuk paketi tekrar istetir.

Protokol	Özellik
Xmodem	128 bayt (Xmodem-1K'da 1024 bayt) paketler. Dosya adı ve boyutu göndermez. AM335x'in **ROM kodu** bunu kullanır.
Ymodem	Xmodem'in geliştirilmiş hali: dosya adını ve boyutunu da gönderir. U-Boot'ta `loady` komutu.

7.2 Seri Boot Fikri



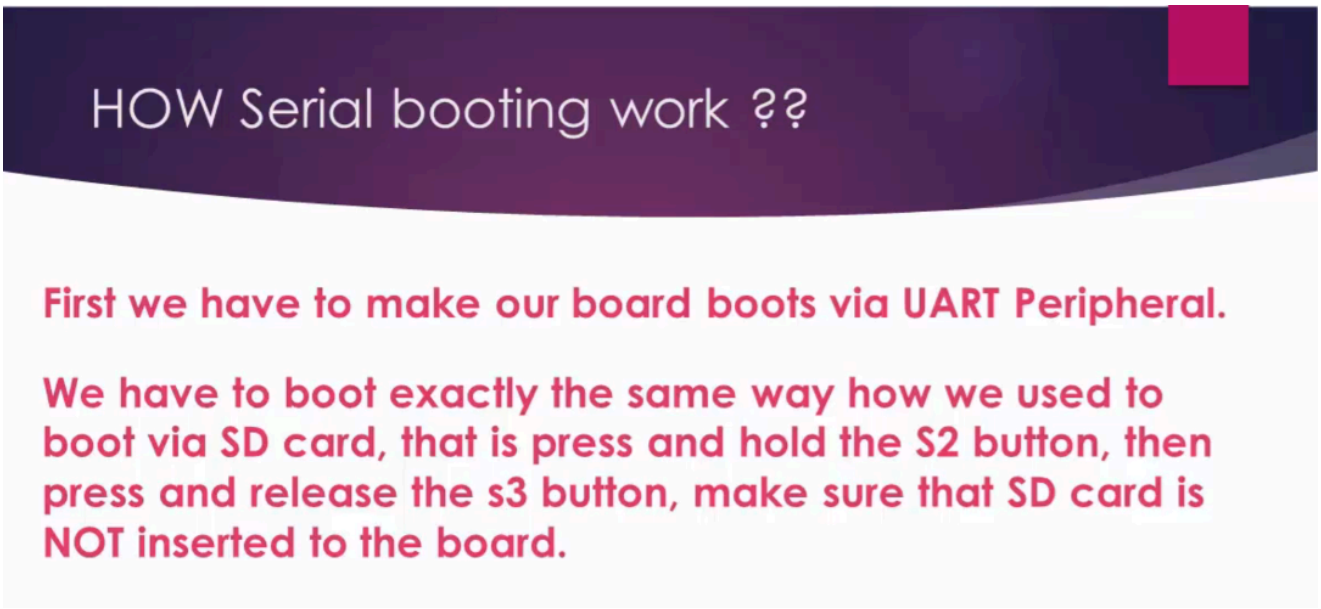
Görsel: kursun slaytı.

Bütün boot dosyaları — **SPL, U-Boot, Linux kernel (uImage), DTB ve dosya sistemi** — kartta değil, **bilgisayarda (HOST)** durur. Kart ile bilgisayar arasındaki tek bağlantı, J1 header'daki **UART0** pinlerine bağlı USB-TTL seri kablodur. Kart açılırken her dosyayı bu kablo üzerinden sırayla göndeririz.

Bu yöntem şu durumlarda işe yarar:

- SD kart ve eMMC boşsa veya içindeki bootloader bozulduysa (kartı "kurtarmak" için).
- Yeni derlenmiş bir SPL / U-Boot'u karta yazmadan denemek için.

7.3 Kartı UART Boot Moduna Sokmak



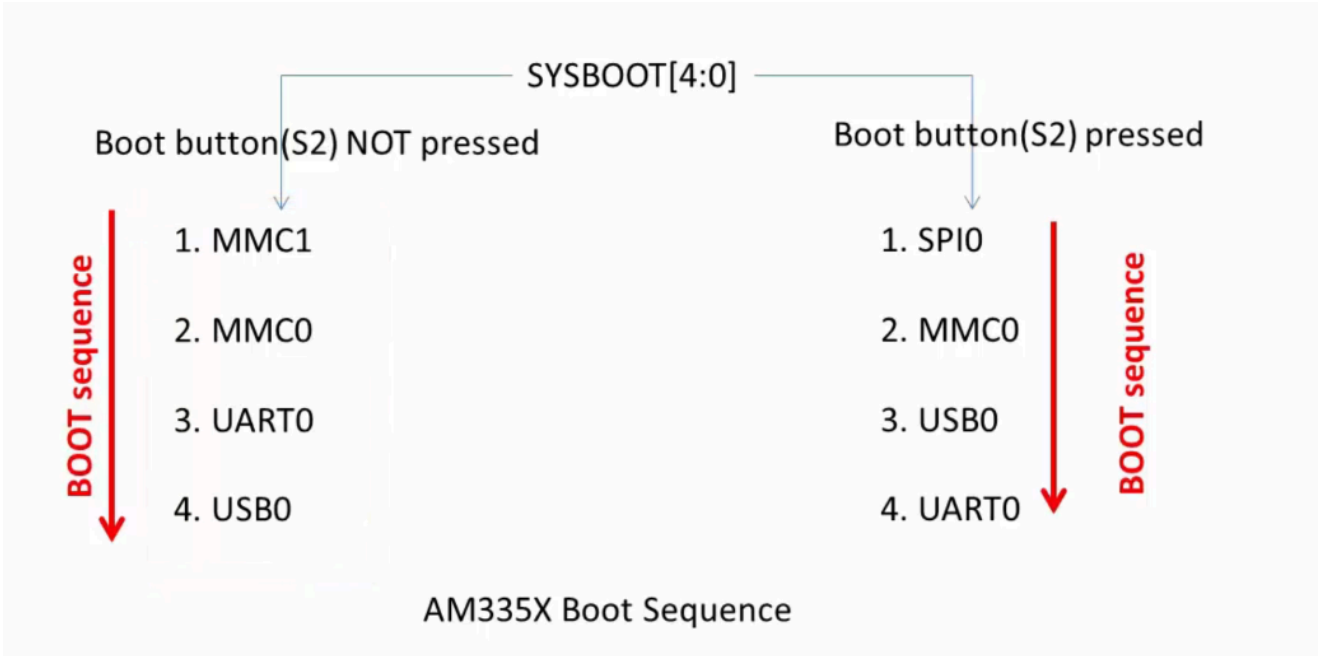
Görsel: kursun slaytı.

Kartı UART'tan boot ettirmek için SD karttan boot ederken yaptığımızın aynısını yaparız:

1. **S2 (BOOT)** butonuna basılı tut,
2. **S3 (POWER)** butonuna basıp bırak (veya beslemeyi tak),
3. Boot başladıktan sonra S2'yi bırak.

Tek fark: **SD kart takılı olmamalı.** Takılıysa ROM kodu boot dosyalarını SD karttan bulur ve UART'a hiç sıra gelmez.

AM335x boot sırası



AM335x'in içindeki ROM kodu, açılışta `SYSBOOT[4:0]` pinlerine bakarak hangi çevre birimlerini hangi sırayla deneyeceğine karar verir (3.5). BBB'de S2 butonu bu pinlerden birinin değerini değiştirir:

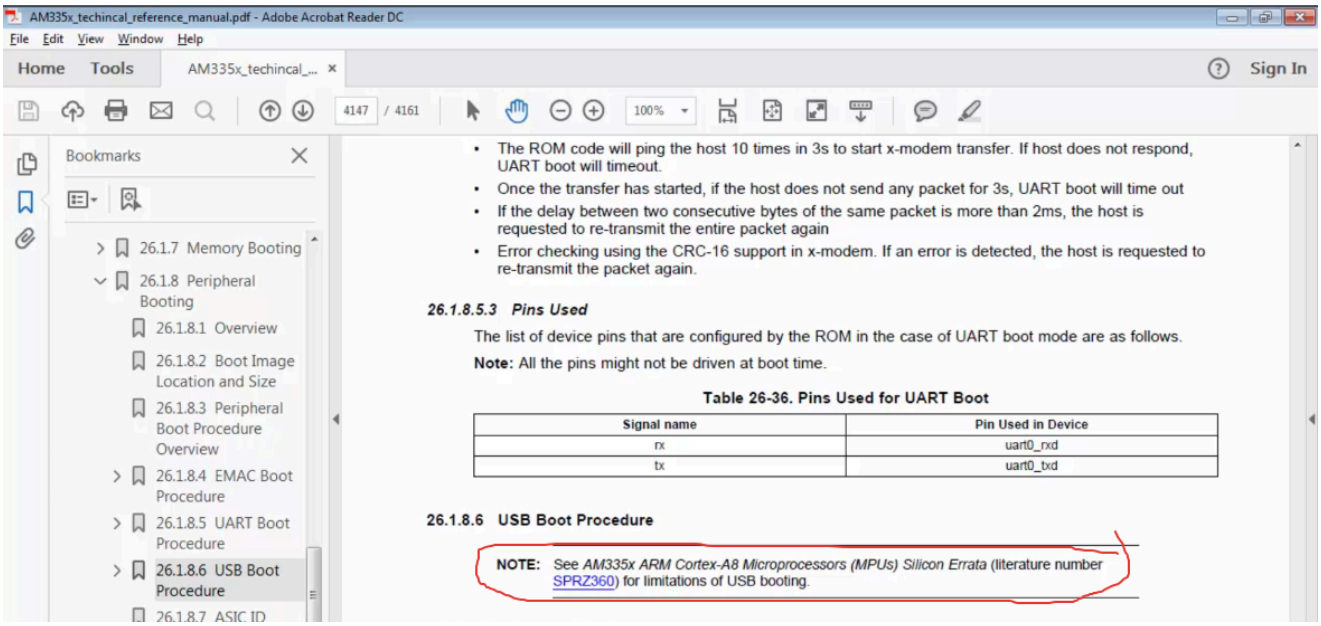
Sıra	S2 basılı **değil**	S2 **basılı**
1	MMC1 (eMMC)	SPI0
2	MMC0 (SD kart)	MMC0 (SD kart)
3	UART0	**USB0**
4	USB0	**UART0**

S2 basılı bir şekilde boot işlemi yaparsak ROM kodu sırayla her çevre biriminde geçerli bir boot image'ı olup olmadığını kontrol eder:

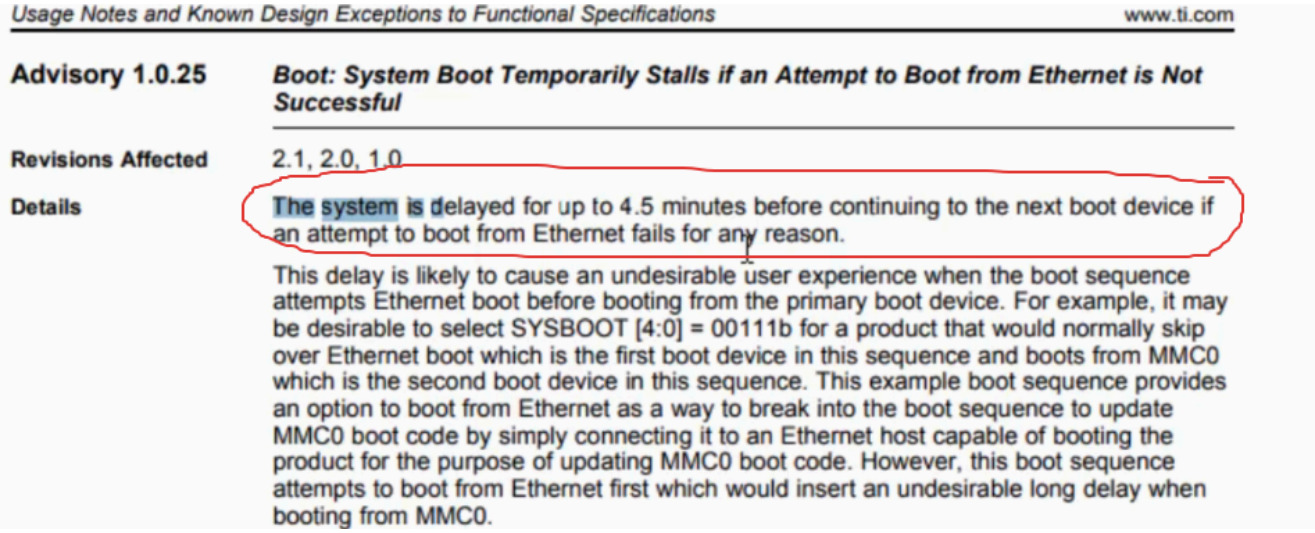
- **SPI0**: BBB'de SPI flash yok → hemen geçer.
- **MMC0**: SD kart takılı değil → hemen geçer.
- **USB0**: Burada farklı bir durum oluşuyor (aşağıda).
- **UART0**: Bizim istediğimiz yer.

USB0 sorunu: 4.5 dakikalık bekleme

Eğer USB kablosu bu sırada karta takılıysa ve image dosyası USB üzerinden gönderilmiyorsa, ROM kodu bir sonraki boot aygıtı olan UART0'a geçmeden önce **4.5 dakikaya kadar** bekliyor. Bu süre dolduktan sonra UART0'a geçiyor.



AM335x Technical Reference Manual'ın (TRM) **26.1.8.6 USB Boot Procedure** bölümünde, USB boot'un kısıtlamaları için **Silicon Errata (SPRZ360)** belgesine bakılması söyleniyor:



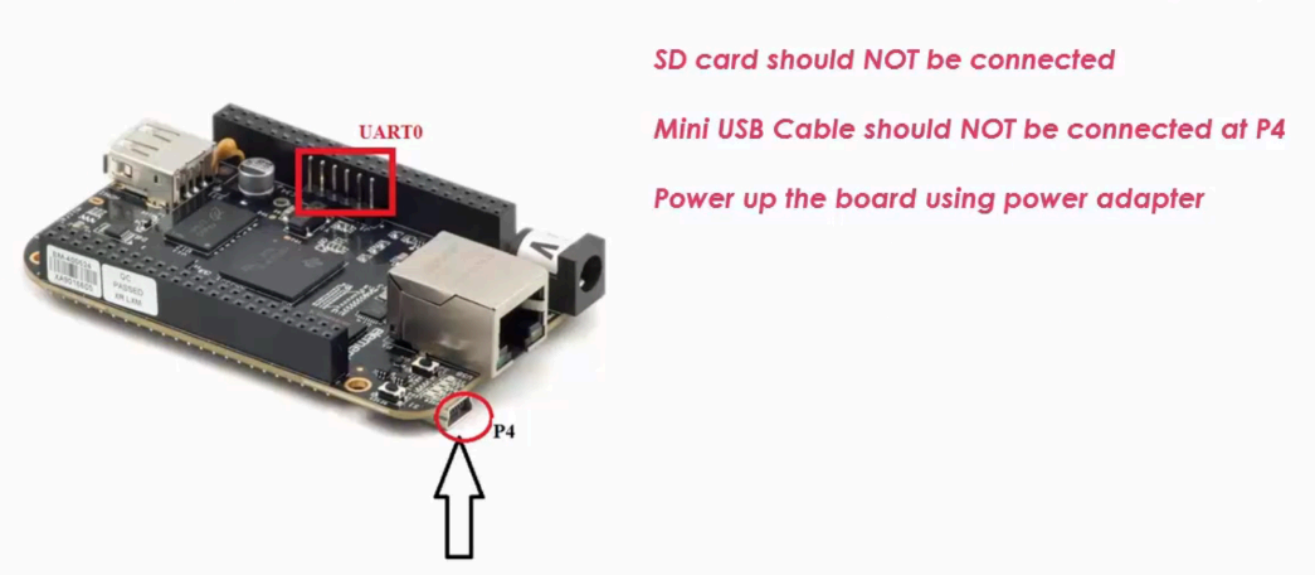
Usage Notes and Known Design Exceptions to Functional Specifications www.ti.com

Advisory 1.0.25	***Boot: System Boot Temporarily Stalls if an Attempt to Boot from Ethernet is Not Successful***
Revisions Affected	2.1, 2.0, 1.0
Details	The system is delayed for up to 4.5 minutes before continuing to the next boot device if an attempt to boot from Ethernet fails for any reason. This delay is likely to cause an undesirable user experience when the boot sequence attempts Ethernet boot before booting from the primary boot device. For example, it may be desirable to select SYSBOOT [4:0] = 00111b for a product that would normally skip over Ethernet boot which is the first boot device in this sequence and boots from MMC0 which is the second boot device in this sequence. This example boot sequence provides an option to boot from Ethernet as a way to break into the boot sequence to update MMC0 boot code by simply connecting it to an Ethernet host capable of booting the product for the purpose of updating MMC0 boot code. However, this boot sequence attempts to boot from Ethernet first which would insert an undesirable long delay when booting from MMC0.

Görsel: TI, AM335x Sitara Processors Silicon Errata (SPRZ360).

Errata'daki **Advisory 1.0.25**'te Ethernet denilmiş, ancak bu durum USB için de geçerli. Çünkü AM335x'in ROM kodu USB boot'u, USB kablosu üzerinden **sanal bir Ethernet bağlantısı (RNDIS)** kurarak yapar. Yani USB boot aslında "USB üzerinden Ethernet boot"tur ve aynı 4.5 dakikalık beklemeye takılır.

Çözüm: Kartı harici adaptörle beslemek



Görsel: kursun slaytı.

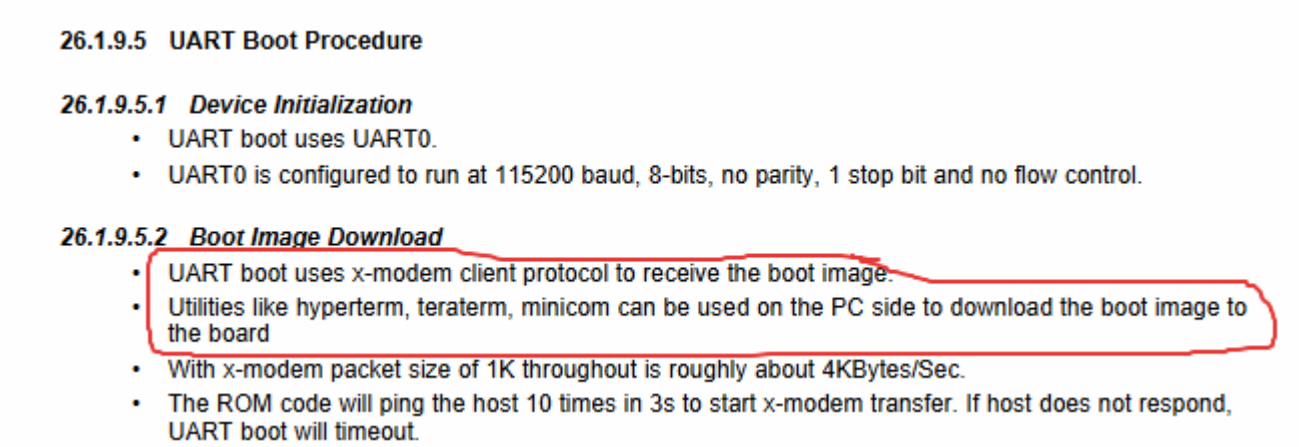
Peki bu bekleme süresinden nasıl kurtulabiliriz? Beslemeyi **P4 mini USB portundan değil**, harici bir **5V - 2A adaptör** yardımıyla (DC jack) yapacağız. USB kablosu takılı olmayınca ROM kodu USB0'da hiçbir şey bulamaz, hiç beklemeden UART0 sekansına geçer ve image yüklemesini oradan yaparız.

Koşul	Neden?
SD kart takılı olmamalı	ROM, UART0'dan önce MMC0'ı dener; SD kartta MLO varsa oradan açılır.
P4 (mini USB) kablosu takılı olmamalı	Takılıysa USB0 boot denenir ve 4.5 dakika beklenir.

Koşul	Neden?
Kart DC jack'ten 5V adaptörle beslenmeli	USB kablosu çıkınca kartın beslemesi başka yerden gelmeli.
UART0 seri kablosu bağlı olmalı	Bütün dosyalar buradan gidecek.

> **Uyarı**
>
> Seri kablonun (USB-TTL) **VCC / 5V** ucunu karta bağlama. J1 header'da sadece **GND, RX ve TX** bağlanır; kart zaten adaptörden besleniyor.

7.4 ROM Kodunun UART Boot Davranışı



26.1.9.5 UART Boot Procedure

26.1.9.5.1 Device Initialization

- UART boot uses UART0.
- UART0 is configured to run at 115200 baud, 8-bits, no parity, 1 stop bit and no flow control.

26.1.9.5.2 Boot Image Download

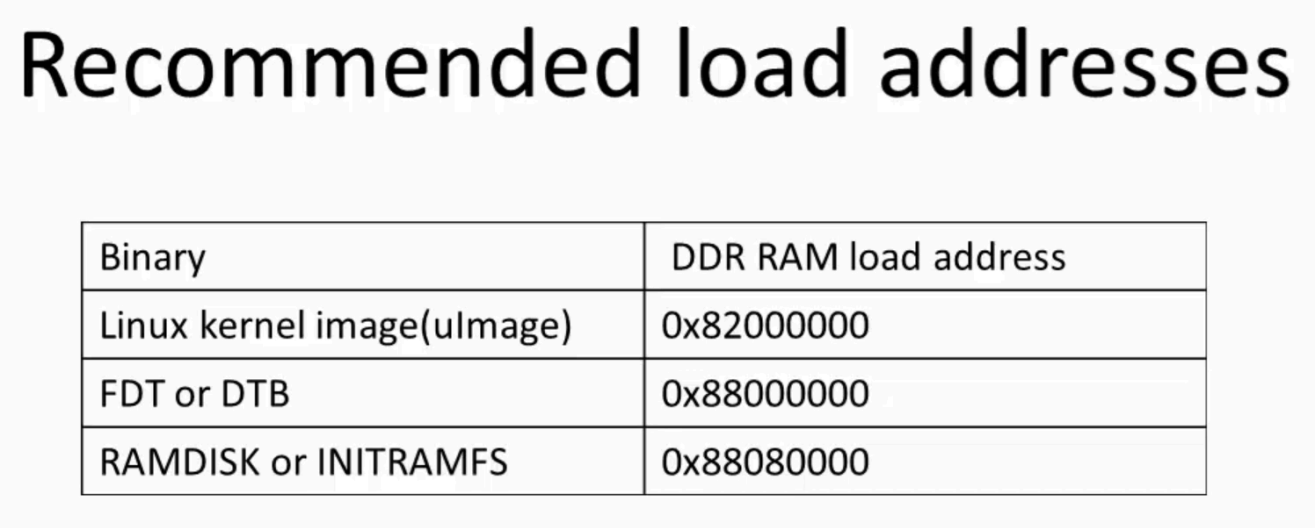
- UART boot uses x-modem client protocol to receive the boot image.
- Utilities like hyperterm, teraterm, minicom can be used on the PC side to download the boot image to the board
- With x-modem packet size of 1K throughout is roughly about 4KBytes/Sec.
- The ROM code will ping the host 10 times in 3s to start x-modem transfer. If host does not respond, UART boot will timeout.

TRM'nin **26.1.9.5 UART Boot Procedure** bölümüne göre:

Konu	Değer
Kullanılan port	**UART0** (pinler: `uart0_rxd` , `uart0_txd`)
Ayarlar	**115200 baud, 8 bit, parity yok, 1 stop bit, flow control yok** (115200 8N1)
Protokol	**Xmodem** (ROM kodu Xmodem istemcisi olarak dosyayı alır)
Bilgisayar tarafı	HyperTerminal, Tera Term, **Minicom** gibi programlar kullanılabilir
Hız	1K paketlerle yaklaşık **4 KB/s**
Zaman aşımı	ROM, 3 saniye içinde 10 kez bilgisayarı "dürter" (ekrana `C` karakteri basar). Cevap gelmezse veya transfer başladıktan sonra 3 saniye paket gelmezse UART boot iptal olur.
Hata kontrolü	Xmodem'in CRC-16 desteği; bozuk paket tekrar istenir

> **İpucu**
>
> Minicom ekranında art arda `cccc...` görmek, kartın UART boot moduna girip **dosya beklediği** anlamına gelir. 3 saniyelik süre kısa olduğu için Minicom'da gönderme menüsünü (`Ctrl-A → S → xmodem`) önceden hazırla.

7.5 Önerilen RAM Yükleme Adresleri



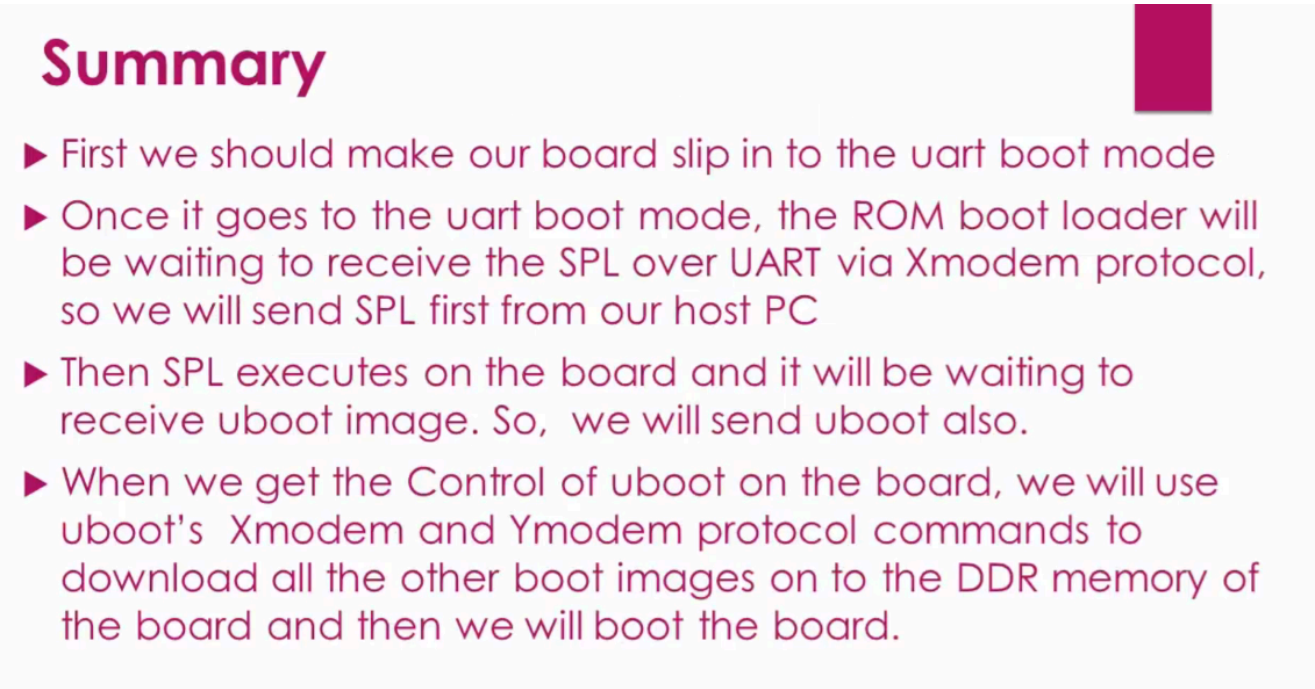
Görsel: kursun slaytı.

U-Boot'a geldikten sonra kernel, DTB ve dosya sistemini RAM'e (DDR, `0x80000000`'dan başlar) yükleyeceğiz. Dosyaların birbirinin üzerine yazmaması için bu adresler kullanılır:

Dosya	Adres
Kernel (`uImage`)	`0x82000000`
Device tree (`.dtb`)	`0x88000000`
initramfs	`0x88080000`

Seri boot'ta SD kart ve eMMC kullanılmadığı için rootfs'i bağlayacak bir disk yoktur. Bu yüzden dosya sistemi de RAM'e yüklenir: **RAMDISK / INITRAMFS**, RAM'de duran küçük bir rootfs'tir. DTB'ye ayrılan alan `0x88000000` – `0x8807FFFF` arası (512 KB) olduğu için initramfs hemen arkasından, `0x88080000`'dan başlar.

7.6 Seri Boot Adımlarına Genel Bakış



Görsel: kursun slaytı.

```
Bilgisayar (HOST)                          BeagleBone Black
─────────────────                          ──────────────────────────────────────────

                                           ROM kodu (S2 basılı, SD kart yok) → UART0
u-boot-spl.bin ─── Xmodem ──────────────►  SPL çalışır (DDR'ı hazırlar)
u-boot.img     ─── Ymodem ──────────────►  U-Boot çalışır → => komut satırı
uImage         ─── loadx / loady ───────►  0x82000000
.dtb           ─── loadx / loady ───────►  0x88000000
initramfs      ─── loadx / loady ───────►  0x88080000
                                           bootm → Linux açılır
```

1. Önce kartı **UART boot moduna** sokarız (SD kart yok, USB kablosu yok, S2 basılı, adaptörle besleme).
2. UART boot moduna girince ROM kodu, **SPL**'i UART üzerinden **Xmodem** protokolüyle bekler. Bu yüzden bilgisayardan önce SPL'i göndeririz.
3. SPL kartta çalışır (DDR RAM'i hazırlar) ve bu sefer **U-Boot** image'ını bekler. U-Boot'u da göndeririz.
4. U-Boot'un kontrolü ele geçince, U-Boot'un **Xmodem / Ymodem** komutlarıyla (`loadx` / `loady`) diğer bütün boot image'larını (kernel, DTB, initramfs) kartın DDR belleğine indirir ve kartı boot ederiz.

> **Not**
>
> ROM kodu SD karttaki `MLO` dosyasından farklı olarak, UART'tan gelen SPL'in başında **GP header** beklemez. Bu yüzden UART boot'ta `MLO` değil, U-Boot derlemesinin ürettiği ham `u-boot-spl.bin` gönderilir.

7.7 Elimizdeki Boot Dosyaları Nedir?

Seri boot için bilgisayarda (HOST) şu 5 dosya hazır olmalı. Her biri boot zincirinin bir halkasıdır; her dosyanın işi bir sonrakini yükleyip çalıştırmaktır:

Dosya	Nedir?	Kim yükler?	Nereye yüklenir?	Protokol
`u-boot-spl.bin`	**SPL** (Secondary Program Loader). U-Boot'un küçük ilk aşaması. Saatleri, pin ayarlarını, UART'ı ve en önemlisi **DDR RAM**'i hazırlar.	ROM kodu	AM335x'in iç SRAM'i (`0x402F0400` , en fazla ~109 KB)	Xmodem
`u-boot.img`	Asıl **U-Boot**. Başında 64 baytlık U-Boot (mkimage) başlığı vardır. Komut satırını (`=>`) bize o verir.	SPL	DDR (`0x80800000`)	Ymodem
`uImage`	**Linux kernel**. `zImage` 'in başına U-Boot başlığı (yükleme adresi, giriş noktası, CRC) eklenmiş hali (4.9).	U-Boot (`loadx`)	`0x82000000`	Xmodem
`am335x-boneblack.dtb`	**Device Tree Blob**. Kernel'e kartta hangi donanımın olduğunu anlatan derlenmiş dosya (Bölüm 11).	U-Boot (`loadx`)	`0x88000000`	Xmodem
`initramfs`	RAM'de çalışan küçük **kök dosya sistemi** (genellikle BusyBox). `cpio` arşivi + gzip + U-Boot başlığı. SD/eMMC olmadığı için rootfs'i buradan alırız.	U-Boot (`loadx`)	`0x88080000`	Xmodem

Bu dosyaları nereden bulurum?

Bu dosyaları **kendin yazmazsın**. Hepsi açık kaynak projelerden (U-Boot, Linux kernel, BusyBox) çıkar; senin işin kaynak kodu indirip ayarlamak ve derlemektir. Üç yol vardır:

Yol	Ne zaman?	Artısı / eksisi
1. Hazır (prebuilt) dosyalar	İlk deneme, "kart çalışıyor mu?" testi	En hızlısı; ama içinde ne olduğunu sen seçmezsin.
2. Kaynak koddan elle derleme	Öğrenirken (kursta yapılan)	Her adımı görürsün; daha uzun sürer.

Yol	Ne zaman?	Artısı / eksisi
3. Buildroot / Yocto	Gerçek projeler	Hepsini tek komutla üretir; ilk derleme uzun sürer.

Yol 1 — Hazır dosyalar

- **Kursun kaynakları:** Kurslar bu dosyaları genelde derlenmiş olarak verir (Udemy ekleri / kursun GitHub reposu). Kursla birebir uyumlu oldukları için ilk seri boot denemesinde en kolayı bunlar.
- **TI Processor SDK (AM335x):** TI'ın sitesinden indirilir. `board-support/prebuilt-images/` klasöründe genellikle `MLO`, `u-boot-spl.bin`, `u-boot.img`, `zImage`, `.dtb` dosyaları ve bir rootfs hazır gelir.
- **BeagleBoard.org Debian imajı:** Bootloader, kernel ve DTB içerir ama hazır initramfs yoktur; seri boot için daha az uygundur.

Yol 2 — Kaynak koddan derleme

Cross toolchain kurulumu ve SPL/U-Boot derlemesi Bölüm 9'da adım adım anlatılıyor. `mkimage` aracı için `sudo apt install u-boot-tools` yeterli.

Dosya	Kaynak kod	Çıktının yeri
`u-boot-spl.bin`, `u-boot.img`	**U-Boot** (Bölüm 9)	`spl/u-boot-spl.bin`, `u-boot.img`
`uImage`	**Linux kernel** (BBB için genelde `github.com/beagleboard/linux`)	`arch/arm/boot/uImage`
`am335x-boneblack.dtb`	Aynı kernel kaynağı	`arch/arm/boot/dts/am335x-boneblack.dtb`
`initramfs`	**BusyBox** (`busybox.net`) ile hazırlanan rootfs klasörü	Senin oluşturduğun dosya

```
# U-Boot  -> spl/u-boot-spl.bin ve u-boot.img  (ayrıntılar: Bölüm 9)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4

# Kernel + DTB  -> arch/arm/boot/uImage ve .../dts/am335x-boneblack.dtb
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000

# BusyBox  -> _install/ klasörü (rootfs'in temeli)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=../rootfs install

# initramfs: rootfs klasöründen cpio arşivi + U-Boot başlığı
cd rootfs
find . | cpio -H newc -o | gzip -9 > ../initramfs.cpio.gz
cd ..
mkimage -A arm -O linux -T ramdisk -C gzip -n initramfs -d initramfs.cpio.gz initramfs
```

Son satırdaki `mkimage` önemli: `bootm` komutu ramdisk'in başında **U-Boot başlığı** görmek ister. Ham `initramfs.cpio.gz` gönderirsen `Wrong Ramdisk Image Format` hatası alırsın (7.10 (g)).

Yol 3 — Buildroot ile hepsini tek seferde

Buildroot toolchain'i, U-Boot'u, kernel'i, DTB'yi ve rootfs'i senin yerine indirip derler. Kurulumu ve ayrıntıları Bölüm 13'te:

```
wget https://buildroot.org/downloads/buildroot-2026.02.3.tar.gz
tar -xzf buildroot-2026.02.3.tar.gz && cd buildroot-2026.02.3
make beaglebone_defconfig
make menuconfig
make                      # ilk derleme 30 dk - 2 saat sürebilir
ls -1 output/images/
```

`make menuconfig`'te seri boot için iki ayar yap:

Menü	Ayar	Neden?
Bootloaders → U-Boot → U-Boot SPL binary name	`MLO spl/u-boot-spl.bin`	Varsayılan sadece `MLO`'yu kopyalar; seri boot için ham SPL gerekir.
Filesystem images → cpio the root filesystem → **Create U-Boot image of the root filesystem**	Aç	`rootfs.cpio.uboot` üretir: başında U-Boot başlığı olan initramfs.

```
mehmet@ubuntu: ~/buildroot
mehmet@ubuntu:~/buildroot$ make beaglebone_defconfig
#
# configuration written to /home/mehmet/buildroot/.config
#
mehmet@ubuntu:~/buildroot$ make
...
mehmet@ubuntu:~/buildroot$ ls -1 output/images/
am335x-boneblack.dtb
MLO                          <-- SD karttan boot için
rootfs.cpio
rootfs.cpio.uboot            <-- seri boot: initramfs
rootfs.ext2
sdcard.img
u-boot-spl.bin               <-- seri boot: SPL
u-boot.img                   <-- seri boot: U-Boot
zImage                       <-- mkimage ile uImage yapılır
```

Buildroot kernel'i `zImage` olarak verir. `bootm` için `uImage`'e çevir (ya da `uImage` yerine `zImage` gönderip `bootm` yerine `bootz` kullan):

```
mkimage -A arm -O linux -T kernel -C none -a 0x80008000 -e 0x80008000 -n Linux -d zImage uImage
```

Her projede baştan mı derleyeceğim?

Hayır. Hangi dosyanın ne zaman yeniden üretildiği:

Dosya	Ne zaman yeniden derlenir?
SPL + U-Boot	Neredeyse hiç. Bir kez derlenir, uzun süre aynısı kullanılır.
Kernel (`uImage`)	Yeni bir sürücü eklendiğinde veya kernel ayarı değiştiğinde.
DTB	Donanım ayarı (pin, I2C/SPI cihazı vb.) değiştiğinde; sadece `make dtbs` yeterli.
initramfs / rootfs	En sık değişen kısım; kendi uygulamanı buraya koyarsın. Sadece `cpio` + `mkimage` adımı tekrarlanır.

Gerçekten **senin yazdığın** kısım: kendi uygulaman, gerekirse bir kernel sürücüsü ve DTS'teki küçük değişiklikler. Geri kalanı hazır projelerin derlenmesidir.

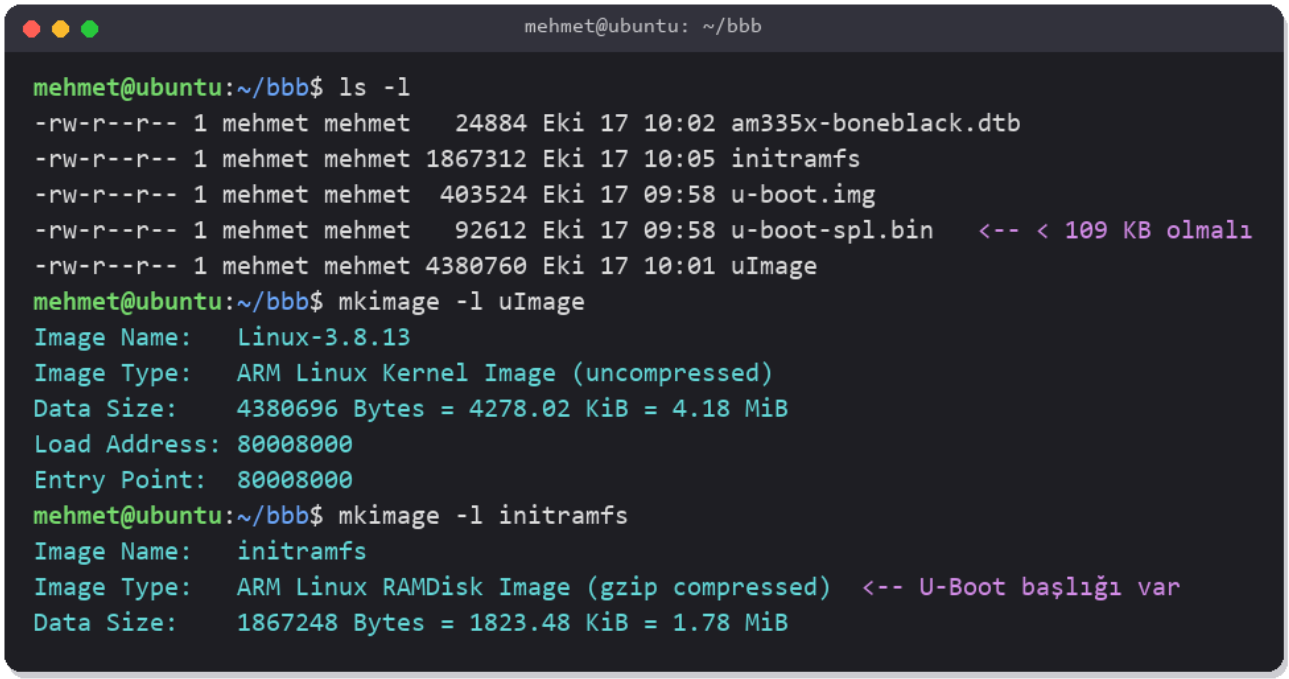
> **İpucu**
>
> Önce kursun hazır dosyalarıyla seri boot'u bir kez çalıştır. Sonra dosyaları **teker teker** kendi derlediklerinle değiştir. Bir şey bozulursa hangi dosyada olduğunu hemen anlarsın.

7.8 Adım Adım Seri Porttan Boot

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm numaraları, tarih, boyut ve hız değerleri senin derlemene göre farklı olacaktır.

Adım 0 — Bilgisayarda hazırlık

Bütün dosyaları tek bir klasörde topla (örnekte `~/bbb`) ve U-Boot başlıklarını `mkimage -l` ile kontrol et:




```
cd ~/bbb
ls -l
mkimage -l uImage
mkimage -l initramfs
```

`u-boot-spl.bin` **109 KB'tan küçük** olmalı; ROM kodu onu iç SRAM'e koyar ve SRAM'de daha fazla yer yoktur.

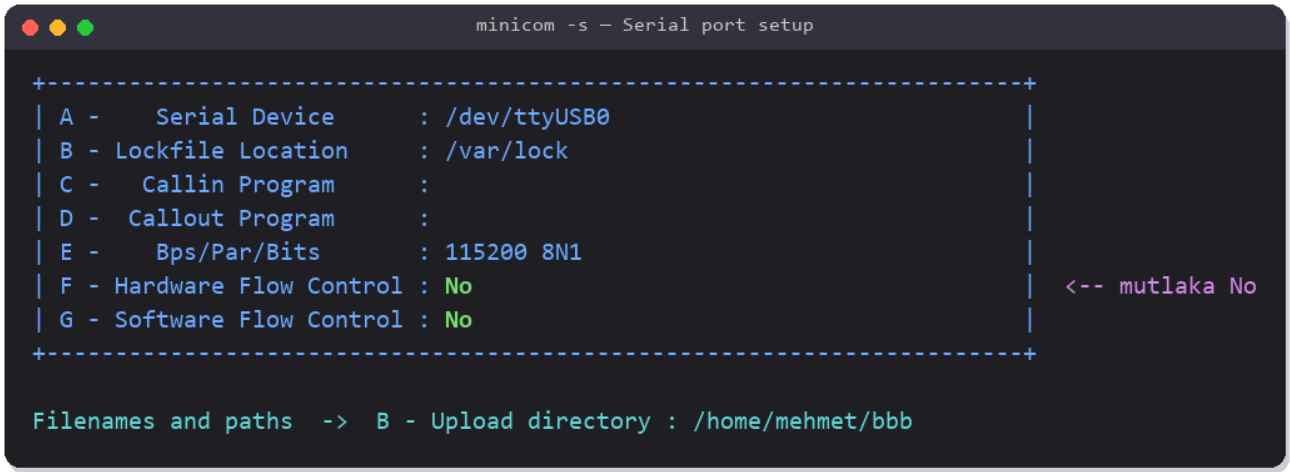
Adım 1 — Seri kabloyu bağla, Minicom'u hazırla

USB-TTL kablosunu J1 header'a bağla (**GND, RX, TX**; kablonun RX'i kartın TX'ine, TX'i kartın RX'ine). Port adını bulma, `dialout` grubu ve Minicom ayarları Bölüm 2'de anlatıldı. Kısaca:

```
sudo dmesg | tail -3
sudo usermod -aG dialout $USER      # bir kez; sonra oturumu kapatıp aç
sudo apt install minicom lrzsz
minicom -s                          # ayarlar (aşağıda)
cd ~/bbb && minicom -D /dev/ttyUSB0 -b 115200
```

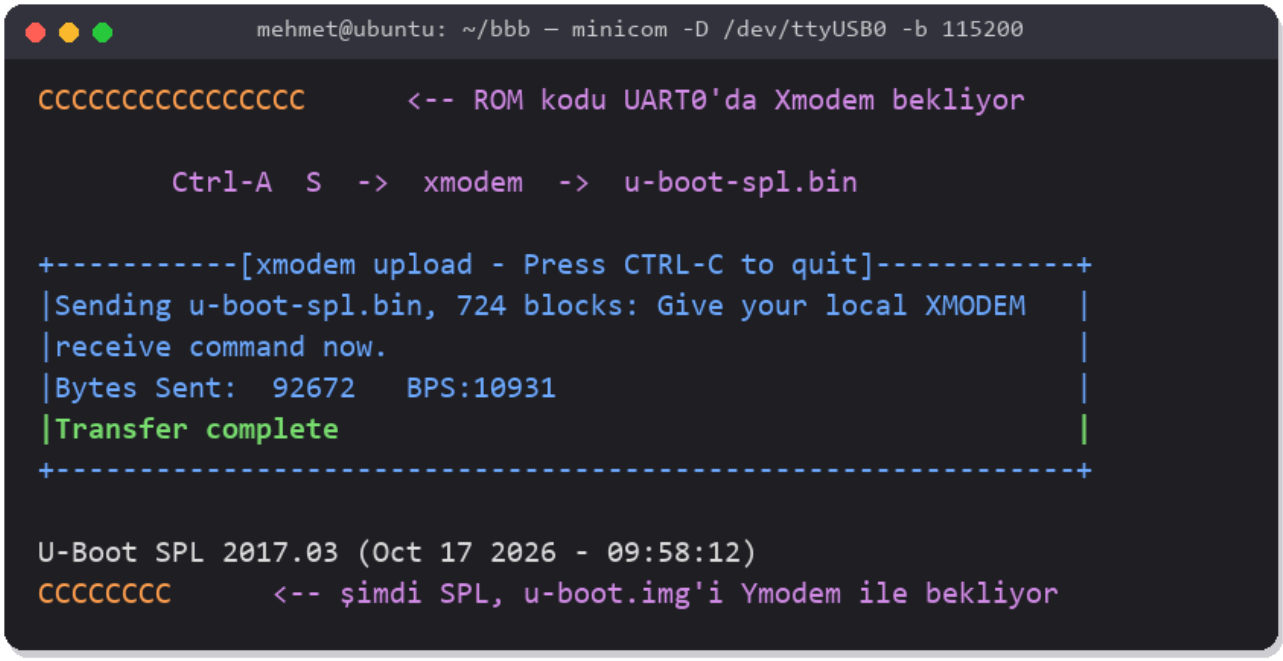
> **Önemli**
>
> Minicom Xmodem/Ymodem gönderirken işi kendisi yapmaz, `lrzsz` paketindeki `sx` (Xmodem) ve `sb` (Ymodem) programlarını çalıştırır. `lrzsz` kurulu değilse dosya gönderilemez.

`minicom -s` → **Serial port setup** ekranında ayarlar şöyle olmalı. **Filenames and paths** menüsünde upload klasörünü `~/bbb` yaparsan dosya seçerken her seferinde klasör aramazsın:



Adım 2 — Kartı UART boot modunda aç ve SPL'i gönder (Xmodem)

1. SD kartı çıkar, P4 mini USB kablosunu çıkar, 5V adaptörü hazırla (7.3).
2. Minicom açıkken **S2**'ye basılı tut, adaptörü tak / **S3**'e bas, sonra S2'yi bırak.
3. Ekranda `CCCC...` akmaya başlar: ROM kodu UART0'da Xmodem ile dosya bekliyor.
4. Minicom'da `Ctrl-A` → `S` → **xmodem** → `u-boot-spl.bin` seç, gönder.

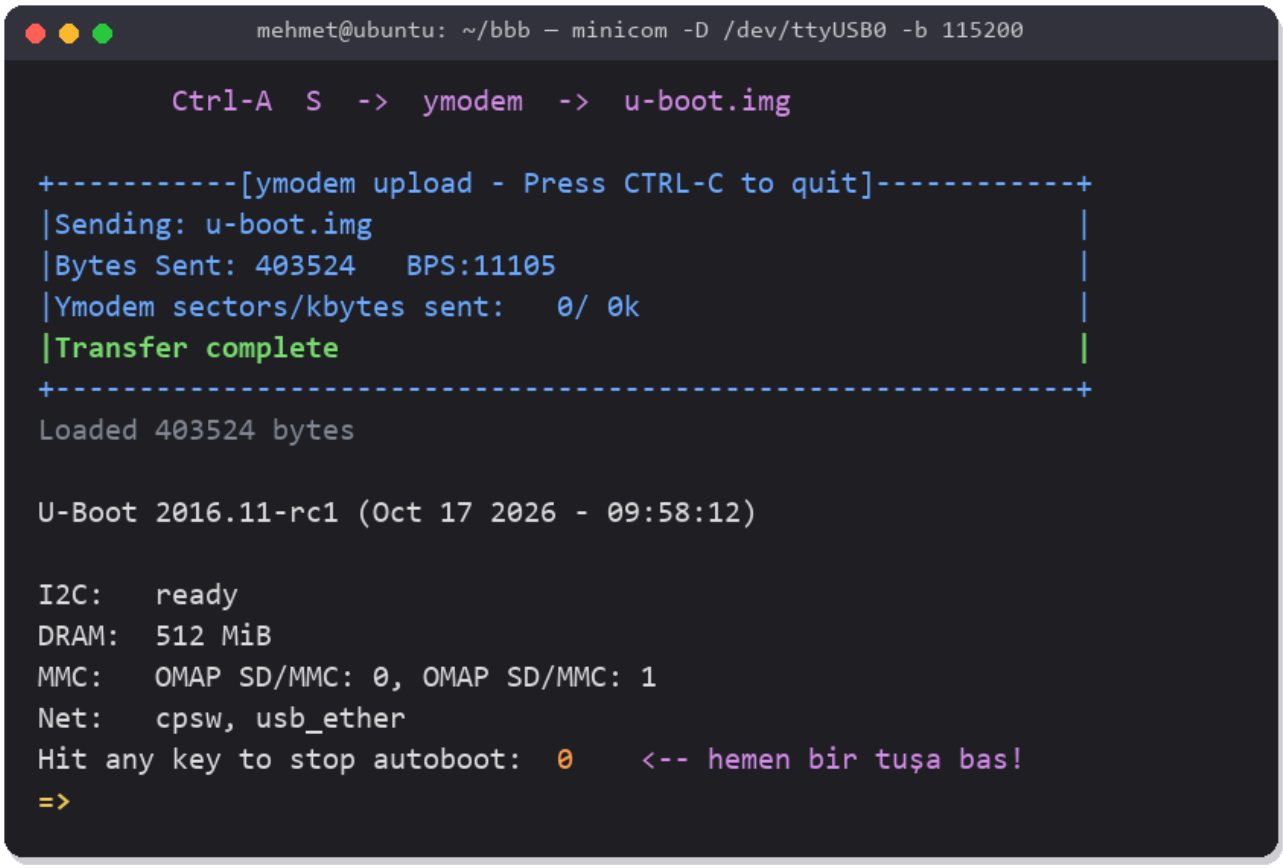


Transfer bitince SPL çalışır, DDR'ı hazırlar, `U-Boot SPL ...` yazısını basar ve tekrar `CCCC` basmaya başlar. Bu sefer bekleyen **SPL**'dir ve **Ymodem** ister.

Adım 3 — U-Boot'u gönder (Ymodem)

> **Not**
>
> Kursun `serial-boot` klasöründeki dosyalar: SPL **2017.03**, U-Boot **2016.11-rc1**. Bu sürümlerde U-Boot'un komut istemi `=>` 'dir. Bölüm 4'teki eski Ångström U-Boot'u (2013.04) ise `U-Boot#` gösterir. Komutlar ikisinde de aynıdır.

1. `Ctrl-A` → `S` → **ymodem** → `u-boot.img` seç, gönder.
2. U-Boot açılır. `Hit any key to stop autoboot` çıkınca **hemen bir tuşa bas.**

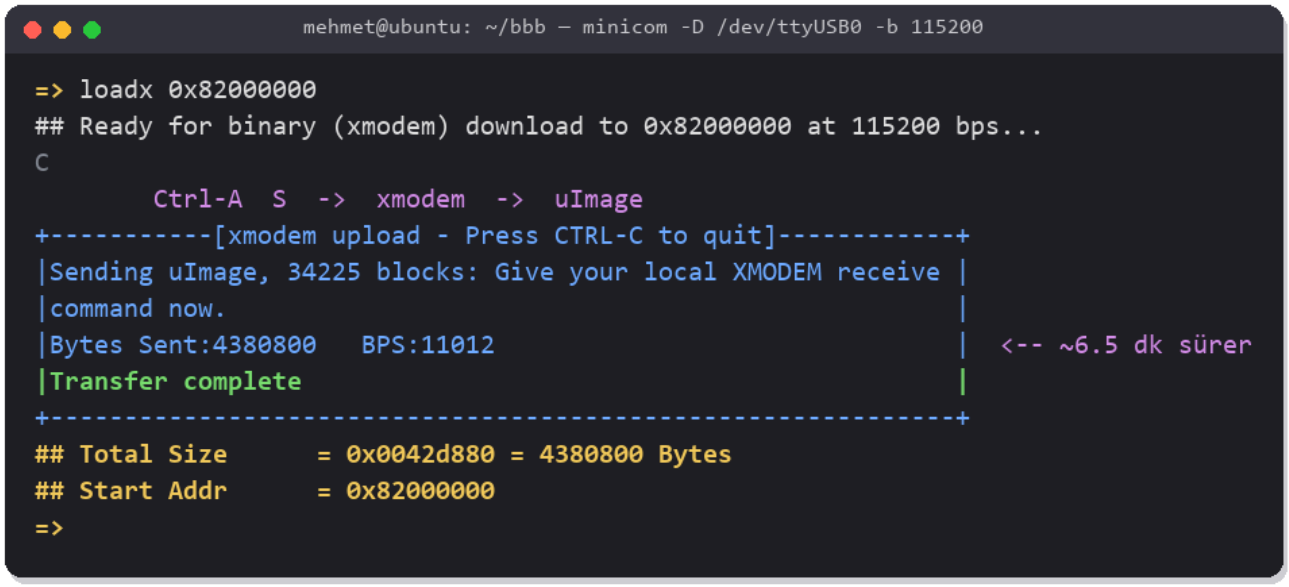


> **Uyarı**
>
> Autoboot'u durdurmazsan U-Boot kendi `bootcmd` 'ini çalıştırır ve kartın **eMMC**'sindeki Linux'u (ör. Debian) açar. O zaman seri boot'un amacı boşa gider; kartı kapatıp Adım 2'den başla.

Adım 4 — Kernel'i RAM'e al: `loadx 0x82000000`

```
=> loadx 0x82000000
```

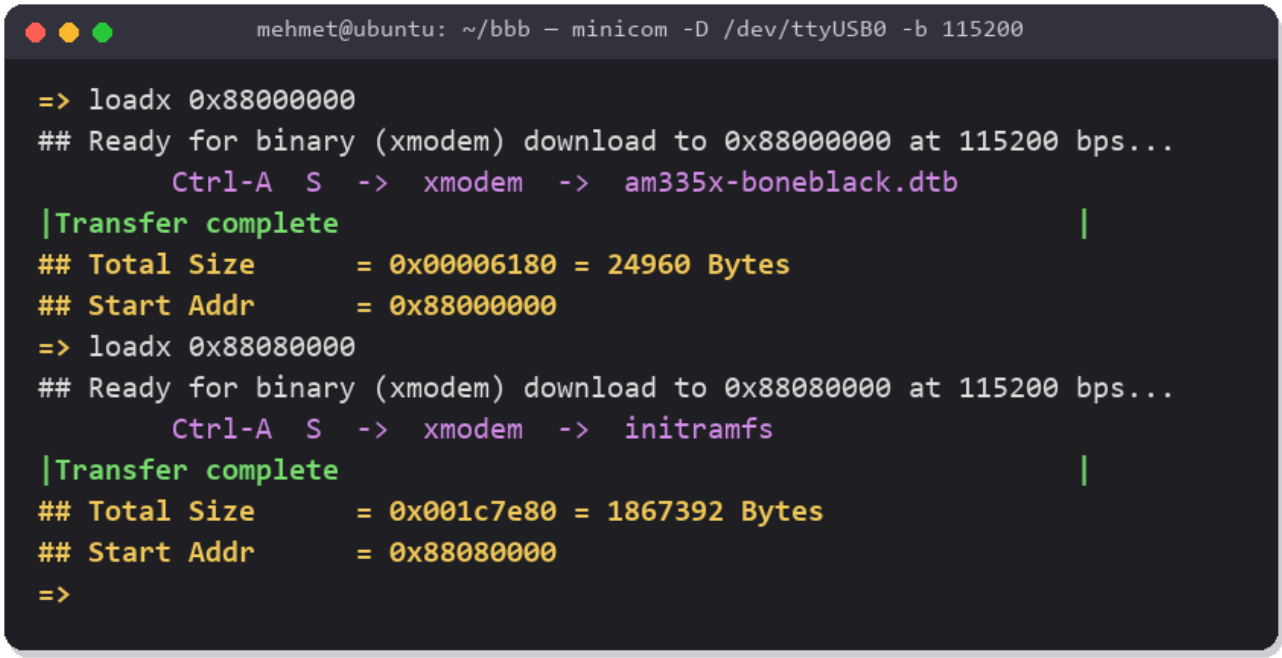
U-Boot Xmodem ile beklemeye başlar. Minicom'da `Ctrl-A` → `S` → **xmodem** → `uImage`.



> **Not**
>
> 115200 baud'da hız en fazla ~11 KB/s'dir. 4 MB'lık bir `uImage` yaklaşık **6–7 dakika** sürer; bu normaldir. Xmodem son paketi 128 bayta tamamladığı için `Total Size` dosya boyutundan biraz büyük görünür, sorun değil.

Adım 5 — DTB ve initramfs'i RAM'e al

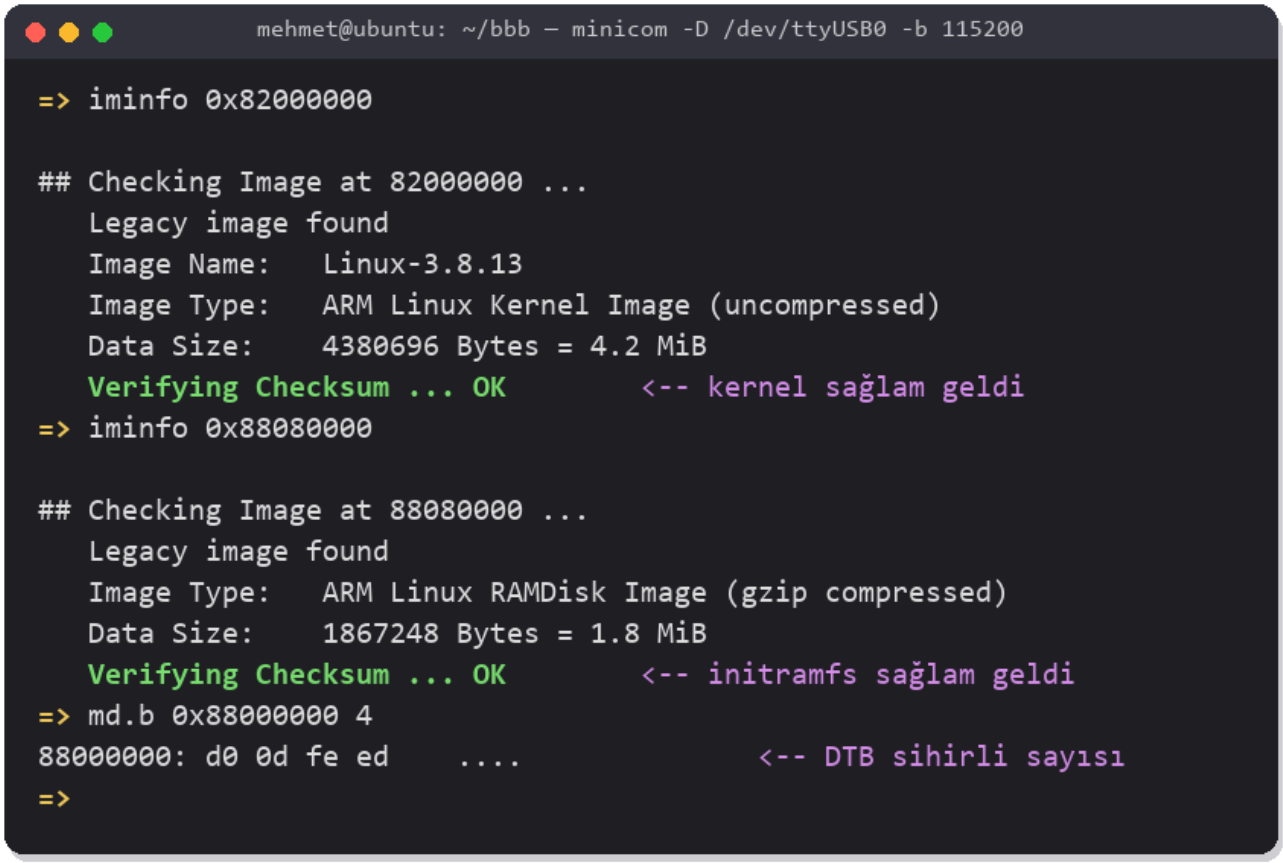
```
=> loadx 0x88000000        (Minicom: xmodem -> am335x-boneblack.dtb)
=> loadx 0x88080000        (Minicom: xmodem -> initramfs)
```



Adım 6 — Yüklenenleri kontrol et (isteğe bağlı ama önerilir)

Boot etmeden önce dosyaların sağlam geldiğini görmek, sonradan "neden açılmadı?" diye uğraşmaktan iyidir:

```
=> iminfo 0x82000000
=> iminfo 0x88080000
=> md.b 0x88000000 4
```



- `iminfo`, U-Boot başlığındaki CRC'yi RAM'deki veriyle karşılaştırır. `Verifying Checksum ... OK` görmelisin.
- DTB'nin U-Boot başlığı yoktur; ilk 4 baytı her zaman `d0 0d fe ed` olmalıdır.

Adım 7 — `bootargs` ayarla ve `bootm` ile boot et

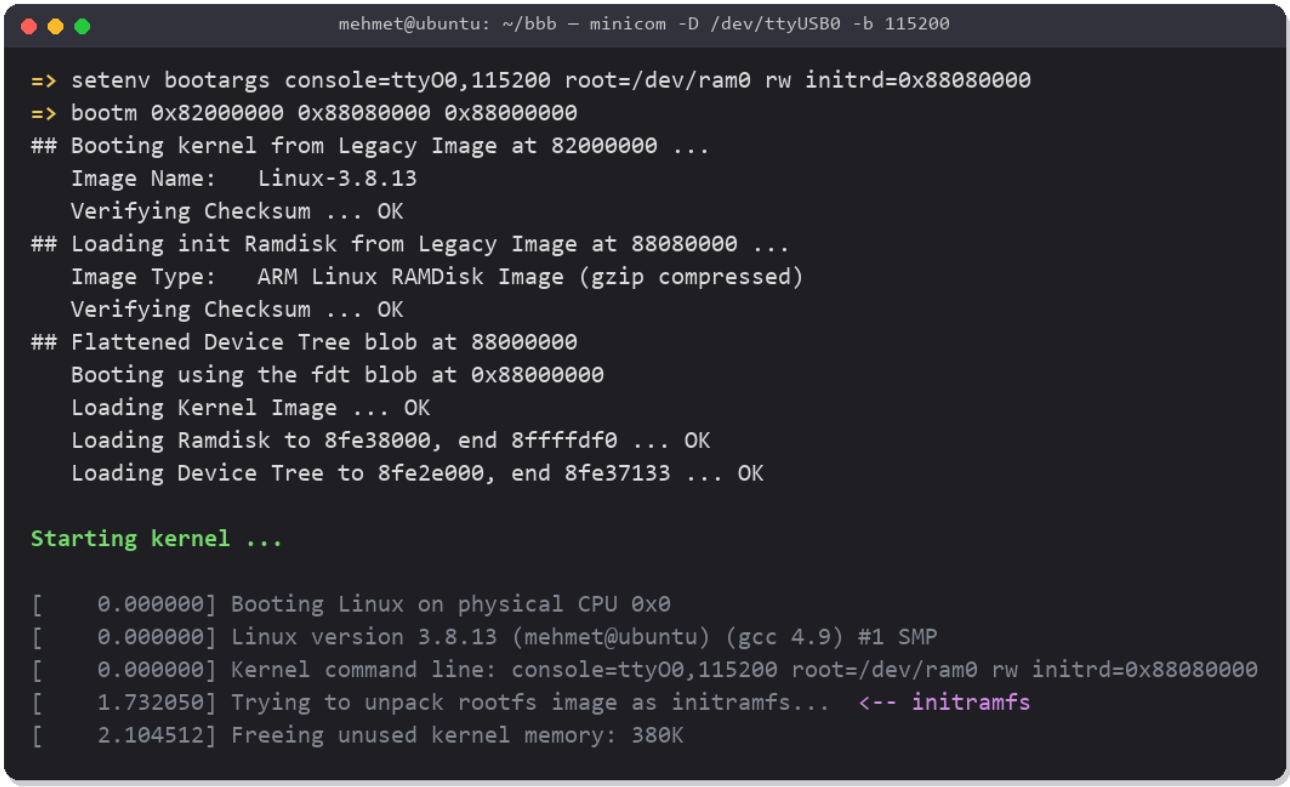
```
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
```

Boot etmeden önce kernel'e **rootfs'in nerede olduğunu** `bootargs` ile bildiriyoruz:

Parametre	Anlamı
`console=ttyO0,115200`	Kernel mesajlarını UART0'a, 115200 baud ile bas (kursun kernel'i 3.8.13 için `ttyO0`; 7.10 (h)).

Parametre	Anlamı
`root=/dev/ram0`	Kök dosya sistemi (rootfs) bir disk değil, **RAM**'de duruyor.
`rw`	Rootfs okunur-yazılır bağlansın.
`initrd=0x88080000`	Kursun komutunda var ama bu haliyle **etkisizdir**: ARM kernel'i bu parametreyi sadece `initrd=<adres>,<boyut>` biçiminde kabul eder; boyut yoksa yok sayar. initramfs'in yerini kernel'e asıl `bootm` bildirir (aşağıdaki not).
`bootm <kernel> <initramfs> <dtb>`	Sıra önemli: **kernel – ramdisk – DTB**. Ramdisk yoksa ortaya `-` yazılır (`bootm 0x82000000 - 0x88000000`).

> **Not**
>
> `bootm` 'in ikinci parametresi (`0x88080000`), initramfs'in U-Boot başlığından boyutunu okur ve adresiyle birlikte kernel'e DTB'nin `/chosen` düğümü üzerinden (`linux,initrd-start` , `linux,initrd-end`) bildirir. Kernel initramfs'i buradan bulur. Bu yüzden `bootargs` 'taki `initrd=` satırı olmasa da açılış aynıdır; kursla aynı kalsın diye komutta bırakıldı.



Adım 8 — Linux açıldı

Kernel initramfs'i açar, içindeki `/init` (BusyBox) çalışır ve seri konsoldan kabuk gelir:

```
BusyBox (initramfs) — seri konsol
Please press Enter to activate this console.
/ # cat /proc/cmdline
console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
/ # mount
rootfs on / type rootfs (rw)        <-- rootfs RAM'de
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
/ # uname -a
Linux (none) 3.8.13 #1 SMP Sat Oct 17 10:01:44 2026 armv7l GNU/Linux
/ # free -m
             total       used       free     shared    buffers
Mem:           497         14        483          0          0
```

`mount` çıktısında `/` 'nin **`rootfs`** olduğunu görürsün: Kart ne SD karta ne eMMC'ye dokunmadan, tamamen seri porttan gelen dosyalarla RAM'den çalışıyor.

> **İpucu**
>
> RAM'deki her şey kart kapanınca silinir. Bir sonraki açılışta bütün adımları (SPL'den itibaren) tekrar yapman gerekir.

Komutların kısa özeti

```
[Kart: S2 basılı + güç, ekranda CCCC]
Minicom: Ctrl-A S -> xmodem -> u-boot-spl.bin
Minicom: Ctrl-A S -> ymodem -> u-boot.img
[Hit any key -> tuşa bas]
=> loadx 0x82000000          -> xmodem -> uImage
=> loadx 0x88000000          -> xmodem -> am335x-boneblack.dtb
=> loadx 0x88080000          -> xmodem -> initramfs
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
```

7.9 Özet

- UART boot için kart **SD kartsız**, **mini USB kablosu takılı değilken** (yoksa ROM USB0'da 4.5 dakika bekler), DC jack'ten beslenerek ve **S2 basılıyken** açılır. Ekrandaki `CCCC...` ROM kodunun Xmodem ile dosya beklediğini gösterir.
- Sıra: ROM ← **`u-boot-spl.bin`** (Xmodem; `MLO` değil, başlıksız ve 109 KB'tan küçük) → SPL ← **`u-boot.img`** (Ymodem) → U-Boot'ta `loadx` ile kernel, DTB ve initramfs.

- RAM adresleri: kernel `0x82000000`, DTB `0x88000000`, initramfs `0x88080000`. initramfs'in başında `mkimage -T ramdisk` ile eklenmiş U-Boot başlığı olmalı.
- Açılış: `setenv bootargs console=... root=/dev/ram0 rw` → `bootm <kernel> <initramfs> <dtb>`. Rootfs RAM'de olduğu için kart kapanınca her şey silinir.
- 115200 baud'da transfer yavaştır (~11 KB/s). Günlük geliştirmede TFTP (Bölüm 8) kullanılır; UART boot bozuk bir kartı kurtarmak ve yeni bir SPL/U-Boot'u karta yazmadan denemek içindir.

7.10 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

=> bootm 0x82000000 0x88080000 0x88000000
## Loading init Ramdisk from Legacy Image at 88080000 ...
Wrong Ramdisk Image Format           <-- initramfs'te U-Boot başlığı yok
Ramdisk image is corrupt or invalid
=> iminfo 0x82000000
## Checking Image at 82000000 ...
   Verifying Checksum ... Bad Data CRC  <-- transfer bozuk / adres çakışması

Starting kernel ...
    <-- sonrası boş: console= yanlış ya da DTB uyumsuz

[    2.11] VFS: Cannot open root device "ram0" or unknown-block(1,0)
[    2.11] Kernel panic - not syncing: VFS: Unable to mount root fs  <-- /init yok

[    2.20] Failed to execute /init.  Attempting defaults...    <-- ARM değil / kütüphane eksik
```

a) Ekranda hiç `cccc` çıkmıyor

Olası neden	Çözüm
SD kart takılı	ROM önce MMC0'ı dener, SD'de MLO varsa oradan açılır. SD kartı çıkar.
P4 mini USB kablosu takılı	ROM USB0'da 4.5 dakika bekler. Kabloyu çıkar, kartı 5V adaptörle besle (7.3).
S2 basılı değildi	Kart eMMC'den açılır (ekranda eMMC'deki U-Boot/Linux çıkar). Kartı kapat, S2 basılıyken tekrar aç.
TX / RX ters bağlı ya da GND bağlı değil	Kablonun TX'i → kartın RX'i, RX'i → kartın TX'i. GND mutlaka bağlı olmalı.
Yanlış port / baud	`dmesg` ile doğru `/dev/ttyUSBx`'i bul; ayar 115200 8N1 olmalı.
Minicom'u kart açıldıktan sonra açtın	`C`'leri kaçırmış olabilirsin. Önce Minicom'u aç, sonra kartı aç.

b) `cccc` var ama transfer başlamıyor / yarıda kesiliyor

Olası neden	Çözüm
Hardware Flow Control: Yes	Minicom karta veri göndermez. `minicom -s` ile **No** yap (en sık yapılan hata!).

Olası neden	Çözüm
`lrzsz` kurulu değil (`sx: not found`)	`sudo apt install lrzsz`
Dosya seçmek çok uzun sürdü, ROM vazgeçti	Upload klasörünü önceden ayarla, `C` görünür görünmez gönder. `C` akışı durduysa kartın gücünü kesip S2 ile baştan aç.
Yanlış protokol	ROM ve `loadx` → **xmodem**; SPL'in `u-boot.img` için beklediği ve `loady` → **ymodem**. Karıştırılırsa transfer hiç başlamaz veya `Transfer incomplete` olur.
Transfer sırasında klavyeye basıldı	Gelen tuşlar Xmodem paketlerini bozar. Transfer bitene kadar dokunma.

c) SPL gönderildi ama hiçbir şey olmuyor

Olası neden	Çözüm
`u-boot-spl.bin` yerine `MLO` gönderildi	MLO'nun başında GP header var; ROM onu kod sanıp çalıştırır ve kilitlenir. UART boot'ta **ham** `u-boot-spl.bin` gönderilir.
SPL 109 KB'tan büyük	ROM iç SRAM'e sığdıramaz. SPL'deki gereksiz özellikleri kapatıp yeniden derle.
SPL farklı bir kart/işlemci için derlenmiş	BBB için uygun defconfig ile yeniden derle (9.3).

d) SPL çalışıyor ama `u-boot.img` yüklenmiyor

Olası neden	Çözüm
SPL, UART'tan Ymodem alma desteğiyle derlenmemiş (`CONFIG_SPL_YMODEM_SUPPORT`)	SPL başka bir aygıt (MMC) arar ve hata verir. Config'te YMODEM desteğini aç, yeniden derle.
`u-boot.img` Xmodem ile gönderildi	SPL Ymodem bekler; **ymodem** seç.
`u-boot.img` yerine `u-boot.bin` gönderildi	SPL, başında mkimage başlığı olan `u-boot.img` 'i bekler.

e) U-Boot açıldı ama kendi kendine eMMC'den boot etti

`Hit any key to stop autoboot` sırasında tuşa basılmadı. Kartı kapatıp baştan başla; geri sayım başlar başlamaz bir tuşa bas.

> **Dikkat**
>
> Seri boot sırasında U-Boot'ta `saveenv` **yapma**. Değişkenler eMMC'deki ortama yazılır ve kartın normal açılışını bozabilir.

f) Transfer çok yavaş

115200 baud'da ~11 KB/s normaldir; `uImage` 6–7 dakika, initramfs birkaç dakika sürer. Hızlandırmak için:

- **Baud yükseltme:** `loadx` / `loady` ikinci parametre olarak baud alır: `loady 0x82000000 460800`. U-Boot hızı değiştirmeni ister; Minicom'da `Ctrl-A` → `P` ile aynı hıza geç, Enter'a bas, gönder. Bitince U-Boot 115200'e dönmeni ister; Minicom'u geri al ve **ESC**'ye bas. (Her USB-TTL kablo yüksek hızlarda güvenilir çalışmaz; hata alırsan 115200'e dön.)
- **Ymodem:** `loady` 1 KB'lık paketlerle çalıştığı için `loadx`'ten biraz daha hızlıdır.
- **TFTP:** U-Boot açıldıktan sonra Ethernet kablosu varsa `tftpboot` ile saniyeler içinde yüklenir (Bölüm 8).

g) `bootm` hataları

Hata	Neden	Çözüm
`Wrong Ramdisk Image Format` / `Ramdisk image is corrupt or invalid`	initramfs'in başında U-Boot başlığı yok veya yanlış adrese yüklendi	Bilgisayarda `mkimage -T ramdisk ...` ile başlık ekle (7.7). `iminfo 0x88080000` ile kontrol et.
`Bad Data CRC`	Transfer sırasında veri bozuldu ya da bir dosya diğerinin üstüne yazıldı	Dosyayı tekrar gönder. Adresleri 7.5'teki gibi kullan; DTB 512 KB'ı aşmamalı.
`Bad Magic Number` / `Wrong Image Format for bootm command`	`uImage` yerine `zImage` gönderildi veya adres yanlış	Kernel'i `make uImage LOADADDR=0x80008000` ile derle ya da `zImage` için `bootz` kullan.
DTB bulunamadı / `FDT_ERR_BADMAGIC`	DTB bozuk ya da yanlış adrese yüklendi	`md.b 0x88000000 4` → `d0 0d fe ed` görmelisin. Görmüyorsan DTB'yi tekrar gönder.

h) `Starting kernel ...` yazıp donuyor

Olası neden	Çözüm
`console=` yanlış	Eski kernel'ler (3.x, TI omap-serial sürücüsü) → `ttyO0`. Daha yeni (4.x ve sonrası) kernel'lerde genellikle `ttyS0`. Kernel aslında açılmış olabilir, sadece mesajları başka yere basıyordur.
DTB, kernel sürümüyle uyumsuz	DTB'yi **aynı kernel kaynağından** derlenmiş olanla değiştir; BBB için `am335x-boneblack.dtb` (`am335x-bone.dtb` değil).
`bootm` parametre sırası yanlış	Sıra: `bootm <kernel> <ramdisk> <dtb>`.

i) Kernel açılıyor ama panic veriyor

Hata	Neden	Çözüm
`VFS: Unable to mount root fs on unknown-block(1,0)`	initramfs içinde `/init` yok; kernel `root=/dev/ram0`'ı bağlamaya çalışıyor ve başarısız oluyor	rootfs klasöründe `ln -s sbin/init init` yapıp initramfs'i yeniden oluştur, ya da `bootargs`'a `rdinit=/sbin/init` ekle.
`VFS: Unable to mount root fs on unknown-block(0,0)`	Kernel ramdisk'i hiç almamış veya `CONFIG_BLK_DEV_INITRD` kapalı	`bootm`'in ikinci parametresini kontrol et; kernel config'te initramfs/initrd desteğini aç.
`Failed to execute /init` / `No working init found`	initramfs'teki programlar ARM için derlenmemiş veya kütüphaneler eksik	BusyBox'ı `CROSS_COMPILE=arm-linux-gnueabihf-` ile ve tercihen **static** derle; dinamik derlediysen `lib/` klasörüne gerekli kütüphaneleri koy. `file bin/busybox` → `ARM` yazmalı.
`RAMDISK: incomplete write`	initramfs yerine ext2 **initrd** imajı kullanılıyor ve kernel'in varsayılan ramdisk boyutundan büyük	`bootargs`'a `ramdisk_size=<KB>` ekle.

j) Ekranda bozuk / anlamsız karakterler

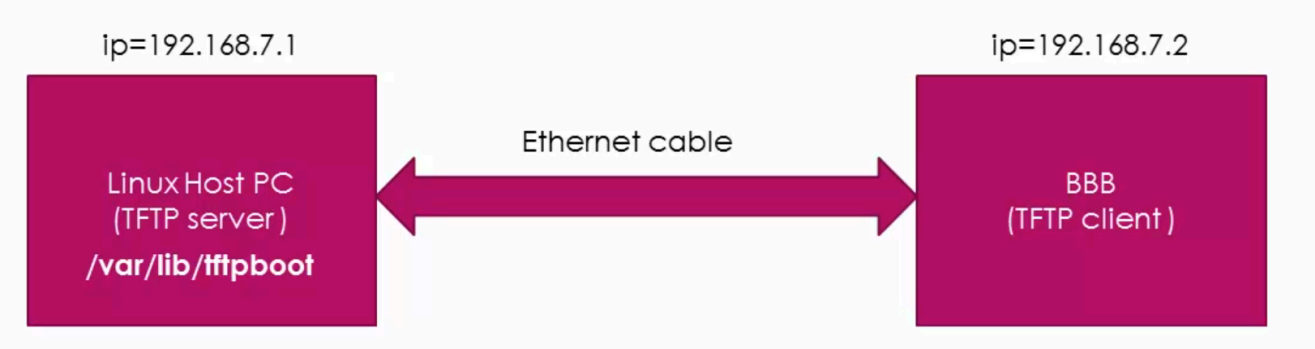
Baud rate uyuşmuyor. Minicom 115200 8N1 olmalı. Baud yükseltme (f) denediysen Minicom'u tekrar 115200'e al.

Bölüm 8 — TFTP ile Ağdan Boot

> **Bu bölümde:**
>
> - Bilgisayarına bir TFTP sunucusu (`tftpd-hpa`) kuracaksın.
> - Kernel, DTB ve initramfs'i her açılışta **Ethernet üzerinden** host'tan çekerek kartı açacaksın: önce komutları elle yazarak, sonra `uEnv.txt` ile otomatik olarak.
> - Neden bazı dosyaların yine de SD kartta durmak zorunda olduğunu anlayacaksın.
> - Challenge: eMMC → SD → TFTP → NFS sırasıyla deneyen genel bir `uEnv.txt` yazacaksın.
>
> **Ön koşul:** Bölüm 4 — SD kart, Bölüm 6 — `uEnv.txt` ve U-Boot komutları, Bölüm 7 — RAM adresleri ve `bootm` .

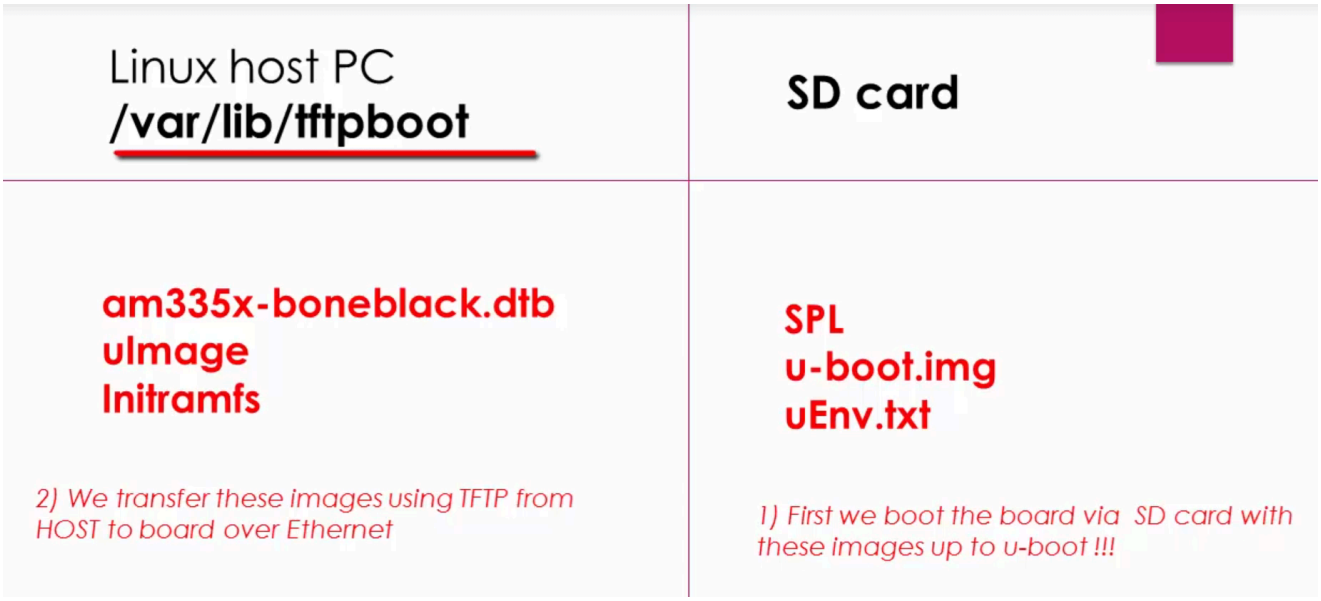
8.1 Genel Bakış

Host bilgisayar **TFTP sunucusu**, BeagleBone Black ise **TFTP istemcisi** olur. İkisi Ethernet kablosuyla bağlanır:



Görsel: kursun slaytı.

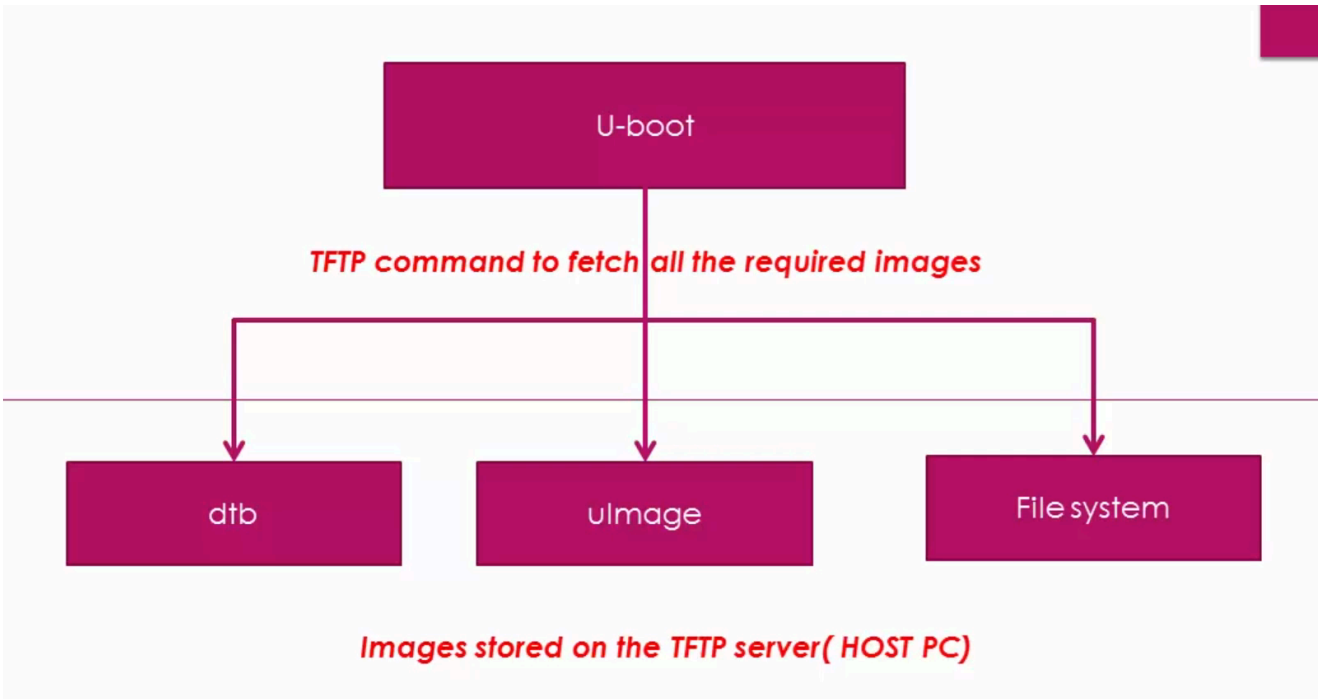
Dosyalar iki gruba ayrılır:

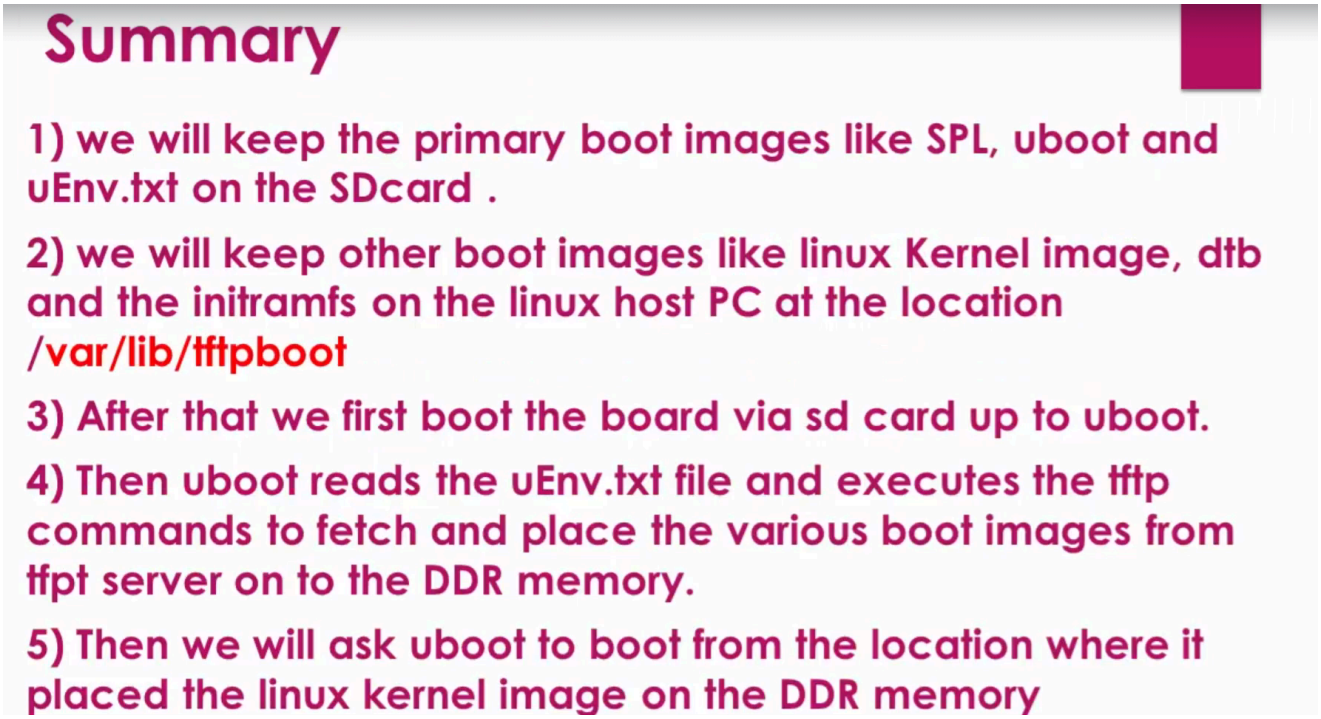


Görsel: kursun slaytı.

Nerede?	Dosyalar
SD kart	SPL (MLO), u-boot.img, uEnv.txt
Host PC `/var/lib/tftpboot`	am335x-boneblack.dtb, uImage, initramfs

Kart önce SD karttan U-Boot'a kadar açılır. Sonra U-Boot, uEnv.txt'deki `tftp` komutlarıyla diğer dosyaları host'tan çekip DDR'a yerleştirir:





Görseller: kursun slaytları.

8.2 Neden Bütün Dosyalar TFTP ile Gönderilmiyor?

Kısaca: **TFTP isteğini yapacak kod U-Boot'un içinde.** U-Boot ağdan gelecek olsaydı, onu ağdan isteyecek bir program olmazdı. Bu bir tavuk-yumurta problemi.

Boot zinciri

```
1. ROM Code (çipin içinde, değiştirilemez)
   │  SD karttan MLO/SPL'i bulur, dahili SRAM'e (~109 KB) yükler
   ▼
2. SPL / MLO
   │  saatleri ve pinleri ayarlar, DDR RAM'i başlatır,
   │  u-boot.img'yi DDR'a yükler
   ▼
3. u-boot.img  (tam U-Boot)
   │  Ethernet + MMC + UART sürücülerini başlatır,
   │  uEnv.txt'yi okur, TFTP istemcisini çalıştırır
   ▼
TFTP ile host'tan:  uImage, .dtb, initramfs  →  DDR  →  bootm
```

- **SPL**, DDR RAM'i başlatır. DDR hazır olmadan büyük dosyalar (U-Boot, kernel) bir yere yüklenemez.
- **u-boot.img**, U-Boot'un kendisidir. Ethernet sürücüsü ve TFTP istemcisi buradadır.
- **uEnv.txt**, U-Boot'a `ipaddr`, `serverip` ve hangi dosyayı hangi adrese çekeceğini söyler. U-Boot bu bilgiyi okuyabilmek için dosyanın yerel bir depolamada durması gerekir.

- **TFTP istemci/sunucu protokolüdür.** Host dosyayı kendiliğinden göndermez, kartın istemesi gerekir. İsteyecek kod (U-Boot) çalışmadan kart ağdan hiçbir şey alamaz.
- **ROM Code'un okuyabileceği yerler sabittir.** BBB'nin boot listesinde Ethernet yoktur (3.5). AM335x'in ROM'u aslında Ethernet'ten boot'u destekler, ama BBB'de bu seçeneği açmak için donanımda değişiklik gerekir.

> **Not**
>
> U-Boot'un başlattığı sürücüler **kalıcı değildir**. U-Boot sadece dosya yükleyip kernel'i çalıştırmak için gereken sürücüleri başlatır. Kernel açılınca bütün donanımı kendi sürücüleriyle **yeniden** başlatır.

Bunun amacı: geliştirmeyi hızlandırmak

Dosya	Ne sıklıkla değişir?	Nerede durur?
SPL, u-boot.img, uEnv.txt	Neredeyse hiç	SD kart (bir kere kopyalanır)
uImage, .dtb, initramfs	Sürekli (kernel derle, DT düzenle, rootfs'e ekle)	Host PC, `/var/lib/tftpboot`

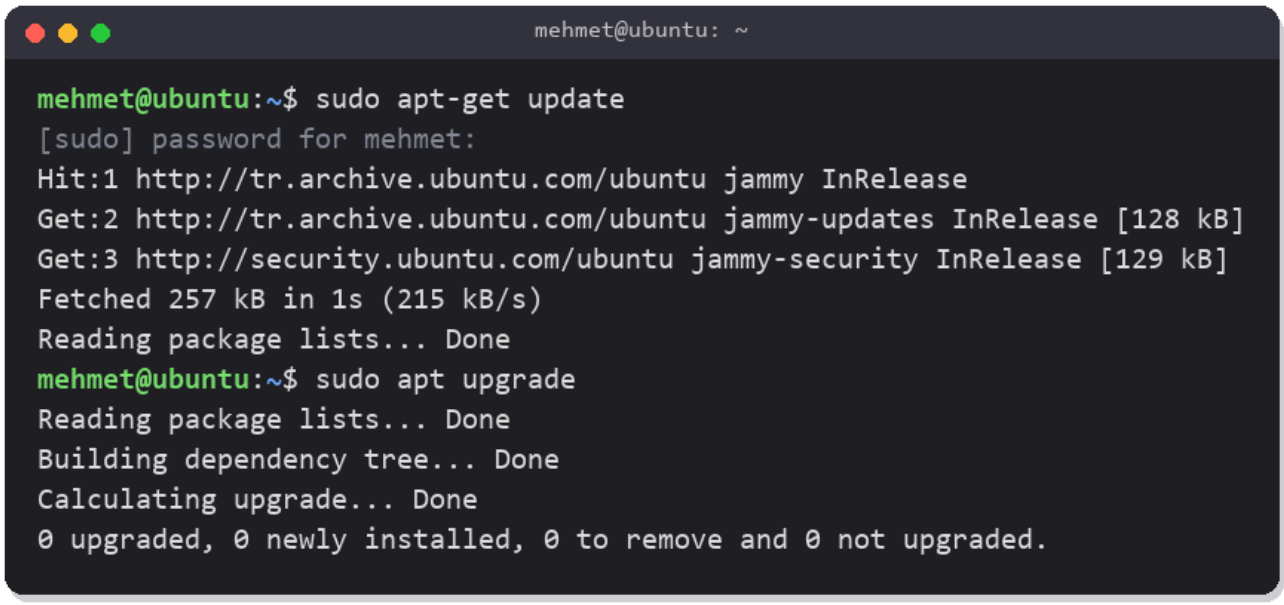
- Kernel ya da device tree değişince SD kartı çıkarıp takmaya gerek kalmaz. Yeni dosyayı `/var/lib/tftpboot` içine kopyalayıp kartı resetlemek yeterli.
- SD kartın yıpranma ve bozulma riski azalır.
- Sektörde de böyle çalışılır: bootloader kartta sabit durur, kernel (ve sonra NFS ile rootfs) ağdan gelir.

> **İpucu**
>
> Tek cümleyle: SD karttaki SPL + U-Boot kartı **"ağdan dosya isteyebilir"** hale getirir. Ondan sonra gelen her şey TFTP ile host'tan alınır.

8.3 TFTP Host'u Hazırlama

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm numaraları, tarih ve PID değerleri senin bilgisayarında farklı olacaktır.

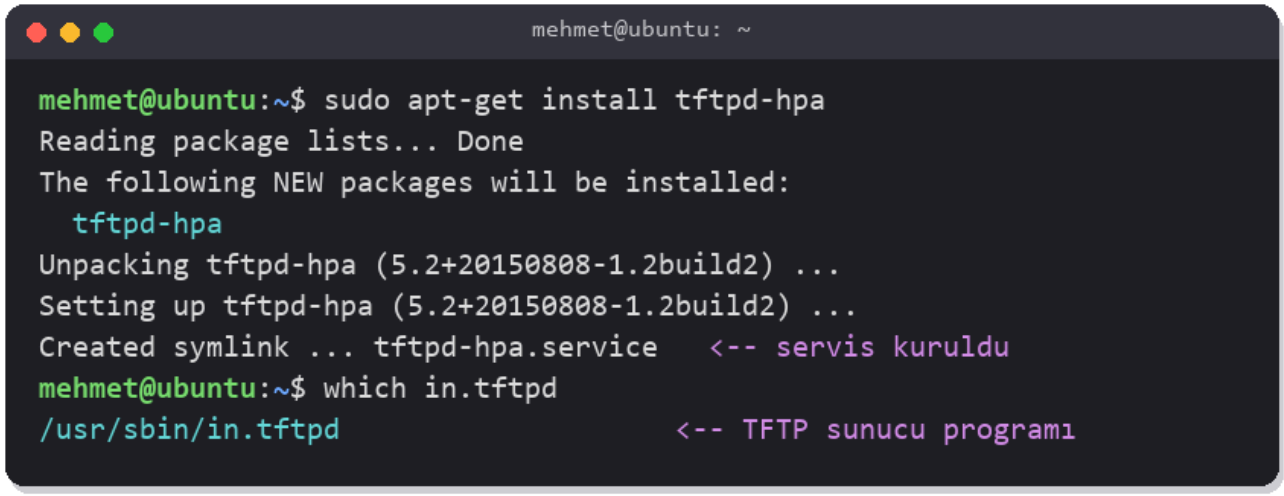
Adım 1 — Paket listesini güncelle




```
sudo apt-get update
sudo apt upgrade
```

Adım 2 — TFTP sunucusunu kur

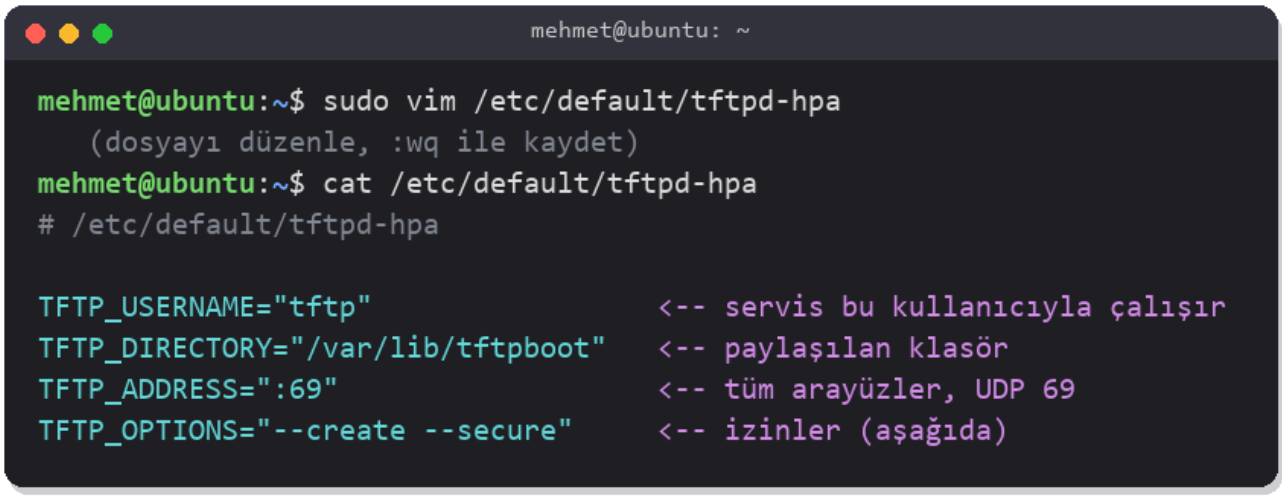
`tftpd-hpa`, Trivial File Transfer Protocol için bir sunucu (daemon) programıdır.




```
sudo apt-get install tftpd-hpa
```

Adım 3 — Ayar dosyasını düzenle

`/etc/default/tftpd-hpa` dosyasını aç ve içeriği aşağıdaki gibi yap:




```
sudo vim /etc/default/tftpd-hpa
```

```
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/var/lib/tftpboot"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="--create --secure"
```

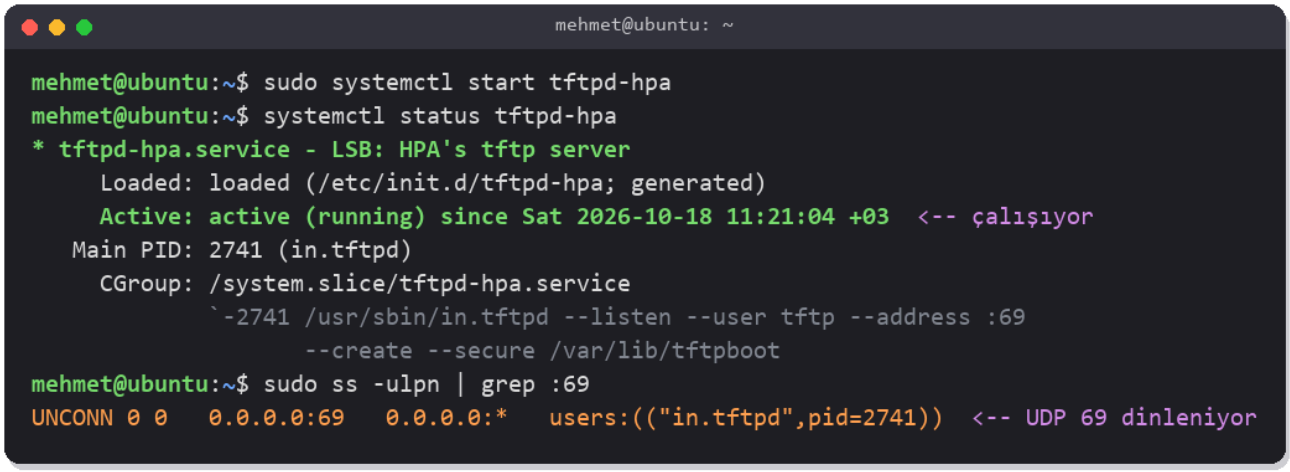
Ayar	Anlamı
`TFTP_USERNAME`	Servis bu kullanıcının yetkileriyle çalışır
`TFTP_DIRECTORY`	Kartın dosya çekeceği klasör
`TFTP_ADDRESS=":69"`	Bütün ağ arayüzlerinde UDP 69 portunu dinle
`--secure`	Dosya yolları bu klasöre göre yazılır (`tftp ... uImage` yeterli, tam yol gerekmez)
`--create`	İstemcinin klasöre yeni dosya yüklemesine izin verir

> **Uyarı**
>
> Ayar dosyasında portu **69** olarak bırak. `TFTP_ADDRESS="0.0.0.0:6900"` gibi farklı bir port yazarsan U-Boot'un `tftpboot` komutu sunucuyu bulamaz, çünkü varsayılan olarak 69'a bağlanır. `TFTP_DIRECTORY` için `/home/<user>/tftp` gibi başka bir klasör de kullanabilirsin, ama bu kitapta her yerde `/var/lib/tftpboot` geçiyor.

Adım 4 — tftpboot klasörünü oluştur

```
sudo mkdir -p /var/lib/tftpboot
sudo chown tftp:tftp /var/lib/tftpboot
sudo chmod -R 777 /var/lib/tftpboot
```

Adım 5 — TFTP servisini başlat ve kontrol et




```
sudo systemctl start tftpd-hpa
systemctl status tftpd-hpa       # "active (running)" görmelisin
sudo ss -ulpn | grep :69         # UDP 69 dinleniyor mu?
```

> **İpucu**
>
> Ayar dosyasını sonradan değiştirirsen servisi yeniden başlat: `sudo systemctl restart tftpd-hpa`

8.4 Bağlantılar

Kablo	Görevi
5V adaptör (barrel jack)	Güç. Bilgisayarın USB'si 500 mA ile sınırlı, kart boot sırasında resetlenebilir.
USB-TTL kablo (J1 header)	Seri konsol (minicom): U-Boot'u durdurmak ve mesajları görmek

Kablo	Görevi
Ethernet kablosu (host ↔ BBB)	**TFTP dosya aktarımı**

Bu bölümde kullanılan adresler:

Cihaz	IP	U-Boot değişkeni
Host PC (TFTP sunucu)	`192.168.27.1`	`serverip`
BBB (TFTP istemci)	`192.168.27.2`	`ipaddr`

İki adres **aynı alt ağda** (`192.168.27.x`) olmalı. Kursun şemasındaki `192.168.7.x` adreslerini de kullanabilirsin; önemli olan host ile kartın aynı ağda olması.

8.5 Yol 1 — Komutları Elle Yazarak TFTP ile Boot

İki yol var:

	Yol 1 — Elle (bu başlık)	Yol 2 — uEnv.txt ile otomatik (8.6)
Nasıl?	Her açılışta U-Boot'u durdurup komutları minicom'a yazarsın	Komutlar SD karttaki `uEnv.txt` 'de durur, U-Boot kendisi çalıştırır
Ne zaman?	İlk deneme, hata ayıklama: hangi komutun nerede takıldığını görürsün	Kurulum bir kez çalıştıktan sonra, günlük geliştirmede

Önce Yol 1 ile bir kere çalıştır, sonra Yol 2'ye geç. Yol 2'deki `uEnv.txt` aynı komutları içerdiği için bir hata olursa Yol 1'e dönüp tek tek denersin.

Hangi dosyalar?

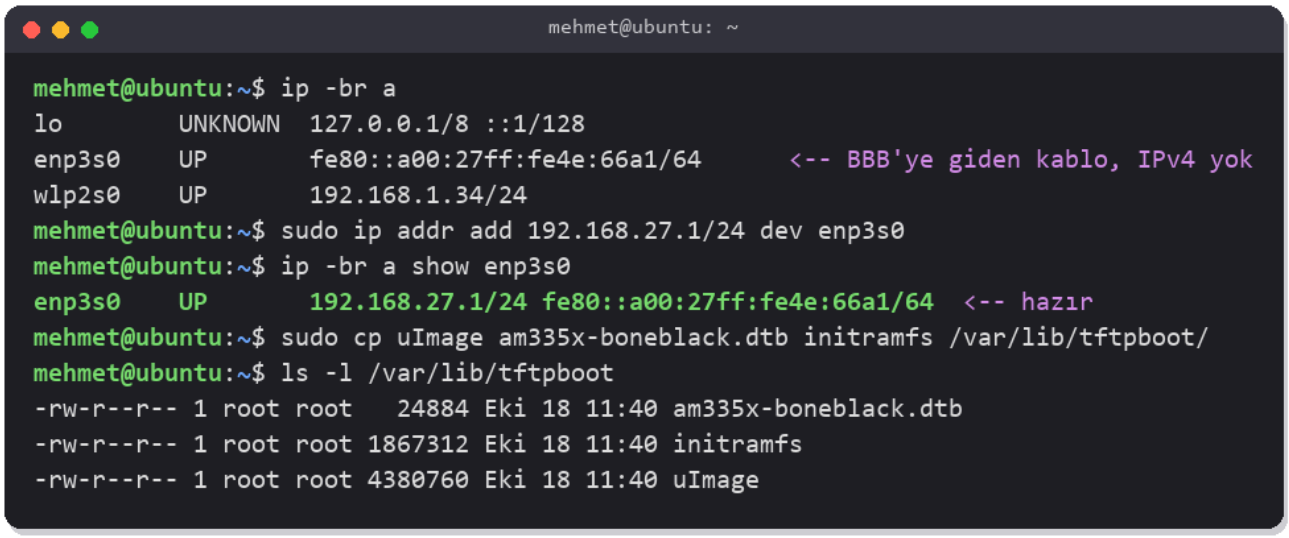
Nerede?	Dosya	Kaynak
SD kart (BOOT)	`MLO` , `u-boot.img`	Kursun güncel `SD-boot` imajları (U-Boot 2022.04) ya da Bölüm 9'da kendi derlediklerin
Host (`/var/lib/tftpboot`)	`uImage` , `am335x-boneblack.dtb` , `initramfs`	Kursun `tftp-boot` klasörü (Ångström kernel 3.8.13) ya da kendi derlediklerin

> **Uyarı**
>
> Kursun güncel `SD-boot` U-Boot'unda `bootdelay=0` . Yani `Hit any key to stop autoboot` geri sayımı yok denecek kadar kısa. U-Boot'u durdurmak için kart açılırken **Space tuşuna basılı tut** ya da art arda bas. Bekleme süresini uzatmak 9.4'teki challenge'ın konusu.

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; arayüz adı (`enp3s0`), sürüm, tarih, boyut ve hız değerleri senin ortamında farklı olacaktır.

Adım 1 — Host'un Ethernet arayüzüne IP ver, dosyaları kopyala

Önce kartın bağlı olduğu Ethernet arayüzünün adını bul. Sonra o arayüze statik IP ver ve boot dosyalarını TFTP klasörüne kopyala:



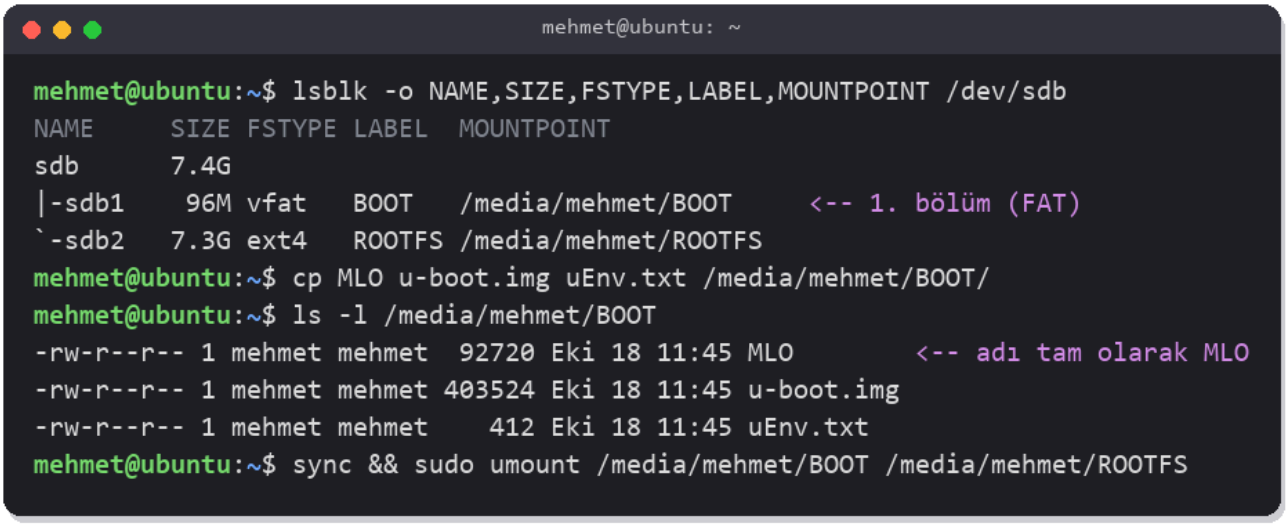

```
ip -br a                                                    # arayüz adını bul (örn. enp3s0, eth0)
sudo ip addr add 192.168.27.1/24 dev enp3s0       # geçici: yeniden başlatınca silinir
sudo cp uImage am335x-boneblack.dtb initramfs /var/lib/tftpboot/
ls -l /var/lib/tftpboot
```

> **İpucu**
>
> `ifconfig` **eski bir komut.** Yeni Ubuntu'larda kurulu bile gelmez (`net-tools` paketi gerekir). Yerine `ip a` / `ip -br a` kullan.
>
> IP adresinin kalıcı olmasını istersen NetworkManager ile bir profil oluştur:
>
> ```
> nmcli con add type ethernet ifname enp3s0 con-name bbb ipv4.method manual ipv4.addresses 192.168.27.1/24
> nmcli con up bbb
> ```

> **Önemli**
>
> **Ubuntu sanal makinedeyse** (VirtualBox/VMware), VM'in ağ ayarını **Bridged** yap ve fiziksel Ethernet kartını seç. NAT modunda kart host'a ulaşamaz.
>
> Güvenlik duvarı açıksa UDP 69'a izin ver: `sudo ufw allow 69/udp`

Adım 2 — SD kartı hazırla (1. bölüm: BOOT)

Dosyalar SD kartın **1. bölümüne (BOOT, FAT)** gider. ROM Code ve U-Boot sadece bu FAT bölümünü okur. 2. bölüm (ROOTFS, ext4) bu yöntemde kullanılmaz, çünkü kök dosya sistemi initramfs olarak ağdan geliyor.




```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT
cp MLO u-boot.img uEnv.txt /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT
```

> **Önemli**
>
> SD karttaki SPL dosyasının adı tam olarak `MLO` olmalı. Seri boot'ta gönderdiğimiz `u-boot-spl.bin` SD kart için uygun değil, çünkü ROM Code SD karttan okurken başında GP başlığı olan `MLO` dosyasını arar. İkisi de U-Boot derlemesinde birlikte çıkar.

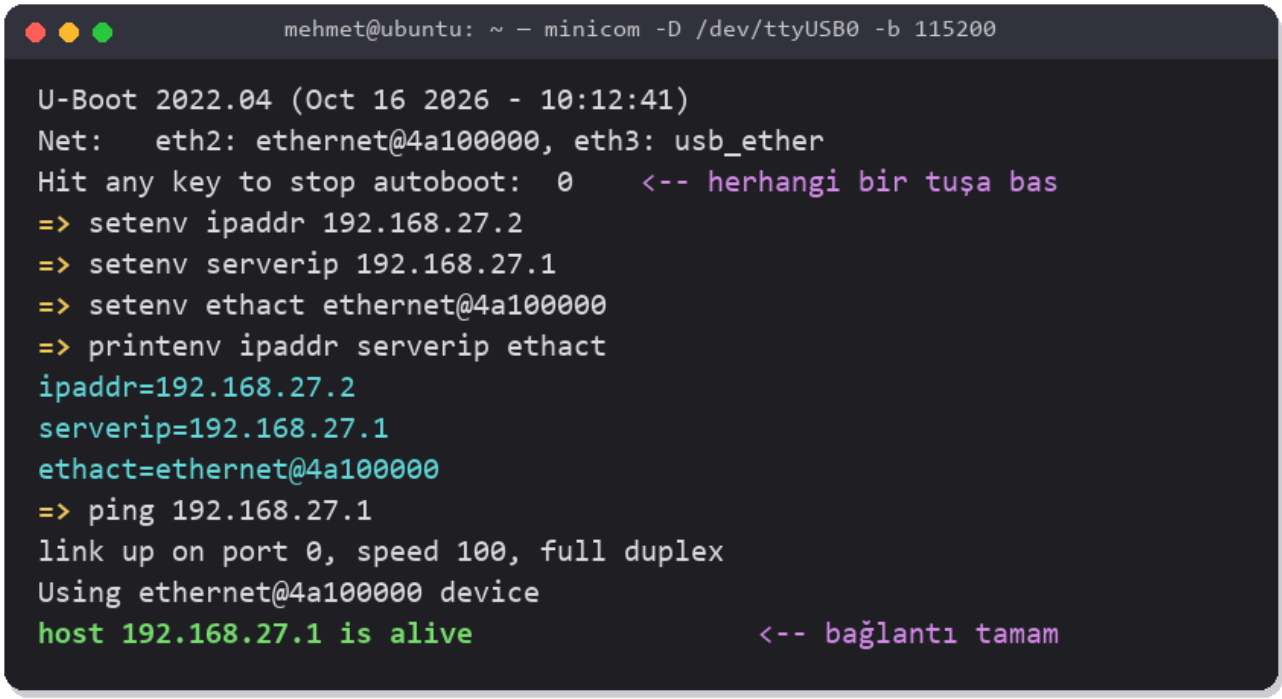
`uEnv.txt`'nin içeriği için iki seçenek var:

- **Elle deneyeceksen (Adım 3–5):** Önceki bölümlerdeki `uEnv.txt`'yi ya hiç koyma ya da U-Boot açılırken bir tuşa basıp autoboot'u durdur. Aksi halde kart kendi kendine SD/eMMC'den boot eder.
- **Otomatik boot için:** 8.6'daki `uEnv.txt`'yi koy.

Kartı SD karttan açmak için **S2 butonuna basılı tutarak** güç ver. Basmazsan kart eMMC'den açılır.

Adım 3 — U-Boot'u durdur, ağ ayarlarını yap, ping at

Minicom açıkken kartı S2'ye basılı tutarak başlat, `Hit any key to stop autoboot` yazısı çıkınca bir tuşa bas:

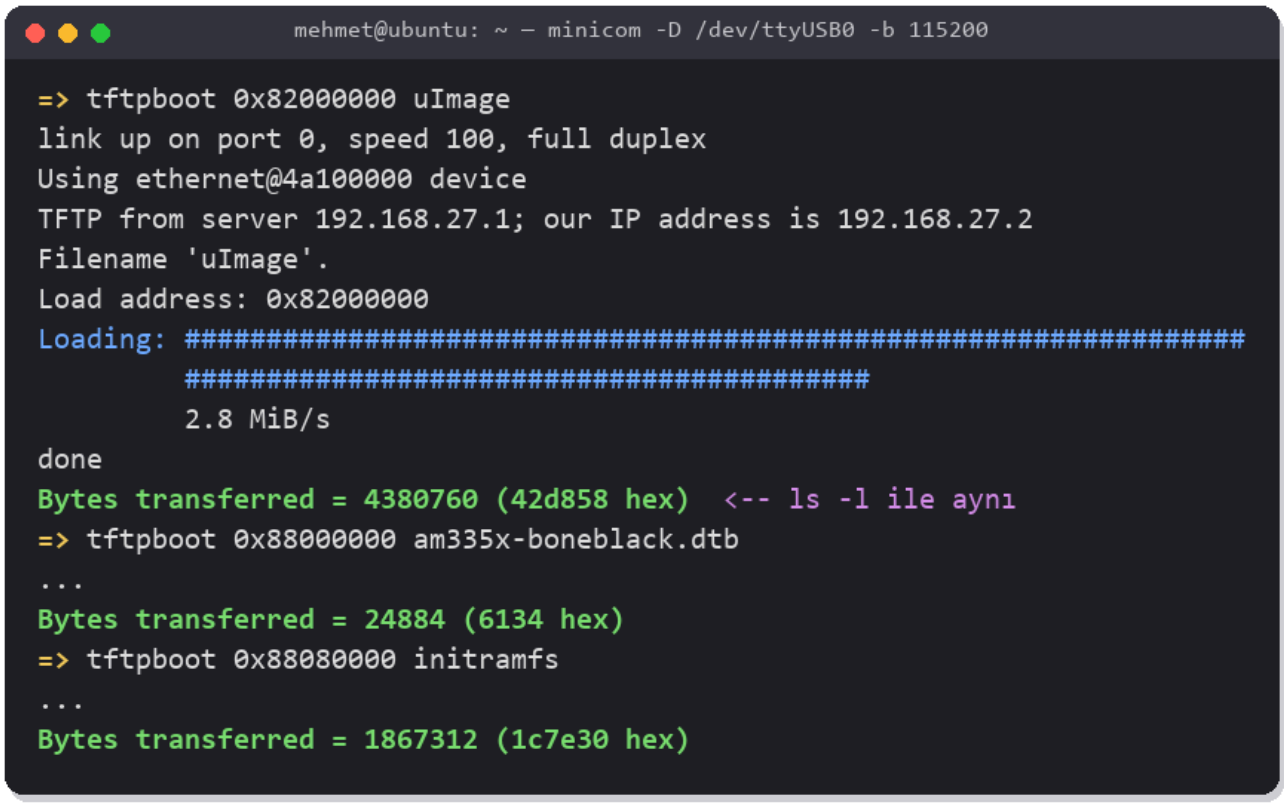



```
=> setenv ipaddr 192.168.27.2
=> setenv serverip 192.168.27.1
=> setenv ethact ethernet@4a100000
=> printenv ipaddr serverip ethact
=> ping 192.168.27.1
```

Komut	Açıklama
`setenv ipaddr ...`	Kartın (istemci) IP adresi
`setenv serverip ...`	TFTP sunucusunun (host) IP adresi. `tftpboot` dosyayı bu adresten ister.
`setenv ethact ethernet@4a100000`	Kullanılacak ağ aygıtı. BBB'nin U-Boot'unda iki aygıt var: Ethernet portu ve USB üzerinden ağ (`usb_ether`). Adlarını açılıştaki `Net:` satırında görürsün. U-Boot 2022.04'te Ethernet'in adı `ethernet@4a100000` , 2016–2017 öncesi sürümlerde `cpsw` 'dir. Çoğu zaman zaten varsayılandır, ama `Using usb_ether device` görürsen bu satır gerekir.
`ping ...`	Kablo ve IP ayarlarını test eder. `is alive` görmelisin.

> **Not**
>
> U-Boot sadece **ping atabilir**, gelen ping'e cevap vermez. Host'tan karta `ping 192.168.27.2` atarsan cevap gelmez, bu normaldir. Testi her zaman kart tarafından yap.

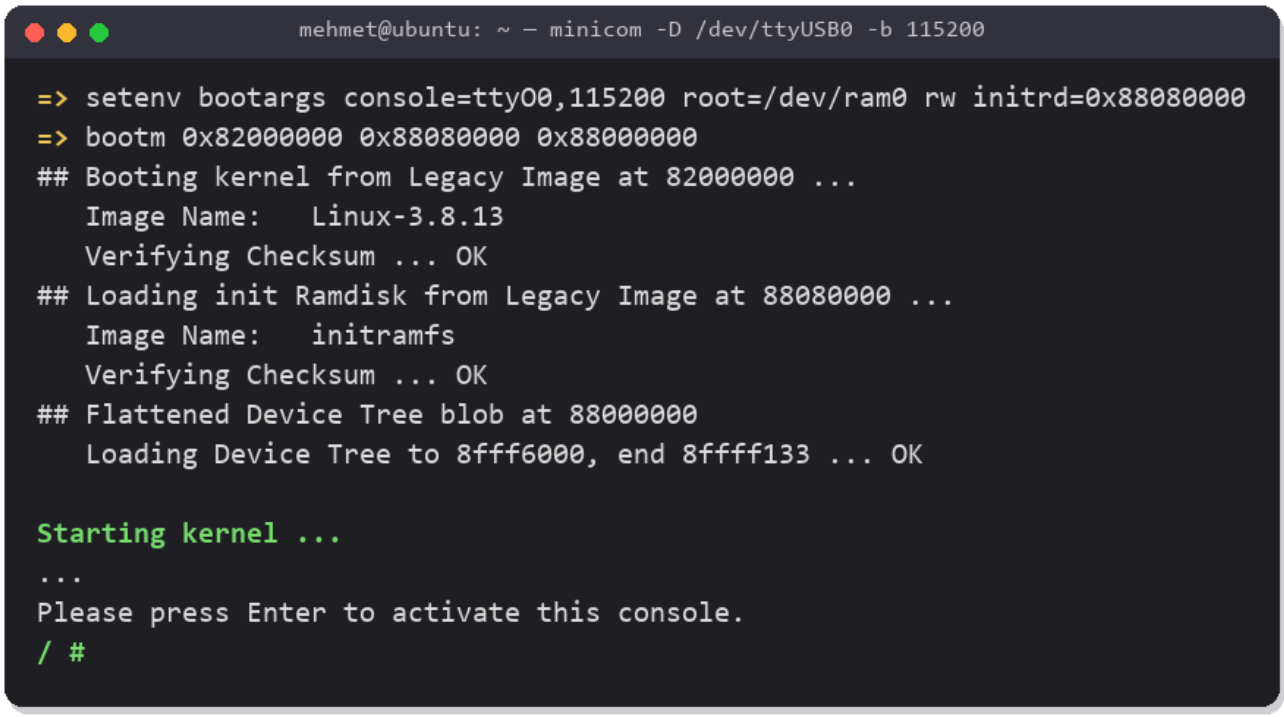
Adım 4 — Dosyaları TFTP ile RAM'e çek




```
=> tftpboot 0x82000000 uImage
=> tftpboot 0x88000000 am335x-boneblack.dtb
=> tftpboot 0x88080000 initramfs
```

Adresler seri boot bölümündekiyle aynı (7.5): kernel `0x82000000`, DTB `0x88000000`, initramfs `0x88080000`. `Bytes transferred` değeri host'taki `ls -l` boyutuyla aynı olmalı. (`tftp` ve `tftpboot` U-Boot'ta aynı komuttur.)

Adım 5 — `bootargs` ayarla ve boot et




```
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
```

`bootm` argüman sırası: **kernel – initramfs – DTB**. Komuta sadece adresler yazılır. `bootargs` 'ın ayrıntıları: 7.8 Adım 7.

Komutların kısa özeti

```
# Host
sudo ip addr add 192.168.27.1/24 dev enp3s0
sudo cp uImage am335x-boneblack.dtb initramfs /var/lib/tftpboot/

# U-Boot (minicom)
setenv ipaddr 192.168.27.2
setenv serverip 192.168.27.1
ping 192.168.27.1
tftpboot 0x82000000 uImage
tftpboot 0x88000000 am335x-boneblack.dtb
tftpboot 0x88080000 initramfs
setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
bootm 0x82000000 0x88080000 0x88000000
```

8.6 Yol 2 — uEnv.txt ile Otomatik TFTP Boot

Yol 1'deki U-Boot komutlarının hepsi `uEnv.txt`'ye yazılır. Kart açılınca U-Boot bu dosyayı SD kartın BOOT bölümünden okur ve `uenvcmd` içindeki komutları kendisi çalıştırır. Böylece **minicom'a hiçbir şey yazmana gerek kalmaz**; minicom'u sadece açılışı izlemek için kullanırsın.

Host tarafındaki hazırlık (IP verme, dosyaları `/var/lib/tftpboot`'a kopyalama) iki yolda da aynı. Yol 2 sadece U-Boot'ta elle yazdığın kısmı (Yol 1, Adım 3–5) otomatik yapar.

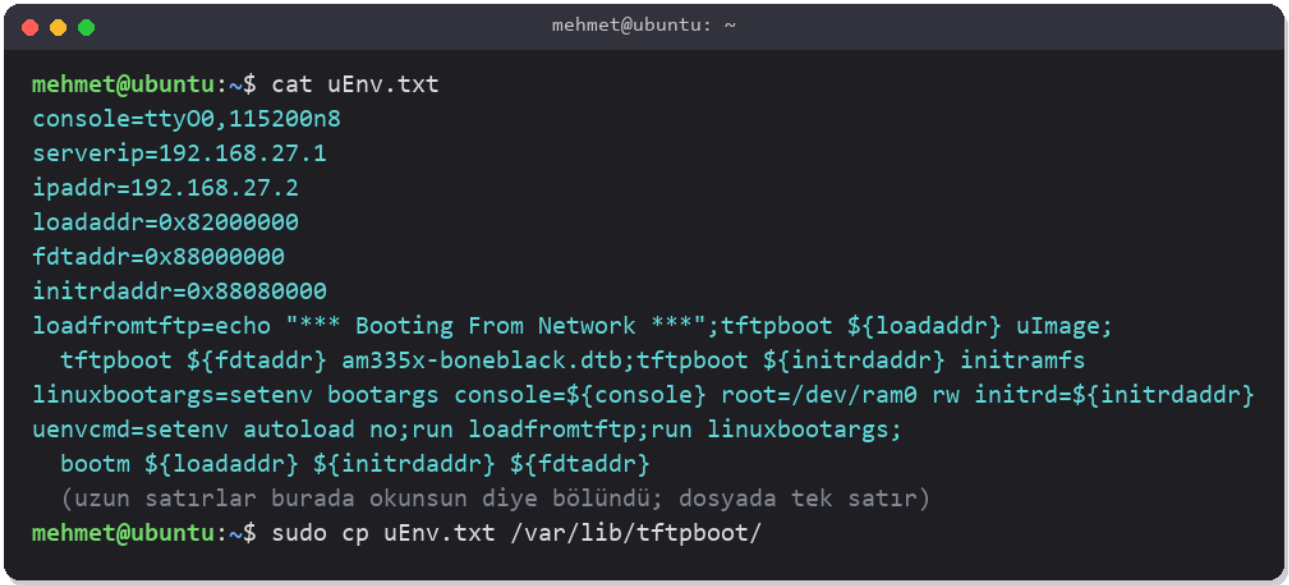
Adım 1 — Host'u hazırla

Yol 1'in Adım 1'i ile aynı: Ethernet arayüzüne `192.168.27.1` ver, `uImage`, `am335x-boneblack.dtb`, `initramfs` dosyalarını `/var/lib/tftpboot`'a kopyala ve `systemctl status tftpd-hpa` ile servisin çalıştığını kontrol et.

Adım 2 — `uEnv.txt` dosyasını yaz

Host'ta aşağıdaki içerikle bir `uEnv.txt` oluştur. Her değişken **tek satırda** olmalı; satırları bölme.

```
console=ttyO0,115200n8
serverip=192.168.27.1
ipaddr=192.168.27.2
loadaddr=0x82000000
fdtaddr=0x88000000
initrdaddr=0x88080000
loadfromtftp=echo "*** Booting From Network ***";tftpboot ${loadaddr} uImage;tftpboot ${fdtaddr}
am335x-boneblack.dtb;tftpboot ${initrdaddr} initramfs
linuxbootargs=setenv bootargs console=${console} root=/dev/ram0 rw initrd=${initrdaddr}
uenvcmd=setenv autoload no;run loadfromtftp;run linuxbootargs;bootm ${loadaddr} ${initrdaddr}
${fdtaddr}
```



Satır	Açıklama	Yol 1'deki karşılığı
`console=ttyO0,115200n8`	Kendi tanımladığımız bir değişken; aşağıda `${console}` ile kullanılıyor. `n8`: parity yok, 8 bit.	`bootargs` içindeki `console=...`
`serverip=...`	TFTP sunucusunun (host) IP'si	`setenv serverip 192.168.27.1`
`ipaddr=...`	Kartın IP'si	`setenv ipaddr 192.168.27.2`
`loadaddr`, `fdtaddr`, `initrdaddr`	Kernel, DTB ve initramfs'in RAM adresleri. Adresleri bir kere yazıp aşağıda `${...}` ile kullanıyoruz; değiştirmek gerekirse tek yerden değişir.	`0x82000000`, `0x88000000`, `0x88080000`
`loadfromtftp=...`	Bir **makro**: ekrana mesaj basar, sonra 3 dosyayı TFTP ile RAM'e çeker. `run loadfromtftp` yazılınca çalışır.	3 adet `tftpboot` komutu
`linuxbootargs=...`	Bir makro: `bootargs`'ı ayarlar	`setenv bootargs ...`
`uenvcmd=...`	U-Boot'un `uEnv.txt`'yi okuduktan sonra **otomatik çalıştırdığı** komut. Sırayla: makroları çalıştır, sonra `bootm`.	Yazdığın bütün komutların sırası
`setenv autoload no`	`dhcp` / `bootp` komutlarının IP aldıktan sonra kendiliğinden TFTP ile dosya çekmesini engeller. Statik IP kullandığımız için şart değil, ama zararı yok ve ileride `dhcp` eklersen sürpriz yaşatmaz.	—

> **Uyarı**
>
> En sık yapılan iki hata:
>
> - `setenv booargs ...` gibi bir yazım hatası. U-Boot hata vermez, `booargs` adında yeni bir değişken oluşturur. Kernel'e gerçek `bootargs` gitmez, `Starting kernel ...` sonrası ekran boş kalabilir veya kernel rootfs bulamadan panic verir.
> - Makroyu bir adla tanımlayıp başka bir adla çağırmak (ör. `loadftp=...` tanımlayıp `run loadfromtftp` yazmak). U-Boot `## Error: "loadfromtftp" not defined` der ve dosyalar hiç çekilmez. Tanımladığın ad ile çağırdığın ad **birebir aynı** olmalı.

> **Önemli**
>
> Komutu `bootcmd` içine değil **`uenvcmd`** içine yaz. BBB'nin U-Boot'u `uEnv.txt`'yi kendi `bootcmd`'si çalışırken okur ve ardından `uenvcmd`'yi çalıştırır. Bu yüzden dosyadaki `bootcmd` o açılışı etkilemez (6.3).

Adım 3 — `uEnv.txt`'yi SD kartın BOOT bölümüne koy

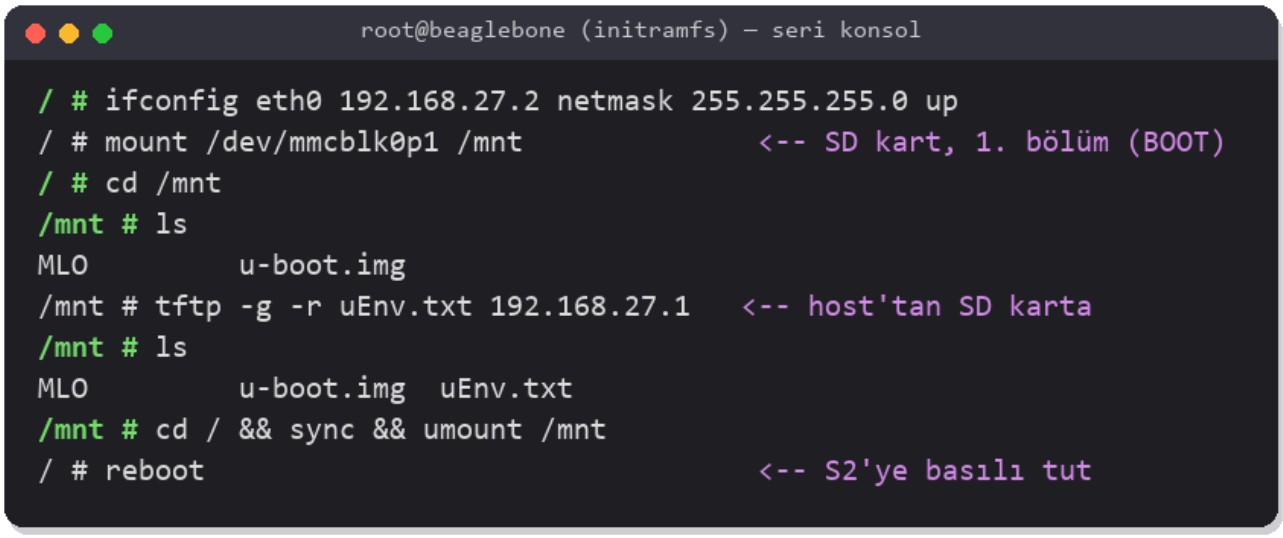
İki seçenek var.

a) Kart okuyucuyla (en basiti): SD kartı bilgisayara tak ve dosyayı 1. bölüme kopyala.

```
cp uEnv.txt /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT
```

b) SD kartı çıkarmadan, TFTP ile: Kart Linux'a açıkken (örneğin Yol 1 ile açtıktan sonra) dosyayı host'tan doğrudan SD karta çekebilirsin. Bunun için BusyBox'taki `tftp -g -r uEnv.txt 192.168.27.1` komutu kullanılır (8.7).

Önce host'ta dosyayı TFTP klasörüne koy (`sudo cp uEnv.txt /var/lib/tftpboot/`). Sonra kartta:




```
ifconfig eth0 192.168.27.2 netmask 255.255.255.0 up
mount /dev/mmcblk0p1 /mnt
cd /mnt
tftp -g -r uEnv.txt 192.168.27.1
ls
cd / && sync && umount /mnt
reboot
```

> **Önemli**
>
> `tftp` dosyayı **bulunduğun klasöre** kaydeder. Önce SD kartın BOOT bölümünü bağlayıp (`mount`) o klasöre girmezsen dosya initramfs'e, yani RAM'e yazılır ve yeniden başlatınca kaybolur. `mmcblk0` SD kart, `mmcblk1` eMMC'dir; `p1` 1. bölüm demek.

Adım 4 — Kartı başlat ve hiçbir tuşa basma

S2'ye basılı tutarak güç ver (veya `reboot` sırasında basılı tut). Bu sefer `Hit any key to stop autoboot` çıkınca **dokunma**; U-Boot `uEnv.txt`'yi okur ve `uenvcmd`'yi çalıştırır:

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200

U-Boot 2022.04 (Oct 16 2026 - 10:12:41)
Hit any key to stop autoboot:  0
mmc0 is current device
reading uEnv.txt                          <-- SD kart, 1. bölüm
412 bytes read in 4 ms (100.6 KiB/s)
Importing environment from mmc ...
Running uenvcmd ...                       <-- bizim komutumuz
*** Booting From Network ***
Using ethernet@4a100000 device
TFTP from server 192.168.27.1; our IP address is 192.168.27.2
Filename 'uImage'.
Bytes transferred = 4380760 (42d858 hex)
Filename 'am335x-boneblack.dtb'.
Bytes transferred = 24884 (6134 hex)
Filename 'initramfs'.
Bytes transferred = 1867312 (1c7e30 hex)
## Booting kernel from Legacy Image at 82000000 ...
Starting kernel ...
```

Ekranda `Running uenvcmd ...`, ardından `*** Booting From Network ***` ve üç tane `Bytes transferred` görmelisin.

Adım 5 — Doğrula

Linux açılınca `bootargs`'ın gerçekten kernel'e gittiğini kontrol et:

```
root@beaglebone (initramfs) — seri konsol

/ # cat /proc/cmdline
console=ttyO0,115200n8 root=/dev/ram0 rw initrd=0x88080000  <-- linuxbootargs çalışmış
```

```
cat /proc/cmdline
```

Burada `console=ttyO0,115200n8 root=/dev/ram0 ...` görmüyorsan `linuxbootargs` çalışmamış demektir (değişken adını kontrol et).

Adım 6 — S2'ye basmadan açılış (Challenge)

Kursun bu bölümdeki challenge'ı: *TFTP boot'u `uEnv.txt` ile otomatikleştir; kart güç alınca ya da reset'e basılınca imajlar kendiliğinden insin ve **S2'ye basmadan** açılsın.*

S2'ye basmadan güç verince ROM Code **eMMC**'deki MLO/U-Boot'u çalıştırır (3.5). Çözüm, eMMC'deki U-Boot'un `uEnv.txt`'yi nerede aradığına bağlı:

1. **Önce kontrol et.** U-Boot'u durdur ve ortamına bak:

   ```
   => printenv mmcdev bootenvfile bootenv
   => printenv bootcmd
   ```

   `mmcdev=0` ve `bootcmd` içinde `loadbootenv` / `envboot` gibi bir adım varsa, U-Boot açılışta önce **SD kartın 1. bölümündeki** `uEnv.txt`'yi okur (`mmc 0` = SD kart).

2. **Bu durumda yapman gereken tek şey** Adım 3'teki `uEnv.txt`'yi SD kartın BOOT bölümüne koymak. Kart eMMC'den açılsa bile SD karttaki dosyayı okur ve `uenvcmd` ile TFTP boot'u başlatır. S2'ye basmana gerek kalmaz.

3. eMMC'deki U-Boot dosyayı başka bir yerde arıyorsa (BeagleBoard'un yeni Debian imajları gibi, ör. eMMC'nin rootfs'indeki `/boot/uEnv.txt`), iki seçenek var: - Komutları U-Boot'un okuduğu dosyaya yaz, ya da - eMMC'deki `MLO`'yu devre dışı bırak. ROM Code eMMC'de geçerli MLO bulamayınca SD karta geçer (3.1).

> **Dikkat**
>
> eMMC'deki `MLO`'yu silmek, eMMC'deki sistemin kendi başına açılmamasına yol açar. Geri dönmek için Bölüm 5'teki gibi eMMC'yi yeniden flash'lamak gerekir. Önce 1. ve 2. maddeyi dene.

Yol 2'nin özeti

Artık geliştirme döngüsü şu kadar:

```
# Host: kernel'i / DTB'yi yeniden derledin
sudo cp uImage am335x-boneblack.dtb /var/lib/tftpboot/
# Kartın reset butonuna bas → U-Boot yeni dosyaları kendisi çeker ve boot eder
```

> **İpucu**
>
> **Yol 2'yi kullanınca Yol 1'deki komutlara gerek kalmaz mı?** Her açılışta yazmana gerek kalmaz, ama komutları bilmek yine de gerekli. Bir şey bozulduğunda U-Boot'u durdurup `run loadfromtftp`, `printenv bootargs` gibi komutları tek tek çalıştırarak hangi adımın bozulduğunu bulursun. `uEnv.txt` sadece aynı komutları senin yerine yazar.

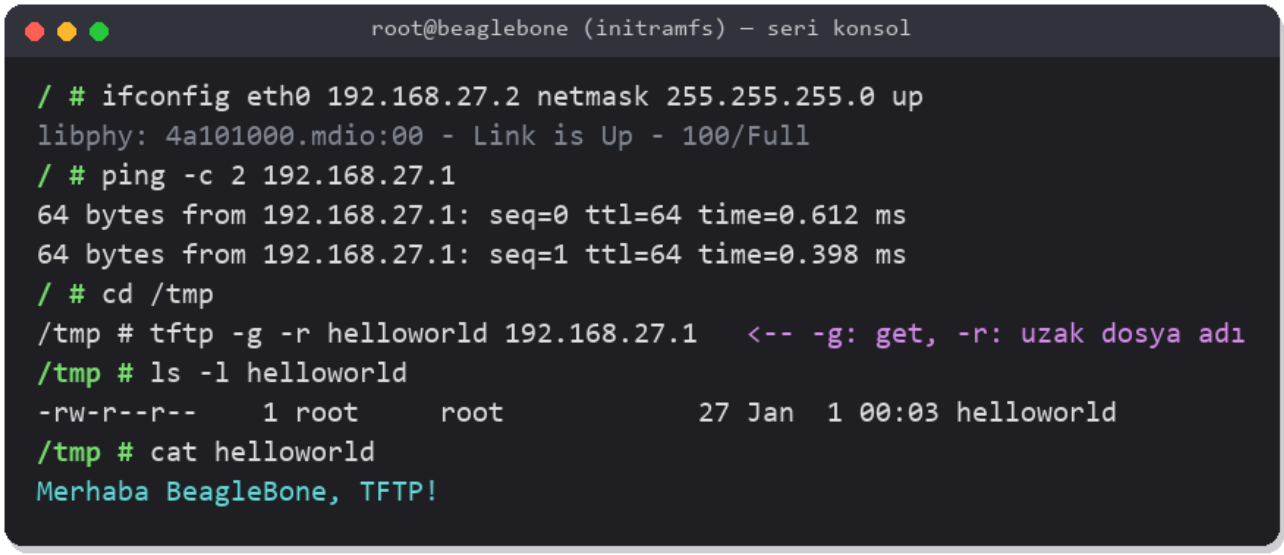
8.7 Linux Açıldıktan Sonra TFTP ile Dosya Çekme

Kart Linux'a açıldıktan sonra BusyBox'taki `tftp` istemcisiyle host'tan dosya alabilirsin.

Host PC'de:

```
echo "Merhaba BeagleBone, TFTP!" > helloworld     # veya: vi helloworld
sudo cp helloworld /var/lib/tftpboot/
```

BeagleBone'da (seri konsol):




```
ifconfig eth0 192.168.27.2 netmask 255.255.255.0 up
ping -c 2 192.168.27.1
cd /tmp
tftp -g -r helloworld 192.168.27.1
cat helloworld
```

Komut / seçenek	Açıklama
`ifconfig eth0 ... up`	U-Boot'un IP ayarı kernel'e **geçmez**. Kernel ağ kartını sıfırdan başlatır, bu yüzden Linux'ta IP'yi her açılışta tekrar vermek gerekir. BusyBox'ta `ifconfig` vardır; `ip addr add 192.168.27.2/24 dev eth0` ve `ip link set eth0 up` da çalışır.
`tftp`	Tek başına yazarsan kullanım yardımını gösterir.
`-g`	get: sunucudan dosya al
`-r helloworld`	remote: sunucudaki dosya adı
`-l isim`	local: kartta farklı bir adla kaydet (isteğe bağlı)
`192.168.27.1`	TFTP sunucusunun IP'si

Komut / seçenek	Açıklama
`-p -l dosya 192.168.27.1`	put: karttan host'a dosya gönder (`--create` ayarı sayesinde)

> **Not**
>
> initramfs RAM'de çalışır. Kartta oluşturduğun veya indirdiğin dosyalar **yeniden başlatınca kaybolur**.

8.8 Challenge: Her Kaynaktan Boot Edebilen Genel uEnv.txt

Görev

Tek bir `uEnv.txt` yazacağız. Bu dosya kaynakları sırayla deneyecek ve ilk çalışan kaynaktan boot edecek: **eMMC → SD → TFTP → NFS**. eMMC'de boot dosyaları yoksa SD karta, SD kart yoksa TFTP'ye geçecek.

Çözüm fikri

U-Boot'un komut satırı (hush) `if ... then ... fi` ve `&&` destekler. Komutların başarılı olup olmadığına göre karar verebiliriz:

- `load` dosyayı bulamazsa hata döner.
- `mmc rescan` kart yoksa hata döner.
- `tftpboot` dosya yoksa ya da sunucu cevap vermezse hata döner.
- `bootm` başarılı olursa **geri dönmez**, kernel çalışır. Başarısız olursa (bozuk imaj vb.) bir sonraki satıra geçilir.

Her kaynak için bir `try_...` makrosu yazıyoruz. Makro önce dosyaları yüklemeyi dener; yükleme başarılıysa `bootargs`'ı ayarlayıp `bootm` çalıştırır. Başarısızsa hiçbir şey yapmaz ve `uenvcmd` sıradaki makroya geçer:

```
uenvcmd
  ├─ try_emmc : eMMC'de uImage + dtb var mı? ─ evet → bootm (rootfs: eMMC p2)
  │                                          └ hayır ↓
  ├─ try_sd   : SD kart takılı ve imajlar var mı? ─ evet → bootm (rootfs: SD p2)
  │                                               └ hayır ↓
  ├─ try_tftp : TFTP'den uImage + dtb + initramfs ─ evet → bootm (rootfs: RAM)
  │                                               └ hayır ↓
  ├─ try_nfs  : TFTP'den uImage + dtb ─ evet → bootm (rootfs: NFS, host'ta)
  │                                   └ hayır ↓
  └─ "Hicbir kaynaktan boot edilemedi"
```

Çözüm: `uEnv.txt`

Her değişken **tek satırda** olmalı. Bu çözümde varsayılan dosya düzeni şu: her MMC aygıtında 1. bölüm (FAT, BOOT) içinde `uImage` ve `am335x-boneblack.dtb`, 2. bölüm (ext4) rootfs.

```
console=ttyO0,115200n8
ipaddr=192.168.27.2
serverip=192.168.27.1
netretry=once
loadaddr=0x82000000
fdtaddr=0x88000000
initrdaddr=0x88080000
nfspath=/srv/nfs/bbb,nolock,wsize=1024,rsize=1024
mmc_load=mmc dev ${mmcnum} && mmc rescan && load mmc ${mmcnum}:1 ${loadaddr} uImage && load mmc
${mmcnum}:1 ${fdtaddr} am335x-boneblack.dtb
mmc_args=setenv bootargs console=${console} root=${mmcroot} rw rootwait
emmc_root=if mmc dev 0 && mmc rescan; then setenv mmcroot /dev/mmcblk1p2; else setenv mmcroot
/dev/mmcblk0p2; fi
try_emmc=echo "[1/4] eMMC deneniyor ..."; setenv mmcnum 1; if run mmc_load; then run emmc_root;
run mmc_args; bootm ${loadaddr} - ${fdtaddr}; fi
try_sd=echo "[2/4] SD kart deneniyor ..."; setenv mmcnum 0; if run mmc_load; then setenv mmcroot
/dev/mmcblk0p2; run mmc_args; bootm ${loadaddr} - ${fdtaddr}; fi
net_load=tftpboot ${loadaddr} uImage && tftpboot ${fdtaddr} am335x-boneblack.dtb
try_tftp=echo "[3/4] TFTP (initramfs) deneniyor ..."; if run net_load && tftpboot ${initrdaddr}
initramfs; then setenv bootargs console=${console} root=/dev/ram0 rw initrd=${initrdaddr}; bootm
${loadaddr} ${initrdaddr} ${fdtaddr}; fi
try_nfs=echo "[4/4] TFTP + NFS deneniyor ..."; if run net_load; then setenv bootargs
console=${console} root=/dev/nfs rw nfsroot=${serverip}:${nfspath}
ip=${ipaddr}:${serverip}::255.255.255.0::eth0:off rootwait; bootm ${loadaddr} - ${fdtaddr}; fi
uenvcmd=if test "${generic_tried}" != "yes"; then setenv generic_tried yes; setenv autoload no;
run try_emmc; run try_sd; run try_tftp; run try_nfs; echo "!!! Hicbir kaynaktan boot edilemedi";
fi
```

Satırların açıklaması

Satır	Açıklama
`console`, `ipaddr`, `serverip`, `*addr`	Yol 2'deki ile aynı: konsol, IP'ler ve RAM adresleri
`netretry=once`	**Önemli.** Varsayılan olarak `tftpboot` sunucu cevap vermezse **sonsuza kadar** tekrar dener (`Retry count exceeded; starting again`). O zaman NFS adımına hiç geçilmez. `once` ile bir kere dener, olmazsa hata döner.
`nfspath`	NFS rootfs'in host'taki yolu ve NFS seçenekleri: `nolock` (kartta kilit servisi yok), `wsize/rsize=1024` (küçük paket boyutu, yavaş ama güvenilir)

Satır	Açıklama
`mmc_load`	Ortak MMC yükleyici. `${mmcnum}` hangi aygıt olduğunu söyler (`1` = eMMC, `0` = SD). Dört komut `&&` ile bağlı: biri başarısız olursa geri kalanlar çalışmaz ve makro hata döner.
`mmc_args`	MMC için `bootargs`. Rootfs, `${mmcroot}` ile belirtilen bölüm.
`emmc_root`	Kernel'de eMMC'nin adını bulur (aşağıdaki uyarıya bak). SD kart varsa eMMC `mmcblk1`, yoksa `mmcblk0` olur.
`try_emmc` , `try_sd`	`mmc_load` başarılıysa → `bootargs` ayarla → `bootm` . `-` : initramfs yok.
`net_load`	TFTP ile kernel + DTB. TFTP ve NFS adımları ortak kullanır.
`try_tftp`	Kernel + DTB + **initramfs**. Rootfs RAM'de (Yol 2 ile aynı).
`try_nfs`	Kernel + DTB TFTP'den gelir, rootfs NFS ile host'tan bağlanır. `ip=...` kernel'e kendi IP'sini verir; U-Boot'un IP'si kernel'e geçmediği için NFS'te şart. Bu adım, host'ta `initramfs` dosyası yoksa devreye girer.
`uenvcmd`	Makroları sırayla çağırır. `generic_tried` bir **koruma bayrağı**: U-Boot sürümüne göre varsayılan `bootcmd` önce SD'de, sonra eMMC'de `uEnv.txt` arayabilir, yani `uenvcmd` iki kez çalışabilir. Bayrak sayesinde zincir sadece bir kez denenir.

> **Uyarı**
>
> **eMMC'nin kernel'deki adı değişebilir.** U-Boot'ta eMMC her zaman `mmc 1` 'dir. Ama eski kernel'lerde (3.8.13 gibi) SD kart **takılı değilse** eMMC `mmcblk0` olarak görünür, takılıysa `mmcblk1` olarak görünür. `emmc_root` makrosu SD kartın olup olmadığına bakıp doğru adı seçer. Yeni kernel'lerde DTB'deki `aliases` ile bu sabitlenebilir.

Dosya nereye konur?

Burada da bir tavuk-yumurta durumu var: `uEnv.txt` 'nin kendisi yerel bir depolamada olmalı. U-Boot onu önce SD'de (`mmcdev 0`), bulamazsa eMMC'de (`mmcdev 1`) arar.

Senaryo	`uEnv.txt` nerede olmalı?
SD kart her zaman takılı	SD kartın 1. bölümü (BOOT) yeterli
"SD kart yoksa TFTP'ye geç" durumu da çalışsın	**eMMC'nin 1. bölümüne de** aynı dosyayı koy

En güvenlisi aynı `uEnv.txt` 'yi hem SD'nin hem eMMC'nin BOOT bölümüne koymak.

Örnek açılış: eMMC'de imaj yok, SD kart takılı değil

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 (SD kart takılı değil)

U-Boot 2022.04 (Oct 16 2026 - 10:12:41)
Card did not respond to voltage select!     <-- mmcdev 0 (SD) yok
SD/MMC found on device 1                    <-- eMMC'ye geçti
reading uEnv.txt
Running uenvcmd ...
[1/4] eMMC deneniyor ...
** Unable to read file uImage **           <-- eMMC'de imaj yok
[2/4] SD kart deneniyor ...
Card did not respond to voltage select!     <-- SD yok
[3/4] TFTP (initramfs) deneniyor ...
Using ethernet@4a100000 device
TFTP from server 192.168.27.1; our IP address is 192.168.27.2
Bytes transferred = 4380760 (42d858 hex)
Bytes transferred = 24884 (6134 hex)
Bytes transferred = 1867312 (1c7e30 hex)
## Booting kernel from Legacy Image at 82000000 ...
Starting kernel ...                         <-- TFTP'den açıldı
```

Nasıl test edilir?

Her satırda tek bir şeyi değiştirip kartı yeniden başlat. Hangi `[n/4]` satırından sonra `Starting kernel ...` geldiğine bak.

Test	Ne yaparsın?	Beklenen
1	Her şey yerinde	`[1/4] eMMC` → boot
2	eMMC'deki `uImage`'ın adını değiştir (`mv uImage uImage.bak`)	eMMC başarısız → `[2/4] SD` → boot
3	SD kartı çıkar (`uEnv.txt` eMMC'de olmalı)	eMMC, SD başarısız → `[3/4] TFTP` → boot
4	Host'ta `/var/lib/tftpboot/initramfs`'i taşı	… → `[4/4] NFS` → boot (NFS sunucusu kurulu olmalı)
5	Ethernet kablosunu da çıkar	`!!! Hicbir kaynaktan boot edilemedi`, U-Boot komut satırı

Bir adımı elle denemek istersen U-Boot'u durdurup `uEnv.txt`'yi yükle (`run loadbootenv; run importbootenv`), sonra makroları tek tek çalıştır: `run try_sd`, `printenv bootargs` gibi.

Sınırlamalar

- Bu zincir sadece **U-Boot aşamasındaki** hataları yakalar (dosya yok, kart yok, sunucu yok). Kernel açılıp sonra panic verirse (yanlış rootfs vb.) U-Boot'a geri dönülmez.
- `MLO` ve `u-boot.img` yine eMMC'de ya da SD'de olmalı. ROM Code BBB'de ağdan boot edemez (8.2).
- `uEnv.txt` içindeki U-Boot `echo` mesajlarında Türkçe karakter kullanma; seri konsolda bozuk görünebilir.

8.9 Özet

- Host: `tftpd-hpa` kur, `/var/lib/tftpboot`'u hazırla, Ethernet arayüzüne `192.168.27.1` ver, dosyaları kopyala.
- SD kart: sadece `MLO`, `u-boot.img`, `uEnv.txt`. Kernel, DTB ve initramfs ağdan gelir.
- U-Boot: `setenv ipaddr` / `serverip` → `ping` → `tftpboot` (3 dosya) → `setenv bootargs` → `bootm kernel initrd dtb`.
- Bu komutları `uEnv.txt`'deki `uenvcmd`'ye yazarsan açılış otomatik olur; geliştirme döngüsü "dosyayı kopyala, reset'e bas"a iner.
- U-Boot'taki IP ayarı kernel'e geçmez; Linux'ta `eth0`'a IP'yi yeniden ver.

8.10 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 (temsili hatalar)

=> tftpboot 0x82000000 uImage
*** ERROR: `serverip' not set              <-- setenv serverip ...
=> tftpboot 0x82000000 uImage
Using usb_ether device                     <-- setenv ethact ...
=> tftpboot 0x82000000 uImage
TFTP from server 192.168.27.1; our IP address is 192.168.27.2
Loading: T T T T T T T T T T
Retry count exceeded; starting again       <-- IP/kablo/firewall/VM ağı
=> tftpboot 0x82000000 uimage
TFTP error: 'File not found' (1)           <-- ad/klasör yanlış (büyük-küçük harf!)
=> ping 192.168.27.1
ping failed; host 192.168.27.1 is not alive <-- host IP'si yok / kablo
```

Hata	Sebep	Çözüm
`*** ERROR: 'serverip' not set`	Sunucu IP'si tanımlı değil	`setenv serverip 192.168.27.1`
`Using usb_ether device`	U-Boot USB ağını kullanmaya çalışıyor	`setenv ethact ethernet@4a100000` (eski U-Boot'larda `setenv ethact cpsw`)

Hata	Sebep	Çözüm
`Loading: T T T T ...` / `Retry count exceeded`	Paket gidiyor ama cevap gelmiyor	Host IP'si ve alt ağ doğru mu? `systemctl status tftpd-hpa` çalışıyor mu? Firewall (`sudo ufw allow 69/udp`)? VM ise Bridged mi? Sunucu port 69'da mı?
`TFTP error: 'File not found' (1)`	Dosya `/var/lib/tftpboot` içinde değil ya da adı yanlış	`ls -l /var/lib/tftpboot`. Linux'ta büyük-küçük harf farklıdır: `uImage` ≠ `uimage`
`TFTP error: 'Permission denied'`	Dosya okunamıyor	`sudo chmod -R 777 /var/lib/tftpboot`
`ping failed; host ... is not alive`	Host'un o arayüzde IP'si yok / kablo takılı değil	`ip -br a` ile kontrol et, Ethernet ışıklarına bak
Kart U-Boot'a gelmeden eMMC'den açılıyor	S2'ye basılmadı	Güç verirken S2'yi basılı tut
Kart SD'den hiç açılmıyor	SPL dosyasının adı yanlış / BOOT bölümü FAT değil	Dosya adı `MLO` olmalı; 1. bölüm FAT ve boot bayrağı açık olmalı
`bootm` sonrası `Wrong Ramdisk Image Format`	initramfs'te U-Boot başlığı yok	`mkimage -A arm -T ramdisk -C gzip -d initramfs.cpio.gz initramfs` (7.10 (g))
`## Error: "loadfromtftp" not defined`	`uenvcmd` içinde çağrılan makro adı, tanımlanan adla aynı değil	`uEnv.txt`'de iki adı birebir aynı yaz
`Running uenvcmd ...` hiç çıkmıyor	`uEnv.txt` okunmadı: BOOT bölümünde değil, adı yanlış (`uenv.txt`) veya kart eMMC'den açıldı	Dosyayı 1. bölüme koy, S2'ye basılı tutarak aç
`Starting kernel ...` sonrası ekran boş	`bootargs` ayarlanmamış (ör. `booargs` yazım hatası) ya da `console` yanlış	U-Boot'ta `printenv bootargs` ile kontrol et

Bölüm 9 — Cross Toolchain, SPL ve U-Boot Derleme

> **Bu bölümde:**
>
> - Cross compile'ın ne olduğunu ve neden gerektiğini öğreneceksin.
> - Linaro cross toolchain'ini kurup `PATH`'e ekleyeceksin.
> - U-Boot 2022.04 kaynağını indirip BeagleBone Black için yapılandıracak ve derleyeceksin: `MLO`, `u-boot-spl.bin`, `u-boot.img`.
> - Kendi derlediğin U-Boot ile kartı açacaksın.
> - Challenge: U-Boot'un açılıştaki bekleme süresini kaynak koddan 10 saniyeye çıkaracaksın.
>
> **Ön koşul:** Bölüm 3 — SPL ve U-Boot'un görevleri, Bölüm 4 — SD karta dosya koyma.

9.1 Bu Bölümde Neden Toolchain Kuruyoruz?

Şimdiye kadar kullandığımız `MLO`, `u-boot.img`, `uImage` hazır dosyalardı. Bu bölümde **SPL ve U-Boot'u kaynak koddan kendimiz derleyeceğiz**. Bunun için ARM kodu üretebilen bir derleyiciye ihtiyacımız var.

	Host PC	**BeagleBone Black**
İşlemci	x86_64 (Intel/AMD)	ARM Cortex-A8 (AM335x)
Komut seti	x86 makine kodu	ARM makine kodu

Ubuntu'daki `gcc` x86 için kod üretir. Onunla derlenen U-Boot'u ROM Code yükler ama ARM işlemci çalıştıramaz.

> **Cross compile:** Bir mimaride (x86) çalışıp **başka bir mimari** (ARM) için çalıştırılabilir kod üretmek. Bunu yapan derleyici ve yardımcı araçların toplamına **cross toolchain** denir.

Neden kartın üzerinde derlemiyoruz?

- Kart yavaş ve RAM'i az. U-Boot dakikalar, kernel saatler sürer.
- Tavuk-yumurta: kartta derleme yapmak için önce çalışan bir Linux gerekir. O Linux'u açacak olan da derleyeceğimiz U-Boot.

Toolchain'in içinde ne var?

Araç	**Görevi**
`arm-linux-gnueabihf-gcc`	C kodunu ARM assembly'ye çevirir

Araç	Görevi
`arm-linux-gnueabihf-as`	Assembly'yi makine koduna çevirir (binutils)
`arm-linux-gnueabihf-ld`	Parçaları birleştirir; kodu linker script'e göre SRAM/DDR adreslerine yerleştirir
`arm-linux-gnueabihf-objcopy`	ELF dosyasından ham binary üretir (`u-boot.bin` , `u-boot-spl.bin`)
glibc ve kütüphaneler	Kernel ve kullanıcı programları için. U-Boot kullanmaz.
`arm-linux-gnueabihf-gdb`	Hata ayıklama

İsimdeki önek (`arm-linux-gnueabihf-`):

Parça	Anlamı
`arm`	Hedef mimari
`linux`	Hedef işletim sistemi: Linux üzerinde çalışacak programlar
`gnueabi`	GNU ABI (Application Binary Interface)
`hf`	**h**ard-**f**loat: ondalıklı sayı işlemleri donanımdaki FPU ile yapılır (Cortex-A8'de FPU var)

Neden U-Boot bölümünde anlatılıyor?

U-Boot, kitapta **derlediğimiz ilk parça**. Toolchain burada bir kere kuruluyor ve sonraki her aşamada aynısı kullanılıyor:

```
cross toolchain ─┬── SPL + U-Boot          (bu bölüm)
                 ├── Linux kernel + DTB     (uImage, am335x-boneblack.dtb)
                 ├── BusyBox / rootfs / initramfs
                 └── kendi uygulamaların ve sürücülerin
```

> **Not**
>
> U-Boot "bare-metal" çalışır: altında işletim sistemi yoktur ve glibc kullanmaz. Teknik olarak `arm-none-eabi-` (işletim sistemi olmayan) toolchain ile de derlenebilir. `arm-linux-gnueabihf-` seçilmesinin sebebi, aynı toolchain'in kernel ve kullanıcı programları için de gerekli olması. Tek toolchain her şeye yetiyor.

9.2 Sürüm Uyumu: Hangi Toolchain?

En çok sorun çıkaran konu bu. **Eski U-Boot/kernel kaynağını yeni GCC ile derlemek çoğu zaman hata verir.** Toolchain'i derleyeceğin kaynağın yaşına göre seç:

Derlenecek kaynak	Uygun toolchain	Nasıl kurulur?
U-Boot 2013.x, kernel 3.8	GCC 4.x (ör. Linaro 4.9-2014.09)	Linaro arşivinden indir
U-Boot 2016.x – 2022.x, kernel 4.x	GCC 6 / 7 (ör. Linaro 6.5, 7.5)	Linaro'dan indir (aşağıda)
U-Boot 2023+, kernel 5.x / 6.x	Güncel GCC	`sudo apt install gcc-arm-linux-gnueabihf`

> **Önemli**
>
> Bu bölümde kursun videolarında kullanılan **U-Boot 2022.04 + Linaro GCC 7.5.0** kullanılıyor (9.3 Adım 4'teki menuconfig ekranında görünüyor). Başka sürüm kullanırsan komutlarda sadece sürüm numaraları değişir, adımlar aynıdır.

> **Not**
>
> Kursun GitHub reposundaki eski derleme notları (`notes/compilation_commands`) Linaro GCC 6.3.1 ve `am335x_boneblack_defconfig` kullanıyor. Bu, kursun 2017 sürümüne ait; güncellenen videolarda U-Boot 2022.04 kullanılıyor. Bu kitap videolardaki güncel sürümü takip ediyor.

İndirme adresleri:

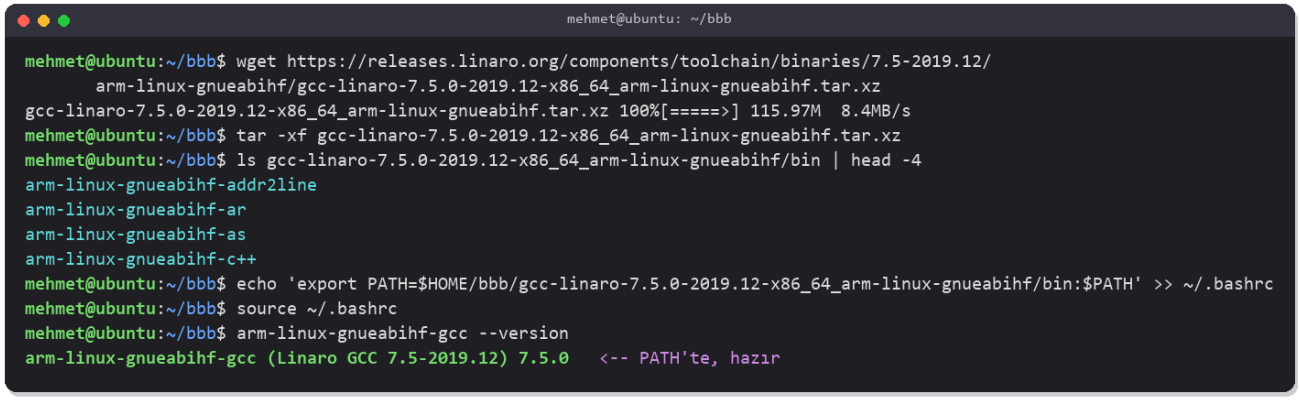
Dosya	Adres
Linaro GCC 7.5	`https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/`
Linaro GCC 6.5	`https://releases.linaro.org/components/toolchain/binaries/6.5-2018.12/arm-linux-gnueabihf/`
Linaro GCC 4.9	`https://releases.linaro.org/archive/14.09/components/toolchain/binaries/`
U-Boot kaynakları	`https://ftp.denx.de/pub/u-boot/`

9.3 Adım Adım: Toolchain Kurulumu ve U-Boot Derleme

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm, tarih ve boyut değerleri senin ortamında farklı olacaktır. Örneklerde çalışma klasörü `~/bbb` .

Adım 1 — Cross toolchain'i kur

Linaro toolchain hazır derlenmiş bir arşiv olarak gelir. İndirip açman ve `bin` klasörünü `PATH`'e eklemen yeterli:



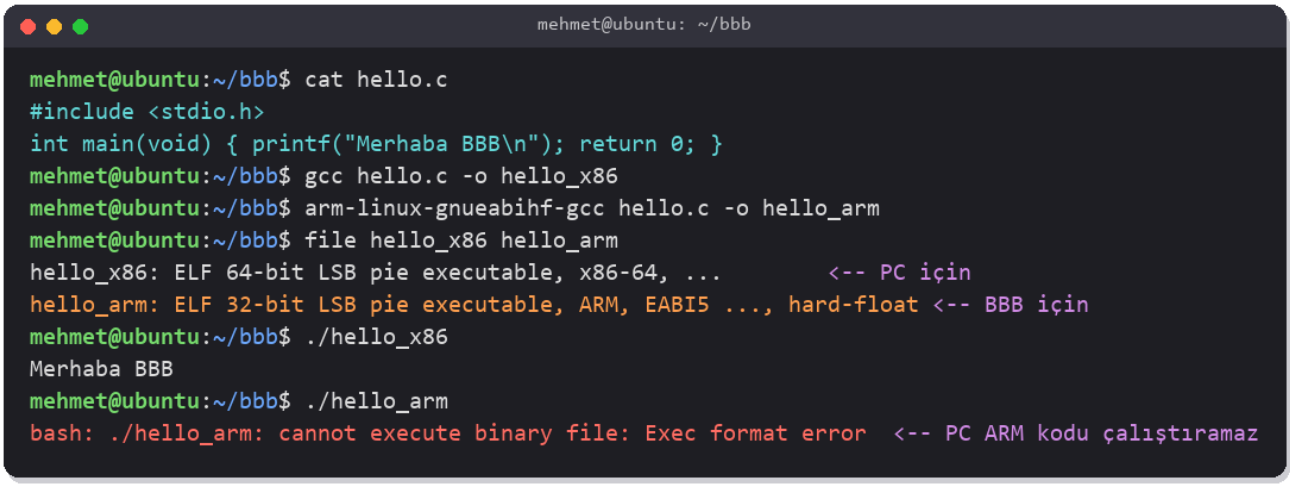

```
cd ~/bbb
wget https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
tar -xf gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
echo 'export PATH=$HOME/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
arm-linux-gnueabihf-gcc --version
```

`~/.bashrc`'ye eklediğimiz için her yeni terminalde `PATH` hazır olur.

> **İpucu**
>
> Güncel U-Boot (2023+) derleyeceksen indirmeye gerek yok: `sudo apt install gcc-arm-linux-gnueabihf` yeterli. Bu paket `/usr/bin`'e kurulur, `PATH` ayarı gerekmez.

Adım 2 — (İsteğe bağlı) Cross compile'ı kendi gözünle gör

Aynı C dosyasını iki derleyiciyle derleyip `file` ile karşılaştır:

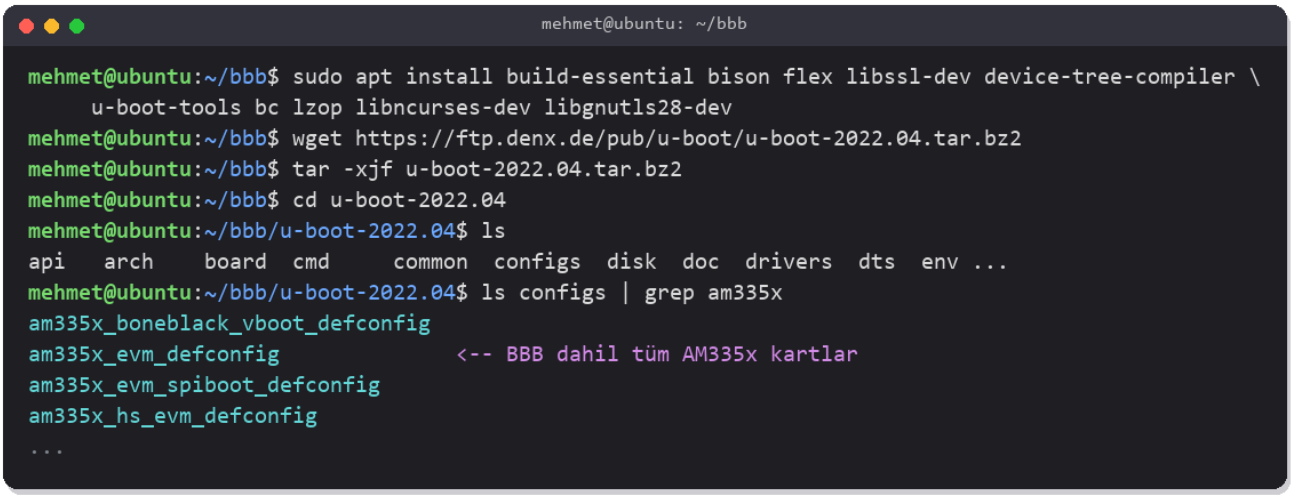



```
gcc hello.c -o hello_x86
arm-linux-gnueabihf-gcc hello.c -o hello_arm
file hello_x86 hello_arm
./hello_arm          # Exec format error: PC ARM kodu çalıştıramaz
```

`hello_arm` dosyasını karta kopyalarsan orada çalışır. U-Boot da aynı şekilde: PC'de derlenir, kartta çalışır. 6.6.3'te rootfs'teki programların PC'de çalışmamasının sebebi de budur.

Adım 3 — Host paketlerini kur ve U-Boot kaynağını indir

U-Boot derlenirken bazı yardımcı programlar (`mkimage`, `kconfig` vb.) **PC için** derlenip çalıştırılır. Bu yüzden host'ta da bazı paketler gerekir:



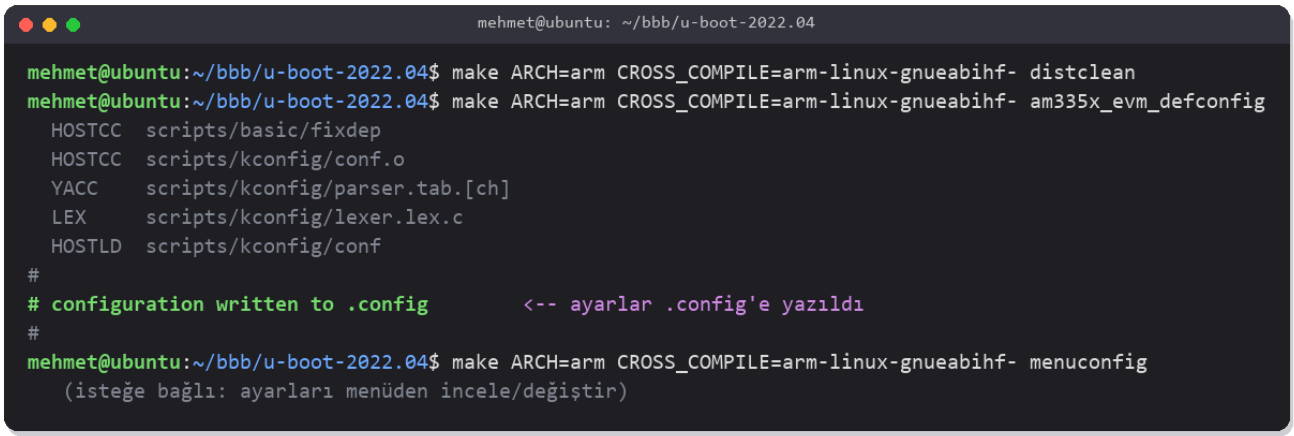

```
sudo apt install build-essential bison flex libssl-dev device-tree-compiler \
     u-boot-tools bc lzop libncurses-dev libgnutls28-dev
wget https://ftp.denx.de/pub/u-boot/u-boot-2022.04.tar.bz2
tar -xjf u-boot-2022.04.tar.bz2
cd u-boot-2022.04
ls configs | grep am335x
```

Paket	Neden?
`build-essential`	Host'un kendi `gcc`'si ve `make` (yardımcı araçlar için)
`bison`, `flex`	Kconfig ve dtc ayrıştırıcıları
`libssl-dev`, `libgnutls28-dev`	`mkimage` ve diğer host araçlarının imza/özet özellikleri
`device-tree-compiler`	`dtc`: U-Boot'un kendi device tree'si için
`libncurses-dev`	`make menuconfig` menüsü
`u-boot-tools`	Hazır `mkimage` (kernel/initramfs'e başlık eklemek için)

> **İpucu**
>
> `git clone https://source.denx.de/u-boot/u-boot.git` ve ardından `git checkout v2022.04` ile de alabilirsin. Kaynağı değiştirip deneyeceksen git daha kullanışlı, çünkü `git diff` ile ne değiştirdiğini görürsün.

Adım 4 — BBB için yapılandır

`configs/` klasöründe her kart için hazır bir ayar dosyası (`defconfig`) bulunur. Önce eski derleme kalıntılarını temizle, sonra BBB'nin ayarını yükle:




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig     # isteğe bağlı
```

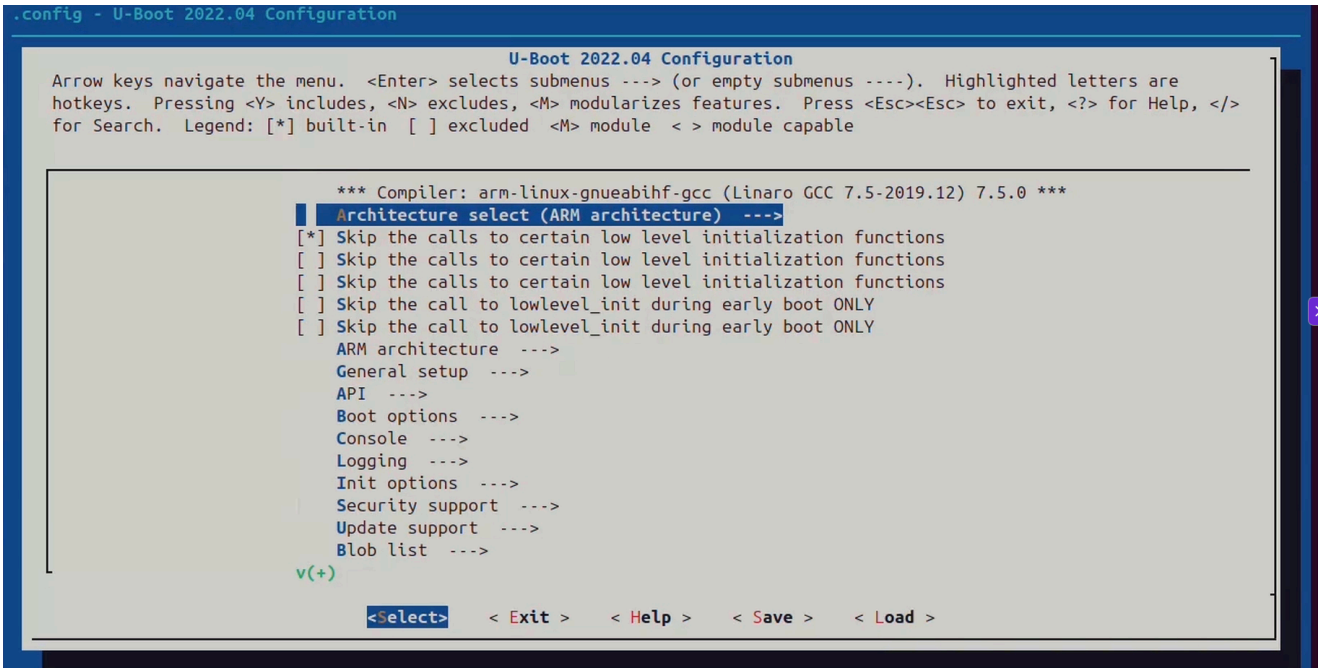
Parça	Anlamı
`ARCH=arm`	Hedef mimari: `arch/arm/` altındaki kodlar kullanılır
`CROSS_COMPILE=arm-linux-gnueabihf-`	Makefile'daki `gcc`, `ld`, `objcopy` komutlarının **başına eklenir**. `gcc` yerine `arm-linux-gnueabihf-gcc` çalışır. Sondaki `-` önemli.
`distclean`	Önceki derlemeden kalan her şeyi (`.config` dahil) siler

Parça	Anlamı
`am335x_evm_defconfig`	BBB'nin hazır ayarlarını `.config` dosyasına yazar
`menuconfig`	`.config`'i menüden düzenler (ör. `bootdelay`, komutlar)

> **Not**
>
> Ayar adı sürüme göre değişir: U-Boot 2013.10'da `am335x_evm_config`, 2017–2019 arası `am335x_boneblack_defconfig`, 2020 ve sonrası `am335x_evm_defconfig`. Bu sonuncusu tek bir SPL/U-Boot ile BBB, Bone, Bone Green, PocketBeagle ve EVM kartlarını kapsar; U-Boot açılırken kartın EEPROM'unu okuyup doğru DTB'yi seçer. Doğru adı `ls configs | grep am335x` ile bul.

`menuconfig` ekranı

`make ... menuconfig` komutu aşağıdaki menüyü açar (kursun ekran görüntüsü). İsteğe bağlı bir adım; hiçbir şeyi değiştirmeden `Exit` ile çıkabilirsin.



Görsel: kursun videosundan ekran görüntüsü.

Ekranda	Anlamı
`U-Boot 2022.04 Configuration`	Derlenen U-Boot sürümü. Başka sürüm indirdiysen burada o yazar.
`*** Compiler: arm-linux-gnueabihf-gcc (Linaro GCC 7.5-2019.12) 7.5.0 ***`	`CROSS_COMPILE` ile verdiğin derleyici. **Toolchain'in doğru bulunduğunu buradan anlarsın.** Burada Ubuntu'nun `gcc`'si görünüyorsa `CROSS_COMPILE` yazılmamış ya da `PATH` yanlış demektir.
`Architecture select (ARM architecture)`	`ARCH=arm` ve defconfig'in seçtiği mimari
`[*]` / `[ ]`	Seçili (U-Boot'a dahil) / seçili değil. **Boşluk** tuşu ile değiştirilir.

Ekranda	Anlamı
`--->`	Alt menü, **Enter** ile girilir
`v(+)`	Listenin devamı var, aşağı ok ile kaydır
`<Save>` / `<Exit>`	Değişiklikleri `.config`'e yazar / çıkar (değişiklik varsa kaydetmek ister)
`/`	Ayar arama (ör. `BOOTDELAY`)

Kursta sık bakılan yerler: **Boot options** (`bootdelay`, `bootcmd`), **Command line interface** (hangi U-Boot komutlarının derleneceği), **Environment** (ortam değişkenlerinin nereye kaydedileceği).

Adım 5 — Derle

```
mehmet@ubuntu: ~/bbb/u-boot-2022.04
mehmet@ubuntu:~/bbb/u-boot-2022.04$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
  CC      arch/arm/mach-omap2/am33xx/board.o
  ...
  LD      spl/u-boot-spl
  OBJCOPY spl/u-boot-spl-nodtb.bin
  COPY    spl/u-boot-spl.bin
  MKIMAGE MLO                          <-- SD kart için SPL
  LD      u-boot
  OBJCOPY u-boot-nodtb.bin
  MKIMAGE u-boot.img                   <-- tam U-Boot
mehmet@ubuntu:~/bbb/u-boot-2022.04$ ls -l MLO u-boot.img spl/u-boot-spl.bin
-rw-rw-r-- 1 mehmet mehmet  86124 Eki 19 14:02 MLO
-rw-rw-r-- 1 mehmet mehmet  86104 Eki 19 14:02 spl/u-boot-spl.bin  <-- seri boot için
-rw-rw-r-- 1 mehmet mehmet 425388 Eki 19 14:02 u-boot.img
mehmet@ubuntu:~/bbb/u-boot-2022.04$ file spl/u-boot-spl u-boot
spl/u-boot-spl: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked
u-boot:         ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked
```

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
ls -l MLO u-boot.img spl/u-boot-spl.bin
file spl/u-boot-spl u-boot
```

`-j4`: 4 iş parçacığıyla paralel derle (çekirdek sayına göre artırabilirsin, `nproc` ile öğren).

Derleme sonunda çıkan dosyalar:

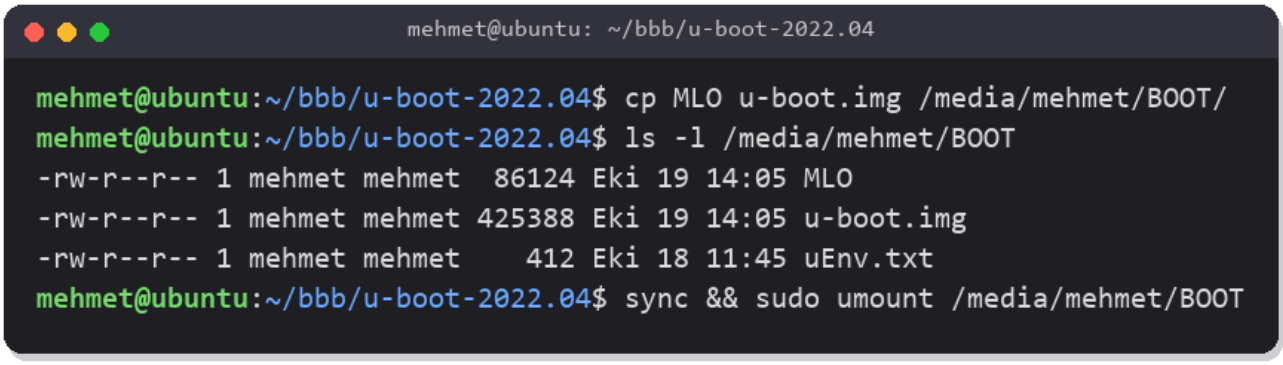
Dosya	Ne işe yarar?	Nerede kullandık?
`MLO`	SPL + başında **GP başlığı** (ROM Code'un okuyacağı yükleme adresi ve boyut)	SD kart / eMMC BOOT bölümü (Bölüm 4)
`spl/u-boot-spl.bin`	Başlıksız ham SPL	Seri (UART) boot, Xmodem ile (Bölüm 7)
`u-boot.img`	Tam U-Boot + U-Boot başlığı (SPL bunu okur)	SD kart, seri boot (Ymodem)

Dosya	Ne işe yarar?	Nerede kullandık?
`u-boot` , `spl/u-boot-spl`	ELF dosyaları (sembollü)	Hata ayıklama (`gdb`)

`file` çıktısındaki **ARM, EABI5** ifadesi dosyanın gerçekten ARM için derlendiğini gösterir.

> **Önemli**
>
> `MLO` **109 KB'tan küçük** olmalı. ROM Code onu AM335x'in dahili SRAM'ine koyar ve SRAM'de daha fazla yer yok. U-Boot'a çok özellik eklersen SPL büyüyebilir; o zaman SPL ayarlarından gereksiz özellikleri kapatmak gerekir.

Adım 6 — SD karta kopyala

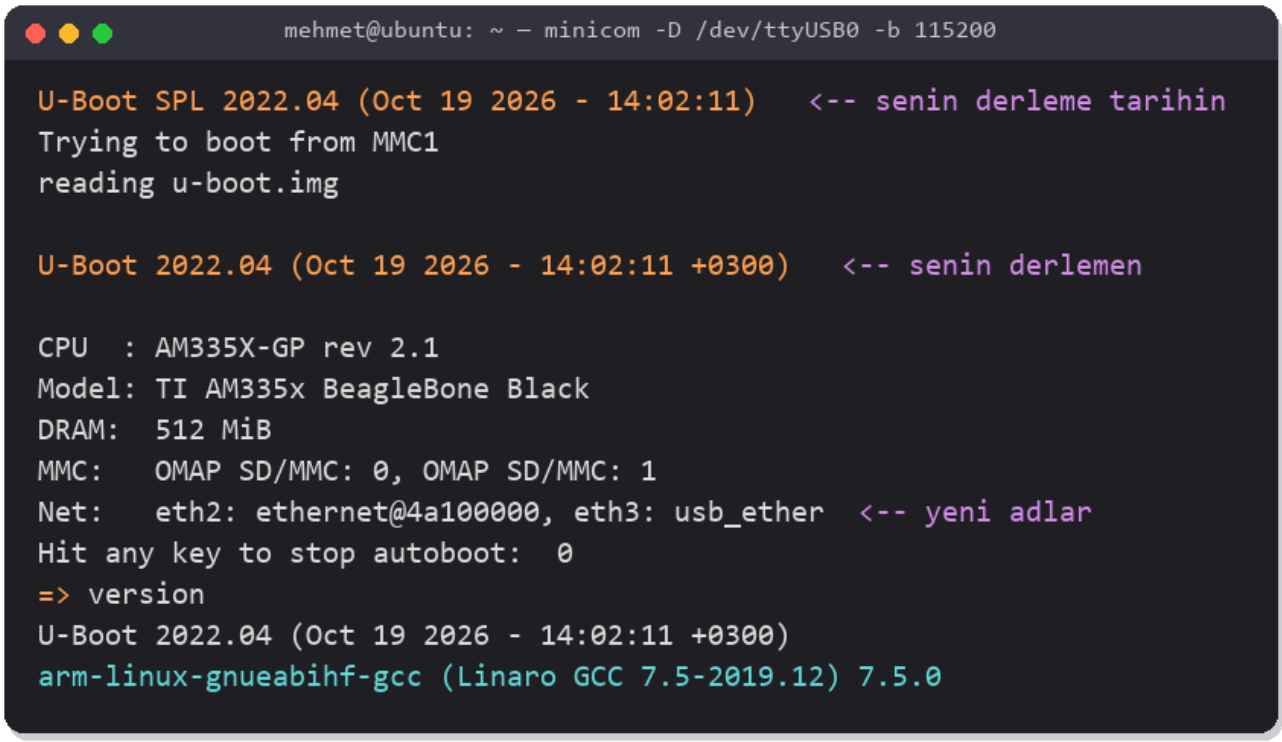



```
cp MLO u-boot.img /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT
```

Seri boot için ise `spl/u-boot-spl.bin` ve `u-boot.img` dosyalarını kullan (Bölüm 7).

Adım 7 — Kartta doğrula

Kartı S2'ye basılı tutarak aç. İlk satırlardaki **tarih ve saat senin derleme zamanın** olmalı:




```
=> version
```

> **Not**
>
> Yeni U-Boot sürümlerinde komut satırı `U-Boot#` yerine `=>` olarak görünür. Önceki bölümlerdeki komutlar aynı şekilde çalışır.
>
> Bir fark var: Ethernet aygıtının adı değişti. Kursun hazır U-Boot'unda (2013.04) `cpsw` olan aygıt 2022.04'te `ethernet@4a100000` olarak görünür (`Net:` satırına bak). Bu yüzden TFTP bölümünde `setenv ethact ethernet@4a100000` kullanılıyor (8.5); eski bir U-Boot kullanıyorsan `cpsw` yaz. Çoğu zaman varsayılan zaten doğrudur, emin olmak için `printenv ethact` ile bak.

Komutların kısa özeti

```
# Toolchain (bir kere)
wget https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
tar -xf gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
echo 'export PATH=$HOME/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

# U-Boot
sudo apt install build-essential bison flex libssl-dev device-tree-compiler u-boot-tools bc lzop libncurses-dev libgnutls28-dev
wget https://ftp.denx.de/pub/u-boot/u-boot-2022.04.tar.bz2 && tar -xjf u-boot-2022.04.tar.bz2 && cd u-boot-2022.04
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
cp MLO u-boot.img /media/$USER/BOOT/ && sync
```

9.4 Challenge: Autoboot Bekleme Süresini 10 Saniyeye Çıkarmak

Görev

Kursun bu bölümdeki challenge'ı:

> *U-Boot açılınca otomatik açılışa (uEnv.txt'yi okuma, uImage'ı yükleme...) geçmeden önce birkaç saniye bekler. Bu sürede bir tuşa basarsan U-Boot komut satırına düşersin. Bu bekleme süresini **10 saniyeye** çıkar ve kartta test et. Bunun için U-Boot **kaynak kodunu** değiştirip yeniden derlemen gerekiyor.*

Bekleme süresi nereden geliyor?

Geri sayımı `bootdelay` ortam değişkeni belirler. Bu değişkenin **varsayılan** değeri derleme sırasında `CONFIG_BOOTDELAY` ayarından gelir ve U-Boot'un içine gömülür:

Kaynak	Değer
U-Boot 2022.04 varsayılanı (`am335x_evm_defconfig` değiştirmiyor)	2 saniye
Kursun güncel `SD-boot` U-Boot'u	0 (geri sayım yok, tuşa basılı tutmak gerekir)
Kursun eski videolarındaki U-Boot	5 saniye

> **Not**
>
> `setenv bootdelay 10 + saveenv` ile de aynı sonuç alınır. Ama bu sadece o kartın kayıtlı ortamını değiştirir; challenge, ayarın **U-Boot imajının kendisinde** değişmesini istiyor. Böylece imajı hangi karta koyarsan koy 10 saniye bekler.

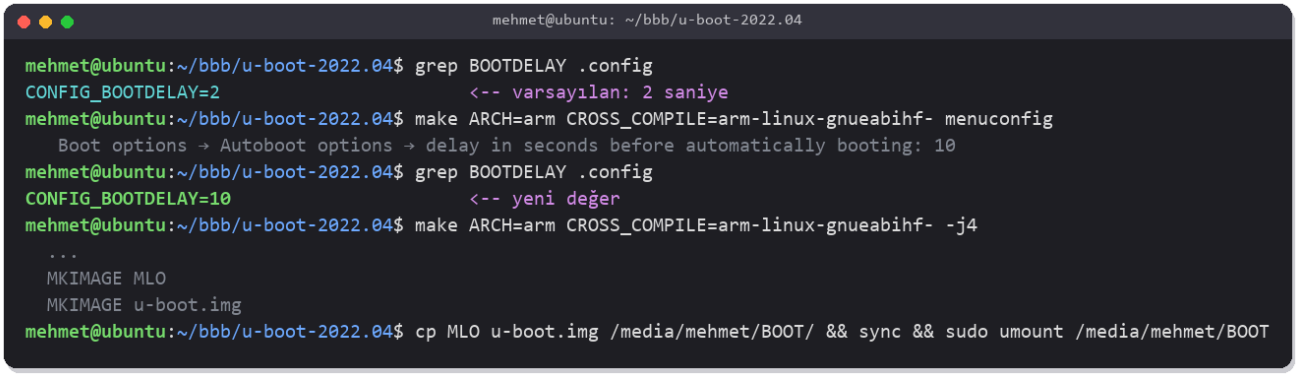
Çözüm

1) Ayarı değiştir. 9.3'teki U-Boot kaynak klasöründe `menuconfig`'i aç ve şu ayarı `10` yap:

```
Boot options  --->
    Autoboot options  --->
        (10) delay in seconds before automatically booting
```

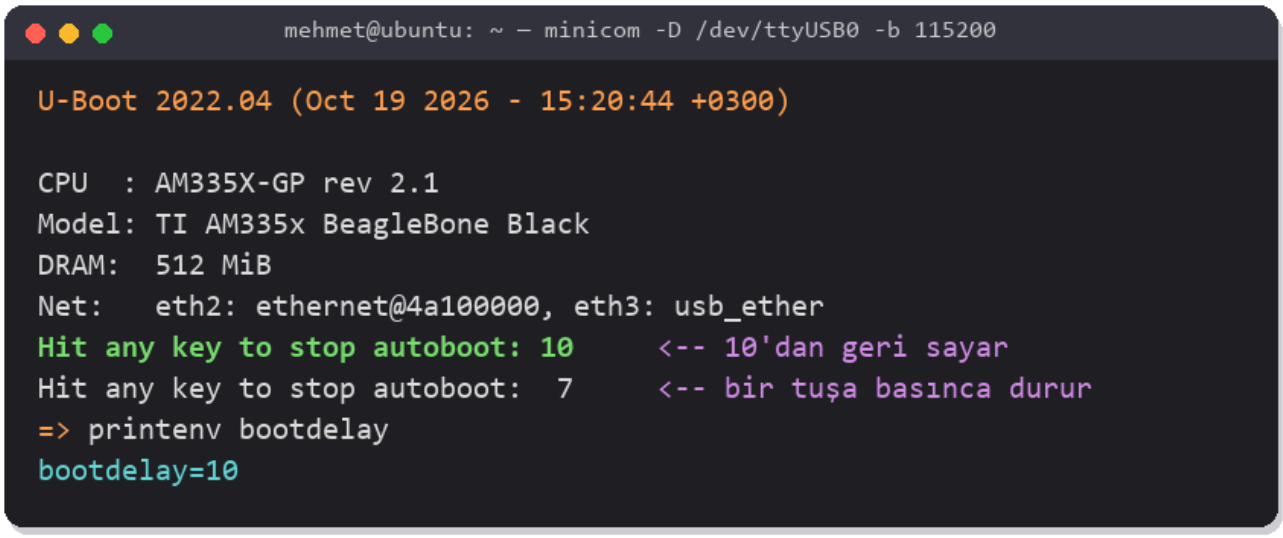
Ayarı bulmak için menüde `/` tuşuna basıp `BOOTDELAY` aramak daha hızlı. Kaydedip çık.

2) Kontrol et, derle, SD karta kopyala.




```
grep BOOTDELAY .config                                  # CONFIG_BOOTDELAY=10 görmelisin
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
cp MLO u-boot.img /media/$USER/BOOT/ && sync && sudo umount /media/$USER/BOOT
```

3) Kartta test et. Kartı S2'ye basılı tutarak aç. Geri sayım 10'dan başlamalı:



> **Önemli**
>
> Geri sayım hâlâ eski değerdeyse, kartta daha önce `saveenv` ile kaydedilmiş bir ortam vardır. Kayıtlı ortam derlemedeki varsayılanı ezer. U-Boot'ta `env default -a` ile varsayılanlara dön ve `saveenv` ile kaydet. Ayrıca `uEnv.txt` 'de `bootdelay=` satırı olmadığından emin ol.

> **İpucu**
>
> Değişikliğin kaynak kodda kalıcı olmasını istersen `make savedefconfig` ile küçük bir `defconfig` dosyası üret ve onu `configs/` altına kendi adınla kaydet (ör. `configs/my_bbb_defconfig`). Sonraki derlemelerde `make my_bbb_defconfig` yeterli olur.

9.5 Özet

- Cross toolchain, PC'de (x86) çalışıp ARM kodu üretir. Önek: `arm-linux-gnueabihf-` .
- Toolchain'i derlenecek kaynağın yaşına göre seç; eski kaynak + yeni GCC = derleme hatası.
- Derleme üç komut: `distclean` → `am335x_evm_defconfig` → `make -j4` . Hepsinde `ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-` .
- Çıktılar: `MLO` (SD/eMMC için), `spl/u-boot-spl.bin` (seri boot için), `u-boot.img` (ikisi için de).
- Kendi U-Boot'unun çalıştığını açılıştaki derleme tarihinden anlarsın.
- U-Boot'un davranışını değiştirmenin kalıcı yolu `menuconfig` ile `.config` 'i değiştirip yeniden derlemektir (ör. `CONFIG_BOOTDELAY`).

9.6 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~/bbb/u-boot-2022.04 (temsili hatalar)
mehmet@ubuntu:~/bbb/u-boot-2022.04$ make CROSS_COMPILE=arm-linux-gnueabihf- -j4
/bin/sh: 1: arm-linux-gnueabihf-gcc: not found     <-- PATH ayarlı değil

include/linux/compiler-gcc.h:121:30: fatal error: linux/compiler-gcc11.h:
No such file or directory                          <-- GCC U-Boot'tan yeni

multiple definition of `yylloc'; ...dtc-lexer.lex.o: first defined here  <-- host GCC 10+

tools/../include/image.h:...: fatal error: openssl/evp.h: No such file  <-- libssl-dev

/bin/sh: 1: bison: not found                       <-- bison/flex eksik
```

Hata	Sebep	Çözüm
`arm-linux-gnueabihf-gcc: not found`	Toolchain `PATH`'te değil	`echo $PATH` ile kontrol et, `source ~/.bashrc` çalıştır, yolu doğru yazdığından emin ol
`fatal error: linux/compiler-gccN.h: No such file or directory`	GCC, U-Boot/kernel kaynağından **daha yeni**. Eski kaynaklar sadece tanıdıkları GCC sürümleri için başlık dosyası içerir.	9.2'deki tablodan kaynağa uygun toolchain'i kullan
`multiple definition of 'yylloc'`	Eski kaynaktaki `dtc` + host'ta GCC 10 veya üstü	`scripts/dtc/dtc-lexer.l` (veya `.lex.c_shipped`) içinde `YYLTYPE yylloc;` satırını `extern YYLTYPE yylloc;` yap
`openssl/evp.h: No such file`	`libssl-dev` kurulu değil	`sudo apt install libssl-dev`
`bison: not found` / `flex: not found`	Host paketleri eksik	`sudo apt install bison flex`
`Your dtc is too old`	Host'taki `dtc` eski	`sudo apt install device-tree-compiler`
`*** Can't find default configuration "..."`	Yanlış `defconfig` adı	`ls configs | grep am335x`
32-bit Linaro 4.x toolchain çalışmıyor: `arm-linux-gnueabihf-gcc: No such file or directory` (dosya varken)	Eski toolchain 32-bit, 64-bit Ubuntu'da 32-bit kütüphaneler eksik	`sudo dpkg --add-architecture i386 && sudo apt update && sudo apt install libc6:i386 libstdc++6:i386 zlib1g:i386`
Kart açılıyor ama ekranda eski U-Boot tarihi var	Kart eMMC'den açıldı ya da SD'ye kopyalanan dosyalar eski	S2'ye basılı tutarak aç, `ls -l /media/$USER/BOOT` ile tarihleri kontrol et

Hata	Sebep	Çözüm
SPL hiç çalışmıyor	`MLO` yerine `u-boot-spl.bin` kopyalandı ya da `MLO` çok büyük	SD karta `MLO` koy, boyutunu kontrol et (< 109 KB)

Kısım IV
Linux Kernel

Bölüm 10 — Linux Kernel: Kaynak Ağacı ve Derleme

> **Bu bölümde:**
>
> - BeagleBone Black için hangi kernel kaynağının kullanıldığını ve kursun kernel sürüm geçmişini öğreneceksin.
> - Kernel kaynak ağacında gezinmeyi öğreneceksin: ARM mimarisi kodu, kart yapılandırma dosyaları ve SoC sürücüleri nerede durur?
> - Kernel'i `bb.org_defconfig` ile yapılandırıp `uImage`, `am335x-boneblack.dtb` ve modülleri derleyeceksin.
> - Kendi derlediğin kernel ile kartı açacaksın.
>
> **Ön koşul:** Bölüm 3 — boot zinciri, Bölüm 9 — cross toolchain kurulumu.

10.1 Hangi Kernel?

Linux kernel'in "resmi" kaynağı Linus Torvalds'ın yönettiği **mainline** ağaçtır (kernel.org). Kart üreticileri bunun üzerine kendi kartlarına özel yamaları ekleyip **vendor** ağaçları yayınlar. BeagleBone Black için iki seçenek var:

Kaynak	Adres	Ne zaman?
BeagleBoard kernel'i (vendor)	`https://github.com/beagleboard/linux.git`	Kursta ve bu kitapta kullanılan. BBB'ye özel ayar (`bb.org_defconfig`), cape ve PRU desteği hazır gelir.
Mainline	`https://git.kernel.org`	En yeni özellikler. BBB'de temel donanım çalışır, bazı ekstralar eksik olabilir.

Kursun kullandığı kernel sürümleri zamanla değişti:

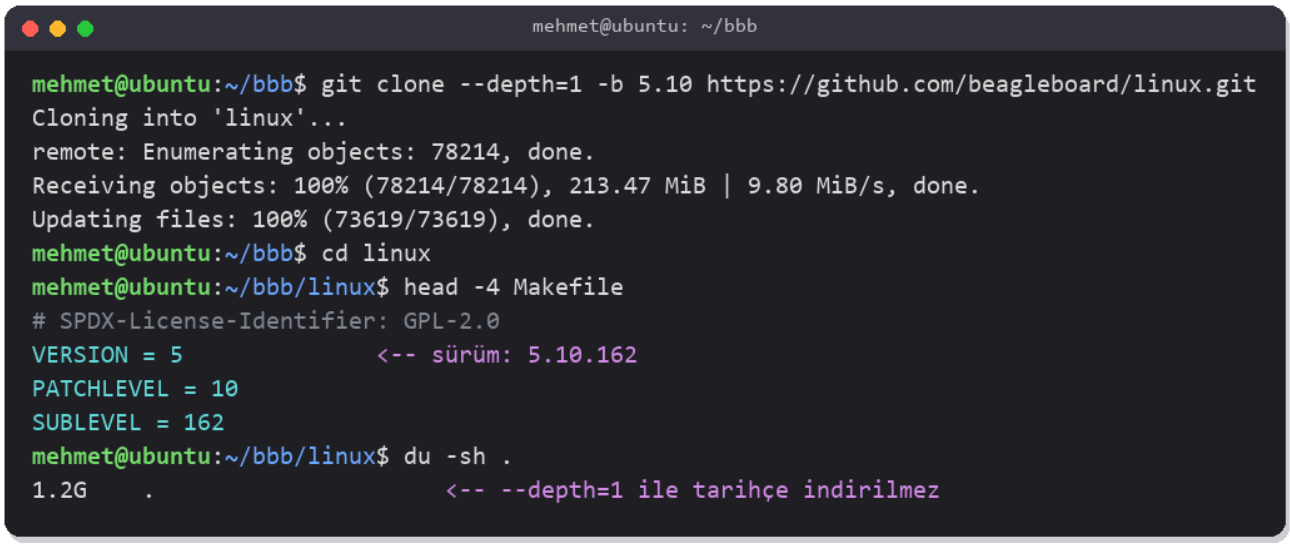
Nerede?	Kernel
Kursun ilk hazır imajları (Ångström)	3.8.13
Kursun eski derleme notları	4.4 (`bb.org_defconfig`) ve 4.11 (`omap2plus_defconfig`)
Kursun güncel imajları	**5.10.168**

Kursun başındaki "Important Update on Linux Kernel Version" notu bu güncellemeyi anlatıyor. Bu kitap BeagleBoard'un **5.10** dalını kullanıyor. 5.10 bir **LTS** (Long Term Support, uzun süre desteklenen) sürümdür.

> **Not**
>
> Dal (branch) zamanla güncellenir. Bu kitap yazılırken `5.10` dalının ucu **5.10.162** sürümündeydi. Kursun hazır imajı 5.10.168. Aradaki fark sadece küçük düzeltmelerdir, anlatılan her şey ikisinde de aynıdır.

10.2 Kaynak Kodu İndirme

```
cd ~/bbb
git clone --depth=1 -b 5.10 https://github.com/beagleboard/linux.git
cd linux
head -4 Makefile
```



Parça	Anlamı
`--depth=1`	Sadece son sürümü indir, binlerce commit'lik geçmişi indirme. İndirme birkaç GB yerine birkaç yüz MB olur.
`-b 5.10`	`5.10` dalını al
`Makefile` başı	`VERSION`, `PATCHLEVEL`, `SUBLEVEL` satırları indirdiğin sürümü gösterir

> **İpucu**
>
> Kaynak kodda bir fonksiyonun ya da sürücünün nerede tanımlandığını bulmak için tarayıcıdan **Elixir**'i kullan: https://elixir.bootlin.com/linux/v5.10/source. Her sembole tıklayınca tanımına ve kullanıldığı yerlere gider. Bilgisayarda ise `grep -rn "aranan" drivers/` iş görür.

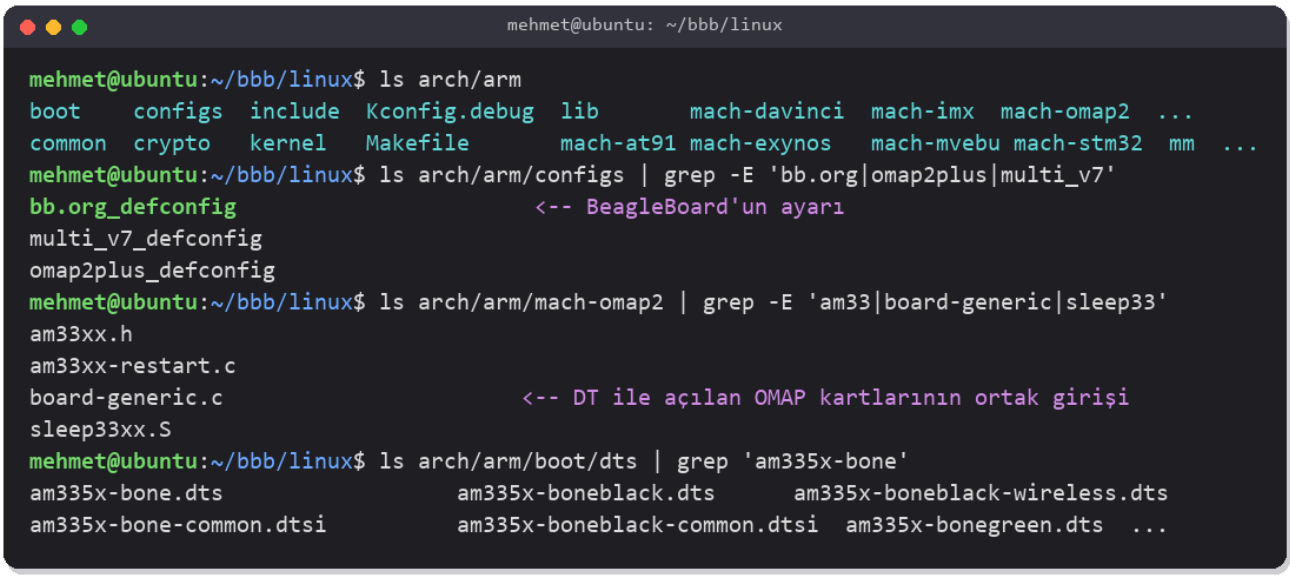
10.3 Kaynak Ağacının Genel Yapısı

Klasör	İçinde ne var?
`arch/`	Mimariye özel kod: `arch/arm`, `arch/arm64`, `arch/x86` ... Açılış kodu, bellek yönetiminin mimariye bağlı kısmı, **device tree** dosyaları ve **defconfig**'ler burada.
`drivers/`	Bütün aygıt sürücüleri: GPIO, I2C, SPI, UART, MMC, USB, ağ... Kaynağın yaklaşık yarısı.
`include/`	Ortak başlık dosyaları (`include/linux/`, `include/dt-bindings/` ...)
`init/`	Kernel'in açılışı: `start_kernel()` (`init/main.c`) ve ilk kullanıcı programını (`init`) başlatan kod
`kernel/`	Çekirdeğin kalbi: zamanlayıcı (scheduler), süreç yönetimi, sinyaller, zamanlayıcılar
`mm/`	Bellek yönetimi
`fs/`	Dosya sistemleri: ext4, FAT, NFS, proc, sysfs...
`net/`	Ağ yığını: TCP/IP, soketler
`block/`	Blok aygıt katmanı (diskler, SD kart)
`lib/`, `crypto/`, `security/`, `sound/`, `ipc/`	Yardımcı kütüphaneler, şifreleme, güvenlik modülleri, ses, süreçler arası iletişim
`Documentation/`	Kernel'in kendi belgeleri. Device tree binding'leri `Documentation/devicetree/bindings/` altında (11.7).
`scripts/`	Derleme sistemi araçları (`kconfig`, `dtc` ...)
`Makefile`, `Kconfig`	Derleme sisteminin girişi ve ayar menüsünün kökü

Kernel'in kullanıcı programlarıyla ilişkisi:

```
Kullanıcı alanı (user space)     uygulamalar, BusyBox, kütüphaneler (glibc)
─────────────────────────────── sistem çağrıları (open, read, write, ioctl...)
Kernel alanı (kernel space)      kernel/ mm/ fs/ net/ ipc/   ← mimariden bağımsız çekirdek
                                 drivers/                    ← aygıt sürücüleri
                                 arch/arm/                   ← ARM'a özel kod
─────────────────────────────── 
Donanım                          AM335x: Cortex-A8, UART, GPIO, I2C, MMC, Ethernet...
```

10.4 ARM Mimarisi Kodunun Düzeni (`arch/arm`)



Klasör	İçinde ne var?
`arch/arm/boot/`	Derlenen imajlar (`Image` , `zImage` , `uImage`) buraya çıkar.
`arch/arm/boot/compressed/`	`zImage` 'in başındaki, kernel'i RAM'de açan küçük program (4.9)
`arch/arm/boot/dts/`	Device tree kaynakları (`am335x-boneblack.dts` ve `.dtsi` 'leri)
`arch/arm/configs/`	Hazır ayar dosyaları (**defconfig**)
`arch/arm/kernel/`	ARM'a özel çekirdek kodu. Kernel'in ilk çalışan komutları `arch/arm/kernel/head.S` 'tedir.
`arch/arm/mm/`	ARM'a özel bellek yönetimi (MMU, cache)
`arch/arm/mach-omap2/`	**TI OMAP/AM335x ailesine özel kod:** güç ve saat alanları, uyku modu (`sleep33xx.S`), DT ile açılan kartların ortak girişi (`board-generic.c`)
`arch/arm/mach-*/`	Diğer SoC aileleri: `mach-imx` (NXP i.MX), `mach-stm32` , `mach-at91` ...

> **Not**
>
> Device tree'den önce her kart için `arch/arm/mach-omap2/board-xxx.c` gibi bir **board file** yazılırdı. Bugün `mach-omap2` içinde kart dosyası yoktur. Kartları device tree tarif eder, `board-generic.c` sadece "bu SoC ailesi DT ile açılıyor" der (11.1).

10.5 Kart Yapılandırma Dosyaları

Bir kart için kernel hazırlarken iki tür dosya kullanılır:

Dosya türü	Ne tarif eder?	BBB için
defconfig (`arch/arm/configs/`)	**Kernel'e hangi özelliklerin ve sürücülerin derleneceğini**	`bb.org_defconfig`
Device tree (`arch/arm/boot/dts/`)	**Kartta hangi donanımın nerede olduğunu**	`am335x-boneblack.dts`

Kullanılabilecek defconfig'ler:

defconfig	Kapsamı
`bb.org_defconfig`	BeagleBoard'un BBB ve diğer Beagle kartları için hazırladığı ayar. Cape, PRU, USB gadget desteği açık. **Bu kitapta kullanılan.**
`omap2plus_defconfig`	Mainline'daki bütün OMAP2+ / AM335x / AM437x / DRA7 kartları için ortak ayar
`multi_v7_defconfig`	Bütün ARMv7 kartları (TI, NXP, ST, Rockchip...) için tek kernel. En büyük imaj.

BBB ailesinin device tree dosyaları:

DTS	Kart
`am335x-boneblack.dts`	BeagleBone Black
`am335x-boneblack-wireless.dts`	BeagleBone Black Wireless
`am335x-bonegreen.dts`	SeeedStudio BeagleBone Green
`am335x-bone.dts`	Eski BeagleBone (White)
`am335x-pocketbeagle.dts`	PocketBeagle

Device tree'nin yapısı ve include zinciri Bölüm 11'de anlatılıyor.

10.6 SoC Sürücülerinin Düzeni (`drivers/`)

Sürücüler mimariye göre değil, **aygıt türüne** göre klasörlenir. AM335x'in çevre birimlerini süren sürücüler:

AM335x birimi	Sürücü dosyası	Kernel ayarı
UART	`drivers/tty/serial/8250/8250_omap.c`	`CONFIG_SERIAL_8250_OMAP`
GPIO	`drivers/gpio/gpio-omap.c`	`CONFIG_GPIO_OMAP`
I2C	`drivers/i2c/busses/i2c-omap.c`	`CONFIG_I2C_OMAP`
SPI (McSPI)	`drivers/spi/spi-omap2-mcspi.c`	`CONFIG_SPI_OMAP24XX`
MMC / SD / eMMC	`drivers/mmc/host/sdhci-omap.c` (eski: `omap_hsmmc.c`)	`CONFIG_MMC_SDHCI_OMAP`
Ethernet (CPSW)	`drivers/net/ethernet/ti/cpsw.c` , `cpsw_new.c`	`CONFIG_TI_CPSW`
USB (MUSB)	`drivers/usb/musb/musb_dsps.c`	`CONFIG_USB_MUSB_DSPS`
USB gadget: Ethernet (`g_ether`)	`drivers/usb/gadget/legacy/ether.c`	`CONFIG_USB_ETH`
RTC	`drivers/rtc/rtc-omap.c`	`CONFIG_RTC_DRV_OMAP`
Watchdog	`drivers/watchdog/omap_wdt.c`	`CONFIG_OMAP_WATCHDOG`
PWM (EHRPWM)	`drivers/pwm/pwm-tiehrpwm.c`	`CONFIG_PWM_TIEHRPWM`
ADC	`drivers/iio/adc/ti_am335x_adc.c`	`CONFIG_TI_AM335X_ADC`
LCD / HDMI	`drivers/gpu/drm/tilcdc/`	`CONFIG_DRM_TILCDC`
PMIC (TPS65217)	`drivers/mfd/tps65217.c`	`CONFIG_MFD_TPS65217`

Bir sürücünün hangi device tree düğümüyle eşleştiğini `compatible` dizisinden bulursun:

```
grep -n "compatible" drivers/gpio/gpio-omap.c          # "ti,omap4-gpio" ...
grep -rn "ti,omap4-gpio" arch/arm/boot/dts/am33xx*.dtsi # DT'deki karşılığı
```

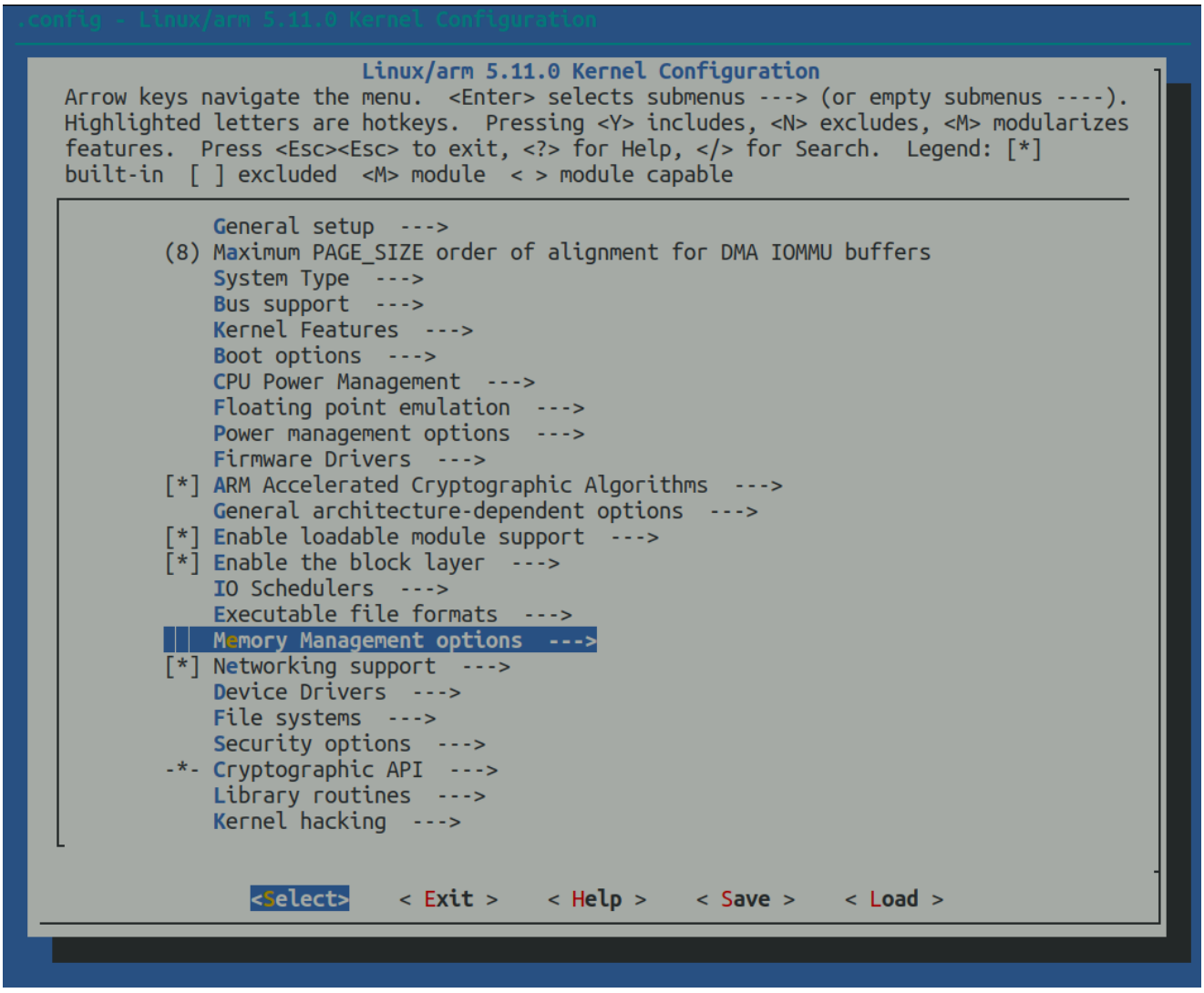
> **Not**
>
> UART sürücüsü konsolun adını belirler: `8250_omap` → `/dev/ttyS0` , eski `omap-serial` → `/dev/ttyO0` . Kernel 5.10'da `8250_omap` kullanıldığı için `bootargs` 'ta **`console=ttyS0`** yazılır (3.4).

10.7 Kernel Yapılandırması: Kconfig, `.config` , menuconfig

Kernel'de binlerce seçenek var ve her biri `CONFIG_` ile başlayan bir isimle temsil edilir.

Parça	Görevi
`Kconfig` dosyaları	Her klasörde bulunur. Seçenekleri, açıklamalarını ve birbirine bağımlılıklarını tanımlar.
`.config`	Derlemede kullanılan **gerçek** ayar dosyası. Binlerce satır, kaynak klasörünün kökünde.

Parça	Görevi
`defconfig`	`.config` 'in kısaltılmış hali: sadece varsayılandan farklı olan satırlar. `make xxx_defconfig` bunu açıp `.config` üretir.
`make menuconfig`	`.config` 'i menüden düzenler. U-Boot'taki menüyle aynı (9.3).
`make savedefconfig`	Değiştirdiğin `.config` 'ten tekrar küçük bir `defconfig` üretir.



Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)

Her sürücü veya özellik için üç seçenek var:

Değer	menuconfig'te	Anlamı
`y`	`[*]` / `<*>`	Kernel imajının **içine** derlenir (built-in). Kernel açılır açılmaz hazırdır.
`m`	`<M>`	**Modül** olarak ayrı bir `.ko` dosyasına derlenir. Rootfs'ten `modprobe` / `insmod` ile sonradan yüklenir.
`n`	`[ ]` / `< >`	Derlenmez

> **Önemli**
>
> Rootfs'i bağlamak için gereken sürücüler (MMC, ext4, NFS için ağ sürücüsü) `y` olmalıdır. Kernel bunlar olmadan rootfs'i bağlayamaz, rootfs'i bağlamadan da modül yükleyemez. Modüller (`m`) ancak rootfs bağlandıktan sonra kullanılabilir.

10.8 Adım Adım: Kernel'i Derleme

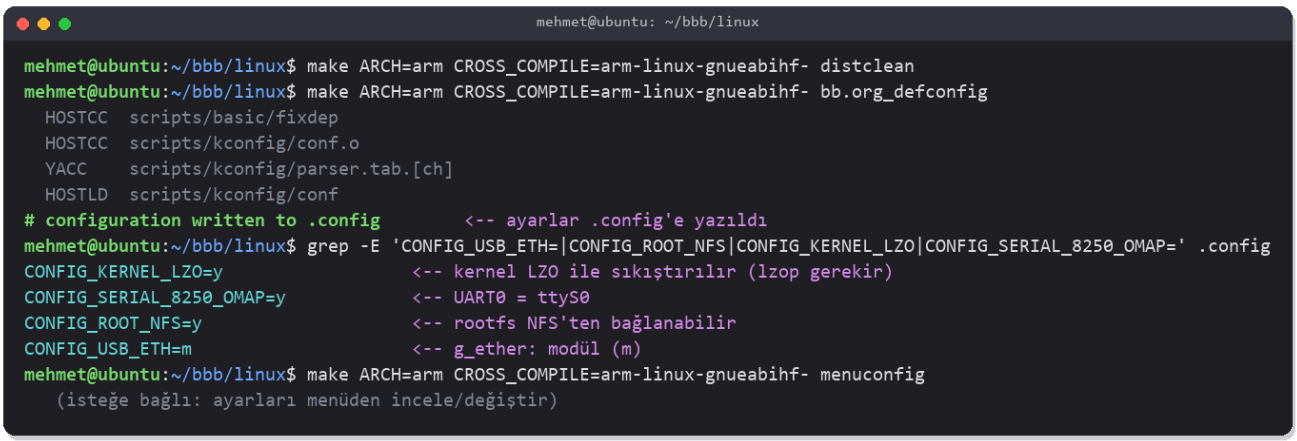
> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**. Örneklerde çalışma klasörü `~/bbb/linux` , toolchain Bölüm 9'da kurulan Linaro GCC 7.5.0.

Adım 1 — Host paketlerini kur

```
sudo apt install build-essential bc bison flex libssl-dev libncurses-dev lzop u-boot-tools git
```

Paket	Neden?
`bc`	Derleme sırasında bazı sabitler `bc` ile hesaplanır
`lzop`	`bb.org_defconfig` kernel'i **LZO** ile sıkıştırır (`CONFIG_KERNEL_LZO=y`)
`libssl-dev`	Modül imzalama ve sertifika araçları
`u-boot-tools`	`mkimage` : `uImage` üretmek için
`bison` , `flex` , `libncurses-dev`	Kconfig ve `menuconfig`

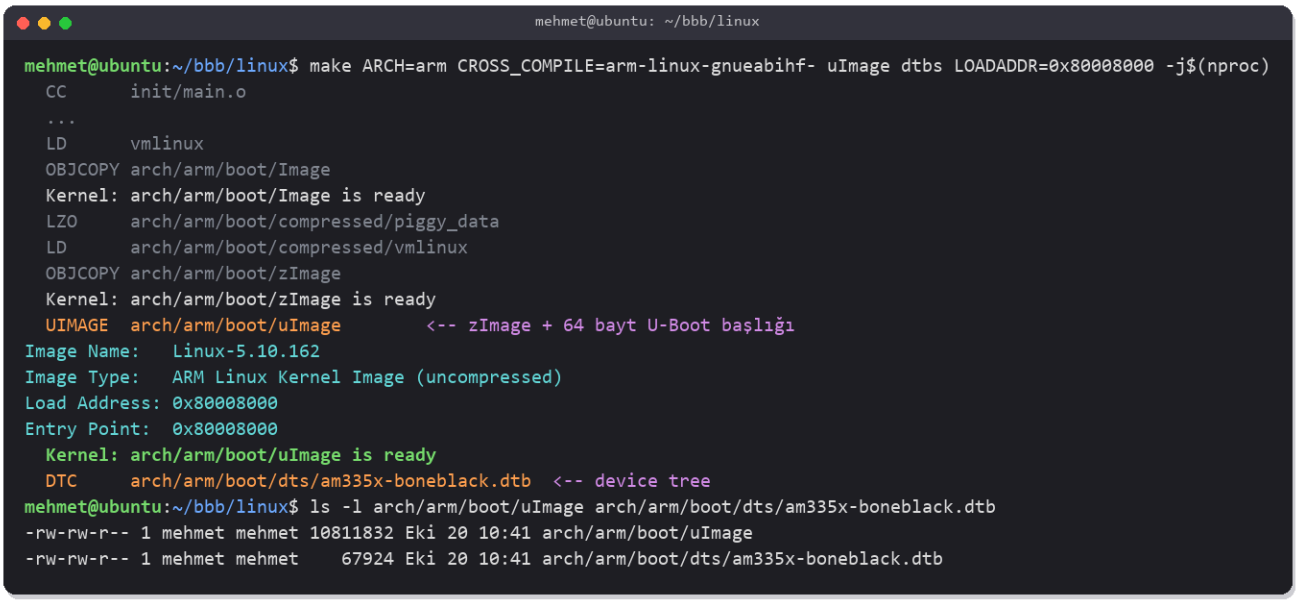
Adım 2 — Yapılandır



```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig     # isteğe bağlı
```

Kernel'de U-Boot'tan farklı olarak `ARCH=arm` **her komutta zorunludur**. Yazmazsan kernel host'un mimarisini (x86) varsayar ve yanlış `.config` üretir.

Adım 3 — Kernel imajını ve device tree'yi derle




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000 -j$(nproc)
```

Parça	Anlamı
`uImage`	`vmlinux` → `Image` → `zImage` → `uImage` zincirini üretir (4.9)
`dtbs`	Bütün `.dts` dosyalarını `.dtb`'ye derler
`LOADADDR=0x80008000`	`uImage` başlığına yazılan yükleme ve giriş adresi. AM335x'te DDR `0x80000000`'da başlar; kernel ilk 32 KB'ın (`0x8000`) arkasına yerleşir.
`-j$(nproc)`	İşlemcinin çekirdek sayısı kadar paralel derle

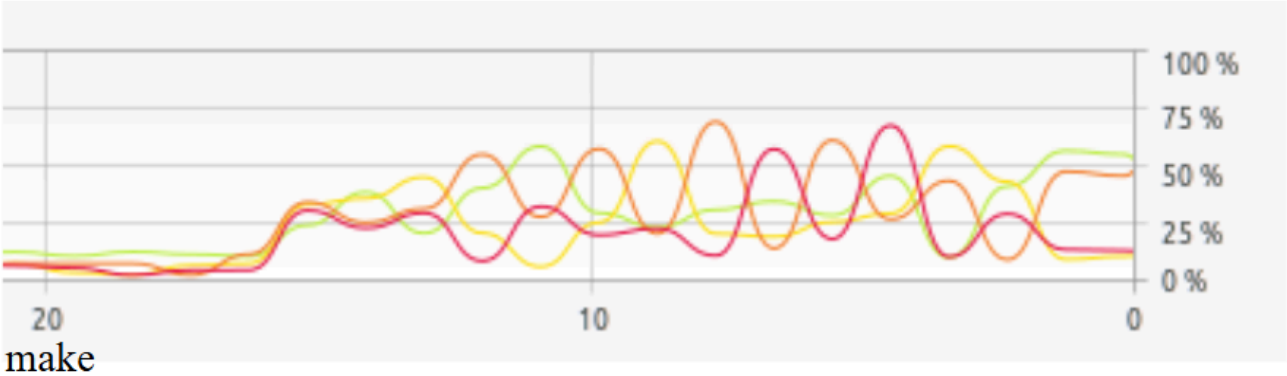
İlk derleme bilgisayarına göre 10–40 dakika sürer. Paralel derlemenin etkisi:

Benefits of parallel compile jobs (make -j<n>)

Tests on Linux 5.11 on arm

make allnoconfig configuration

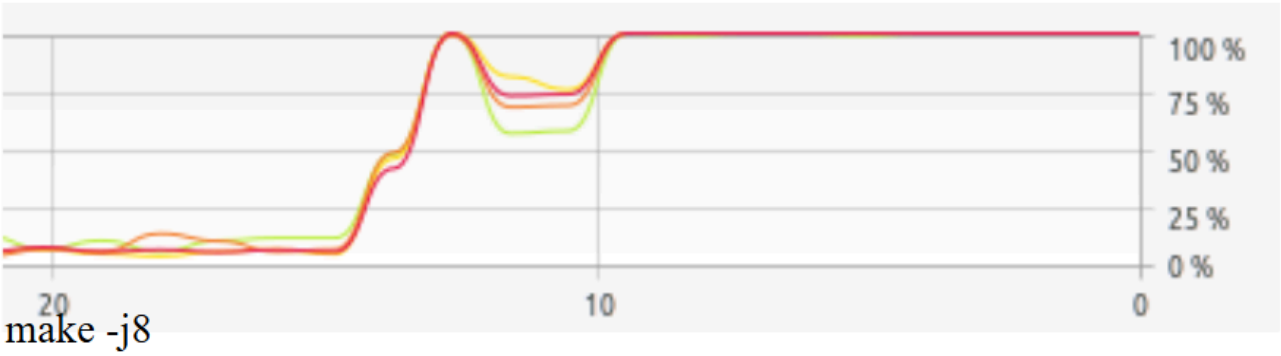
gnome-system-monitor showing the load on 4 threads / 2 CPUs



make

Command: make

Total time: 129 s



make -j8

Command: make -j8

Total time: 67 s

Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)

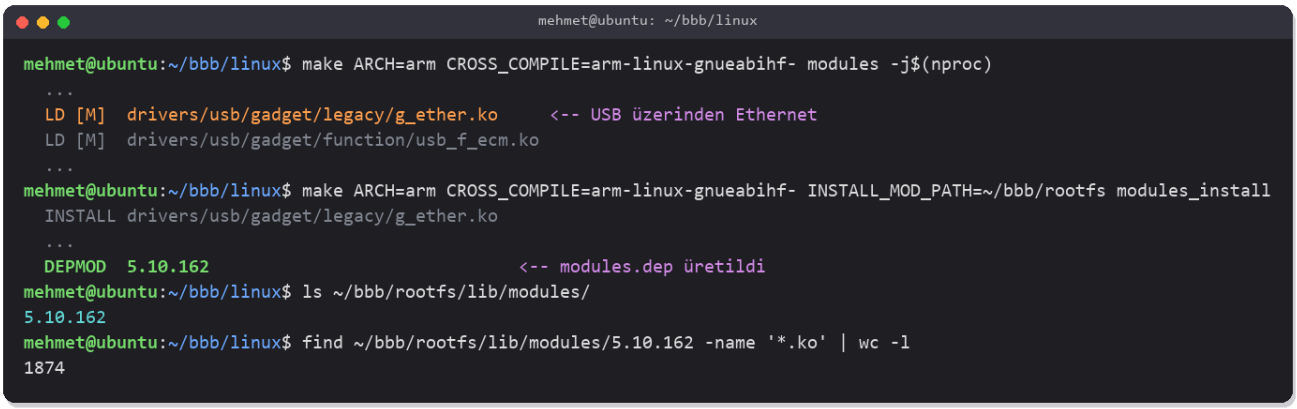
Derleme sonunda çıkan dosyalar:

Dosya	Ne?
`vmlinux`	Derlenmiş kernel, ELF formatında (sembollü, hata ayıklama için)
`arch/arm/boot/Image`	Ham binary kernel
`arch/arm/boot/zImage`	Sıkıştırılmış, kendini açan kernel

Dosya	Ne?
`arch/arm/boot/uImage`	`zImage` + U-Boot başlığı. **Karta giden dosya.**
`arch/arm/boot/dts/am335x-boneblack.dtb`	BBB'nin device tree'si. **Karta giden dosya.**
`System.map`	Kernel sembollerinin adresleri

Adım 4 — Modülleri derle ve kur

`m` olarak seçilen sürücüler ayrı derlenir ve rootfs'e kurulur:




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=~/bbb/rootfs modules_install
```

`modules_install`, modülleri `<INSTALL_MOD_PATH>/lib/modules/5.10.162/` altına kopyalar ve `modprobe`'un kullandığı `modules.dep` dosyasını üretir. Buradaki rootfs klasörünü Bölüm 12'de BusyBox ile dolduracağız.

> **Dikkat**
>
> `INSTALL_MOD_PATH`'i **unutma**. Yazmazsan `sudo make modules_install` modülleri **host bilgisayarının** `/lib/modules` klasörüne kurar; ARM modülleri Ubuntu'nun klasörüne karışır.

Komutların kısa özeti

```
cd ~/bbb
git clone --depth=1 -b 5.10 https://github.com/beagleboard/linux.git
cd linux
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000 -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=~/bbb/rootfs modules_install
```

10.9 Yeni Kernel'i Kartta Denemek

Kernel'i tam olarak denemek için bir rootfs gerekir. Kendi rootfs'imizi Bölüm 12'de hazırlayacağız. Kernel'in açıldığını görmek için şimdilik **eMMC'deki Debian'ın rootfs'ini** kullanabilirsin.

1) Dosyaları SD kartın BOOT bölümüne kopyala:

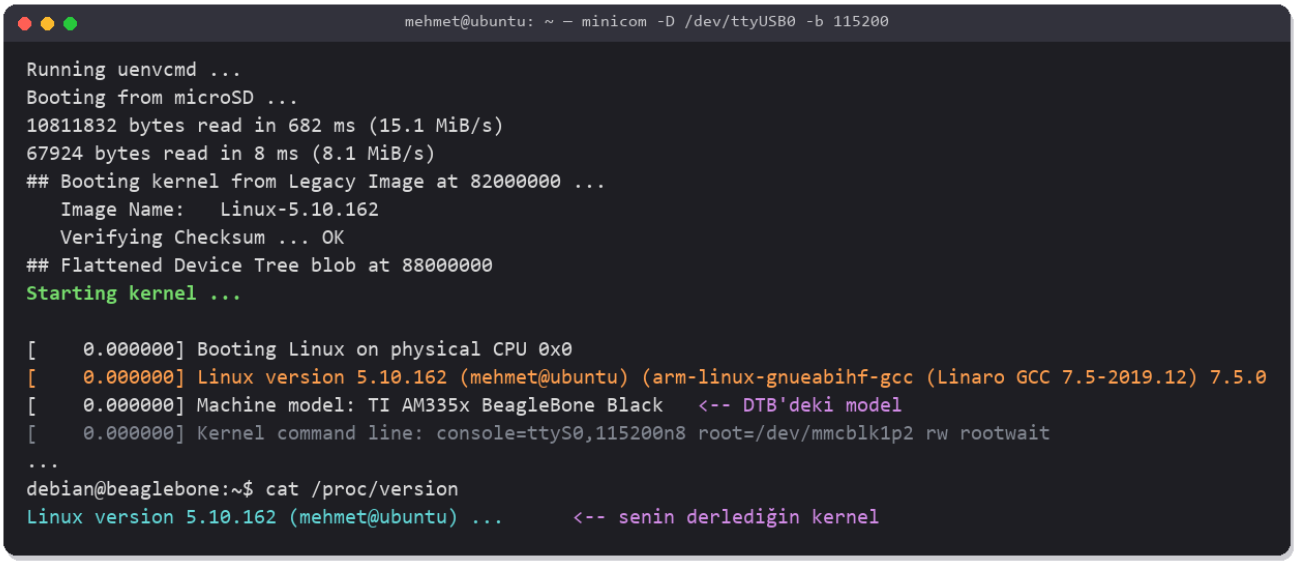
```
cp arch/arm/boot/uImage arch/arm/boot/dts/am335x-boneblack.dtb /media/$USER/BOOT/
```

2) BOOT bölümündeki `uEnv.txt` 'yi yaz. Kursun `uEnv-SD.txt` dosyası temel alındı; konsol `ttyS0`, rootfs eMMC:

```
console=ttyS0,115200n8
netargs=setenv bootargs console=${console} root=/dev/mmcblk1p2 rw rootwait
netboot=echo Booting from microSD ...; setenv autoload no; load mmc 0:1 ${loadaddr} uImage; load
mmc 0:1 ${fdtaddr} am335x-boneblack.dtb; run netargs; bootm ${loadaddr} - ${fdtaddr}
uenvcmd=run netboot
```

> **Önemli**
>
> eMMC'deki rootfs'in hangi bölümde olduğu Debian imajına göre değişir. Eski imajlarda 2. bölüm (`mmcblk1p2`), yenilerinde tek bölüm (`mmcblk1p1`) olabilir. U-Boot'ta `mmc dev 1` ve `mmc part` ile kontrol et (6.5).

3) Kartı S2'ye basılı tutarak aç. Açılış mesajlarında kendi kernel sürümünü, derleme makineni ve toolchain'ini görmelisin:




```
cat /proc/version
```

> **Not**
>
> eMMC'deki Debian kendi kernel modüllerini (`/lib/modules/<eski sürüm>`) içerir, senin kernel'inin modüllerini içermez. Kernel açılır ama bazı sürücüler (Wi-Fi, USB gadget...) yüklenmeyebilir. Bu bir sorun değil; amacımız sadece kernel'in açıldığını görmek. Modüller Bölüm 12'deki rootfs'e kurulacak.

10.10 Özet

- BeagleBone Black için BeagleBoard'un kernel ağacı kullanılıyor: `https://github.com/beagleboard/linux.git` , dal `5.10` .
- `arch/arm` mimariye özel kodu, `arch/arm/configs` defconfig'leri, `arch/arm/boot/dts` device tree'leri, `drivers/` sürücüleri tutar. AM335x'e özel SoC kodu `arch/arm/mach-omap2` 'de.
- `defconfig` → `.config` → `menuconfig` ile ayarlanır. `y` = kernel'in içinde, `m` = modül.
- Derleme: `distclean` → `bb.org_defconfig` → `uImage dtbs LOADADDR=0x80008000` → `modules` → `modules_install INSTALL_MOD_PATH=...` . Her komutta `ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-` .
- Karta giden dosyalar: `uImage` , `am335x-boneblack.dtb` ve rootfs'e kurulan modüller.

10.11 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~/bbb/linux (temsili hatalar)
mehmet@ubuntu:~/bbb/linux$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs -j4
multiple (or no) load addresses:
This is incompatible with uImages
Specify LOADADDR on the commandline to build an uImage  <-- LOADADDR=... eksik

"mkimage" command not found - U-Boot images will not be built  <-- u-boot-tools

/bin/sh: 1: lzop: not found                              <-- sudo apt install lzop

scripts/extract-cert.c:21:10: fatal error: openssl/bio.h: No such file  <-- libssl-dev

/bin/sh: 1: bc: not found                                <-- sudo apt install bc
```

Belirti	Sebep	Çözüm
`Specify LOADADDR on the commandline to build an uImage`	`uImage` için yükleme adresi verilmedi	Komuta `LOADADDR=0x80008000` ekle
`"mkimage" command not found`	`u-boot-tools` kurulu değil	`sudo apt install u-boot-tools`
`lzop: not found`	Kernel LZO ile sıkıştırılıyor	`sudo apt install lzop`

Belirti	Sebep	Çözüm
`openssl/bio.h: No such file`	`libssl-dev` eksik	`sudo apt install libssl-dev`
`bc: not found`	`bc` eksik	`sudo apt install bc`
Derleme x86 için yapılıyor, `.config`'te ARM seçenekleri yok	Bir komutta `ARCH=arm` unutuldu	`distclean` ile temizleyip her komutta `ARCH=arm` yazarak baştan başla
`Starting kernel ...` sonrası hiçbir şey yok	`console=` yanlış ya da DTB uyumsuz	Kernel 5.10'da `console=ttyS0`. DTB'yi aynı kernel kaynağından derle.
`VFS: Unable to mount root fs on unknown-block(179,...)`	`root=` yanlış bölümü gösteriyor	U-Boot'ta `mmc part`, Linux'ta (açılabiliyorsa) `lsblk` ile doğru bölümü bul
Ubuntu'nun `/lib/modules` klasöründe ARM modülleri var	`modules_install` `INSTALL_MOD_PATH` olmadan `sudo` ile çalıştırıldı	O sürüm klasörünü `/lib/modules`'ten sil, komutu `INSTALL_MOD_PATH` ile tekrar çalıştır

Bölüm 11 — Linux Device Tree

> **Bu bölümde:**
>
> - Kernel'in, kendini tanıtmayan donanımı (UART, I2C, GPIO...) nereden öğrendiğini anlayacaksın.
> - DTS, DTSI, DTB, DTBO dosyalarını ve `dtc` derleyicisini tanıyacaksın.
> - Device tree söz dizimini, `compatible` ile driver eşleşmesini ve BeagleBone Black'in include zincirini göreceksin.
> - Çalışan sistemde device tree'yi `/proc/device-tree` altından inceleyeceksin.
>
> **Ön koşul:** Bölüm 3 — boot zinciri, Bölüm 4 — U-Boot'un DTB'yi RAM'e yüklemesi.

11.1 Problem: Kernel Donanımı Nereden Bilecek?

Kernel'in çalışabilmesi için kartta **hangi donanımların olduğunu, nerede durduklarını ve nasıl bağlandıklarını** bilmesi gerekir. Örneğin:

- UART denetleyicisinin register'ları hangi adreste?
- Kaç MB RAM var, hangi adresten başlıyor?
- I2C hattına hangi çip, hangi adresle bağlı?
- USR0 LED'i hangi GPIO pinine bağlı?
- Hangi cihaz hangi interrupt hattını kullanıyor?

Keşfedilebilen ve keşfedilemeyen donanım

	Keşfedilebilir (discoverable)	**Keşfedilemez (non-discoverable)**
Örnek	PCI, PCIe, USB	UART, I2C, SPI, GPIO, Ethernet MAC, NAND, SoC içindeki çoğu çevre birimi
Nasıl bulunur?	Cihaz kendini tanıtır (Vendor ID / Device ID). Kernel bus'ı tarayarak bulur.	Cihaz kendini tanıtmaz. Kernel'e **birinin söylemesi gerekir**.

Masaüstü PC'lerde (x86) bu bilgi BIOS/UEFI tarafından **ACPI tabloları** ile verilir. Gömülü sistemlerde (ARM, RISC-V, PowerPC) ise donanımın büyük kısmı SoC'un içinde, belirli adreslerde duran ve kendini tanıtmayan bloklardır. Bu yüzden bu bilgiyi kernel'e bir şekilde aktarmak gerekir.

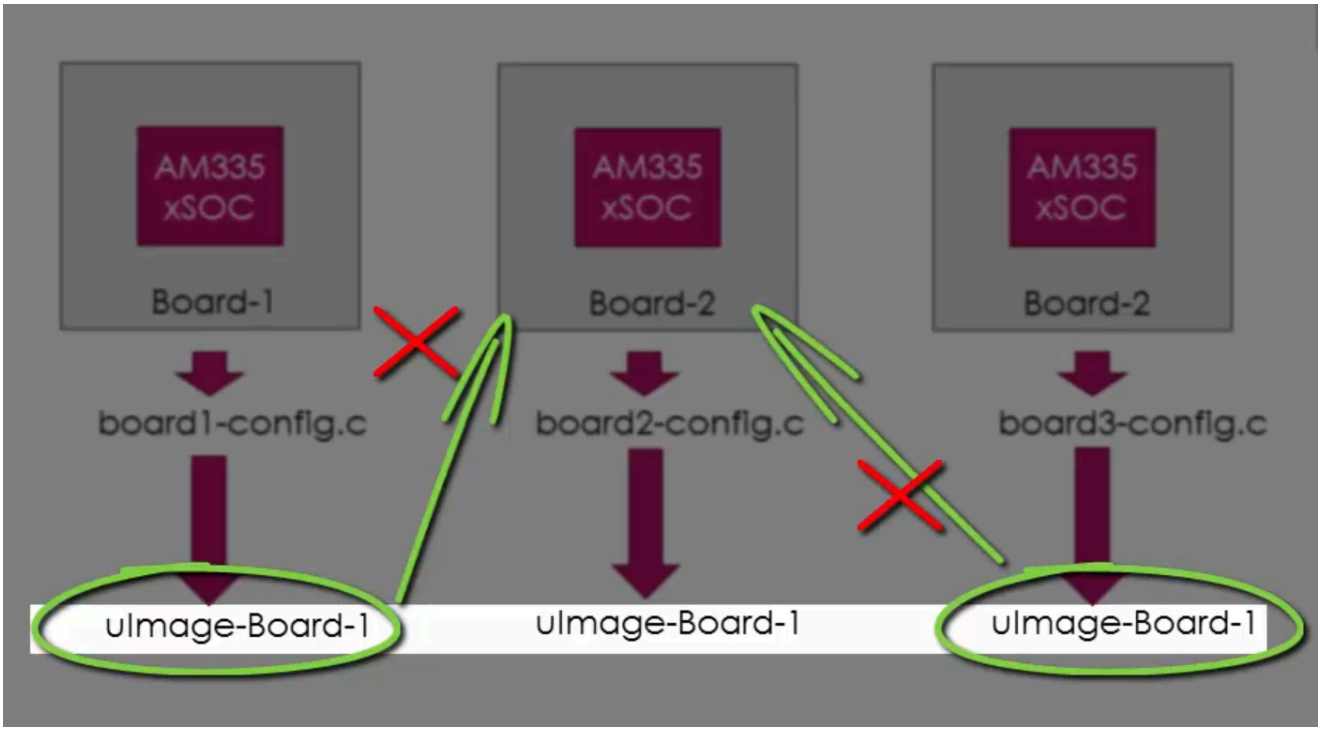
Eski yöntem: board file'lar

Device Tree'den önce ARM'da her kart için kernel kaynak kodunda bir C dosyası (**board file**) yazılırdı (`arch/arm/mach-xxx/board-yyy.c`). Donanım bilgisi bu dosyalara sabit olarak gömülürdü:

```
/* Eski usül: donanım C koduna gömülü (temsili örnek) */
static struct resource uart0_resources[] = {
    { .start = 0x44E09000, .end = 0x44E09FFF, .flags = IORESOURCE_MEM },
    { .start = 72,         .end = 72,         .flags = IORESOURCE_IRQ },
};
```

Bu yöntemin sorunları:

- Her yeni kart = kernel'e yeni C kodu. `arch/arm` altında binlerce kart dosyası birikti.
- Kartta küçük bir değişiklik (ör. LED'in başka pine taşınması) bile **kernel'i yeniden derlemeyi** gerektiriyordu.
- Her kart için ayrı bir kernel image'ı gerekiyordu. **Tek bir kernel, birden fazla kartta çalışamıyordu.**



Görsel: kursun slaytı. Her kartın kendi `boardN-config.c` dosyası ve kendi kernel imajı var; Board-1 için derlenen imaj diğer kartlarda çalışmaz.

2011'de Linus Torvalds ARM tarafındaki bu karmaşadan açıkça şikayet etti. Bunun üzerine ARM dünyası, PowerPC'de zaten kullanılan **Device Tree**'ye geçti (kernel 3.x serisi). Bugün yeni ARM32 ve ARM64 kartları için Device Tree zorunludur.

> **Not**
>
> Temel fikir: **Donanım tarifi koddan ayrılır.** Kernel aynı kalır, kartı tarif eden küçük bir veri dosyası (DTB) değişir. Böylece aynı `zImage` hem BeagleBone Black'te hem de başka bir AM335x kartında çalışabilir. Sadece DTB farklıdır.

11.2 Device Tree Nedir?

Device Tree (DT), bir sistemdeki donanımı tarif eden, **ağaç (tree) şeklinde bir veri yapısıdır**.

- Ağaç **node**'lardan (düğüm) oluşur. Her node bir donanım bloğunu temsil eder (CPU, bellek, UART, I2C denetleyicisi, EEPROM...).
- Her node'un **property**'leri (özellikleri) vardır: adres, interrupt numarası, uyumlu driver adı, durum vb.
- Node'lar iç içe olabilir. Bir I2C denetleyicisinin altında ona bağlı I2C cihazları alt node olarak yer alır. Bu, donanımdaki **fiziksel bağlantı hiyerarşisini** yansıtır.
- Kökeni Sun'ın **OpenFirmware** standardına dayanır. Bugün **devicetree.org** tarafından Devicetree Specification olarak standartlaştırılmıştır.
- **İşletim sisteminden bağımsızdır**: Linux, U-Boot, FreeBSD, Zephyr gibi projeler de kullanır.

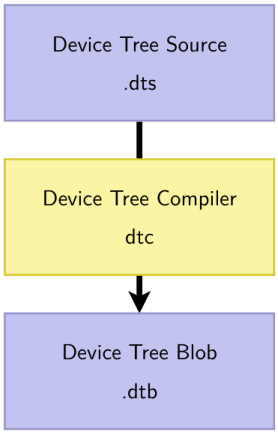
> **Önemli**
>
> Device Tree **donanımın ne olduğunu** tarif eder, **nasıl kullanılacağını** değil. "Bu kartta 0x44E09000 adresinde bir UART var" bir DT bilgisidir. "Bu UART'ı 115200 baud ile konsol olarak kullan" ise bir yapılandırma tercihidir ve genelde kernel command line'a (`bootargs`) aittir.

Device Tree'de olmayan bir şey kernel tarafından **bilinemez**. Driver kernel'e derlenmiş olsa bile, DT'de ona uyan bir node yoksa o driver hiç çalışmaz (probe edilmez).

11.3 Dosya Türleri: DTS, DTSI, DTC, DTB, DTBO

Device Tree: from source to blob

- A tree data structure describing the hardware is written by a developer in a **Device Tree Source** file, `.dts`
- Processed by the **Device Tree Compiler**, `dtc`
- Produces a more efficient representation: **Device Tree Blob**, `.dtb`
- Additional C preprocessor pass
- `.dtb` → accurately describes the hardware platform in an **OS-agnostic** way.
- `.dtb` ≈ few dozens of kilobytes
- DTB also called **FDT**, *Flattened Device Tree*, once loaded into memory.
  - `fdt` command in U-Boot
  - `fdt_` APIs



Görsel: Bootlin, "Linux kernel and driver development" eğitim slaytları (CC BY-SA 3.0)

Kısaltma	Açılımı	Ne?
DT	Device Tree	Genel kavram: donanımı tarif eden ağaç yapısı
.dts	Device Tree **Source**	İnsanın okuyup yazabildiği **metin** kaynak dosyası. Bir kartı tarif eder. Örn: `am335x-boneblack.dts`
.dtsi	Device Tree Source **Include**	Başka dosyalara `#include` edilen ortak parça. Genelde SoC'u ya da bir kart ailesini tarif eder. Örn: `am33xx.dtsi`. C'deki `.h` dosyalarına benzer.
dtc	Device Tree **Compiler**	`.dts` dosyasını `.dtb`'ye derleyen (ve tersini yapabilen) araç
.dtb	Device Tree **Blob** (Binary)	Derlenmiş, **ikili (binary)** çıktı. Bootloader bunu RAM'e yükler ve kernel bunu okur. Örn: `am335x-boneblack.dtb`
FDT	Flattened Device Tree	DTB'nin bellekteki "düzleştirilmiş" formatının adı. U-Boot'taki `fdt` komutu ve `fdtaddr` değişkeni buradan gelir.
.dtbo	Device Tree Blob **Overlay**	Var olan bir DTB'ye çalışma/boot anında eklenen **yama**. Cape/HAT gibi eklenti kartlar için kullanılır. (11.12)

Derleme akışı

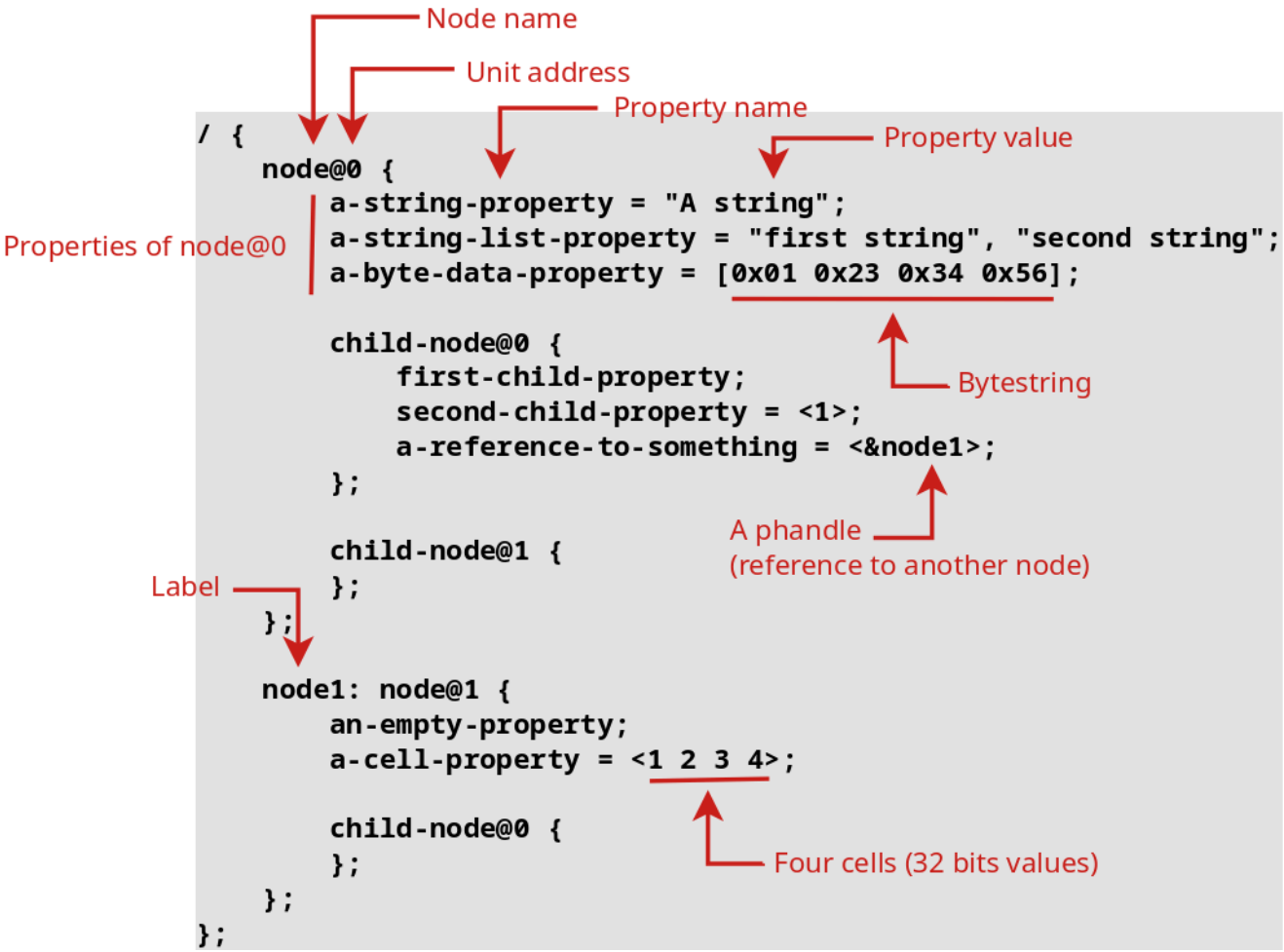
```
am33xx.dtsi ─┐
am335x-bone-common.dtsi ─┤  #include
am335x-boneblack-common.dtsi ─┤
                              ▼
            am335x-boneblack.dts
                    │
                    │  1) C preprocessor (cpp): #include, #define çözülür
                    │  2) dtc: metin → binary
                    ▼
            am335x-boneblack.dtb   (~60-70 KB)
                    │
                    │  U-Boot RAM'e yükler (fdtaddr)
                    ▼
                Linux kernel
```

- Kernel kaynak ağacında `.dts` dosyaları önce **C ön işlemcisinden (cpp)** geçer. Bu sayede `#include <dt-bindings/gpio/gpio.h>` ve `GPIO_ACTIVE_HIGH` gibi sabitler kullanılabilir. Sonra `dtc` ile derlenir.
- DTB küçüktür (birkaç on KB) ve kernel'in hızlıca okuyabileceği bir formattadır.
- Her DTB dosyası `0xd00dfeed` sihirli sayısı (magic number) ile başlar. Bir dosyanın gerçekten DTB olup olmadığını böyle anlayabilirsin:

```
$ hexdump -C am335x-boneblack.dtb | head -1     # ilk 4 byte: d0 0d fe ed
00000000  d0 0d fe ed 00 01 0a 5c  00 00 00 38 00 00 f8 a8  |.......\...8....|
```

> **Not**
>
> Kernel tarafındaki `.dts` dosyaları kernel kaynak kodunun içindedir: ARM32 için `arch/arm/boot/dts/`, ARM64 için `arch/arm64/boot/dts/<vendor>/`. Kernel 6.5'ten itibaren ARM32 dosyaları da vendor klasörlerine taşındı. BeagleBone Black'in dosyası yeni kernel'lerde `arch/arm/boot/dts/ti/omap/am335x-boneblack.dts` konumundadır.

11.4 Söz Dizimi (Syntax)



Görsel: Bootlin eğitim materyalleri (CC BY-SA 3.0)

Node

```
[label:] node-name[@unit-address] {
    [properties]
    [child nodes]
};
```

Parça	Açıklama	Örnek
`/`	Kök (root) node. Her DT'de bir tane vardır ve tüm sistemi temsil eder.	`/ { ... };`
node-name	Cihazın **türünü** anlatan genel bir isim	`serial`, `i2c`, `gpio`, `memory`, `eeprom`
@unit-address	Node'un adresi. Genelde `reg` property'sinin ilk değeriyle aynıdır.	`serial@44e09000`, `eeprom@50`
label	Node'a verilen takma ad. Başka yerlerden `&label` diye referans verilir.	`uart0: serial@44e09000`
`;`	Her node ve property `;` ile biter. Unutmak en sık yapılan derleme hatasıdır.	

Bir node'un tam yolu (path) dosya sistemi yoluna benzer: `/ocp/interconnect@44c00000/.../serial@0`.

Property

Property'ler `isim = değer;` şeklindedir. Değer türleri:

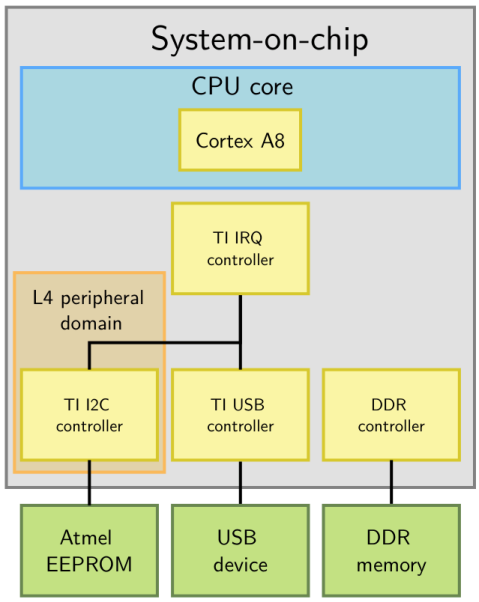
Tür	Yazım	Örnek
Boş (boolean)	Sadece isim. Varsa "true", yoksa "false" demektir.	`gpio-controller;`
String	Çift tırnak	`status = "okay";`
String listesi	Virgülle ayrılmış string'ler	`compatible = "ti,am3352-uart", "ti,omap3-uart";`
Cell (u32) listesi	`< >` içinde 32-bit sayılar	`reg = <0x44e09000 0x2000>;`
Byte dizisi	`[ ]` içinde hex byte'lar	`local-mac-address = [00 11 22 33 44 55];`
phandle (referans)	`< >` içinde `&label`	`gpios = <&gpio1 21 GPIO_ACTIVE_HIGH>;`

> **İpucu**
>
> **phandle**, bir node'un başka bir node'a "işaretçi" (pointer) gibi referans vermesidir. Yukarıdaki `gpios` örneği şunu söyler: "Bu LED, `gpio1` denetleyicisinin **21.** pinine bağlı ve **active-high**." dtc derleme sırasında `&gpio1` 'i o node'a ait benzersiz bir sayıya (phandle) çevirir.

11.5 Bir Device Tree'nin Genel Yapısı

DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0x80000000 { ... };
  chosen { ... };
  ocp {
    intc: interrupt-controller@48200000 { ... };
    usb0: usb@47401300 { ... };
    l4_per: interconnect@44c00000 {

      i2c0: i2c@40012000 {
        compatible = "ti,omap4-i2c";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x0 0x1000>;
        interrupts = <70>;
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&i2c0_pins>;
        clock-frequency = <400000>;

        baseboard_eeprom: eeprom@50 {
          compatible = "atmel,24c256";
          reg = <0x50>;
        };
      };
    };
  };
};
```



Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0). Soldaki ağaç, sağdaki AM335x tabanlı donanımı tarif ediyor.

Görseldeki eşleşmeye dikkat et: DT'deki iç içe yapı **donanımdaki bağlantıyı** birebir yansıtır. EEPROM, I2C denetleyicisine bağlı olduğu için DT'de de `i2c@40012000` node'unun **altındadır**.

Hemen her DT'de bulunan standart node'lar:

Node	Görevi
`/`	Kök. `model` (kartın adı) ve `compatible` (kartın kimliği) burada tanımlanır.
`cpus`	İşlemci çekirdekleri (BBB'de tek bir Cortex-A8)
`memory@...`	RAM'in başlangıç adresi ve boyutu
`chosen`	Donanım değil, bootloader ile kernel arasındaki bilgiler: `bootargs`, `stdout-path` (konsol)
`aliases`	Kısa isimler. Örn: `serial0 = &uart0;` sayesinde `uart0` → `/dev/ttyS0` olur.
`ocp` / `soc`	SoC içindeki çevre birimlerinin bulunduğu bus. AM335x'te adı `ocp`'dir (On-Chip Peripherals).

BeagleBone Black'ten gerçek bir örnek

BBB'nin ana DTS dosyası aslında çok kısadır. Asıl iş include ettiği dosyalardadır:

```
/* am335x-boneblack.dts (kısaltılmış) */
/dts-v1/;

#include "am33xx.dtsi"                    /* AM335x SoC tanımı */
#include "am335x-bone-common.dtsi"        /* Tüm BeagleBone'larda ortak */
#include "am335x-boneblack-common.dtsi"   /* BBB ve BBB Wireless'ta ortak */
#include "am335x-boneblack-hdmi.dtsi"     /* HDMI desteği */

/ {
    model = "TI AM335x BeagleBone Black";
    compatible = "ti,am335x-bone-black", "ti,am335x-bone", "ti,am33xx";
};
```

`am335x-bone-common.dtsi` içinden USR LED tanımı (kısaltılmış):

```
leds {
    pinctrl-names = "default";
    pinctrl-0 = <&user_leds_s0>;      /* pin mux ayarı */
    compatible = "gpio-leds";         /* hangi driver? → leds-gpio */

    led2 {
        label = "beaglebone:green:usr0";
        gpios = <&gpio1 21 GPIO_ACTIVE_HIGH>;    /* GPIO1_21 pini */
        linux,default-trigger = "heartbeat";    /* kalp atışı gibi yanıp söner */
        default-state = "off";
    };
    /* usr1, usr2, usr3 ... */
};
```

> **İpucu**
>
> Kart açıldığında USR0 LED'inin "kalp atışı" gibi yanıp sönmesinin sebebi tam olarak bu satırdır: `linux,default-trigger = "heartbeat";` . Yani bu davranış kernel kodunda değil, **Device Tree'de** tanımlıdır.

11.6 Önemli Property'ler

Property	Anlamı	Örnek
`compatible`	**En önemli property.** Cihazın ne olduğunu ve hangi driver'ın onu yöneteceğini belirler. Format: `"üretici,model"` . En özelden en genele doğru sıralanır.	`"ti,am3352-uart"`, `"ti,omap3-uart"`

Property	Anlamı	Örnek
`reg`	Cihazın adresi. Memory-mapped cihazlarda **(başlangıç adresi, boyut)** çifti. I2C/SPI cihazlarında bus üzerindeki adres.	`reg = <0x44e09000 0x2000>;` `reg = <0x50>;`
`#address-cells`	Alt node'ların `reg` değerindeki **adresin** kaç adet 32-bit cell olduğu	`<1>`
`#size-cells`	Alt node'ların `reg` değerindeki **boyutun** kaç cell olduğu. I2C'de boyut yoktur, bu yüzden `<0>`.	`<1>` veya `<0>`
`status`	Cihaz kullanılıyor mu? `"okay"` → evet, `"disabled"` → hayır (driver probe edilmez).	`status = "okay";`
`interrupts`	Cihazın kullandığı interrupt hattı	`interrupts = <72>;`
`interrupt-parent`	Interrupt'ın hangi interrupt denetleyicisine gittiği (phandle)	`<&intc>`
`clocks`	Cihazın beslendiği saat (clock) kaynakları	`clocks = <&l4ls_gclk>;`
`pinctrl-0`, `pinctrl-names`	Pin multiplexing: SoC pinlerinin bu cihaz için hangi moda alınacağı	`pinctrl-0 = <&uart0_pins>;`
`gpios`, `*-gpios`	Kullanılan GPIO pinleri (phandle + pin no + flag)	`<&gpio1 21 GPIO_ACTIVE_HIGH>`

reg, #address-cells ve #size-cells ilişkisi

`reg`'in nasıl okunacağını **parent node'daki** `#address-cells` ve `#size-cells` belirler:

```
ocp {
    #address-cells = <1>;   /* alt node'larda adres = 1 cell */
    #size-cells = <1>;      /* alt node'larda boyut = 1 cell */

    serial@44e09000 {
        reg = <0x44e09000 0x2000>;   /* adres=0x44E09000, boyut=8 KB */
    };
};

i2c0: i2c@44e0b000 {
    #address-cells = <1>;
    #size-cells = <0>;      /* I2C cihazlarının "boyutu" yok */

    baseboard_eeprom: eeprom@50 {
        compatible = "atmel,24c256";
        reg = <0x50>;       /* I2C adresi 0x50 */
    };
};
```

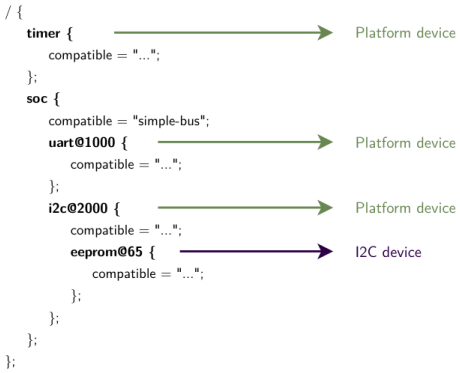
> **Not**
>
> BBB'nin üzerindeki **EEPROM** (kart adını ve seri numarasını tutan çip) gerçekten de I2C0 hattında **0x50** adresindedir. U-Boot bu EEPROM'u okuyarak kartın BeagleBone Black olduğunu anlar ve doğru DTB'yi (`am335x-boneblack.dtb`) seçer.

11.7 compatible ile Driver Eşleşmesi

Kernel DTB'yi okuduğunda her node için bir **device** oluşturur. Sonra her device için `compatible` string'ine bakar ve **aynı string'i destekleyen bir driver** arar. Eşleşme bulunursa driver'ın `probe()` fonksiyonu çağrılır ve cihaz o driver'a **bağlanır (bind)**.

`compatible` property and Linux kernel drivers

- Linux identifies as **platform devices**:
  - Top-level DT nodes with a `compatible` string
  - Sub-nodes of `simple-bus`
    - Instantiated automatically at boot time
- Sub-nodes of I2C controllers → *I2C devices*
- Sub-nodes of SPI controllers → *SPI devices*
- Each Linux driver has a table of compatible strings it supports
  - struct of_device_id[]
- When a DT node compatible string matches a given driver, the device is *bound* to that driver.



Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)

- Kök node'un hemen altındaki ve `simple-bus` altındaki node'lar → **platform device** olur.
- I2C denetleyicisinin altındaki node'lar → **I2C device** olur.
- SPI denetleyicisinin altındaki node'lar → **SPI device** olur.

Driver tarafında bu eşleşme `of_device_id` tablosu ile yapılır (`of` = OpenFirmware):

```
/* drivers/i2c/busses/i2c-omap.c (kısaltılmış) */
static const struct of_device_id omap_i2c_of_match[] = {
    { .compatible = "ti,omap4-i2c", .data = &omap4_pdata },
    { .compatible = "ti,omap3-i2c", .data = &omap3_pdata },
    { },
};
MODULE_DEVICE_TABLE(of, omap_i2c_of_match);

static struct platform_driver omap_i2c_driver = {
    .probe  = omap_i2c_probe,
    .remove = omap_i2c_remove,
    .driver = {
        .name           = "omap_i2c",
        .of_match_table = omap_i2c_of_match,   /* DT eşleşme tablosu */
    },
};
```

```
DT node:   compatible = "ti,omap4-i2c"
                    │
                    │  string eşleşmesi
                    ▼
Driver:    { .compatible = "ti,omap4-i2c" }  →  omap_i2c_probe() çağrılır
```

Driver `probe()` içinde `reg`, `interrupts`, `clocks` gibi property'leri okuyarak cihazı başlatır (`platform_get_resource()`, `of_property_read_u32()` vb.).

> **İpucu**
>
> `compatible` listesinde birden fazla string olmasının sebebi **geriye dönük uyumluluktur**. `"ti,am3352-uart", "ti,omap3-uart"` şunu söyler: "Ben aslında AM3352 UART'ıyım. Bana özel driver yoksa OMAP3 UART driver'ı da beni sürebilir." Kernel listeyi soldan sağa dener.

> **Not**
>
> Bir cihazın property'lerinin nasıl yazılacağı (hangi property zorunlu, hangi değerler geçerli) **binding** denen belgelerle tanımlanır. Bunlar kernel'de `Documentation/devicetree/bindings/` altındadır ve günümüzde YAML formatındadır. Yeni bir cihaz eklerken ilk bakılacak yer burasıdır.

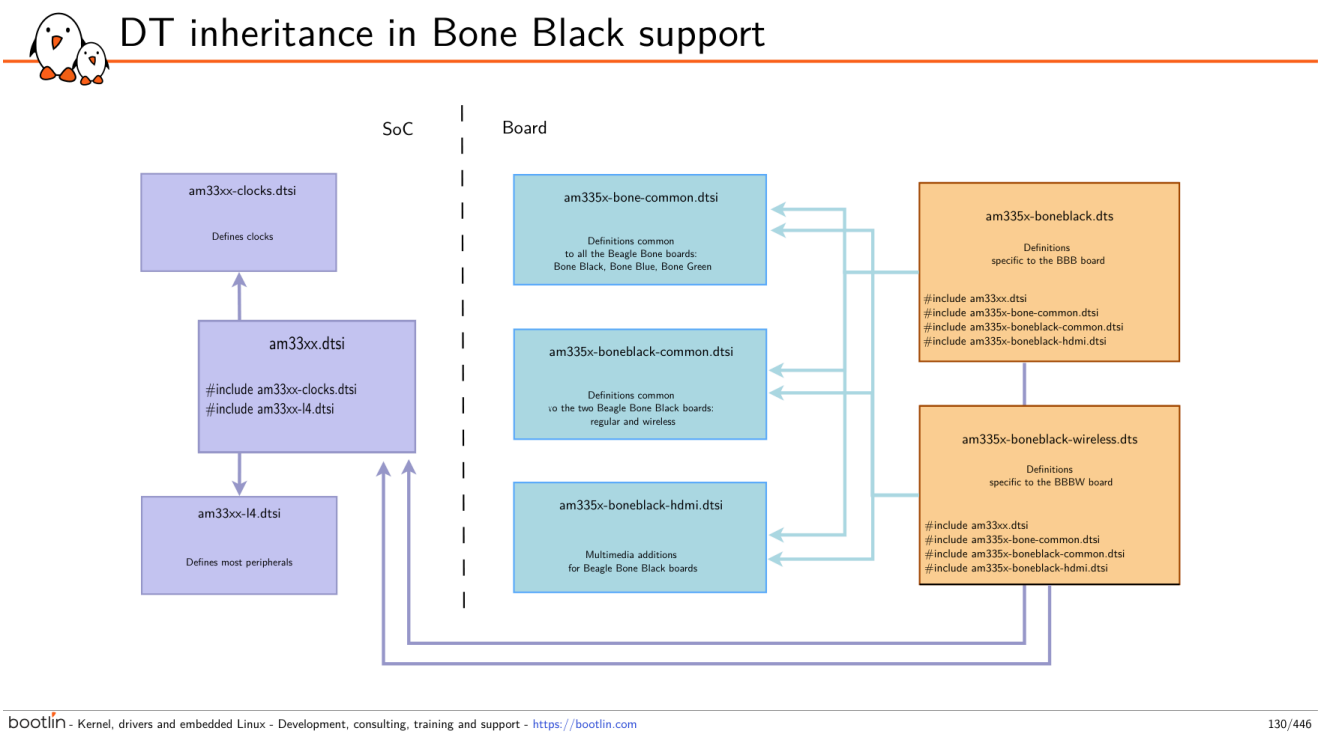
11.8 Kalıtım: .dtsi, include ve Label ile Override

Aynı SoC'u kullanan onlarca kart vardır. Her kart için SoC'un tamamını baştan yazmak yerine:

1. SoC'un tüm çevre birimleri bir `.dtsi` dosyasında tanımlanır. Çoğu `status = "disabled"` olarak bırakılır.
2. Kartın `.dts` dosyası bu `.dtsi`'yi include eder. Kartta kullanılan birimleri `status = "okay"` yaparak açar ve karta özel cihazları (EEPROM, LED, PMIC...) ekler.

Aynı node birden fazla kez tanımlanırsa dtc bunları **birleştirir**. Aynı property iki yerde varsa **sonra gelen kazanır**.

BeagleBone Black'in include zinciri



Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)

Dosya	Seviye	İçerik
`am33xx.dtsi`	SoC	AM335x işlemcinin genel tanımı. `am33xx-l4.dtsi` (çevre birimleri) ve `am33xx-clocks.dtsi` (saatler) dosyalarını include eder.
`am335x-bone-common.dtsi`	Kart ailesi	Tüm BeagleBone'larda ortak olanlar: LED'ler, EEPROM, PMIC (TPS65217), 256 MB RAM
`am335x-boneblack-common.dtsi`	Kart ailesi	BBB ve BBB Wireless'ta ortak olanlar: eMMC, 512 MB RAM
`am335x-boneblack-hdmi.dtsi`	Ek özellik	HDMI çıkışı (TDA19988 çipi)
`am335x-boneblack.dts`	Kart	Sadece BBB'ye özel olanlar: `model`, `compatible`

Override'a güzel bir örnek RAM boyutudur. `am335x-bone-common.dtsi` RAM'i 256 MB tanımlar (eski BeagleBone White'ta 256 MB vardı). `am335x-boneblack-common.dtsi` aynı node'u tekrar yazarak bunu 512 MB'a çıkarır:

```
/* am335x-bone-common.dtsi */
memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x10000000>;   /* 256 MB */
};

/* am335x-boneblack-common.dtsi (sonra include edildiği için kazanır) */
memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x20000000>;   /* 512 MB */
};
```

Label ile override (tercih edilen yöntem)

Bir node'u değiştirmek için tam yolunu tekrar yazmak yerine, **label**'ı ile doğrudan ona erişmek tercih edilir:

Inheritance and labels

Doing:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  ocp {
    serial@0 {
      status = "okay";
    };
  };
};
```

Is exactly equivalent to:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

&uart0 {
  status = "okay";
};
```

→ this solution is preferred

Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0). Soldaki ve sağdaki board.dts tamamen aynı sonucu üretir. Sağdaki `&uart0 { ... };` yazımı tercih edilir.

```
/* Kart dosyasında: SoC'ta "disabled" olan I2C2'yi aç ve hızını ayarla */
&i2c2 {
    status = "okay";
    clock-frequency = <100000>;
};
```

11.9 dtc Kullanımı

dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dtb -O dts foo.dtb
/dts-v1/;

/ {
        welcome = <0xbadcafe>;

        bootlin {
                webinar = "great";
                demo = <0x01 0x02 0x03>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```


Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)

Kurulum

```
sudo apt install device-tree-compiler
```

Temel komutlar

```
# DTS → DTB (derleme)
dtc -I dts -O dtb -o foo.dtb foo.dts

# DTB → DTS (geri derleme / decompile)
dtc -I dtb -O dts -o foo_geri.dts foo.dtb

# Çalışan sistemin canlı device tree'sini DTS olarak dök
dtc -I fs -O dts /proc/device-tree
```

Parametre	Anlamı
`-I`	Girdi formatı: `dts`, `dtb` veya `fs` (dosya sistemi, ör. `/proc/device-tree`)
`-O`	Çıktı formatı: `dts`, `dtb` veya `asm`
`-o`	Çıktı dosyası
`-@`	Label'ları (`__symbols__`) DTB'ye ekle. Overlay kullanılacaksa gereklidir.

> **İpucu**
>
> Geri derleme (`-I dtb -O dts`) çok işe yarar: Elinde sadece bir `.dtb` varsa (ör. hazır bir image'dan çıkan) içinde ne olduğunu görebilirsin. Ancak çıkan DTS'te yorumlar, `#include` 'lar, `#define` isimleri ve label'lar kaybolmuştur. Sayılar ham hex olarak görünür (`GPIO_ACTIVE_HIGH` yerine `0x00` gibi).

Kernel kaynak ağacında DTB derleme

Kernel'in içindeki `.dts` dosyaları elle `dtc` çağırmak yerine kernel'in build sistemiyle derlenir. Bu yöntem cpp ön işleme adımını da otomatik yapar:

```
# Tüm DTB'leri derle (kernel kaynak klasöründe)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- dtbs

# Sadece BBB'nin DTB'sini derle (kernel 5.10)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x-boneblack.dtb

# Çıktı:
#   arch/arm/boot/dts/am335x-boneblack.dtb
```

> **Not**
>
> Kernel 6.5 ve sonrasında (Bölüm 13'teki 6.18.1 gibi) hedef `ti/omap/am335x-boneblack.dtb` 'dir ve çıktı `arch/arm/boot/dts/ti/omap/am335x-boneblack.dtb` konumuna yazılır.

11.10 Boot Sürecinde Device Tree

Linux boot sürecinde gördüğümüz gibi U-Boot, kernel'i başlatmadan önce **hem kernel image'ını hem de DTB'yi** RAM'e yükler:

```
eMMC / SD kart
  /boot/zImage, /boot/am335x-boneblack.dtb
        │  U-Boot: load
        ▼
DDR RAM
  loadaddr = 0x82000000 → kernel
  fdtaddr  = 0x88000000 → DTB
        │  bootz / bootm   (r2 = DTB adresi)
        ▼
Linux kernel
  DTB'yi okur (unflatten), device'ları oluşturur
        │
        ▼
Driver'lar compatible ile eşleşir ve probe edilir
```

SD karttan boot bölümündeki `uEnv.txt` ile birebir aynı mantık:

```
fdtaddr=0x88000000                                  # DTB'nin RAM'deki adresi
load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb  # DTB'yi RAM'e yükle
bootm ${loadaddr} - ${fdtaddr}                      # kernel'i başlat ve DTB adresini ver
```

`bootm` / `bootz` komutunun üç parametresi:

Parametre	Anlamı
`${loadaddr}`	Kernel image'ının RAM adresi
`-`	initramfs adresi (`-` = yok)
`${fdtaddr}`	**DTB'nin RAM adresi**

ARM32'de U-Boot kernel'e atlarken DTB'nin fiziksel adresini `r2` **register'ında** verir. Kernel açılışta bu adresteki blob'u okur, `0xd00dfeed` magic number'ını kontrol eder ve DTB'yi bellekte gezilebilir bir ağaca dönüştürür (**unflatten**).

> **Not**
>
> U-Boot, kernel'e vermeden önce DTB'yi **değiştirebilir** de. Örneğin `bootargs` değişkenini DTB'deki `/chosen` node'una yazar ve RAM boyutunu `/memory` node'una işler. Kernel command line'ının kernel'e ulaşma yollarından biri budur.

U-Boot'ta DTB'yi incelemek

U-Boot'un `fdt` komutuyla RAM'deki DTB'ye bakılabilir:

```
=> load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
=> fdt addr ${fdtaddr}           # hangi adresteki DTB ile çalışılacak
=> fdt print /                   # tüm ağacı yazdır
=> fdt print /leds               # sadece leds node'u
=> fdt list /ocp                 # sadece alt node isimleri
```

11.11 Çalışan Sistemde Device Tree'yi İncelemek

Kernel, okuduğu device tree'yi bir dosya sistemi olarak dışarı açar:

```
/sys/firmware/devicetree/base/     # asıl konum
/proc/device-tree                  # → yukarıdakine sembolik link
```

Burada **her node bir klasör, her property bir dosyadır**:

```
# Kartın modeli
$ cat /proc/device-tree/model
TI AM335x BeagleBone Black

# Kök node'un compatible listesi (string'ler \0 ile ayrılmıştır)
$ cat /proc/device-tree/compatible | tr '\0' '\n'
ti,am335x-bone-black
ti,am335x-bone
ti,am33xx

# LED node'larına bak
$ ls /proc/device-tree/leds/

# Bootloader'ın kernel'e verdiği bootargs
$ cat /proc/device-tree/chosen/bootargs

# Sayısal property'leri hex olarak gör (ör. RAM adresi ve boyutu)
$ hexdump -C /proc/device-tree/memory@80000000/reg
00000000  80 00 00 00 20 00 00 00                           |.... ...|
#         adres=0x80000000  boyut=0x20000000 (512 MB)
```

Tüm ağacı tek seferde okunabilir DTS olarak görmek için:

```
sudo apt install device-tree-compiler
dtc -I fs -O dts /proc/device-tree > canli.dts 2>/dev/null
less canli.dts
```

Bir cihazın hangi driver'a bağlandığını görmek için:

```
ls /sys/bus/platform/devices/          # DT'den oluşturulan platform device'lar
ls -l /sys/bus/platform/devices/44e09000.serial/driver
#  → .../drivers/omap8250   (UART0'ı süren driver, kernel config'e göre serial-omap da olabilir)
```

BBB'nin Debian image'larında DTB dosyaları `/boot/dtbs/<kernel-sürümü>/` klasöründedir:

```
ls /boot/dtbs/$(uname -r)/ | grep boneblack
```

> **İpucu**
>
> Bir donanım çalışmıyorsa ilk kontrol: `/proc/device-tree` altında o cihazın node'u var mı ve `status` değeri `okay` mi? Node yoksa ya da `disabled` ise driver hiç devreye girmez. `dmesg | grep -i <cihaz>` ile de probe hatalarına bakılabilir.

11.12 Device Tree Overlay (.dtbo)

BeagleBone'a **cape** (P8/P9 header'larına takılan eklenti kart) takıldığında donanım değişir. Her cape kombinasyonu için ayrı bir `.dts` yazmak mantıklı değildir. Bunun çözümü **overlay**'dir.

- Overlay, ana DTB'ye **sonradan uygulanan bir yamadır**. Node ekleyebilir veya mevcut node'ları değiştirebilir.
- `.dts` kaynağından `dtc -@` ile `.dtbo` olarak derlenir.
- Ana DTB'nin de `-@` ile derlenmiş olması gerekir (label'ların bulunabilmesi için).
- Genellikle U-Boot tarafından boot sırasında uygulanır.

Basit bir overlay örneği. Yapıyı göstermek için I2C2'nin ayarını değiştiriyor; BBB'de I2C2 aslında zaten açıktır (19.4):

```
/dts-v1/;
/plugin/;                /* bu bir overlay */

&i2c2 {                  /* ana DTB'deki i2c2 label'ına yama */
    status = "okay";
    clock-frequency = <100000>;
};
```

```
dtc -@ -I dts -O dtb -o BB-I2C2-ORNEK.dtbo BB-I2C2-ORNEK.dts
```

BBB'nin resmi Debian image'larında overlay'ler `/boot/uEnv.txt` dosyasından yönetilir. Hazır overlay'ler `/lib/firmware/` klasöründedir:

```
# /boot/uEnv.txt (ilgili satırlar)
enable_uboot_overlays=1
uboot_overlay_addr0=/lib/firmware/BB-I2C2-00A0.dtbo
#disable_uboot_overlay_video=1      # HDMI'ı kapatıp o pinleri serbest bırakır
```

> **Not**
>
> Raspberry Pi'deki `config.txt` içindeki `dtoverlay=...` satırları da aynı mekanizmadır. Overlay kavramı BeagleBone'a özel değildir.

11.13 Özet

Kavram	Bir cümlede
Device Tree	Keşfedilemeyen donanımı kernel'e tarif eden, ağaç yapısındaki veri
.dts	Bir kartın insan tarafından okunabilir kaynak tarifi
.dtsi	Birden fazla `.dts` tarafından include edilen ortak (genelde SoC) tarif
dtc	DTS ↔ DTB dönüşümü yapan derleyici
.dtb	Derlenmiş binary. Bootloader RAM'e yükler, kernel okur.
.dtbo	DTB'ye sonradan uygulanan yama (cape/HAT için)
compatible	Node ile driver'ı eşleştiren string
reg	Cihazın adresi (ve memory-mapped ise boyutu)
status	`"okay"` → kullan, `"disabled"` → kullanma
label / phandle	Node'a takma ad ve başka node'dan ona referans (`&label`)
/proc/device-tree	Çalışan kernel'in gördüğü device tree

BeagleBone Black için akış:

```
am33xx.dtsi + am335x-bone*.dtsi + am335x-boneblack.dts
        │  (cpp + dtc  /  make dtbs)
        ▼
am335x-boneblack.dtb  ──►  SD/eMMC: /boot/...
        │  U-Boot: load → fdtaddr (0x88000000)
        ▼
bootz/bootm ${loadaddr} - ${fdtaddr}   (r2 = DTB adresi)
        │
        ▼
Kernel DTB'yi okur → device'ları oluşturur → compatible ile driver'ları probe eder
```

Kaynaklar

- Görseller: Bootlin - Linux kernel and driver development training slaytları ve bootlin/training-materials deposu. Lisans: CC BY-SA 3.0
- Devicetree Specification - devicetree.org
- Linux kernel dokümantasyonu: Devicetree
- Bootlin - Device Tree 101 webinar
- eLinux.org - Device Tree Usage

Kısım V
Root Filesystem

Bölüm 12 — BusyBox ile Minimal Root Filesystem

> **Bu bölümde:**
>
> - BusyBox'ın ne olduğunu ve neden gömülü sistemlerde bu kadar yaygın kullanıldığını öğreneceksin.
> - BusyBox 1.36.1'i ARM için **statik** derleyip kendi root filesystem'ini sıfırdan kuracaksın.
> - Kernel modüllerini rootfs'e kurup kartı kendi kernel'in ve kendi rootfs'inle açacaksın.
> - BusyBox `init`'ini, `inittab`'ı ve `rcS` başlangıç betiklerini anlayacaksın.
> - USB üzerinden Ethernet sürücüsünü (`g_ether`) açılışta otomatik yükleyeceksin.
> - Rootfs'i NFS ile bilgisayardan bağlamayı göreceksin.
> - İki challenge: SSH ve TFTP desteği eklemek, BusyBox'ı dinamik derlemek.
>
> **Ön koşul:** Bölüm 6 — rootfs klasörleri, Bölüm 9 — cross toolchain, Bölüm 10 — kernel ve modüller.

12.1 Root Filesystem'de Neler Olmalı?

Kernel açılışın sonunda `root=` ile gösterilen dosya sistemini `/` olarak bağlar ve içindeki `/sbin/init`'i çalıştırır (3.4). Bundan sonra olan her şey rootfs'in işidir. Klasörlerin görevlerini 6.6.1'de görmüştük. Çalışan en küçük bir Linux için gerekenler şunlar:

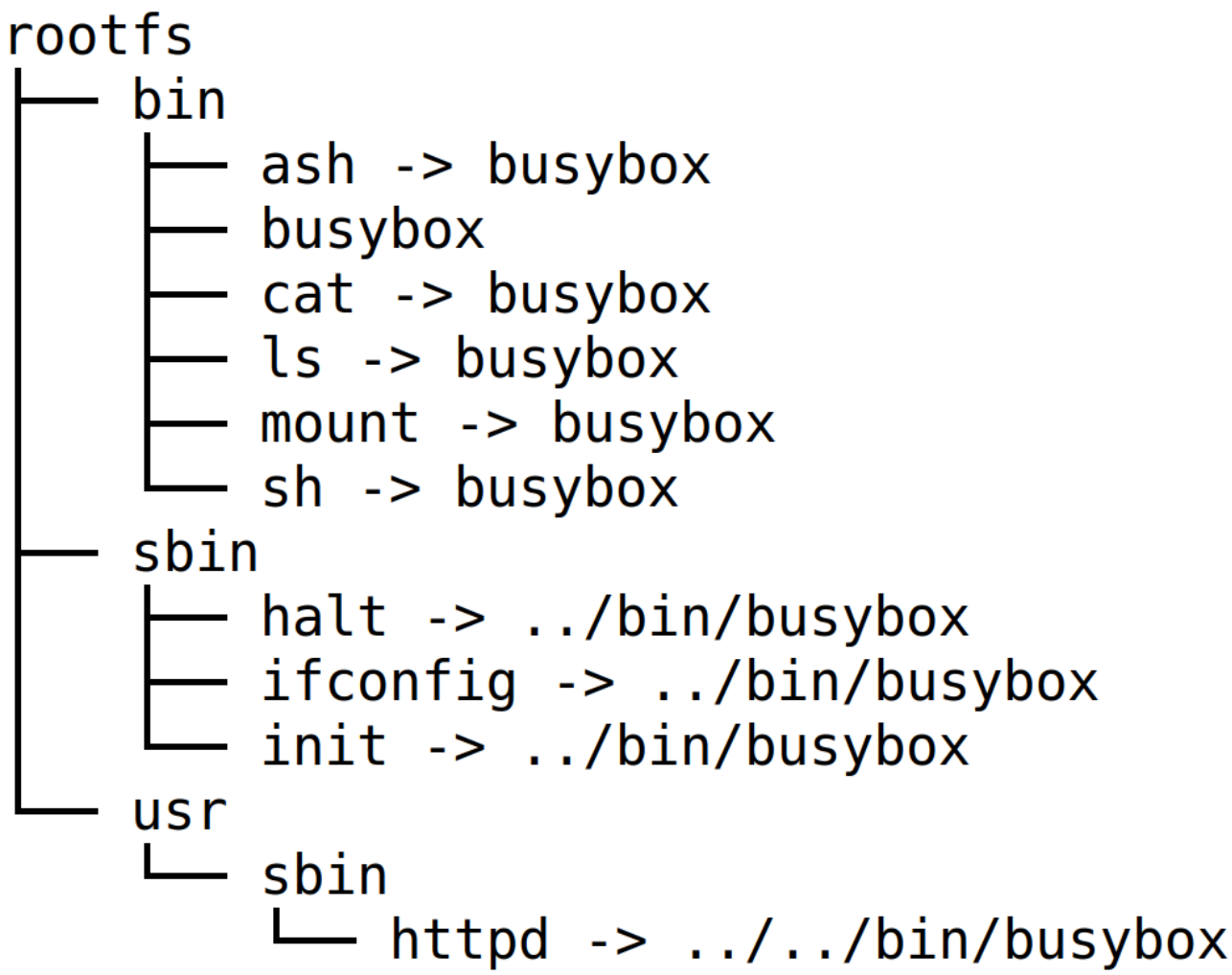
Gereken	Neden?	Bu bölümde nereden?
`/sbin/init`	Kernel'in çalıştırdığı ilk program (PID 1)	BusyBox
Bir kabuk (`/bin/sh`) ve temel komutlar	Sisteme komut verebilmek	BusyBox
`/dev` , `/proc` , `/sys` klasörleri	Kernel'in sanal dosya sistemleri buraya bağlanır	`mkdir`
`/etc/inittab` , `/etc/init.d/rcS`	`init`'in açılışta ne yapacağı	Elle yazacağız
`/lib/modules/<sürüm>/`	Modül olarak derlenen sürücüler	Bölüm 10'daki `modules_install`
`/lib/*.so` (sadece dinamik derlemede)	Paylaşılan kütüphaneler	12.11

> **Not**
>
> Kursun "About RFS" ve "About initramfs" makaleleri bu konunun teorisini anlatıyor. initramfs, bu bölümde kuracağımız klasör yapısının sıkıştırılıp RAM'e yüklenen halidir. Bölüm 7'de kullandığımız `initramfs` dosyası da böyle bir BusyBox rootfs'idir.

12.2 BusyBox Nedir?

BusyBox, `ls`, `cp`, `mount`, `sh`, `init`, `ifconfig`, `vi`, `tftp` gibi yüzlerce Unix komutunu **tek bir küçük programda** toplayan bir projedir. Her komuta **applet** denir.

- Tek bir `busybox` dosyası vardır (birkaç yüz KB ile 2 MB arası).
- `/bin/ls`, `/sbin/init` gibi komutlar bu dosyaya giden **sembolik linklerdir**.
- `ls` olarak çağrılınca BusyBox, çağrıldığı adı (`argv[0]`) okur ve `ls` gibi davranır. `busybox ls` yazmak da aynı işi yapar.



Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)

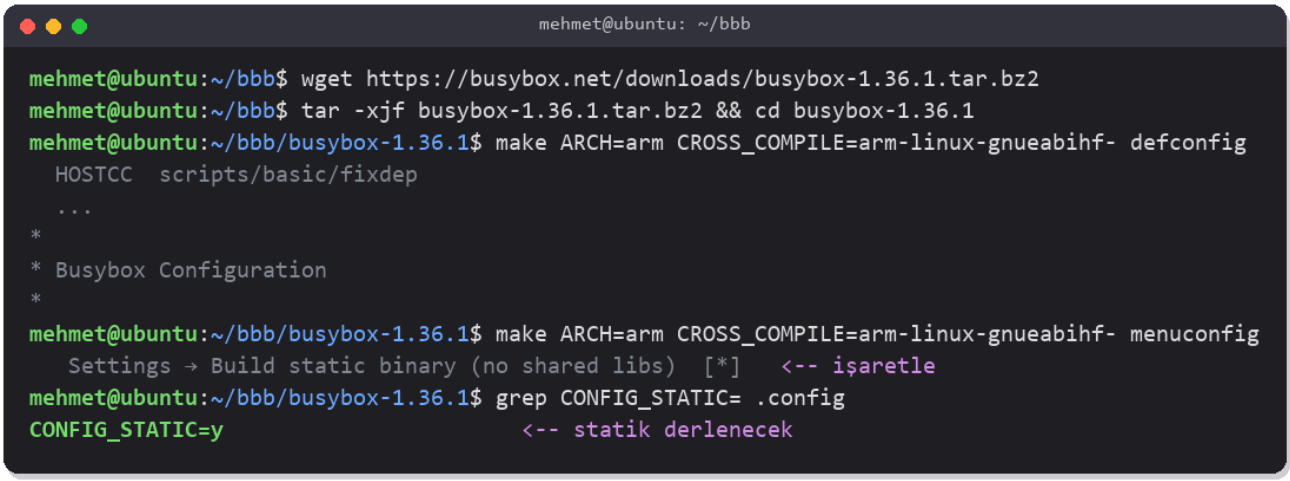
	Masaüstü Linux (GNU coreutils vb.)	BusyBox
Boyut	Yüzlerce MB, her komut ayrı program	Tek dosya, ~1–2 MB
Özellik	Bütün seçenekler	En çok kullanılan seçenekler
Yapılandırma	—	Kernel gibi `menuconfig` ile hangi applet'lerin derleneceği seçilir

---

12.3 Adım Adım: BusyBox'ı Derlemek

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**. Çalışma klasörü `~/bbb` , rootfs `~/bbb/rootfs` . Toolchain Bölüm 9'da kurulan Linaro GCC 7.5.0.

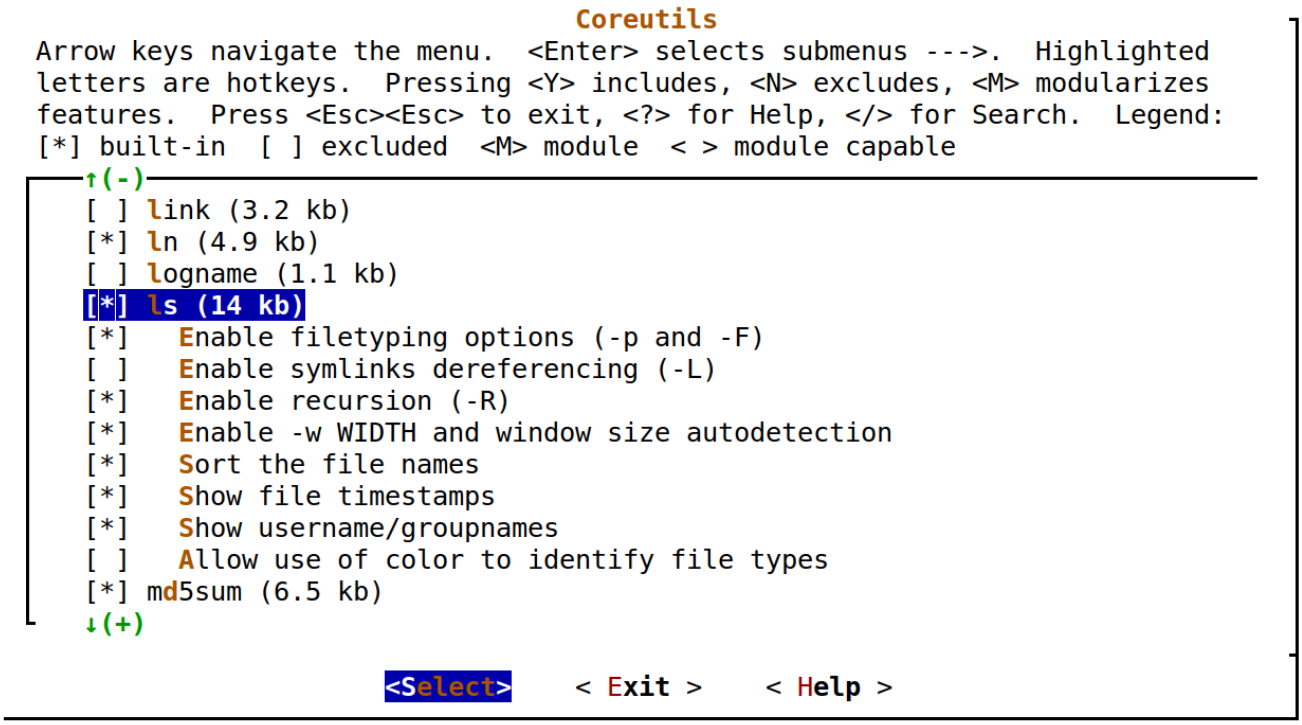
Adım 1 — İndir ve yapılandır




```
cd ~/bbb
wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2
tar -xjf busybox-1.36.1.tar.bz2
cd busybox-1.36.1
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
```

`menuconfig` 'te tek bir ayarı değiştir:

```
Settings  --->
    [*] Build static binary (no shared libs)
```



Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)

	Statik (`CONFIG_STATIC=y`)	**Dinamik**
Kütüphaneler	`busybox` dosyasının **içine** gömülür	Çalışırken rootfs'teki `/lib/*.so` dosyalarından yüklenir
Rootfs'te `/lib` gerekir mi?	Hayır	Evet (12.11)
Ne zaman?	İlk deneme, en basit rootfs. **Kursta kullanılan.**	Rootfs'e başka dinamik programlar da konacaksa

Adım 2 — Derle ve rootfs klasörüne kur

```
mehmet@ubuntu: ~/bbb/busybox-1.36.1
mehmet@ubuntu:~/bbb/busybox-1.36.1$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)
  ...
  LINK    busybox_unstripped
Static linking against glibc, can't use --gc-sections
Trying libraries: crypt m resolv rt
Final link with: m resolv rt
mehmet@ubuntu:~/bbb/busybox-1.36.1$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=~/bbb/rootfs install
  ~/bbb/rootfs/bin/ash -> busybox
  ~/bbb/rootfs/bin/cat -> busybox
  ...
--------------------------------------------------
You will probably need to make your busybox binary
setuid root to ensure all configured applets will
work properly.
--------------------------------------------------
mehmet@ubuntu:~/bbb/busybox-1.36.1$ file ~/bbb/rootfs/bin/busybox
~/bbb/rootfs/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
statically linked, for GNU/Linux 3.2.0, stripped      <-- ARM + statik
```

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=~/bbb/rootfs install
file ~/bbb/rootfs/bin/busybox
```

`install` hedefi `busybox`'ı `~/bbb/rootfs/bin/` altına koyar ve bütün applet linklerini (`bin/`, `sbin/`, `usr/bin/`, `usr/sbin/`) oluşturur. `file` çıktısında **ARM** ve **statically linked** görmelisin.

> **İpucu**
>
> Statik derlemede `Using 'getpwnam' in statically linked applications requires at runtime the shared libraries...` gibi uyarılar görebilirsin. Bunlar glibc'nin statik bağlamadaki bilinen uyarılarıdır, derleme başarılıdır.

Adım 3 — Kernel modüllerini kur

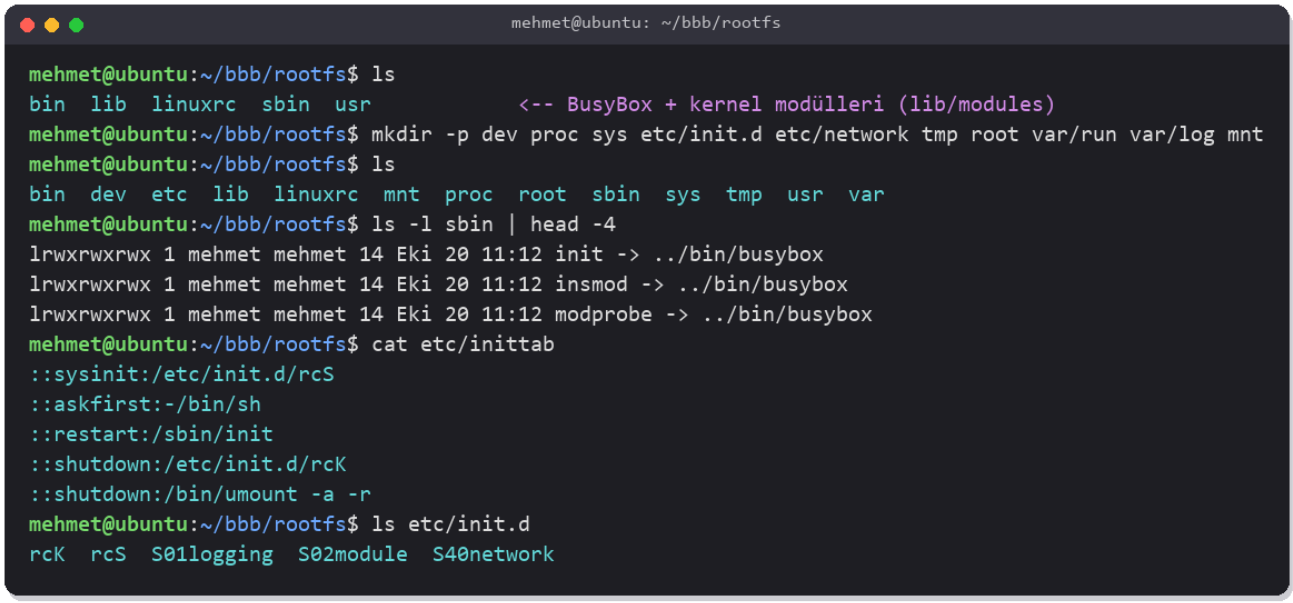
Kursun "Kernel Modules installation" dersi: Bölüm 10'da derlediğin modülleri aynı rootfs klasörüne kur. Bunu daha önce yaptıysan atlayabilirsin.

```
cd ~/bbb/linux
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=~/bbb/rootfs modules_install
ls ~/bbb/rootfs/lib/modules/          # 5.10.162
```

> **Önemli**
>
> `/lib/modules/` altındaki klasör adı, çalışan kernel'in `uname -r` çıktısıyla **birebir aynı** olmalı. Kernel'i yeniden derleyip sürüm değiştiysen modülleri de yeniden kur. Aksi halde `modprobe` modülleri bulamaz.

12.4 Rootfs'i Tamamlamak

BusyBox sadece programları kurar. Kalan klasörleri ve ayar dosyalarını elle oluşturuyoruz:




```
cd ~/bbb/rootfs
mkdir -p dev proc sys etc/init.d tmp root var/run var/log mnt
mkdir -p etc/network/if-pre-up.d etc/network/if-up.d etc/network/if-down.d etc/network/if-post-down.d
```

Klasör	Neden boş da olsa gerekli?
`dev`	Kernel buraya `devtmpfs`'i bağlar ve aygıt dosyalarını (`/dev/ttyS0`, `/dev/mmcblk0` ...) kendisi oluşturur. `bb.org_defconfig`'te `CONFIG_DEVTMPFS_MOUNT=y` olduğu için ayrıca bir şey yapmana gerek yok.
`proc`, `sys`	`rcS` buraya `proc` ve `sysfs` sanal dosya sistemlerini bağlayacak
`tmp`, `var/run`, `var/log`	Geçici dosyalar, PID dosyaları ve loglar
`etc/network/if-*.d`	BusyBox'ın `ifup` komutu bu klasörleri arar; yoksa hata verir

> **Not**
>
> `/dev` klasörünü elle doldurmana (`mknod`) gerek yok. Eskiden aygıt dosyaları elle oluşturulurdu; bugün `devtmpfs` bunu kernel tarafında otomatik yapar.

12.5 BusyBox `init`, `inittab` ve `rcS`

Kernel rootfs'i bağladıktan sonra şu zincir çalışır:

```
kernel  ──►  /sbin/init  (BusyBox, PID 1)
                 │  /etc/inittab'ı okur
                 ├── ::sysinit   → /etc/init.d/rcS        (bir kez, açılışta)
                 │                    └── /etc/init.d/S??*  sırayla "start" ile
                 ├── ::askfirst  → /bin/sh                (konsolda Enter'a basınca kabuk)
                 └── ::shutdown  → /etc/init.d/rcK        (kapanırken, ters sırayla "stop")
```

`/etc/inittab`

`inittab` her satırda `<tty>:<runlevel>:<eylem>:<program>` biçimindedir. BusyBox `runlevel` alanını kullanmaz. `tty` boşsa konsol kullanılır.

```
::sysinit:/etc/init.d/rcS
::askfirst:-/bin/sh
::restart:/sbin/init
::shutdown:/etc/init.d/rcK
::shutdown:/bin/umount -a -r
```

Eylem	Anlamı
`sysinit`	Açılışta bir kez çalışır, bitene kadar beklenir
`askfirst`	"Please press Enter to activate this console." yazar, Enter'a basınca programı başlatır. Program kapanırsa yeniden başlatır. `-/bin/sh`'teki `-` kabuğu "login shell" yapar.
`respawn`	`askfirst` gibi ama sormadan başlatır
`restart`	`init` yeniden başlatılırken çalışır
`shutdown`	`reboot` / `poweroff` sırasında çalışır

> **İpucu**
>
> `inittab` hiç yoksa BusyBox varsayılan bir tane kullanır: `rcS`'i çalıştırır, konsolda ve **tty2–tty4**'te kabuk açmaya çalışır. BBB'de bu terminaller olmadığı için `can't open /dev/tty2` hataları görürsün. Kendi `inittab`'ını yazmak bunu önler.

`/etc/init.d/rcS` — kursun betiği

Kursun reposundaki (`scripts/etc/init.d/rcS`) betik `/proc`'u bağlar ve `/etc/init.d` altındaki `S` ile başlayan iki haneli numaralı betikleri **sırayla** `start` parametresiyle çalıştırır. Sona `.sh` eklenmiş betikler ayrı süreç açmadan, aynı kabukta çalıştırılır:

```
#!/bin/sh

# Start all init scripts in /etc/init.d
# executing them in numerical order.
#
echo "Mounting proc"
mount -t proc /proc /proc

for i in /etc/init.d/S??* ;do

     # Ignore dangling symlinks (if any).
     [ ! -f "$i" ] && continue

     case "$i" in
    *.sh)
        # Source shell script for speed.
        (
        trap - INT QUIT TSTP
        set start
        . $i
        )
        ;;
    *)
        # No sh extension, so fork subprocess.
        $i start
        ;;
    esac
done
```

`echo "Mounting proc"` satırından sonraya şu iki satırı da eklemeni öneririm. `sysfs` birçok aracın ve Bölüm 1'deki LED komutlarının, `devpts` ise SSH'ın (12.10) çalışması için gerekli:

```
mount -t sysfs sysfs /sys
mkdir -p /dev/pts && mount -t devpts devpts /dev/pts
```

`rcK` aynı döngüyü **ters sırada** ve `stop` parametresiyle çalıştırır. Numaralar sırayı belirler: `S01` önce, `S50` sonra başlar; kapanırken tersi.

Kursun reposundaki betikler:

Betik	Görevi
`S01logging`	`syslogd` ve `klogd`'yi başlatır (loglar `/var/log/messages`'a)
`S02module`	Açılışta kernel modüllerini yükler (12.7)
`S40network`	`/etc/network/interfaces`'teki arayüzleri `ifup -a` ile açar

Betik	Görevi
`S50sshd`	OpenSSH sunucusunu başlatır. BusyBox rootfs'inde OpenSSH olmadığı için kendi başına çalışmaz (12.10'da yerine Dropbear kullanıyoruz)

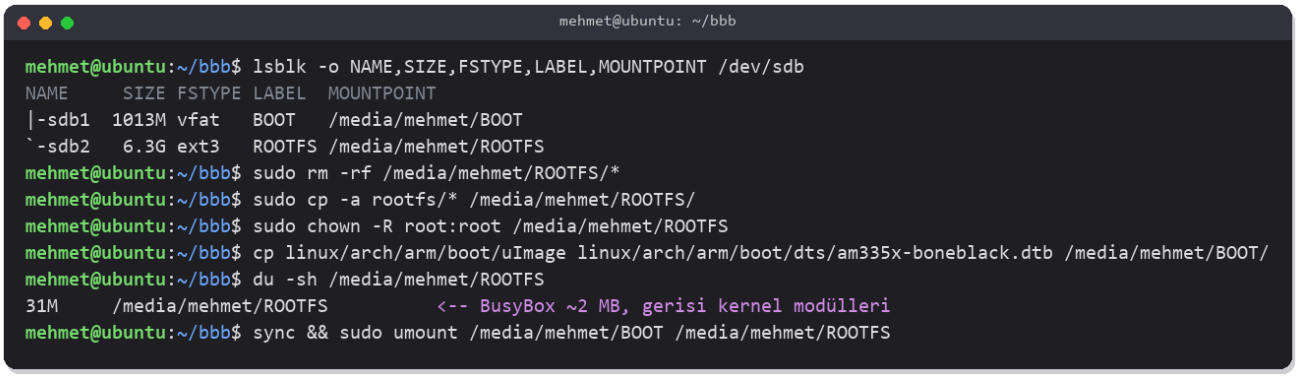
Betiklerin hepsini rootfs'e koy ve **çalıştırılabilir** yap:

```
cd ~/bbb/rootfs/etc/init.d
# rcS, rcK, S01logging, S02module, S40network dosyalarını buraya kopyala
chmod +x rcS rcK S*
```

12.6 Kartta Test: Rootfs SD Kartta

Kursun "Testing boot images and busybox on BBB" dersleri: kernel ve DTB SD kartın BOOT bölümünden, rootfs ROOTFS bölümünden gelir.

1) Rootfs'i SD karta kopyala.



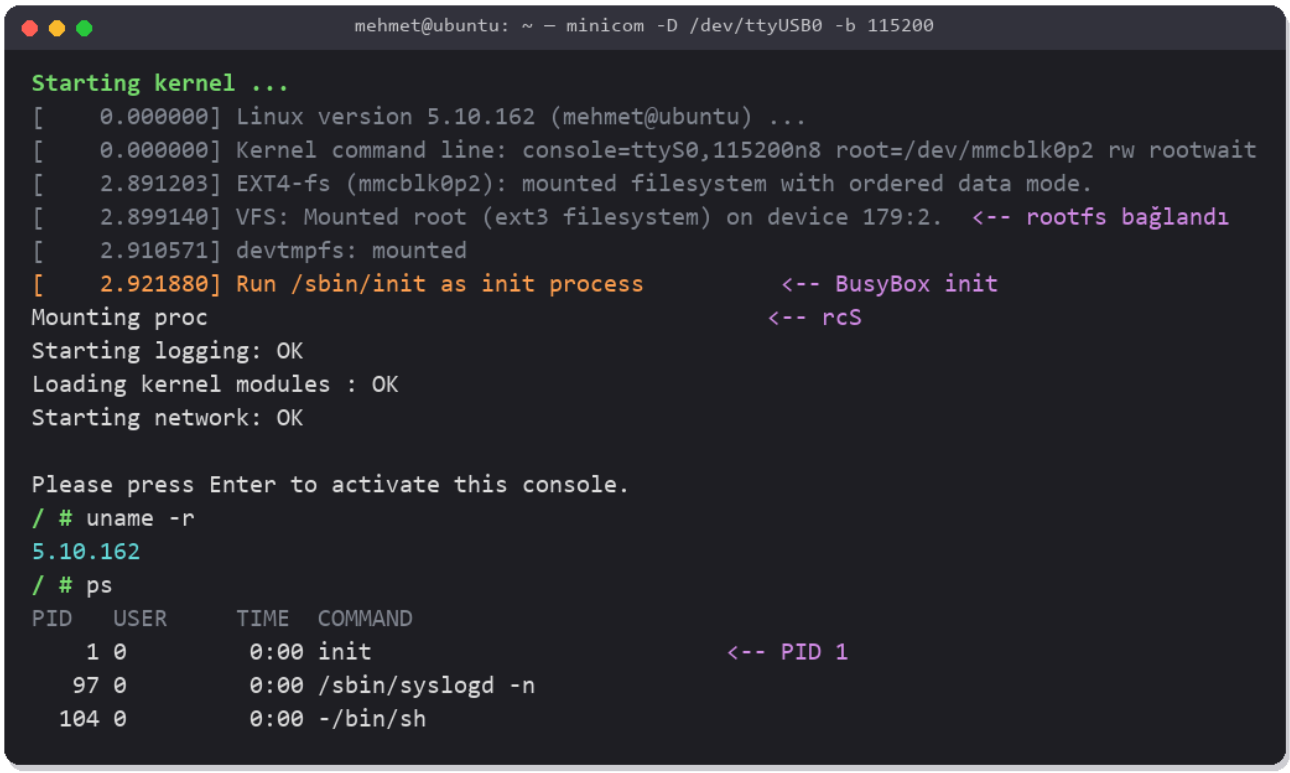

```
cd ~/bbb
sudo rm -rf /media/$USER/ROOTFS/*
sudo cp -a rootfs/* /media/$USER/ROOTFS/
sudo chown -R root:root /media/$USER/ROOTFS
cp linux/arch/arm/boot/uImage linux/arch/arm/boot/dts/am335x-boneblack.dtb /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT /media/$USER/ROOTFS
```

> **Dikkat**
>
> `sudo rm -rf /media/$USER/ROOTFS/*` komutundaki yolu iki kez kontrol et. Yol yanlış yazılırsa bilgisayarındaki dosyaları silebilir.

2) BOOT bölümündeki `uEnv.txt` (10.9'dakinin rootfs'i SD kart olan hali):

```
console=ttyS0,115200n8
netargs=setenv bootargs console=${console} root=/dev/mmcblk0p2 rw rootwait
netboot=echo Booting from microSD ...; setenv autoload no; load mmc 0:1 ${loadaddr} uImage; load
mmc 0:1 ${fdtaddr} am335x-boneblack.dtb; run netargs; bootm ${loadaddr} - ${fdtaddr}
uenvcmd=run netboot
```

3) Kartı S2'ye basılı tutarak aç. `rcS`'in mesajlarından sonra Enter'a bas:



`ps` çıktısında PID 1'in `init` olduğunu görürsün. Artık kartta çalışan her şeyi sen derledin: SPL, U-Boot, kernel, DTB ve rootfs.

12.7 USB Üzerinden Ethernet ve Sürücüleri Açılışta Yüklemek

Kursun "Enabling ethernet over usb by driver integration" ve "Auto loading drivers during system startup" dersleri.

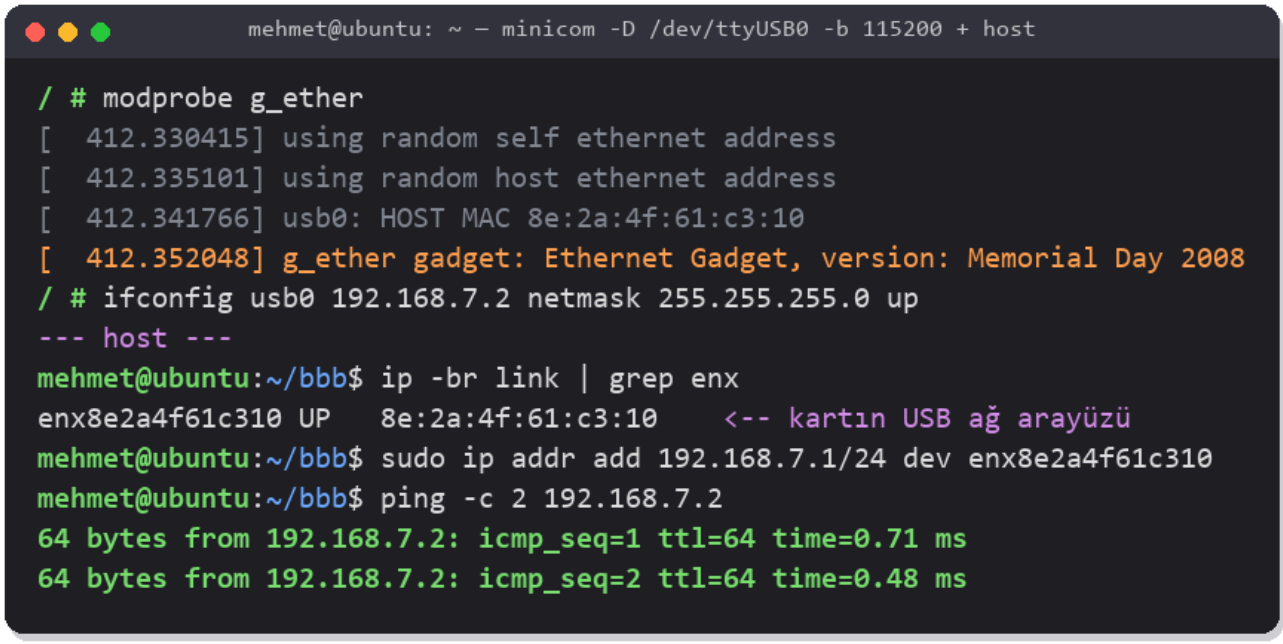
USB gadget: `g_ether`

BBB'nin mini-USB portu (USB0) **gadget** (cihaz) modunda çalışabilir. `g_ether` sürücüsü yüklenince kart, bilgisayara USB üzerinden bir **ağ kartı** gibi görünür. Ethernet kablosu olmadan SSH, TFTP, NFS yapılabilir. Kursun `192.168.7.x` adresleri bu ağdan gelir.

`bb.org_defconfig`'te bu sürücü **modül** olarak derlenir (`CONFIG_USB_ETH=m`, 10.8). Başka bir defconfig kullanıyorsan `menuconfig`'te açman gerekir:

```
Device Drivers  --->
    [*] USB support  --->
        <*>   USB Gadget Support  --->
            <M>   USB Gadget precomposed configurations
                <M>   Ethernet Gadget (with CDC Ethernet support)
```

Elle denemek için kartta:




```
/ # modprobe g_ether
/ # ifconfig usb0 192.168.7.2 netmask 255.255.255.0 up
```

Bilgisayarda yeni bir ağ arayüzü (`enx...` veya `usb0`) belirir. Ona `192.168.7.1` ver ve ping at:

```
ip -br link
sudo ip addr add 192.168.7.1/24 dev enx8e2a4f61c310     # arayüz adını kendi çıktından al
ping 192.168.7.2
```

Açılışta otomatik yüklemek

Her açılışta bu komutları yazmamak için iki betik kullanılır.

`S02module` (kursun reposundan) `g_ether` 'i yükler:

```
#!/bin/sh
#
# load the kernel modules at startup
#

case "$1" in
  start)
    printf "Loading kernel modules : "
    /sbin/modprobe g_ether
    [ $? = 0 ] && echo "OK" || echo "FAIL"
    ;;
  stop)
    printf "Unloading kernel modules : "
    /sbin/rmmod g_ether
    [ $? = 0 ] && echo "OK" || echo "FAIL"
    ;;
  restart|reload)
    "$0" stop
    "$0" start
    ;;
  *)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
esac

exit $?
```

`S40network` `ifup -a` ile `/etc/network/interfaces` dosyasındaki arayüzleri açar. Bu dosyayı oluştur:

```
# /etc/network/interfaces
auto lo
iface lo inet loopback

auto usb0
iface usb0 inet static
    address 192.168.7.2
    netmask 255.255.255.0
```

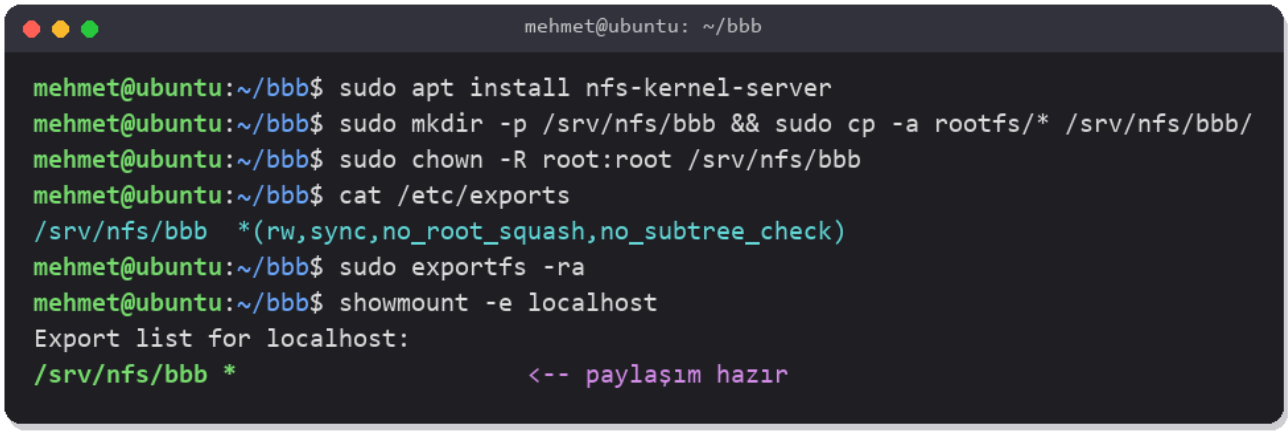
Numaralar sırayı garanti eder: önce `S02` sürücüyü yükler, sonra `S40` arayüzü açar. Kartı yeniden başlatınca ekranda `Loading kernel modules : OK` ve `Starting network: OK` görürsün.

12.8 Rootfs'i NFS ile Bilgisayardan Bağlamak

Rootfs'i her değişiklikte SD karta kopyalamak yorucu. **NFS** (Network File System) ile kart, rootfs'i doğrudan bilgisayardaki bir klasörden bağlar. Bilgisayarda bir dosyayı değiştirdiğin anda kart da görür. Bölüm 8'deki TFTP ile birlikte geliştirme döngüsü şöyle olur:

```
Host PC                                    BBB
 /var/lib/tftpboot/uImage, .dtb  ── TFTP ──►  U-Boot: RAM'e yükle, bootm
 /srv/nfs/bbb/  (rootfs)     ── NFS  ──►  Kernel: / olarak bağla
```

Adım 1 — Host'ta NFS sunucusu




```
sudo apt install nfs-kernel-server
sudo mkdir -p /srv/nfs/bbb
sudo cp -a ~/bbb/rootfs/* /srv/nfs/bbb/
sudo chown -R root:root /srv/nfs/bbb
echo "/srv/nfs/bbb  *(rw,sync,no_root_squash,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra
showmount -e localhost
```

`/etc/exports` seçeneği	Anlamı
`*`	Her IP'den erişime izin ver (yerine `192.168.27.2` yazarak sadece karta açabilirsin)
`rw`	Okuma-yazma
`sync`	Yazmalar diske yazıldıktan sonra onay ver
`no_root_squash`	Karttaki `root` kullanıcısı paylaşımda da `root` olsun. Rootfs için şart.
`no_subtree_check`	Alt klasör kontrollerini kapat (daha hızlı ve sorunsuz)

Adım 2 — `uEnv.txt`

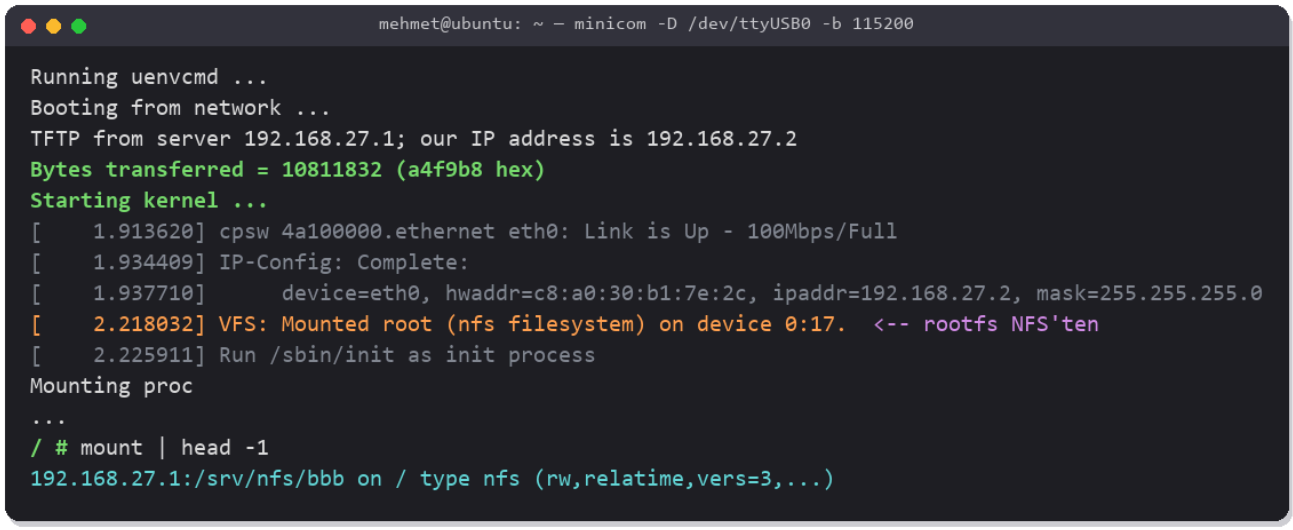
Kursun `uEnv-nfs.txt` dosyası temel alındı. Kursun dosyası USB ağı (`192.168.7.x`) kullanıyor; burada Bölüm 8'deki Ethernet kablosu ve adresleri kullanılıyor:

```
console=ttyS0,115200n8
ipaddr=192.168.27.2
serverip=192.168.27.1
rootpath=/srv/nfs/bbb
loadtftp=echo Booting from network ...; tftpboot ${loadaddr} uImage; tftpboot ${fdtaddr} am335x-boneblack.dtb
netargs=setenv bootargs console=${console} root=/dev/nfs rw
nfsroot=${serverip}:${rootpath},nolock,v3,tcp rootwait
ip=${ipaddr}:${serverip}::255.255.255.0::eth0:off init=/sbin/init
uenvcmd=setenv autoload no; run loadtftp; run netargs; bootm ${loadaddr} - ${fdtaddr}
```

`bootargs` parçası	Anlamı
`root=/dev/nfs`	Rootfs bir disk değil, NFS
`nfsroot=<sunucu>:<klasör>,...`	NFS sunucusu ve paylaşılan klasör
`nolock`	NFS dosya kilitlemesini kapat (kartta kilit servisi yok)
`v3,tcp`	NFS sürüm 3, TCP üzerinden. **Önemli:** kernel varsayılan olarak NFSv2 dener; Ubuntu 22.04 ve sonrasında NFSv2 kapalıdır.
`ip=<kart>:<sunucu>::<maske>::eth0:off`	Kernel'in IP'sini kendisi ayarlaması. U-Boot'un IP'si kernel'e geçmez (8.7).
`init=/sbin/init`	Çalıştırılacak ilk program (varsayılan da budur)

`uImage` ve `am335x-boneblack.dtb`'yi `/var/lib/tftpboot`'a kopyalamayı unutma.

Adım 3 — Aç ve doğrula




```
/ # mount | head -1
192.168.27.1:/srv/nfs/bbb on / type nfs (rw,...,vers=3,...)
```

> **Uyarı**
>
> Rootfs NFS'ten geliyorken kartta **`eth0` 'ı yeniden yapılandırma** (`ifconfig eth0 ...` , `/etc/network/interfaces` 'e `eth0` eklemek). Ağ bir an bile kesilirse kernel rootfs'e ulaşamaz ve sistem donar. `eth0` 'ı kernel `ip=` ile ayarlıyor; `interfaces` dosyasında sadece `lo` ve `usb0` kalsın.

> **Not**
>
> Kursun `uEnv-nfs.txt` 'si NFS'i USB ağı (`usb0`) üzerinden yapar. Bunun için `g_ether` 'in modül değil **kernel'e gömülü** (`CONFIG_USB_ETH=y`) olması gerekir; çünkü rootfs bağlanmadan modül yüklenemez (10.7). Ethernet kablosunda bu sorun yok, CPSW sürücüsü zaten gömülü.

12.9 Geliştirme Döngüsü

Ne değişti?	Ne yaparsın?
Kendi uygulaman / bir betik	Dosyayı `/srv/nfs/bbb` içine kopyala. Kart hemen görür, yeniden başlatma yok.
Kernel veya DTB	`uImage` / `.dtb` 'yi `/var/lib/tftpboot` 'a kopyala, kartı resetle.
Kernel modülleri	`modules_install INSTALL_MOD_PATH=/srv/nfs/bbb` (sudo ile), kartta `modprobe` ya da reset
BusyBox ayarları	BusyBox'ı yeniden derle, `CONFIG_PREFIX=/srv/nfs/bbb` ile kur

12.10 Challenge 1: SSH ve TFTP Desteği

Kursun bu bölümdeki ilk challenge'ı:

> *BusyBox ile minimal bir dosya sistemi kurup kartta test ettin. Şimdi onu birkaç araçla zenginleştir: kart ile bilgisayar arasında **TFTP** ve **SSH** ile dosya aktarımı yapılabilsin. BusyBox'ta SSH desteği yoktur; üçüncü parti bir kaynaktan eklenmesi gerekir. İpucu: https://busybox.net/tinyutils.html*

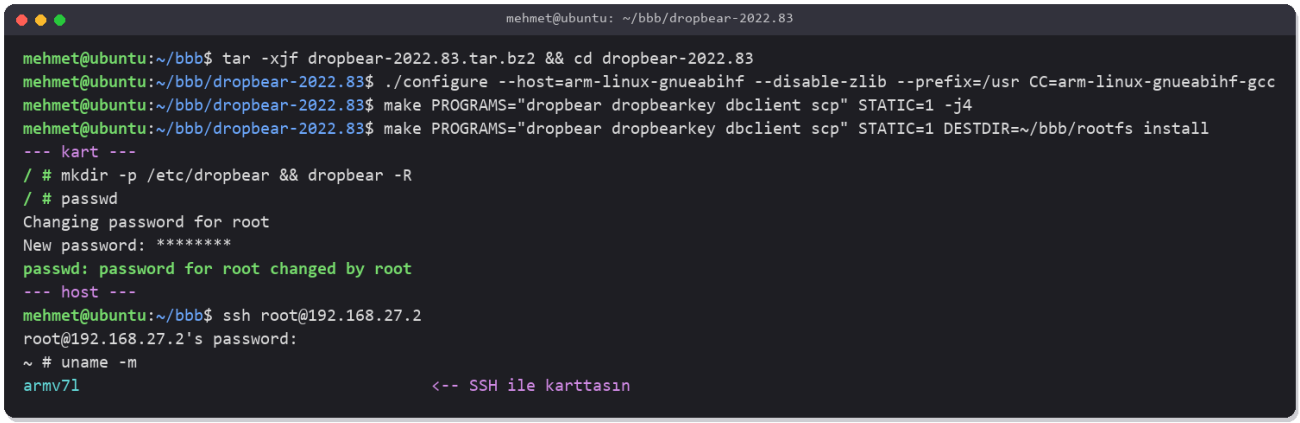
TFTP

BusyBox'ın `defconfig`'inde `tftp` applet'i zaten açık (`Networking Utilities → tftp`). Kullanımı 8.7'deki gibi:

```
/ # tftp -g -r dosya 192.168.27.1        # host'tan al
/ # tftp -p -l dosya 192.168.27.1        # host'a gönder
```

SSH: Dropbear

İpucundaki sayfa küçük sistemler için araçları listeler. SSH için önerilen **Dropbear**: SSH sunucusu, istemcisi ve `scp`'yi birkaç yüz KB'ta sunan bir proje. Kursun derleme notlarındaki adımlar:




```
cd ~/bbb
wget https://matt.ucc.asn.au/dropbear/releases/dropbear-2022.83.tar.bz2
tar -xjf dropbear-2022.83.tar.bz2 && cd dropbear-2022.83
./configure --host=arm-linux-gnueabihf --disable-zlib --prefix=/usr CC=arm-linux-gnueabihf-gcc
make PROGRAMS="dropbear dropbearkey dbclient scp" STATIC=1 -j4
make PROGRAMS="dropbear dropbearkey dbclient scp" STATIC=1 DESTDIR=~/bbb/rootfs install
```

Parça	Anlamı
`--host=arm-linux-gnueabihf`	ARM için cross compile

Parça	Anlamı
`--disable-zlib`	zlib sıkıştırma kütüphanesi olmadan derle (rootfs'te zlib yok)
`--prefix=/usr` + `DESTDIR=...`	Kartta `/usr/sbin/dropbear` olacak şekilde rootfs klasörüne kur
`STATIC=1`	BusyBox gibi statik derle, kütüphane gerekmesin

Kartta SSH'a giriş yapabilmek için bir **kullanıcı ve şifre** gerekir. Rootfs'e şu iki dosyayı ekle:

```
cd ~/bbb/rootfs
echo 'root:x:0:0:root:/root:/bin/sh' > etc/passwd
echo 'root::0:0:99999:7:::' > etc/shadow
chmod 600 etc/shadow
```

Kartta şifreyi belirle ve sunucuyu başlat:

```
/ # passwd
/ # mkdir -p /etc/dropbear
/ # dropbear -R
```

`-R`, sunucu anahtarlarını ilk bağlantıda otomatik üretir. Bilgisayardan bağlan ve dosya gönder:

```
ssh root@192.168.27.2
scp dosya root@192.168.27.2:/root/
```

Açılışta otomatik başlaması için `S50sshd` yerine şu küçük betiği koy:

```
#!/bin/sh
# /etc/init.d/S50dropbear
case "$1" in
  start) printf "Starting dropbear: "; mkdir -p /etc/dropbear; /usr/sbin/dropbear -R && echo "OK"
;;
  stop)  printf "Stopping dropbear: "; killall dropbear; echo "OK" ;;
  *)     echo "Usage: $0 {start|stop}"; exit 1 ;;
esac
```

> **Önemli**
>
> Dropbear, kabuk açmak için `/dev/pts` 'e ihtiyaç duyar. 12.5'te `rcS` 'e eklediğimiz `mount -t devpts devpts /dev/pts` satırı yoksa bağlantı `PTY allocation request failed` ile kapanır.

12.11 Challenge 2: BusyBox'ı Dinamik Derlemek

Kursun ikinci challenge'ı:

> *BusyBox derslerinin hepsinde "statik" derlenmiş programlar kullandık. Dinamik bağlanarak (dynamic linking) cross compile edilmiş bir uygulamayı bu dosya sisteminde çalıştırırsan çalışmaz. BusyBox'ı **dinamik** bağlanmış programlar üretecek şekilde yeniden derle ve dinamik bağlanmış bir uygulamanın çalıştığını test et.*

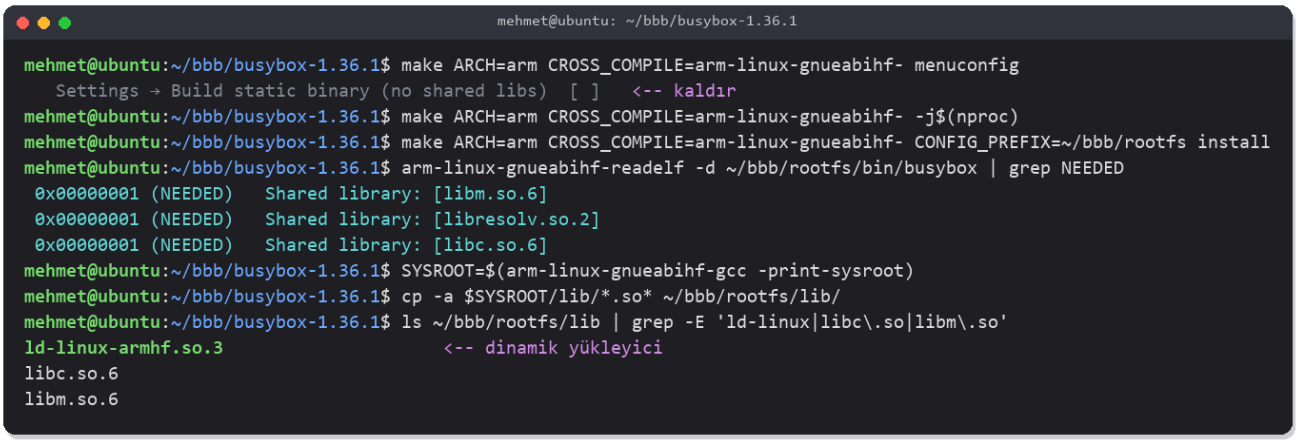
Neden dinamik program çalışmıyor?

Dinamik bağlanmış bir program, çalışırken iki şeye ihtiyaç duyar:

1. **Dinamik yükleyici**: ARM hard-float'ta `/lib/ld-linux-armhf.so.3`. Kernel programı başlatırken önce bunu çalıştırır.
2. Programın kullandığı **paylaşılan kütüphaneler**: `libc.so.6`, `libm.so.6` ...

Statik BusyBox rootfs'inde `/lib` altında sadece kernel modülleri vardır. Yükleyici yoksa kernel programı hiç başlatamaz ve kabuk yanıltıcı bir şekilde `not found` der.

Çözüm



1) Statik ayarı kapat, yeniden derle ve kur:

```
cd ~/bbb/busybox-1.36.1
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig      # Settings → Build static
binary: [ ]
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=~/bbb/rootfs install
```

2) Hangi kütüphanelerin gerektiğine bak:

```
arm-linux-gnueabihf-readelf -d ~/bbb/rootfs/bin/busybox | grep NEEDED
arm-linux-gnueabihf-readelf -l ~/bbb/rootfs/bin/busybox | grep interpreter
```

3) Kütüphaneleri toolchain'den rootfs'e kopyala. Toolchain, ARM için derlenmiş glibc'yi kendi **sysroot** klasöründe taşır:

```
SYSROOT=$(arm-linux-gnueabihf-gcc -print-sysroot)
cp -a $SYSROOT/lib/*.so* ~/bbb/rootfs/lib/
```

> **İpucu**
>
> Bu komut toolchain'deki bütün kütüphaneleri kopyalar (birkaç on MB). Sadece gerekenleri istiyorsan `readelf` çıktısındaki dosyaları ve `ld-linux-armhf.so.3`'ü kopyala. Kopyaladıktan sonra `arm-linux-gnueabihf-strip` ile küçültebilirsin.

4) Dinamik bir uygulamayla test et:

```
arm-linux-gnueabihf-gcc hello.c -o hello          # varsayılan: dinamik
file hello                                            # "dynamically linked, interpreter /lib/ld-
linux-armhf.so.3"
cp hello ~/bbb/rootfs/root/                         # NFS kullanıyorsan /srv/nfs/bbb/root/
```

Kartta `/root/hello` çalışıyorsa challenge tamam.

12.12 Özet

- BusyBox, yüzlerce komutu tek bir programda toplar; komutlar ona giden sembolik linklerdir.
- Minimal rootfs = BusyBox + `dev proc sys etc tmp var` klasörleri + `/etc/inittab` + `/etc/init.d/rcS` + `/lib/modules`.
- Açılış: kernel → `/sbin/init` (PID 1) → `inittab` → `rcS` → `S??*` betikleri sırayla → konsolda kabuk.
- `g_ether` kartı USB üzerinden bir ağ kartı yapar; `S02module` + `S40network` ile açılışta otomatik yüklenir.
- NFS + TFTP ile rootfs ve kernel bilgisayardan gelir; dosyayı kopyalamak, kartta denemek için yeterli olur.
- SSH için Dropbear, dinamik programlar için `/lib`'e toolchain sysroot'undaki kütüphaneler gerekir.

12.13 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 (temsili hatalar)

Kernel panic - not syncing: No working init found.   <-- init yok / ARM değil / kütüphane eksik

can't open /dev/tty2: No such file or directory      <-- /etc/inittab yok

/etc/init.d/rcS: Permission denied                   <-- chmod +x

modprobe: can't change directory to '/lib/modules': No such file  <-- modules_install

VFS: Unable to mount root fs via NFS.                <-- NFS sürümü / exports / ip=

PTY allocation request failed on channel 0           <-- /dev/pts bağlı değil
```

Belirti	Sebep	Çözüm
`Kernel panic - not syncing: No working init found`	`/sbin/init` yok, ARM için derlenmemiş ya da dinamik derlenmiş ama `/lib`'te kütüphane yok	`file bin/busybox` → ARM ve statik olmalı (veya 12.11)
`can't open /dev/tty2: No such file or directory`	`/etc/inittab` yok, BusyBox varsayılan olarak tty2–tty4'ü açmaya çalışıyor	12.5'teki `inittab`'ı ekle
`/etc/init.d/rcS: Permission denied`	Betik çalıştırılabilir değil	`chmod +x etc/init.d/rcS etc/init.d/S*`
`modprobe: can't change directory to '/lib/modules'`	Modüller kurulmamış ya da sürüm klasörü `uname -r` ile uyuşmuyor	`modules_install INSTALL_MOD_PATH=...` ile yeniden kur
`VFS: Unable to mount root fs via NFS`	NFS sürümü (v2), `exports` hatası, yanlış `ip=` / `nfsroot=`	`nfsroot=...,v3,tcp` ; host'ta `showmount -e` ; firewall'da NFS'e izin
`ifup: can't open '/etc/network/interfaces'`	Dosya yok	12.7'deki dosyayı oluştur
`run-parts: /etc/network/if-pre-up.d: No such file or directory`	BusyBox `ifup`'ın aradığı klasörler yok	`mkdir -p etc/network/if-{pre-up,up,down,post-down}.d`
SSH: `PTY allocation request failed`	`/dev/pts` bağlı değil	`rcS`'e `mount -t devpts devpts /dev/pts` ekle
SSH: `Permission denied`	root'un şifresi yok	Kartta `passwd`
Derleme: `networking/tc.c: error: 'TCA_CBQ_MAX' undeclared`	Toolchain'in kernel başlıkları çok yeni (6.8+), BusyBox 1.36'nın `tc` applet'i derlenmiyor	`menuconfig` → Networking Utilities → `tc`'yi kapat
Kartta dosyalar `root` yerine `nobody` sahipliğinde görünüyor	`no_root_squash` eksik	`/etc/exports`'a ekle, `sudo exportfs -ra`

Bölüm 13 — Buildroot

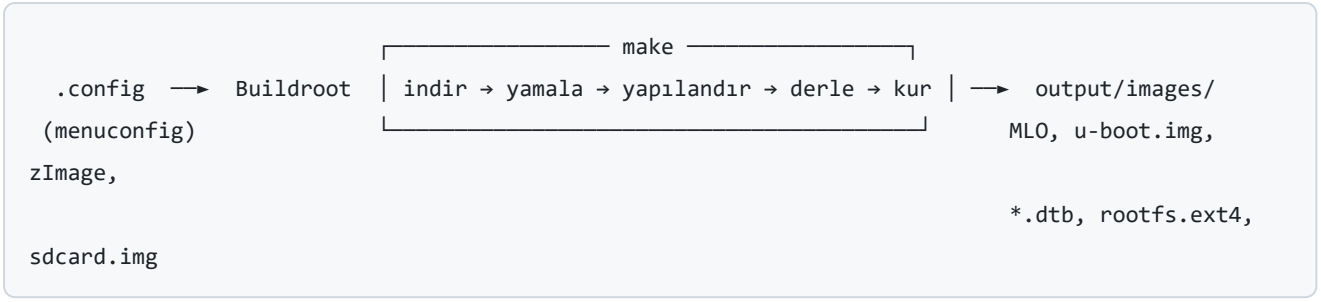
> **Bu bölümde:**
>
> - Buildroot'un ne olduğunu ve önceki bölümlerde elle yaptığımız işleri nasıl tek komuta indirdiğini öğreneceksin.
> - Buildroot'un klasör yapısını ve temel komutlarını tanıyacaksın.
> - BeagleBone Black için toolchain, U-Boot, kernel ve rootfs'i tek seferde derleyip hazır bir SD kart imajı üreteceksin.
> - Rootfs'e SSH (Dropbear), root şifresi ve ağ ayarı ekleyeceksin.
> - Buildroot içinden kernel ve U-Boot ayarlarını değiştirip yeniden derleyeceksin.
>
> **Ön koşul:** Bölüm 9 — U-Boot derleme, Bölüm 10 — kernel derleme, Bölüm 12 — BusyBox rootfs.

13.1 Buildroot Nedir?

Şimdiye kadar bir Embedded Linux sisteminin dört parçasını **tek tek, elle** hazırladık:

Parça	Elle (önceki bölümler)	Buildroot ile
Cross toolchain	Linaro arşivini indir, `PATH`'e ekle (Bölüm 9)	Hazır toolchain'i kendisi indirir ya da sıfırdan derler
U-Boot + SPL	Kaynağı indir, `defconfig`, `make` (Bölüm 9)	`make`
Kernel + DTB + modüller	Kaynağı indir, `defconfig`, `uImage dtbs modules` (Bölüm 10)	`make`
Rootfs	BusyBox derle, klasörleri ve betikleri elle yaz (Bölüm 12)	`make`
Ek programlar (Dropbear, kütüphaneler...)	Her birini ayrı cross compile et	Menüden seç, `make`
SD kart	GParted ile bölümle, dosyaları kopyala (Bölüm 4)	Hazır `sdcard.img`'yi karta yaz

Buildroot, bütün bu adımları yapan bir **derleme sistemidir**. Makefile'lar ve Kconfig'den oluşur: ne istediğini `menuconfig` ile seçersin, `make` her şeyi indirip doğru sırayla derler ve kartına yazılacak imajları üretir.



> **Not**
>
> Benzer bir araç olan **Yocto Project** çok daha esnek ama öğrenmesi de çok daha zordur. Buildroot küçük ve orta ölçekli projeler, öğrenme ve hızlı prototip için idealdir. İkisi de aynı fikre dayanır: bütün sistemi tarif eden bir yapılandırma ve onu derleyen bir sistem.

13.2 Sürüm ve Kurulum

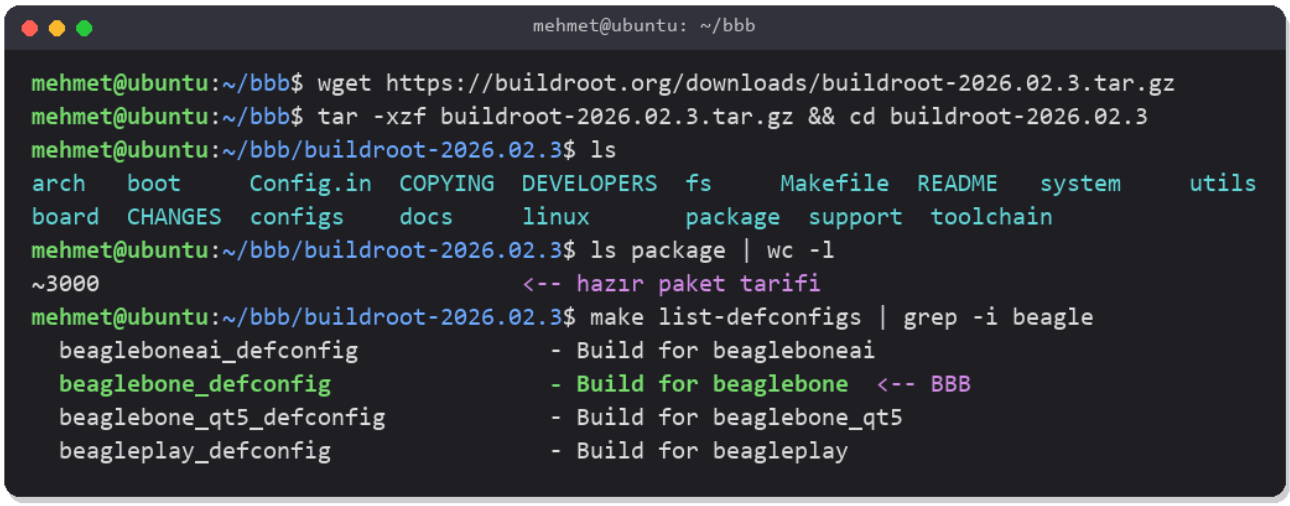
Buildroot her üç ayda bir sürüm çıkarır (`YYYY.02` , `.05` , `.08` , `.11`). Şubat sürümleri bir yıl desteklenen **LTS** sürümlerdir. Bu kitapta güncel LTS olan **2026.02.3** kullanılıyor.

> **Not**
>
> Kursun videoları Buildroot'un 2017 sürümünü kullanıyor. Menüler ve komutlar büyük ölçüde aynı; değişenler: kernel artık `uImage` yerine `zImage` olarak üretiliyor, varsayılan toolchain artık hazır **Bootlin** toolchain'i ve kart dosyaları `board/beaglebone/` yerine `board/beagleboard/beaglebone/` altında.

Host paketleri (Buildroot kılavuzundaki zorunlu liste + `menuconfig`):

```
sudo apt install which sed make binutils build-essential diffutils gcc g++ bash patch gzip bzip2
perl \
     tar cpio unzip rsync file bc findutils wget libncurses-dev python3 git
```

İndir ve aç:




```
cd ~/bbb
wget https://buildroot.org/downloads/buildroot-2026.02.3.tar.gz
tar -xzf buildroot-2026.02.3.tar.gz
cd buildroot-2026.02.3
make list-defconfigs | grep -i beagle
```

13.3 Klasör Yapısı

Klasör	İçinde ne var?
`arch/`	Mimari seçenekleri (ARM, ARM64, RISC-V...)
`board/`	Karta özel dosyalar: SD kart düzeni (`genimage.cfg`), `uEnv.txt` , betikler, yamalar. BBB için `board/beagleboard/beaglebone/` .
`boot/`	Bootloader tarifleri (U-Boot, Barebox, GRUB...)
`configs/`	Hazır kart ayarları (`*_defconfig`). BBB için `beaglebone_defconfig` .
`docs/`	Buildroot kılavuzu
`fs/`	Rootfs imaj türleri (ext4, squashfs, cpio/initramfs, tar...)
`linux/`	Kernel'in nasıl derleneceğinin tarifi
`package/`	**~3000 paketin** tarifi. Her paket bir klasör: `Config.in` (menü), `<paket>.mk` (nasıl derlenir), `.hash` (indirmenin doğrulaması).
`support/`	Yardımcı betikler (ör. `genimage.sh`)
`system/`	Rootfs iskeleti ve "System configuration" menüsü (init, hostname, şifre...)
`toolchain/`	Toolchain seçenekleri
`utils/`	Geliştirici araçları

`make` çalıştıktan sonra iki klasör daha oluşur:

Klasör	İçinde ne var?
`dl/`	İndirilen kaynak arşivleri. Bir kere indirilir, sonraki derlemelerde tekrar kullanılır.
`output/`	Bütün derleme çıktıları (aşağıda)

```
mehmet@ubuntu: ~/bbb/buildroot-2026.02.3
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ ls output
build  host  images  staging  target
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ ls output/build | head -6
busybox-1.37.0  host-dosfstools-4.2  host-genimage-18  linux-6.18.1  ...
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ ls output/target
bin  dev  etc  lib  lib32  linuxrc  media  mnt  opt  proc  root  run  sbin  sys  tmp  usr  var
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ file output/target/bin/busybox
output/target/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ ls output/host/bin | grep -E 'gcc$|mkimage|genimage' | head -4
arm-buildroot-linux-gnueabihf-gcc        <-- Buildroot'un cross toolchain'i
genimage
mkimage
```

`output/` altı	İçinde ne var?
`build/`	Her paketin açılıp derlendiği klasör (`busybox-1.37.0/` , `linux-6.18.1/` ...)
`host/`	**Host'ta** çalışan araçlar: cross toolchain (`arm-buildroot-linux-gnueabihf-gcc`), `mkimage` , `genimage` ...
`staging/`	Hedefin kütüphaneleri ve başlık dosyaları (sysroot). Uygulama derlerken bunlara karşı derlenir.
`target/`	**Rootfs'in kendisi**, sadeleştirilmiş haliyle. İmajlar bundan üretilir. Elle değiştirme; bir sonraki derlemede ezilir.
`images/`	**Karta gidecek dosyalar**

13.4 Temel Komutlar

Komut	Görevi
`make list-defconfigs`	Hazır kart ayarlarını listeler
`make <kart>_defconfig`	Hazır ayarı `.config` 'e yükler
`make menuconfig`	Buildroot'un ayar menüsü
`make`	Her şeyi derler. `-j` **verme**: Buildroot paketlerin içinde zaten paralel derler.
`make <paket>-rebuild`	Bir paketi yeniden derler (ör. `make busybox-rebuild`)
`make <paket>-dirclean`	Paketin derleme klasörünü siler; bir sonraki `make` 'te sıfırdan derlenir

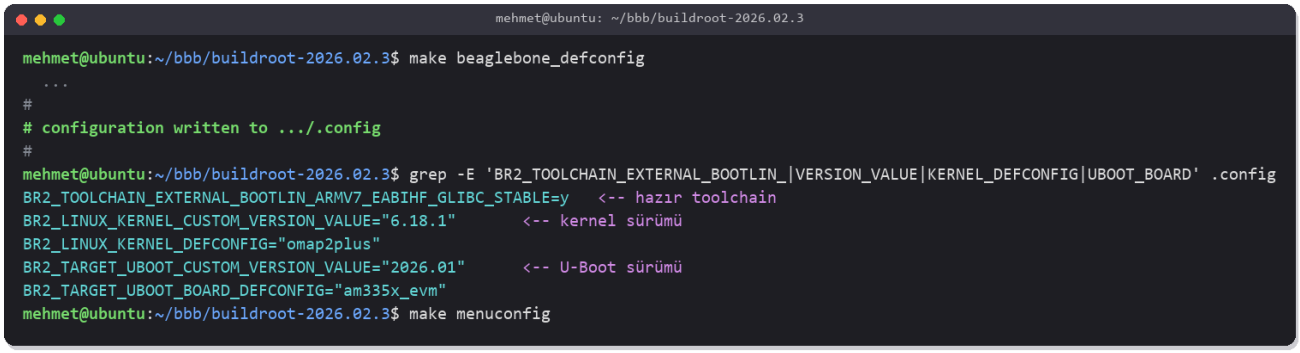
Komut	Görevi
`make linux-menuconfig` / `uboot-menuconfig` / `busybox-menuconfig`	Kernel'in, U-Boot'un, BusyBox'ın kendi menüsü (13.9)
`make savedefconfig`	Ayarlarını küçük bir defconfig olarak kaydeder
`make clean`	`output/`'u siler (indirmeler `dl/`'de kalır)
`make help`	Bütün hedefleri listeler

> **Önemli**
>
> Buildroot bir ayarı değiştirdiğinde neyin etkilendiğini **her zaman takip edemez**. Bir paketi eklemek için `make` yeterlidir. Ama **toolchain'i, mimariyi, init sistemini** veya bir paketi kaldırmak gibi büyük değişikliklerde `make clean all` ile baştan derle.

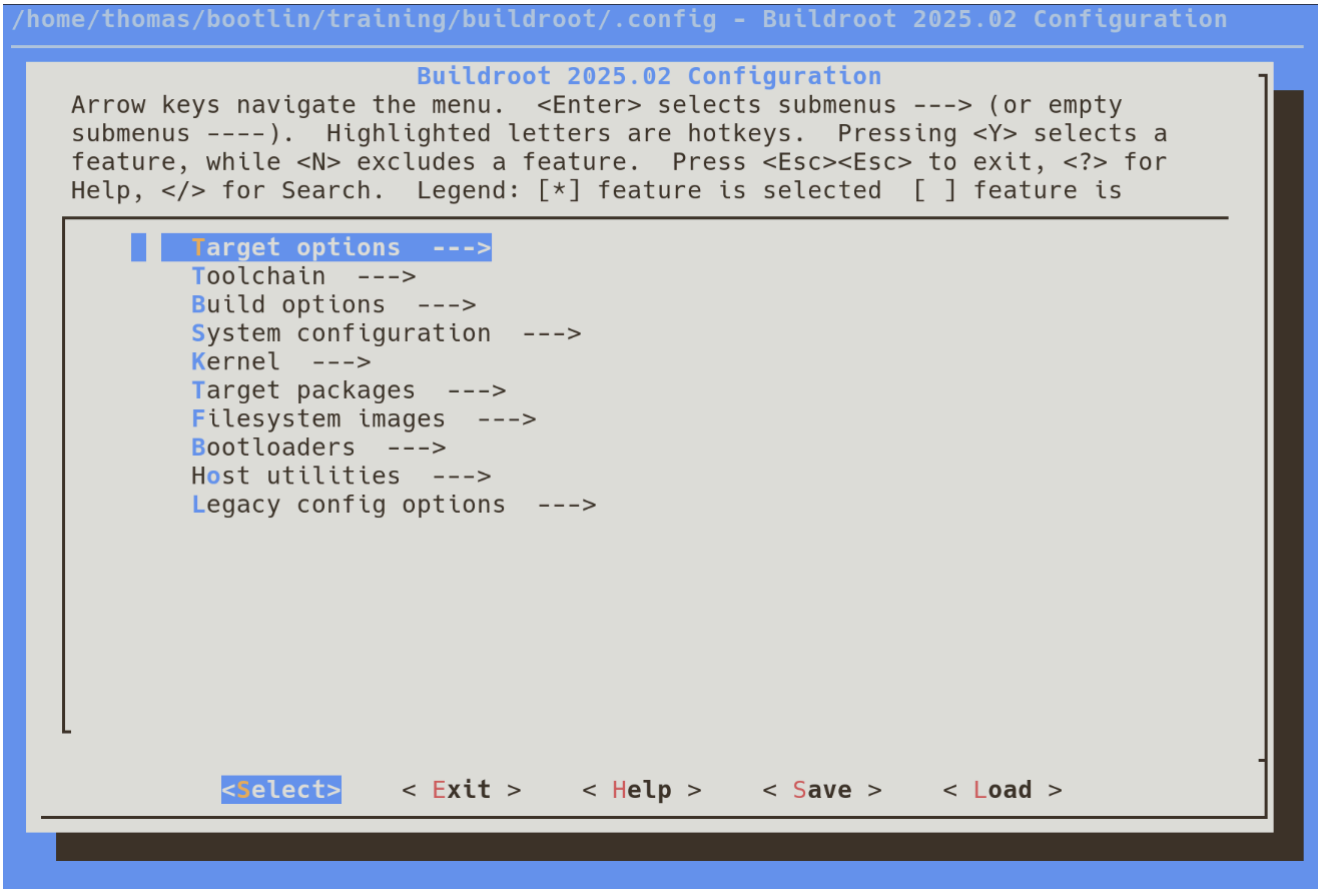
13.5 Adım Adım: BBB İçin Derleme

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**. İlk derleme indirmelerle birlikte bilgisayarına göre **30 dakika ile 2 saat** sürer ve birkaç GB yer kaplar.

Adım 1 — Hazır ayarı yükle

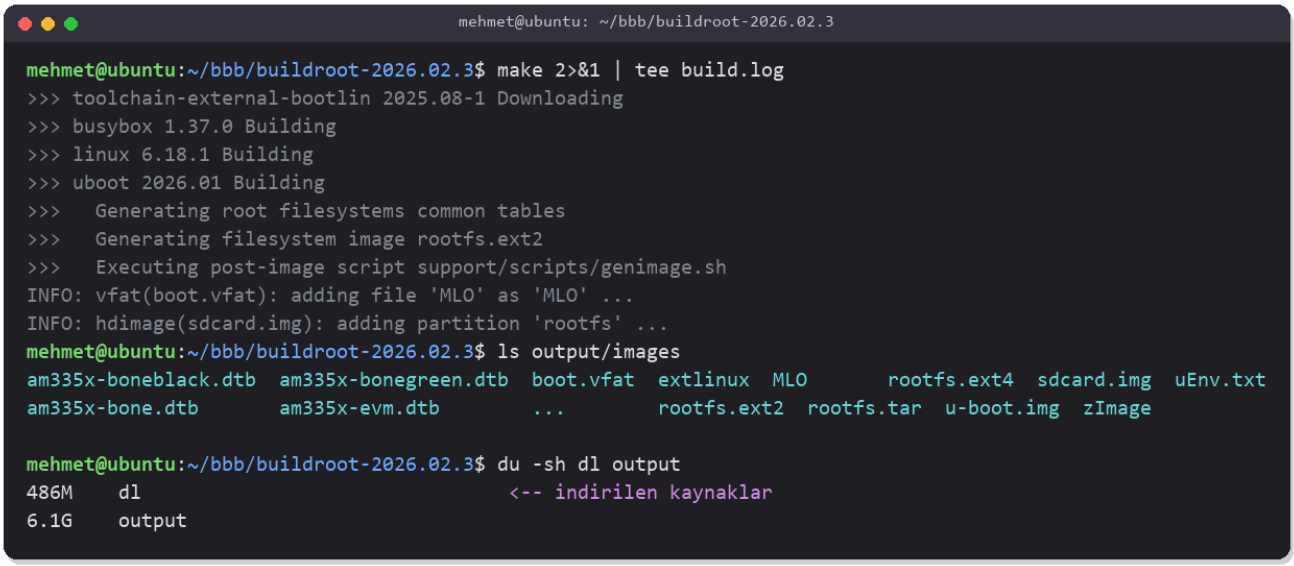



```
make beaglebone_defconfig
make menuconfig          # isteğe bağlı: incele
```



Görsel: Bootlin, "Buildroot" eğitim slaytları (CC BY-SA 3.0)

Adım 2 — Derle




```
make 2>&1 | tee build.log
ls output/images
```

`tee` ile çıktıyı `build.log`'a da yazmak, bir hata olduğunda log'a geri bakmanı sağlar.

`beaglebone_defconfig` ne seçiyor?

Ayar	Değer	Anlamı
`BR2_arm`, `BR2_cortex_a8`, `BR2_ARM_FPU_VFPV3`		Hedef: ARM Cortex-A8, donanımsal FPU
`BR2_TOOLCHAIN_EXTERNAL_BOOTLIN`	Bootlin armv7-eabihf glibc stable 2025.08-1	Bootlin'in hazır derlenmiş toolchain'i indirilir. Toolchain'i sıfırdan derlemekten çok daha hızlıdır.
`BR2_LINUX_KERNEL_CUSTOM_VERSION_VALUE`	6.18.1	Mainline kernel sürümü
`BR2_LINUX_KERNEL_DEFCONFIG`	`omap2plus`	Kernel'in `omap2plus_defconfig`'i (10.5)
`BR2_LINUX_KERNEL_INTREE_DTS_NAME`	`ti/omap/am335x-boneblack` ve diğer Beagle kartları	Derlenecek DTB'ler. Kernel 6.5'ten beri DTS'ler `ti/omap/` altında (11.9).
`BR2_TARGET_UBOOT_CUSTOM_VERSION_VALUE`	2026.01	U-Boot sürümü
`BR2_TARGET_UBOOT_BOARD_DEFCONFIG`	`am335x_evm`	U-Boot'un `am335x_evm_defconfig`'i (9.3)
`BR2_TARGET_UBOOT_SPL_NAME`	`MLO`	SPL'in SD kart için başlıklı hali
`BR2_ROOTFS_DEVICE_CREATION_DYNAMIC_MDEV`		`/dev` için BusyBox'ın `mdev`'i kullanılır
`BR2_TARGET_ROOTFS_EXT2_4`		Rootfs ext4 imajı olarak üretilir
`BR2_ROOTFS_POST_IMAGE_SCRIPT`	`genimage.sh` + `genimage.cfg`	Bütün dosyalardan hazır bir SD kart imajı (`sdcard.img`) üretilir

Üretilen dosyalar (`output/images/`)

Dosya	Ne?
`MLO`, `u-boot.img`	SPL ve U-Boot 2026.01
`zImage`	Kernel 6.18.1. **`uImage` değil**; U-Boot `bootz` ile başlatır.
`am335x-boneblack.dtb` (+ diğer Beagle'lar)	Device tree'ler
`uEnv.txt`, `extlinux/extlinux.conf`	U-Boot'a kernel'i nasıl açacağını söyleyen dosyalar
`rootfs.ext4` (= `rootfs.ext2`), `rootfs.tar`	Rootfs: imaj olarak ve arşiv olarak
`boot.vfat`	BOOT bölümünün (FAT, 16 MB) imajı
`sdcard.img`	Bölüm tablosu + `boot.vfat` + `rootfs.ext4`. **Karta doğrudan yazılacak dosya.**

13.6 Kartta Test

SD karta yazmak

`sdcard.img`, Bölüm 5'teki Debian imajı gibi **bütün bir diskin kopyasıdır**. Karta byte byte yazılır, bölümlemeye gerek yoktur:

```
lsblk                                          # SD kartın adını bul (ör. /dev/sdb)
sudo umount /dev/sdb1 /dev/sdb2
sudo dd if=output/images/sdcard.img of=/dev/sdb bs=4M status=progress conv=fsync
```

> **Dikkat**
>
> `dd`'nin `of=` kısmına yanlış diski yazarsan (ör. `/dev/sda`) **bilgisayarının diski silinir**. `lsblk` ile boyuta bakarak kontrol et. Emin değilsen 5.4.5'teki gibi balenaEtcher kullan.

U-Boot kernel'i nasıl açıyor?

Buildroot'un BOOT bölümüne koyduğu `uEnv.txt` (`board/beagleboard/beaglebone/uEnv.txt`):

```
bootpart=0:1
devtype=mmc
bootdir=
bootfile=zImage
bootpartition=mmcblk0p2
console=ttyS0,115200n8
loadaddr=0x82000000
fdtaddr=0x88000000
set_mmc1=if test $board_name = A33515BB; then setenv bootpartition mmcblk1p2; fi
set_bootargs=setenv bootargs console=${console} root=/dev/${bootpartition} rw rootfstype=ext4
rootwait
uenvcmd=run set_mmc1; run set_bootargs;run loadimage;run loadfdt;printenv bootargs;bootz
${loadaddr} - ${fdtaddr}
```

Parça	Anlamı
`bootpart` , `devtype` , `bootdir` , `bootfile`	U-Boot'un kendi `loadimage` makrosunun kullandığı değişkenler: " `mmc 0:1` 'deki `zImage` 'ı yükle"
`run loadimage` , `run loadfdt`	U-Boot'un varsayılan ortamındaki hazır makrolar. Bölüm 6'daki `load mmc 0:1 ...` komutlarını onlar çalıştırır.
`set_mmc1`	Kartın EEPROM'undaki ada (`board_name`) göre rootfs'i eMMC'ye çevirir
`bootz`	**`zImage`** başlatır. `bootm` ise U-Boot başlıklı **`uImage`** ister (4.9).

Buildroot ayrıca `extlinux/extlinux.conf` da koyar. Yeni U-Boot sürümleri BOOT bölümünde bu dosyayı da arar ("standard boot"). Hangisinin kullanılacağı U-Boot'un ortamına bağlıdır; ikisi de aynı kernel'i, aynı DTB'yi ve aynı `root=` ayarını kullanır.

Aç

Kartı S2'ye basılı tutarak aç. `Welcome to Buildroot` ve `buildroot login:` geldiğinde `root` yaz. Varsayılan ayarda şifre yoktur.

13.7 SSH, Şifre ve Ağ Ayarı

Kursun "Testing buildroot RFS and ssh support on BBB" dersi. Önce `menuconfig` 'te üç değişiklik yap:

Menü	Ayar	Neden?
Target packages → Networking applications	`[*] dropbear`	SSH sunucusu (12.10'daki Dropbear, bu sefer Buildroot derliyor ve açılışta başlatıyor)
System configuration → Root password	Bir şifre	Dropbear şifresiz root girişine izin vermez
System configuration → Root filesystem overlay directories	`board/bbb/overlay`	Rootfs'e kendi dosyalarını eklemek için (aşağıda)

Rootfs overlay, Buildroot'un rootfs'i ürettikten sonra üzerine **kopyaladığı** bir klasördür. `output/target/` 'ı elle değiştirmek yerine kendi dosyalarını buraya koyarsın. Statik IP için:

```
mkdir -p board/bbb/overlay/etc/network
cat > board/bbb/overlay/etc/network/interfaces <<'EOF'
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet static
    address 192.168.27.2
    netmask 255.255.255.0
EOF
```

```
mehmet@ubuntu: ~/bbb/buildroot-2026.02.3
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ make menuconfig
   Target packages → Networking applications → [*] dropbear
   System configuration → Root password: ********
   System configuration → Root filesystem overlay directories: board/bbb/overlay
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ cat board/bbb/overlay/etc/network/interfaces
auto lo
iface lo inet loopback
auto eth0
iface eth0 inet static
    address 192.168.27.2
    netmask 255.255.255.0
mehmet@ubuntu:~/bbb/buildroot-2026.02.3$ make
>>> dropbear 2026.91 Building
>>>   Generating filesystem image rootfs.ext2
>>>   Executing post-image script support/scripts/genimage.sh  <-- sdcard.img yenilendi
```

```
make
```

> **İpucu**
>
> Ağında bir DHCP sunucusu (modem) varsa overlay yerine System configuration → **Network interface to configure through DHCP** = `eth0` ayarı yeterli.

Yeni `sdcard.img`'yi karta yaz ve aç. Bilgisayardan bağlan:

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 + host
U-Boot SPL 2026.01 (Oct 21 2026 - 14:02:33 +0300)
U-Boot 2026.01 (Oct 21 2026 - 14:02:33 +0300)
...
Starting kernel ...
[    0.000000] Linux version 6.18.1 (mehmet@ubuntu) (arm-buildroot-linux-gnueabihf-gcc ...)
Starting syslogd: OK
Starting network: OK
Starting dropbear sshd: OK

Welcome to Buildroot
buildroot login: root
Password:
# uname -r
6.18.1
--- host ---
mehmet@ubuntu:~/bbb$ ssh root@192.168.27.2
#                                        <-- SSH ile Buildroot sistemindesin
```

```
ssh root@192.168.27.2
```

13.8 Buildroot Rootfs'ini Tek Başına Kullanmak

Bazen Buildroot'un sadece rootfs'ini istersin; kernel'i ve U-Boot'u kendin derlemeye devam edersin. `rootfs.tar` bunun içindir:

Nereye?	Komut
SD kartın ROOTFS bölümüne	`sudo tar -xf output/images/rootfs.tar -C /media/$USER/ROOTFS`
NFS klasörüne (12.8)	`sudo tar -xf output/images/rootfs.tar -C /srv/nfs/bbb`

> **Not**
>
> Buildroot rootfs'indeki kernel modülleri Buildroot'un derlediği kernel (6.18.1) içindir. Kendi kernel'ini kullanıyorsan modülleri 10.8'deki gibi `modules_install` ile aynı rootfs'e kur.

13.9 Buildroot'ta Kernel ve U-Boot Ayarları

Kursun "Buildroot Linux and U-Boot configurations" ve "image generation and testing" dersleri.

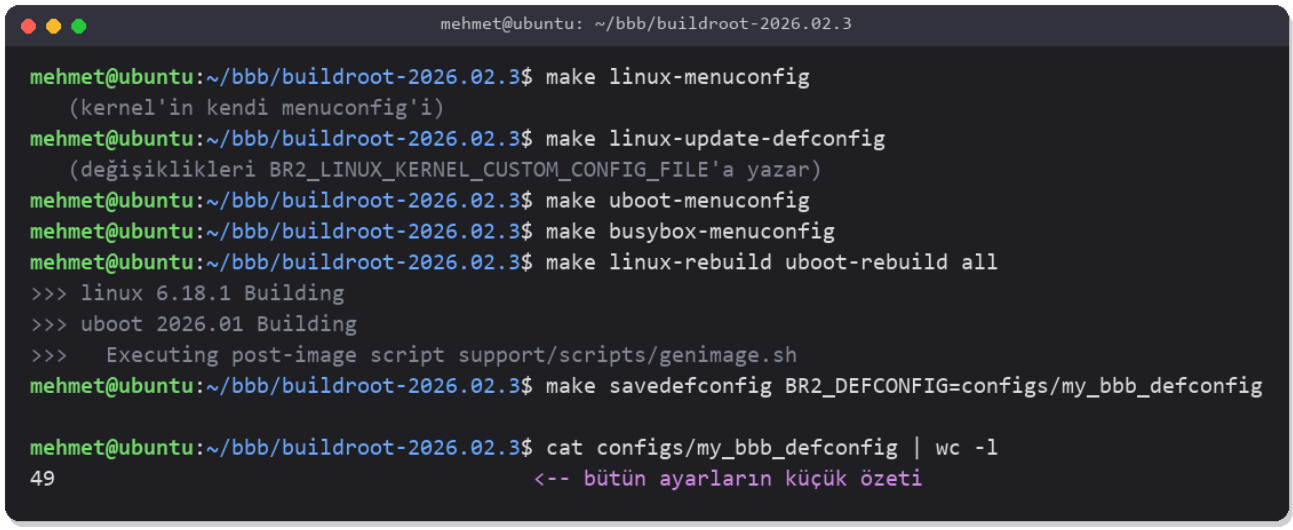
Hangi sürüm, hangi ayar?

Buildroot `menuconfig`'teki **Kernel** ve **Bootloaders** → **U-Boot** menüleri, Bölüm 9 ve Bölüm 10'da elle verdiğimiz bilgileri sorar:

Menü (Kernel)	Elle karşılığı
Kernel version	Hangi kaynak: resmi sürüm, özel sürüm numarası, özel git deposu veya arşiv
Kernel configuration → Defconfig name	`make xxx_defconfig`
Kernel binary format	`zImage` / `uImage` (uImage için *load address* = `0x80008000`)
Build a Device Tree Blob → In-tree Device Tree Source file names	`make ... am335x-boneblack.dtb`

Menü (Bootloaders → U-Boot)	Elle karşılığı
U-Boot Version	Hangi U-Boot kaynağı
Board defconfig	`make am335x_evm_defconfig`
U-Boot binary format → u-boot.img	`u-boot.img`
Install U-Boot SPL binary image + SPL name `MLO`	`MLO`

Kernel'in ve U-Boot'un kendi menüleri




```
make linux-menuconfig          # kernel'in menuconfig'i (output/build/linux-.../.config)
make uboot-menuconfig          # U-Boot'un menuconfig'i
make busybox-menuconfig        # BusyBox'ın menuconfig'i
make linux-rebuild uboot-rebuild all
```

> **Önemli**
>
> `linux-menuconfig` ile yaptığın değişiklikler `output/build/` altında durur ve `make clean` 'de **kaybolur**. Kalıcı yapmak için:
>
> 1. Kernel menüsünde *Kernel configuration* = **Using a custom (def)config file** seç ve bir dosya yolu ver (ör. `board/bbb/linux.config`).
> 2. `make linux-update-defconfig` ile değişiklikleri o dosyaya yaz.
>
> U-Boot ve BusyBox için de aynısı: `uboot-update-defconfig` , `busybox-update-config` .

Örnek: Kursun BeagleBoard kernel'ini kullanmak

Kursta Buildroot'un içinde BeagleBoard'un kernel'i ve `bb.org_defconfig` kullanılıyor. 2026.02.3'te bunu şöyle seçersin:

Menü	Değer
Kernel version	**Custom Git repository**
URL of custom repository	`https://github.com/beagleboard/linux.git`
Custom repository version	`5.10`
Defconfig name	`bb.org` (sonuna `_defconfig` yazılmaz)

Menü	Değer
In-tree Device Tree Source file names	`am335x-boneblack` (kernel 6.5'ten eski olduğu için `ti/omap/` öneki yok)
Build options → Force all downloads to have a valid hash	**Kapat** (`beaglebone_defconfig` bunu açıyor; git dalının önceden bilinen bir hash'i yok)

> **Uyarı**
>
> Eski bir kernel'i (5.10) Buildroot'un yeni toolchain'iyle derlemek, 9.2'deki "eski kaynak + yeni GCC" sorununa yol açabilir. Derleme hata verirse Buildroot'un varsayılan kernel'ine (6.18.1) dön; o, bu toolchain ile test edilmiştir.

Kendi defconfig'ini kaydet

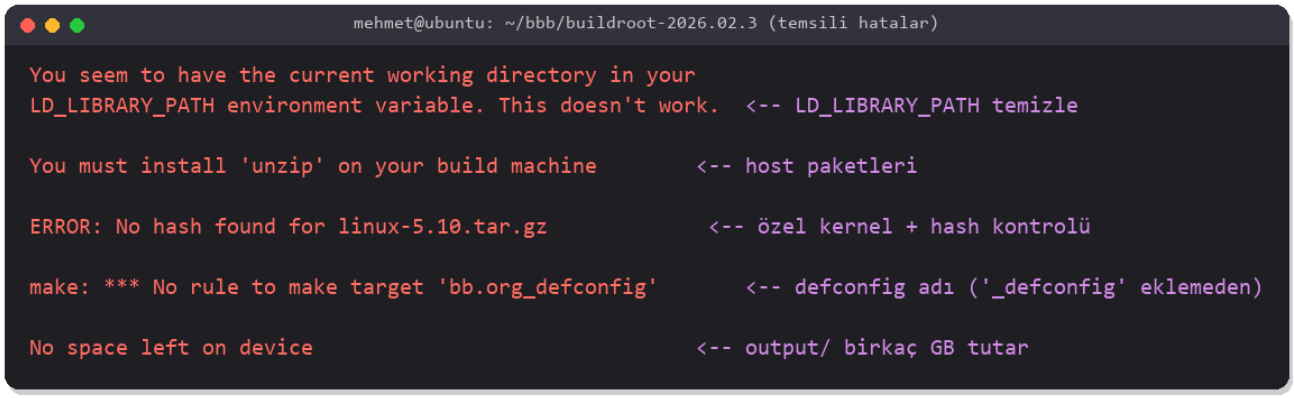
Bütün değişikliklerden sonra ayarlarını küçük bir dosyada sakla. Böylece aynı sistemi başka bir bilgisayarda ya da `make clean` sonrasında tek komutla yeniden üretebilirsin:

```
make savedefconfig BR2_DEFCONFIG=configs/my_bbb_defconfig
# sonra, sıfırdan:
make my_bbb_defconfig && make
```

13.10 Özet

- Buildroot; toolchain, U-Boot, kernel ve rootfs'i tek bir `.config` 'ten derleyip hazır `sdcard.img` üretir.
- Akış: `make beaglebone_defconfig` → `make menuconfig` → `make` → `output/images/sdcard.img` → `dd` ile SD karta.
- `output/` : `build` (derleme), `host` (toolchain ve araçlar), `staging` (sysroot), `target` (rootfs), `images` (karta gidenler).
- Paket eklemek: menüden seç, `make` . Kendi dosyaların: **rootfs overlay**. Büyük değişiklik: `make clean all` .
- Kernel/U-Boot/BusyBox ayarları: `make linux-menuconfig` vb. Kalıcı yapmak için `*-update-defconfig` , bütün sistem için `savedefconfig` .

13.11 Karşılaşılabilecek Problemler



Belirti	Sebep	Çözüm
`You seem to have the current working directory in your LD_LIBRARY_PATH`	Ortam değişkeni Buildroot'u bozuyor	`unset LD_LIBRARY_PATH` ve tekrar `make`
`You must install 'unzip' on your build machine` (veya başka bir araç)	Host paketi eksik	13.2'deki paketleri kur
`ERROR: No hash found for linux-...`	Özel kernel/U-Boot kaynağı + "Force all downloads to have a valid hash" açık	Build options'tan bu ayarı kapat ya da `.hash` dosyası ekle
`No rule to make target 'bb.org_defconfig_defconfig'`	Defconfig adına `_defconfig` eklenmiş	Menüye sadece `bb.org` yaz
Ayar değiştirdim ama sonuç değişmedi	Buildroot o paketi yeniden derlemedi	`make <paket>-rebuild` veya `make clean all`
`No space left on device`	`output/` ve `dl/` birkaç GB tutar	En az 15 GB boş yer bırak
Kart `Starting kernel ...`'dan sonra sessiz	`console=` yanlış	Kernel 6.18.1'de `console=ttyS0,115200n8`
SSH: `Permission denied`	Root şifresi boş	System configuration → Root password
`output/target/`'ta yaptığım değişiklik kayboldu	`target/` her derlemede yeniden üretilir	Dosyalarını rootfs overlay'e koy

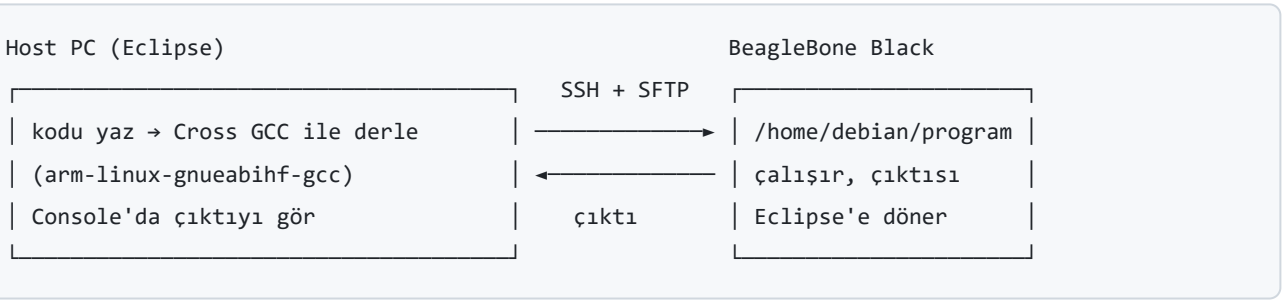
Kısım VI
Uygulama Geliştirme ve Donanım

Bölüm 14 — Eclipse ile Uygulama Geliştirme

> **Bu bölümde:**
>
> - Eclipse'i (2026-09) kurup Bölüm 9'daki cross toolchain'e bağlayacaksın.
> - Bir C projesi oluşturup ARM için derleyeceksin ve derleme ayarlarını öğreneceksin.
> - Programı Eclipse'ten tek tıkla karta gönderip çalıştıracaksın.
> - Kursun hazır Eclipse projelerini içe aktaracaksın.
> - Aynı işi Eclipse olmadan yapacaksın: `make` + `scp` / `ssh`, VS Code ve `gdbserver` ile uzaktan hata ayıklama.
>
> **Ön koşul:** Bölüm 9 — cross toolchain, kartta SSH ile bağlanılabilen bir Linux (Bölüm 5'teki Debian ya da Bölüm 13'teki Buildroot sistemi).

14.1 Neden Eclipse?

Şimdiye kadar her şeyi terminalden derledik. Kendi uygulamalarını yazmaya başladığında bir IDE işini kolaylaştırır: kod tamamlama, hataların satırında gösterilmesi, tek tuşla derleme ve programı karta gönderip çalıştırma.



Eclipse kendisi derlemez. Arka planda Bölüm 9'da kurduğun `arm-linux-gnueabihf-gcc`'yi çağırır. Bu yüzden komut satırında öğrendiğin her şey burada da geçerlidir. Eclipse kullanmak istemiyorsan 14.8'e geç.

> **Not**
>
> Kurs bu bölümü **Windows** üzerinde anlatıyor; kitapta Ubuntu kullanılıyor. Adımlar iki sistemde de aynıdır, farkları 14.3'te belirtildi.

14.2 Kursa Göre Değişenler

Kursta	Bugün (2026-09)
Önce **Java JDK** kurulur, Eclipse onsuz açılmaz	Eclipse paketleri **kendi Java'sıyla (JRE)** gelir. Ayrıca Java kurmana gerek yok.
Uygulama karta **Remote System Explorer (RSE)** ile gönderilir	RSE artık Eclipse'in C/C++ araçlarıyla gelmiyor. Aynı işi CDT'nin **C/C++ Remote Launch** özelliği SSH üzerinden yapar (14.6).

14.3 Eclipse'i Kurmak

1) İndir. https://www.eclipse.org/downloads/packages/ adresinden **Eclipse IDE for C/C++ Developers** paketini, işletim sistemine uygun olanı indir. ("Eclipse IDE for Embedded C/C++ Developers" paketi de olur; o, mikrodenetleyiciler için ek araçlar içerir.)

2) Aç ve çalıştır.

```
cd ~
tar -xzf ~/Downloads/eclipse-cpp-2026-09-R-linux-gtk-x86_64.tar.gz
~/eclipse/eclipse &
```

İlk açılışta bir **workspace** (projelerin duracağı klasör) sorar. Varsayılan `~/eclipse-workspace` uygundur.

3) Cross toolchain'in hazır olduğunu kontrol et. Eclipse'in toolchain'i bulabilmesi için tam yolunu bilmen gerekir:

```
which arm-linux-gnueabihf-gcc
```

Host	Toolchain
Ubuntu	9.3'te kurduğun Linaro toolchain: `~/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin`
Windows	Linaro'nun Windows sürümü: `gcc-linaro-7.5.0-2019.12-i686-mingw32_arm-linux-gnueabihf.tar.xz` (indir). Bir klasöre aç; içindeki `bin` klasörünün yolu Eclipse'e verilecek. Windows'ta `make` de gerekir (ör. MSYS2 veya "Windows Build Tools").

4) Gerekli eklentileri kontrol et. *Help → About Eclipse IDE → Installation Details* listesinde şu iki özellik olmalı. Yoksa *Help → Install New Software*'da *Work with* kutusuna `https://download.eclipse.org/releases/2026-09/` yaz ve aratıp kur:

Özellik	Ne işe yarar?
C/C++ GCC Cross Compiler Support	Proje sihirbazındaki "Cross GCC" araç zinciri
C/C++ Remote Launch	Programı SSH ile karta gönderip çalıştırma

---

14.4 Birinci Proje: Oluşturma ve Derleme

Kursun "Eclipse project creation and build settings" dersi.

Adım 1 — Projeyi oluştur

File → New → C/C++ Project → C Managed Build, sonra:

Alan	Değer
Project name	`hello_bbb`
Project type	Executable → **Hello World ANSI C Project**
Toolchains	**Cross GCC**

Next ile ilerle. **Cross GCC Command** sayfasında:

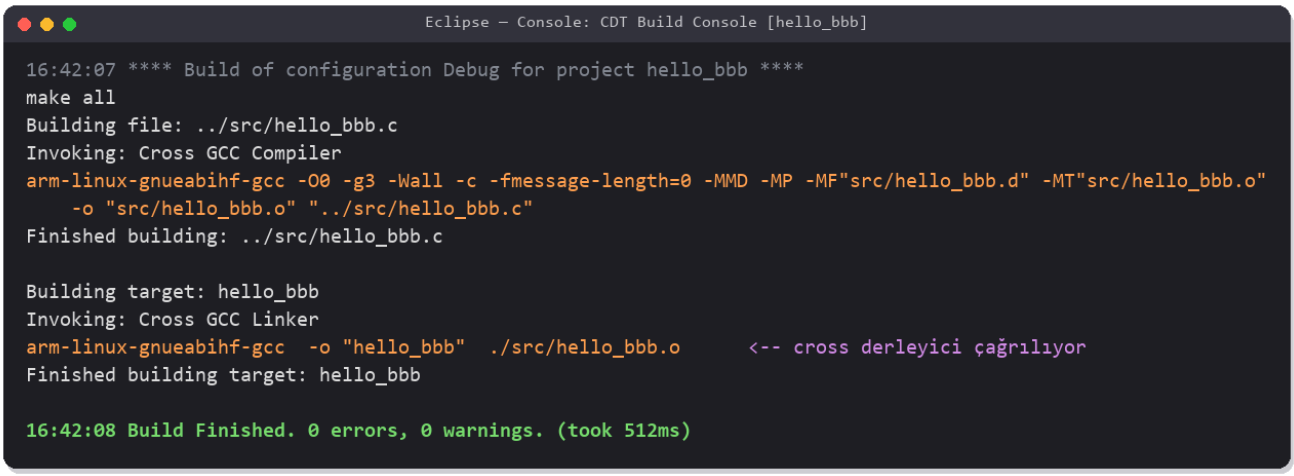
Alan	Değer
Cross compiler prefix	`arm-linux-gnueabihf-` (sondaki `-` dahil)
Cross compiler path	Toolchain'in `bin` klasörü, ör. `/home/<kullanıcı>/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin`

Finish'e bas. Eclipse `src/hello_bbb.c` dosyasını oluşturur. İçeriğini kartta çalıştığını gösterecek şekilde değiştir:

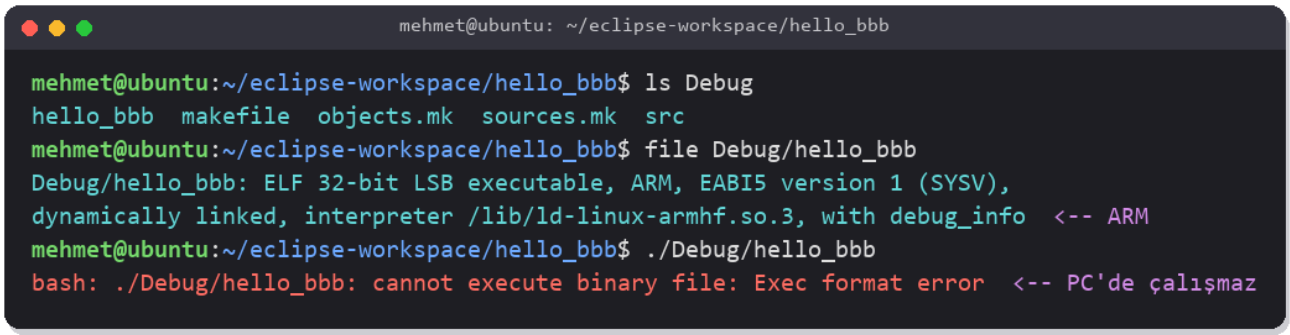
```c
#include <stdio.h>
#include <sys/utsname.h>

int main(void)
{
    struct utsname u;

    printf("Merhaba BeagleBone! (Eclipse'ten uzaktan çalıştırıldı)\n");
    if (uname(&u) == 0)
        printf("Kernel: %s %s %s %s\n", u.sysname, u.nodename, u.release, u.machine);
    return 0;
}
```

Adım 2 — Derle

Project → Build Project (veya çekiç simgesi). **Console** görünümünde Eclipse'in çağırdığı komutları görürsün:



Eclipse proje klasöründe bir `Debug/` klasörü ve içine kendi `makefile`'ını üretir. Çıktının gerçekten ARM için olduğunu terminalden doğrulayabilirsin:




```
file ~/eclipse-workspace/hello_bbb/Debug/hello_bbb
```

Derleme ayarları

Project → Properties → C/C++ Build → Settings → Tool Settings:

Bölüm	Ne ayarlanır?
Cross Settings	**Prefix** ve **Path**: hangi toolchain kullanılacak. Toolchain'i taşıdıysan ya da başka bir bilgisayardan proje aldıysan burayı düzelt.
Cross GCC Compiler → Optimization	`-O0` (Debug) / `-O2` (Release)
Cross GCC Compiler → Warnings	`-Wall` (tüm uyarılar)
Cross GCC Compiler → Includes	Ek başlık klasörleri (`-I`)
Cross GCC Linker → Libraries	Bağlanacak kütüphaneler (ör. `m` → `-lm` , `pthread`)
Cross GCC Linker → Miscellaneous → Linker flags	Ek bağlayıcı seçenekleri. Programı `-static` derlemek için buraya yaz (14.5).

Üst kısımdaki **Configuration** kutusu **Debug** ve **Release** arasında seçim yapar. Her yapılandırmanın kendi ayarları ve kendi çıktı klasörü vardır.

14.5 Karttaki Sistemle Uyum: Dinamik mi, Statik mi?

Eclipse varsayılan olarak **dinamik** bağlanmış program üretir. Dinamik program kartta çalışırken iki koşul sağlanmalı (12.11):

Karttaki sistem	Dinamik program çalışır mı?	Ne yapmalı?
eMMC'deki Debian (Bölüm 5)	Genelde evet	Toolchain'in glibc'si karttakinden **eski veya aynı** olmalı. Kartta `ldd --version` ile bak.
Buildroot (Bölüm 13)	Buildroot'un kendi toolchain'iyle derlenirse evet	Cross Settings'te Path = `output/host/bin` , Prefix = `arm-buildroot-linux-gnueabihf-`
BusyBox (statik) rootfs (Bölüm 12)	Hayır, `/lib` 'te kütüphane yok	Linker flags'e `-static` ekle ya da kütüphaneleri kopyala

> **İpucu**
>
> Emin değilsen `-static` ile derle. Program biraz büyür ama karttaki kütüphanelere bağımlı olmaz, her rootfs'te çalışır.

14.6 Uygulamayı Kartta Çalıştırmak

Kursun "Installing Remote System Explorer and Testing Application" dersi. Eclipse programı **SSH** ile karta kopyalar, çalıştırır ve çıktısını Console'da gösterir.

> **Önemli**
>
> Dosya kopyalama için kartta bir **SFTP sunucusu** olmalı.
>
> - Debian'da OpenSSH ile birlikte gelir, ek bir şey gerekmez.
> - Buildroot'ta Dropbear tek başına SFTP sunmaz. *Target packages → Networking applications*'dan **gesftpserver** paketini de ekle (ya da Dropbear yerine `openssh` seç).

Adım 1 — Bağlantı ve çalıştırma ayarı

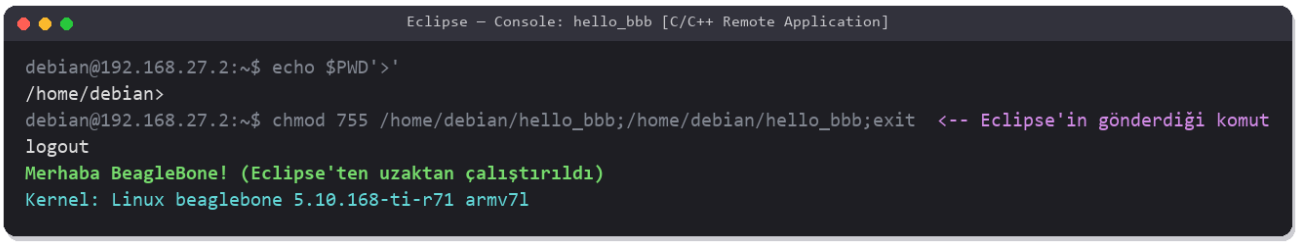
Run → Run Configurations... → sol listede **C/C++ Remote Application**'a sağ tık → *New Configuration*:

Alan	Değer
Project / C/C++ Application	`hello_bbb` / `Debug/hello_bbb`
Connection	*New...* → **SSH** → Host: `192.168.27.2`, User: `debian`, şifre
Remote Absolute File Path for C/C++ Application	`/home/debian/hello_bbb`
Commands to execute before application	Boş bırakabilirsin
Skip download to target path	İşaretleme (her çalıştırmada yeni derlenen dosya kopyalansın)

> **Not**
>
> Kartın IP'si bağlantı yöntemine göre değişir: Ethernet ile `192.168.27.2` (8.4), USB ile Debian'da `192.168.7.2` / `192.168.6.2` (5.4.1). Buildroot'ta kullanıcı `root` 'tur.

Adım 2 — Çalıştır

Run'a bas. Eclipse programı `/home/debian/` altına kopyalar, çalıştırılabilir yapar ve çalıştırır:

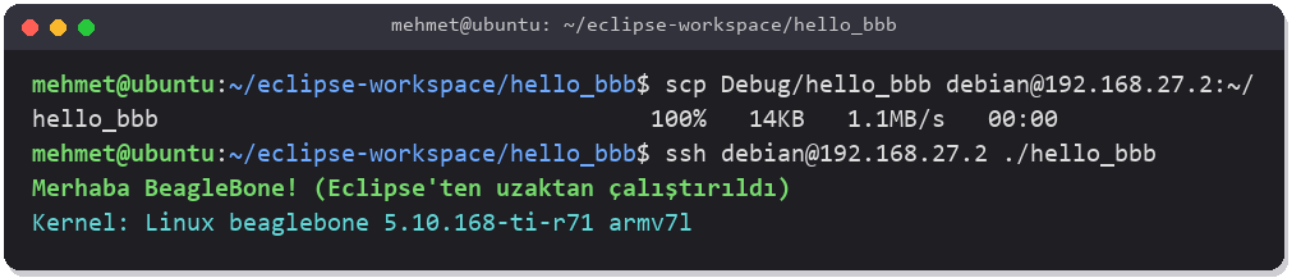


Bundan sonra yeşil *Run* düğmesi bu ayarı hatırlar: kodu değiştir, kaydet, *Run*'a bas. Eclipse önce derler, sonra karta gönderip çalıştırır.

> **İpucu**
>
> Kartın dosyalarını görmek ve kartta kabuk açmak için *Window → Show View → Other...*'dan **Connections** ve **Terminal** görünümlerini aç. Kursta RSE'nin "Remote Systems" görünümüyle yapılan dosya gezme ve terminal işlerini bunlar karşılar.

Eclipse olmadan: `scp` + `ssh`

Aynı işi terminalden de yapabilirsin (14.8'de ayrıntılı). Bir şey ters giderse sorunun Eclipse'te mi, kartta mı olduğunu anlamak için iyi bir testtir:




```
scp Debug/hello_bbb debian@192.168.27.2:~/
ssh debian@192.168.27.2 ./hello_bbb
```

14.7 Kursun Projelerini Eclipse'e Aktarmak

Kursun "Importing Eclipse Projects" dersi. Kursun reposundaki `Project_Src/` klasörü sonraki bölümlerin bütün uygulamalarını içeriyor.

1) Repoyu indir:

```
cd ~
git clone https://github.com/niekiran/EmbeddedLinuxBBB.git
```

2) İçe aktar: *File → Import → General → Existing Projects into Workspace → Select root directory*: `~/EmbeddedLinuxBBB/Project_Src` → istediğin projeleri işaretle → *Copy projects into workspace*'i işaretle → *Finish*.

3) Toolchain yolunu düzelt. Projelerdeki **Path** eğitmenin bilgisayarındaki yolu gösterir. Her proje için *Properties → C/C++ Build → Settings → Cross Settings → Path*'i kendi toolchain klasörünle değiştir. Prefix (`arm-linux-gnueabihf-`) zaten doğru.

4) Temiz derle: *Project → Clean...*, sonra *Build Project*. Repoda eğitmenin derlediği eski `Debug/` dosyaları da var; temizlemek bunları siler.

Proje	Ne yapar?	Kitapta
`BBB_led_control`	Kartın USR3 LED'ini sysfs ile kontrol eder	Bölüm 15
`counter_7seg`, `button_counter_7seg`	Tek haneli 7-segment gösterge, sayıcı	Bölüm 17
`4digitcount`	4 haneli 7-segment sayıcı	Bölüm 18
`MPU6050_raw_read`	I2C ile ivmeölçer/jiroskop okuma	Bölüm 19
`lcd_text`, `lcd_scrollable_text`, `lcd_news`	16x2 LCD'ye yazı yazma	Bölüm 20
`Drivers/gpio`, `Drivers/lcd`	Projelerin ortak kullandığı GPIO ve LCD fonksiyonları	Bölüm 16, Bölüm 20

> **Önemli**
>
> Kursun projelerinde hatalar var (ör. LED programında ham bayt yazımı, LCD'de eksik başlatma dizisi). Kitaptaki bölümler her projenin hatalarını anlatıp düzeltilmiş halini repodaki `code/` klasöründe veriyor; kendi çalışmanda o sürümleri kullan.

> **Not**
>
> Repodaki `.metadata` ve `RemoteSystemsTempFiles` klasörleri eğitmenin Eclipse çalışma alanına ait. İçe aktarırken bunları seçme.

14.8 Eclipse'siz Geliştirme: make, VS Code ve gdbserver

Eclipse zorunlu değil. Kurs kullandığı için bu bölümde anlatıldı, ama yaptığı her şeyin terminalde bir karşılığı var:

Eclipse'te	Eclipse'siz
Cross GCC proje ayarları	`Makefile` (`CROSS_COMPILE`, `CFLAGS`, `LDFLAGS`)
Build Project	`make`
C/C++ Remote Application	`scp` + `ssh` (`make run`)
Debug	Kartta `gdbserver`, host'ta `gdb-multiarch`
Editör, kod tamamlama	Herhangi bir editör; VS Code'da C/C++ eklentisi

Sonraki bölümlerin programları terminalden de derleniyor, Bölüm 21'den sonraki kernel sürücülerinde ise IDE hiç kullanılmıyor. Sektörde de en yaygın düzen budur: kod herhangi bir editörde yazılır, derleme `make` ya da CMake ile yapılır. Eclipse bir yerde takılırsa sorunu bulmak için de bu komutlara dönersin.

> **Not**
>
> Bu kısımdaki terminal görüntüleri temsilidir; sürüm numaraları, adresler ve süreler sende farklı olabilir.

Bu kısmın dosyaları repoda `code/14_no_ide/` klasöründe: `hello_bbb.c`, `Makefile` ve VS Code ayarlarının bulunduğu `.vscode/`.

14.8.1 Makefile ile derleme ve karta gönderme

`Makefile`'ın önemli kısmı:

```
PROG   ?= hello_bbb
SRCS   ?= hello_bbb.c
OBJS    = $(SRCS:.c=.o)

CROSS_COMPILE ?= arm-linux-gnueabihf-
CC      = $(CROSS_COMPILE)gcc
CFLAGS ?= -Wall -O0 -g

ifeq ($(STATIC),1)
LDFLAGS += -static
endif

BOARD  ?= debian@192.168.7.2
RDIR   ?= .

$(PROG): $(OBJS)
    $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)

deploy: $(PROG)
    scp $(PROG) $(BOARD):$(RDIR)/

run: deploy
    ssh $(BOARD) $(RDIR)/$(PROG)
```

- `CC`, Eclipse'teki **Prefix** ayarının karşılığıdır: `arm-linux-gnueabihf-` + `gcc`. Toolchain'in `bin` klasörü `PATH`'te olmalı (Bölüm 9). Değilse tam yolu ver: `make CROSS_COMPILE=~/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin/arm-linux-gnueabihf-`.
- `?=` ile tanımlanan her değişken komut satırından değiştirilebilir. Varsayılan kart, USB ile bağlı Debian'dır (5.4.1).
- `make STATIC=1`, Eclipse'te Linker flags'e `-static` yazmanın karşılığıdır (14.5).
- Makefile'da komut satırları **sekme (Tab)** ile başlar, boşlukla değil.

Derle, karta gönder ve çalıştır:

```
cd ~/code/14_no_ide
make
make run
make run BOARD=root@192.168.27.2      # Ethernet'teki Buildroot kartı
```

Her `scp` / `ssh` çağrısında şifre sorulmasın diye SSH anahtarını bir kez karta kopyala:

```
ssh-keygen -t ed25519                 # anahtarın yoksa; sorulara Enter
ssh-copy-id debian@192.168.7.2
```

> **İpucu**
>
> Kart rootfs'ini NFS'ten alıyorsa (Bölüm 12) `scp` 'ye de gerek yok: host'taki NFS klasörüne kopyaladığın dosya o an kartta görünür.
>
> ```
> make STATIC=1 && sudo cp hello_bbb /srv/nfs/bbb/
> ```
>
> Kartta `/hello_bbb` ile çalıştır.

Başka bir programı derlemek için `PROG` ve `SRCS` 'i değiştirmen yeterli. Örneğin Bölüm 17'deki sayıcı:

```
make PROG=counter_7seg \
     SRCS="../17_7segment/counter_7seg.c ../17_7segment/seg7.c ../lib/gpio_driver.c" \
     CFLAGS="-Wall -O2 -I../lib"
```

14.8.2 VS Code ile

VS Code'da Eclipse'in proje ayarlarının yerini `.vscode/` klasöründeki birkaç JSON dosyası alır. Derleme yine `make` ile yapılır, VS Code sadece onu çağırır.

1) Kur. VS Code'u (https://code.visualstudio.com) ve Microsoft'un **C/C++** eklentisini (`ms-vscode.cpptools`) kur. Hata ayıklama için host'ta `gdb-multiarch` gerekir:

```
sudo apt install gdb-multiarch
code ~/code/14_no_ide
```

2) Kod tamamlama: `c_cpp_properties.json` **.** Bu dosya yoksa VS Code host'un `/usr/include` başlıklarını kullanır ve ARM'a özgü kodda yanlış uyarılar verir. `compilerPath` cross derleyiciyi gösterince başlıkları ve makroları toolchain'den okur:

```
"compilerPath": "${env:HOME}/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin/arm-linux-gnueabihf-gcc",
"intelliSenseMode": "linux-gcc-arm"
```

3) Derleme ve çalıştırma: `tasks.json` **.** Üç görev tanımlı:

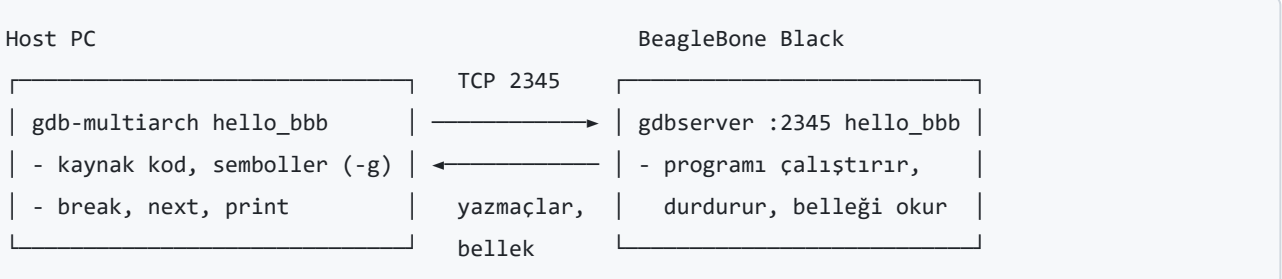
Görev	Komut	Nasıl çalıştırılır
`build`	`make`	**Ctrl+Shift+B**. Derleme hataları Problems panelinde, kendi satırında görünür.
`run on board`	`make run`	*Terminal → Run Task...*
`start gdbserver`	`make deploy` , sonra kartta `gdbserver`	Hata ayıklama başlarken kendiliğinden (14.8.4)

Kartın adresi `.vscode/settings.json` 'daki `bbb.board` değeridir.

> **Uyarı**
>
> VS Code'un **Remote-SSH** eklentisiyle doğrudan kartın içinde de çalışılabilir. Ama bunun için kartta VS Code'un sunucu tarafı çalışır; BBB'nin 512 MB RAM'i ve tek çekirdeği için bu ağırdır, BusyBox rootfs'te ise hiç çalışmaz. Kodu host'ta yazıp karta göndermek daha hızlıdır.

14.8.3 gdbserver ile uzaktan hata ayıklama

`printf` ile hata aramak her zaman yetmez. **gdb** ile programı bir satırda durdurabilir, değişkenlere bakabilir, adım adım ilerleyebilirsin. Program kartta çalıştığı için gdb iki parçaya bölünür:



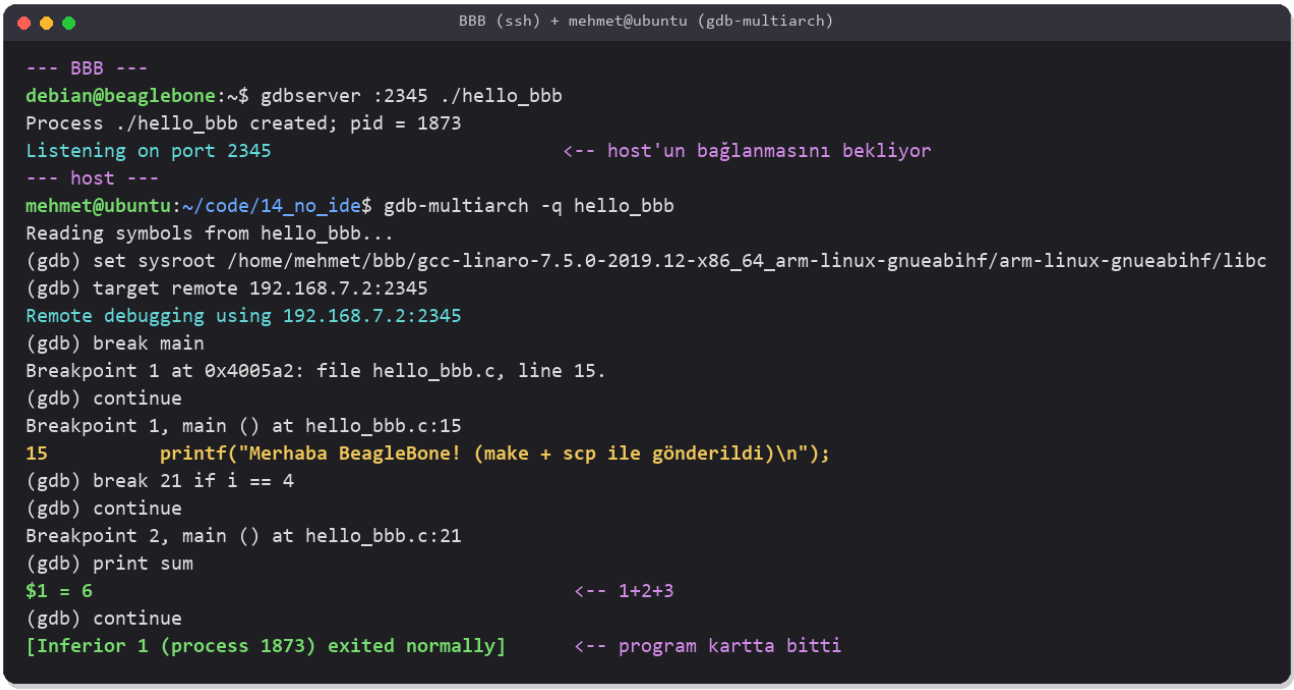
- **gdbserver** kartta çalışır. Küçüktür; kaynak koda ve sembollere ihtiyaç duymaz.
- **gdb** host'ta çalışır, sembolleri (`-g` ile derlenmiş dosyadan) ve kaynak kodu okur. Host'un normal `gdb`'si sadece x86'yı anlar; ARM için **`gdb-multiarch`** gerekir. (Linaro toolchain'inin içindeki `arm-linux-gnueabihf-gdb` de olur.)

gdbserver'ı karta kur:

Karttaki sistem	Nasıl?
Debian (Bölüm 5)	`sudo apt install gdbserver` (kartın internete çıkabilmesi gerekir)
Buildroot (Bölüm 13)	*Target packages* → *Debugging, profiling and benchmark* → *gdb*'yi seç; altındaki **gdbserver** seçeneği varsayılan olarak işaretli gelir. Sonra yeniden `make`.

Programı `-g -O0` ile derle. Makefile'daki varsayılan `CFLAGS` bunu yapar. `-O2` ile derlenmiş programda derleyici satırların sırasını değiştirir ve bazı değişkenleri yok eder; gdb onlar için `<optimized out>` gösterir.

Çalıştır:



Kartta:

```
gdbserver :2345 ./hello_bbb
```

Host'ta:

```
cd ~/code/14_no_ide
gdb-multiarch -q hello_bbb
```

```
(gdb) set sysroot ~/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/arm-linux-gnueabihf/libc
(gdb) target remote 192.168.7.2:2345
(gdb) break main
(gdb) continue
(gdb) next
(gdb) print sum
(gdb) continue
```

Komut	Kısaltma	Ne yapar?
`break main`, `break 21`	`b`	Fonksiyonda ya da satırda durma noktası koyar. `break 21 if i == 4` sadece koşul doğruysa durur.
`continue`	`c`	Sonraki durma noktasına kadar çalıştırır. gdbserver programı başlatıp ilk komutta durdurduğu için `run` değil `continue` yazılır.
`next` / `step`	`n` / `s`	Bir satır ilerler; `step` çağrılan fonksiyonun içine girer.
`print sum`	`p`	Değişkenin değerini gösterir.
`backtrace`	`bt`	Program çöktüyse hangi fonksiyon zincirinde olduğunu gösterir.
`info locals`		Fonksiyonun bütün yerel değişkenleri.

`set sysroot`, gdb'ye programın kullandığı paylaşılan kütüphanelerin (libc) host'taki kopyasını gösterir. Yol, `arm-linux-gnueabihf-gcc -print-sysroot` komutunun çıktısıdır. Program `-static` derlenmişse bu satıra gerek yoktur.

14.8.4 VS Code'da hata ayıklama

Aynı işi VS Code'un arayüzünden yapmak için `.vscode/launch.json`:

```
{
    "name": "Debug on BBB",
    "type": "cppdbg",
    "request": "launch",
    "program": "${workspaceFolder}/hello_bbb",
    "MIMode": "gdb",
    "miDebuggerPath": "/usr/bin/gdb-multiarch",
    "miDebuggerServerAddress": "192.168.7.2:2345",
    "preLaunchTask": "start gdbserver",
    "stopAtEntry": true,
    "setupCommands": [
        { "text": "set sysroot ${env:HOME}/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/arm-linux-gnueabihf/libc" }
    ]
}
```

F5'e bastığında:

1. `preLaunchTask` (14.8.2'deki `start gdbserver` görevi) programı derleyip karta kopyalar ve kartta `gdbserver`'ı başlatır.
2. Görev "Listening on port" satırını görünce VS Code `gdb-multiarch`'ı başlatır ve `miDebuggerServerAddress`'e bağlanır.
3. Program `main`'de durur. Satır numarasının soluna tıklayarak durma noktası koyar, **F10** (next) ve **F11** (step) ile ilerler, değişkenleri soldaki *Variables* panelinde görürsün.

Kartın IP'si değişirse hem `settings.json`'daki `bbb.board`'u hem `launch.json`'daki `miDebuggerServerAddress`'i güncelle.

14.9 Özet

- Eclipse (Eclipse IDE for C/C++ Developers) Java'sıyla birlikte gelir. Gereken eklentiler: **C/C++ GCC Cross Compiler Support** ve **C/C++ Remote Launch**.
- Proje: *C Managed Build → Executable → Cross GCC*. Toolchain'i **Prefix** (`arm-linux-gnueabihf-`) ve **Path** (`.../bin`) belirler.
- Karta gönderip çalıştırma: *Run Configurations → C/C++ Remote Application*, SSH bağlantısı; kartta SFTP sunucusu gerekir.
- Karttaki rootfs'te kütüphane yoksa ya da glibc eskiyse programı `-static` derle.
- Kursun projeleri: *Import → Existing Projects into Workspace*, sonra Cross Settings → Path'i düzelt.
- Eclipse zorunlu değil: `make` ile derle, `scp` + `ssh` ile çalıştır (`make run`). VS Code'da aynı işi `.vscode/` altındaki `tasks.json` ve `launch.json` yapar.
- Hata ayıklama: kartta `gdbserver :2345 ./program`, host'ta `gdb-multiarch` → `target remote <kart>:2345`. Programı `-g -O0` ile derle.

14.10 Karşılaşılabilecek Problemler

Eclipse — Problems / Console (temsili hatalar)

```
Program "arm-linux-gnueabihf-gcc" not found in PATH              <-- Cross GCC Path yanlış

make: *** No rule to make target 'all'.  Stop.          <-- proje 'Makefile project' seçilmiş

./hello_bbb: /lib/arm-linux-gnueabihf/libc.so.6: version `GLIBC_2.34' not found
    <-- toolchain'in glibc'si karttakinden yeni: aynı/eski toolchain ya da -static

-sh: ./hello_bbb: not found                                <-- BusyBox rootfs'te /lib yok: -static

Error during file upload. Failure                    <-- kartta SFTP sunucusu yok

Remote 'g' packet reply is too long (expected 260 bytes, got 328 bytes)
    <-- host'un x86 gdb'si kullanıldı: gdb-multiarch

192.168.7.2:2345: Connection timed out.           <-- kartta gdbserver çalışmıyor
```

Belirti	Sebep	Çözüm
`Program "arm-linux-gnueabihf-gcc" not found in PATH`	Cross Settings → Path yanlış ya da boş	`which arm-linux-gnueabihf-gcc` çıktısının klasörünü yaz
Proje sihirbazında "Cross GCC" yok	C/C++ GCC Cross Compiler Support eklentisi kurulu değil	14.3 Adım 4
Run Configurations'ta "C/C++ Remote Application" yok	C/C++ Remote Launch eklentisi kurulu değil	14.3 Adım 4
`make: *** No rule to make target 'all'`	Proje "Makefile Project" türünde açılmış, Eclipse makefile üretmiyor	Projeyi "C Managed Build → Executable" olarak yeniden oluştur
Kartta `version 'GLIBC_2.xx' not found`	Toolchain'in glibc'si karttakinden yeni	Karttakiyle uyumlu toolchain kullan ya da `-static`
Kartta `./program: not found` (dosya varken)	Dinamik program, rootfs'te `/lib/ld-linux-armhf.so.3` yok	`-static` ile derle
`Error during file upload`	Kartta SFTP sunucusu yok	Buildroot'a `gesftpserver` ekle
`Permission denied` (çalıştırırken)	Dosya çalıştırılabilir değil ya da yanlış klasöre kopyalandı	Remote path'in kullanıcının yazabildiği bir yer olduğundan emin ol
Kartta `Exec format error`	Program ARM için değil, host için derlenmiş	Toolchain olarak Cross GCC seçildiğini, `file` çıktısında ARM yazdığını kontrol et
`missing separator. Stop.`	Makefile'da komut satırı boşlukla başlıyor	Komut satırlarının başına **Tab** koy
`make: arm-linux-gnueabihf-gcc: No such file or directory`	Toolchain `PATH`'te değil	`make CROSS_COMPILE=<toolchain>/bin/arm-linux-gnueabihf-` ya da `PATH`'e ekle (14.8.1)
gdb'de `Remote 'g' packet reply is too long`	ARM programa host'un x86 `gdb`'siyle bağlanıldı	`gdb-multiarch` kullan
`target remote` → `Connection timed out` / `refused`	Kartta `gdbserver` çalışmıyor, IP ya da port yanlış	Kartta `gdbserver :2345 ./program` çalışıyor mu bak
gdb'de değişken `<optimized out>`, satırlar atlanıyor	Program `-O2` ile derlenmiş	`-g -O0` ile derle
VS Code ARM başlıklarında yanlış hata gösteriyor	IntelliSense host derleyicisini kullanıyor	`c_cpp_properties.json`'da `compilerPath`'i cross gcc yap

Bölüm 15 — AM335x GPIO'ları, Pin Ayarları ve sysfs

> **Bu bölümde:**
>
> - AM335x'in GPIO alt sistemini ve GPIO numaralarının nasıl hesaplandığını öğreneceksin.
> - P8/P9 header'larında hangi pinlerin GPIO olarak kullanılabileceğini göreceksin.
> - Pin multiplexing'i (pinmux) ve pin ayar register'larının bitlerini çözeceksin.
> - Çalışan sistemde pinlerin durumunu sysfs/debugfs'ten okuyacaksın.
> - USER LED'leri ve GPIO'ları komut satırından, sonra da bir C programıyla kontrol edeceksin.
>
> **Ön koşul:** Bölüm 1 — P8/P9 header'ları, Bölüm 11 — device tree ve pinctrl, Bölüm 14 — kartta program çalıştırma.

> **Not**
>
> Bu bölümdeki komutlar kartta açık bir Linux'ta (eMMC'deki Debian ya da kendi rootfs'in) **root** olarak çalıştırılır. Debian'da `sudo -i` ile root olabilirsin. Terminal görüntüleri temsilidir; GPIO numaraları kernel 5.10'a göredir (15.6).

15.1 AM335x GPIO Alt Sistemi

AM335x'te **4 GPIO modülü (bankası)** vardır ve her biri **32 pin** yönetir. Toplam 128 GPIO:

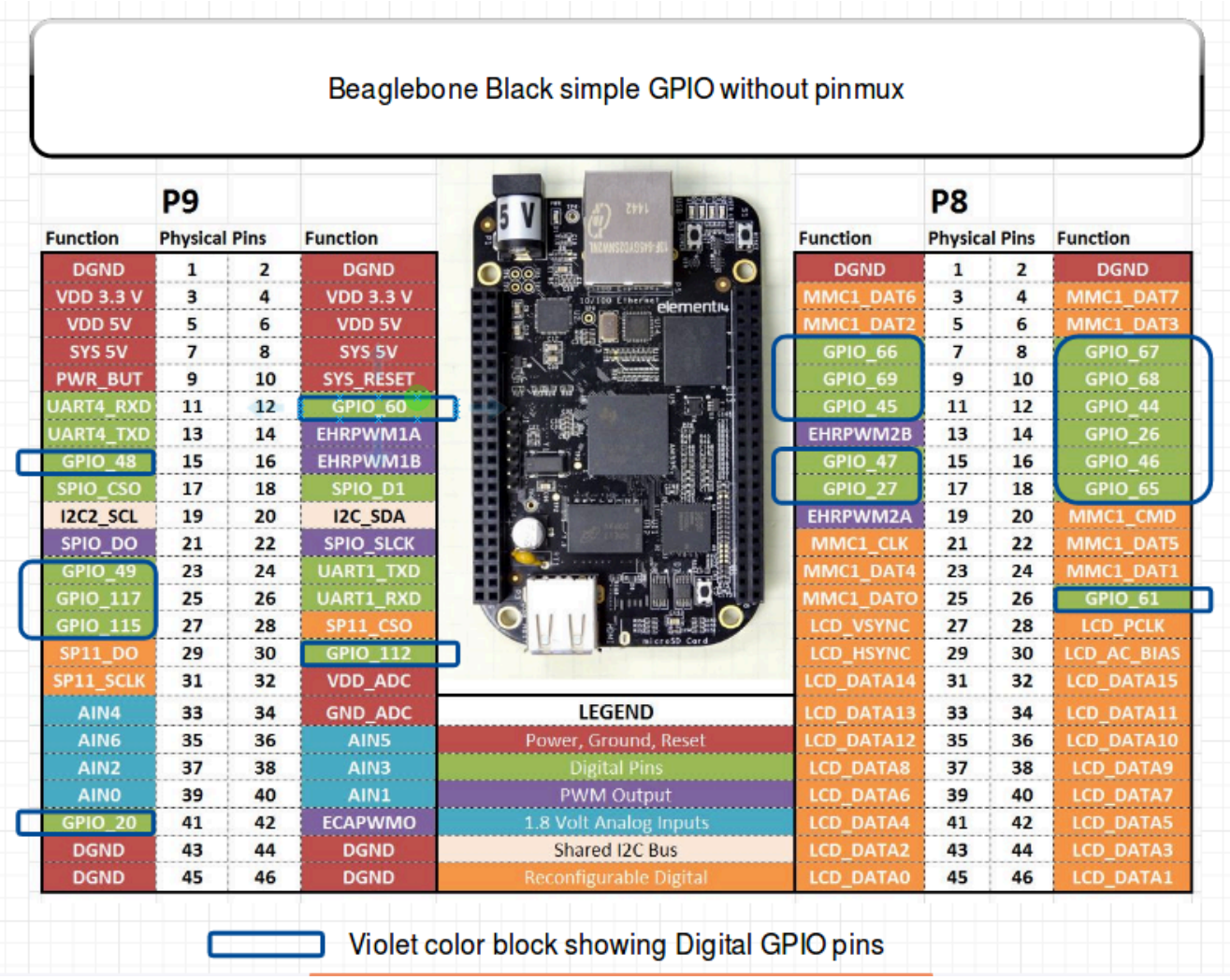
Banka	Register adresi	Linux'taki adı	Kernel 5.10'daki GPIO numaraları
GPIO0	`0x44E07000`	`gpiochip0`	0 – 31
GPIO1	`0x4804C000`	`gpiochip1`	32 – 63
GPIO2	`0x481AC000`	`gpiochip2`	64 – 95
GPIO3	`0x481AE000`	`gpiochip3`	96 – 127

Datasheet ve şemalarda pinler **`GPIOx_y`** diye geçer: x banka, y bankadaki bit numarası. Linux'un eski GPIO numarası şöyle hesaplanır:

```
GPIO numarası = banka × 32 + bit

GPIO1_21 (USR0 LED)  →  1 × 32 + 21 =  53
GPIO1_28 (P9_12)     →  1 × 32 + 28 =  60
GPIO2_2  (P8_7)      →  2 × 32 +  2 =  66
```

Her GPIO modülü giriş/çıkış yönü, çıkış değeri, giriş okuma ve **kesme (interrupt)** için register'lar içerir. Linux'ta bu register'larla `gpio-omap` sürücüsü konuşur (10.6); biz sürücünün kullanıcı alanına açtığı arayüzleri kullanacağız.

15.2 Genişleme Header'larındaki GPIO'lar



Görsel: Bootlin, "Embedded Linux system development" lab materyalleri (CC BY-SA 3.0)

Görselde mor çerçeveli pinler, başka bir ayar yapmadan doğrudan GPIO olarak kullanılabilen pinlerdir. Sık kullanılanlar:

Header pini	AM335x adı	GPIO no (5.10)	Pin ayar register'ı (`0x44E10000` +)
P8_7	GPIO2_2	66	`0x890`
P8_12	GPIO1_12	44	`0x830`
P9_12	GPIO1_28	60	`0x878`
P9_15	GPIO1_16	48	`0x840`
USR0 – USR3 (kart üstü LED)	GPIO1_21 – GPIO1_24	53 – 56	`0x854` – `0x860`

> **Dikkat**
>
> - Pinler **3.3V**'tur ve 5V toleranslı değildir (1.9).
> - Bir pin ancak birkaç mA akım verebilir. LED'i mutlaka seri dirençle bağla; röle, motor gibi yükleri transistör üzerinden sür.
> - P8_3 – P8_6, P8_20 – P8_25 **eMMC**'ye, P8_27 – P8_46 **HDMI**'ye bağlıdır. eMMC'yi veya HDMI'ı kapatmadan bu pinleri GPIO olarak kullanma.

15.3 Pin Multiplexing ve Pin Ayar Register'ları

AM335x'in her fiziksel pini **8 farklı görevden** birini üstlenebilir (mod 0 – mod 7). Örneğin P9_24 pini UART1'in TX'i, I2C1'in SCL'si ya da bir GPIO olabilir. Hangi görevin seçileceğine **pin multiplexing (pinmux)** denir. GPIO görevi her pinde **mod 7**'dir.

Pinmux, SoC'nin **Control Module**'ündeki pin ayar register'larıyla yapılır. Control Module `0x44E10000` adresindedir; pin register'ları `0x800` ofsetinden başlar ve her pinin kendi register'ı vardır (adı `conf_<modül>_<pin>`, ör. USR0 için `conf_gpmc_a5`, `0x44E10854`).

Register'ın bitleri (AM335x TRM, Control Module):

Bit	Ad	Anlamı
2:0	`MUXMODE`	Mod 0 – 7. **7 = GPIO.**
3	`PULLUDEN`	**0 = pull direnci açık**, 1 = kapalı (ters mantık)
4	`PULLTYPESEL`	0 = pull-down, 1 = pull-up
5	`RXACTIVE`	1 = giriş tamponu açık (pin okunabilir). Giriş olarak kullanılacak pinlerde 1 olmalı.
6	`SLEWCTRL`	0 = hızlı, 1 = yavaş kenar geçişi

Örnekler:

Değer	İkili	Anlamı
`0x07`	`000 0111`	Mod 7 (GPIO), pull-down, giriş kapalı. USER LED'ler böyle (sadece çıkış).
`0x27`	`010 0111`	Mod 7, pull-down, **giriş açık**
`0x37`	`011 0111`	Mod 7, **pull-up**, giriş açık. Butonlar için tipik.

> **Önemli**
>
> Bu register'lar **kullanıcı alanından yazılamaz**: AM335x'te Control Module'e yazmak için işlemcinin ayrıcalıklı (privileged) modda olması gerekir. Pinmux bu yüzden **device tree**'de `pinctrl` düğümleriyle yapılır (11.6), kernel açılırken uygular. BeagleBoard'un Debian imajlarında hazır overlay'ler sayesinde `config-pin` aracıyla da değiştirilebilir (ör. `config-pin P9_12 gpio`, `config-pin -q P9_12`).

Device tree'de bir pin grubu şöyle tanımlanır (`am335x-bone-common.dtsi`'den, USER LED'ler):

```
user_leds_s0: user_leds_s0 {
    pinctrl-single,pins = <
        AM33XX_PADCONF(AM335X_PIN_GPMC_A5, PIN_OUTPUT_PULLDOWN, MUX_MODE7)  /* gpmc_a5.gpio1_21
*/
        AM33XX_PADCONF(AM335X_PIN_GPMC_A6, PIN_OUTPUT_PULLUP, MUX_MODE7)    /* gpmc_a6.gpio1_22
*/
        AM33XX_PADCONF(AM335X_PIN_GPMC_A7, PIN_OUTPUT_PULLDOWN, MUX_MODE7)  /* gpmc_a7.gpio1_23
*/
        AM33XX_PADCONF(AM335X_PIN_GPMC_A8, PIN_OUTPUT_PULLUP, MUX_MODE7)    /* gpmc_a8.gpio1_24
*/
    >;
};
```

`PIN_OUTPUT_PULLDOWN`, `MUX_MODE7` gibi isimler yukarıdaki bitlerin sabitleridir.

15.4 Çalışan Sistemde Pin Bilgisini Okumak

Kursun "Exploring Pin details using SYSFS entries" dersleri. Kernel, pinlerin ve GPIO'ların durumunu **debugfs**'ten (`/sys/kernel/debug`) okunabilir hale getirir. Boşsa önce bağla:

```
mount | grep debugfs || mount -t debugfs none /sys/kernel/debug
```

GPIO'ların durumu

```
cat /sys/kernel/debug/gpio
```

Her satır bir GPIO'yu, onu kullanan sürücüyü (ör. `beaglebone:green:usr0` LED sürücüsü), yönünü (`in` / `out`) ve değerini (`lo` / `hi`) gösterir.

Pinmux durumu

```
ls /sys/kernel/debug/pinctrl/
cd /sys/kernel/debug/pinctrl/44e10800.pinmux*/
grep 44e10854 pins          # USR0'ın pin register'ı ve değeri
grep "pin 21 " pinmux-pins  # bu pini hangi sürücü/grup kullanıyor?
```

Dosya	İçinde ne var?
`pins`	Her pinin register adresi ve **şu anki değeri**. 15.3'teki tabloyla çözülür.
`pinmux-pins`	Pini hangi sürücünün, hangi pin grubuyla aldığı
`pingroups`	Device tree'de tanımlı pin grupları ve içindeki pinler

`pin 21 ... 44e10854 00000007` satırını çözelim: register `0x854` = `conf_gpmc_a5` , değer `0x07` = mod 7 (GPIO), pull-down, giriş kapalı. Yani USR0 bir GPIO çıkışı.

15.5 USER LED'leri sysfs ile Kontrol Etmek

Kursun "Controlling USER LEDs using SYSFS entries" dersleri. USER LED'ler kernel'in **LED alt sistemi** (`leds-gpio` sürücüsü) tarafından yönetilir ve `/sys/class/leds/` altında görünür. Her LED bir klasördür:

```
debian@beaglebone: ~ (SSH)
debian@beaglebone:~$ ls /sys/class/leds/
beaglebone:green:usr0  beaglebone:green:usr1  beaglebone:green:usr2  beaglebone:green:usr3
root@beaglebone:~# cd /sys/class/leds/beaglebone:green:usr3
root@beaglebone:usr3# ls
brightness  device  max_brightness  power  subsystem  trigger  uevent
root@beaglebone:usr3# cat trigger
none ... timer oneshot heartbeat ... default-on ... [mmc1] ...  <-- [..] = şu anki
root@beaglebone:usr3# echo none > trigger

root@beaglebone:usr3# echo 1 > brightness
                                  <-- LED yandı
root@beaglebone:usr3# echo timer > trigger
root@beaglebone:usr3# echo 100 > delay_on; echo 900 > delay_off
                                  <-- 100 ms yanık, 900 ms sönük
root@beaglebone:usr3# echo mmc1 > trigger
                                  <-- varsayılana dön (eMMC erişimi)
```

Dosya	Anlamı
`brightness`	`0` = sönük, `1` (veya `max_brightness`'a kadar) = yanık
`trigger`	LED'i kernel'de hangi olayın süreceği. `cat` ile okununca seçenekler listelenir, köşeli parantezdeki şu anki tetikleyicidir.
`delay_on`, `delay_off`	Sadece `timer` tetikleyicisinde görünür: yanık / sönük kalma süresi (ms)

Trigger	LED neye göre yanar?
`none`	Hiçbir şeye; `brightness`'a ne yazarsan
`heartbeat`	Kalp atışı gibi; sistem yükü arttıkça hızlanır
`timer`	`delay_on` / `delay_off` sürelerine göre yanıp söner
`oneshot`	`shot` dosyasına her yazılışta bir kez yanıp söner
`default-on`	Sürekli yanık
`mmc0`, `mmc1`, `cpu0` ...	SD kart / eMMC erişimi, işlemci etkinliği

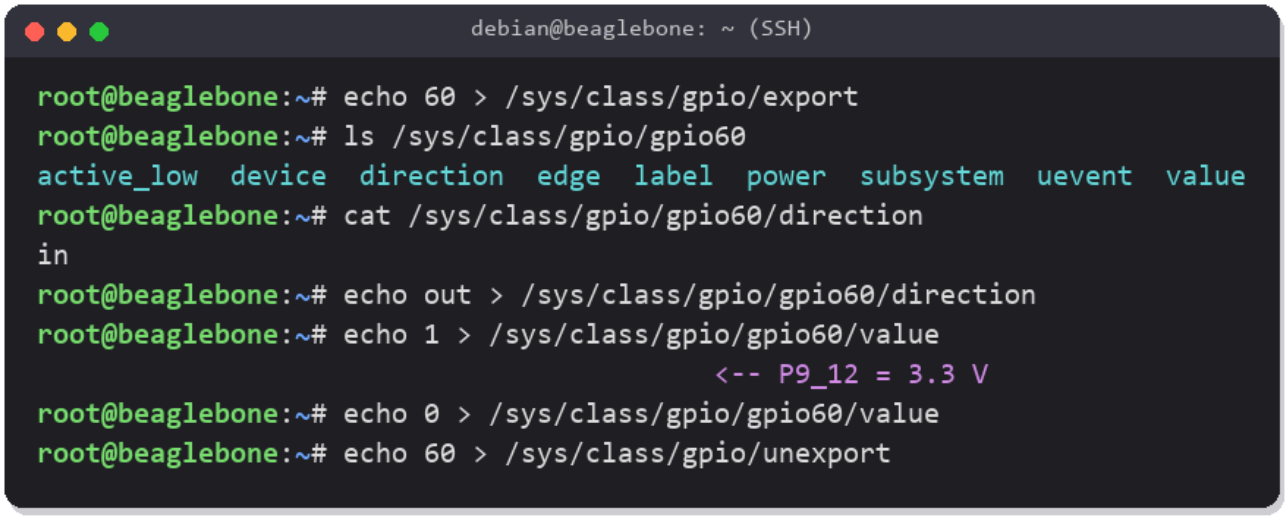
```
cd /sys/class/leds/beaglebone:green:usr3
cat trigger
echo none > trigger
echo 1 > brightness                          # yak
echo 0 > brightness                          # söndür
echo timer > trigger
echo 100 > delay_on; echo 900 > delay_off    # 100 ms yanık, 900 ms sönük
echo mmc1 > trigger                          # varsayılana dön
```

> **İpucu**
>
> Bir tetikleyici aktifken `brightness`'a `1` yazarsan tetikleyici değeri hemen ezer. Önce `trigger`'ı `none` yap. (`brightness`'a `0` yazmak ise tetikleyiciyi kendiliğinden kapatır.)

15.6 GPIO'ları Kullanıcı Alanından Kontrol Etmek

USER LED'ler bir sürücüye ait. Header'daki boş bir GPIO'yu (ör. P9_12) kontrol etmek için iki arayüz var.

a) sysfs GPIO arayüzü (kursta kullanılan)




```
echo 60 > /sys/class/gpio/export             # GPIO 60'ı (P9_12) kullanıcı alanına aç
echo out > /sys/class/gpio/gpio60/direction  # çıkış
echo 1 > /sys/class/gpio/gpio60/value        # 3.3 V
echo 0 > /sys/class/gpio/gpio60/value        # 0 V
echo 60 > /sys/class/gpio/unexport           # bırak
```

Dosya	Anlamı
`direction`	`in` / `out` (`high` / `low` yazmak çıkış yapıp değeri de ayarlar)

Dosya	Anlamı
`value`	Okunan veya yazılan değer
`edge`	Giriş için kesme: `none`, `rising`, `falling`, `both` (`poll()` ile beklenir)
`active_low`	`1` ise mantık terslenir

> **Uyarı**
>
> Bu arayüz kernel'de **eskimiş (deprecated)** olarak işaretlidir. BeagleBoard'un 5.10 kernel'inde hâlâ açık (`CONFIG_GPIO_SYSFS=y`) ve banka numaraları sabit (0, 32, 64, 96). Ama güncel mainline kernel'lerde (ör. Bölüm 13'teki 6.18.1) GPIO numaraları **dinamik** verilir ve genelde 512'den başlar: `echo 60 > export` orada başka bir pine denk gelir ya da hata verir. Doğru numarayı `cat /sys/kernel/debug/gpio` veya `ls /sys/class/gpio/` (`gpiochipNNN` = bankanın ilk numarası) ile bul. Yeni projelerde aşağıdaki libgpiod'u kullan.

b) libgpiod (güncel yöntem)

Kernel'in karakter aygıtı arayüzünü (`/dev/gpiochipN`) kullanır. GPIO'yu **banka + bit** ile adresler; numara hesabı ve kernel sürümü farkı yoktur. Debian'da `sudo apt install gpiod`, Buildroot'ta *Target packages → Libraries → Hardware handling → libgpiod* (araçlarla birlikte).

```
debian@beaglebone: ~ (SSH)
root@beaglebone:~# gpiodetect
gpiochip0 [44e07000.gpio] (32 lines)
gpiochip1 [4804c000.gpio] (32 lines)        <-- GPIO1 bankası
gpiochip2 [481ac000.gpio] (32 lines)
gpiochip3 [481ae000.gpio] (32 lines)
root@beaglebone:~# gpioinfo gpiochip1 | sed -n '22p;29p'
□line  21: "P8_.."   "beaglebone:green:usr0" output active-high [used]
□line  28: "P9_12"   unused input active-high       <-- GPIO1_28
--- libgpiod 1.x (Debian 10/11/12) ---
root@beaglebone:~# gpioset gpiochip1 28=1
root@beaglebone:~# gpioget gpiochip1 28
1
--- libgpiod 2.x (Debian 13 ve sonrası) ---
root@beaglebone:~# gpioset -c gpiochip1 28=1
^C                                          <-- v2: Ctrl-C'ye kadar tutar
```

Araç	Görevi
`gpiodetect`	GPIO bankalarını listeler
`gpioinfo`	Her hattın adı, yönü ve kim tarafından kullanıldığı

Araç	Görevi
`gpioget`	Girişi okur
`gpioset`	Çıkışı ayarlar
`gpiomon`	Kenar (edge) olaylarını bekler

libgpiod'un iki sürümünün komut sözdizimi farklıdır:

İş	libgpiod 1.x (Debian 10/11/12)	libgpiod 2.x (Debian 13 ve sonrası)
P9_12'yi 1 yap	`gpioset gpiochip1 28=1`	`gpioset -c gpiochip1 28=1` (Ctrl-C'ye kadar tutar)
P9_12'yi oku	`gpioget gpiochip1 28`	`gpioget -c gpiochip1 28`
Sürümü öğren	`gpioset --version`	`gpioset --version`

15.7 C Programıyla USER LED Kontrolü

Kursun "Controlling user LEDs using 'C' Application" dersleri ve repodaki `BBB_led_control` projesi (14.7). Program, 15.5'teki `echo` komutlarının aynısını C'de `open()` + `write()` ile yapar.

Kursun kodundaki hata

Kursun `write_brightness_values()` fonksiyonu değeri şöyle yazıyor:

```
ret = write(fd, (void *)&value, sizeof(value));   /* value bir int */
```

Bu satır `1` sayısının bellekteki **4 ham baytını** (`01 00 00 00`) yazar. Oysa sysfs dosyaları `echo`'nun yazdığı gibi **metin** bekler: `"1"`. Kernel gelen veriyi sayıya çeviremez, `write()` hata döner ve program `trigger value write error` basar. `trigger` kısmı metin yazdığı için çalışır, `brightness` kısmı çalışmaz. Ayrıca açılan dosyalar hiç kapatılmıyor.

Doğrusu, değeri metin olarak yazmak:

```
write(fd, "1", 1);
```

Düzeltilmiş ve genelleştirilmiş program

Kursun "TODO" notundaki öneriye uygun olarak program dört LED'in hepsini destekleyecek şekilde genişletildi. Tam kod: `code/15_gpio/usr_led.c`.

```
#define LED_ATTR_PATH "/sys/class/leds/beaglebone:green:usr%d/%s"

/* /sys/class/leds/beaglebone:green:usr<led>/<attr> dosyasına value'yu yazar. */
static int write_attr(int led, const char *attr, const char *value)
{
    char path[128];
    ssize_t n;
    int fd;

    snprintf(path, sizeof(path), LED_ATTR_PATH, led, attr);

    fd = open(path, O_WRONLY);
    if (fd < 0) {
        perror(path);
        return -1;
    }

    n = write(fd, value, strlen(value));   /* metin olarak: "1", "timer" ... */
    if (n < 0)
        perror("write");

    close(fd);
    return n < 0 ? -1 : 0;
}
```

`main()`, argümanları kontrol eder ve `write_attr(led, "trigger", value)` ya da `write_attr(led, "brightness", value)` çağırır.

Derle ve çalıştır

Eclipse ile (14.4) ya da terminalden:

```
arm-linux-gnueabihf-gcc -Wall -o usr_led code/15_gpio/usr_led.c
scp usr_led debian@192.168.27.2:~/
```

Kartta (root olarak):

```
./usr_led 3 trigger none
./usr_led 3 brightness 1
./usr_led 3 trigger heartbeat
```

15.8 Özet

- AM335x: 4 GPIO bankası × 32 pin. `GPIOx_y` → kernel 5.10'da numara `x × 32 + y`; yeni kernel'lerde numara dinamik.
- Her pinin 8 modu var; GPIO = mod 7. Pin ayar register'ı `0x44E10800`'den başlar: mod, pull-up/down, giriş açık/kapalı. Kullanıcı alanından yazılamaz; device tree (`pinctrl`) ya da `config-pin` ile ayarlanır.
- Durumu okumak: `/sys/kernel/debug/gpio`, `/sys/kernel/debug/pinctrl/.../pins`.
- USER LED'ler: `/sys/class/leds/beaglebone:green:usrN/{trigger,brightness}`.
- Boş GPIO'lar: eski `/sys/class/gpio` (kursta) ya da güncel **libgpiod** (`gpioset` / `gpioget`).
- sysfs'e C'den yazarken değer **metin** olmalı.

15.9 Karşılaşılabilecek Problemler

```
debian@beaglebone: ~ (SSH) (temsili hatalar)
debian@beaglebone:~$ echo 1 > /sys/class/leds/beaglebone:green:usr3/brightness
-bash: /sys/class/leds/.../brightness: Permission denied   <-- root/sudo

root@beaglebone:~# echo 60 > /sys/class/gpio/export
bash: echo: write error: Device or resource busy         <-- pin başka sürücüde / zaten export

root@beaglebone:~# echo 60 > /sys/class/gpio/export
bash: echo: write error: Invalid argument             <-- yeni kernel: numaralar 512+

root@beaglebone:~# ./led_control brightness 1
trigger value write error                          <-- kursun kodundaki hata (15.7)

root@beaglebone:~# ls /sys/kernel/debug
(boş)                                          <-- mount -t debugfs none /sys/kernel/debug
```

Belirti	Sebep	Çözüm
`Permission denied` (`echo ... > brightness`)	Kullanıcı root değil; `sudo echo 1 > dosya` da çalışmaz (yönlendirmeyi kabuk yapar)	`sudo -i` ile root ol ya da `echo 1 | sudo tee dosya`
`export`: `Device or resource busy`	Pin başka bir sürücüde (ör. LED, eMMC) ya da zaten export edilmiş	`cat /sys/kernel/debug/gpio` ile kimin kullandığına bak
`export`: `Invalid argument`	Yeni kernel'de GPIO numaraları 512'den başlıyor	Doğru numarayı `/sys/kernel/debug/gpio`'dan bul ya da libgpiod kullan
Pine `1` yazıyorum ama voltaj yok	Pin GPIO modunda değil (pinmux)	`pins` dosyasında register değerinin mod 7 olduğunu kontrol et; değilse `config-pin` / device tree
Giriş hep aynı değeri okuyor	`RXACTIVE` kapalı ya da pin boşta ve pull direnci yok	Pin ayarını `0x27` / `0x37` yap; butonda pull-up/down kullan
`brightness` yazılıyor ama LED hemen eski haline dönüyor	Tetikleyici (`heartbeat`, `mmc1`) aktif	Önce `echo none > trigger`
Kursun programı `trigger value write error` basıyor	Değer metin değil, ham bayt olarak yazılıyor	15.7'deki düzeltilmiş kod
`/sys/kernel/debug` boş	debugfs bağlı değil	`mount -t debugfs none /sys/kernel/debug`
`gpioset: invalid option -- 'c'` / `unable to request lines`	libgpiod 1.x ile 2.x sözdizimi karıştı	`gpioset --version` ile sürüme bak, 15.6'daki tabloya göre yaz

Bölüm 16 — Harici LED Bağlamak

> **Bu bölümde:**
>
> - Bir LED'i doğru dirençle bir GPIO pinine bağlayacaksın.
> - Pinin GPIO modunda olduğunu doğrulayıp LED'i komut satırından yakıp söndüreceksin.
> - Kursun GPIO kütüphanesini (düzeltilmiş haliyle) kullanan bir C programıyla LED'i yanıp söndüreceksin.
> - Ek: LED'i bir device tree overlay ile kernel'in LED alt sistemine bağlayacaksın.
>
> **Ön koşul:** Bölüm 15 — GPIO numaraları, pinmux ve sysfs.

> **Not**
>
> Bu bölümdeki komutlar kartta **root** olarak çalıştırılır. Örneklerde LED **P9_12**'ye (GPIO1_28, kernel 5.10'da GPIO 60) bağlı. Terminal görüntüleri temsilidir.

16.1 Gerekenler ve Devre

Malzeme	Not
LED (5 mm, kırmızı)	Uzun bacak **anot (+)**, kısa bacak / düz kenar **katot (−)**
Direnç: 470 Ω – 1 kΩ	Akımı sınırlar (hesap aşağıda)
Breadboard ve erkek-erkek kablolar	

Direnç hesabı

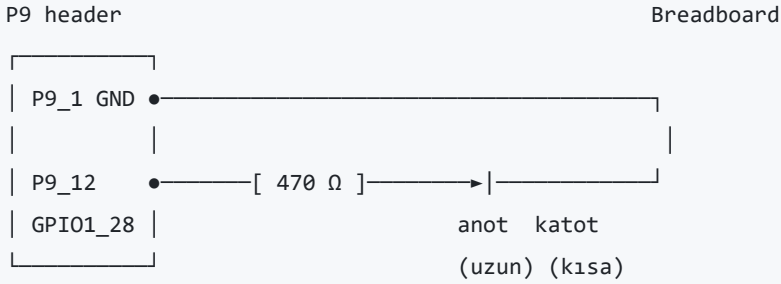
GPIO pini yüksek olunca 3.3 V verir. LED'in üzerinde kendi **ileri yön gerilimi** (V_F) kadar gerilim düşer, kalan gerilim dirence kalır:

$$R = \frac{V_{GPIO} - V_F}{I} = \frac{3.3\ V - 2.0\ V}{3\ mA} \approx 430\ \Omega \rightarrow 470\ \Omega \text{ (standart değer)}$$

LED rengi	Tipik V_F	470 Ω ile akım	1 kΩ ile akım
Kırmızı	~2.0 V	~2.8 mA	~1.3 mA
Yeşil / sarı	~2.1 V	~2.6 mA	~1.2 mA
Mavi / beyaz	~3.0 V	~0.6 mA (soluk)	çok soluk

> **Dikkat**
>
> AM335x pinleri sadece **birkaç mA** akım verebilir. LED'i dirençsiz bağlama; akımı birkaç mA ile sınırlı tut. Daha fazla akım çeken yükleri (parlak LED'ler, röle, motor) bir **transistör** üzerinden sür: GPIO → ~1 kΩ → NPN transistörün (ör. BC547, 2N2222) bazı; yük, 5 V/3.3 V ile transistörün kolektörü arasında.

Bağlantı



Bu bağlantıda pin **1** olunca LED yanar (aktif yüksek). Tersi de mümkündür: LED'i 3.3 V (P9_3) ile pin arasına bağlarsan pin **0** olunca yanar (aktif düşük); o zaman sysfs'te `active_low` dosyasına `1` yazmak mantığı düzeltir (15.6).

> **Uyarı**
>
> Kabloları kart **kapalıyken** bağla. P9_1/P9_2 GND, P9_3/P9_4 3.3 V, P9_5–P9_8 5 V'tur. Bir GPIO pinini yanlışlıkla 5 V'a değdirmek işlemciyi bozabilir (1.9).

16.2 Pinin GPIO Modunda Olduğunu Doğrula

P9_12 varsayılan olarak GPIO modundadır. Yine de bir pine LED bağlamadan önce kontrol etmek iyi alışkanlıktır (15.4):

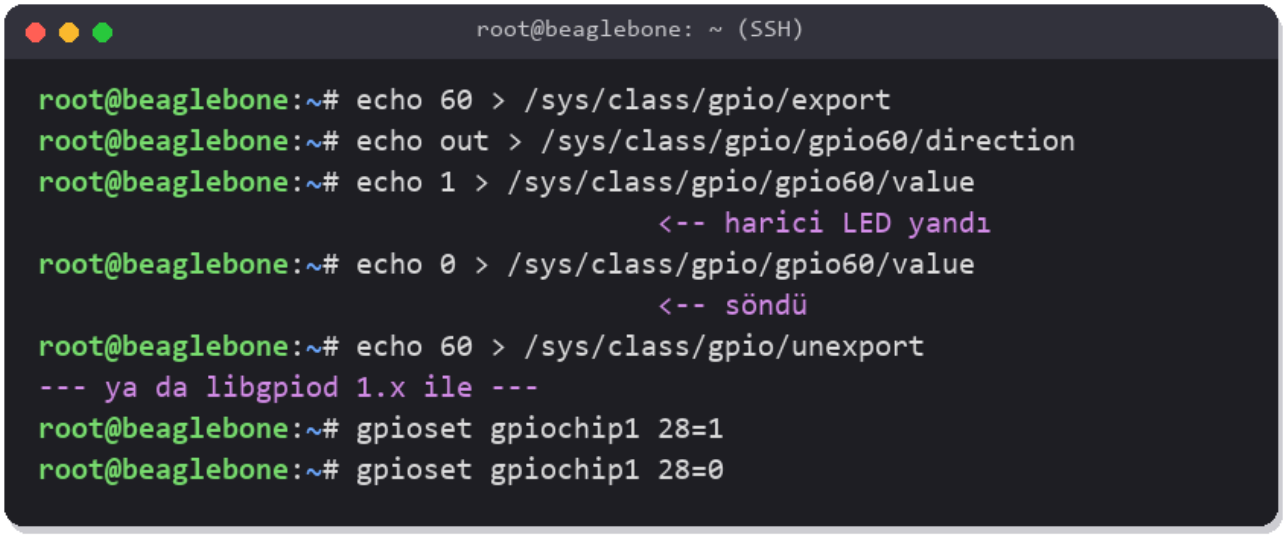
```
root@beaglebone: ~ (SSH)
root@beaglebone:~# config-pin -q P9_12
P9_12 Mode: default Direction: in Value: 0          <-- BeagleBoard Debian'da
root@beaglebone:~# config-pin P9_12 gpio
root@beaglebone:~# config-pin -q P9_12
P9_12 Mode: gpio Direction: in Value: 0
root@beaglebone:~# grep 44e10878 /sys/kernel/debug/pinctrl/44e10800.pinmux*/pins
pin 30 (PIN30) 28:gpio-32-63 44e10878 00000027 pinctrl-single  <-- mod 7 = GPIO
```

```
config-pin -q P9_12                # BeagleBoard Debian imajlarında
config-pin P9_12 gpio
grep 44e10878 /sys/kernel/debug/pinctrl/44e10800.pinmux*/pins
```

Register değerinin son hanesi `7` (mod 7) ise pin GPIO'dur. `config-pin` BeagleBoard'un Debian imajlarına özel bir araçtır; kendi rootfs'inde (Bölüm 12) yoktur, orada pinmux device tree ile yapılır.

16.3 Komut Satırından Test

Kursun "Interfacing external LED - Part 1" dersi. 15.6'daki komutlarla:




```
echo 60 > /sys/class/gpio/export
echo out > /sys/class/gpio/gpio60/direction
echo 1 > /sys/class/gpio/gpio60/value      # yanar
echo 0 > /sys/class/gpio/gpio60/value      # söner
echo 60 > /sys/class/gpio/unexport
```

LED yanmıyorsa önce devreyi kontrol et: LED'in yönü, GND bağlantısı ve direnç değeri. LED'i ters takmak bir şeye zarar vermez, sadece yanmaz.

16.4 Kursun GPIO Kütüphanesi

Kursun reposundaki `Project_Src/Drivers/gpio/` klasörü, sysfs GPIO işlemlerini fonksiyonlara saran küçük bir kütüphanedir. Sonraki bölümlerin projeleri (7-segment, LCD) de bunu kullanır.

Fonksiyon	Yaptığı iş	Karşılığı
`gpio_export(n)`	GPIO'yu kullanıcı alanına açar	`echo n > export`

Fonksiyon	Yaptığı iş	Karşılığı
`gpio_configure_dir(n, dir)`	Yön: `GPIO_DIR_OUT` / `GPIO_DIR_IN`	`echo out > direction`
`gpio_write_value(n, v)`	Çıkışı 0/1 yapar	`echo 1 > value`
`gpio_read_value(n)`	Girişi okur	`cat value`
`gpio_configure_edge(n, edge)`	Kesme kenarı: `rising`, `falling`, `both`	`echo rising > edge`
`gpio_file_open(n)` / `gpio_file_close(fd)`	`value` dosyasını `poll()` ile beklemek için açar	—

Kütüphanedeki hata

`gpio_read_value()` `value` dosyasını **sadece yazma** modunda (`O_WRONLY`) açıp ardından `read()` çağırıyor. Yazmak için açılmış bir dosyadan okunamaz; `read()` her seferinde hata verir ve fonksiyon hep `0` döndürür. Ayrıca okunan değer `'0'` / `'1'` karakteri olarak (48/49) dönecekti.

Düzeltilmiş kütüphane: `code/lib/gpio_driver.c`, `code/lib/gpio_driver.h`. Fonksiyon adları kursla aynı, sadece:

- `gpio_read_value()` dosyayı `O_RDONLY` açar ve `0` / `1` (hata durumunda `-1`) döner.
- `write()` sonuçları kontrol edilir; sona gereksiz `NUL` karakteri yazılmaz.
- GPIO'yu bırakmak için `gpio_unexport()` eklendi.

```
/* Girişi okur: 0 veya 1, hata durumunda -1 döner. */
int gpio_read_value(uint32_t gpio_num)
{
    char path[SOME_BYTES];
    char c = 0;
    ssize_t n;
    int fd;

    snprintf(path, sizeof(path), SYS_GPIO_PATH "/gpio%u/value", gpio_num);

    fd = open(path, O_RDONLY);          /* kursta: O_WRONLY */
    ...
    n = read(fd, &c, 1);
    ...
    return c == '1' ? 1 : 0;            /* kursta: ASCII karakterin kendisi */
}
```

16.5 C Programıyla LED'i Yanıp Söndürmek

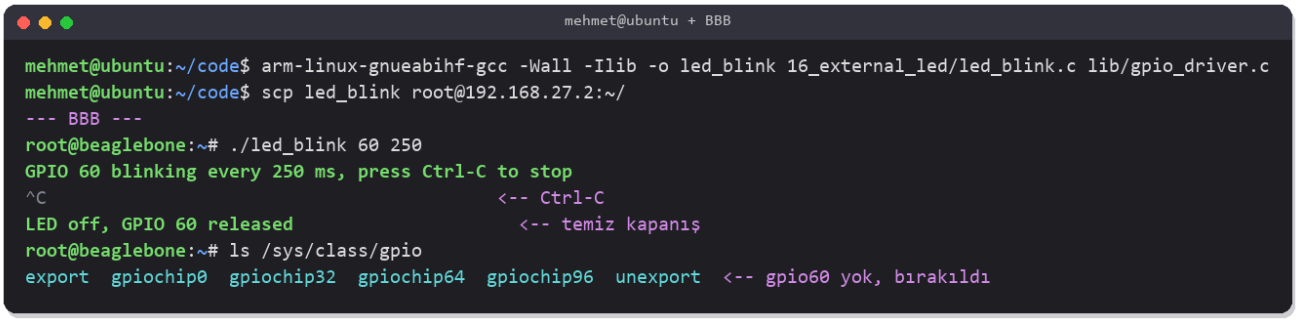
Kursun "Interfacing external LED - Part 2" dersi. `code/16_external_led/led_blink.c`, kütüphaneyi kullanarak verilen GPIO'daki LED'i yanıp söndürür ve **Ctrl-C** ile durdurulunca LED'i söndürüp GPIO'yu bırakır:

```
static volatile sig_atomic_t running = 1;

static void on_sigint(int sig)
{
    (void)sig;
    running = 0;        /* döngüye "dur" der; temizliği main yapar */
}

int main(int argc, char *argv[])
{
    ...
    gpio_export(gpio);
    gpio_configure_dir(gpio, GPIO_DIR_OUT);
    signal(SIGINT, on_sigint);

    while (running) {
        on = !on;
        gpio_write_value(gpio, on);
        usleep(half_period_ms * 1000);
    }

    gpio_write_value(gpio, GPIO_LOW_VALUE);
    gpio_unexport(gpio);
    ...
}
```

> **İpucu**
>
> `SIGINT` (Ctrl-C) yakalanmazsa program o anda ölür ve LED yanık kalabilir, GPIO da export edilmiş halde kalır. Sinyal fonksiyonu içinde sadece bir bayrağı değiştirip asıl temizliği `main`'de yapmak güvenli yöntemdir.

Derle ve çalıştır

Repodaki `code/` klasöründe:




```
arm-linux-gnueabihf-gcc -Wall -Ilib -o led_blink 16_external_led/led_blink.c lib/gpio_driver.c
scp led_blink root@192.168.27.2:~/
```

Kartta:

```
./led_blink 60 250       # 250 ms yanık, 250 ms sönük; Ctrl-C ile dur
```

Eclipse kullanıyorsan (Bölüm 14): projeye `lib/gpio_driver.c`'yi ekle, *Cross GCC Compiler → Includes*'a `lib` klasörünü yaz.

16.6 Ek: LED'i Kernel'in LED Alt Sistemine Bağlamak

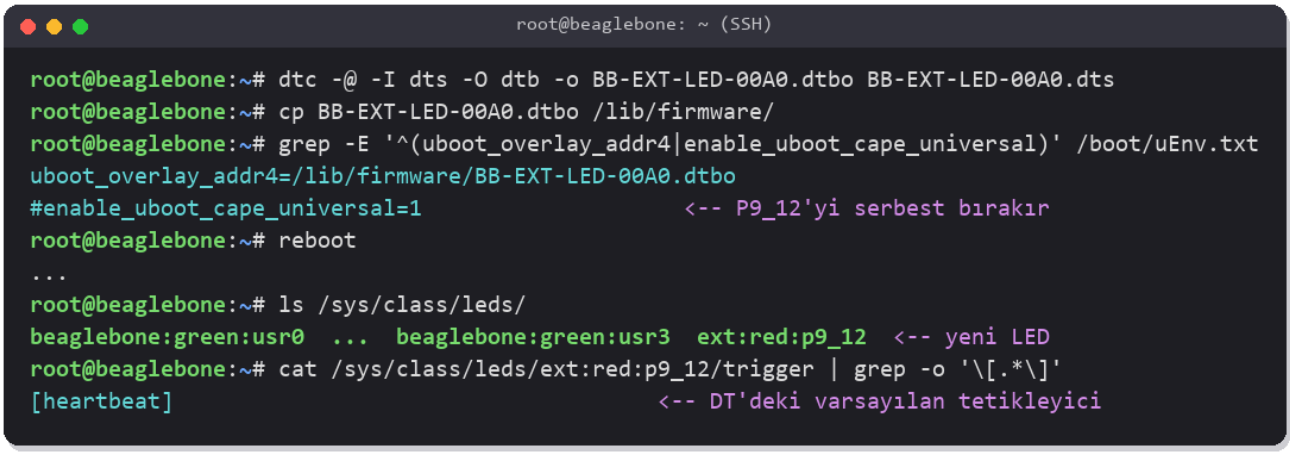
> **Not**
>
> Bu başlık kursta yok; Bölüm 11 ve Bölüm 15'i birleştiren bir uygulama. İlk okumada atlayabilirsin.

USER LED'ler `/sys/class/leds` altında görünüyor ve `heartbeat`, `timer` gibi tetikleyicilerle çalışıyordu (15.5), çünkü device tree'de `gpio-leds` sürücüsüne tanıtılmışlardı. Harici LED'imizi de aynı şekilde tanıtabiliriz. Bunun için ana DTB'yi değiştirmek yerine bir **overlay** yazarız (11.12): `code/16_external_led/BB-EXT-LED-00A0.dts`.

```
/dts-v1/;
/plugin/;

&am33xx_pinmux {
    ext_led_pins: ext-led-pins {
        pinctrl-single,pins = <
            0x078 0x07  /* P9_12: gpmc_be1n.gpio1_28, mod 7, çıkış */
        >;
    };
};

&{/} {
    ext-leds {
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&ext_led_pins>;

        ext-led {
            label = "ext:red:p9_12";
            gpios = <&gpio1 28 0>;          /* GPIO1_28, aktif yüksek */
            linux,default-trigger = "heartbeat";
            default-state = "off";
        };
    };
};
```

Parça	Anlamı
`&am33xx_pinmux { ... }`	AM335x'in pin ayar düğümüne yeni bir pin grubu ekler. `0x078`, `0x44E10878` register'ının `0x44E10800`'e göre ofseti; `0x07` değeri 15.3'teki tablodan.
`&{/} { ... }`	Kök düğüme yeni bir düğüm ekler
`compatible = "gpio-leds"`	Bu düğümü `leds-gpio` sürücüsü sürer (11.7)
`gpios = <&gpio1 28 0>`	GPIO1 bankasının 28. hattı, `0` = aktif yüksek
`linux,default-trigger`	Açılıştaki tetikleyici

Derleyip kurmak (BeagleBoard Debian imajında, kartta):




```
dtc -@ -I dts -O dtb -o BB-EXT-LED-00A0.dtbo BB-EXT-LED-00A0.dts
cp BB-EXT-LED-00A0.dtbo /lib/firmware/
nano /boot/uEnv.txt
#   uboot_overlay_addr4=/lib/firmware/BB-EXT-LED-00A0.dtbo   satırını ekle/aç
#   enable_uboot_cape_universal=1                            satırını # ile kapat
reboot
ls /sys/class/leds/
```

Açılıştan sonra LED kalp atışı gibi yanıp sönmeye başlar ve `/sys/class/leds/ext:red:p9_12/` altında USER LED'lerle aynı dosyalara sahip olur.

> **Önemli**
>
> - BeagleBoard Debian imajlarında **cape-universal** açıkken P9_12 gibi pinler `config-pin` için ayrılmış durumdadır; overlay aynı pini alamaz. Bu yüzden overlay'i kullanırken cape-universal'ı kapatıyoruz (bu, `config-pin` 'i de devre dışı bırakır).
> - Overlay, ana DTB'deki `am33xx_pinmux` ve `gpio1` etiketlerine başvurur; ana DTB'nin `-@` ile (sembollerle) derlenmiş olması gerekir. BeagleBoard'un kernel'leri böyle derler.
> - Overlay kurulduktan sonra GPIO 60'ı `/sys/class/gpio` ile export edemezsin; artık LED sürücüsüne aittir.

16.7 Özet

- LED'i GPIO'ya **seri direnç**le bağla: `R = (3.3 V − V_F) / I`, birkaç mA yeterli. Büyük yükler için transistör.
- Pinin GPIO modunda olduğunu `config-pin -q` ya da debugfs'teki `pins` dosyasıyla doğrula.
- Komut satırından: `/sys/class/gpio` veya libgpiod. C'den: kursun GPIO kütüphanesi (düzeltilmiş haliyle `code/lib/`).
- Program sonlanırken LED'i söndürüp GPIO'yu bırakmak için `SIGINT` 'i yakala.
- LED'i device tree overlay ile `gpio-leds` 'e bağlarsan kernel'in LED tetikleyicileri de kullanılabilir.

16.8 Karşılaşılabilecek Problemler

```
root@beaglebone: ~ (SSH) (temsili hatalar)
LED hiç yanmıyor, value=1                <-- LED ters takılı / GND bağlı değil / pinmux

root@beaglebone:~# echo 60 > /sys/class/gpio/export
bash: echo: write error: Device or resource busy   <-- pin overlay'de / başka sürücüde

root@beaglebone:~# ./led_blink 60
/sys/class/gpio/gpio60/direction: Permission denied   <-- root olarak çalıştır

root@beaglebone:~# config-pin P9_12 gpio
ERROR: open() for /sys/devices/platform/ocp/ocp:P9_12_pinmux/state failed
                    <-- cape-universal kapalı / overlay pini aldı
```

Belirti	Sebep	Çözüm
`value` 1 ama LED yanmıyor	LED ters, GND bağlı değil, yanlış header pini ya da pin GPIO modunda değil	Bacakları kontrol et; multimetreyle pinde 3.3 V ölç; 16.2
LED çok soluk	Direnç büyük ya da LED mavi/beyaz (V_F ~3 V)	470 Ω kullan; mavi/beyaz LED'i transistörle sür
`export`: `Device or resource busy`	Pin bir overlay'de ya da başka sürücüde	`cat /sys/kernel/debug/gpio` ile kimin kullandığına bak
`Permission denied`	Program root olarak çalışmıyor	`sudo ./led_blink 60` ya da root olarak çalıştır
Ctrl-C sonrası LED yanık kaldı / sonraki çalıştırmada `export` hatası	Program sinyali yakalamadan sonlandı, GPIO export edilmiş kaldı	`echo 60 > /sys/class/gpio/unexport`; kendi programında `SIGINT`'i yakala
`config-pin: ... P9_12_pinmux/state failed`	cape-universal kapalı ya da pin bir overlay'de	Overlay kullanıyorsan beklenen durum; değilse `/boot/uEnv.txt`'de cape-universal'ı aç
`gpio_read_value()` hep 0 döndürüyor (kursun kodu)	Dosya `O_WRONLY` açılıp okunuyor	16.4'teki düzeltilmiş kütüphane

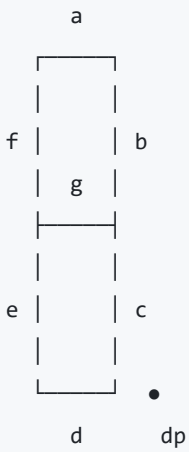
Bölüm 17 — 7-Segment Gösterge

> **Bu bölümde:**
>
> - 7-segment göstergenin yapısını, ortak katot ve ortak anot farkını öğreneceksin.
> - Tek haneli bir göstergeyi sekiz GPIO pinine dirençlerle bağlayacaksın.
> - Segmentleri komut satırından tek tek yakıp devreyi test edeceksin.
> - Rakamları bir segment tablosuyla kodlayan, yukarı/aşağı/yukarı-aşağı/rastgele sayan bir C programı yazacaksın.
> - Ek: Butonun her basışında sayıyı artıran, kesme (edge) ile çalışan bir program yazacaksın.
>
> **Ön koşul:** Bölüm 15 — GPIO numaraları ve sysfs, Bölüm 16 — LED bağlama ve kursun GPIO kütüphanesi.

> **Not**
>
> Bu bölümdeki komutlar kartta **root** olarak çalıştırılır. Kart BeagleBoard Debian imajı ve kernel 5.10 ile çalışıyor; GPIO numaraları bu kernel'e göre. Terminal görüntüleri temsilidir.

17.1 7-Segment Gösterge

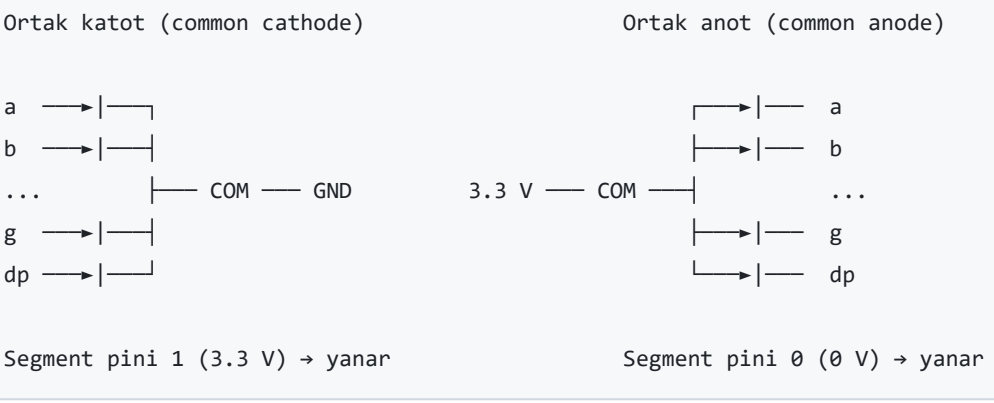
7-segment gösterge, tek bir kılıfa konmuş **8 LED**'dir: rakamı çizen 7 çubuk (segment) ve bir nokta (dp, decimal point). Segmentler `a`'dan `g`'ye harflerle adlandırılır:



Hangi segmentlerin yandığı hangi rakamın göründüğünü belirler. Örneğin `1` için sadece `b` ve `c`, `7` için `a`, `b` ve `c` yanar.

Ortak katot ve ortak anot

Sekiz LED'in birer bacağı içeride birleştirilip tek bir **COM** (ortak) pinine bağlanmıştır. Hangi bacağın birleştirildiğine göre iki tür gösterge vardır:

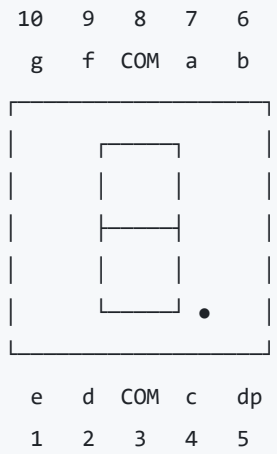


	Ortak katot	**Ortak anot**
COM bağlantısı	GND	3.3 V
Segmenti yakmak	GPIO = **1** (aktif yüksek)	GPIO = **0** (aktif düşük)
Örnek parça	5161AS	5161BS

Kursun sayaç programı **ortak katot** gösterge kullanır. Elindeki göstergenin türünü parça numarasından ya da datasheet'inden öğren; emin değilsen COM'u GND'ye bağla, bir segment pinini dirençle 3.3 V'a değdir. Yanıyorsa ortak katottur.

Pin dizilimi

Yaygın 10 pinli tek haneli göstergelerde (0.56", 5161 serisi) pinler şöyledir. Göstergeye önden bakıldığında pin 1 sol alttadır:



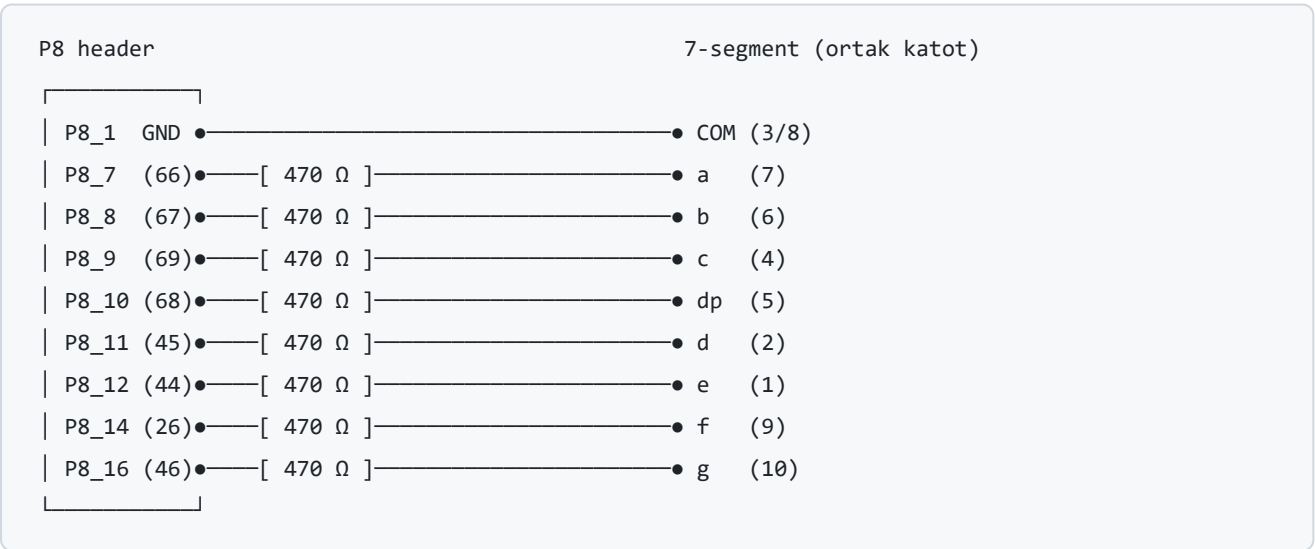
İki COM pini içeride birbirine bağlıdır; birini bağlamak yeter.

> **İpucu**
>
> Farklı üreticilerin dizilimi farklı olabilir. Datasheet'in yoksa COM'u bağlayıp diğer pinleri dirençle tek tek dene; her pinin hangi segmenti yaktığını not al.

17.2 Devre

Kursun bağlantısı: segmentler P8 header'ındaki sekiz GPIO pinine bağlanır. Bu pinler varsayılan olarak GPIO modundadır ve eMMC/HDMI tarafından kullanılmaz (15.2).

P8 pini	AM335x	GPIO numarası	Segment	Gösterge pini (5161)
P8_7	GPIO2_2	66	a	7
P8_8	GPIO2_3	67	b	6
P8_9	GPIO2_5	69	c	4
P8_10	GPIO2_4	68	dp	5
P8_11	GPIO1_13	45	d	2
P8_12	GPIO1_12	44	e	1
P8_14	GPIO0_26	26	f	9
P8_16	GPIO1_14	46	g	10
P8_1 / P8_2	GND	—	COM (ortak katot)	3 veya 8



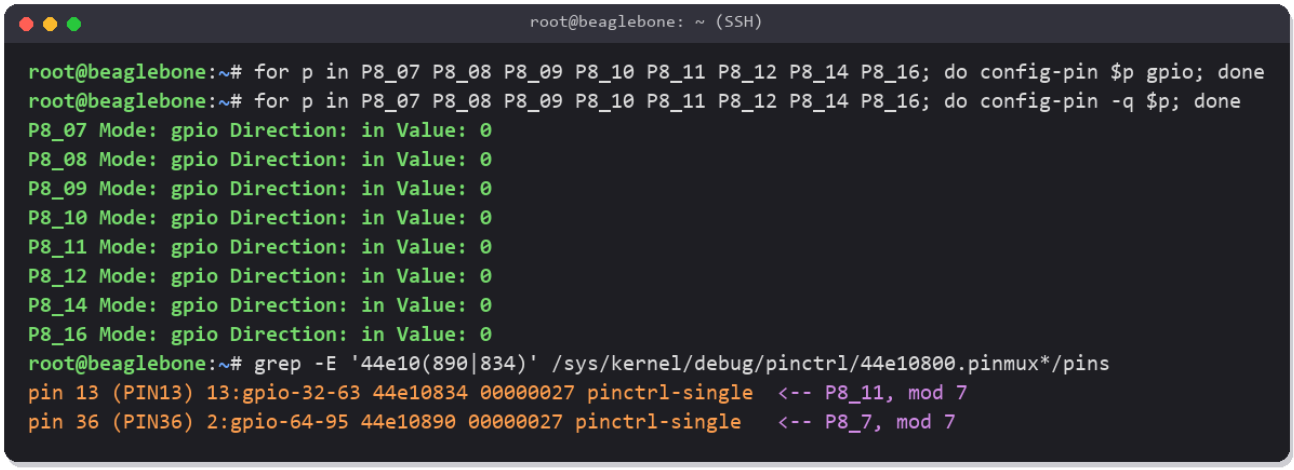
Her segment ayrı bir LED olduğu için **her segmente ayrı bir direnç** konur. Direnç hesabı 16.1'dekiyle aynı: 470 Ω ile kırmızı bir segmentten ~3 mA akar.

> **Önemli**
>
> Tek bir direnci COM hattına koymak cazip gelebilir ama yanlıştır: o zaman bütün segmentler aynı akımı paylaşır. `1` (2 segment) parlak, `8` (7 segment) soluk görünür.

> **Uyarı**
>
> - Kabloları kart **kapalıyken** bağla.
> - Ortak anot gösterge kullanıyorsan COM'u **3.3 V**'a (P9_3/P9_4) bağla, asla 5 V'a bağlama. COM 5 V'ta olursa GPIO pini 3.3 V'ta iken bile segmentten pine akım akar, segment sönmez ve pin zarar görebilir.

17.3 Pinlerin GPIO Modunda Olduğunu Doğrula

Kursun "7-Segment Display basics and Interfacing" dersleri. Pinleri bağlamadan önce sekizinin de GPIO modunda olduğunu kontrol et (16.2):




```
for p in P8_07 P8_08 P8_09 P8_10 P8_11 P8_12 P8_14 P8_16; do config-pin $p gpio; done
for p in P8_07 P8_08 P8_09 P8_10 P8_11 P8_12 P8_14 P8_16; do config-pin -q $p; done
grep -E '44e10(890|834)' /sys/kernel/debug/pinctrl/44e10800.pinmux*/pins
```

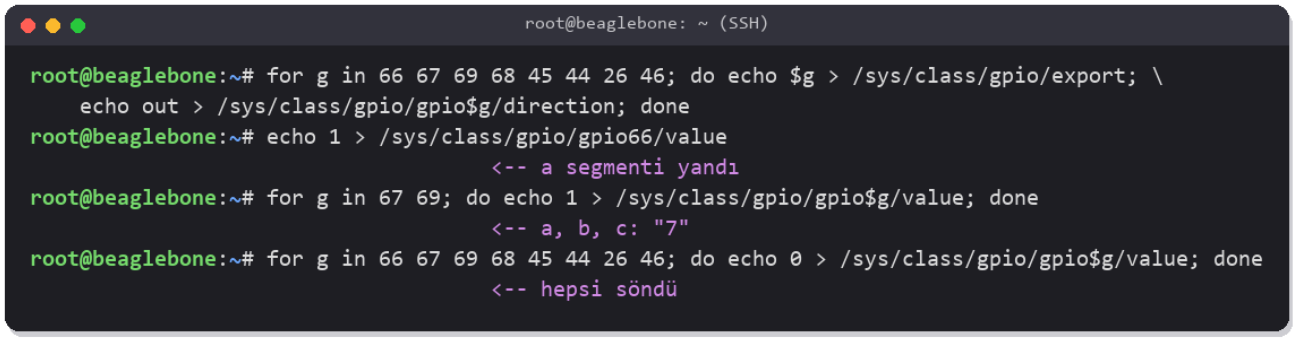
> **Not**
>
> Güncel `config-pin` P8'in tek haneli pinlerini başında sıfırla ister: `P8_7` değil `P8_07`. Eski sürümler iki yazımı da kabul eder.

Pin ayar register'larının adresleri (15.3):

Pin	Register	Pin	Register
P8_7	`0x44E10890`	P8_11	`0x44E10834`
P8_8	`0x44E10894`	P8_12	`0x44E10830`
P8_9	`0x44E1089C`	P8_14	`0x44E10828`
P8_10	`0x44E10898`	P8_16	`0x44E10838`

17.4 Komut Satırından Segment Testi

Program yazmadan önce her segmentin doğru pine bağlandığını görmek için segmentleri tek tek yak:




```
for g in 66 67 69 68 45 44 26 46; do
    echo $g > /sys/class/gpio/export
    echo out > /sys/class/gpio/gpio$g/direction
done

echo 1 > /sys/class/gpio/gpio66/value                        # a
for g in 67 69; do echo 1 > /sys/class/gpio/gpio$g/value; done   # b, c → "7"

for g in 66 67 69 68 45 44 26 46; do echo 0 > /sys/class/gpio/gpio$g/value; done
for g in 66 67 69 68 45 44 26 46; do echo $g > /sys/class/gpio/unexport; done
```

Beklemediğin bir segment yanıyorsa o pinin kablosu yer değiştirmiştir. Hiçbiri yanmıyorsa COM bağlantısına ve gösterge türüne bak (17.1).

17.5 Rakamları Kodlamak: Segment Tablosu

Bir rakamı göstermek, 7 segmentin her birini açıp kapatmaktır. Her segmente bir bit verirsek bir rakam tek bir bayt olur:

```
  bit:     6   5   4   3   2   1   0
  segment: g   f   e   d   c   b   a
```

Rakam	Yanan segmentler	g f e d c b a	Hex
0	a b c d e f	0 1 1 1 1 1 1	`0x3F`
1	b c	0 0 0 0 1 1 0	`0x06`
2	a b d e g	1 0 1 1 0 1 1	`0x5B`
3	a b c d g	1 0 0 1 1 1 1	`0x4F`
4	b c f g	1 1 0 0 1 1 0	`0x66`
5	a c d f g	1 1 0 1 1 0 1	`0x6D`
6	a c d e f g	1 1 1 1 1 0 1	`0x7D`
7	a b c	0 0 0 0 1 1 1	`0x07`
8	a b c d e f g	1 1 1 1 1 1 1	`0x7F`
9	a b c d f g	1 1 0 1 1 1 1	`0x6F`

Kursun programı her rakam için 7 satırlık bir `case` yazar (`Write_number_to_7segdisplay()`, 10 rakam × 7 satır). Tablo kullanınca aynı iş birkaç satıra iner ve bir rakamın şeklini değiştirmek tek bir sayıyı değiştirmek olur. Bu bölümün kodunda göstergeyi süren kısım ayrı bir dosyada: `code/17_7segment/seg7.c`, `code/17_7segment/seg7.h`.

```
/* Segment sırası: a, b, c, d, e, f, g */
static const uint32_t seg_gpio[7] = { 66, 67, 69, 45, 44, 26, 46 };

static const uint8_t digit_segments[11] = {
    0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F,
    0x00,   /* SEG7_BLANK: hepsi sönük */
};

uint8_t seg7_pattern(uint8_t digit)
{
    return digit_segments[digit <= 9 ? digit : SEG7_BLANK];
}

void seg7_show(uint8_t digit)
{
    uint8_t bits = seg7_pattern(digit);
    int i;

    for (i = 0; i < 7; i++)
        gpio_write_value(seg_gpio[i], (bits >> i) & 1 ? SEGMENT_ON : SEGMENT_OFF);
}
```

`SEGMENT_ON` ve `SEGMENT_OFF`, gösterge türüne göre derleme sırasında seçilir:

```
#ifdef COMMON_ANODE
#define SEGMENT_ON          LOW_VALUE
#define SEGMENT_OFF         HIGH_VALUE
#else
#define SEGMENT_ON          HIGH_VALUE      /* ortak katot (varsayılan) */
#define SEGMENT_OFF         LOW_VALUE
#endif
```

Fonksiyon	Yaptığı iş
`seg7_init()`	8 GPIO'yu export eder, çıkış yapar, segmentleri söndürür
`seg7_show(n)`	`0` – `9` arası rakamı gösterir; `SEG7_BLANK` göstergeyi söndürür
`seg7_pattern(n)`	Rakamın segment desenini (bayt) döndürür; Bölüm 18'deki 4 haneli gösterge de bunu kullanır
`seg7_dp(on)`	Noktayı yakar/söndürür
`seg7_release()`	Göstergeyi söndürür ve GPIO'ları bırakır

`seg7.c`, GPIO işlemleri için 16.4'teki düzeltilmiş kütüphaneyi (`code/lib/`) kullanır. Sonraki bölümlerde de aynı dosyalar kullanılacak.

17.6 Kursun Sayaç Programı ve Düzeltmeler

Kursun "7segment display up-down counter implementation" dersleri `Project_Src/counter_7seg/src/counter_7seg.c` programını yazar. Program iki argüman alır: sayma yönü ve iki rakam arasındaki bekleme süresi (ms).

Mod	Davranış
`up`	0, 1, … 9, 0, 1, …
`down`	9, 8, … 0, 9, 8, …
`updown`	0, 1, … 9, 8, … 1, 0, 1, …
`random`	Rastgele rakamlar

Kursun kodunda `updown` ve `random` modları öğrenciye bırakılmış (`TODOs for the students`), fonksiyonlar sadece bir mesaj yazdırıyor. Kodda ayrıca şu sorunlar var:

Sorun	Sonucu	Düzeltme
GPIO'lar export edilmiyor	Program, pinlerin önceden elle export edilmiş olmasını bekliyor; değilse her `open()` hata verir ama program yine de "sayıyor"	`seg7_init()` export eder

Sorun	Sonucu	Düzeltme
`initialize_all_gpios()` hataları kontrol etmiyor, her zaman `0` döndürüyor (kodda `TODO`)	Pin export edilmemişse ya da root değilsen hata görmezsin, gösterge boş kalır	Her adımın sonucu kontrol edilir
Program sonsuz döngüde, Ctrl-C ile öldürülüyor	Gösterge son rakamda yanık kalır, GPIO'lar export edilmiş kalır	`SIGINT` yakalanır, gösterge söndürülüp GPIO'lar bırakılır (16.5)
`write(fd, "1", 2)`	Sona fazladan `NUL` karakteri yazılıyor	Düzeltilmiş kütüphane (16.4)

Çözüm: updown ve random

Düzeltilmiş program: `code/17_7segment/counter_7seg.c`. `updown` modu bir adım değişkeni tutar ve uçlarda yönünü değiştirir; böylece 9 ve 0 iki kez gösterilmez:

```
static void start_updowncounting(unsigned int delay_ms)
{
    int i = 0, step = 1;

    while (running) {
        show_and_wait(i, delay_ms);
        if (i == 9)
            step = -1;
        else if (i == 0)
            step = 1;
        i += step;
    }
}
```

`random` modu `rand() % 10` ile rakam seçer. Aynı rakam art arda gelirse gösterge değişmez ve program donmuş gibi görünür; bu yüzden bir önceki rakam tekrar seçilmez:

```
static void start_randomcounting(unsigned int delay_ms)
{
    int last = -1, n;

    srand((unsigned int)time(NULL));    /* her çalıştırmada farklı dizi */

    while (running) {
        do
            n = rand() % 10;
        while (n == last);
        last = n;
        show_and_wait(n, delay_ms);
    }
}
```

> **İpucu**
>
> `srand()` çağrılmazsa `rand()` her çalıştırmada **aynı** diziyi üretir. Dizi "rastgele" görünür ama programı her açışında aynı rakamlar aynı sırayla gelir.

Derle ve çalıştır

Repodaki `code/` klasöründe:

```
arm-linux-gnueabihf-gcc -Wall -Ilib -o counter_7seg 17_7segment/counter_7seg.c \
      17_7segment/seg7.c lib/gpio_driver.c
scp counter_7seg root@192.168.27.2:~/
```

Ortak anot gösterge kullanıyorsan derleme komutuna `-DCOMMON_ANODE` ekle.

Kartta:

```
./counter_7seg up 500         # yarım saniyede bir artar
./counter_7seg down 500
./counter_7seg updown 300
./counter_7seg random 1000
```

Eclipse kullanıyorsan (Bölüm 14): projeye `seg7.c` ve `lib/gpio_driver.c`'yi ekle, *Cross GCC Compiler → Includes*'a `lib` klasörünü yaz, ortak anot için *Preprocessor → Defined symbols*'a `COMMON_ANODE` ekle.

17.7 Ek: Butonla Sayma

> **Not**
>
> Kursun reposunda `Project_Src/button_counter_7seg/` olarak bulunan program. Göstergeye bir buton ekler: her basışta sayı bir artar. Buton girişini **kesme** (edge) ile beklemeyi gösterir.

Buton devresi

Buton basılı değilken pin, **pull-down** direnci sayesinde 0'da kalır; basılınca 3.3 V'a bağlanır ve 1 olur. Pull-down olmazsa pin **boşta** (floating) kalır: elini yaklaştırmak bile 0/1 arasında rastgele değişime ve sahte basışlara yol açar.

AM335x'in her pininde dahili pull-up/pull-down vardır (15.3). BeagleBoard Debian'da `config-pin P9_23 gpio_pd` pini GPIO yapıp dahili pull-down'ı açar. Harici 10 kΩ direnç yine de önerilir; dahili direnç zayıftır ve pinmux yanlışsa iş görmez.

Kesme ile beklemek: poll()

Butonu bir döngüde sürekli okumak (polling) işlemciyi boşuna meşgul eder. sysfs bunun yerine kesmeyi kullanmana izin verir:

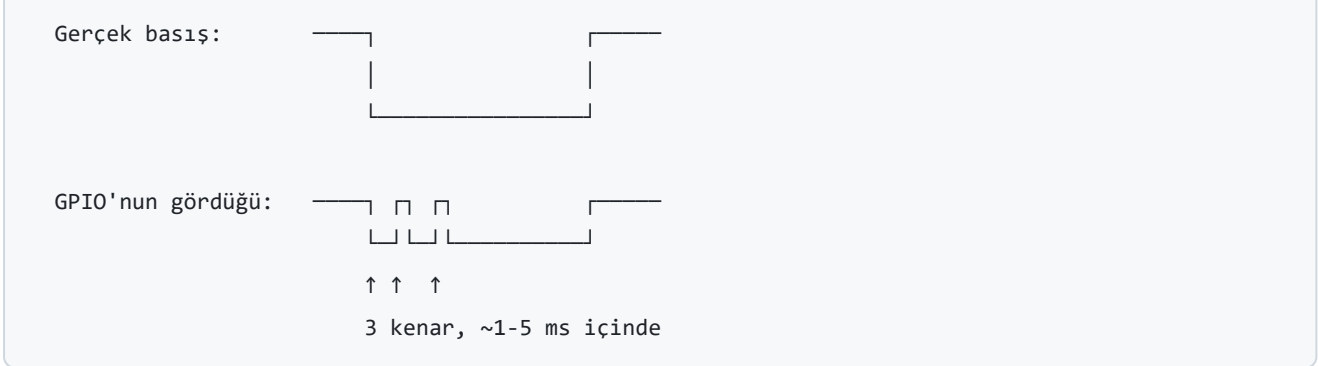
1. `edge` dosyasına `rising` yaz: yükselen kenarda (0 → 1, basış anı) kernel bir olay üretir.

2. `value` dosyasını aç ve `poll()` ile `POLLPRI` olayını bekle. Program olay gelene kadar uyur, CPU harcamaz.
3. Olay gelince `value`'yu **başa sarıp oku** (`lseek` + `read`). Okumazsan olay temizlenmez ve `poll()` hemen tekrar döner.

```
gpio_configure_dir(BUTTON_GPIO, GPIO_DIR_IN);
gpio_configure_edge(BUTTON_GPIO, "rising");
fd = gpio_file_open(BUTTON_GPIO);         /* value, O_RDONLY | O_NONBLOCK */

read_value_fd(fd);                        /* açılıştaki bekleyen olayı temizle */

while (running) {
    pfd.fd = fd;
    pfd.events = POLLPRI | POLLERR;
    pfd.revents = 0;

    rc = poll(&pfd, 1, -1);               /* basılana kadar uyu */
    ...
    if (read_value_fd(fd) != '1')         /* olayı temizle, hâlâ basılı mı? */
        continue;

    if (now_ms() - last_press < DEBOUNCE_MS)
        continue;                         /* kontak sekmesi */
    last_press = now_ms();

    counter = (counter + 1) % 10;
    seg7_show(counter);
}
```

Kontak sekmesi (debounce)

Mekanik bir butonun metal kontakları basıldığı anda birkaç milisaniye boyunca birbirine çarpıp ayrılır. GPIO bunu art arda birkaç yükselen kenar olarak görür:



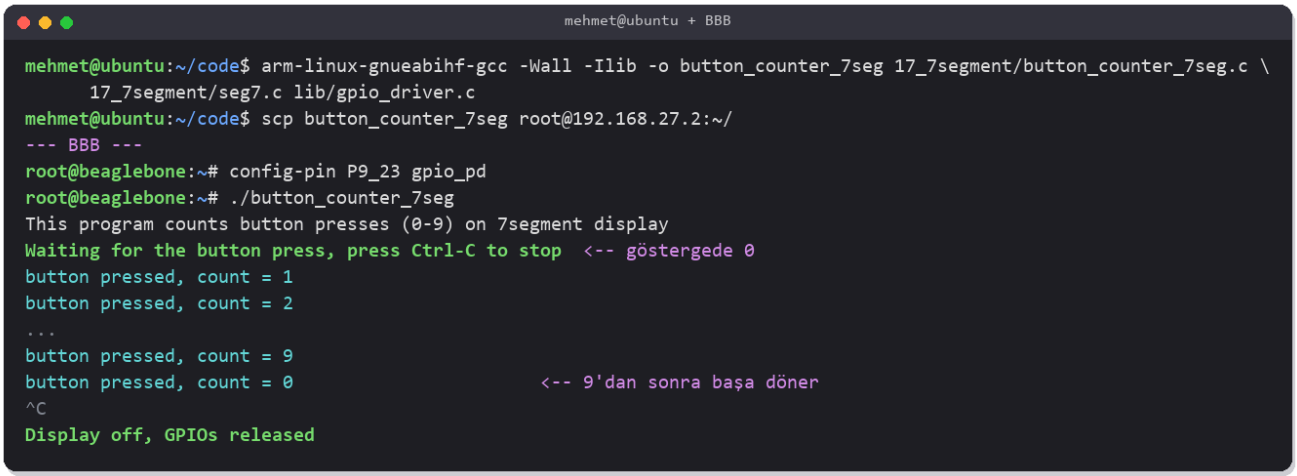
Önlem alınmazsa tek basış sayıyı 2–3 artırır. Programda bir basıştan sonraki 200 ms içinde gelen kenarlar sayılmaz (yazılımla debounce). Donanımla çözmek de mümkündür: butona paralel ~100 nF kondansatör.

Kursun kodundaki hatalar

`code/17_7segment/button_counter_7seg.c` bunları düzeltir:

Sorun	Sonucu
Sayaç (`uint8_t led_counter`) ilk değer verilmeden kullanılıyor	Başlangıç değeri belirsiz; ilk basışta herhangi bir şey görünebilir
Sayaç 9'dan sonra başa dönmüyor	10'da gösterge söner (`case 10`), 11'den sonra `switch` 'te karşılığı yok: gösterge 245 basış boyunca boş kalır
İlk `poll()` 'dan önce `value` okunmuyor	sysfs, dosya açılınca bir olay bekler halde başlar; ilk `poll()` hemen döner ve basılmadan bir sayılır
Debounce yok	Tek basış birkaç kez sayılır
`initialize_button_gpio()` buton yerine `GPIO_68` 'e (nokta segmenti) yazıyor	Kopyala-yapıştır hatası; butona `value` yazmak zaten anlamsız (pin giriş)
`SEGMENT_ON` = `LOW` (sayaç programında `HIGH`)	Aynı gösterge ile iki program birbirinin tersini gösterir: biri ortak katot, öbürü ortak anot varsayar. Bizim kodda tür tek yerden seçilir (17.5)
`pollfd` dizisinin ilk elemanı boş (`fd = 0` , stdin)	Zararsız ama gereksiz; tek eleman yeterli

Derle ve çalıştır




```
arm-linux-gnueabihf-gcc -Wall -Ilib -o button_counter_7seg 17_7segment/button_counter_7seg.c \
      17_7segment/seg7.c lib/gpio_driver.c
scp button_counter_7seg root@192.168.27.2:~/
```

Kartta:

```
config-pin P9_23 gpio_pd
./button_counter_7seg
```

Program çalışırken `top`'a bakarsan CPU kullanımının ~%0 olduğunu görürsün: `poll()` beklerken süreç uyur.

> **Not**
>
> Aynı işi libgpiod ile de yapabilirsin (15.6): `gpiomon` komutu ya da kütüphanenin olay fonksiyonları. libgpiod, kernel'de debounce süresini doğrudan ayarlama seçeneği de sunar; yazılım tarafında zaman tutmaya gerek kalmaz.

17.8 Özet

- 7-segment gösterge 8 LED'dir (a–g ve dp). **Ortak katot**: COM GND'ye, segment 1 ile yanar. **Ortak anot**: COM 3.3 V'a, segment 0 ile yanar.
- Her segmente ayrı direnç (470 Ω) koy; COM'a tek direnç parlaklığı rakama göre değiştirir.
- Kursun bağlantısı: a–g ve dp → P8_7, P8_8, P8_9, P8_11, P8_12, P8_14, P8_16 ve P8_10 (GPIO 66, 67, 69, 45, 44, 26, 46, 68).
- Bir rakam bir bayttır (bit 0 = a … bit 6 = g); 10 rakamlık tablo 70 satırlık `switch`'in yerini alır.
- `updown` modu uçlarda yön değiştiren bir adım değişkeni, `random` modu `srand()` + `rand() % 10` ile yazılır.
- Butonu `edge` + `poll()` ile bekle; olaydan sonra `value`'yu başa sarıp oku, ilk `poll()`'dan önce de bir kez oku. Mekanik buton için debounce şart.

17.9 Karşılaşılabilecek Problemler

```
root@beaglebone: ~ (SSH) (temsili hatalar)
Rakamlar ters: "8" sönük, boşlukta segmentler yanık  <-- ortak anot gösterge
                  -DCOMMON_ANODE ile derle, COM'u 3.3 V'a bağla

debian@beaglebone:~$ ./counter_7seg up 500
/sys/class/gpio/gpio66/direction: Permission denied   <-- root olarak çalıştır
GPIO 66 could not be configured as output
Error: GPIO init failed !

root@beaglebone:~# ./button_counter_7seg
button pressed, count = 1
button pressed, count = 2      <-- hiç basılmadı: pin boşta (pull-down yok)
button pressed, count = 3
```

Belirti	Sebep	Çözüm
Rakamlar ters: `8` gösterilirken her şey sönük, boşken her şey yanık	Ortak anot gösterge, program ortak katot için derlenmiş (ya da tersi)	`-DCOMMON_ANODE` ile derle; COM'u 3.3 V'a bağla

Belirti	Sebep	Çözüm
Hiçbir segment yanmıyor	COM bağlı değil ya da yanlış yerde; pinler GPIO modunda değil	COM'u kontrol et; 17.3, 17.4
Rakamlar bozuk görünüyor (ör. `2` yerine anlamsız bir şekil)	İki segmentin kablosu yer değiştirmiş	17.4'teki gibi segmentleri tek tek yak
Bazı rakamlar soluk, bazıları parlak	Direnç segmentlerde değil COM hattında	Her segmente ayrı direnç
`Permission denied` / `GPIO init failed`	Program root olarak çalışmıyor	`sudo ./counter_7seg up 500`
`export`: `Device or resource busy`	Pin bir overlay'de ya da başka sürücüde	`cat /sys/kernel/debug/gpio` ile kimin kullandığına bak
`config-pin P8_7 gpio` hata veriyor	Güncel `config-pin` sıfırlı yazım istiyor	`P8_07` yaz
Butona basmadan sayı artıyor	Pull-down yok, pin boşta	10 kΩ pull-down ekle, `config-pin P9_23 gpio_pd`
Tek basışta sayı 2–3 artıyor	Kontak sekmesi	Yazılımla debounce (17.7) ya da 100 nF kondansatör
Program başlar başlamaz bir basış sayıyor (kursun kodu)	İlk `poll()`'dan önce `value` okunmamış	`poll()` döngüsünden önce `lseek` + `read`
`poll()` sürekli dönüyor, CPU %100	Olaydan sonra `value` okunmuyor	Her olaydan sonra `lseek(fd, 0, SEEK_SET)` + `read()`
GPIO numaraları yeni kernel'de çalışmıyor	Kernel 6.x'te GPIO numaraları 512'den başlayabilir	15.6'daki uyarı; `ls /sys/class/gpio/` ile bankaların ilk numarasını bul

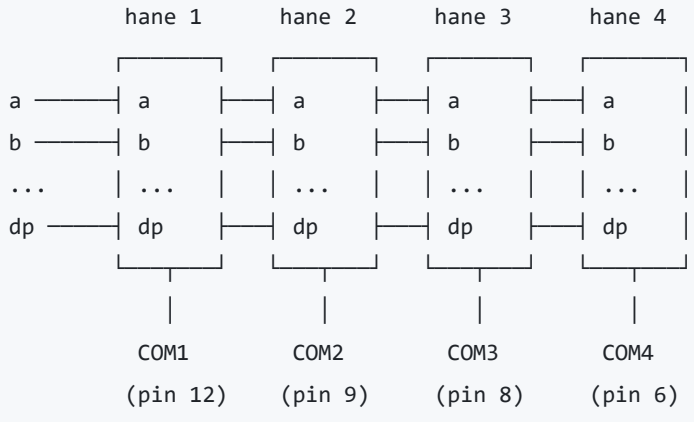
Bölüm 18 — 4 Haneli 7-Segment Gösterge

> **Bu bölümde:**
>
> - 4 haneli bir göstergenin segmentleri neden ortak kullandığını ve çoğullamanın (multiplexing) nasıl çalıştığını öğreneceksin.
> - Göstergeyi 12 GPIO pinine bağlayacak, hane pinlerinin akım sınırını göreceksin.
> - Göstergeyi kullanıcı alanından çoğullayan bir C modülü yazacaksın.
> - 0000–9999 sayan bir sayaç ve kursun ödevi olan dijital saati yazacaksın.
> - Kullanıcı alanında çoğullamanın sınırlarını göreceksin.
>
> **Ön koşul:** Bölüm 17 — tek haneli 7-segment gösterge, segment tablosu ve `seg7.c`.

> **Not**
>
> Bu bölümdeki komutlar kartta **root** olarak çalıştırılır. Kart BeagleBoard Debian imajı ve kernel 5.10 ile çalışıyor; GPIO numaraları bu kernel'e göre. Terminal görüntüleri temsilidir.

18.1 4 Haneli Göstergenin Yapısı

4 haneli bir göstergede 4 × 8 = 32 LED vardır. Her LED'e ayrı pin verilseydi 32 segment pini ve COM pinleri gerekirdi. Bunun yerine üretici, dört hanenin **aynı harfli segmentlerini** içeride birbirine bağlar: dört hanenin `a` segmentleri tek bir `a` pinine çıkar, `b`'ler tek bir `b` pinine, ve böyle devam eder. Her hanenin sadece ortak pini (COM) ayrı kalır:



Böylece 8 segment pini + 4 hane pini = **12 pin** yeter. Bedeli şudur: segment pinlerine bir desen yazıp iki haneyi birden açarsan iki hanede de **aynı rakam** görünür. Farklı hanelerde farklı rakam göstermenin tek yolu, haneleri sırayla açmaktır (18.2).

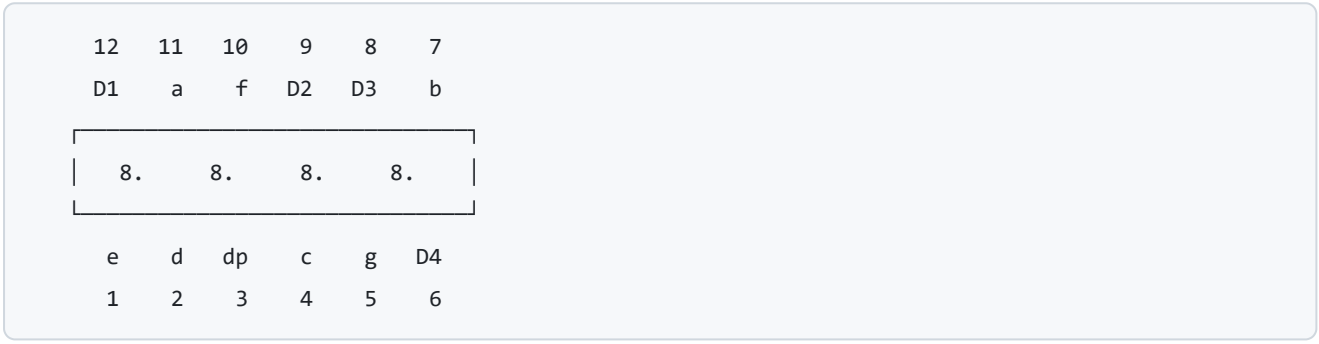
Hane pinleri 17.1'deki ortak katot/anot mantığıyla çalışır:

	Segment yakmak	**Haneyi açmak**
Ortak anot (ör. 5641BS)	segment pini **0**	hane pini **1** (3.3 V)
Ortak katot (ör. 5641AS)	segment pini **1**	hane pini **0** (0 V)

Kursun programı **ortak anot** gösterge kullanır.

Pin dizilimi

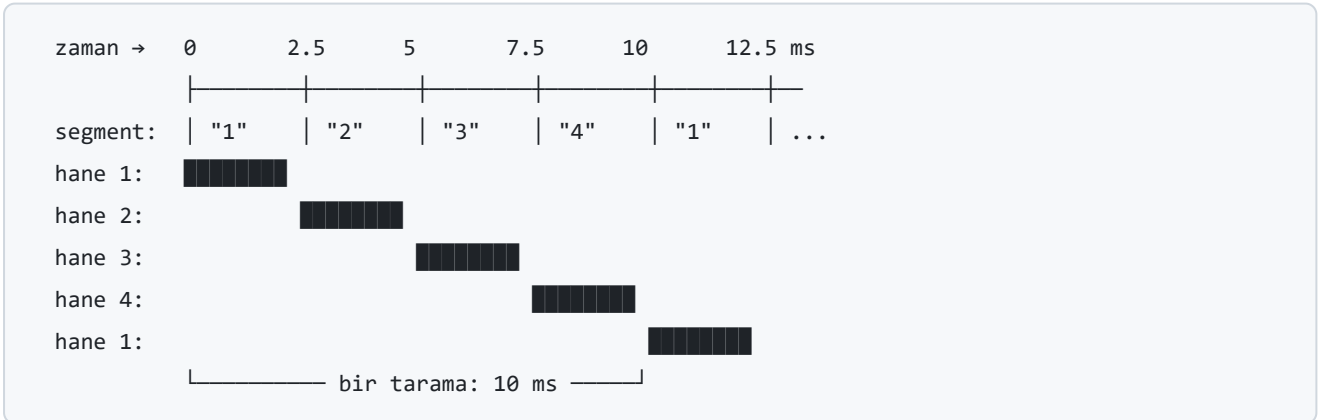
Yaygın 12 pinli 0.56" göstergelerde (5641 serisi) pinler şöyledir. Göstergeye önden bakıldığında pin 1 sol alttadır:



`D1` – `D4` hanelerin ortak pinleridir; `D1` en soldaki hane. Farklı üreticilerin dizilimi farklı olabilir; datasheet'e bak ya da 17.1'deki gibi pinleri tek tek dene.

18.2 Çoğullama (Multiplexing)

Çoğullama, aynı anda sadece **bir** haneyi açıp ona ait rakamı göstermek ve bunu dört hane için hızla tekrarlamaktır:



Göz, saniyede ~50 kereden hızlı yanıp sönen bir ışığı sürekli yanıyor algılar (görmenin sürekliliği). Burada her hane her 10 ms'de bir yanar, yani tarama frekansı 100 Hz'dir; dört hane aynı anda yanıyor görünür.

Hesap:

```
hane başına süre × hane sayısı = tarama periyodu
     2.5 ms     ×      4      =     10 ms    →   1 / 10 ms = 100 Hz
```

Tarama yavaşlarsa (ör. hane başına 10 ms → 25 Hz) titreme görünür; daha da yavaşlarsa rakamların sırayla yandığını gözle görürsün.

> **Önemli**
>
> Gösterge sadece **program onu taradığı sürece** yanar. Program uyursa ya da başka bir işle meşgul olursa ekranda tek bir hane kalır ya da gösterge söner. Bu yüzden sayaç programı sayıyı değiştirmek için `sleep()` ile beklemez; sürekli tarar ve sırası gelince sayıyı değiştirir (18.7).

Her hane zamanın sadece dörtte birinde yanık olduğu için aynı dirençle tek haneli göstergeden daha **soluk** görünür.

18.3 Devre

Kursun bağlantısı: segmentler Bölüm 17'deki P8 pinlerinde kalır, dört hane pini P9'a bağlanır.

Pin	AM335x	GPIO	Gösterge pini		Pin	AM335x	GPIO	Gösterge pini
P8_7	GPIO2_2	66	11 (a)		P9_15	GPIO1_16	48	12 (D1)
P8_8	GPIO2_3	67	7 (b)		P9_23	GPIO1_17	49	9 (D2)
P8_9	GPIO2_5	69	4 (c)		P9_30	GPIO3_16	112	8 (D3)
P8_11	GPIO1_13	45	2 (d)		P9_27	GPIO3_19	115	6 (D4)
P8_12	GPIO1_12	44	1 (e)					
P8_14	GPIO0_26	26	10 (f)					
P8_16	GPIO1_14	46	5 (g)					
P8_10	GPIO2_4	68	3 (dp)					

Segment pinlerine 17.2'deki gibi birer direnç konur. Hane pinlerine direnç konmaz.

> **Not**
>
> Kursun kodunda P9_30'un makro adı `GPIO_117_P9_30_DIGIT3`, değeri ise `112`. Doğrusu 112: P9_30 = GPIO3_16 = 3 × 32 + 16 (15.1). P9_23'ü Bölüm 17'de buton için kullandıysan butonu söküp pull-down direncini de çıkar.

Hane pininin akımı

Bir hane açıkken o hanenin bütün segment akımları **tek bir hane pininden** geçer. Ortak anot göstergede `8.` rakamı için:

```
8 segment × ~3 mA (470 Ω) ≈ 24 mA   → hepsi hane pininden çıkar
```

AM335x'in bir GPIO pini ise sadece birkaç mA verebilir (16.1). Kursun bağlantısında hane pinleri göstergeye doğrudan bağlıdır; bu, pini sınırının üzerinde zorlar. İki çözüm var:

Çözüm	Bağlantı	Program
Büyük segment direnci (deneme için)	Segmentlere 2.2 kΩ: segment başına ~0.6 mA, hane pininden en fazla ~5 mA	Kursla aynı (`-DCOMMON_ANODE`). Gösterge soluk olur.
Hane başına transistör (önerilen)	Ortak anotta PNP (BC557, 2N3906): emitör 3.3 V'a, kolektör hane pinine, baz 1 kΩ ile GPIO'ya	`-DCOMMON_ANODE -DDIGIT_TRANSISTOR` : GPIO **0** olunca transistör iletir, hane açılır

```
Hane başına PNP (ortak anot)

P9_3 (3.3 V) ●──────────────┐
                            │ E
P9_15 (48) ●──[ 1 kΩ ]──B─┤  BC557 (PNP)
                            │ C
                            └──────● D1 (gösterge pin 12)

(D2, D3, D4 için aynısı: P9_23, P9_30, P9_27)
```

Ortak katot göstergede transistör NPN olur (BC547, 2N2222): emitör GND'ye, kolektör hane pinine; GPIO **1** olunca hane açılır. Program bunu da `-DDIGIT_TRANSISTOR` ile bilir.

> **Uyarı**
>
> Kabloları kart **kapalıyken** bağla. Ortak anot göstergenin hane pinlerini ya da PNP'lerin emitörünü **5 V'a bağlama**; GPIO pinleri 3.3 V'tur (17.2).

18.4 Pinleri Kontrol Et ve Komut Satırından Dene

Kursun "4 digit 7 segment LED Counter implementation" dersi. Segment pinlerini 17.3'te kontrol etmiştin; şimdi hane pinlerine bak:

```
root@beaglebone:~# for p in P9_15 P9_23 P9_30 P9_27; do config-pin $p gpio; config-pin -q $p; done
P9_15 Mode: gpio Direction: in Value: 0
P9_23 Mode: gpio Direction: in Value: 0
P9_30 Mode: gpio Direction: in Value: 0
P9_27 Mode: gpio Direction: in Value: 0
root@beaglebone:~# grep -E '44e10(840|844|998|9a4)' /sys/kernel/debug/pinctrl/44e10800.pinmux*/pins
pin 16 (PIN16) 16:gpio-32-63 44e10840 00000027 pinctrl-single   <-- P9_15
pin 17 (PIN17) 17:gpio-32-63 44e10844 00000027 pinctrl-single   <-- P9_23
pin 102 (PIN102) 16:gpio-96-127 44e10998 00000027 pinctrl-single <-- P9_30
pin 105 (PIN105) 19:gpio-96-127 44e109a4 00000027 pinctrl-single <-- P9_27
```

```
for p in P9_15 P9_23 P9_30 P9_27; do config-pin $p gpio; config-pin -q $p; done
grep -E '44e10(840|844|998|9a4)' /sys/kernel/debug/pinctrl/44e10800.pinmux*/pins
```

Pin	Register
P9_15	`0x44E10840`
P9_23	`0x44E10844`
P9_30	`0x44E10998`
P9_27	`0x44E109A4`

Sonra segmentlerin gerçekten ortak olduğunu gör. Ortak anot göstergede bir `7` deseni yazıp önce 1. haneyi, sonra 4. haneyi aç:

```
root@beaglebone:~# for g in 66 67 69 68 45 44 26 46 48 49 112 115; do \
    echo $g > /sys/class/gpio/export; echo out > /sys/class/gpio/gpio$g/direction; done
root@beaglebone:~# for g in 66 67 69 68 45 44 26 46; do echo 1 > /sys/class/gpio/gpio$g/value; done
                                  <-- ortak anot: segment 1 = sönük
root@beaglebone:~# for g in 66 67 69; do echo 0 > /sys/class/gpio/gpio$g/value; done
                                  <-- a, b, c seçili: "7"
root@beaglebone:~# echo 1 > /sys/class/gpio/gpio48/value
                                  <-- sadece 1. hanede 7
root@beaglebone:~# echo 1 > /sys/class/gpio/gpio115/value
                                  <-- 4. hanede de aynı 7: segmentler ortak
root@beaglebone:~# echo 0 > /sys/class/gpio/gpio48/value; echo 0 > /sys/class/gpio/gpio115/value
```

```
for g in 66 67 69 68 45 44 26 46 48 49 112 115; do
    echo $g > /sys/class/gpio/export
    echo out > /sys/class/gpio/gpio$g/direction
done

for g in 66 67 69 68 45 44 26 46; do echo 1 > /sys/class/gpio/gpio$g/value; done  # hepsi sönük
for g in 66 67 69; do echo 0 > /sys/class/gpio/gpio$g/value; done                 # a, b, c: "7"
echo 1 > /sys/class/gpio/gpio48/value      # 1. hanede 7
echo 1 > /sys/class/gpio/gpio115/value     # 4. hanede de 7
echo 0 > /sys/class/gpio/gpio48/value; echo 0 > /sys/class/gpio/gpio115/value

for g in 66 67 69 68 45 44 26 46 48 49 112 115; do echo $g > /sys/class/gpio/unexport; done
```

Ortak katot göstergede 0 ve 1'leri tersle. Hane transistörle sürülüyorsa hane pinleri için de tersle.

18.5 Kursun Programı ve Düzeltmeler

Kursun programı `Project_Src/4digitcount/src/4_digit_7seg.c`. Tek haneli sayaçla aynı argümanları alır (`up`, `down`, `updown`, `random` ve bir gecikme). Çoğullamayı `dispaly_numbers()` fonksiyonu yapar: haneyi açar, rakamı yazar, `usleep(10)` bekler, segmentleri ve haneleri kapatır.

Kodda `updown`, `random` ve dijital saat öğrenciye bırakılmış (`TODOs for the students`, "Convert this application to implement Digital clock"). Ayrıca şu sorunlar var:

Sorun	Sonucu	Düzeltme
Gecikme argümanı milisaniye değil, **tarama sayısı** (`for (i = 0; i < delay_value; i++) dispaly_numbers(number);`)	Sayma hızı CPU'ya ve sysfs'in hızına bağlı; aynı değer farklı kartlarda farklı hız verir	Gecikme ms cinsinden; zaman `clock_gettime()` ile ölçülür
`down` modunda `uint32_t number` 0'dan sonra azaltılıyor	0000'dan sonra 9999 değil **7295** görünür (4294967295'in son dört hanesi)	0'dan sonra 9999'a dön
Her segment yazımı `open()` + `write()` + `close()`	Hane başına 19 dosya yazımı (~57 sistem çağrısı); tarama yavaş, gösterge soluk ve titrek	`value` dosyaları bir kez açılır, sadece `pwrite()` yapılır; sadece değişen segment yazılır (18.6)
Hane `usleep(10)` kadar açık, ardından 11 dosya yazımıyla kapatılıyor	Hane, zamanın küçük bir kısmında açık: gösterge soluk	Her hane 2.5 ms açık
Hane önce açılıyor, sonra segmentler tek tek yazılıyor	Rakam yazılırken hane yanık: ara desenler kısa süre görünür	Segmentler bütün haneler kapalıyken yazılır, sonra hane açılır

Sorun	Sonucu	Düzeltme
`COMMON_CATHODE` tanımlanırsa sadece segment mantığı değişiyor, hane hep `HIGH` ile açılıyor	Ortak katot göstergede hiçbir hane yanmaz	Hane mantığı da gösterge türüne göre seçilir
Hane pinleri doğrudan göstergeye bağlı	Hane pininden ~24 mA çekilir (18.3)	Transistör seçeneği (`-DDIGIT_TRANSISTOR`)
Hata kontrolü yok, Ctrl-C ile öldürülüyor	Gösterge son durumda kalır, GPIO'lar export edilmiş kalır	Hatalar kontrol edilir, `SIGINT` yakalanır

---

18.6 Çoğullama Kodu: seg7x4

Çoğullama ayrı bir modülde: `code/18_4digit_7segment/seg7x4.c` , `seg7x4.h` . Rakamların segment desenleri Bölüm 17'deki `seg7_pattern()` 'dan gelir.

Fonksiyon	Yaptığı iş
`seg7x4_init()`	12 GPIO'yu export eder, çıkış yapar, `value` dosyalarını açar; haneler kapalı başlar
`seg7x4_refresh(digits, dp_mask)`	Dört haneyi bir kez tarar (~10 ms). `digits` : soldan sağa 4 rakam; `dp_mask` : hangi hanenin noktası yanacak
`seg7x4_release()`	Göstergeyi söndürür, dosyaları kapatır, GPIO'ları bırakır

value dosyasını açık tutmak

Bölüm 16'daki `gpio_write_value()` her çağrıda dosyayı açıp kapatır. Saniyede birkaç kez yazan bir LED programı için sorun değil; ama çoğullama saniyede yüzlerce kez yazar. Dosyayı bir kez açıp hep aynı tanımlayıcıya yazmak çok daha ucuzdur:

```
/* Açık value dosyasına 0/1 yazar. pwrite() dosya konumunu değiştirmez. */
static int put(int fd, int value)
{
    return pwrite(fd, value ? "1" : "0", 1, 0) == 1 ? 0 : -1;
}
```

`pwrite()` , `write()` 'ın belirli bir konuma (burada 0) yazan halidir; her yazmadan önce `lseek()` gerekmez.

Tarama

```
int seg7x4_refresh(const uint8_t digits[SEG7X4_DIGITS], uint8_t dp_mask)
{
    uint8_t bits;
    int i;

    for (i = 0; i < SEG7X4_DIGITS; i++) {
        bits = seg7_pattern(digits[i]);
        if (dp_mask & (1 << i))
            bits |= 1 << 7;

        /* Bütün haneler kapalıyken segmentleri değiştir, sonra haneyi aç. */
        if (set_segments(bits) < 0 || put(digit_fd[i], DIGIT_ON) < 0)
            return -1;

        usleep(DIGIT_ON_TIME_US);           /* 2500 µs */

        if (put(digit_fd[i], DIGIT_OFF) < 0)
            return -1;
    }
    return 0;
}
```

Sıralama önemli: önce hane kapanır, segmentler değişir, sonra yeni hane açılır. Bir hane açıkken segmentler değişirse önceki hanenin rakamı yeni hanede soluk bir "gölge" (ghosting) olarak görünür.

`set_segments()` her segmentin son yazılan değerini tutar ve sadece değişenleri yazar. `1111` gösterilirken tarama başına segment yazımı sıfıra iner; sadece hane pinleri yazılır.

Gösterge türü

Segment ve hane mantığı derleme sırasında seçilir:

Derleme seçeneği	Gösterge	Segment yanar	Hane açılır
(yok)	Ortak katot	1	0
`-DDIGIT_TRANSISTOR`	Ortak katot + NPN	1	1
`-DCOMMON_ANODE`	Ortak anot (kursun göstergesi)	0	1
`-DCOMMON_ANODE -DDIGIT_TRANSISTOR`	Ortak anot + PNP	0	0

```
#ifdef COMMON_ANODE
#define SEGMENT_ON          LOW_VALUE
#else
#define SEGMENT_ON          HIGH_VALUE
#endif

#ifdef DIGIT_TRANSISTOR
#define DIGIT_ON            SEGMENT_ON
#else
#define DIGIT_ON            (!SEGMENT_ON)
#endif
```

18.7 Sayaç ve Dijital Saat

Program: `code/18_4digit_7segment/counter_4digit.c`.

Mod	Davranış
`up <ms>`	0000 → 9999 → 0000 …
`down <ms>`	9999 → 0000 → 9999 …
`updown <ms>`	0000 → 9999 → 0000 …; uçlar tekrarlanmaz
`random <ms>`	0000–9999 arası rastgele sayılar; aynı sayı art arda gelmez
`clock`	Saat `SS.DD` (ör. `14.37`); 2. hanenin noktası saniyede bir yanıp söner

Ana döngü hiç uyumaz; her turda bir kez tarar ve sayıyı değiştirme zamanı gelip gelmediğine bakar:

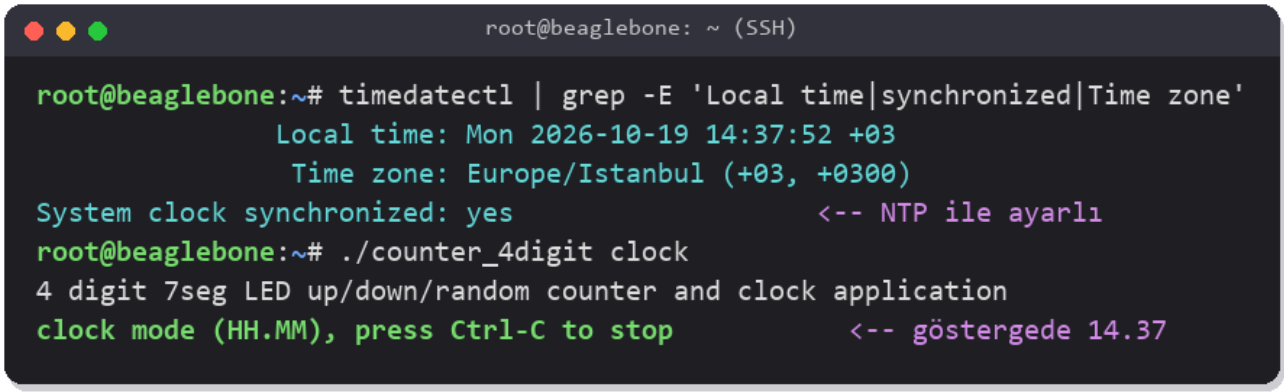
```
last_step = now_ms();

while (running) {
    if (mode == CLOCK) {
        t = time(NULL);
        localtime_r(&t, &tm);
        split(tm.tm_hour * 100 + tm.tm_min, digits);
        dp_mask = tm.tm_sec % 2 ? 0 : 1 << 1;
    } else {
        if (now_ms() - last_step >= delay_ms) {
            last_step = now_ms();
            number = next_number(mode, number, &step);
        }
        split(number, digits);
    }

    if (seg7x4_refresh(digits, dp_mask) < 0)
        break;
}
```

`split()` sayıyı soldan sağa dört rakama ayırır (`42` → `0 0 4 2`). `next_number()` her modun bir sonraki değerini hesaplar; `updown` 17.6'daki gibi bir adım değişkeniyle yön değiştirir.

`now_ms()` `CLOCK_MONOTONIC` kullanır: sistem saati NTP ile ileri geri ayarlansa bile geriye gitmez. Saat modu ise duvar saatini (`time()` + `localtime_r()`) gösterir.

Saat doğru mu?

AM335x'in içinde bir RTC var ama BeagleBone Black'te pil yok; kart kapanınca saat sıfırlanır. Debian, ağ varsa saati NTP ile (`systemd-timesyncd`) ayarlar. Saat modunu çalıştırmadan önce kontrol et:

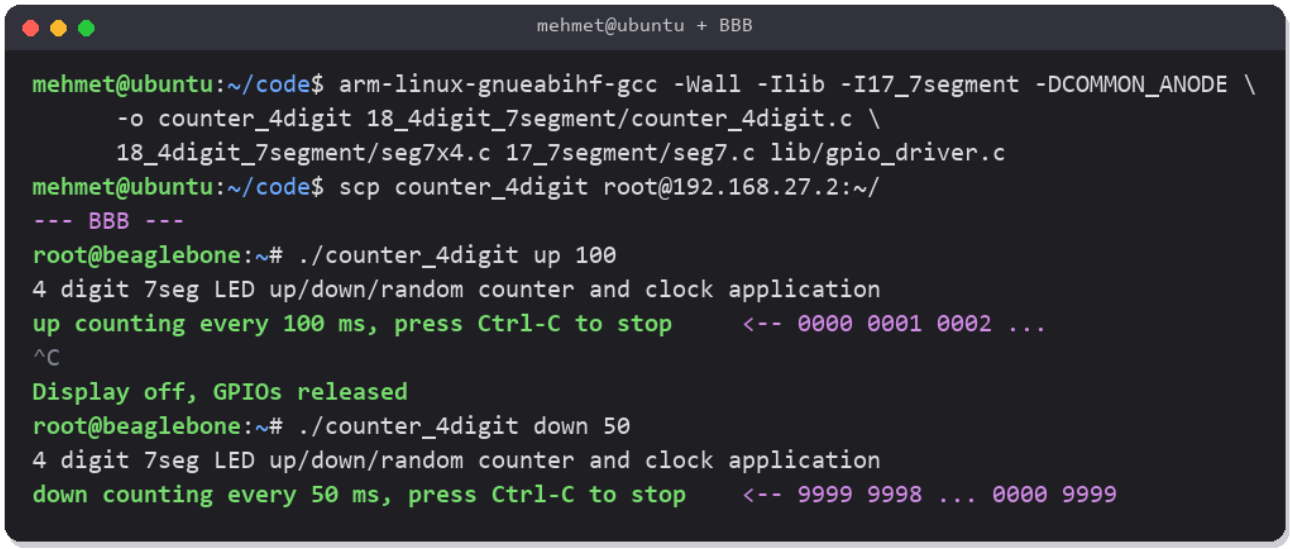



```
timedatectl                                 # "System clock synchronized: yes" olmalı
timedatectl set-timezone Europe/Istanbul    # gerekiyorsa saat dilimi
./counter_4digit clock
```

Kart internete çıkamıyorsa saati elle ayarla: `date -s "2026-10-19 14:37"`.

Derle ve çalıştır

Repodaki `code/` klasöründe:




```
arm-linux-gnueabihf-gcc -Wall -Ilib -I17_7segment -DCOMMON_ANODE \
      -o counter_4digit 18_4digit_7segment/counter_4digit.c \
      18_4digit_7segment/seg7x4.c 17_7segment/seg7.c lib/gpio_driver.c
scp counter_4digit root@192.168.27.2:~/
```

Haneler transistörle sürülüyorsa `-DDIGIT_TRANSISTOR` ekle; ortak katot göstergede `-DCOMMON_ANODE`'u çıkar.

Kartta:

```
./counter_4digit up 100        # saniyede 10 artar
./counter_4digit down 50
./counter_4digit updown 10
./counter_4digit random 1000
./counter_4digit clock
```

Eclipse kullanıyorsan (Bölüm 14): projeye `seg7x4.c`, `17_7segment/seg7.c` ve `lib/gpio_driver.c`'yi ekle, *Includes*'a `lib` ve `17_7segment` klasörlerini, *Defined symbols*'a `COMMON_ANODE` (ve gerekirse `DIGIT_TRANSISTOR`) yaz.

18.8 Kullanıcı Alanında Çoğullamanın Sınırları

Linux gerçek zamanlı bir işletim sistemi değildir. Kernel, programını herhangi bir anda durdurup başka bir süreci çalıştırabilir. Program durduğu sürece o an açık olan hane yanık kalır, diğerleri söner. Kart meşgulken (ör. `apt upgrade` sırasında) bunu parlaklık dalgalanması ve titreme olarak görürsün.

Yapabileceklerin:

- Programı gerçek zamanlı öncelikle çalıştır: `chrt -f 50 ./counter_4digit up 100`. Kernel, programını sıradan süreçlerden önce çalıştırır. Program `usleep()` ile beklediği için CPU'yu tekeline almaz.
- `top` ile programın CPU kullanımına bak. Sürekli tarama, sysfs üzerinden saniyede yüzlerce yazma demektir; bunun da bir maliyeti vardır.

Gerçek projelerde çoğullama genellikle yazılıma bırakılmaz:

Yöntem	Açıklama
Sürücü entegresi	MAX7219 (SPI), TM1637 (2 kablo), HT16K33 (I2C) gibi entegreler taramayı kendileri yapar; işlemci sadece gösterilecek rakamları bir kez gönderir. Pin sayısı da 12'den 2–3'e iner.
Mikrodenetleyici	Taramayı küçük bir mikrodenetleyici zamanlayıcı kesmesiyle yapar; BBB ona UART/I2C ile rakam gönderir. AM335x'in PRU çekirdekleri de bu tür zamanlama işleri için vardır.
Kernel sürücüsü	Tarama kernel'de bir zamanlayıcıyla yapılır; kullanıcı alanına sadece "göster" arayüzü açılır.

Kursun sonraki konusu MPU6050 sensörü I2C ile okur; HT16K33 gibi I2C'li bir gösterge sürücüsüyle de aynı yolla konuşulur.

18.9 Özet

- 4 haneli göstergede dört hanenin aynı harfli segmentleri içeride ortaktır; sadece hane pinleri (COM) ayrıdır. 8 + 4 = 12 pin.
- Çoğullama: segmentlere bir hanenin rakamını yaz, sadece o haneyi aç, kısa süre bekle, sıradakine geç. 4 × 2.5 ms = 10 ms tarama = 100 Hz, titreme görünmez.
- Gösterge sadece program taradığı sürece yanar; sayıyı değiştirmek için `sleep()` ile beklenmez, zaman `clock_gettime()` ile ölçülür.
- Segmentleri bütün haneler kapalıyken değiştir; yoksa gölgelenme olur.
- Hızlı tarama için `value` dosyalarını bir kez aç ve `pwrite()` ile yaz; sadece değişen segmentleri yaz.
- Bir hane pininden 8 segmentin akımı geçer; GPIO'yu korumak için hane başına transistör kullan (ortak anotta PNP, ortak katotta NPN).
- Kullanıcı alanında çoğullama Linux'un zamanlamasına bağlıdır; gerçek projelerde sürücü entegresi (MAX7219, TM1637, HT16K33) kullanılır.

18.10 Karşılaşılabilecek Problemler

```
root@beaglebone: ~ (SSH) (temsili hatalar)
Bütün hanelerde aynı rakam görünüyor        <-- haneler aynı anda açık / tarama yok
Rakamlar titriyor, parlaklık dalgalanıyor    <-- CPU meşgul, tarama gecikiyor
root@beaglebone:~# chrt -f 50 ./counter_4digit up 100
                                <-- gerçek zamanlı öncelikle çalıştır

root@beaglebone:~# ./counter_4digit up 100
/sys/class/gpio/export: Device or resource busy     <-- P9_23 hâlâ butonda / overlay'de
GPIO 49 could not be configured as output
Error: GPIO init failed !
```

Belirti	Sebep	Çözüm
Bütün hanelerde aynı rakam	Haneler aynı anda açık: hane mantığı ters ya da tarama yapılmıyor	Gösterge türüne uygun `-DCOMMON_ANODE` / `-DDIGIT_TRANSISTOR` seçeneklerini kontrol et (18.6)
Hiçbir hane yanmıyor	Hane mantığı ters (ör. ortak katot göstergeyi ortak anot için derlemek) ya da COM pinleri yanlış	18.4'teki gibi elle dene
Rakamlar ters (`8` yerine boşluk)	Segment mantığı ters	`-DCOMMON_ANODE` seçeneğini ekle/çıkar
Rakamlar titriyor, parlaklık dalgalanıyor	CPU meşgul, tarama gecikiyor	`chrt -f 50 ./counter_4digit ...`; diğer yükleri azalt
Bir hanede komşu hanenin rakamı soluk görünüyor	Segmentler hane açıkken değiştiriliyor (kursun kodu)	Önce haneyi kapat, segmentleri yaz, sonra sıradaki haneyi aç
Gösterge çok soluk	Segment direnci büyük; hane pini akımı sınırlıyor	Hane başına transistör ve 470 Ω segment direnci (18.3)
Geri sayımda 0000'dan sonra 7295 (kursun kodu)	`uint32_t` 0'dan sonra 4294967295'e döner	0'dan sonra 9999'a dön
Aynı gecikme değeri farklı hızda sayıyor (kursun kodu)	Gecikme tarama sayısı olarak kullanılıyor	Zamanı `clock_gettime()` ile ölç
`export`: `Device or resource busy` (GPIO 49)	P9_23 hâlâ başka bir yerde kullanılıyor (17.7)	`cat /sys/kernel/debug/gpio`; programı ya da overlay'i kapat
Saat yanlış	Kartta RTC pili yok, NTP ile eşitlenmedi ya da saat dilimi UTC	`timedatectl`, `timedatectl set-timezone ...`, gerekirse `date -s`
`Permission denied`	Program root olarak çalışmıyor	`sudo ./counter_4digit ...`

Bölüm 19 — MPU6050 ile Eğim Ölçmek (I2C)

> **Bu bölümde:**
>
> - İvmeölçer ve jiroskobun neyi ölçtüğünü ve birbirlerinin eksiğini nasıl kapattığını öğreneceksin.
> - MPU6050'nin register'larını, ölçek aralıklarını ve ham değerin birime nasıl çevrildiğini göreceksin.
> - I2C veriyolunun nasıl çalıştığını ve BBB'nin hangi I2C hattını kullanacağını öğreneceksin.
> - Sensörü önce `i2c-tools` ile komut satırından, sonra `/dev/i2c-2` üzerinden bir C programıyla okuyacaksın.
> - İvmeölçerden eğim açısını hesaplayacak, tamamlayıcı filtreyle jiroskopla birleştireceksin.
>
> **Ön koşul:** Bölüm 14 — kartta program derleyip çalıştırma, Bölüm 15 — pinmux.

> **Not**
>
> Bu bölümdeki komutlar kartta **root** olarak çalıştırılır. Kart BeagleBoard Debian imajı ve kernel 5.10 ile çalışıyor. Terminal görüntüleri temsilidir; sensörden okunan sayılar her çalıştırmada biraz farklıdır.

19.1 Neden İvmeölçer ve Jiroskop?

Kursun "Why Accelerometers are used?" ve "Why Gyroscope sensors are used?" dersleri.

İvmeölçer (accelerometer), üç eksende sensöre etki eden ivmeyi ölçer. Sensör hareketsizken ölçtüğü tek şey **yerçekimidir**: düz bir masada Z ekseni +1 g, X ve Y 0 g okur. Sensörü eğince yerçekimi vektörü eksenler arasında paylaşılır; buradan eğim açısı hesaplanır. Telefonun ekranını yan çevirince dönmesi bu şekilde çalışır.

Jiroskop (gyroscope), üç eksen etrafındaki **açısal hızı** (°/s) ölçer. Sensör dönmüyorsa 0 okur. Açıyı bulmak için açısal hızı zamanla toplamak (integral) gerekir.

	İvmeölçer	**Jiroskop**
Ölçtüğü	İvme (g): yerçekimi + hareket ivmesi	Açısal hız (°/s)
Eğim açısı	Doğrudan hesaplanır	Açısal hız toplanarak bulunur
Kısa vadede	Gürültülü: titreşim ve hareket ivmesi açıyı bozar	Temiz ve hızlı tepki verir
Uzun vadede	Kararlı: yerçekimi hep aynı yönde	Kayar (drift): küçük ofset hatası zamanla birikir
Dikey eksen etrafında dönme (yaw)	Ölçemez (yerçekimi değişmez)	Ölçer ama kayar

İkisi birbirinin zayıf yanını kapatır. Bu yüzden ikisi genellikle tek bir pakette bulunur ve birlikte kullanılır (19.10).

19.2 MPU6050

Kursun "MPU6050 Introduction" dersi. MPU6050 (InvenSense, şimdi TDK), 3 eksenli ivmeölçer ve 3 eksenli jiroskobu tek bir çipte birleştiren 6 eksenli bir sensördür. Her eksen 16 bitlik bir ADC ile ölçülür. Bilgisayarla **I2C** üzerinden konuşur (en fazla 400 kHz).

Özellik	Değer
Besleme (VDD)	2.375 – 3.46 V
I2C adresi	`0x68` (AD0 pini 0) veya `0x69` (AD0 pini 1)
Çıkış	İşaretli 16 bit (−32768 … 32767), yüksek bayt önce (big endian)
Açılış durumu	**Uyku modu**: register'lar okunur ama ölçüm yapılmaz

Bu bölümde kullanılan register'lar (Register Map Rev 4.2):

Adres	Ad	Görevi
`0x1B`	`GYRO_CONFIG`	Jiroskop ölçek aralığı (bit 4:3, `FS_SEL`)
`0x1C`	`ACCEL_CONFIG`	İvmeölçer ölçek aralığı (bit 4:3, `AFS_SEL`)
`0x3B` – `0x40`	`ACCEL_XOUT_H` … `ACCEL_ZOUT_L`	İvme X, Y, Z (her biri 2 bayt)
`0x41` – `0x42`	`TEMP_OUT_H/L`	Sıcaklık
`0x43` – `0x48`	`GYRO_XOUT_H` … `GYRO_ZOUT_L`	Açısal hız X, Y, Z
`0x6B`	`PWR_MGMT_1`	Uyku (bit 6, `SLEEP`) ve saat kaynağı (bit 2:0, `CLKSEL`). Açılışta `0x40`
`0x75`	`WHO_AM_I`	Her zaman `0x68` okunur; doğru çipe konuştuğunu doğrulamak için

Ölçüm register'ları art arda dizilmiştir: `0x3B`'den başlayarak **14 bayt** okumak ivme, sıcaklık ve jiroskobun hepsini verir. Register Map bunu önerir: tek seferde (burst) okunan değerler aynı ölçüm anına aittir.

19.3 Ölçek Aralığı ve Birim Dönüşümü

Kursun "Understanding MPU6050 ACC full scale range", "converting raw Acc value in to g values" ve "Understanding MPU6050 Gyro Full Scale range" dersleri.

16 bitlik ham değer her zaman −32768 … 32767 arasındadır. Bu aralığın kaç g'ye (ya da kaç °/s'ye) karşılık geleceğini **ölçek aralığı** (full scale range) belirler. Dar aralık daha hassas ama daha çabuk doyar; geniş aralık büyük hareketleri ölçer ama küçük değişimleri kaba ölçer.

`AFS_SEL`	İvme aralığı	Hassasiyet		`FS_SEL`	Jiroskop aralığı	Hassasiyet
0	±2 g	16384 LSB/g		0	±250 °/s	131 LSB/(°/s)
1	±4 g	8192 LSB/g		1	±500 °/s	65.5 LSB/(°/s)
2	±8 g	4096 LSB/g		2	±1000 °/s	32.8 LSB/(°/s)
3	±16 g	2048 LSB/g		3	±2000 °/s	16.4 LSB/(°/s)

(Product Specification Rev 3.4, bölüm 6.1 ve 6.2.)

Dönüşüm bir bölmedir:

```
ivme (g)      = ham değer / hassasiyet      ör. 16360 / 16384 ≈ 1.00 g   (±2 g)
açısal hız    = ham değer / hassasiyet      ör.  -177 / 131   ≈ -1.35 °/s (±250 °/s)
sıcaklık (°C) = ham değer / 340 + 36.53     ör. -4226 / 340 + 36.53 ≈ 24.1 °C
```

Sıcaklık formülü Register Map'in 4.18. bölümündendir. Bu sensör çipin kendi sıcaklığını ölçer; ortam sıcaklığından birkaç derece yüksek okuması normaldir.

Kursun programı ve bu bölümün kodu en hassas aralıkları (±2 g, ±250 °/s) kullanır. Eğim ölçmek için yeterlidir; sensörü sallarsan ya da hızlı çevirirsen değerler doyar.

19.4 I2C'ye Kısa Bakış

I2C (Inter-Integrated Circuit), iki kablo üzerinden çok sayıda aygıtın konuştuğu bir seri veriyoludur:

Hat	Görevi
SCL	Saat; her zaman ana aygıt (master, burada AM335x) üretir
SDA	Veri; iki yönlü

Hatlar **open-drain**'dir: aygıtlar hattı sadece 0'a çekebilir, 1'e **pull-up dirençleri** çeker. Pull-up yoksa hat hiç 1 olmaz ve iletişim kurulamaz. Veriyolundaki her bağımlı aygıtın (slave) 7 bitlik bir adresi vardır; master her işlemde önce adresi gönderir, sadece o adresteki aygıt cevap verir (ACK).

Bir register'a yazmak:

```
START | 0x68 + W | ACK | 0x6B (register) | ACK | 0x01 (veri) | ACK | STOP
```

Bir register'dan okumak iki aşamalıdır: önce okunacak register adresi yazılır, sonra okunur. Sensör, her okunan bayttan sonra register adresini kendisi bir artırır; böylece art arda register'lar tek seferde okunur:

```
  START | 0x68 + W | ACK | 0x3B | ACK | START | 0x68 + R | ACK | b0 | ACK | b1 | ... | b13 | NACK
| STOP
                                        ↑
                                repeated start: araya STOP girmez
```

AM335x'te üç I2C denetleyicisi var. BeagleBone Black'te bunlardan biri, dahili aygıtlara bağlıdır. P9 header'ında kullanıcıya açık ve hazır olan ise **I2C2**'dir:

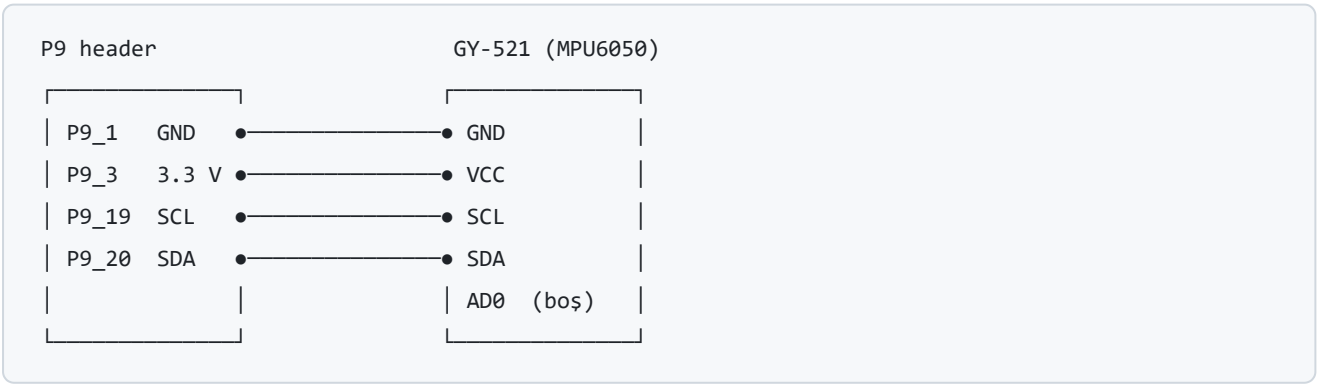
Denetleyici	Pinler	Kullanım
I2C0	Dışarı çıkmaz	PMIC (TPS65217), kartın EEPROM'u, HDMI çerçevecisi
I2C1	P9_17 (SCL), P9_18 (SDA)	Varsayılan olarak kapalı; pinmux ile açılır
I2C2	**P9_19 (SCL), P9_20 (SDA)**	Açık ve pull-up'lı; cape'lerin kimlik EEPROM'ları (0x54–0x57) da bu hatta

---

19.5 Devre

Kursun "MPU6050 breakout board details" ve "Deciding BBB I2C pins for sensor interfacing" dersleri. MPU6050 genellikle **GY-521** adlı küçük bir modül olarak satılır. Modülün üzerinde voltaj regülatörü ve SDA/SCL pull-up dirençleri vardır.

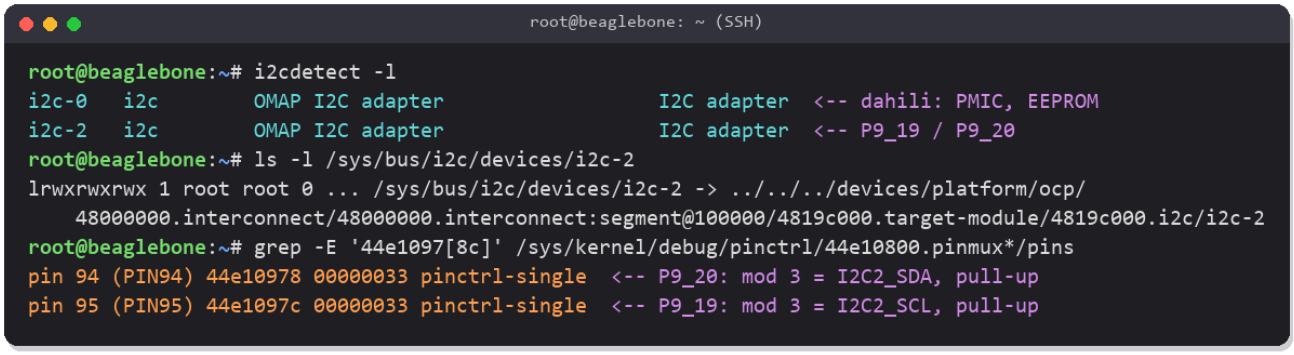
GY-521 pini	BBB pini	Not
VCC	P9_3 (3.3 V)	
GND	P9_1 (GND)	
SCL	P9_19 (I2C2_SCL)	
SDA	P9_20 (I2C2_SDA)	
AD0	Boş	Modülde GND'ye çekili: adres `0x68`. 3.3 V'a bağlarsan `0x69` olur
INT, XDA, XCL	Boş	Kesme çıkışı ve yardımcı I2C; bu bölümde kullanılmıyor



> **Uyarı**
>
> - Kabloları kart **kapalıyken** bağla.
> - Modülü **3.3 V**'tan besle. Modülün regülatörü 5 V'u da kabul eder, ama modülün şemasını bilmiyorsan pull-up dirençlerinin nereye bağlı olduğunu da bilemezsin. 3.3 V'tan beslemek SDA/SCL'nin hiçbir durumda 3.3 V'u aşmamasını garanti eder; AM335x'in pinleri 5 V'a dayanmaz.

Pinmux ve veriyolu numarası

P9_19 ve P9_20, BBB'nin device tree'sinde zaten I2C2 moduna (mod 3) ayarlıdır; bir şey yapmana gerek yok. Kontrol etmek ve kernel'in bu veriyoluna verdiği numarayı bulmak için:




```
i2cdetect -l
ls -l /sys/bus/i2c/devices/i2c-2
grep -E '44e1097[8c]' /sys/kernel/debug/pinctrl/44e10800.pinmux*/pins
```

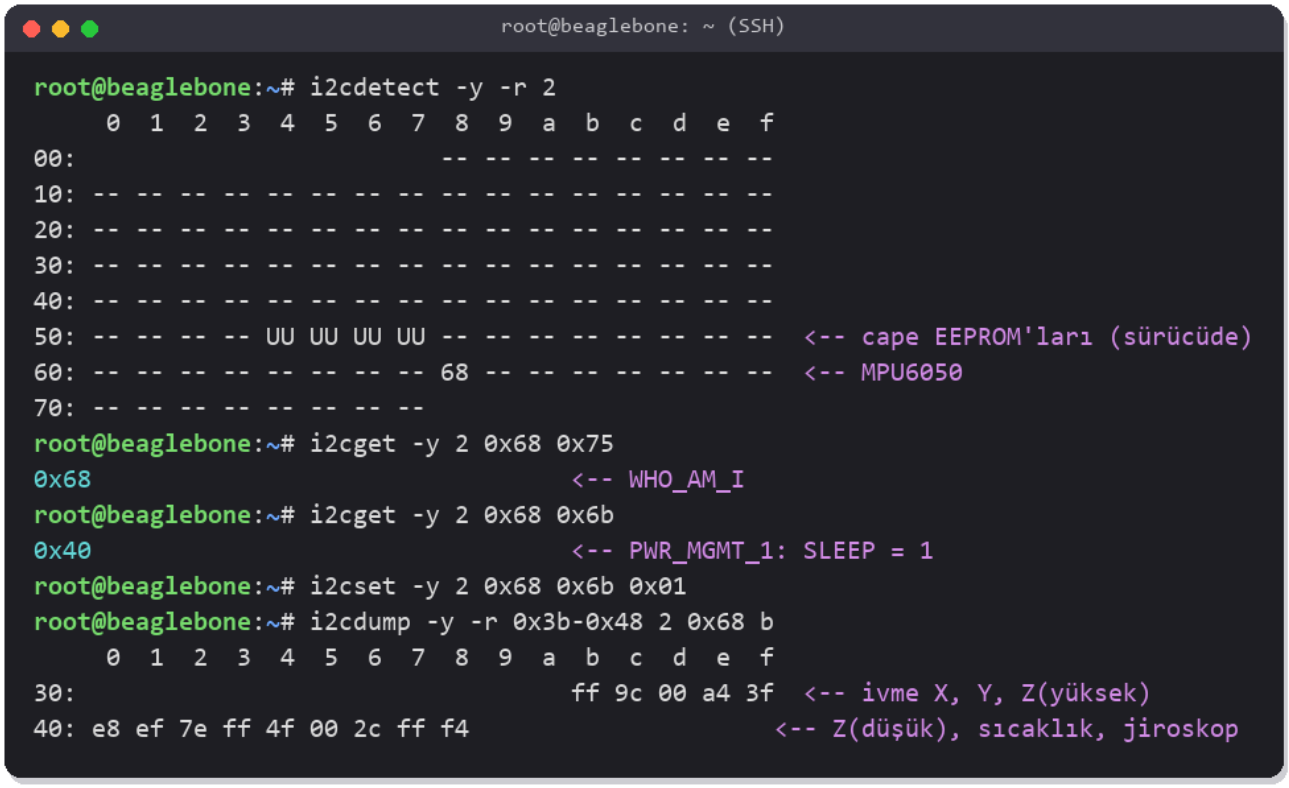
`/sys/bus/i2c/devices/i2c-2`'nin gösterdiği yolda `4819c000.i2c` geçiyor: bu, I2C2 denetleyicisinin adresi (AM335x TRM'deki bellek haritası). Yani kernel 5.10'da I2C2 = `/dev/i2c-2`.

> **Not**
>
> Eski 3.8 kernel'lerde (kursun ilk sürümündeki Angstrom imajı) veriyolları açılış sırasına göre numaralanırdı ve I2C2 `/dev/i2c-1` olarak görünürdü. Güncel kernel'ler device tree'deki `aliases` ile sabit numara verir. Emin olmak için her zaman `i2cdetect -l` ve yukarıdaki `ls -l` ile denetleyicinin adresine bak.

Pin	Register	Değer
P9_20 (I2C2_SDA)	`0x44E10978`	`0x33` : mod 3, giriş açık, pull-up
P9_19 (I2C2_SCL)	`0x44E1097C`	`0x33`

19.6 Komut Satırından: i2c-tools

Program yazmadan önce sensörün veriyolunda göründüğünü ve doğru cevap verdiğini `i2c-tools` ile kontrol et. BeagleBoard Debian imajlarında kuruludur; değilse `apt install i2c-tools`.




```
i2cdetect -y -r 2                  # veriyolu 2'yi tara
i2cget -y 2 0x68 0x75              # WHO_AM_I → 0x68
i2cget -y 2 0x68 0x6b              # PWR_MGMT_1 → 0x40 (uykuda)
i2cset -y 2 0x68 0x6b 0x01         # uyandır, saat kaynağı: jiroskop PLL
i2cdump -y -r 0x3b-0x48 2 0x68 b   # 14 ölçüm baytı
```

Komut	Görevi
`i2cdetect -y -r 2`	Veriyolundaki her adresi dener. `68` = cevap veren aygıt, `UU` = bir kernel sürücüsünün kullandığı adres, `--` = cevap yok
`i2cget -y 2 0x68 0x75`	Veriyolu 2, aygıt `0x68`, register `0x75`'i okur
`i2cset -y 2 0x68 0x6b 0x01`	Register `0x6B`'ye `0x01` yazar
`i2cdump -y -r 0x3b-0x48 2 0x68 b`	`0x3B` – `0x48` arasını bayt bayt okur

`-y` onay sorusunu atlar; `-r`, `i2cdetect`'e aygıtları okuma ile yoklamasını söyler (bazı aygıtlar yazma yoklamasına kötü tepki verir).

`i2cdump` çıktısını elle çözmek iyi bir alıştırmadır: `0x3B` – `0x3C` = `ff 9c` → `0xFF9C` = −100 → −100 / 16384 ≈ −0.01 g. `0x3F` – `0x40` = `3f e8` → 16360 → 1.00 g: sensör düz duruyor.

19.7 Linux'ta I2C: /dev/i2c-N

Kursun "MPU6050 Raw read Source code explanation" dersleri. Kernel'in `i2c-dev` sürücüsü her I2C veriyolu için bir `/dev/i2c-N` aygıt dosyası açar. Bu dosya kullanıcı alanından I2C işlemi yapmanı sağlar:

Adım	Çağrı
Veriyolunu aç	`fd = open("/dev/i2c-2", O_RDWR)`
Konuşulacak aygıtı seç	`ioctl(fd, I2C_SLAVE, 0x68)`
Bayt yaz	`write(fd, buf, n)` : START, adres+W, `buf` , STOP
Bayt oku	`read(fd, buf, n)` : START, adres+R, `n` bayt, STOP
Birleşik işlem	`ioctl(fd, I2C_RDWR, &xfer)` : birden çok mesaj, aralarında repeated start

Kursun kodu register okumak için önce `write()` ile register adresini, sonra `read()` ile veriyi gönderir. Bu iki ayrı işlemdir; arada STOP gider. MPU6050 register adresini STOP'tan sonra da hatırladığı için çalışır. Ama aynı veriyolunda başka bir program (ya da kernel sürücüsü) araya girerse adres değişebilir. `I2C_RDWR` ile ikisi tek bir işlemde, 19.4'teki repeated start ile gider:

```
static int read_regs(int fd, uint8_t reg, uint8_t *buf, uint16_t len)
{
    struct i2c_msg msgs[2] = {
        { .addr = dev_addr, .flags = 0,        .len = 1,   .buf = &reg },
        { .addr = dev_addr, .flags = I2C_M_RD, .len = len, .buf = buf  },
    };
    struct i2c_rdwr_ioctl_data xfer = { .msgs = msgs, .nmsgs = 2 };

    if (ioctl(fd, I2C_RDWR, &xfer) != 2) {
        perror("i2c read");
        return -1;
    }
    return 0;
}
```

> **İpucu**
>
> `/dev/i2c-*` yoksa `i2c-dev` modülü yüklenmemiştir: `modprobe i2c-dev` . Dosyalar `i2c` grubuna aittir; root olmayan bir kullanıcıyla çalışacaksan kullanıcıyı bu gruba ekle (`usermod -aG i2c debian`).

19.8 Kursun Programı ve Düzeltmeler

Kursun programı `Project_Src/MPU6050_raw_read/mpu6050_rawread.c`. Sensörü uyandırır, ölçek aralıklarını ayarlar ve sonsuz döngüde ivme ile jiroskobu okuyup yazdırır. Kodda şu sorunlar var:

Sorun	Sonucu	Düzeltme
Okunan baytlar `char` dizisinde tutulup `buf[0] << 8` ile birleştiriliyor	ARM'da `char` işaretsiz olduğu için tesadüfen doğru. x86'da `char` işaretlidir: düşük bayt `0x80` veya üstüyse işaretle genişler ve yüksek baytı siler (`0x00 0x80` → 128 yerine −128). Jiroskopta baytlar `+` ile toplanıyor; o da aynı sebeple yanlış sonuç verir	Baytlar `uint8_t`
`mpu6050_read()`, `read()` başarısız olsa da `0` döndürüyor	Kablo çıksa bile program eski/çöp değerlerle sessizce devam eder	Hata çağırana bildirilir, program durur
İvme ve jiroskop iki ayrı okumayla alınıyor	İki değer farklı ölçüm anlarına ait olabilir	`0x3B`'den 14 bayt tek seferde (19.2)
Register adresi ve veri iki ayrı I2C işleminde	Veriyolu paylaşılırsa yanlış register okunabilir	`I2C_RDWR` ile repeated start (19.7)
`mpu6050_init()` hataları kontrol etmiyor, `WHO_AM_I`'a bakmıyor	Yanlış aygıta ya da hiçbir şeye konuşulsa da program "çalışır"	`WHO_AM_I` = `0x68` kontrol edilir, her yazma kontrol edilir
`PWR_MGMT_1` = `0x00`: saat kaynağı dahili 8 MHz osilatör	Çalışır, ama Register Map jiroskop PLL'lerinden birinin seçilmesini "highly recommended" diye önerir (daha kararlı)	`0x01`: X jiroskop PLL
Yorum "250 ms bekle" diyor, kod `usleep(50 * 1000)`	Küçük tutarsızlık	Bekleme süresi tek bir sabitte

Düzeltilmiş kod

`code/19_mpu6050/mpu6050.c` ve `mpu6050.h` sensörle ilgili her şeyi toplar:

Fonksiyon	Yaptığı iş
`mpu6050_open(dev, addr)`	`/dev/i2c-N`'i açar, aygıt adresini seçer
`mpu6050_init(fd, afs, gfs)`	`WHO_AM_I`'ı doğrular, sensörü uyandırır, ölçek aralıklarını ayarlar
`mpu6050_read_raw(fd, &raw)`	14 baytı tek seferde okur; ham ivme, sıcaklık, jiroskop
`mpu6050_convert(&raw, afs, gfs, &data)`	Ham değerleri g, °C ve °/s'ye çevirir (19.3)

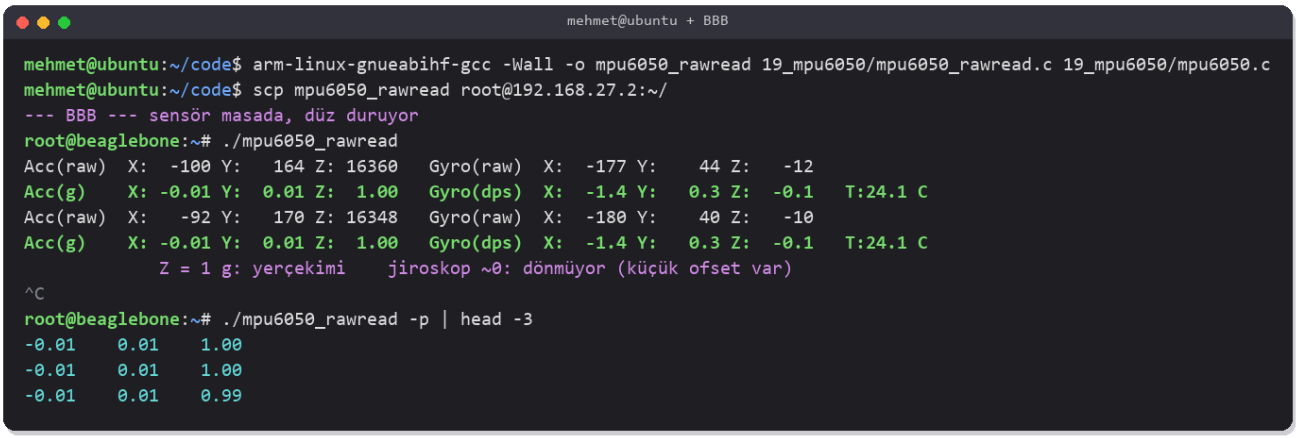
Baytları birleştirmek:

```
/* Yüksek bayt önce (big endian) gelen 16 bitlik işaretli değer */
static int16_t be16(const uint8_t *p)
{
    return (int16_t)((p[0] << 8) | p[1]);
}

int mpu6050_read_raw(int fd, struct mpu6050_raw *raw)
{
    uint8_t b[14];
    int i;

    if (read_regs(fd, MPU6050_REG_ACCEL_XOUT_H, b, sizeof(b)) < 0)
        return -1;

    for (i = 0; i < 3; i++) {
        raw->acc[i]  = be16(&b[i * 2]);         /* 0x3B-0x40 */
        raw->gyro[i] = be16(&b[8 + i * 2]);     /* 0x43-0x48 */
    }
    raw->temp = be16(&b[6]);                    /* 0x41-0x42 */
    return 0;
}
```

`(int16_t)` dönüşümü önemli: `0xFF9C` bir `int` olarak 65436'dır; 16 bite sığdırıldığında −100 olur.

19.9 Ham Okuma Programı

Kursun "Testing MPU6050 Raw read application on BBB" dersi. `code/19_mpu6050/mpu6050_rawread.c` her 250 ms'de ham ve dönüştürülmüş değerleri yazdırır. `-p` seçeneğiyle kursun ikinci çıktı biçimini kullanır: sadece ivme (g), sekmeyle ayrılmış üç sütun.



```
arm-linux-gnueabihf-gcc -Wall -o mpu6050_rawread 19_mpu6050/mpu6050_rawread.c
19_mpu6050/mpu6050.c
scp mpu6050_rawread root@192.168.27.2:~/
```

Kartta:

```
./mpu6050_rawread         # Ctrl-C ile dur
./mpu6050_rawread -p      # sadece ivme, sütun biçiminde
```

Sensörü elinde çevirerek değerlere bak:

- Düz dururken Z ≈ +1 g, X ve Y ≈ 0. Ters çevirince Z ≈ −1 g.
- Bir kenarı üzerine dikince o eksen ±1 g olur. Üç eksenin vektör toplamı hareketsizken hep ~1 g'dir.
- Jiroskop sadece döndürürken 0'dan ayrılır. Dururken bile birkaç °/s'lik sabit bir değer (ofset) okuyabilir; bu normaldir.

> **İpucu**
>
> `-p` çıktısını bilgisayarına alıp grafiğini çizebilirsin: `ssh root@192.168.27.2 ./mpu6050_rawread -p > acc.tsv` . Dosyayı bir tablo programında ya da Python/gnuplot ile aç.

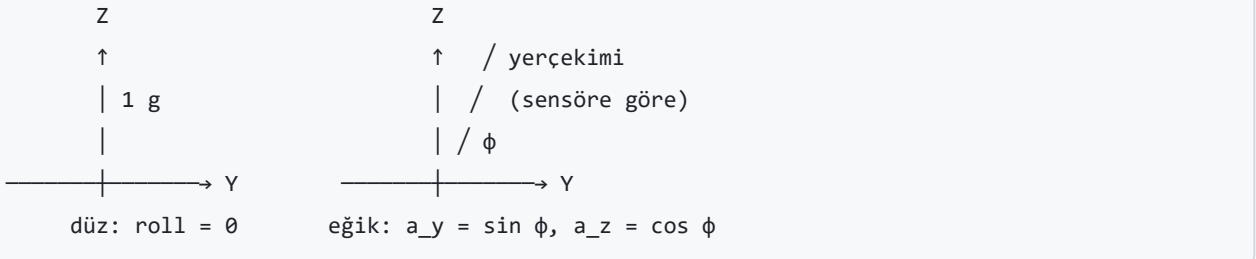
19.10 Eğim Açısı

Kursun "Measuring Tilt Angle of the object" dersleri.

İvmeölçerden

Sensör hareketsizken ölçülen ivme vektörü sadece yerçekimidir. Sensör X ekseni etrafında φ kadar dönerse yerçekimi Y ve Z arasında paylaşılır: `a_y = sin ϕ` , `a_z = cos ϕ` . Buradan:

```
roll  (X ekseni etrafında) = atan2(a_y, a_z)
pitch (Y ekseni etrafında) = atan2(−a_x, √(a_y² + a_z²))
```



`atan` yerine `atan2` kullanılır: iki bileşenin işaretine bakarak dört çeyreği ayırır ve `a_z` = 0 olunca sıfıra bölme olmaz. Pitch formülündeki karekök, sensör aynı anda iki eksende eğik olsa da doğru açıyı verir.

> **Not**
>
> Dikey eksen etrafındaki dönme (yaw, pusula yönü) ivmeölçerden hesaplanamaz: sensör masada dönerken yerçekimi vektörü değişmez. Yaw için jiroskop (kayar) ya da manyetometre (ör. MPU9250, MPU6050 + HMC5883L) gerekir.

Tamamlayıcı filtre

İvmeölçerin açısı titreşimde ve hareket sırasında sıçrar (hareket ivmesi yerçekimine eklenir). Jiroskobun açısal hızını toplamak temiz bir açı verir ama ofset hatası birikir ve açı yavaşça kayar. **Tamamlayıcı filtre** (complementary filter) ikisini birleştirir:

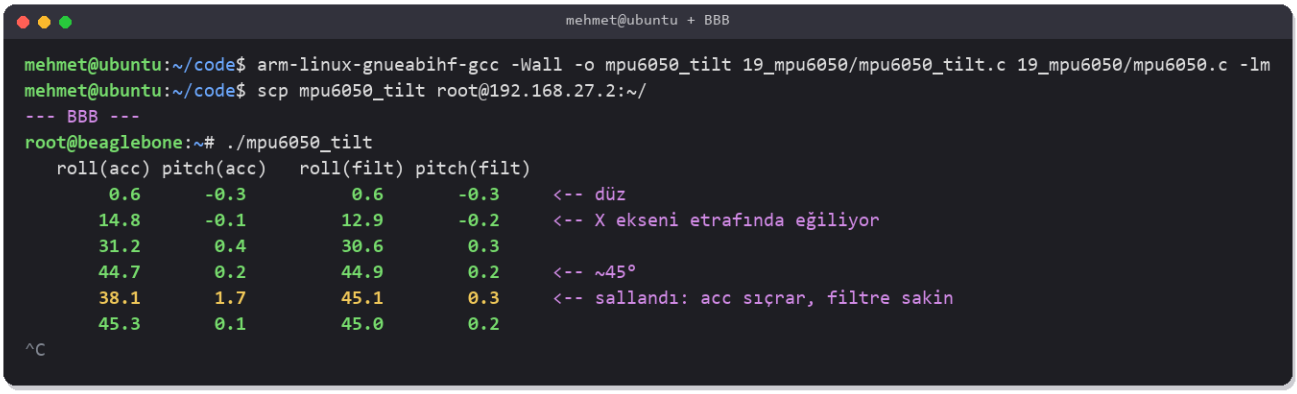
```
açı = α × (açı + jiroskop × Δt)  +  (1 − α) × ivmeölçer açısı
       └ kısa vade: jiroskop ┘      └ uzun vade: ivmeölçer ┘
```

α = 0.98 ve 50 Hz örneklemede (Δt = 20 ms) filtre, ~1 saniyeden kısa değişimlerde jiroskoba, daha yavaş olanlarda ivmeölçere güvenir (zaman sabiti ≈ α·Δt / (1 − α) ≈ 1 s).

`code/19_mpu6050/mpu6050_tilt.c` iki açıyı yan yana yazdırır:

```
static void acc_angles(const double a[3], double *roll, double *pitch)
{
    *roll  = atan2(a[1], a[2]) * RAD_TO_DEG;
    *pitch = atan2(-a[0], sqrt(a[1] * a[1] + a[2] * a[2])) * RAD_TO_DEG;
}
...
    while (running) {
        usleep(PERIOD_MS * 1000);
        ...
        t = now_s();
        dt = t - last_t;            /* gerçek geçen süre; usleep'e güvenme */
        last_t = t;

        acc_angles(d.acc, &roll_acc, &pitch_acc);

        roll  = ALPHA * (roll  + d.gyro[0] * dt) + (1 - ALPHA) * roll_acc;
        pitch = ALPHA * (pitch + d.gyro[1] * dt) + (1 - ALPHA) * pitch_acc;
        ...
    }
```

Δt, `usleep()`'e verilen değerden değil saatten ölçülür: Linux, programı istenenden geç uyandırabilir (18.8).




```
arm-linux-gnueabihf-gcc -Wall -o mpu6050_tilt 19_mpu6050/mpu6050_tilt.c 19_mpu6050/mpu6050.c -lm
scp mpu6050_tilt root@192.168.27.2:~/
```

`-lm`, `atan2()` ve `sqrt()` için matematik kütüphanesini bağlar. Kartta `./mpu6050_tilt` ile çalıştır, sensörü yavaşça eğ ve sonra salla: sallarken `acc` sütunları sıçrar, `filt` sütunları sakin kalır.

> **İpucu**
>
> Jiroskobun ofsetini çıkarırsan filtre daha iyi çalışır: program açılışında sensör hareketsizken jiroskobu örneğin 200 kez oku, ortalamasını al ve sonraki her okumadan çıkar.

> **Not**
>
> Linux kernel'de MPU6050 için hazır bir sürücü de var (`inv_mpu6050` , IIO alt sistemi). Device tree ile bağlanınca değerler `/sys/bus/iio/devices/iio:deviceN/` altında dosya olarak görünür. Bu bölümde kursu takip ederek sensörle doğrudan `/dev/i2c-2` üzerinden konuştuk; aynı veriyolu adresini bir kernel sürücüsü alırsa `i2cdetect` o adreste `UU` gösterir ve kullanıcı alanından `I2C_SLAVE` hata verir.

19.11 Özet

- İvmeölçer yerçekimini ölçer: hareketsizken eğim açısı doğrudan hesaplanır ama titreşimde gürültülüdür. Jiroskop açısal hızı ölçer: kısa vadede temizdir ama toplandıkça kayar.
- MPU6050: 6 eksen, 16 bit, I2C adresi `0x68` (AD0 = 1 ise `0x69`). Açılışta uykudadır: `PWR_MGMT_1` 'e (`0x6B`) yazarak uyandır. `WHO_AM_I` (`0x75`) = `0x68` .
- Ham değer / hassasiyet = birim: ±2 g'de 16384 LSB/g, ±250 °/s'de 131 LSB/(°/s). Sıcaklık = ham / 340 + 36.53.
- BBB'de kullanıcıya açık I2C hattı I2C2: P9_19 (SCL), P9_20 (SDA), kernel 5.10'da `/dev/i2c-2` . `i2cdetect -l` ile doğrula.
- `i2cdetect` , `i2cget` , `i2cset` , `i2cdump` ile program yazmadan sensörü dene.
- C'den: `open("/dev/i2c-2")` , `ioctl(I2C_SLAVE)` , register okumak için `ioctl(I2C_RDWR)` (repeated start). 14 baytı tek seferde oku; baytları `uint8_t` tut.

- Eğim: `roll = atan2(a_y, a_z)`, `pitch = atan2(-a_x, √(a_y² + a_z²))`. Tamamlayıcı filtre jiroskopla ivmeölçeri birleştirir.

19.12 Karşılaşılabilecek Problemler

```
root@beaglebone: ~ (SSH) (temsili hatalar)
root@beaglebone:~# ./mpu6050_rawread
i2c read: Remote I/O error                <-- sensör yanıt vermiyor (NACK): kablo, adres
Error: MPU6050 init failed

root@beaglebone:~# i2cdetect -y -r 2
60: -- -- -- -- -- -- -- -- -- 69 -- -- -- -- -- --   <-- AD0 = 1: adres 0x69

root@beaglebone:~# ./mpu6050_rawread
/dev/i2c-2: No such file or directory      <-- i2c-dev modülü yok / yanlış veriyolu
root@beaglebone:~# modprobe i2c-dev; i2cdetect -l

Acc(g)    X:  0.00 Y:  0.00 Z:  0.00   Gyro(dps)  X:   0.0 Y:   0.0 Z:   0.0
                                    <-- sensör uykuda (PWR_MGMT_1 yazılmadı)
```

Belirti	Sebep	Çözüm
`i2cdetect`'te `68` yok, program `Remote I/O error` veriyor	Sensör cevap vermiyor (NACK): SDA/SCL ters, besleme yok, yanlış veriyolu	Kabloları kontrol et; SDA = P9_20, SCL = P9_19; modülün LED'i yanıyor mu?
`i2cdetect`'te `69` görünüyor	AD0 pini 1'de	AD0'ı boş bırak/GND'ye bağla ya da programda adresi `0x69` yap
`i2cdetect` her adreste aygıt gösteriyor ya da çok yavaş	Pull-up yok ya da SDA/SCL kısa devre	Modülde pull-up olduğundan emin ol; kabloları kontrol et
`/dev/i2c-2: No such file or directory`	`i2c-dev` yüklü değil ya da veriyolu numarası farklı	`modprobe i2c-dev`; `i2cdetect -l`
`Permission denied`	Root değilsin, kullanıcı `i2c` grubunda değil	`sudo` ya da `usermod -aG i2c <kullanıcı>`
`I2C_SLAVE: Device or resource busy`	Adres bir kernel sürücüsünde (`i2cdetect`'te `UU`)	Sürücüyü/overlay'i kaldır ya da sürücünün sysfs arayüzünü kullan
Bütün değerler 0	Sensör uykuda: `PWR_MGMT_1` yazılmadı	`i2cset -y 2 0x68 0x6b 0x01` ya da programın init'ini kontrol et
Değerler mantıksız sıçrıyor, ±1 g yerine tuhaf sayılar (x86'da derlenmiş kursun kodu)	`char` dizisinde işaret genişlemesi	Baytları `uint8_t` tut (19.8)
İvme değerleri 2 g'de, jiroskop 250 °/s'de takılı kalıyor	Ölçek aralığı doydu	Daha geniş aralık seç (`ACC_FS_8G`, `GYRO_FS_1000`); hassasiyet değeri de değişir

Belirti	Sebep	Çözüm
Filtrelenmiş açı yavaşça kayıyor	Jiroskop ofseti	Açılışta ofseti ölçüp çıkar; α'yı biraz küçült (ör. 0.96)
Sıcaklık ortamdan birkaç derece yüksek	Çipin kendi sıcaklığı ölçülüyor	Normal; ortam sıcaklığı için ayrı sensör kullan
`undefined reference to 'atan2'`	Matematik kütüphanesi bağlanmadı	Derleme komutunun sonuna `-lm` ekle

Bölüm 20 — 16x2 Karakter LCD (HD44780)

> **Bu bölümde:**
>
> - 16x2 karakter LCD'nin pinlerini ve içindeki HD44780 denetleyicisini tanıyacaksın.
> - DDRAM, CGROM ve CGRAM'in ne olduğunu ve ekrandaki her karakterin nereden geldiğini öğreneceksin.
> - HD44780 komutlarını ve LCD'yi 4 bit modunda doğru başlatma dizisini göreceksin.
> - LCD'yi BBB'ye 7 GPIO ile bağlayacak ve düzeltilmiş bir sürücüyle yazı, IP adresi, saat ve kayan yazı göstereceksin.
> - CGRAM'e kendi karakterlerini (Türkçe harfler) tanımlayacaksın.
>
> **Ön koşul:** Bölüm 16 — kursun GPIO kütüphanesi, Bölüm 17 — P8 pinlerine çoklu bağlantı.

> **Not**
>
> Bu bölümdeki komutlar kartta **root** olarak çalıştırılır. Kart BeagleBoard Debian imajı ve kernel 5.10 ile çalışıyor; GPIO numaraları bu kernel'e göre. Terminal görüntüleri ve LCD görselleri temsilidir.

20.1 Karakter LCD ve HD44780

Kursun "LCD Introduction" dersi. 16x2 karakter LCD, 2 satırda 16'şar karakter gösteren bir ekrandır. Her karakter 5×8 noktalık bir matristen oluşur. Ekranın arkasındaki küçük kartta bir **HD44780** denetleyicisi (ya da uyumlu bir kopyası: KS0066, SPLC780...) bulunur. Ekranı tarayan, karakterlerin noktalarını üreten ve komutları işleyen odur. Biz sadece denetleyiciye "şu adrese şu karakteri yaz" deriz.

Aynı denetleyici 8x1, 16x1, 20x4, 40x2 gibi farklı boyutlarda ekranlarda da kullanılır. Bu bölümde öğrendiklerin hepsinde geçerlidir; sadece satır adresleri değişir.

Bu bölümdeki bilgiler, kursun reposundaki `Docs_n_datasheets/HD44780.pdf` (Hitachi HD44780U datasheet'i) kaynaklıdır.

20.2 LCD'nin Pinleri

Kursun "16x2 LCD pin details" dersi. 16 pinli standart modülün pinleri:

Pin	Ad	Görevi
1	VSS	GND
2	VDD	Besleme, **5 V**

Pin	Ad	Görevi
3	V0 (VEE)	Kontrast ayarı; bir potansiyometrenin orta ucuna bağlanır
4	RS	Register Select: **0** = komut, **1** = karakter (veri)
5	R/W	**0** = LCD'ye yaz, **1** = LCD'den oku
6	E	Enable: LCD, veri hatlarını bu pinin **düşen kenarında** okur
7–10	D0–D3	Veri hatları (8 bit modunda kullanılır)
11–14	D4–D7	Veri hatları (4 bit modunda sadece bunlar kullanılır)
15	A (LED+)	Arka ışık anodu
16	K (LED−)	Arka ışık katodu

Veri yolu 8 bittir ama HD44780 **4 bit modunda** da çalışabilir: her bayt iki parça (nibble) halinde D4–D7 üzerinden gönderilir. Böylece 11 yerine 6 sinyal (RS, E, D4–D7) yeter. Kurs da bu modu kullanır.

20.3 HD44780'in Belleği: DDRAM, CGROM, CGRAM

Kursun "HD44780 LCD controller Internal Details" ve "Understanding DDRAM, CGRAM, CGROM" dersleri. Denetleyicinin içinde üç bellek vardır:

Bellek	Boyut	Görevi
DDRAM (Display Data RAM)	80 bayt	Ekranda gösterilecek karakterlerin **kodları**. Ekrana bir şey yazmak, DDRAM'e bir karakter kodu yazmaktır.
CGROM (Character Generator ROM)	Sabit	Her karakter kodunun 5×8 nokta deseni. `0x41` kodu için `A`'nın deseni buradan gelir. 0x20–0x7D arası ASCII ile aynıdır; üst yarısı ROM sürümüne göre Japonca kana (A00) ya da Avrupa karakterleri (A02) içerir.
CGRAM (Character Generator RAM)	64 bayt	Kullanıcının tanımladığı **8 karakterin** desenleri. Bu karakterler `0x00` – `0x07` (ya da aynı anlama gelen `0x08` – `0x0F`) kodlarıyla gösterilir (20.10).

Ekrana `A` yazdığında olan şey: DDRAM'in o anki adresine `0x41` yazılır, adres sayacı (AC) bir artar. Denetleyici ekranı tararken o hücrede `0x41` görür, deseni CGROM'dan alır ve noktaları yakar.

DDRAM adresleri

2 satır modunda DDRAM'in her satıra 40 baytı düşer, ama 16x2 ekranda her satırın sadece ilk 16'sı görünür:

```
              görünen 16 sütun                  görünmeyen kısım
          ┌──────────────────────────────┬──────────────────────────┐
1. satır  │ 00 01 02 03 ... 0D 0E 0F     │ 10 11 ... 27             │
2. satır  │ 40 41 42 43 ... 4D 4E 4F     │ 50 51 ... 67             │
          └──────────────────────────────┴──────────────────────────┘
```

İkinci satırın `0x10` değil **`0x40`**'tan başladığına dikkat: 1. satırın sonuna 17. karakteri yazarsan 2. satıra geçmez, görünmeyen `0x10` adresine gider. İmleci bir satır ve sütuna götürmek için DDRAM adresini ayarlarsın: 2. satır 3. sütun = `0x40 + 2` = `0x42`, komutu `0x80 | 0x42` = `0xC2` (20.4).

1 satır modunda (`N = 0`) DDRAM tek bir satırdır: `0x00` – `0x4F`.

20.4 Komutlar

Kursun "Understanding LCD command sets" dersleri. RS = 0 iken gönderilen her bayt bir komuttur. Komutun türünü en yüksek 1 biti belirler; altındaki bitler seçeneklerdir (datasheet Table 6):

Komut	D7 D6 D5 D4 D3 D2 D1 D0	Görevi	Süre
Clear display	`0 0 0 0 0 0 0 1`	Ekranı temizler (DDRAM'e boşluk yazar), adresi 0 yapar	1.52 ms
Return home	`0 0 0 0 0 0 1 -`	Adresi 0 yapar, kaydırılmış ekranı geri alır	1.52 ms
Entry mode set	`0 0 0 0 0 1 I/D S`	I/D: yazdıktan sonra adres artsın (1) / azalsın (0). S: ekran da kaysın	37 µs
Display on/off	`0 0 0 0 1 D C B`	D: ekran, C: imleç (alt çizgi), B: yanıp sönen blok	37 µs
Cursor/display shift	`0 0 0 1 S/C R/L - -`	S/C: ekranı (1) ya da imleci (0) kaydır; R/L: sağa (1) / sola (0)	37 µs
Function set	`0 0 1 DL N F - -`	DL: 8 bit (1) / 4 bit (0); N: 2 satır (1) / 1 satır (0); F: 5×10 (1) / 5×8 (0)	37 µs
Set CGRAM address	`0 1 A A A A A A`	Sonraki yazmalar CGRAM'e gider	37 µs
Set DDRAM address	`1 A A A A A A A`	Sonraki yazmalar DDRAM'e gider (imleci taşır)	37 µs

Süreler 270 kHz osilatör için en fazla değerlerdir. Denetleyici bir komutu işlerken yenisini kabul etmez; ya meşgul bayrağı (busy flag, R/W = 1 ile okunur) beklenir ya da her komuttan sonra bu süre kadar beklenir. Biz R/W'yi hep 0 tuttuğumuz için ikinci yolu kullanıyoruz.

Bu bölümün kodunda kullanılan değerler:

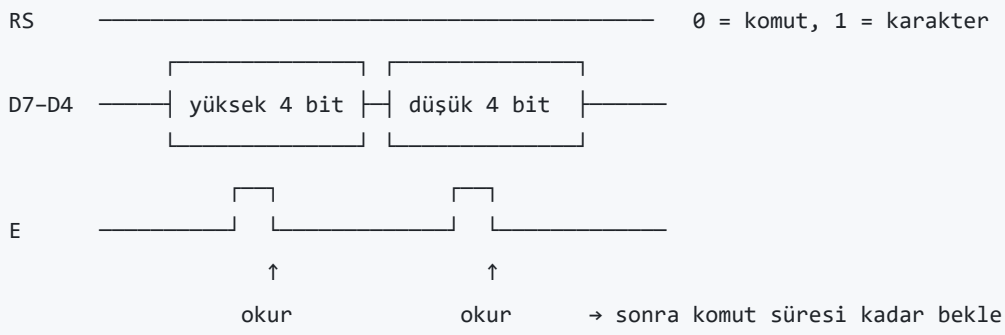
Değer	Anlamı
`0x28`	Function set: 4 bit, 2 satır, 5×8
`0x08`	Ekran kapalı
`0x01`	Temizle
`0x06`	Entry mode: yazdıkça sağa git, ekran kaymasın
`0x0C`	Ekran açık, imleç yok

Değer	Anlamı
`0x04`	Entry mode: yazdıkça sola git
`0x80` + adres	İmleci DDRAM adresine götür (ör. `0xC2`)
`0x40` + adres	CGRAM adresini seç

20.5 4 Bit Modu ve Başlatma

Bir baytın gönderilmesi

4 bit modunda her bayt önce **yüksek** 4 bit, sonra **düşük** 4 bit olarak D4–D7'ye konur. Her parçadan sonra E pini 1 yapılıp 0'a çekilir; LCD veriyi E'nin düşen kenarında okur:



Datasheet'e göre E en az 230–450 ns yüksek kalmalıdır. sysfs üzerinden bir GPIO yazmak bundan çok daha uzun sürdüğü için ayrıca beklemeye gerek yoktur.

Başlatma dizisi

HD44780 açılışta **8 bit modundadır**. Bizim sadece D4–D7 bağlı; D0–D3 boşta. LCD'yi 4 bit moduna geçirmek için datasheet bir dizi verir (Figure 24, "Initializing by Instruction"):

```
besleme geldi
     |  > 40 ms bekle
     ▼
0x3  (D7-D4 = 0011, sadece bir nibble)  "8 bit"     > 4.1 ms bekle
0x3                                     "8 bit"     > 100 µs bekle
0x3                                     "8 bit"
0x2                                     "4 bit"     ← artık 4 bit modundayız
     |
     ▼  bundan sonra her komut iki nibble
0x28  function set: 4 bit, 2 satır, 5×8       satır sayısı ve font sadece burada
0x08  ekran kapalı
0x01  temizle
0x06  entry mode
0x0C  ekran açık
```

Neden üç kez `0x3`? LCD'nin o anki durumunu bilemeyiz:

- Yeni açıldıysa 8 bit modundadır; her `0x3` tek başına "8 bit" komutudur.
- Program daha önce çalışıp yarıda kaldıysa LCD 4 bit modunda ve hatta bir baytın **ortasında** (ilk nibble'ı almış, ikincisini bekliyor) olabilir.

Üç `0x3`, hangi durumda olursa olsun LCD'yi bilinen 8 bit moduna getirir. Ardından gelen `0x2` 4 bit moduna geçirir.

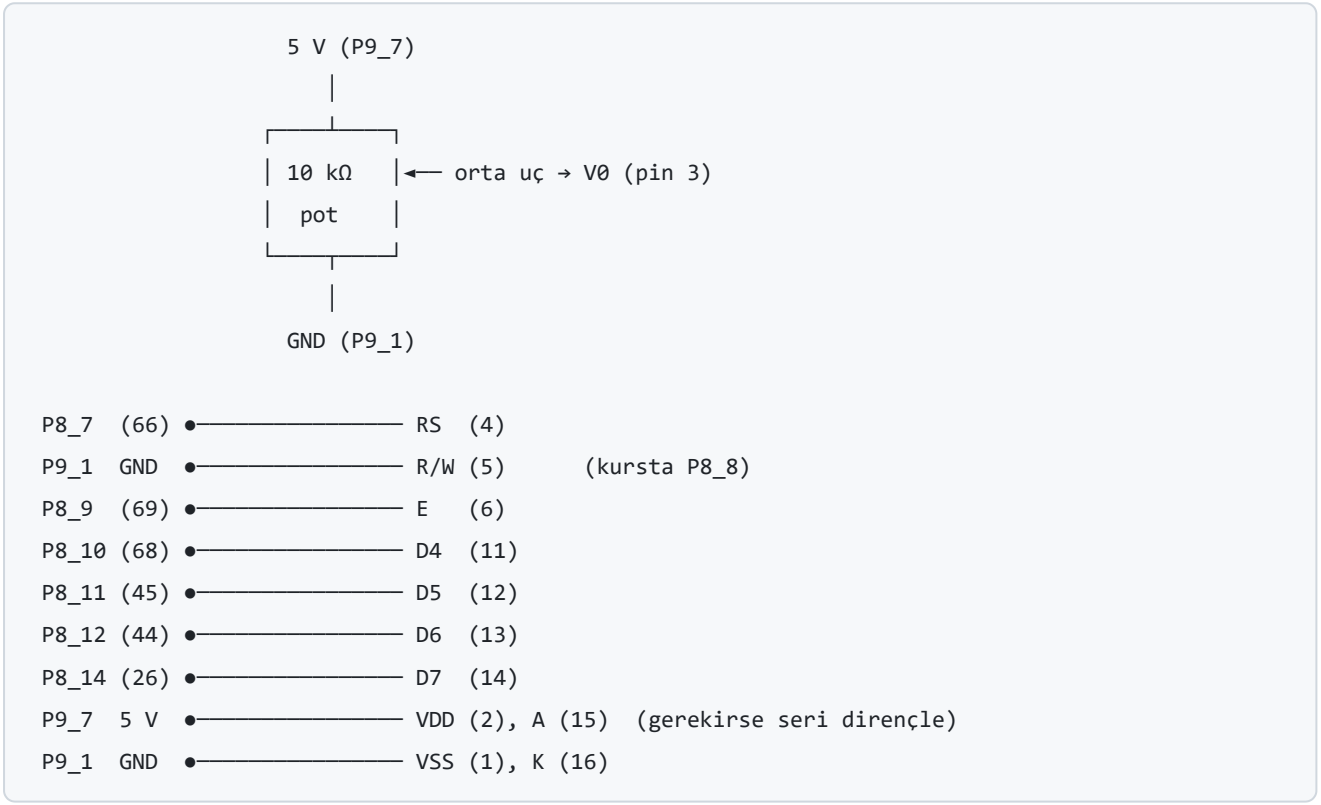
> **Önemli**
>
> Kursun sürücüsü bu diziyi yapmaz; doğrudan `0x28`'i iki nibble olarak gönderir. LCD 8 bit modundayken ilk nibble (`0x2`) tek başına bir komut sayılır ve LCD'yi 4 bit moduna geçirir. İkinci nibble (`0x8`) ise bir sonraki komutun yarısı sanılır. Sonraki bütün komutlar yarım bayt kaymış gider. Bu yüzden kursun koduyla LCD bazen çalışır, bazen anlamsız karakterler gösterir; çoğu zaman programı ikinci kez çalıştırınca düzelir.

20.6 Devre

Kursun "Connecting BBB and LCD" dersi. LCD, Bölüm 17'deki P8 pinlerini kullanır:

BBB pini	GPIO	LCD pini	Not
P9_1	GND	1 (VSS)	Kursun tablosunda "P9.0" yazıyor; böyle bir pin yok
P9_7	SYS_5V	2 (VDD)	5 V
—	—	3 (V0)	10 kΩ potansiyometrenin orta ucu
P8_7	66	4 (RS)	
P8_8	67	5 (R/W)	**GND'ye bağlanması önerilir** (aşağıda)

BBB pini	GPIO	LCD pini	Not
P8_9	69	6 (E)	
P8_10	68	11 (D4)	
P8_11	45	12 (D5)	
P8_12	44	13 (D6)	
P8_14	26	14 (D7)	
—	—	15 (A), 16 (K)	Arka ışık: 5 V ve GND'ye (aşağıda)
—	—	7–10 (D0–D3)	Boş



5 V'luk LCD, 3.3 V'luk GPIO

LCD 5 V ile beslenir ama BBB'nin pinleri 3.3 V'tur. Bu bağlantı iki koşulla güvenlidir:

1. **BBB → LCD yönü**: HD44780, 4.5–5.5 V beslemede girişte 2.2 V'u "1" kabul eder (datasheet, DC Characteristics, V_{IH1}). 3.3 V yeter.
2. **LCD → BBB yönü hiç olmamalı**: R/W = 1 olursa LCD veri hatlarını **5 V** ile sürer ve bu gerilim AM335x'in pinlerine zarar verir. Bu yüzden R/W'yi doğrudan **GND'ye bağla**. Kursta R/W P8_8'e bağlı ve programda 0 yapılıyor; ama kart açılırken, program çalışmadan önce pinin ne durumda olacağı programın elinde değil.

Bu bölümün sürücüsü P8_8'i yine de 0 yapar; kursun bağlantısıyla da çalışır.

Kontrast ve arka ışık

- **Kontrast (V0)**: 10 kΩ potansiyometrenin uçları 5 V ve GND'ye, orta ucu V0'a. Potansiyometre olmadan ya hiçbir şey görünmez ya da bütün noktalar yanar. İlk denemede potansiyometreyi, 1. satırda dolu kareler belirene kadar çevir; LCD başlatılınca kareler kaybolur ve yazı görünür.
- **Arka ışık (A, K)**: Bir LED'dir ve 10–30 mA çeker. Kursun başlık dosyası A'yı GPIO 46'ya, K'yı GPIO 48'e bağlıyor; ama programlar bu pinleri hiç sürmüyor, sürselerdi de bir GPIO bu akımı veremez (16.1). A'yı 5 V'a, K'yı GND'ye bağla. Çoğu modülde akım sınırlayıcı direnç kartın üzerindedir; modülünün datasheet'inde yoksa A'ya seri ~100–220 Ω koy. Arka ışığı programdan açıp kapatmak istersen K'yı bir NPN transistör (BC547) üzerinden GND'ye bağla ve transistörün bazını ~1 kΩ ile bir GPIO'ya ver.

> **Uyarı**
>
> Kabloları kart **kapalıyken** bağla. VDD'yi P9_7/P9_8 (SYS_5V) ya da P9_5/P9_6'ya (VDD_5V, sadece DC jack'ten beslenirken) bağla. 5 V'u hiçbir GPIO pinine değdirme.

20.7 Kursun Sürücüsü ve Düzeltmeler

Kursun LCD kütüphanesi `Project_Src/Drivers/lcd/lcd_driver.{c,h}` ve onu kullanan üç program var: `lcd_text` (mesaj, IP, saat), `lcd_scrollable_text` (kayan yazı) ve `lcd_news` (internetten haber başlıkları). Sorunlar:

Sorun	Sonucu
4 bit başlatma dizisi (3-3-3-2) yok, açılıştaki 40 ms bekleme yok	Kart açıldıktan sonraki ilk çalıştırmada ya da program yarıda kaldıktan sonra anlamsız karakterler (20.5)
Komut gönderdikten sonra beklenmiyor (temizle 1.52 ms sürer)	E darbesi 2 ms tutulduğu için tesadüfen çalışıyor; her karakter ~9 ms sürüyor
`lcd_print_string()` `do { } while` kullanıyor	Boş string'te `'\0'` karakterini yazar ve string'in **dışını** okumaya devam eder
`lcd_printf()` 32 baytlık tampona `vsprintf()` ile yazıyor	Uzun metin tamponu taşırır
`lcd_text` : `lcd_printf(msg_buf)`	Kullanıcının mesajı biçim string'i olarak kullanılıyor: mesajda `%s` varsa program çöker (format string açığı)
`lcd_text` : `strncpy(msg_buf, argv[1], 32)` , tampon 32 bayt	32 karakter ve üstü mesajda string sonlanmaz
`lcd_text` : `tm_mon` olduğu gibi yazılıyor, sıfır dolgusu yok	Ay bir eksik görünür (`tm_mon` 0'dan başlar), `2026-9-5` gibi. Koddaki TODO: "LCD is not displaying correct time & date"
`lcd_text` : IP adresi sabit `usb0` arayüzünden, hatalar kontrol edilmeden	Kart Ethernet'teyse ya da `usb0` yoksa çöp ya da `0.0.0.0`

Sorun	Sonucu
`lcd_scrollable_text` : `lcd_send_command(LCD_CMD_CLEAR_DISPLAY yo);`	Program **derlenmez**
`lcd_scrollable_text` : başlatmadan sonra 1 satır moduna geçmeye çalışıyor	Datasheet: satır sayısı ve font function set dışında değiştirilemez; komut yok sayılır
`lcd_load_cgram()` CGRAM adresini ayarlamıyor	Çağıran önce `0x40` göndermezse desenler DDRAM'e, yani ekrana yazılır
Arka ışık GPIO'lara bağlı	GPIO arka ışığın akımını veremez (20.6)

> **Not**
>
> `lcd_news` programı `newsapi.org` 'un eski v1 uç noktasına, kodun içine gömülü bir API anahtarıyla bağlanıyor ve libcurl'ü cross-compile etmeyi gerektiriyor. API'nin bu sürümü güncel değil; bu yüzden program kitaba alınmadı.

20.8 Düzeltilmiş Sürücü

`code/20_lcd/lcd_driver.c` , `lcd_driver.h` . Fonksiyon adları kursla aynı:

Fonksiyon	Yaptığı iş
`lcd_init()`	GPIO'ları export edip çıkış yapar, 4 bit başlatma dizisini uygular. Başarılıysa 0
`lcd_send_command(cmd)`	Komut gönderir (RS = 0) ve komutun süresi kadar bekler
`lcd_print_char(c)`	Bir karakter yazar (RS = 1)
`lcd_print_string(s)`	String yazar
`lcd_printf(fmt, ...)`	`printf` gibi yazar; en fazla 32 karakter, `\n` 'de durur
`lcd_set_cursor(row, col)`	İmleci 1. ya da 2. satıra, 1–16. sütuna götürür
`lcd_clear()`	Ekranı temizler
`lcd_load_cgram(tab, n)`	`n` özel karakteri CGRAM'e yükler (20.10)
`lcd_release()`	Ekranı temizler, GPIO'ları bırakır

Bir baytın gönderilmesi ve başlatma:

```
static void write_nibble(uint8_t nibble)
{
    gpio_write_value(LCD_GPIO_D4, (nibble >> 0) & 1);
    gpio_write_value(LCD_GPIO_D5, (nibble >> 1) & 1);
    gpio_write_value(LCD_GPIO_D6, (nibble >> 2) & 1);
    gpio_write_value(LCD_GPIO_D7, (nibble >> 3) & 1);

    gpio_write_value(LCD_GPIO_EN, 1);
    gpio_write_value(LCD_GPIO_EN, 0);      /* LCD burada okur */
}

static void write_byte(uint8_t value, uint8_t rs)
{
    gpio_write_value(LCD_GPIO_RS, rs);
    write_nibble(value >> 4);
    write_nibble(value & 0x0F);
}

int lcd_init(void)
{
    ...
    usleep(50 * 1000);      /* > 40 ms */

    write_nibble(0x3);
    usleep(4500);           /* > 4.1 ms */
    write_nibble(0x3);
    usleep(150);            /* > 100 µs */
    write_nibble(0x3);
    usleep(CMD_WAIT_US);

    write_nibble(0x2);      /* artık 4 bit */
    usleep(CMD_WAIT_US);

    lcd_send_command(LCD_CMD_FUNCTION_SET | DISPLAY_2_LINES);             /* 0x28 */
    lcd_send_command(LCD_CMD_DISPLAY_CURSOR_ONOFF_CONTROL);               /* 0x08 */
    lcd_clear();                                                          /* 0x01 */
    lcd_send_command(LCD_CMD_ENTRY_MODESET | INC_CURSOR);                 /* 0x06 */
    lcd_send_command(LCD_CMD_DISPLAY_CURSOR_ONOFF_CONTROL | DISPLAY_ON);  /* 0x0C */
    return 0;
}
```

Sürücünün ürettiği nibble dizisi, GPIO yazmalarını kaydeden bir testle kontrol edildi. `lcd_init()` + `lcd_set_cursor(2, 3)` + `lcd_print_string("Hi")` için:

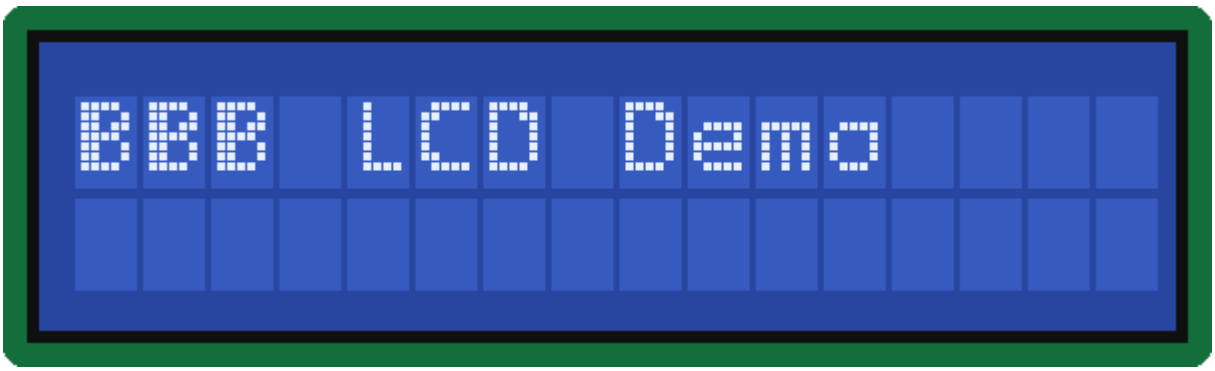
```
komut: 3 3 3 2 | 2 8 | 0 8 | 0 1 | 0 6 | 0 C | C 2      veri: 4 8 | 6 9
       başlatma  0x28  0x08  0x01  0x06  0x0C  0xC2           'H'   'i'
```

20.9 Örnek Programlar

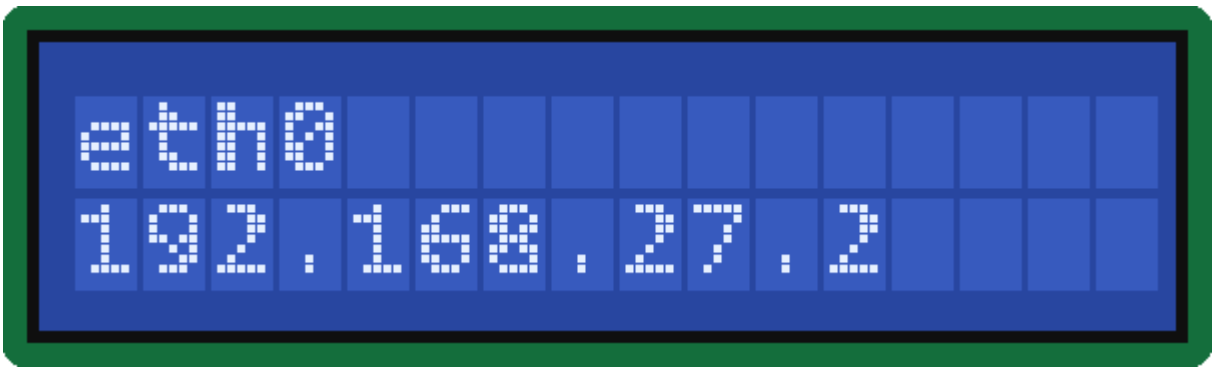
lcd_text: mesaj, IP adresi, tarih ve saat

`code/20_lcd/lcd_text.c`, kursun `lcd_text` programının düzeltilmiş hali. Sırayla şunları gösterir, aralarında kursun yıldızlı geçiş efektiyle:

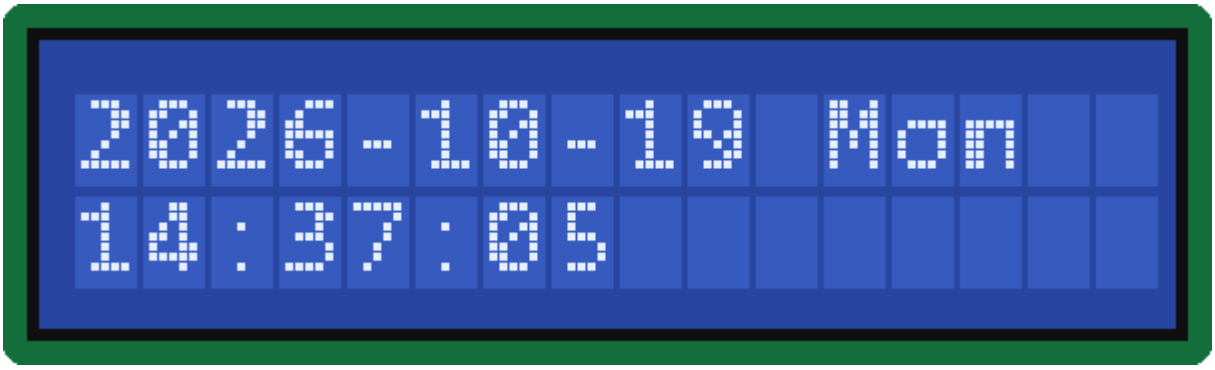
1. Başlık:



2. Çalışan ilk IPv4 arayüzünün adı ve adresi (`getifaddrs()` ile; kursta sabit `usb0`):



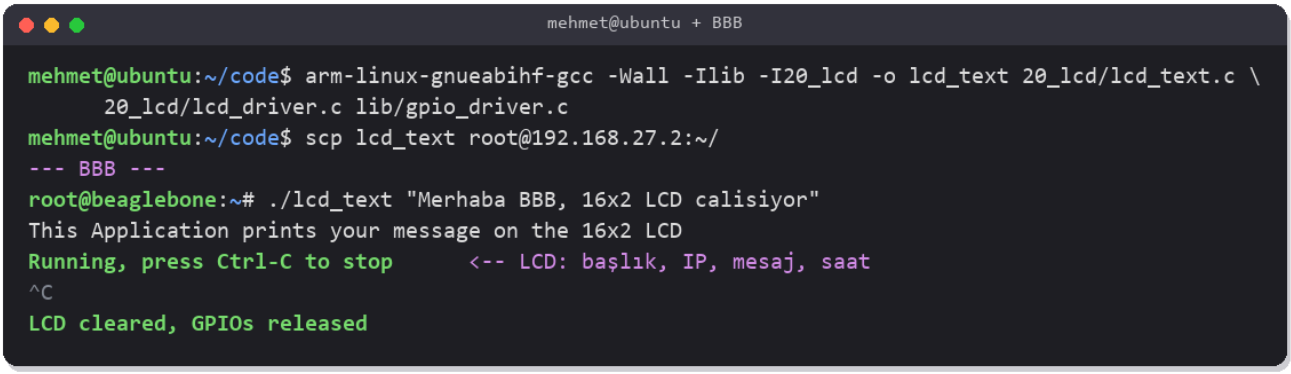
3. Komut satırından verilen mesaj; 16 karakterden uzunsa 2. satıra devam eder.
4. Tarih ve saat, 5 saniye boyunca her saniye güncellenir:



Tarih ve saat `strftime()` ile biçimlendirilir; ay ve sıfır dolgusu doğru çıkar, saat dilimi sistemden gelir (18.7'deki `timedatectl` notu burada da geçerli):

```
strftime(line, sizeof(line), "%Y-%m-%d %a", &tm);      /* 2026-10-19 Mon */
...
strftime(line, sizeof(line), "%H:%M:%S", &tm);         /* 14:37:05 */
```

Geçiş efektinin ikinci satırı entry mode'u değiştirerek sağdan sola yazar: `0x04` ile adres her karakterden sonra **azalır**; imleç 2. satırın 16. sütununa konur ve yıldızlar sola doğru dizilir. Sonra `0x06` ile normal yöne dönülür.




```
arm-linux-gnueabihf-gcc -Wall -Ilib -I20_lcd -o lcd_text 20_lcd/lcd_text.c \
      20_lcd/lcd_driver.c lib/gpio_driver.c
scp lcd_text root@192.168.27.2:~/
```

Kartta:

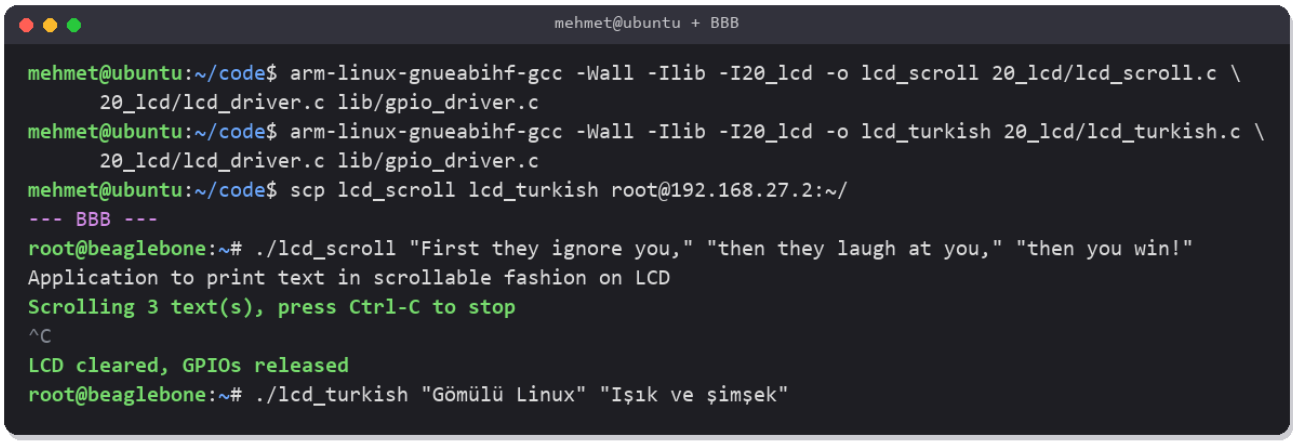
```
./lcd_text "Merhaba BBB, 16x2 LCD calisiyor"     # Ctrl-C ile dur
```

lcd_scroll: kayan yazı

Kursun `lcd_scrollable_text` programında yazılar koda gömülüydü; komut satırından alınmaları öğrenciye bırakılmıştı. `code/20_lcd/lcd_scroll.c` her argümanı 1. satırda sağdan sola kaydırır, 2. satırda kaçıncı yazıda olduğunu gösterir.

HD44780'in kendi kaydırma komutu (`0x18` : ekranı sola kaydır) da var, ama satır başına 40 baytlık DDRAM'i kaydırır: 40 karakterden uzun yazıyı gösteremez ve iki satırı birlikte kaydırır. Kurs bunu aşmak için 1 satır moduna (80 bayt) geçmeye çalışıyordu; HD44780 başlatmadan sonra buna izin vermez (20.4). Bu yüzden kaydırma yazılımla yapılır: her adımda metnin 16 karakterlik bir penceresi 1. satıra yeniden yazılır:

```
for (pos = -LCD_COLUMNS; pos <= len && running; pos++) {
    lcd_set_cursor(1, 1);
    for (i = 0; i < LCD_COLUMNS; i++) {
        c = pos + i;
        lcd_print_char(c >= 0 && c < len ? (uint8_t)text[c] : ' ');
    }
    usleep(STEP_MS * 1000);
}
```




```
arm-linux-gnueabihf-gcc -Wall -Ilib -I20_lcd -o lcd_scroll 20_lcd/lcd_scroll.c \
      20_lcd/lcd_driver.c lib/gpio_driver.c
```

```
./lcd_scroll "First they ignore you," "then they laugh at you," "then you win!"
```

Eclipse kullanıyorsan (Bölüm 14): projeye `20_lcd/lcd_driver.c` ve `lib/gpio_driver.c`'yi ekle, *Includes*'a `lib` ve `20_lcd` klasörlerini yaz.

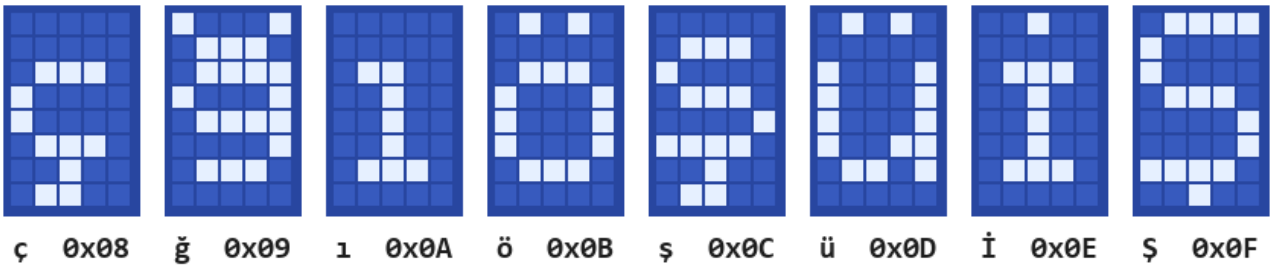
20.10 Özel Karakterler: CGRAM ile Türkçe Harfler

> **Not**
>
> Bu örnek kursta yok; "Understanding DDRAM, CGRAM, CGROM" dersindeki CGRAM'in bir uygulaması.

CGROM'da `ç`, `ğ`, `ı`, `ş`, `İ` yok. CGRAM'e en fazla 8 karakter tanımlanabilir. Her karakter 8 bayttır: her bayt bir satır (üstten alta), baytın düşük 5 biti o satırın noktaları (bit 4 en soldaki nokta):

```
ş:  0x00   . . . . .
    0x0E   . # # # .
    0x10   # . . . .
    0x0E   . # # # .
    0x01   . . . . #
    0x1E   # # # # .
    0x04   . . # . .      8. satır normalde imleç satırıdır;
    0x0C   . # # . .      imleç kapalıyken kullanılabilir (çengel)
```

`code/20_lcd/lcd_turkish.c` küçük `ç ğ ı ö ş ü` ile büyük `İ Ş`'yi tanımlar:

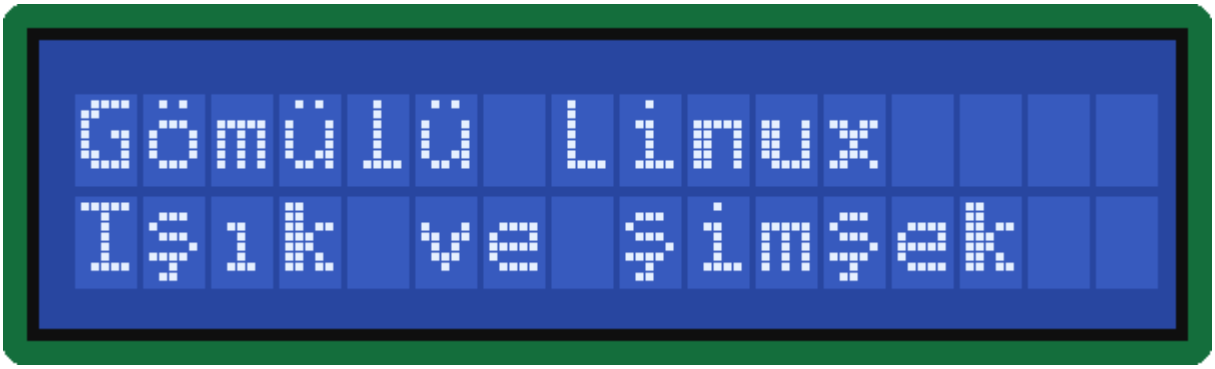


Yüklemek: CGRAM adresini 0 yap (`0x40`), 64 baytı yaz, DDRAM'e dön (`0x80`):

```
void lcd_load_cgram(const uint8_t tab[], uint8_t charnum)
{
    int i;

    lcd_send_command(LCD_CMD_SET_CGRAM_ADDRESS);    /* CGRAM adresi 0 */
    for (i = 0; i < charnum * 8; i++)
        lcd_print_char(tab[i]);

    lcd_send_command(LCD_CMD_SET_DDRAM_ADDRESS);    /* yazmaya DDRAM'den devam */
}
```

Göstermek: tanımlanan karakterler `0x00` – `0x07` kodlarıyla yazdırılır. Datasheet'e göre karakter kodunun 3. biti CGRAM için önemsizdir; `0x08` – `0x0F` aynı karakterleri verir. Biz `0x08` – `0x0F` kullanıyoruz, çünkü `0x00` C'de string'in sonu demektir.

Program UTF-8 metni çözer: Türkçe harfler UTF-8'de 2 bayttır (`ş` = `0xC5 0x9F` → U+015F). Her harfin kod noktası bir tablodan LCD koduna çevrilir. CGRAM'de yer kalmayan `Ç Ğ Ö Ü` için `C G O U` yazılır.




```
arm-linux-gnueabihf-gcc -Wall -Ilib -I20_lcd -o lcd_turkish 20_lcd/lcd_turkish.c \
      20_lcd/lcd_driver.c lib/gpio_driver.c
```

```
./lcd_turkish "Gömülü Linux" "Işık ve şimşek"
```

> **İpucu**
>
> CGRAM, LCD'nin beslemesi kesilene kadar kalır; ama başka bir program LCD'yi başlatınca da silinmez, sadece üzerine yazılabilir. 8 karakter yetmiyorsa ekranda o an gereken harfleri yükleyip değiştirebilirsin; o anda ekranda duran eski karakterler de değişir.

> **Not**
>
> Linux kernel'de HD44780 için hazır bir sürücü de var (`hd44780` , `drivers/auxdisplay/`). Device tree'de `compatible = "hit,hd44780"` ve GPIO'larla tanıtılınca LCD bir karakter aygıtı (`/dev/lcd`) olarak görünür; yazdığın metin ekrana çıkar. Kernel'in bu sürücüyle derlenmiş olması (`CONFIG_HD44780`) ve bir overlay (16.6) gerekir.

20.11 Özet

- 16x2 LCD'nin arkasında bir HD44780 denetleyicisi var. RS = 0 komut, RS = 1 karakter; LCD veriyi E'nin düşen kenarında okur.
- DDRAM ekrandaki karakter kodlarıdır: 1. satır `0x00` , 2. satır `0x40` 'tan başlar. İmleci götürmek: `0x80 | adres` .
- CGROM sabit karakter desenleridir; CGRAM'e 8 özel karakter tanımlanır (`0x40 | adres` ile seçilir, `0x08` – `0x0F` kodlarıyla gösterilir).
- 4 bit modunda her bayt iki nibble. Açılışta datasheet'teki dizi şart: 40 ms bekle, `0x3` , `0x3` , `0x3` , `0x2` , sonra `0x28` . Satır sayısı ve font sadece bu noktada ayarlanır.
- Her komuttan sonra bekle: temizle/başa dön 1.52 ms, diğerleri 37 µs.
- LCD 5 V, GPIO 3.3 V: yazma yönü güvenli (V_{IH} 2.2 V), ama LCD hiç okunmamalı: R/W'yi GND'ye bağla.
- Kontrast için 10 kΩ potansiyometre şart; arka ışığı GPIO'dan değil 5 V'tan besle.

20.12 Karşılaşılabilecek Problemler

```
root@beaglebone: ~ (SSH) (temsili hatalar)
LCD'nin 1. satırı dolu kareler, 2. satır boş  <-- besleme var, LCD başlatılmamış
                                (ya da kontrast çok yüksek)
LCD tamamen boş, arka ışık yanıyor           <-- kontrast (V0) ayarsız
Anlamsız karakterler, her çalıştırmada farklı  <-- D4-D7 sırası yanlış / 4 bit başlatma yok

root@beaglebone:~# ./lcd_text "test"
/sys/class/gpio/gpio66/direction: Permission denied   <-- root olarak çalıştır
GPIO 66 could not be configured as output
Error: LCD init failed
```

Belirti	Sebep	Çözüm
Arka ışık yanıyor, ekranda hiçbir şey yok	Kontrast (V0) ayarsız ya da bağlı değil	Potansiyometreyi çevir; 5 V'luk LCD'lerde V0 genelde 0–1 V arasında iyi görünür
1. satır dolu kareler, 2. satır boş	LCD beslendi ama başlatılmadı; ya da kontrast çok yüksek	Programı çalıştır; kareler kalıyorsa RS/E/D4–D7 bağlantılarını kontrol et
Anlamsız karakterler, her çalıştırmada farklı	4 bit başlatma dizisi yok (kursun kodu) ya da D4–D7 sırası yanlış	Düzeltilmiş sürücü; D4 = pin 11 … D7 = pin 14 (kursun `lcd_text_scroll.c` yorumundaki 7–10 yanlış)
İlk çalıştırmada bozuk, ikincide düzgün (kursun kodu)	LCD'nin modu ve nibble senkronu bilinmiyor	3-3-3-2 dizisi (20.5)
Bazı karakterler eksik, özellikle temizlemeden sonra	Komutlardan sonra yeterince beklenmiyor	Temizle/başa dön sonrası ≥ 1.52 ms, diğerleri ≥ 37 µs
17. karakter 2. satıra geçmiyor	1. satırın devamı DDRAM `0x10` – `0x27`, 2. satır `0x40`'tan başlar	2. satır için `lcd_set_cursor(2, 1)`
Türkçe harfler yerine Japonca karakterler	UTF-8 baytları doğrudan LCD'ye gidiyor	`lcd_turkish`'teki gibi CGRAM'e yükle ve kodları çevir
`Permission denied` / `LCD init failed`	Root değilsin ya da pin başka yerde kullanılıyor	`sudo` ; `cat /sys/kernel/debug/gpio`
`lcd_scrollable_text` derlenmiyor (kursun kodu)	Kaynakta `LCD_CMD_CLEAR_DISPLAY yo` yazım hatası	`lcd_scroll.c`
Kart açılırken LCD'de rastgele karakterler	Pinler açılışta tanımsız; E'de oluşan darbeler LCD'ye bir şey yazar	Normal; program başlayınca `lcd_init()` düzeltir. R/W'nin GND'de olduğundan emin ol

Kısım VII
Kernel Sürücüleri

Bölüm 21 — Kernel Modülü Yazmak

> **Bu bölümde:**
>
> - Kernel modülünün ne olduğunu, kullanıcı programından farkını ve neden modül yazıldığını öğreneceksin.
> - İlk modülünü yazıp kernel ağacının dışında (out-of-tree) cross compile edeceksin.
> - Modülü kartta `insmod`, `lsmod`, `rmmod`, `modinfo` ve `dmesg` ile yükleyip izleyeceksin.
> - Modüle parametre verecek ve bu parametreleri çalışırken sysfs'ten değiştireceksin.
> - Bir modülün dışa açtığı fonksiyonu başka bir modülde kullanacak, bağımlılığı `modprobe`'a çözdüreceksin.
>
> **Ön koşul:** Bölüm 10 — kernel'i ve modülleri derlemek, Bölüm 12 — kendi kernel'in ve BusyBox rootfs'inle (tercihen NFS) açılan kart.

> **Not**
>
> Bu bölüm kursta yok; kitabın kursun ötesine geçen kısmının ilk bölümü. Kart, Bölüm 10'da derlediğin kernel 5.10.162 ile ve Bölüm 12'deki BusyBox rootfs'iyle açılıyor; komutlar kartta root olarak çalıştırılır. Terminal görüntüleri temsilidir.

21.1 Kernel Modülü Nedir?

Şimdiye kadar yazdığımız programlar (LED, 7-segment, LCD) **kullanıcı alanında** (user space) çalıştı. Donanıma kernel'in açtığı dosyalar (`/sys/class/gpio`, `/dev/i2c-2`) üzerinden ulaştılar. Bu dosyaların arkasında kernel'in içindeki **sürücüler** çalışıyordu.

Kernel modülü, kernel çalışırken içine eklenip çıkarılabilen bir kod parçasıdır (`.ko` dosyası, *kernel object*). Yüklendiği anda kernel'in bir parçası olur:

	Kullanıcı programı	**Kernel modülü**
Nerede çalışır?	Kullanıcı alanı, kendi korumalı belleğinde	Kernel alanı, kernel ile aynı bellekte
Başlangıç noktası	`main()`	`module_init()` ile belirtilen fonksiyon
Kütüphane	glibc (`printf`, `malloc` ...)	Sadece kernel'in fonksiyonları (`printk`, `kmalloc` ...)
Hata yaparsa	Program çöker (`Segmentation fault`), sistem çalışmaya devam eder	**Bütün sistem** çökebilir (`Oops`, kernel panic)
Donanıma erişim	Kernel'in açtığı arayüzler üzerinden	Doğrudan (register'lar, kesmeler)

	Kullanıcı programı	Kernel modülü
Çalıştırma	`./program`	`insmod modul.ko`

```
Kullanıcı alanı     ./led_blink  ./lcd_text        (Bölüm 16-20)
─────────────────────── open / read / write / ioctl ─────────────
Kernel alanı        kernel  +  hello.ko  +  gpio-omap  +  i2c-omap ...
                            ▲
                            └── insmod ile eklendi, rmmod ile çıkarılır
```

Neden modül?

- Kernel'i yeniden derleyip kartı yeniden başlatmadan sürücü eklemek ve denemek için. Sürücü geliştirirken döngü "derle → `insmod` → dene → `rmmod`" olur.
- Her donanımın sürücüsünü kernel imajına gömmek imajı büyütür. Dağıtımlar sürücülerin çoğunu modül olarak verir; sadece donanımda olan yüklenir (10.7'deki `m` seçeneği).

> **Uyarı**
>
> Modül kernel'in içinde çalıştığı için bir hata (NULL işaretçi, taşma) bütün sistemi kilitleyebilir. Modülleri geliştirme kartında dene; NFS rootfs ile çalışmak, kart kilitlendiğinde dosyalarının bilgisayarda güvende olması demektir.

21.2 Hazırlık: Hangi Kernel'e Karşı Derleniyor?

Bir modül, **belirli bir kernel için** derlenir. Derlenirken o kernel'in başlık dosyaları, `.config`'i ve sembol listesi (`Module.symvers`) kullanılır. Bu yüzden:

1. Elinde kartta çalışan kernel'in **derlenmiş kaynak ağacı** olmalı: Bölüm 10'daki `~/bbb/linux`. Orada `make ... modules` de çalıştırılmış olmalı; `Module.symvers` dosyası bu adımda üretilir.
2. Kartta **aynı kaynaktan, aynı ayarlarla** derlenmiş kernel çalışmalı. Modülün içine yazılan **vermagic** (kernel sürümü + bazı ayarlar) yükleme anında çalışan kernel'le karşılaştırılır; uyuşmazsa modül reddedilir (21.10).

```
ls ~/bbb/linux/Module.symvers ~/bbb/linux/.config     # ikisi de olmalı
```

Kartta:

```
~ # uname -r
5.10.162
```

> **Not**
>
> **Kartta eMMC'deki Debian (v5.10-ti) çalışıyorsa** modülü Debian'ın kernel'ine karşı derlemen gerekir; Bölüm 10'daki ağaç o kernel'in birebir aynısı değildir. En kolayı kartın kendisinde derlemek: `sudo apt install linux-headers-$(uname -r) build-essential` , sonra bu bölümün `code/` klasörünü karta kopyalayıp orada `make KDIR=/lib/modules/$(uname -r)/build` çalıştır. Debian armhf'te derleyicinin adı da `arm-linux-gnueabihf-gcc` olduğu için Makefile değişmeden çalışır.

21.3 Birinci Modül: `hello.c`

Bölümün kodu: `code/21_kernel_module/` . En küçük modül iki fonksiyondan oluşur: biri yüklenirken, biri çıkarılırken çalışır.

```c
#define pr_fmt(fmt) KBUILD_MODNAME ": " fmt     /* her mesajın başına "hello: " */

#include <linux/init.h>
#include <linux/module.h>
#include <linux/moduleparam.h>
#include <linux/utsname.h>

static char *name = "BeagleBone";
module_param(name, charp, 0444);
MODULE_PARM_DESC(name, "Name to greet");

static int count = 1;
module_param(count, int, 0644);
MODULE_PARM_DESC(count, "How many times to greet (1-10)");

static int __init hello_init(void)
{
    int i;

    if (count < 1 || count > 10) {
        pr_err("count must be 1-10, got %d\n", count);
        return -EINVAL;
    }

    for (i = 0; i < count; i++)
        pr_info("Hello %s! (%d/%d)\n", name, i + 1, count);

    pr_info("running on Linux %s (%s)\n", utsname()->release, utsname()->machine);
    return 0;
}

static void __exit hello_exit(void)
{
    pr_info("Goodbye %s, count was %d\n", name, count);
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Embedded Linux kitabı");
MODULE_DESCRIPTION("First kernel module: hello with parameters");
```

Parça	Anlamı
`module_init(hello_init)`	`insmod` sırasında çağrılacak fonksiyon. `0` **dönerse** modül yüklenir; negatif bir hata kodu (`-EINVAL` , `-ENOMEM` ...) dönerse yükleme iptal olur ve modül bellekten atılır.
`module_exit(hello_exit)`	`rmmod` sırasında çağrılır. Init'te alınan her kaynak (bellek, GPIO, kesme...) burada bırakılmalıdır.
`__init` / `__exit`	Fonksiyonları özel bölümlere koyar. `__init` kodu yükleme bittikten sonra bellekten atılır; `__exit` kodu modül kernel'e gömülü derlenirse hiç derlenmez (gömülü kod çıkarılamaz).
`pr_info()` , `pr_err()`	Kernel log'una yazar: `printk(KERN_INFO ...)` kısaltması (21.8). Çıktı ekrana değil, `dmesg` 'e gider.
`pr_fmt`	Bütün `pr_*` mesajlarının başına eklenecek metin. `KBUILD_MODNAME` derlemede modülün adıyla (`hello`) değiştirilir.
`MODULE_LICENSE("GPL")`	Lisans. GPL dışı bir lisans kernel'i "tainted" (lekeli) yapar ve modül `EXPORT_SYMBOL_GPL` ile açılmış fonksiyonları kullanamaz.
`MODULE_AUTHOR` , `MODULE_DESCRIPTION`	`modinfo` çıktısında görünen bilgiler
`utsname()`	Çalışan kernel'in adı ve sürümü (`uname` komutunun kernel'deki karşılığı)

> **Önemli**
>
> Kernel kodunda `printf` , `malloc` , `<stdio.h>` yoktur; glibc kullanıcı alanının kütüphanesidir. Kernel'in kendi karşılıkları vardır: `printk` / `pr_info` , `kmalloc` / `kfree` , `<linux/...>` başlıkları. Kernel'de kayan nokta (`float` , `double`) da kullanılmaz.

Kernel mesajlarında Türkçe karakter kullanmadık: kernel log'u seri konsola da gider ve orada bozuk görünebilir (8.8'deki U-Boot mesajları gibi).

21.4 Ağaç Dışında Derlemek: Makefile ve kbuild

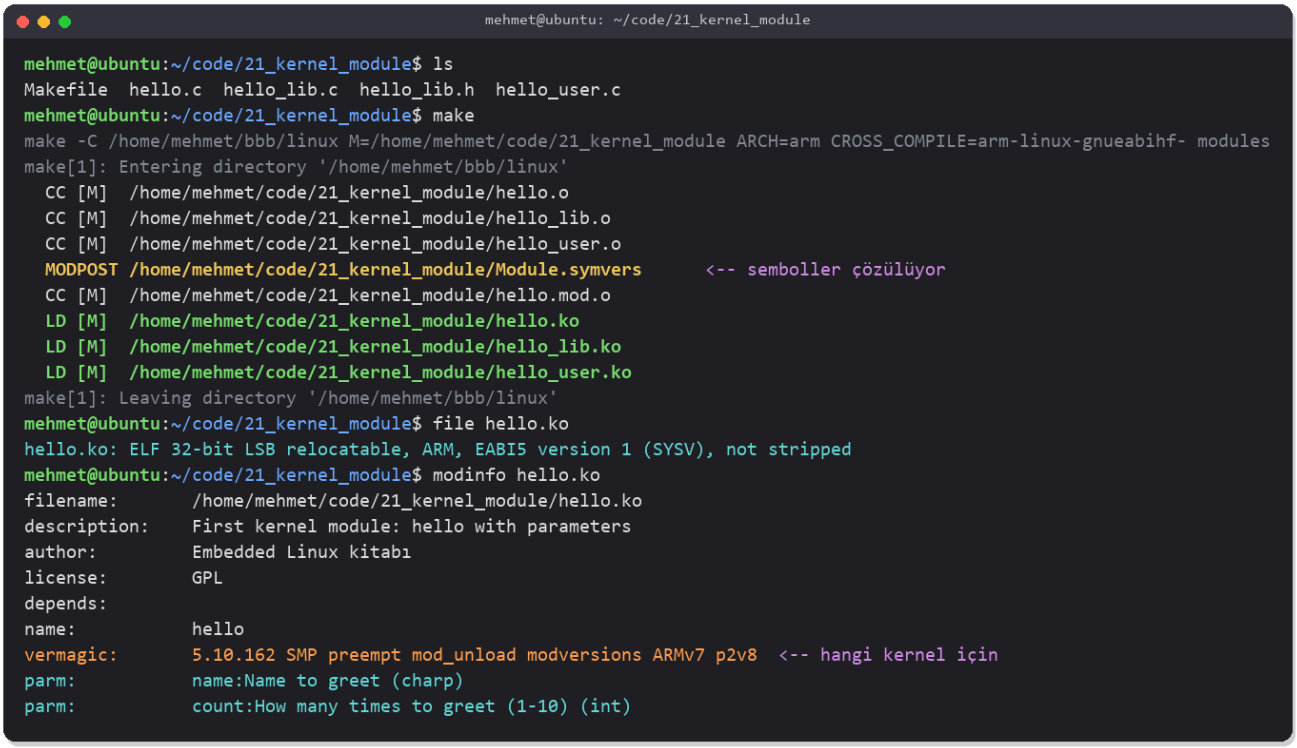
Modül kernel kaynağının içine konmadan, kendi klasöründe derlenir (**out-of-tree**). Derlemeyi yine kernel'in derleme sistemi (**kbuild**) yapar; biz sadece hangi dosyaların modül olacağını söyleriz:

```
ifneq ($(KERNELRELEASE),)

obj-m := hello.o hello_lib.o hello_user.o

else

KDIR ?= $(HOME)/bbb/linux
ARCH ?= arm
CROSS_COMPILE ?= arm-linux-gnueabihf-

KMAKE = $(MAKE) -C $(KDIR) M=$(CURDIR) ARCH=$(ARCH) CROSS_COMPILE=$(CROSS_COMPILE)

all:
    $(KMAKE) modules

clean:
    $(KMAKE) clean

install:
    $(KMAKE) INSTALL_MOD_PATH=$(INSTALL_MOD_PATH) modules_install

endif
```

Parça	Anlamı
`obj-m := hello.o`	`hello.c`'den `hello.ko` modülü üret (`m` = modül; kernel'e gömülü olsaydı `obj-y`)
`-C $(KDIR)`	`make`'i kernel kaynak klasöründe çalıştır; derleme kuralları, `.config` ve başlıklar oradan gelir
`M=$(CURDIR)`	Derlenecek modülün klasörü (bu klasör)
`ARCH`, `CROSS_COMPILE`	Kernel'i derlerken verdiğin değerlerin aynısı (10.8)
`ifneq ($(KERNELRELEASE),)`	Makefile iki kez okunur: önce senin `make` komutunla, sonra kbuild tarafından. kbuild okurken `KERNELRELEASE` tanımlıdır ve sadece `obj-m` kısmı kullanılır.

> **İpucu**
>
> Kernel kaynağın başka bir yerdeyse `make KDIR=/yol/linux` yaz. Bir modül birden fazla `.c` dosyasından oluşacaksa: `obj-m := surucu.o` ve `surucu-objs := a.o b.o`.

Derle




```
cd ~/code/21_kernel_module        # repodaki code/21_kernel_module klasörü
make
file hello.ko
modinfo hello.ko
```

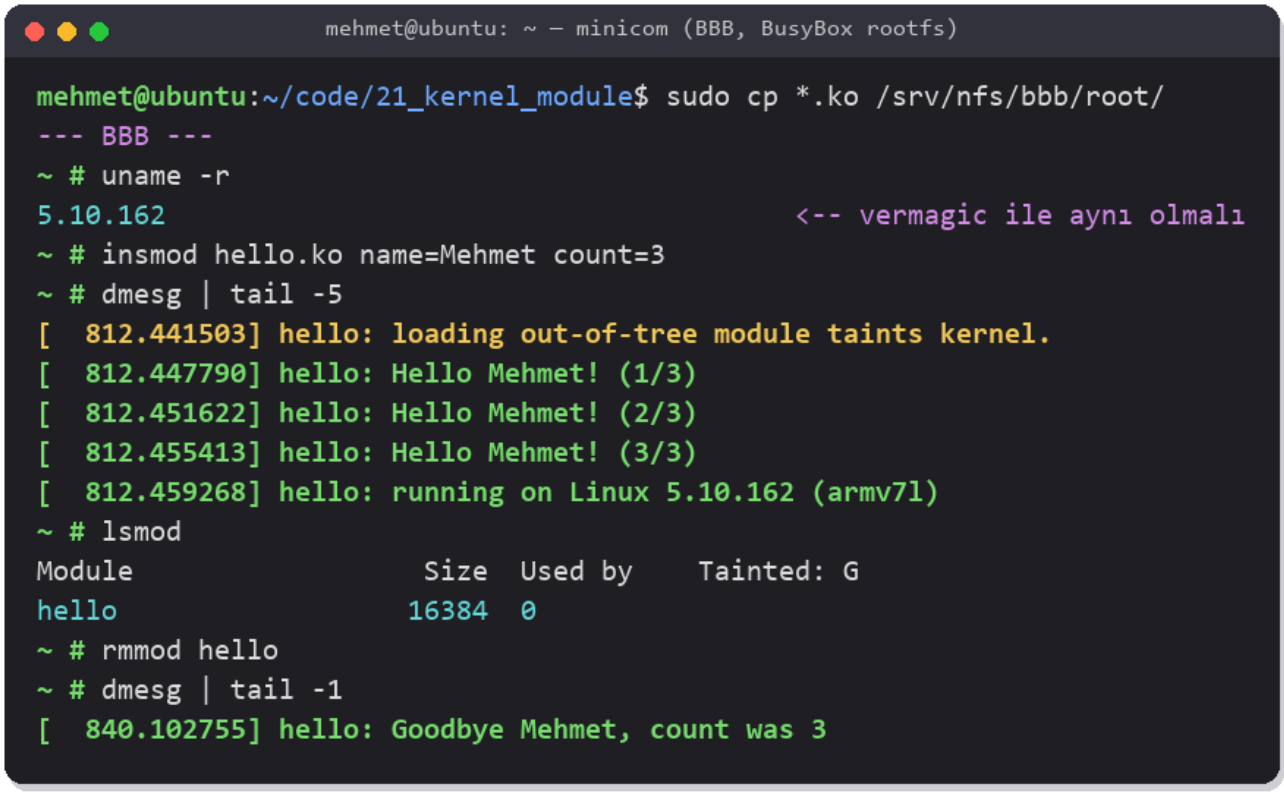
Çıktıdaki adımlar:

Satır	Ne oluyor?
`CC [M] hello.o`	C kodu ARM için derleniyor (`[M]` = modül)
`MODPOST Module.symvers`	Modülün kullandığı kernel fonksiyonları kernel'in `Module.symvers` 'ındaki listeyle eşleştiriliyor. Kernel'de olmayan bir fonksiyon kullanırsan uyarı burada çıkar.
`CC [M] hello.mod.o`	Modülün kimlik bilgileri (vermagic, bağımlılıklar, parametreler) ayrı bir dosyada derleniyor
`LD [M] hello.ko`	Hepsi `hello.ko` 'da birleştiriliyor

`modinfo` host'ta da çalışır ve modülün içine gömülü bilgileri gösterir. En önemlisi `vermagic` : modülün hangi kernel için derlendiği.

21.5 Kartta Yüklemek: insmod, lsmod, dmesg, rmmod

Rootfs NFS'ten geliyorsa (12.8) modülleri NFS klasörüne kopyalamak yeterli; kart hemen görür. SD kart kullanıyorsan `scp` ya da SD kartı bilgisayara takıp kopyala.



Host'ta:

```
sudo cp *.ko /srv/nfs/bbb/root/
```

Kartta:

```
~ # insmod hello.ko name=Mehmet count=3
~ # dmesg | tail -5
~ # lsmod
~ # rmmod hello
~ # dmesg | tail -1
```

Komut	Görevi
`insmod dosya.ko [parametre=değer ...]`	Modülü verilen dosyadan yükler. Bağımlılıklara bakmaz.
`lsmod`	Yüklü modüller, boyutları ve kaç modül/kullanıcının onları kullandığı (`/proc/modules` 'un okunaklı hali)
`rmmod ad`	Modülü çıkarır (`.ko` değil, modülün **adı** yazılır)
`modinfo dosya.ko`	Modülün bilgileri. Host'taki `modinfo` ARM modüllerini de okur; BusyBox'ın `modinfo` 'su daha sınırlıdır, host'ta kullan.
`dmesg`	Kernel log'u. `pr_info()` mesajları burada.

`dmesg`'teki ilk satır dikkat çekici: `loading out-of-tree module taints kernel.` Kernel, kendi kaynağında olmayan bir modül yüklendiğini not eder (**taint**, "leke"). Bu bir hata değildir; kernel bir hata raporu (Oops) basarsa geliştiricilere "sisteme dışarıdan kod yüklenmişti" bilgisini verir. `lsmod`'daki `Tainted: G` ve `/proc/sys/kernel/tainted` de bunu gösterir.

Modül yüklüyken `/sys/module/hello/` klasörü oluşur. Kernel her modül için burada bilgi açar: `refcnt` (kaç kullanıcısı var), `parameters/` (21.6), `sections/`.

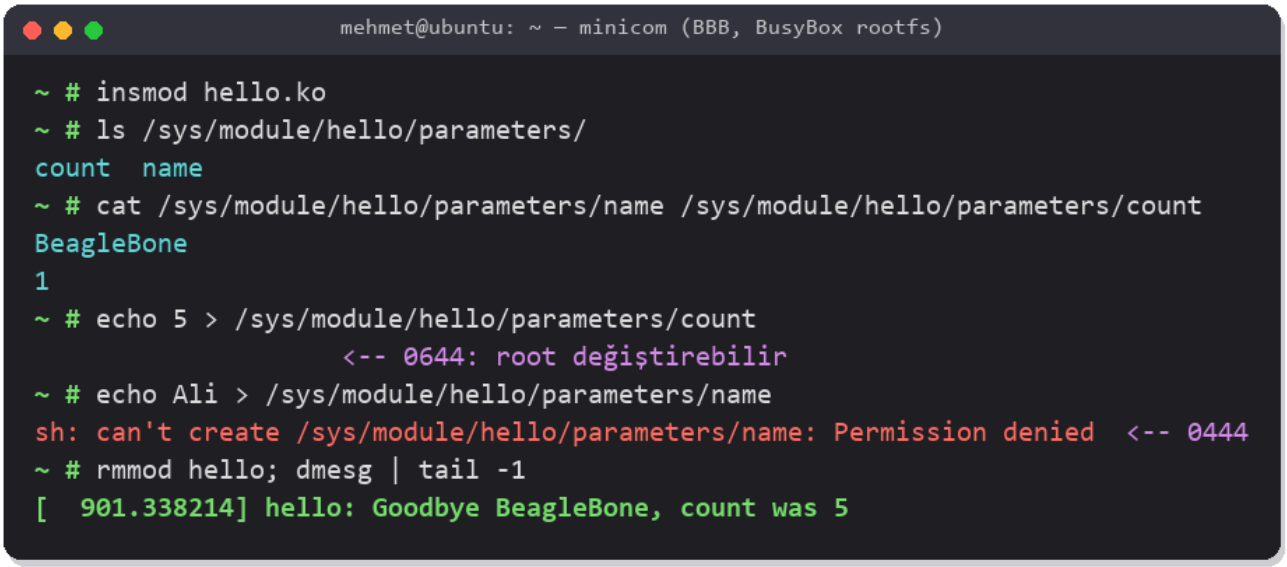
21.6 Modül Parametreleri

Kullanıcı programına komut satırı argümanı verir gibi modüle de değer verebilirsin. `module_param(isim, tür, izin)`:

Tür	C türü
`int`, `uint`, `long`, `ulong`, `short`, `ushort`	Tamsayılar
`bool`, `invbool`	`true`/`false` (`1`/`0`, `y`/`n`)
`charp`	Metin (`char *`); kernel değeri kendi belleğine kopyalar

Üçüncü argüman, parametrenin `/sys/module/<modül>/parameters/` altındaki dosyasının izinleridir:

İzin	Anlamı
`0`	sysfs'te dosya yok; değer sadece yüklerken verilir
`0444`	Herkes okuyabilir, kimse değiştiremez (`name`)
`0644`	Herkes okur, root çalışırken değiştirebilir (`count`)



```
~ # insmod hello.ko
~ # cat /sys/module/hello/parameters/name /sys/module/hello/parameters/count
~ # echo 5 > /sys/module/hello/parameters/count
~ # echo Ali > /sys/module/hello/parameters/name        # Permission denied: 0444
~ # rmmod hello; dmesg | tail -1                         # count was 5
```

> **Önemli**
>
> sysfs'ten yazılan değer değişkene **hemen** yazılır ama modül bundan haberdar edilmez. `count` init'te kontrol edildi; çalışırken `echo 50 > count` yazılırsa bu kontrol tekrar yapılmaz. Değerin değişince bir şey tetiklemesi ya da doğrulanması gerekiyorsa `module_param_cb()` ile kendi set fonksiyonunu yazarsın.

Modül kernel'e gömülü derlenmişse aynı parametreler kernel komut satırından (`bootargs`) `modül.parametre=değer` biçiminde verilir (ör. `hello.count=3`).

21.7 Modüller Arası Sembol Paylaşımı ve modprobe

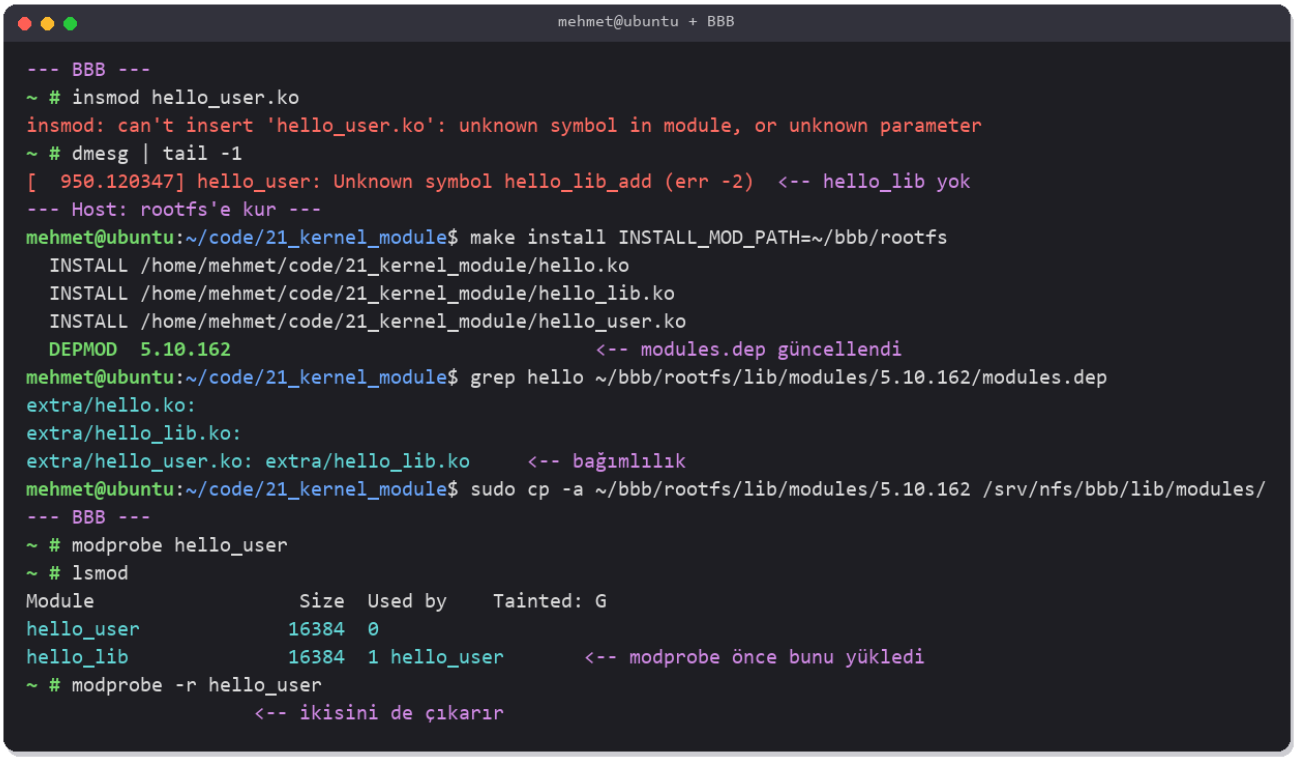
Kernel'in fonksiyonları modüllere kendiliğinden açık değildir; sadece **`EXPORT_SYMBOL()`** ya da **`EXPORT_SYMBOL_GPL()`** ile işaretlenenler kullanılabilir. Bir modül de kendi fonksiyonunu böyle dışa açabilir. Örnekte iki modül var:

Modül	Ne yapar?
`hello_lib.c`	`hello_lib_add()` fonksiyonunu `EXPORT_SYMBOL_GPL` ile dışa açar
`hello_user.c`	Yüklenirken `hello_lib_add(2, 3)` 'ü çağırır

```
/* hello_lib.c */
int hello_lib_add(int a, int b)
{
    calls++;
    pr_info("hello_lib_add(%d, %d) called, %d call(s) so far\n", a, b, calls);
    return a + b;
}
EXPORT_SYMBOL_GPL(hello_lib_add);

/* hello_user.c */
static int __init hello_user_init(void)
{
    pr_info("2 + 3 = %d\n", hello_lib_add(2, 3));
    return 0;
}
```

`hello_user.ko`, `hello_lib.ko`'ya **bağımlıdır**: `modinfo hello_user.ko` çıktısında `depends: hello_lib` yazar. `insmod` bağımlılıklara bakmaz; `hello_lib` yüklü değilse `Unknown symbol` hatası verir. Bağımlılıkları `modprobe` çözer:

1. Modüller `/lib/modules/$(uname -r)/` altına kurulur.
2. `depmod` bütün modüllerin bağımlılıklarını `modules.dep` dosyasına yazar.
3. `modprobe hello_user` bu dosyaya bakar, önce `hello_lib`'i, sonra `hello_user`'ı yükler. `modprobe -r` tersini yapar.



Host'ta, modülleri Bölüm 12'deki rootfs klasörüne kur ve NFS klasörüne kopyala:

```
make install INSTALL_MOD_PATH=~/bbb/rootfs
grep hello ~/bbb/rootfs/lib/modules/5.10.162/modules.dep
sudo cp -a ~/bbb/rootfs/lib/modules/5.10.162 /srv/nfs/bbb/lib/modules/
```

`modules_install`, dışarıdan derlenen modülleri `lib/modules/<sürüm>/extra/` altına koyar ve `depmod`'u kendisi çalıştırır. `make install`'ı **`sudo` olmadan** çalıştır: `sudo` altında `HOME` ve `PATH` değişir; Makefile kernel kaynağını (`$(HOME)/bbb/linux`) ve cross derleyiciyi bulamaz.

Kartta:

```
~ # insmod hello_user.ko        # Unknown symbol: hello_lib yüklü değil
~ # modprobe hello_user         # önce hello_lib, sonra hello_user
~ # lsmod
~ # modprobe -r hello_user      # ikisini de çıkarır
```

`lsmod`'da `hello_lib`'in "Used by" sütununda `1 hello_user` görünür. Bir modülü kullanan başka modül varken onu `rmmod` ile çıkaramazsın.

> **İpucu**
>
> Bir modülün açılışta otomatik yüklenmesi için 12.7'deki `S02module` betiğine `modprobe hello_user` satırını eklemen yeterli.

21.8 printk ve Log Seviyeleri

`pr_info()` gibi makrolar `printk()`'ya bir **log seviyesi** ekler:

Makro	Seviye	Ne zaman?
`pr_emerg`	0 `KERN_EMERG`	Sistem kullanılamaz
`pr_alert`	1 `KERN_ALERT`	Hemen müdahale gerekli
`pr_crit`	2 `KERN_CRIT`	Kritik durum
`pr_err`	3 `KERN_ERR`	Hata
`pr_warn`	4 `KERN_WARNING`	Uyarı
`pr_notice`	5 `KERN_NOTICE`	Önemli ama normal
`pr_info`	6 `KERN_INFO`	Bilgi
`pr_debug`	7 `KERN_DEBUG`	Hata ayıklama; sadece dosyanın başında `#define DEBUG` varsa (ya da dinamik debug açıksa) derlenir

Bütün mesajlar `dmesg`'e gider. Hangilerinin **seri konsola da** basılacağını konsol log seviyesi belirler: seviyesi bu değerden küçük olanlar konsolda da görünür.

```
~ # cat /proc/sys/kernel/printk       # ilk sayı: konsol log seviyesi
~ # dmesg -n 8                        # her şeyi konsolda da göster
~ # dmesg -n 1                        # konsolu sustur (sadece panik mesajları)
~ # dmesg -c                          # log'u gösterip temizle
```

Kernel komut satırına (`bootargs`) `loglevel=8` eklemek açılıştan itibaren aynı etkiyi yapar; `quiet` ise konsolu susturur.

> **İpucu**
>
> Modül yüklerken mesajları görmek için seri konsolda `dmesg -n 8` yapabilir ya da ikinci bir terminalde (SSH) `dmesg -w` ile log'u canlı izleyebilirsin.

21.9 Özet

- Kernel modülü (`.ko`) kernel çalışırken eklenip çıkarılan koddur; kernel ile aynı bellekte, aynı yetkiyle çalışır. Hatası bütün sistemi çökertebilir.
- En küçük modül: `module_init()` (0 dönerse yüklenir), `module_exit()` , `MODULE_LICENSE("GPL")` . Çıktı `pr_info()` ile `dmesg` 'e yazılır.
- Out-of-tree derleme: `obj-m := x.o` + `make -C <kernel ağacı> M=$(PWD) ARCH=arm CROSS_COMPILE=... modules` . Ağaç, kartta çalışan kernel'in derlenmiş kaynağı olmalı (vermagic).
- Kartta: `insmod x.ko param=değer` , `lsmod` , `rmmod x` , `dmesg` . Parametreler `/sys/module/x/parameters/` altında.
- `EXPORT_SYMBOL[_GPL]` ile modüller birbirinin fonksiyonunu kullanır. `modules_install` + `depmod` → `modprobe` bağımlılıkları kendisi yükler.
- Log seviyeleri `pr_err` … `pr_debug` ; konsolda neyin görüneceğini `dmesg -n` belirler.

21.10 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom (BBB, BusyBox rootfs) (temsili hatalar)

~ # insmod hello.ko
insmod: can't insert 'hello.ko': invalid module format
~ # dmesg | tail -1
hello: version magic '5.10.168-ti-r71 SMP preempt mod_unload modversions ARMv7 p2v8'
       should be '5.10.162 SMP preempt mod_unload modversions ARMv7 p2v8'
            <-- modül başka bir kernel için derlenmiş

~ # insmod hello.ko count=20
insmod: can't insert 'hello.ko': Invalid argument
[ 1022.517008] hello: count must be 1-10, got 20  <-- init -EINVAL döndü

~ # rmmod hello_lib
rmmod: can't unload module 'hello_lib': Resource temporarily unavailable
            <-- hello_user hâlâ kullanıyor (Used by 1)
```

Belirti	Sebep	Çözüm
`insmod: can't insert ...: invalid module format` , `dmesg` 'te `version magic '...' should be '...'`	Modül, kartta çalışan kernel'den farklı bir kernel için derlenmiş	Kartta çalışan kernel'in kaynağıyla derle (21.2); kernel'i yeniden derlediysen kartı da yeni kernel'le aç
`invalid module format` , vermagic doğru	Modül x86 için derlenmiş (`ARCH` / `CROSS_COMPILE` unutuldu)	`file hello.ko` → ARM yazmalı
`make: *** /home/.../bbb/linux: No such file or directory`	`KDIR` yanlış	`make KDIR=<kernel kaynağının yolu>`
`WARNING: Module.symvers is missing`	Kernel ağacında `make modules` hiç çalıştırılmamış	Kernel klasöründe `make ARCH=arm CROSS_COMPILE=... modules` (10.8)

Belirti	Sebep	Çözüm
`insmod: ... unknown symbol in module`, `dmesg`'te `Unknown symbol xxx`	Kullanılan fonksiyonu açan modül yüklü değil ya da fonksiyon dışa açılmamış	`modprobe` kullan; kendi kodunda `EXPORT_SYMBOL` eksik mi bak
`Unknown symbol`, ama sembolü açan modül yüklü	Modülün lisansı GPL değil; `EXPORT_SYMBOL_GPL` ile açılan semboller ona görünmez	`MODULE_LICENSE("GPL")`
`insmod: ... Invalid argument`	Init fonksiyonu `-EINVAL` döndü (ör. `count=20`) ya da parametre türü yanlış	`dmesg`'teki mesaja bak
`insmod: ... File exists`	Modül zaten yüklü	`lsmod`; önce `rmmod`
`rmmod: can't unload module ...: Resource temporarily unavailable`	Modülü başka bir modül ya da açık bir dosya kullanıyor	`lsmod`'da "Used by"a bak; önce kullananı çıkar ya da `modprobe -r`
`modprobe: module hello_user not found in modules.dep`	Modül kurulmamış ya da `depmod` çalışmamış	21.7'deki `make install`; kartta `depmod`
`pr_info()` mesajları konsolda görünmüyor	Konsol log seviyesi düşük	`dmesg` ile oku ya da `dmesg -n 8`
`module verification failed: signature and/or required key missing - tainting kernel`	Kernel imzalı modül bekliyor ama zorunlu kılmıyor	Uyarıdır, modül yüklenir

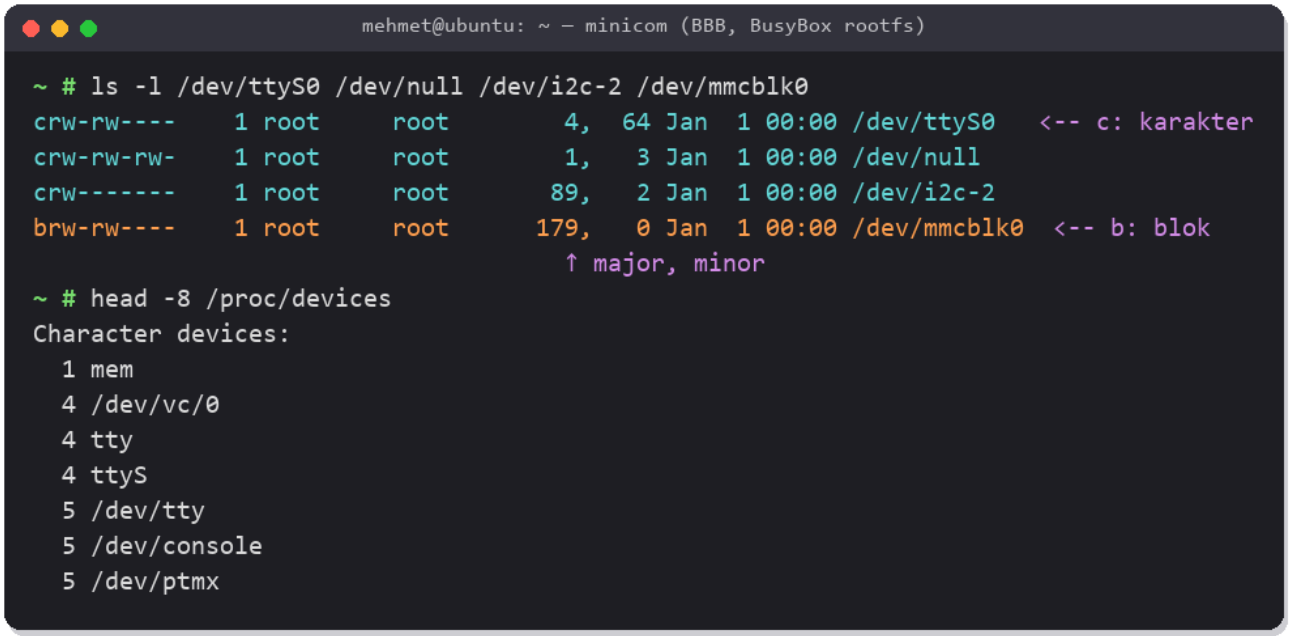
Bölüm 22 — Karakter Aygıt Sürücüsü

> **Bu bölümde:**
>
> - `/dev` altındaki dosyaların ne olduğunu, karakter ve blok aygıtlarını, major/minor numaralarını öğreneceksin.
> - Bir programın `open()` / `read()` / `write()` çağrısının sürücüdeki fonksiyonlara nasıl ulaştığını göreceksin.
> - `/dev/bbbmsg` adında, kendiliğinden oluşan bir aygıt dosyası açan karakter sürücüsü yazacaksın.
> - Kullanıcı belleğiyle güvenli veri alışverişini (`copy_to_user` , `copy_from_user`), dosya konumunu, mutex'i ve `ioctl` 'ü öğreneceksin.
> - Sürücüyü kabuktan ve bir C programından deneyeceksin.
>
> **Ön koşul:** Bölüm 21 — modül derleme, `insmod` / `rmmod` , `dmesg` .

> **Not**
>
> Bu bölüm kursta yok. Kart, Bölüm 21'deki gibi kendi kernel'in (5.10.162) ve BusyBox rootfs'inle açılıyor; komutlar kartta root olarak çalıştırılır. Terminal görüntüleri temsilidir.

22.1 /dev Altındaki Dosyalar

Önceki bölümlerde `/dev/ttyS0` 'a (seri port), `/dev/i2c-2` 'ye (Bölüm 19) ve `/dev/mmcblk0` 'a (SD kart) eriştik. Bunlar diskte veri tutan dosyalar değildir; **aygıt dosyalarıdır** (device node). Bir programın bu dosyaya yaptığı her `open` , `read` , `write` , `ioctl` çağrısı kernel'de o aygıtın **sürücüsüne** gider.



```
~ # ls -l /dev/ttyS0 /dev/null /dev/i2c-2 /dev/mmcblk0
~ # head -8 /proc/devices
```

`ls -l` çıktısının ilk harfi aygıtın türünü, boyut sütunundaki iki sayı **major** ve **minor** numaralarını gösterir:

Tür	Harf	Nasıl erişilir?	Örnek
Karakter aygıtı	`c`	Bayt bayt, sırayla (akış gibi)	Seri port, `/dev/null`, `/dev/i2c-*`, `/dev/gpiochip*`
Blok aygıtı	`b`	Sabit boyutlu bloklar halinde, rastgele; arada kernel'in önbelleği var	SD kart, eMMC, USB bellek
Ağ aygıtı	—	`/dev`'de dosyası yok; soketlerle	`eth0`, `usb0`

Numara	Anlamı
Major	Hangi **sürücü**? Kernel, aygıt dosyası açılınca bu numaraya bakarak sürücüyü bulur. `/proc/devices` hangi major'ın hangi sürücüye ait olduğunu listeler.
Minor	Sürücünün yönettiği aygıtlardan **hangisi**? Ör. `ttyS0` = 4/64, `ttyS1` = 4/65: aynı sürücü, farklı UART.

Bu bölümde kendi karakter aygıtımızı yazacağız. Gerçek bir donanım sürmüyor: aygıta yazılan metni kernel belleğinde saklıyor ve okununca geri veriyor. Böylece sürücünün parçalarına donanım ayrıntısı karışmadan bakabiliriz; Bölüm 23'te device tree'den tanıtılan gerçek bir LED'i ve butonu süren bir sürücü yazacağız.

22.2 Bir read() Çağrısının Yolculuğu

```
Kullanıcı alanı      cat /dev/bbbmsg
                        │  read(fd, buf, 4096)
─────────────────────────┼──────────────────────────────── sistem çağrısı
Kernel alanı             ▼
                    VFS (sanal dosya sistemi)
                        │  /dev/bbbmsg → karakter aygıtı, major 246, minor 0
                        ▼
                    cdev tablosu: 246:0 → bbbmsg_fops
                        │
                        ▼
                    bbbmsg_read(file, buf, 4096, &pos)    ← bizim fonksiyonumuz
                        │  copy_to_user(buf, msg, n)
                        ▼
                    n bayt döner → cat ekrana basar
```

Sürücü, kernel'e bir `struct file_operations` tablosu verir. Tablodaki her alan bir sistem çağrısına karşılık gelir; doldurulmayan alanlar için kernel varsayılan davranışı uygular ya da hata döner:

```
static const struct file_operations bbbmsg_fops = {
    .owner          = THIS_MODULE,
    .open           = bbbmsg_open,
    .release        = bbbmsg_release,
    .read           = bbbmsg_read,
    .write          = bbbmsg_write,
    .llseek         = bbbmsg_llseek,
    .unlocked_ioctl = bbbmsg_ioctl,
};
```

Alan	Hangi çağrıda?
`open` / `release`	`open()` / son `close()`
`read` / `write`	`read()` / `write()`
`llseek`	`lseek()`
`unlocked_ioctl`	`ioctl()`
`owner = THIS_MODULE`	Aygıt açıkken modülün kullanımda sayılmasını sağlar; aygıt açıkken `rmmod` reddedilir

Bölümün kodu: `code/22_char_driver/`.

22.3 Aygıtı Kaydetmek: Numara, cdev, class

Sürücü yüklenirken üç şey yapar:

```
static int __init bbbmsg_init(void)
{
    struct device *dev;
    int ret;

    /* 1) Boş bir major numara ve 1 minor numara iste. */
    ret = alloc_chrdev_region(&devno, 0, 1, DEV_NAME);
    if (ret)
        return ret;

    /* 2) Bu numarayı file_operations'a bağla. */
    cdev_init(&bbbmsg_cdev, &bbbmsg_fops);
    bbbmsg_cdev.owner = THIS_MODULE;
    ret = cdev_add(&bbbmsg_cdev, devno, 1);
    if (ret)
        goto err_region;

    /* 3) /sys/class/bbbmsg/bbbmsg oluştur; devtmpfs/mdev /dev/bbbmsg'yi açar. */
    bbbmsg_class = class_create(THIS_MODULE, DEV_NAME);
    ...
    dev = device_create(bbbmsg_class, NULL, devno, NULL, DEV_NAME);
    ...
    return 0;

err_class:
    class_destroy(bbbmsg_class);
err_cdev:
    cdev_del(&bbbmsg_cdev);
err_region:
    unregister_chrdev_region(devno, 1);
    return ret;
}
```

Adım	Fonksiyon	Ne yapar?
1	`alloc_chrdev_region()`	Kernel'den boş bir major ve istenen sayıda minor numara alır. `devno` (`dev_t`) ikisini birlikte tutar: `MAJOR(devno)`, `MINOR(devno)`. Numara sabit yazılmaz; başka bir sürücüyle çakışabilir.
2	`cdev_init()` + `cdev_add()`	"Bu numaraya gelen çağrılar şu `file_operations`'a gitsin" kaydını yapar. `cdev_add`'den sonra aygıt **canlıdır**.
3	`class_create()` + `device_create()`	`/sys/class/bbbmsg/bbbmsg/` oluşturur. Kernel bunu bir **uevent** ile duyurur; `/dev/bbbmsg` düğümünü **devtmpfs** (kernel'in kendisi) ya da `mdev` / `udev` açar. Bu adım olmasaydı düğümü elle `mknod /dev/bbbmsg c 246 0` ile açmak gerekirdi.

> **Not**
>
> Kernel 6.4'ten itibaren `class_create()` sadece ismi alır: `class_create("bbbmsg")` . Koddaki `#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0)` satırı, sürücünün hem 5.10'da hem yeni kernel'lerde derlenmesini sağlar.

Hata yolu: Her adım başarısız olabilir. Bir adım başarısız olursa o ana kadar alınanlar **ters sırada** bırakılır (`goto err_...` zinciri). Kernel kodunda bu kalıp çok yaygındır; init yarıda kalırsa bırakılmayan bir numara ya da sınıf, modül tekrar yüklenene kadar sistemde kalır. `bbbmsg_exit()` de aynı sırayla temizler.

22.4 read ve write: Kullanıcı Belleğiyle Çalışmak

copy_to_user ve copy_from_user

`read()` 'e verilen `buf` işaretçisi **kullanıcı programının** belleğini gösterir. Kernel bu adrese doğrudan (`memcpy`) yazmamalıdır:

- Adres geçersiz olabilir (program yanlış adres verdi ya da kötü niyetli). Kernel'in o adrese yazması sistemi çökertir ya da güvenlik açığı olur.
- O sayfa o an bellekte olmayabilir.

`copy_to_user(hedef, kaynak, n)` ve `copy_from_user(hedef, kaynak, n)` adresi kontrol eder ve kopyalayamadığı bayt sayısını döner (0 = hepsi kopyalandı). Kopyalanamadıysa sürücü `-EFAULT` döner; program `Bad address` hatası alır. Fonksiyon parametrelerindeki `__user` işareti bu adresin kullanıcı alanına ait olduğunu belirtir; statik analiz araçları (`sparse`) onu doğrudan kullanan kodu yakalar.

Dosya konumu: *ppos

```
static ssize_t bbbmsg_read(struct file *file, char __user *ubuf,
               size_t count, loff_t *ppos)
{
    ssize_t ret;

    if (mutex_lock_interruptible(&msg_lock))
        return -ERESTARTSYS;

    if (*ppos >= msg_len) {
        ret = 0;                        /* dosya sonu (EOF) */
        goto out;
    }
    if (count > msg_len - *ppos)
        count = msg_len - *ppos;

    if (copy_to_user(ubuf, msg + *ppos, count)) {
        ret = -EFAULT;
        goto out;
    }
    *ppos += count;
    ret = count;
out:
    mutex_unlock(&msg_lock);
    return ret;
}
```

`*ppos`, açık dosyanın içinde nerede olduğumuzu tutar. `read()`'in dönüş değeri kuralları:

Dönüş	Program ne anlar?
`> 0`	Bu kadar bayt okundu; tekrar oku
`0`	**Dosya sonu.** `cat` burada durur.
`< 0`	Hata (`-EFAULT`, `-EINTR`...); `errno` olarak programa gider

> **Önemli**
>
> `read` her çağrıda aynı veriyi döner ve hiç `0` dönmezse `cat /dev/bbbmsg` sonsuza kadar aynı metni basar. `*ppos`'u ilerletmek ve sonda `0` dönmek bu yüzden gerekli.

`write` aynı mantıkla çalışır: veriyi `*ppos`'tan itibaren `msg`'ye kopyalar, yazılan yerin sonu mesajın yeni sonu olur. Böylece:

- `echo a > /dev/bbbmsg`: dosya baştan (konum 0) açılır, mesaj değişir.

- `echo b >> /dev/bbbmsg` : `O_APPEND` ile açılır. Normal dosyalarda sona eklemeyi dosya sistemi yapar; karakter aygıtlarında **sürücü** yapmalıdır: `if (file->f_flags & O_APPEND) *ppos = msg_len;`
- Tampon (256 bayt) dolarsa `write` yazabildiği kadarını yazar; yer hiç kalmadıysa `-ENOSPC` döner.

`llseek` için kernel'in hazır yardımcısı kullanılıyor: `generic_file_llseek_size(file, offset, whence, BUF_SIZE, msg_len)` . `SEEK_END` mesajın sonunu, en büyük geçerli konum tampon boyutunu kullanır.

22.5 Eşzamanlılık: Mutex

Aynı anda iki program aygıta yazabilir; biri yazarken öbürü okuyabilir. `msg` ve `msg_len` 'i aynı anda iki yerden değiştirmek veriyi bozar. Bu yüzden onlara her erişim bir **mutex** ile korunur:

```
static DEFINE_MUTEX(msg_lock);

if (mutex_lock_interruptible(&msg_lock))
    return -ERESTARTSYS;
/* ... msg ve msg_len'e erişim ... */
mutex_unlock(&msg_lock);
```

Fonksiyon	Fark
`mutex_lock()`	Kilit boşalana kadar bekler; beklerken Ctrl-C ile bile kesilemez
`mutex_lock_interruptible()`	Beklerken bir sinyal gelirse (Ctrl-C) vazgeçer ve 0 dışı döner. Kullanıcının çağrısıyla gelen uzun beklemelerde bu tercih edilir; sürücü `-ERESTARTSYS` döner, kernel çağrıyı gerekirse yeniden başlatır.

> **İpucu**
>
> AM335x tek çekirdekli ama kernel **preemptive**: bir süreç `write` 'ın ortasında durdurulup başka bir süreç çalıştırılabilir. Tek çekirdekte de kilit gerekir.

22.6 ioctl: Okuma/Yazma Dışındaki Komutlar

Bazı işler "veri oku/yaz" kalıbına uymaz: baud hızını değiştirmek, I2C adresini seçmek (Bölüm 19'daki `ioctl(fd, I2C_SLAVE, 0x68)`), bir ayarı sormak. Bunlar **`ioctl()`** ile yapılır. Sürücümüzde iki komut var; komut numaraları sürücü ile programın ortak kullandığı `bbbmsg.h` 'de:

```
#define BBBMSG_IOC_MAGIC    'b'
#define BBBMSG_CLEAR        _IO(BBBMSG_IOC_MAGIC, 0)          /* mesajı sil */
#define BBBMSG_GET_LEN      _IOR(BBBMSG_IOC_MAGIC, 1, int)    /* uzunluğu oku */
```

Makro	Anlamı
`_IO(tip, no)`	Veri taşımayan komut
`_IOR(tip, no, tür)`	Sürücüden programa veri (Read: program okur)
`_IOW(tip, no, tür)`	Programdan sürücüye veri
`_IOWR(tip, no, tür)`	İki yönlü

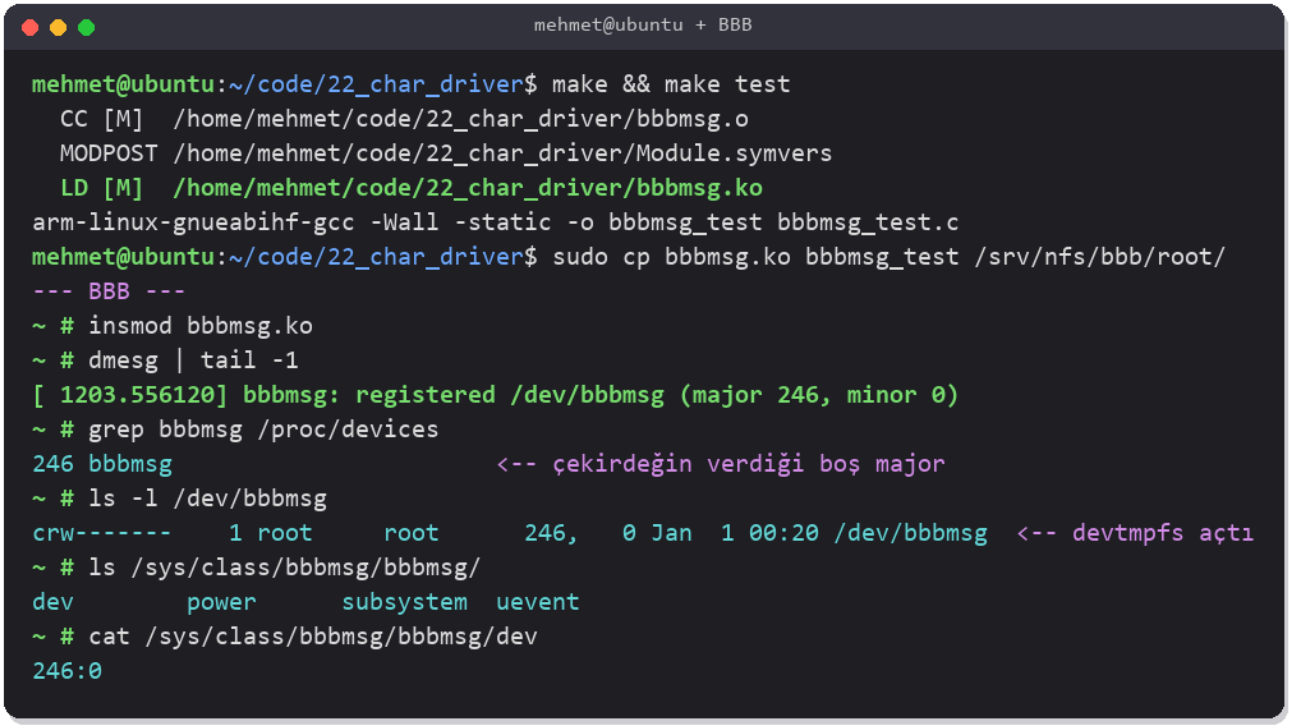
Makrolar komut numarasına sihirli harfi (`'b'`), sıra numarasını, veri yönünü ve boyutunu kodlar; böylece başka bir sürücünün komutu yanlışlıkla bizimkine gönderilirse tanınmaz. Sürücü tarafı:

```
static long bbbmsg_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    int len;

    switch (cmd) {
    case BBBMSG_CLEAR:
        mutex_lock(&msg_lock);
        msg_len = 0;
        mutex_unlock(&msg_lock);
        return 0;

    case BBBMSG_GET_LEN:
        mutex_lock(&msg_lock);
        len = msg_len;
        mutex_unlock(&msg_lock);
        return put_user(len, (int __user *)arg);

    default:
        return -ENOTTY;                /* bu sürücünün komutu değil */
    }
}
```

`put_user()` , tek bir değişken için `copy_to_user()` 'ın kısa yoludur. Bilinmeyen komuta `-ENOTTY` dönmek gelenektir (program `Inappropriate ioctl for device` görür).

22.7 Derle ve Kabuktan Dene

`Makefile` Bölüm 21'dekinin aynısı; `make test` ayrıca kullanıcı programını statik derler.



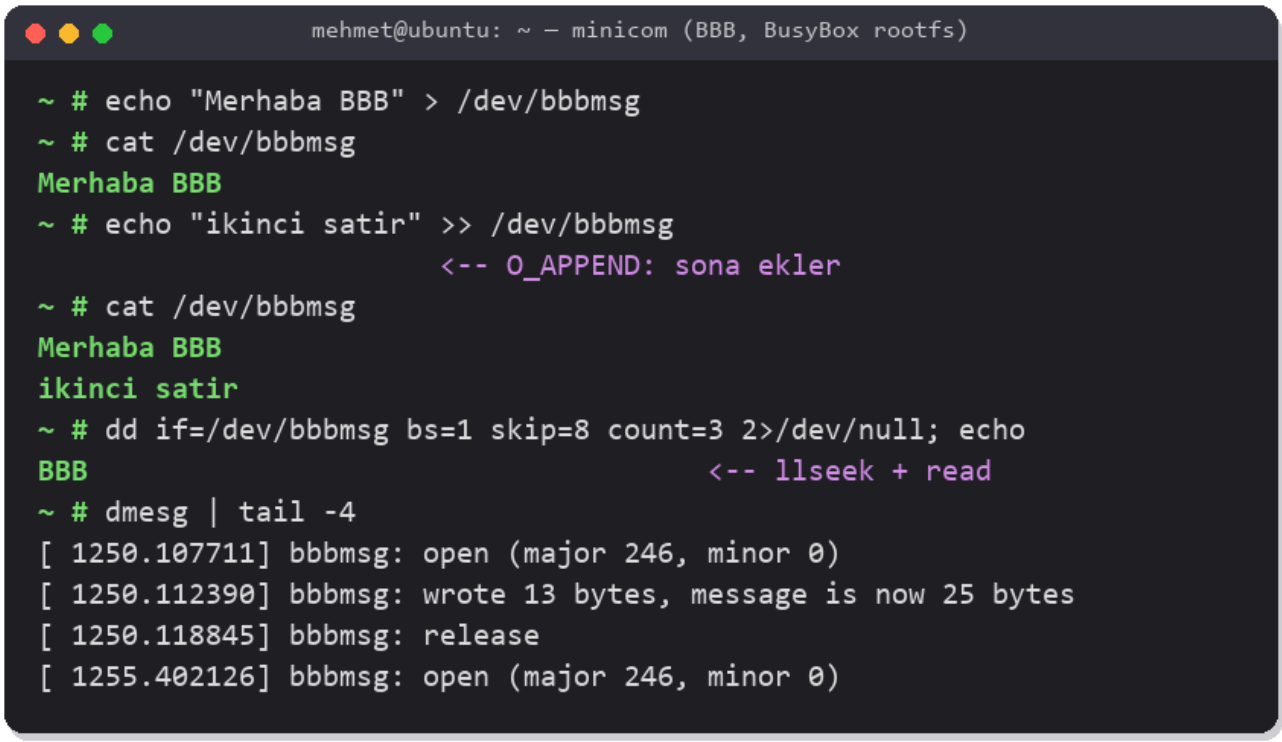
Host'ta:

```
cd ~/code/22_char_driver
make && make test
sudo cp bbbmsg.ko bbbmsg_test /srv/nfs/bbb/root/
```

Kartta:

```
~ # insmod bbbmsg.ko
~ # dmesg | tail -1
~ # grep bbbmsg /proc/devices
~ # ls -l /dev/bbbmsg
~ # cat /sys/class/bbbmsg/bbbmsg/dev
```

Kernel boş bir major numara verdi (örnekte 246; sende farklı olabilir) ve `/dev/bbbmsg` kendiliğinden oluştu. Artık bu dosya her program için sıradan bir dosya gibi:



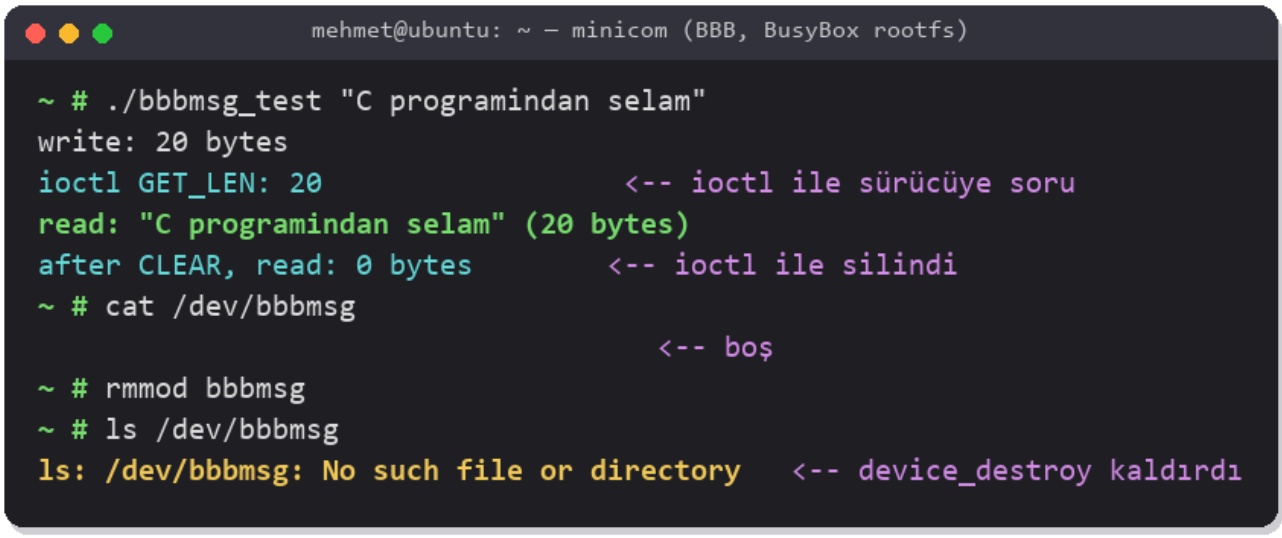

```
~ # echo "Merhaba BBB" > /dev/bbbmsg
~ # cat /dev/bbbmsg
~ # echo "ikinci satir" >> /dev/bbbmsg
~ # cat /dev/bbbmsg
~ # dd if=/dev/bbbmsg bs=1 skip=8 count=3 2>/dev/null; echo
~ # dmesg | tail -4
```

`dd ... skip=8` önce `lseek` ile 8. bayta gider, sonra 3 bayt okur: "Merhaba BBB" metnindeki `BBB`. `dmesg`'te her `echo` / `cat` için bir `open` ve `release` satırı görürsün: her komut aygıtı açıp kapatıyor.

22.8 Kullanıcı Programıyla Dene

`bbbmsg_test.c` aygıtı bir C programından kullanır: yazar, `ioctl` ile uzunluğu sorar, başa dönüp okur, `ioctl` ile siler ve tekrar okur.

```
fd = open(DEV_PATH, O_RDWR);

n = write(fd, argv[1], strlen(argv[1]));
ioctl(fd, BBBMSG_GET_LEN, &len);

lseek(fd, 0, SEEK_SET);                 /* yazdıktan sonra konum sonda */
n = read(fd, buf, sizeof(buf) - 1);

ioctl(fd, BBBMSG_CLEAR);
```




```
~ # ./bbbmsg_test "C programindan selam"
~ # rmmod bbbmsg
~ # ls /dev/bbbmsg          # yok: device_destroy düğümü kaldırdı
```

Sürücü ile program aynı `bbbmsg.h`'yi kullandığı için komut numaraları iki tarafta da aynıdır. Gerçek sürücülerde bu başlıklar kernel'in `include/uapi/` klasöründe durur ve kullanıcı programlarına `/usr/include/linux/` olarak gelir; Bölüm 19'daki `<linux/i2c-dev.h>` böyle bir başlıktır.

22.9 Özet

- `/dev` altındaki dosyalar aygıt düğümleridir. `c` karakter, `b` blok aygıtı; **major** sürücüyü, **minor** sürücünün aygıtını seçer (`/proc/devices`).
- Karakter sürücüsü: `alloc_chrdev_region()` → `cdev_init()` + `cdev_add()` → `class_create()` + `device_create()` (düğümü devtmpfs/mdev açar). Çıkarken ters sırada bırakılır.
- `struct file_operations` sistem çağrılarını sürücünün fonksiyonlarına bağlar: `open` , `release` , `read` , `write` , `llseek` , `unlocked_ioctl` .
- Kullanıcı belleğine sadece `copy_to_user()` / `copy_from_user()` / `put_user()` ile erişilir; başarısızsa `-EFAULT` .
- `read` konumu (`*ppos`) ilerletir ve sonda `0` döner; yoksa `cat` hiç durmaz. `O_APPEND` 'i karakter sürücüsü kendisi uygular.
- Paylaşılan veri mutex ile korunur (`mutex_lock_interruptible`); tek çekirdekte de gerekir.
- Okuma/yazma dışındaki komutlar `ioctl` ile: `_IO` , `_IOR` , `_IOW` ; bilinmeyen komuta `-ENOTTY` .

22.10 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom (BBB, BusyBox rootfs) (temsili hatalar)

~ # printf '%300s' x > /dev/bbbmsg
printf: write error: No space left on device   <-- 256 bayt yazıldı, kalanına yer yok

debian@beaglebone:~$ echo hi > /dev/bbbmsg
-bash: /dev/bbbmsg: Permission denied       <-- düğüm 0600, root değilsin

~ # ./bbbmsg_test x
/dev/bbbmsg: No such file or directory      <-- modül yüklü değil

~ # rmmod bbbmsg
rmmod: can't unload module 'bbbmsg': Resource temporarily unavailable
                                  <-- aygıt başka bir süreçte açık
```

Belirti	Sebep	Çözüm
`/dev/bbbmsg` oluşmuyor	`device_create()` çağrılmadı ya da rootfs'te devtmpfs/mdev yok	`dmesg`'e bak; 12.4'teki `CONFIG_DEVTMPFS_MOUNT`'u kontrol et ya da elle `mknod /dev/bbbmsg c <major> 0`
`cat /dev/bbbmsg` aynı metni durmadan basıyor	`read` `*ppos`'u ilerletmiyor ya da sonda `0` dönmüyor	22.4'teki gibi
`>>` ile ekleme yapılmıyor, metin baştan yazılıyor	`O_APPEND` sürücüde uygulanmamış	`write`'ta `f_flags & O_APPEND` kontrolü
`write error: No space left on device`	256 baytlık tampon doldu	Beklenen davranış; mesajı `BBBMSG_CLEAR` ioctl'üyle sil (`./bbbmsg_test`). `echo -n > /dev/bbbmsg` silmez: sıfır baytlık yazma sürücünün `write`'ını hiç çağırmaz.
`Permission denied`	Düğüm `0600` ve root değilsin	Root ol; kalıcı çözüm için `mdev.conf` / `udev` kuralıyla izni değiştir
`rmmod: ... Resource temporarily unavailable`	Aygıt bir süreçte açık (`owner = THIS_MODULE`)	Aygıtı kullanan programı kapat (`fuser /dev/bbbmsg`)
`insmod`'da `-EBUSY` / `-EEXIST`, `dmesg`'te `sysfs: cannot create duplicate filename '/class/bbbmsg'`	Önceki yükleme init'te yarıda kaldı ve sınıfı bırakmadı	Hata yolunu (`goto err_...`) kontrol et; kartı yeniden başlat
Program `Inappropriate ioctl for device` alıyor	Komut numarası sürücüdekiyle aynı değil	Sürücü ve program aynı `bbbmsg.h`'yi kullanmalı; ikisini birlikte yeniden derle
Kernel `Oops`, `Unable to handle kernel paging request`	Kullanıcı adresine `memcpy` ile erişildi ya da tampon taşırıldı	`copy_*_user` kullan, `count`'u tampon boyutuyla sınırla

Belirti	Sebep	Çözüm
`implicit declaration of function 'class_create'` / argüman sayısı hatası	Kernel 6.4+ ile derleniyor	Koddaki `LINUX_VERSION_CODE` koşulu (22.3)

Bölüm 23 — Device Tree ile GPIO Platform Sürücüsü

> **Bu bölümde:**
>
> - Device tree'ye kendi donanımını tarif eden bir düğüm ekleyecek ve `compatible` ile sürücüne bağlayacaksın.
> - Bir **platform sürücüsü** yazacak; `probe()` içinde LED'i ve butonu kernel'in GPIO API'siyle (`gpiod`) alacaksın.
> - Butonu kesmeyle (threaded IRQ) okuyup kontak sekmesini sürücüde çözeceksin.
> - Sürücünün durumunu sysfs dosyalarıyla dışarı açacak, kullanıcı programını `poll()` ile uyandıracaksın.
> - `devm_` fonksiyonlarıyla kaynakların kendiliğinden bırakılmasını, `bind` / `unbind` ile sürücü-aygıt bağını göreceksin.
>
> **Ön koşul:** Bölüm 11 — device tree ve `compatible`, Bölüm 16 — harici LED, Bölüm 17 — buton ve kesme, Bölüm 22 — sürücü yazmak.

> **Not**
>
> Bu bölüm kursta yok. Kart, Bölüm 21'deki gibi kendi kernel'in (5.10.162) ve BusyBox rootfs'inle, kernel ve DTB TFTP'den gelerek (12.8) açılıyor. Kartta Debian çalıştırıyorsan 23.3'teki overlay yolunu kullan. Terminal görüntüleri temsilidir.

23.1 Kullanıcı Alanından Kernel'e

Bölüm 16 ve Bölüm 17'de LED'i ve butonu kullanıcı alanından, `/sys/class/gpio` ile sürdük. Program GPIO'yu export ediyor, kesme için `edge` dosyasını ayarlıyor, `poll()` ile bekliyor ve kontak sekmesini kendisi süzüyordu. Bu bölümde aynı donanımı **kernel'in içinden** süreceğiz:

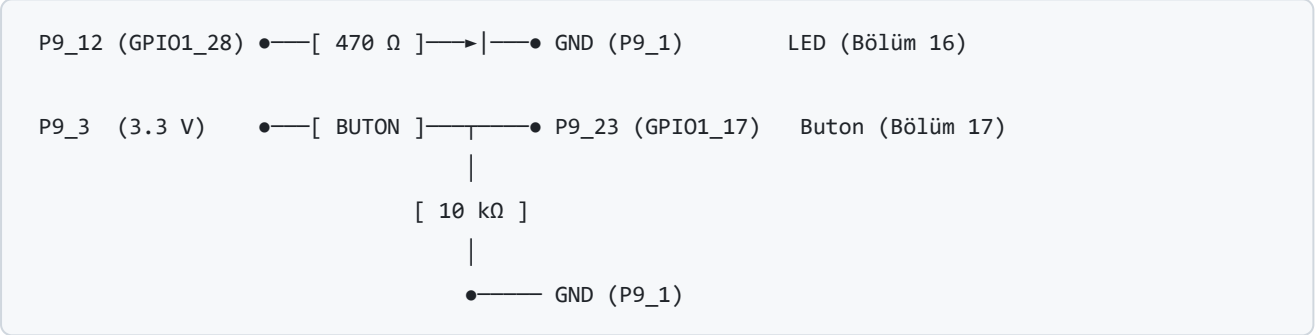
	Kullanıcı alanında (Bölüm 16–17)	**Platform sürücüsünde (bu bölüm)**
Donanım nerede tarif ediliyor?	Programın kaynak kodunda: `#define BUTTON_GPIO 49`	**Device tree**'de: `button-gpios = <&gpio1 17 ...>`
GPIO numarası	Elle hesaplanıyor, kernel sürümüne bağlı (15.6)	Yok: sürücü GPIO'yu adıyla (`"button"`) ister
Pinmux	`config-pin` ya da varsayılana güven	DT'deki `pinctrl-0`, sürücü yüklenmeden kernel ayarlar
Kesme ve debounce	Program `poll()` + zaman kontrolü	Sürücü: kesme fonksiyonu + debounce

	Kullanıcı alanında (Bölüm 16–17)	Platform sürücüsünde (bu bölüm)
Kullanıcıya arayüz	—	Sürücünün açtığı `led` ve `presses` dosyaları

Kernel'in kendi sürücüleri de böyle çalışır: USR LED'leri süren `leds-gpio` (11.5) ya da Bölüm 19'daki I2C denetleyicisi device tree'den tanıtılır ve `compatible` ile sürücüsüne bağlanır.

Devre

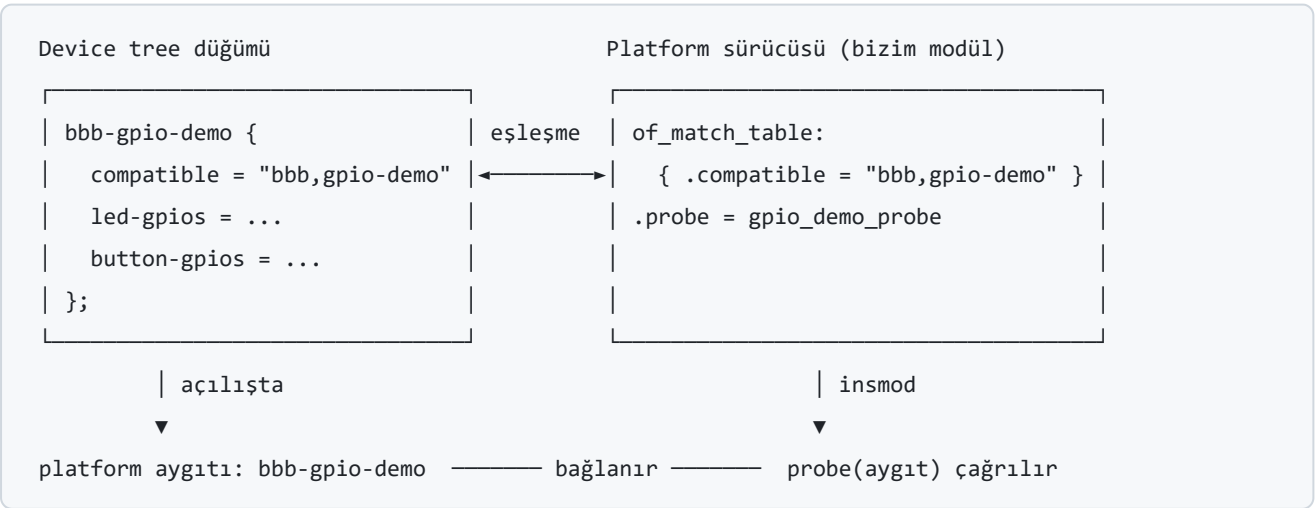
Bölüm 16'daki LED ve Bölüm 17'deki buton, birlikte:



> **Uyarı**
>
> Bölüm 18'deki 4 haneli gösterge de P9_23'ü kullanıyor; o devre takılıysa sök.

23.2 Platform Aygıtı ve Platform Sürücüsü

USB veya PCI aygıtları kendini tanıtır; kernel onları veriyolunu tarayarak bulur. SoC'nin içindeki birimler ve GPIO'ya bağlı bir LED ise kendini tanıtamaz (11.1). Kernel bunları **platform veriyolu** (platform bus) üzerinde tutar:



1. Kernel açılırken DTB'deki düğümden bir **platform aygıtı** oluşturur (`/sys/bus/platform/devices/bbb-gpio-demo`). Henüz bir sürücüsü yoktur.
2. Sürücü (`insmod` ya da kernel'e gömülü) kendini platform veriyoluna kaydeder.
3. Veriyolu, aygıtların `compatible` 'ını sürücülerin `of_match_table` 'ıyla karşılaştırır. Eşleşen her aygıt için sürücünün **`probe()`** fonksiyonunu çağırır.
4. Sürücü ayrılırken (`rmmod` , `unbind`) aldığı kaynakları bırakır.

Sıra fark etmez: sürücü önce yüklenirse aygıt oluştuğunda, aygıt önceden varsa sürücü yüklendiğinde eşleşme olur.

23.3 Device Tree'ye Düğüm Eklemek

Bölümün kodu: `code/23_gpio_platform_driver/` .

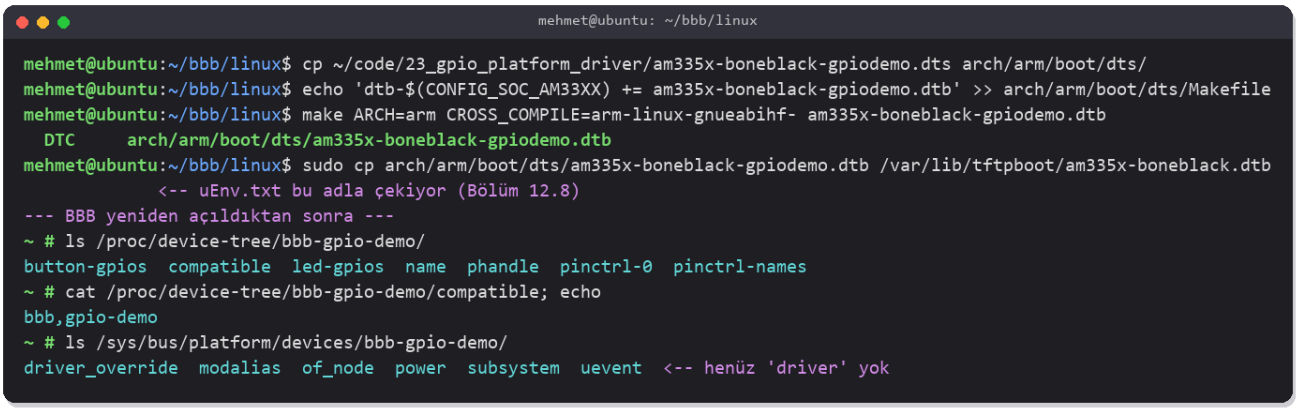
Ana yol: kendi kernel'ine kart DTS'i

`am335x-boneblack-gpiodemo.dts` , BBB'nin DTS'ini olduğu gibi alıp üzerine düğümümüzü ekler (11.8'deki kalıtım):

```
#include "am335x-boneblack.dts"

&am33xx_pinmux {
    gpio_demo_pins: gpio-demo-pins {
        pinctrl-single,pins = <
            AM33XX_PADCONF(AM335X_PIN_GPMC_BEN1, PIN_OUTPUT_PULLDOWN, MUX_MODE7)    /* P9_12:
gpio1_28, LED */
            AM33XX_PADCONF(AM335X_PIN_GPMC_A1, PIN_INPUT_PULLDOWN, MUX_MODE7)   /* P9_23:
gpio1_17, buton */
        >;
    };
};

/ {
    bbb-gpio-demo {
        compatible = "bbb,gpio-demo";
        pinctrl-names = "default";
        pinctrl-0 = <&gpio_demo_pins>;
        led-gpios = <&gpio1 28 GPIO_ACTIVE_HIGH>;
        button-gpios = <&gpio1 17 GPIO_ACTIVE_HIGH>;
    };
};
```

Satır	Anlamı
`&am33xx_pinmux { ... }`	AM335x'in pin ayar düğümüne bir pin grubu ekler: P9_12 mod 7 çıkış, P9_23 mod 7 giriş + pull-down (15.3)
`bbb-gpio-demo`	Kök düğümün altında yeni bir düğüm; kernel bundan aynı adlı bir platform aygıtı oluşturur
`compatible = "bbb,gpio-demo"`	Sürücüyü seçen string. Biçim `"üretici,model"` ; `bbb` burada kitabın uydurduğu bir önek
`pinctrl-0`	Sürücünün `probe()` 'u çağrılmadan hemen önce kernel bu pin grubunu uygular
`led-gpios` , `button-gpios`	`<&gpio1 28 GPIO_ACTIVE_HIGH>` : GPIO1 bankasının 28. hattı, aktif yüksek. Sürücü bunları `"led"` ve `"button"` adıyla ister; property adının sonundaki `-gpios` sabittir.

Kernel kaynağında derle (Bölüm 10):




```
cd ~/bbb/linux
cp ~/code/23_gpio_platform_driver/am335x-boneblack-gpiodemo.dts arch/arm/boot/dts/
echo 'dtb-$(CONFIG_SOC_AM33XX) += am335x-boneblack-gpiodemo.dtb' >> arch/arm/boot/dts/Makefile
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x-boneblack-gpiodemo.dtb
sudo cp arch/arm/boot/dts/am335x-boneblack-gpiodemo.dtb /var/lib/tftpboot/am335x-boneblack.dtb
```

DTS bir `.dts` 'i include ettiği ve `AM33XX_PADCONF` gibi makrolar kullandığı için elle `dtc` ile değil, kernel'in derleme sistemiyle derleniyor (önce C ön işlemcisi, sonra `dtc` ; 11.9). `Makefile` 'a eklenen satır, kernel'e bu DTB'nin AM335x için derlenebileceğini söyler. Son satır yeni DTB'yi, 12.8'deki `uEnv.txt` 'nin TFTP'den çektiği adla koyar; asıl `am335x-boneblack.dtb` 'yi istersen önce yedekle.

> **Not**
>
> Kernel 6.5 ve sonrasında DTS'ler `arch/arm/boot/dts/ti/omap/` altında ve hedef `ti/omap/am335x-boneblack-gpiodemo.dtb` (11.9).

Kartı yeniden aç ve düğümün geldiğini kontrol et:

```
~ # ls /proc/device-tree/bbb-gpio-demo/
~ # cat /proc/device-tree/bbb-gpio-demo/compatible; echo
~ # ls /sys/bus/platform/devices/bbb-gpio-demo/
```

Platform aygıtı oluştu ama içinde `driver` bağlantısı yok: henüz onu süren bir sürücü yok.

Debian kullanıyorsan: overlay

Kartta BeagleBoard'un Debian imajı (v5.10-ti) çalışıyorsa ana DTB'yi değiştirmek yerine `BB-GPIO-DEMO-00A0.dts` overlay'ini kullan. Aynı düğümü ekler; derleme ve kurulum 16.6'daki gibidir:

```
dtc -@ -I dts -O dtb -o BB-GPIO-DEMO-00A0.dtbo BB-GPIO-DEMO-00A0.dts
sudo cp BB-GPIO-DEMO-00A0.dtbo /lib/firmware/
# /boot/uEnv.txt: uboot_overlay_addr4=/lib/firmware/BB-GPIO-DEMO-00A0.dtbo
#                 enable_uboot_cape_universal=1 satırını # ile kapat
```

> **Önemli**
>
> Cape-universal açıkken P9_12 ve P9_23 `config-pin` için ayrılmıştır ve overlay bu pinleri alamaz (23.10). 16.6'daki `BB-EXT-LED` overlay'i de P9_12'yi kullanıyor; ikisini aynı anda yükleme. Sürücüyü de Debian'ın kernel'ine karşı derle (21.2).

23.4 Sürücünün İskeleti

```
static const struct of_device_id gpio_demo_of_match[] = {
    { .compatible = "bbb,gpio-demo" },
    { }
};
MODULE_DEVICE_TABLE(of, gpio_demo_of_match);

static struct platform_driver gpio_demo_driver = {
    .probe = gpio_demo_probe,
    .driver = {
        .name           = "bbb-gpio-demo",
        .of_match_table = gpio_demo_of_match,
        .dev_groups     = gpio_demo_groups,
    },
};
module_platform_driver(gpio_demo_driver);
```

Parça	Anlamı
`of_match_table`	Sürücünün sürebildiği `compatible` string'leri; boş eleman listenin sonu
`MODULE_DEVICE_TABLE(of, ...)`	Tabloyu modülün bilgilerine de yazar (`modinfo` → `alias: of:N*T*Cbbb,gpio-demo`). `udev` gibi araçlar bununla, eşleşen bir aygıt görünce modülü kendiliğinden yükler.
`.probe`	Eşleşen her aygıt için çağrılır
`.dev_groups`	Sürücü bağlanınca aygıtın sysfs klasörüne eklenecek dosyalar (23.6)
`module_platform_driver()`	`module_init` 'te `platform_driver_register()` , `module_exit` 'te `platform_driver_unregister()` yazan kısa yol

Sürücünün bir `.remove` fonksiyonu yok; gerek de yok. Her kaynak `devm_` fonksiyonlarıyla alındığı için sürücü ayrılırken kernel hepsini kendisi bırakır (23.7).

23.5 probe(): GPIO'ları ve Kesmeyi Almak

```
static int gpio_demo_probe(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct gpio_demo *d;
    int ret;

    d = devm_kzalloc(dev, sizeof(*d), GFP_KERNEL);
    if (!d)
        return -ENOMEM;
    ...
    d->led = devm_gpiod_get(dev, "led", GPIOD_OUT_LOW);
    if (IS_ERR(d->led))
        return dev_err_probe(dev, PTR_ERR(d->led), "cannot get led-gpios\n");

    d->button = devm_gpiod_get(dev, "button", GPIOD_IN);
    ...
    gpiod_set_debounce(d->button, DEBOUNCE_MS * 1000);

    d->irq = gpiod_to_irq(d->button);
    ...
    platform_set_drvdata(pdev, d);

    ret = devm_add_action_or_reset(dev, demo_led_off, d);
    ...
    ret = devm_request_threaded_irq(dev, d->irq, NULL, demo_button_thread,
                    IRQF_TRIGGER_RISING | IRQF_ONESHOT,
                    dev_name(dev), d);
    ...
    dev_info(dev, "ready: LED off, button on irq %d\n", d->irq);
    return 0;
}
```

Çağrı	Ne yapar?
`devm_kzalloc()`	Sürücünün aygıta özel verisi (`struct gpio_demo`) için sıfırlanmış bellek. Aynı sürücü birden fazla aygıtı sürebileceği için durum global değişkende değil bu yapıda tutulur.
`devm_gpiod_get(dev, "led", GPIOD_OUT_LOW)`	DT'deki `led-gpios` 'u bulur, GPIO'yu ister (başka biri kullanıyorsa `-EBUSY`) ve çıkış/0 yapar. Dönen `gpio_desc` artık GPIO'nun "tutamacı"dır; numara hiç görünmez.
`GPIO_ACTIVE_HIGH/LOW`	DT'deki bayrak mantığı belirler: `gpiod_set_value(led, 1)` "LED'i yak" demektir; aktif düşük bir devrede kernel pini 0 yapar. Sürücü kodu değişmez.

Çağrı	Ne yapar?
`gpiod_set_debounce()`	AM335x'in GPIO modülünde donanımsal debounce var; destekleniyorsa açılır. Desteklenmezse hata döner, sürücü yazılımdaki kontrolle devam eder.
`gpiod_to_irq()`	GPIO hattının kesme numarası
`platform_set_drvdata()`	Veriyi aygıta bağlar; sysfs fonksiyonları `dev_get_drvdata()` ile geri alır
`dev_err_probe()`	Hatayı aygıtın adıyla `dmesg`'e yazar ve hata kodunu döner. `-EPROBE_DEFER`'de susar (aşağıdaki not).

> **Not**
>
> **Ertelenen probe (`-EPROBE_DEFER`):** Aygıtın ihtiyaç duyduğu başka bir sürücü (ör. GPIO denetleyicisi) henüz hazır değilse `devm_gpiod_get()` `-EPROBE_DEFER` döner. Bu bir hata değildir: kernel probe'u sonra tekrar dener. Sürücüler bu hatayı yukarı iletir ve log'a yazmaz; `dev_err_probe()` bunu kendisi yapar.

Threaded kesme

```
static irqreturn_t demo_button_thread(int irq, void *data)
{
    struct gpio_demo *d = data;
    unsigned long now = jiffies;
    int n;

    if (time_before(now, d->last_press + msecs_to_jiffies(DEBOUNCE_MS)))
        return IRQ_HANDLED;
    d->last_press = now;

    mutex_lock(&d->lock);
    demo_set_led(d, !d->led_on);
    mutex_unlock(&d->lock);

    n = atomic_inc_return(&d->presses);
    dev_info(d->dev, "button pressed (%d), LED %s\n", n, d->led_on ? "on" : "off");

    sysfs_notify(&d->dev->kobj, NULL, "presses");
    return IRQ_HANDLED;
}
```

Kesme fonksiyonu normalde işlemcinin o an yaptığı işi keserek, **uyuyamayan** bir bağlamda çalışır: içinde mutex, `msleep` ya da uyuyabilen GPIO işlemleri kullanılamaz. `devm_request_threaded_irq()`'nin ikinci fonksiyonu ise kernel'in açtığı ayrı bir thread'de çalışır ve uyuyabilir. İlk fonksiyonu `NULL` verdik; bu durumda `IRQF_ONESHOT` zorunludur: thread bitene kadar kesme kapalı tutulur.

Parça	Anlamı
`IRQF_TRIGGER_RISING`	Yükselen kenar: butona basıldığı an (17.7'deki `edge = rising`)
`jiffies`, `time_before()`	Kernel'in zaman sayacı (saniyede `HZ` kez artar). 50 ms içindeki ikinci kenar kontak sekmesidir, sayılmaz (17.7'deki debounce'un aynısı).
`atomic_t`	Kilitsiz, bölünmez sayaç; sysfs'ten okunurken yarım değer görülmez
`IRQ_HANDLED`	"Bu kesme benimdi, işlendi"

23.6 sysfs ile Dışarıya Açmak

Bölüm 22'de kullanıcıya bir `/dev` dosyası açmıştık. Basit değerler için sürücünün kendi sysfs klasörüne dosya eklemek daha kolaydır; kernel'in LED sürücüsündeki `brightness` dosyası (15.5) de böyledir:

```
static ssize_t led_show(struct device *dev, struct device_attribute *attr, char *buf)
{
    struct gpio_demo *d = dev_get_drvdata(dev);

    return sysfs_emit(buf, "%d\n", d->led_on);
}

static ssize_t led_store(struct device *dev, struct device_attribute *attr,
             const char *buf, size_t count)
{
    ...
    ret = kstrtobool(buf, &on);         /* "0", "1", "y", "n", "on", "off" */
    ...
    return count;
}
static DEVICE_ATTR_RW(led);             /* led_show + led_store → "led" dosyası, 0644 */
static DEVICE_ATTR_RO(presses);         /* presses_show → "presses" dosyası, 0444 */
```

Parça	Anlamı
`xxx_show()`	Dosya okununca çağrılır; değeri `buf`'a yazar (`sysfs_emit()` taşmayı önler)
`xxx_store()`	Dosyaya yazılınca çağrılır; metni çözer (`kstrtobool`, `kstrtoint`) ve `count` döner
`DEVICE_ATTR_RW/RO(ad)`	`ad_show` / `ad_store` fonksiyonlarından dosya tanımı üretir
`ATTRIBUTE_GROUPS()` + `.dev_groups`	Dosyalar sürücü bağlanınca oluşur, ayrılınca silinir
`sysfs_notify(..., "presses")`	`presses` dosyasını `poll()` ile bekleyenleri uyandırır

Kullanıcı alanında sysfs dosyalarına C'den yazarken de değerin **metin** olduğunu hatırla (15.7).

23.7 devm: Kaynakları Kendiliğinden Bırakmak

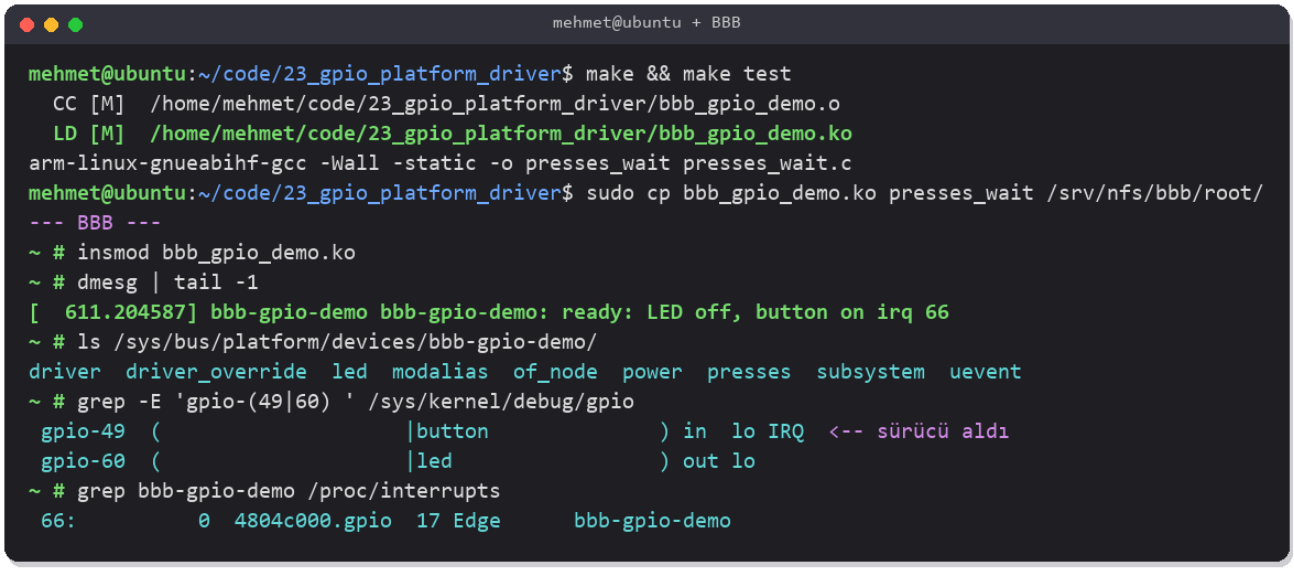
`devm_` (device-managed) önekli fonksiyonlar aldıkları kaynağı aygıta kaydeder. Sürücü ayrılırken (ya da `probe` hata dönerse) kernel bu kaynakları **ters sırada** kendisi bırakır:

Alınan	Kendiliğinden bırakılan
`devm_kzalloc()`	`kfree()`
`devm_gpiod_get()`	`gpiod_put()`
`devm_request_threaded_irq()`	`free_irq()`
`devm_add_action_or_reset(dev, demo_led_off, d)`	`demo_led_off(d)` : LED'i söndürür

Bölüm 22'deki `goto err_...` zinciri burada gerekmedi: `probe` herhangi bir adımda hata dönerse o ana kadar alınanlar bırakılır. Sıralama da önemli: kesme LED'i söndürme eyleminden **sonra** alındığı için önce kesme bırakılır, sonra LED söner; kesme fonksiyonu sönmüş LED'i yeniden yakamaz.

23.8 Kartta Dene

Yükle



Host'ta:

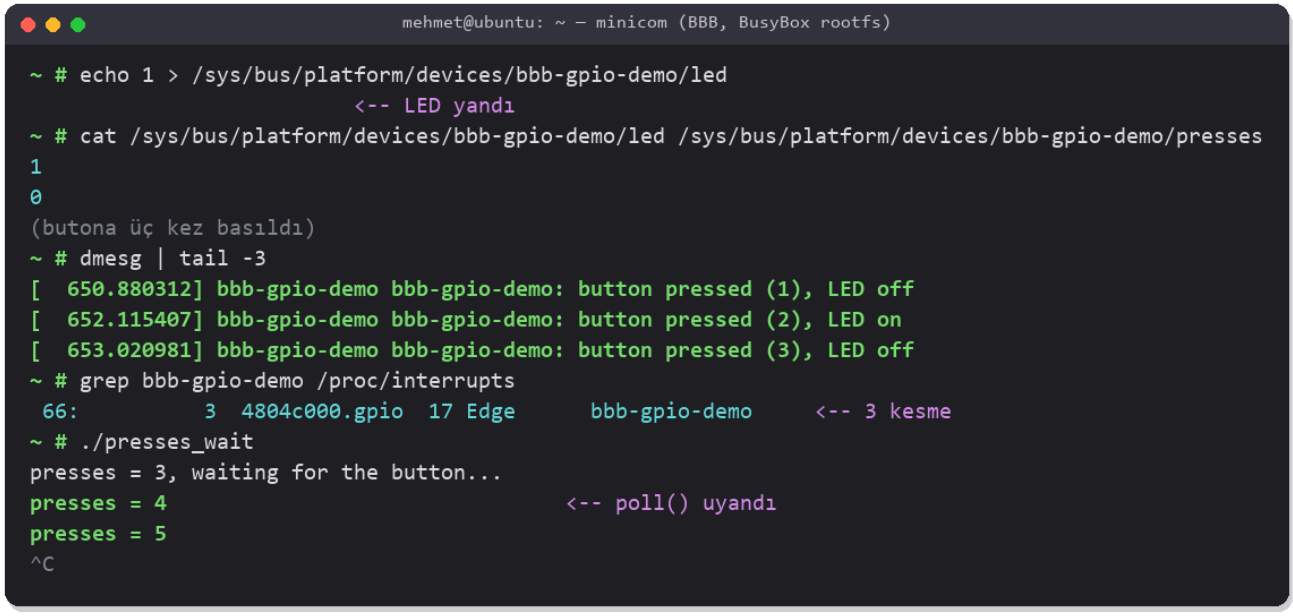
```
cd ~/code/23_gpio_platform_driver
make && make test
sudo cp bbb_gpio_demo.ko presses_wait /srv/nfs/bbb/root/
```

Kartta:

```
~ # insmod bbb_gpio_demo.ko
~ # dmesg | tail -1
~ # ls /sys/bus/platform/devices/bbb-gpio-demo/
~ # grep -E 'gpio-(49|60) ' /sys/kernel/debug/gpio
~ # grep bbb-gpio-demo /proc/interrupts
```

`probe` çalıştı: aygıtın klasöründe artık `driver` bağlantısı ve sürücünün açtığı `led`, `presses` dosyaları var. `/sys/kernel/debug/gpio` GPIO'ları kimin aldığını (`led`, `button`), `/proc/interrupts` kesmenin kaç kez geldiğini gösterir.

Kullan



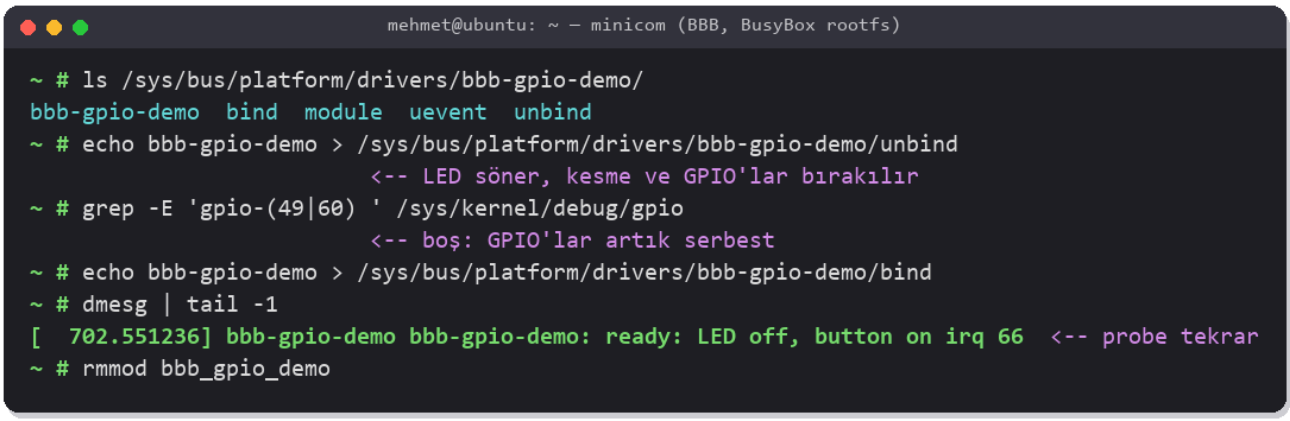

```
~ # echo 1 > /sys/bus/platform/devices/bbb-gpio-demo/led
~ # cat /sys/bus/platform/devices/bbb-gpio-demo/presses
~ # dmesg | tail -3                      # butona birkaç kez bastıktan sonra
~ # ./presses_wait
```

Her basışta LED'in durumu değişir, `presses` bir artar. `presses_wait.c`, 17.7'deki `poll()` döngüsünün aynısıdır; ama kesmeyi ve debounce'u artık sürücü yapıyor, program sadece uyanıp sayıyı okuyor:

```
read_presses(fd);                       /* bir kez okumadan poll() hemen döner */

for (;;) {
    pfd.fd = fd;
    pfd.events = POLLPRI | POLLERR;
    poll(&pfd, 1, -1);                  /* sysfs_notify gelene kadar uyu */
    printf("presses = %d\n", read_presses(fd));
}
```

bind ve unbind

Sürücü ile aygıt arasındaki bağ sysfs'ten koparılıp yeniden kurulabilir. Bir sürücüyü modülü çıkarmadan denemek ya da bir aygıtı başka bir sürücüye vermek için kullanılır:




```
~ # echo bbb-gpio-demo > /sys/bus/platform/drivers/bbb-gpio-demo/unbind
~ # echo bbb-gpio-demo > /sys/bus/platform/drivers/bbb-gpio-demo/bind
~ # rmmod bbb_gpio_demo
```

`unbind` ile devm kaynakları bırakılır (LED söner, GPIO'lar ve kesme serbest kalır); `bind` ile `probe` tekrar çağrılır.

23.9 Özet

- Kendini tanıtmayan donanım device tree'de bir düğümle tarif edilir; kernel her düğümden bir **platform aygıtı** oluşturur.
- Platform sürücüsü `of_match_table`'daki `compatible` ile aygıtı bulur, `probe()`'ta kaynaklarını alır. `module_platform_driver()` kayıt işini yapar.
- GPIO'lar numarayla değil adla alınır: DT'de `led-gpios`, sürücüde `devm_gpiod_get(dev, "led", ...)`. Aktif yüksek/düşük DT'de belirlenir.
- Buton: `gpiod_to_irq()` + `devm_request_threaded_irq(..., IRQF_TRIGGER_RISING | IRQF_ONESHOT)`; uyuyabilen işler thread'de, debounce `jiffies` ile.
- Kullanıcıya basit arayüz: `DEVICE_ATTR_RW/RO` + `.dev_groups`; `sysfs_notify()` ile `poll()` edenleri uyandır.

- `devm_` fonksiyonları kaynakları ters sırada kendiliğinden bırakır; `.remove` ve `goto` zinciri gerekmez.
- Sürücü-aygıt bağı `/sys/bus/platform/drivers/<sürücü>/bind` ve `unbind` ile elle de kurulup koparılabilir.

23.10 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom (BBB, BusyBox rootfs) (temsili hatalar)
~ # insmod bbb_gpio_demo.ko; dmesg | tail -1
            <-- hiçbir mesaj yok: DT'de compatible eşleşen düğüm yok

~ # echo 60 > /sys/class/gpio/export; insmod bbb_gpio_demo.ko; dmesg | tail -1
bbb-gpio-demo bbb-gpio-demo: error -EBUSY: cannot get led-gpios
            <-- GPIO 60 sysfs'te export edilmiş

~ # dmesg | grep pinctrl
pinctrl-single 44e10800.pinmux: pin PIN30 already requested by ocp:P9_12_pinmux; cannot claim for bbb-gpio-demo
            <-- Debian'da cape-universal açık (Bölüm 16.6)
```

Belirti	Sebep	Çözüm
`insmod` sessiz, `dmesg`'te hiçbir şey yok, `led` dosyası yok	DT'de `compatible`'ı eşleşen düğüm yok: eski DTB yüklendi ya da `compatible` yazımı farklı	`ls /proc/device-tree/bbb-gpio-demo`; DTB'nin TFTP klasöründe güncel olduğunu kontrol et
`error -EBUSY: cannot get led-gpios`	GPIO başka biri tarafından alınmış: sysfs'te export edilmiş, `BB-EXT-LED` overlay'i ya da başka bir sürücü	`cat /sys/kernel/debug/gpio`; `echo 60 > /sys/class/gpio/unexport`
`error -ENOENT: cannot get led-gpios`	Düğümde `led-gpios` yok ya da adı farklı (`led-gpio`, `leds-gpios`)	Property adı `<ad>-gpios`, sürücüde `"<ad>"`
`pin PIN30 already requested by ocp:P9_12_pinmux`	Debian'da cape-universal pinleri tutuyor	`/boot/uEnv.txt`'de `enable_uboot_cape_universal=1`'i kapat (16.6)
LED yanıyor ama buton hiçbir şey yapmıyor; `/proc/interrupts`'ta sayı artmıyor	Buton devresi, pull-down ya da pinmux (giriş açık değil)	P9_23'ü multimetreyle ölç; pin ayarı `0x27` olmalı (15.3)
Tek basış birkaç kez sayılıyor	Kontak sekmesi 50 ms'den uzun	`DEBOUNCE_MS`'yi artır; butona 100 nF kondansatör
`BUG: sleeping function called from invalid context`	Uyuyan bir fonksiyon (mutex, `msleep`) ilk (hard) kesme fonksiyonunda çağrıldı	İşi threaded fonksiyona taşı (23.5)
`genirq: Threaded irq requested with handler=NULL and !ONESHOT`	`IRQF_ONESHOT` unutuldu	Bayrağa ekle

Belirti	Sebep	Çözüm
`make: *** No rule to make target 'am335x-boneblack-gpiodemo.dtb'`	DTS `arch/arm/boot/dts/`'e kopyalanmadı ya da `Makefile` satırı eklenmedi; kernel 6.5+'ta yol farklı	23.3'teki adımlar
`presses_wait` ilk anda bir kez uyanıyor	`poll()` öncesi dosya okunmadı	Döngüden önce bir kez oku (programdaki gibi)

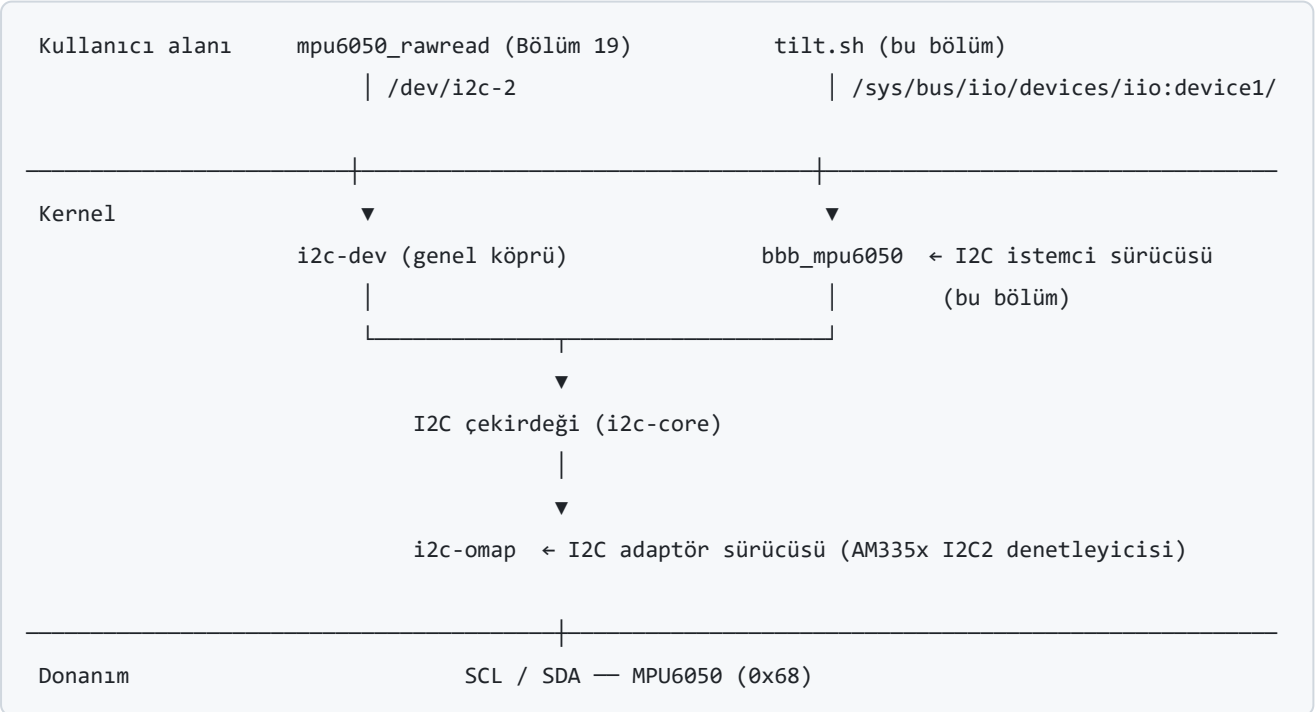
Bölüm 24 — MPU6050 için I2C Sürücüsü (IIO)

> **Bu bölümde:**
>
> - Linux'ta I2C'nin iki tarafını, **adaptör** (denetleyici) ve **istemci** (sensör) sürücülerini tanıyacaksın.
> - Bölüm 19'da kullanıcı alanından okuduğun MPU6050'yi kernel'de bir `i2c_driver` ile okuyacaksın.
> - Sürücüyü kernel'in sensör alt sistemi **IIO**'ya kaydedip değerleri standart dosyalarla (`in_accel_x_raw`, `in_accel_scale` ...) açacaksın.
> - Sensörü önce `new_device` ile anında, sonra device tree ile kalıcı olarak tanıtacaksın.
> - Eğim açısını IIO dosyalarından bir kabuk betiğiyle hesaplayacaksın.
>
> **Ön koşul:** Bölüm 19 — MPU6050, register'lar ve I2C, Bölüm 21 — modül derleme ve `modprobe`, Bölüm 23 — `probe()`, `devm_` ve device tree düğümü.

> **Not**
>
> Bu bölüm kursta yok. Kart, Bölüm 21'deki gibi kendi kernel'in (5.10.162) ve BusyBox rootfs'inle (NFS) açılıyor; MPU6050 19.5'teki gibi I2C2'ye (P9_19, P9_20) bağlı. Terminal görüntüleri temsilidir.

24.1 Linux'ta I2C'nin İki Tarafı

Bölüm 19'da `/dev/i2c-2`'yi açıp `ioctl(I2C_SLAVE)` ile sensörün adresini seçtik ve register'ları kullanıcı alanından okuduk. Bunun altında kernel'de iki tür sürücü vardır:



Sürücü	Neyi sürer?	Örnek
Adaptör (bus/controller) sürücüsü	SoC'nin I2C denetleyicisini: START/STOP, bayt gönderme, ACK	`i2c-omap` (10.6); her I2C hattı için bir `i2c-N`
İstemci (client) sürücüsü	Hattaki bir aygıtı: hangi register'ın ne anlama geldiğini bilir	EEPROM için `at24`, PMIC için `tps65217`, bu bölümdeki `bbb_mpu6050`
`i2c-dev`	Kullanıcı alanına genel bir kapı (`/dev/i2c-N`)	Bölüm 19'daki yol

İstemci sürücüsü register adreslerini ve birim dönüşümünü kendisi bilir; kullanıcı programı sadece "ivme X ne?" diye sorar. Aynı sensörü kullanan her program aynı sürücüden geçer, iki program aynı anda register adresini karıştıramaz.

24.2 IIO: Sensörler için Ortak Arayüz

Kernel'de her tür aygıtın bir **alt sistemi** vardır: LED'ler için `leds` (15.5), ağ kartları için `net`, giriş aygıtları için `input`. İvmeölçer, jiroskop, ADC, sıcaklık, basınç, ışık sensörleri için olan alt sistem **IIO**'dur (Industrial I/O).

Sürücü IIO'ya "şu kanallarım var" der; IIO her kanal için sysfs'te standart adlı dosyalar açar:

Dosya	Anlamı
`name`	Aygıtın adı
`in_accel_x_raw`	İvme X ekseni, ham değer (register'daki sayı)
`in_accel_scale`	Ham değeri birime çeviren çarpan: `raw × scale` = **m/s²**
`in_anglvel_x_raw`, `in_anglvel_scale`	Açısal hız; `raw × scale` = **rad/s**
`in_temp_raw`, `in_temp_offset`, `in_temp_scale`	Sıcaklık; `(raw + offset) × scale` = **milli °C**

Birimler IIO'da sabittir. Hangi üreticinin sensörünü takarsan tak, programın `in_accel_*` dosyalarını okuyup aynı formülü uygulaması yeter. AM335x'in kendi ADC'si de (`AIN0`–`AIN6`, 1.9) bir IIO aygıtıdır; kartında genelde `iio:device0` odur.

> **Not**
>
> Kernel'de MPU6050 için tam özellikli bir IIO sürücüsü zaten var: `inv_mpu6050` (`compatible = "invensense,mpu6050"`). Kesme, tampon (buffer) ve FIFO desteği de içerir. Bu bölümdeki `bbb_mpu6050` öğrenmek için yazılmış küçük bir sürücü; ötekiyle çakışmasın diye farklı bir ad ve `compatible` kullanıyor.

24.3 Sürücü

Bölümün kodu: `code/24_i2c_iio_driver/bbb_mpu6050.c`.

Kanallar

```
#define MPU6050_CHAN(_type, _axis, _reg) {              \
    .type = _type,                          \
    .modified = 1,                          \
    .channel2 = IIO_MOD_##_axis,                    \
    .address = _reg,                        \
    .info_mask_separate = BIT(IIO_CHAN_INFO_RAW),           \
    .info_mask_shared_by_type = BIT(IIO_CHAN_INFO_SCALE),       \
}

static const struct iio_chan_spec bbb_mpu_channels[] = {
    MPU6050_CHAN(IIO_ACCEL, X, MPU6050_REG_ACCEL_XOUT_H),
    MPU6050_CHAN(IIO_ACCEL, Y, MPU6050_REG_ACCEL_XOUT_H + 2),
    MPU6050_CHAN(IIO_ACCEL, Z, MPU6050_REG_ACCEL_XOUT_H + 4),
    MPU6050_CHAN(IIO_ANGL_VEL, X, MPU6050_REG_GYRO_XOUT_H),
    ...
    {
        .type = IIO_TEMP,
        .address = MPU6050_REG_TEMP_OUT_H,
        .info_mask_separate = BIT(IIO_CHAN_INFO_RAW) |
                      BIT(IIO_CHAN_INFO_SCALE) |
                      BIT(IIO_CHAN_INFO_OFFSET),
    },
};
```

Alan	Anlamı
`.type`	Ne ölçülüyor: `IIO_ACCEL`, `IIO_ANGL_VEL`, `IIO_TEMP`
`.modified`, `.channel2`	Eksen: dosya adına `_x`, `_y`, `_z` eklenir
`.address`	Sürücünün kendi kullanımı için serbest alan; burada kanalın register adresi (19.2)
`info_mask_separate`	Her kanalın kendi dosyası olan bilgiler: `in_accel_x_raw`
`info_mask_shared_by_type`	Aynı türdeki kanalların ortak dosyası: tek bir `in_accel_scale`

read_raw

IIO, bir dosya okunduğunda sürücünün `read_raw()` fonksiyonunu hangi kanal ve hangi bilgi istendiğiyle çağırır:

```
static int bbb_mpu_read_raw(struct iio_dev *indio_dev,
                struct iio_chan_spec const *chan,
                int *val, int *val2, long mask)
{
    struct bbb_mpu *m = iio_priv(indio_dev);
    int ret;

    switch (mask) {
    case IIO_CHAN_INFO_RAW:
        mutex_lock(&m->lock);
        ret = i2c_smbus_read_word_swapped(m->client, chan->address);
        mutex_unlock(&m->lock);
        if (ret < 0)
            return ret;
        *val = (s16)ret;
        return IIO_VAL_INT;

    case IIO_CHAN_INFO_SCALE:
        switch (chan->type) {
        case IIO_ACCEL:         /* ±2 g: 9.80665 / 16384 = 0.000598550 m/s^2 */
            *val = 0;
            *val2 = 598550;
            return IIO_VAL_INT_PLUS_NANO;
        ...
        }

    case IIO_CHAN_INFO_OFFSET:      /* sadece sıcaklık: 36.53 °C x 340 = 12420.2 */
        *val = 12420;
        *val2 = 200000;
        return IIO_VAL_INT_PLUS_MICRO;
    }

    return -EINVAL;
}
```

- **Ham değer:** `i2c_smbus_read_word_swapped()` register'dan 2 bayt okur. MPU6050 yüksek baytı önce gönderir; fonksiyonun adındaki "swapped" baytları buna göre çevirir. `(s16)` dönüşümü işaretli 16 bit yapar: 19.8'deki `be16()`'nın işi.
- **Kayan nokta yok:** Kernel'de `float` kullanılmaz (21.3). Ondalıklı bir çarpan iki tamsayıyla verilir: `IIO_VAL_INT_PLUS_NANO` → `val + val2 × 10⁻⁹`. IIO dosyaya `0.000598550` yazar.
- **Çarpanlar** 19.3'teki hassasiyetlerden ve IIO'nun birimlerinden gelir:

Kanal	Hesap	Değer
İvme (±2 g, 16384 LSB/g)	9.80665 m/s² / 16384	0.000598550 m/s²
Açısal hız (±250 °/s, 131 LSB/(°/s))	(π / 180) / 131	0.000133231 rad/s

Kanal	Hesap	Değer
Sıcaklık: scale	1000 / 340	2.941176 milli °C
Sıcaklık: offset	36.53 × 340	12420.2

Sıcaklık formülü 19.3'tekinin aynısıdır: `raw / 340 + 36.53` = `(raw + 12420.2) × (1000 / 340)` milli °C.

probe

```
static int bbb_mpu_probe(struct i2c_client *client)
{
    ...
    ret = i2c_smbus_read_byte_data(client, MPU6050_REG_WHO_AM_I);
    if (ret < 0)
        return dev_err_probe(dev, ret, "no answer at 0x%02x\n", client->addr);
    if (ret != MPU6050_WHO_AM_I_VALUE)
        return dev_err_probe(dev, -ENODEV, "WHO_AM_I = 0x%02x, not an MPU6050\n", ret);

    ret = i2c_smbus_write_byte_data(client, MPU6050_REG_PWR_MGMT_1, MPU6050_PWR_CLK_PLL_X);
    ...
    indio_dev = devm_iio_device_alloc(dev, sizeof(*m));
    m = iio_priv(indio_dev);
    m->client = client;
    ...
    ret = devm_add_action_or_reset(dev, bbb_mpu_sleep, m);   /* ayrılırken uyut */
    ...
    indio_dev->name = "bbb_mpu6050";
    indio_dev->info = &bbb_mpu_info;
    indio_dev->channels = bbb_mpu_channels;
    indio_dev->num_channels = ARRAY_SIZE(bbb_mpu_channels);
    indio_dev->modes = INDIO_DIRECT_MODE;

    return devm_iio_device_register(dev, indio_dev);
}
```

Adım	Karşılığı
`struct i2c_client *client`	Hattaki aygıt: adaptörü (`client->adapter`) ve adresi (`client->addr`) içerir. Bölüm 19'daki `open("/dev/i2c-2")` + `ioctl(I2C_SLAVE, 0x68)` 'in kernel'deki hali.
`i2c_smbus_read_byte_data()` / `write_byte_data()`	Register oku/yaz. Kullanıcı alanındaki `I2C_RDWR` mesajlarını (19.7) bu fonksiyonlar kurar.
`WHO_AM_I` kontrolü	Doğru aygıta konuştuğumuzdan emin ol; değilse `probe` başarısız
`devm_iio_device_alloc()`	IIO aygıtı + sürücünün verisi için tek seferde bellek; veriye `iio_priv()` ile ulaşılır

Adım	Karşılığı
`INDIO_DIRECT_MODE`	Değerler istendiğinde tek tek okunur (dosya okununca). Sürekli örnekleme (buffer) bu örnekte yok.
`devm_iio_device_register()`	sysfs dosyaları burada oluşur; sürücü ayrılırken kendiliğinden kaldırılır

Eşleşme tabloları

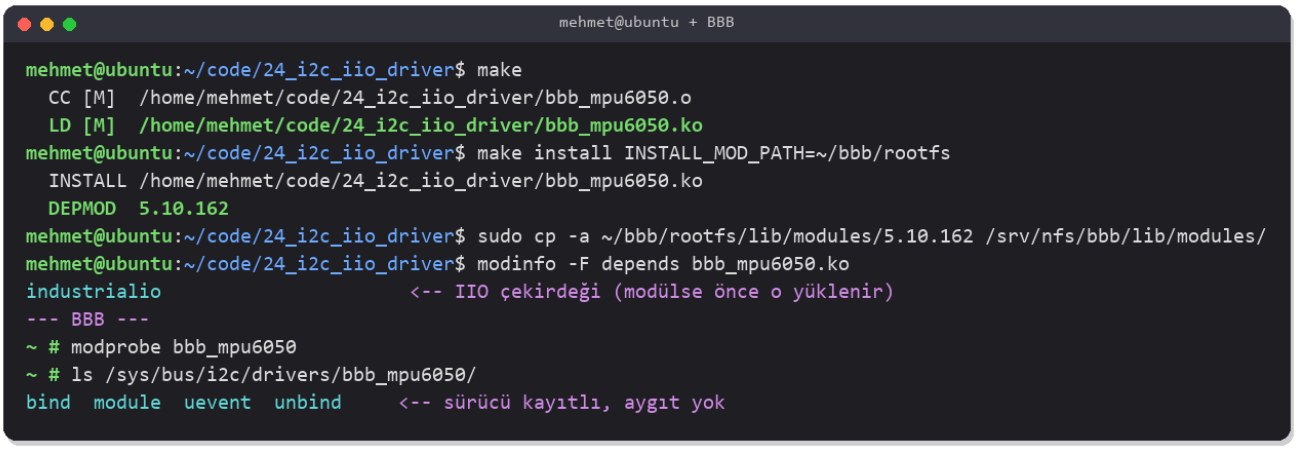
```
static const struct i2c_device_id bbb_mpu_id[] = {
    { "bbb_mpu6050", 0 },
    { }
};
MODULE_DEVICE_TABLE(i2c, bbb_mpu_id);

static const struct of_device_id bbb_mpu_of_match[] = {
    { .compatible = "bbb,mpu6050" },
    { }
};
MODULE_DEVICE_TABLE(of, bbb_mpu_of_match);

static struct i2c_driver bbb_mpu_driver = {
    .driver = {
        .name           = "bbb_mpu6050",
        .of_match_table = bbb_mpu_of_match,
    },
#if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 3, 0)
    .probe          = bbb_mpu_probe,
#else
    .probe_new      = bbb_mpu_probe,
#endif
    .id_table       = bbb_mpu_id,
};
module_i2c_driver(bbb_mpu_driver);
```

İki tablo, aygıtın iki tanıtılma yolu içindir: `id_table` adla (24.4'teki `new_device`), `of_match_table` device tree ile. Kernel 6.3'te `probe`'un imzası değişti (`id` argümanı kalktı); 5.10'da tek argümanlı hali `probe_new` adıyla verilir. `#if` ikisini de karşılar.

24.4 Aygıtı Tanıtmak

I2C aygıtları kendini tanıtmaz; kernel `0x68`'de ne olduğunu bilemez. Birinin söylemesi gerekir.

Derle, kur, yükle



Host'ta:

```
cd ~/code/24_i2c_iio_driver
make
make install INSTALL_MOD_PATH=~/bbb/rootfs
sudo cp -a ~/bbb/rootfs/lib/modules/5.10.162 /srv/nfs/bbb/lib/modules/
sudo cp tilt.sh /srv/nfs/bbb/root/
```

Kartta:

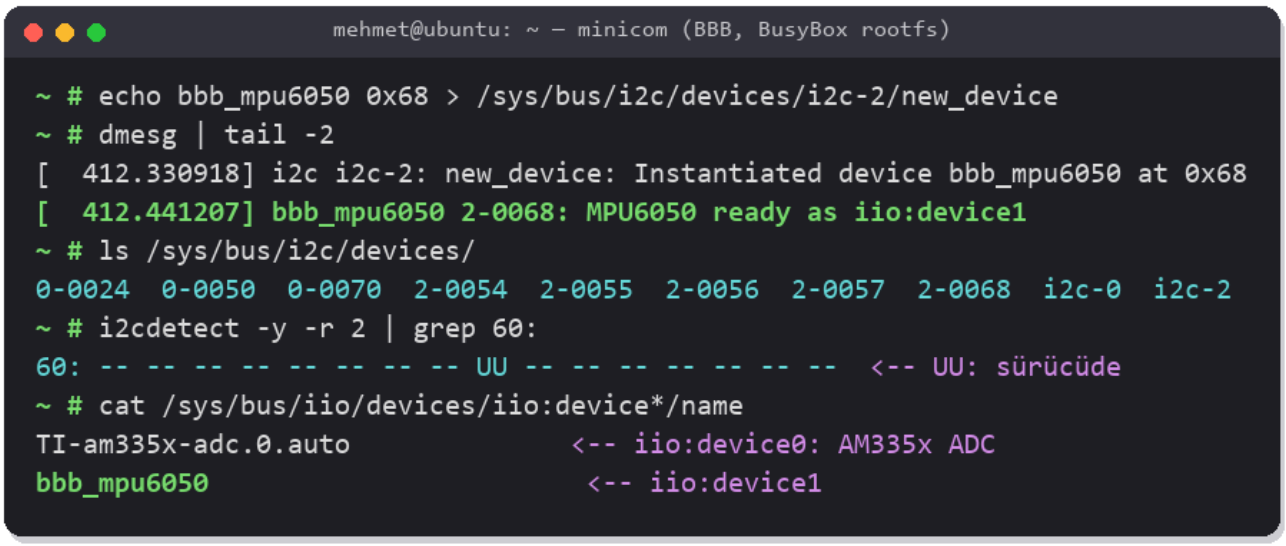
```
~ # modprobe bbb_mpu6050
~ # ls /sys/bus/i2c/drivers/bbb_mpu6050/
```

Sürücü IIO'nun çekirdek fonksiyonlarını kullanır (`modinfo -F depends` → `industrialio`). IIO kernel'de modül olarak derlendiyse `insmod` `Unknown symbol` verir; `modprobe` bağımlılığı önce yükler (21.7). Bu yüzden modül `make install` ile kurulup `modprobe` ile yükleniyor.

Sürücü kayıtlı ama henüz sürdüğü bir aygıt yok.

Hızlı yol: new_device

Her I2C hattının sysfs klasöründe `new_device` dosyası vardır. "Bu hatta, bu adreste, bu adla bir aygıt var" demek, aygıtı DTB'yi değiştirmeden anında oluşturur:




```
~ # echo bbb_mpu6050 0x68 > /sys/bus/i2c/devices/i2c-2/new_device
~ # dmesg | tail -2
~ # ls /sys/bus/i2c/devices/
~ # i2cdetect -y -r 2 | grep 60:
~ # cat /sys/bus/iio/devices/iio:device*/name
```

Kernel `2-0068` adında bir I2C aygıtı oluşturdu (hat 2, adres `0x68`), `id_table`'daki adla sürücüyü buldu ve `probe`'u çağırdı. `i2cdetect` artık `0x68`'de `UU` gösteriyor: adres bir kernel sürücüsünde (19.6).

Aygıtı silmek: `echo 0x68 > /sys/bus/i2c/devices/i2c-2/delete_device`. `new_device` kalıcı değildir; kart yeniden açılınca tekrar yazmak gerekir.

Kalıcı yol: device tree

Bölüm 23'teki gibi bir kart DTS'i: `am335x-boneblack-mpu6050.dts`. Bu kez düğüm kökün değil, **I2C2 hattının** altında; çünkü aygıt o hatta bağlı (11.5'teki EEPROM örneği gibi):

```
#include "am335x-boneblack.dts"

&i2c2 {
    imu@68 {
        compatible = "bbb,mpu6050";
        reg = <0x68>;           /* I2C adresi (AD0 = 0) */
    };
};
```

I2C2 BBB'nin DTS'inde zaten açık ve pinmux'u ayarlı (19.4); `pinctrl` ya da `status` yazmaya gerek yok. `reg`, I2C düğümlerinde aygıtın adresidir (11.6).

```
cd ~/bbb/linux
cp ~/code/24_i2c_iio_driver/am335x-boneblack-mpu6050.dts arch/arm/boot/dts/
echo 'dtb-$(CONFIG_SOC_AM33XX) += am335x-boneblack-mpu6050.dtb' >> arch/arm/boot/dts/Makefile
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x-boneblack-mpu6050.dtb
sudo cp arch/arm/boot/dts/am335x-boneblack-mpu6050.dtb /var/lib/tftpboot/am335x-boneblack.dtb
```

Kart bu DTB ile açılınca kernel açılışta `2-0068` aygıtını kendisi oluşturur; `modprobe bbb_mpu6050` (ya da sürücüyü açılışta yükleyen 12.7'deki betik) yeterli olur.

> **İpucu**
>
> Bölüm 23'teki LED/buton düğümünü de kullanmak istiyorsan iki düğümü tek bir DTS'te birleştir: `am335x-boneblack-gpiodemo.dts` 'in sonuna buradaki `&i2c2 { ... };` bloğunu ekle.

24.5 Değerleri Okumak

```
mehmet@ubuntu: ~ — minicom (BBB, BusyBox rootfs)

~ # cd /sys/bus/iio/devices/iio:device1 && ls
dev              in_accel_z_raw    in_anglvel_z_raw  in_temp_scale  subsystem
in_accel_scale   in_anglvel_scale  in_temp_offset    name           uevent
in_accel_x_raw   in_anglvel_x_raw  in_temp_raw       of_node
in_accel_y_raw   in_anglvel_y_raw  power
iio:device1 # cat in_accel_x_raw in_accel_y_raw in_accel_z_raw in_accel_scale
-100
164
16360                              <-- Z: yerçekimi
0.000598550                        <-- m/s^2 / LSB
iio:device1 # awk 'BEGIN { print 16360 * 0.000598550 }'
9.79228                            <-- m/s^2 ~ 1 g
iio:device1 # cat in_temp_raw in_temp_offset in_temp_scale
-4226
12420.200000
2.941176
iio:device1 # awk 'BEGIN { print (-4226 + 12420.2) * 2.941176 / 1000 }'
24.1006                            <-- °C
```

```
~ # cd /sys/bus/iio/devices/iio:device1
iio:device1 # ls
iio:device1 # cat in_accel_x_raw in_accel_y_raw in_accel_z_raw in_accel_scale
iio:device1 # awk 'BEGIN { print 16360 * 0.000598550 }'
iio:device1 # cat in_temp_raw in_temp_offset in_temp_scale
iio:device1 # awk 'BEGIN { print (-4226 + 12420.2) * 2.941176 / 1000 }'
```

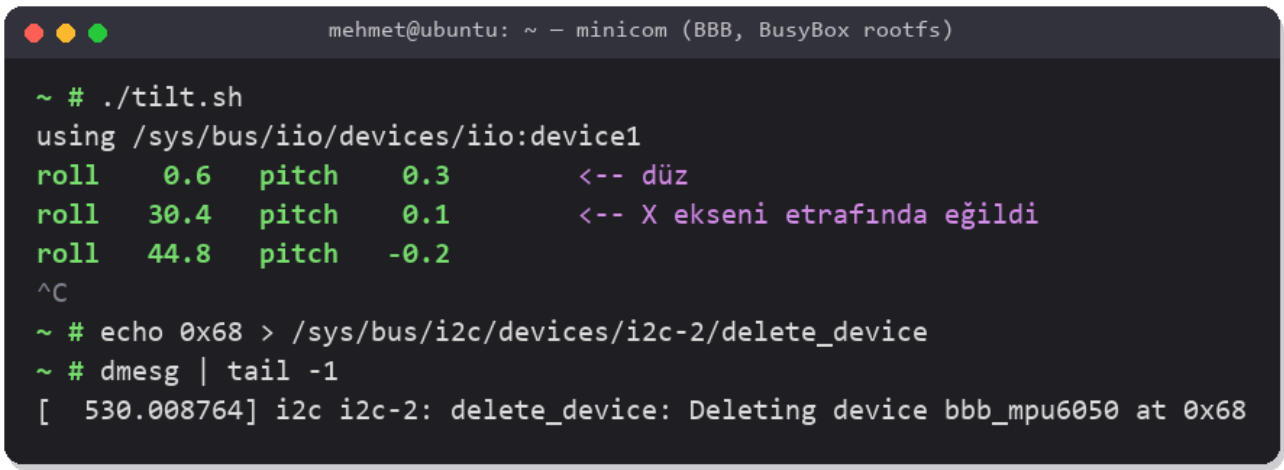
Düz dururken Z ekseni ≈ 9.8 m/s² (1 g) okur; sıcaklık 19.9'daki 24.1 °C ile aynı. Aygıt numarası (`iio:device1`) sistemden sisteme değişebilir; doğru klasörü `name` dosyasından bul.

Eğim betiği

`tilt.sh`, 19.10'daki roll/pitch formülünü IIO dosyalarıyla hesaplar. C programına, `/dev/i2c-2`'ye ya da register adreslerine gerek kalmadı; sürücü hepsini üstlendi:

```
DEV=
for d in /sys/bus/iio/devices/iio:device*; do
    [ "$(cat "$d/name" 2>/dev/null)" = "bbb_mpu6050" ] && DEV=$d && break
done
...
while true; do
    x=$(cat "$DEV/in_accel_x_raw")
    y=$(cat "$DEV/in_accel_y_raw")
    z=$(cat "$DEV/in_accel_z_raw")
    awk -v x="$x" -v y="$y" -v z="$z" 'BEGIN {
        r2d = 180 / 3.14159265
        printf "roll %6.1f   pitch %6.1f\n", atan2(y, z) * r2d, atan2(-x, sqrt(y*y + z*z)) * r2d
    }'
    sleep 1
done
```

Açı hesaplanırken `scale` ile çarpmaya gerek yok: üç eksen aynı çarpanla çarpılacağı için oranları, yani açı değişmez.




```
~ # chmod +x tilt.sh
~ # ./tilt.sh
~ # echo 0x68 > /sys/bus/i2c/devices/i2c-2/delete_device
```

> **Not**
>
> IIO'nun komut satırı araçları da var: kernel kaynağında `tools/iio/` altında `iio_info`, `lsiio`, `iio_generic_buffer`. Ayrıca `libiio` kütüphanesi IIO aygıtlarına C, Python ve ağ üzerinden erişim sağlar.

24.6 Özet

- Linux'ta I2C'nin iki tarafı var: **adaptör** sürücüsü denetleyiciyi (`i2c-omap`), **istemci** sürücüsü hattaki aygıtı sürer. `/dev/i2c-N` (`i2c-dev`) kullanıcı alanına açılan genel kapıdır.
- İstemci sürücüsü: `struct i2c_driver` + `module_i2c_driver()`; `probe(struct i2c_client *)` içinde `i2c_smbus_read_byte_data()`, `i2c_smbus_read_word_swapped()`, `i2c_smbus_write_byte_data()`.
- Sensörler **IIO**'ya kaydolur: `iio_chan_spec` kanalları, `read_raw()` ve `devm_iio_device_register()`. Değerler `in_<tür>_<eksen>_raw`, birim çarpanı `in_<tür>_scale` (ivme m/s^2, açısal hız rad/s, sıcaklık milli °C).
- Kernel'de kayan nokta yok: çarpanlar `IIO_VAL_INT_PLUS_MICRO/NANO` ile iki tamsayı olarak verilir.
- Aygıtı tanıtmak: anında `echo <ad> <adres> > /sys/bus/i2c/devices/i2c-N/new_device` (`id_table`), kalıcı olarak DT'de I2C hattının altına `reg = <adres>` ile düğüm (`of_match_table`).
- Adresi bir kernel sürücüsü aldığında `i2cdetect` `UU` gösterir ve kullanıcı alanından `I2C_SLAVE` `EBUSY` döner.

24.7 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom (BBB, BusyBox rootfs) (temsili hatalar)

~ # insmod bbb_mpu6050.ko
insmod: can't insert 'bbb_mpu6050.ko': unknown symbol in module, or unknown parameter
bbb_mpu6050: Unknown symbol devm_iio_device_register (err -2)  <-- IIO modülü yüklü değil

~ # echo bbb_mpu6050 0x68 > /sys/bus/i2c/devices/i2c-2/new_device; dmesg | tail -1
bbb_mpu6050 2-0068: error -EREMOTEIO: no answer at 0x68  <-- sensör bağlı değil / adres yanlış

~ # ./mpu6050_rawread
I2C_SLAVE: Device or resource busy        <-- adres sürücüde (Bölüm 19'un programı)
```

Belirti	Sebep	Çözüm
`insmod: ... unknown symbol`, `dmesg`'te `Unknown symbol devm_iio_device_register`	IIO çekirdeği (`industrialio`) modül ve yüklü değil	`make install` + `modprobe bbb_mpu6050` (24.4)
`error -EREMOTEIO: no answer at 0x68`	Sensör cevap vermiyor: bağlantı, besleme ya da adres	19.12'deki kontroller; `i2cdetect -y -r 2`

Belirti	Sebep	Çözüm
`WHO_AM_I = 0x.., not an MPU6050`	Bu adreste başka bir aygıt var (ya da MPU6050 klonu farklı bir değer dönüyor)	Datasheet'e bak; klonsa `MPU6050_WHO_AM_I_VALUE`'yu değiştir
`new_device` yazıldı ama `probe` çağrılmadı	Sürücü yüklü değil ya da addaki yazım `id_table`'dan farklı	`ls /sys/bus/i2c/drivers/`; ad `bbb_mpu6050` olmalı
`new_device`: `Device or resource busy` / `already in use`	O adreste zaten bir aygıt tanımlı (DT'den ya da önceki `new_device`)	`ls /sys/bus/i2c/devices/`; önce `delete_device`
Bölüm 19'un programı `I2C_SLAVE: Device or resource busy`	Adres artık `bbb_mpu6050` sürücüsünde	Sürücüyü ayır (`delete_device` ya da `rmmod`) ya da IIO dosyalarını kullan
DT'li açılışta `inv-mpu6050` sürücüsü bağlandı	`compatible`'ı `invensense,mpu6050` yazdın	Bu sürücü için `bbb,mpu6050`; ikisi aynı aygıtı alamaz
Değerler hep `0`	Sensör uykuda: `probe`'daki `PWR_MGMT_1` yazılmadı ya da başka bir program sensörü uyuttu	`dmesg`'te `MPU6050 ready` satırına bak; aygıtı silip yeniden tanıt
`tilt.sh: bbb_mpu6050 IIO device not found`	Aygıt tanıtılmamış	24.4